

○ 高等学校教材

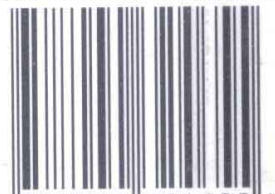
# 数值计算方法

○ 丁丽娟 程杞元 编著



高等教育出版社  
HIGHER EDUCATION PRESS

ISBN 978-7-04-032465-5



9 787040 324655 >

定价 36.20 元

# 数值计算方法

## Shuzhi Jisuan Fangfa

丁丽娟 程杞元 编著



高等教育出版社·北京  
HIGHER EDUCATION PRESS BEIJING

## 内容提要

本书是根据理工科数值计算方法课程的基本要求,结合作者多年的教学实践经验和成果编写而成的。在编写过程中注重数值计算方法的实用性,并介绍了各类方法的新发展,以 MATLAB 为平台,强调计算效率。

全书内容包括数值代数、数值逼近与常微分方程数值解法的基本内容。各章均配备了丰富的例题与应用实例,给出了各种基本算法的计算机实现过程。书末还附有 MATLAB 数学软件简介,便于读者编程进行数值实验。

本书可作为工科专业研究生及理科各专业本科生的数值计算课程教材,也可供相关工程技术人员参考。

## 图书在版编目(CIP)数据

数值计算方法/丁丽娟,程杞元编著. —北京:高等教育出版社, 2011.8

ISBN 978 - 7 - 04 - 032465 - 5

I. ①数… II. ①丁… ②程… III. ①数值计算 - 计算方法 - 高等学校 - 教材 IV. ①O241

中国版本图书馆 CIP 数据核字(2011)第 128931 号

策划编辑 张长虹  
版式设计 余 杨

责任编辑 张长虹  
插图绘制 尹 莉

封面设计 王凌波  
责任校对 殷 然

责任印制 胡晓旭

出版发行 高等教育出版社  
社 址 北京市西城区德外大街 4 号  
邮政编码 100120  
印 刷 北京四季青印刷厂  
开 本 787mm × 960mm 1/16  
印 张 23.25  
字 数 430 千字  
购书热线 010 - 58581118

咨询电话 400 - 810 - 0598  
网 址 <http://www.hep.edu.cn>  
<http://www.hep.com.cn>  
网上订购 <http://www.landraco.com>  
<http://www.landraco.com.cn>  
版 次 2011 年 8 月第 1 版  
印 次 2011 年 8 月第 1 次印刷  
定 价 36.20 元

本书如有缺页、倒页、脱页等质量问题,请到所购图书销售部门联系调换

版权所有 侵权必究

物 料 号 32465 - 00



# 前 言

随着计算机的广泛使用与科学技术的迅速发展,科学计算已成为科学研究和工程应用领域中的一种重要研究工具,它是与理论分析、科学试验并驾齐驱的一种科学研究方法。科学计算能力也成为理工科大学生必需具备的基本素质之一。数值计算方法课程是计算科学的重要基础课程,是理工科院校各专业本科生和工科专业硕士研究生的学位课,主要内容是引导科学计算的入门、介绍数值计算的基本方法以及如何利用现代计算工具高效率解决数值计算问题。

本书是作者团队根据数值计算方法课程的基本要求,在多年的教学实践和原有教材基础上编写而成的,包含了数值代数、数值逼近和常微分方程数值解法的基本内容。力求全面、系统地介绍求解各类数学问题近似解的最基本、最常用的方法,并且着重阐明构造算法的基本思想与原理。根据“重概念、重方法、重应用、重能力”的培养原则,我们在编写中强调以下几点:

1. 注重数值计算基本方法的介绍,简明介绍数值计算中的“病态”问题和工程计算应用中常见问题的处理方法。如病态线性方程组和稀疏分块方程组的求解、二维数据插值拟合、重积分的数值计算、刚性微分方程(组)的求解等。

2. 注重方法的实用性。精编了一些例题,各章配备了应用实例,以各章的数值方法为工具,比较各类数值方法的优劣性并帮助读者如何选择数值方法求解问题,更好地体会应用数值计算方法有效地解决工程技术问题。

3. 注重提高计算效率,以 MATLAB 为平台,培养学生的编程能力。教材附录中简单介绍了 MATLAB 软件的基本功能,同时各章中介绍实现各种数值方法的函数命令和程序,使学生在了解数值计算方法的原理之后,快速实现算法并能编程求解工程计算问题。

4. 注重各类数值方法的发展。本书对每一章的数值计算方法作了简要的评述,介绍近几年发展起来的计算效率高的各种数值方法,并给出参考书目,以便读者自学。

全书讲授约需 60 学时,也可根据具体学时选择部分章节讲授。在本书的编写过程中,我们参考了许多国内外相关教材和专著,并将它们列在参考文献中,在此表示衷心的感谢。由于编者水平有限,书中不当之处难免,恳请读者批评指正。

编者

2011 年 3 月

## 郑重声明

高等教育出版社依法对本书享有专有出版权。任何未经许可的复制、销售行为均违反《中华人民共和国著作权法》，其行为人将承担相应的民事责任和行政责任；构成犯罪的，将被依法追究刑事责任。为了维护市场秩序，保护读者的合法权益，避免读者误用盗版书造成不良后果，我社将配合行政执法部门和司法机关对违法犯罪的单位和个人进行严厉打击。社会各界人士如发现上述侵权行为，希望及时举报，本社将奖励举报有功人员。

反盗版举报电话 (010) 58581897 58582371 58581879

反盗版举报传真 (010) 82086060

反盗版举报邮箱 dd@hep.com.cn

通信地址 北京市西城区德外大街4号 高等教育出版社法务部

邮政编码 100120

# 目 录

|                                  |               |
|----------------------------------|---------------|
| <b>第一章 数值计算中的误差 .....</b>        | <b>1</b>      |
| § 1.1 数值计算的内容与特点 .....           | 1             |
| § 1.2 误差的基本概念 .....              | 1             |
| § 1.3 数值计算中误差的传播 .....           | 5             |
| § 1.4 数值计算中应注意的问题 .....          | 9             |
| 评注 .....                         | 12            |
| 习题一 .....                        | 13            |
| 数值实验 .....                       | 14            |
| <br><b>第二章 解线性方程组的直接方法 .....</b> | <br><b>15</b> |
| § 2.1 消去法 .....                  | 16            |
| § 2.2 直接三角分解法 .....              | 22            |
| § 2.3 特殊矩阵的三角分解法 .....           | 34            |
| § 2.4 误差分析 .....                 | 41            |
| § 2.5 超定线性方程组的最小二乘解 .....        | 51            |
| § 2.6 应用实例 .....                 | 54            |
| 评注 .....                         | 57            |
| 习题二 .....                        | 58            |
| 数值实验 .....                       | 61            |
| <br><b>第三章 解线性方程组的迭代法 .....</b>  | <br><b>63</b> |
| § 3.1 迭代法概述 .....                | 63            |
| § 3.2 几种基本的迭代法 .....             | 65            |
| § 3.3 迭代法的收敛条件 .....             | 72            |
| § 3.4 最速下降法与共轭梯度法 .....          | 80            |
| § 3.5 应用实例 .....                 | 84            |
| 评注 .....                         | 86            |

|                                |            |
|--------------------------------|------------|
| 习题三 .....                      | 87         |
| 数值实验 .....                     | 89         |
| <b>第四章 矩阵特征值与特征向量的计算.....</b>  | <b>91</b>  |
| § 4.1 幂法和反幂法 .....             | 91         |
| § 4.2 雅可比 (Jacobi) 方法 .....    | 100        |
| § 4.3 QR 方法 .....              | 105        |
| § 4.4 应用实例 .....               | 117        |
| 评注 .....                       | 119        |
| 习题四 .....                      | 120        |
| 数值实验 .....                     | 121        |
| <b>第五章 插值法 .....</b>           | <b>123</b> |
| § 5.1 拉格朗日 (Lagrange) 插值 ..... | 123        |
| § 5.2 牛顿 (Newton) 插值 .....     | 130        |
| § 5.3 分段线性插值 .....             | 141        |
| § 5.4 埃尔米特 (Hermite) 插值 .....  | 144        |
| § 5.5 样条插值 .....               | 151        |
| § 5.6 二维插值 .....               | 159        |
| § 5.7 快速傅里叶变换 (FFT) .....      | 162        |
| § 5.8 应用实例 .....               | 165        |
| 评注 .....                       | 171        |
| 习题五 .....                      | 171        |
| 数值实验 .....                     | 175        |
| <b>第六章 函数逼近 .....</b>          | <b>177</b> |
| § 6.1 数据拟合的最小二乘法 .....         | 178        |
| § 6.2 正交多项式 .....              | 187        |
| § 6.3 函数的最佳平方逼近 .....          | 193        |
| § 6.4 应用实例 .....               | 197        |
| 评注 .....                       | 202        |
| 习题六 .....                      | 202        |
| 数值实验 .....                     | 204        |



|                                         |            |
|-----------------------------------------|------------|
| <b>第七章 数值微分与数值积分</b> .....              | <b>206</b> |
| § 7.1 数值微分 .....                        | 206        |
| § 7.2 牛顿 - 科茨 (Newton-Cotes) 求积公式 ..... | 210        |
| § 7.3 复化求积公式 .....                      | 217        |
| § 7.4 龙贝格 (Romberg) 求积公式 .....          | 226        |
| § 7.5 高斯 (Gauss) 型求积公式 .....            | 230        |
| § 7.6 振荡函数的积分 .....                     | 242        |
| § 7.7 重积分的数值计算 .....                    | 245        |
| § 7.8 应用实例 .....                        | 250        |
| 评注 .....                                | 256        |
| 习题七 .....                               | 257        |
| 数值实验 .....                              | 260        |
| <br>                                    |            |
| <b>第八章 非线性方程及非线性方程组的解法</b> .....        | <b>262</b> |
| § 8.1 对分区间法 .....                       | 262        |
| § 8.2 简单迭代法 .....                       | 265        |
| § 8.3 牛顿 (Newton) 法与弦截法 .....           | 273        |
| § 8.4 抛物线 (Müller) 法 .....              | 277        |
| § 8.5 非线性方程组的解法 .....                   | 279        |
| § 8.6 应用实例 .....                        | 283        |
| 评注 .....                                | 287        |
| 习题八 .....                               | 288        |
| 数值实验 .....                              | 290        |
| <br>                                    |            |
| <b>第九章 常微分方程数值解法</b> .....              | <b>292</b> |
| § 9.1 欧拉 (Euler) 方法及其改进方法 .....         | 294        |
| § 9.2 龙格 - 库塔 (Runge-Kutta) 法 .....     | 300        |
| § 9.3 线性多步法 .....                       | 305        |
| § 9.4 相容性、收敛性与稳定性 .....                 | 314        |
| § 9.5 微分方程组的数值解法 .....                  | 320        |
| § 9.6 边值问题的数值解法 .....                   | 327        |
| § 9.7 应用实例 .....                        | 331        |
| 评注 .....                                | 339        |

---

|                               |            |
|-------------------------------|------------|
| 习题九 .....                     | 340        |
| 数值实验 .....                    | 342        |
| <b>附录 MATLAB 数学软件简介 .....</b> | <b>344</b> |
| <b>参考文献 .....</b>             | <b>362</b> |

# 第一章 数值计算中的误差

## § 1.1 数值计算的内容与特点

数值计算方法是研究求解数学问题(数学模型)近似解的方法、过程及其理论的一个数学分支,由于它所研究的数学问题往往来源于科学研究与工程计算问题,故数值计算方法也称为科学计算方法.随着计算机技术的发展,使用计算机通过数值计算或数值模拟的手段解决工程实际问题和科学研究中关键问题已成为越来越不可缺少的重要环节,在某些领域内代替甚至超过了工程实验所起的作用.数值计算与模拟已成为与理论研究、科学实验同样重要和有效的第三种手段.目前,掌握和运用科学计算的基本方法已是各学科科学研究者和工程技术人员必备知识.

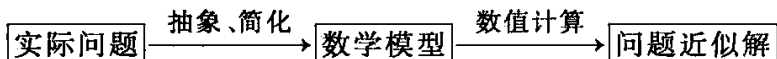
从科学与工程计算中抽象出的数学问题常常涉及求解微分方程、线性与非线性代数方程系统、微分与积分计算、特征值计算、数据拟合与函数逼近问题以及数据处理问题,数值计算方法利用离散、迭代、插值与拟合等最基础的方法,介绍针对各个问题的基本数值算法,再编程计算,得到数学问题的近似解.进一步分析数值算法的误差、精确度、稳定性和收敛性,建立相关的计算数学理论.数值计算方法不同于纯数学,它不仅具有数学的抽象性和严密性,又具有广泛的应用性和实验技术性.它是一门与计算机紧密结合的、实用性很强的、有着自身研究方法 with 理论体系的数学课程.

学习数值计算方法时,首先应掌握各种数值计算方法的基本原理和思想,学会运用各种方法的处理技巧,了解各种方法的优缺点,并会利用数学计算软件编程计算,求得数学问题的数值解.目前流行的数值计算软件有 Maple, MATLAB, Mathematica 等,更多的软件可以在<http://www.netlib.org>, <http://gmas.nist.gov>, <http://sourceforge.net> 等网站上查询,本书中的计算实例用 MATLAB 编程实现.

## § 1.2 误差的基本概念

实际问题的精确解与数值计算所得的近似解之间的差别称为误差.在科学

计算中,误差是不可避免的.由于用数学方法解决实际问题时,常按以下过程进行:



从而引起误差的因素很多,主要有以下几种:

### 1. 模型误差

实际问题的解与数学模型的解之差称为“模型误差”

### 2. 观测误差

数学问题中所出现的一些参量,其值往往由观测得到,而观测不可能绝对准确,由此产生的误差称为“观测误差”.

### 3. 截断误差

一般数学问题常常难以求出精确解,需要简化为较易求解的问题,以简化问题的解作为原问题解的近似.如求一个收敛的无穷级数之和,总是用它的部分和作为近似值,也就是截去该级数后面的无穷多项.这样由于简化问题所引起的解的误差称为“截断误差”或“方法误差”.例如

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots + \frac{(-1)^n x^{2n}}{(2n)!} + \cdots$$

当  $|x|$  很小时,可以用  $1 - \frac{x^2}{2}$  作为  $\cos x$  近似值.由交错级数判敛的莱布尼茨 (Leibniz) 准则,它的截断误差的绝对值不超过  $\frac{x^4}{24}$ .

### 4. 舍入误差

在计算过程中往往要对数字进行舍入.如受机器字长的限制,无穷小数和位数很多的数必须舍入成一定的位数.这样产生的误差称为“舍入误差”.

本课程中只讨论截断误差与舍入误差对计算结果的影响.

## 1.2.1 绝对误差与相对误差

设  $x^*$  为准确值  $x$  的一个近似值,称

$$e(x^*) = x - x^* \quad (1-1)$$

为近似值  $x^*$  的绝对误差,简称误差.一般情况下准确值  $x$  难以求出,从而也不能算出绝对误差  $e(x^*)$  的准确值,但可以根据测量工具或计算的情况估计出它的取值范围,即估计出误差绝对值的一个上界  $\varepsilon$

$$|e(x^*)| = |x - x^*| \leq \varepsilon \quad (1-2)$$

通常称  $\varepsilon$  为近似值  $x^*$  的绝对误差限,简称误差限.显然,误差限不是唯一的.

有了误差限及近似值,就可以得到准确值的范围

$$x^* - \varepsilon \leq x \leq x^* + \varepsilon$$

即准确值  $x$  必定在区间  $[x^* - \varepsilon, x^* + \varepsilon]$  内, 也常记作

$$x = x^* \pm \varepsilon$$

容易看出, 经过四舍五入得到的数, 其误差必定不超过被保留的最后数位上的半个单位, 即最后数位上的半个单位为其误差限. 例如, 若取  $\pi$  的近似值为 3.14, 则

$$|\pi - 3.14| \leq 0.0016 \leq \frac{1}{2} \times 10^{-2}$$

若取  $\pi \approx 3.142$ , 则

$$|\pi - 3.142| \leq 0.00041 \leq \frac{1}{2} \times 10^{-3}$$

误差限的大小不能完全反映近似值的精确程度. 要刻画近似值的精确程度, 不仅要看绝对误差的大小, 还必须考虑精确值本身的大小, 由此引出了相对误差的概念.

仍设  $x^*$  为准确值  $x$  的近似值, 称绝对误差与准确值之比为近似值  $x^*$  的相对误差, 记为  $e_r(x^*)$ , 即

$$e_r(x^*) = \frac{e(x^*)}{x} = \frac{x - x^*}{x} \quad (1-3)$$

由于在计算过程中准确值  $x$  总是未知的, 故一般取相对误差为

$$e_r(x^*) = \frac{e(x^*)}{x^*}$$

可以证明, 当  $|e_r(x^*)|$  很小时,  $\frac{e(x^*)}{x} - \frac{e(x^*)}{x^*}$  是  $e_r(x^*)$  的高阶无穷小, 可以忽略不计. 所以, 取绝对误差与近似值之比为相对误差是合理的.

同样, 相对误差也只能估计其上限. 如果存在正数  $\varepsilon_r$ , 使得

$$|e_r(x^*)| = \left| \frac{e(x^*)}{x^*} \right| \leq \varepsilon_r \quad (1-4)$$

则称  $\varepsilon_r$  为  $x^*$  的相对误差限. 显然, 误差限与近似值绝对值之比  $\frac{\varepsilon}{|x^*|}$  为  $x^*$  的一个相对误差限.

例如, 由实验测得光速近似值为  $c^* = 2.997\,925 \times 10^5$  km/s, 其误差限为 0.1 km/s, 于是

$$\frac{\varepsilon}{|c^*|} = \frac{0.1}{2.997\,925 \times 10^5} < 4 \times 10^{-7}$$

所以  $4 \times 10^{-7}$  是  $c^*$  的一个相对误差限.

### 1.2.2 有效数字

有效数字是近似值的一种表示法. 它既能表示近似值的大小, 又能表示其精

确程度.

在计算过程中,常常按四舍五入的原则取数  $x$  的前几位数  $x^*$  为其近似值. 例如,  $x = \sqrt{2} = 1.414\ 213\ 562\cdots$ , 取前四位数得近似值  $x^* = 1.414$ , 取前八位数得近似值  $x^* = 1.414\ 213\ 6$ , 它们的绝对误差

$$|\sqrt{2} - 1.414| \leq \frac{1}{2} \times 10^{-3}$$

$$|\sqrt{2} - 1.414\ 213\ 6| \leq \frac{1}{2} \times 10^{-7}$$

如果近似值  $x^*$  的误差限是  $\frac{1}{2} \times 10^{-n}$ , 则称  $x^*$  准确到小数点后第  $n$  位, 并从第一个非零数字到这一位的所有数字均称为有效数字. 例如,  $\sqrt{2}$  的近似值 1.414 准确到小数点后第 3 位, 它具有 4 位有效数字. 1.414 213 6 作为  $\sqrt{2}$  的近似值准确到小数点后第 7 位, 有 8 位有效数字. 一般地, 如果近似值  $x^*$  的规格化形式为

$$x^* = \pm 0.a_1a_2\cdots a_n\cdots \times 10^m \quad (1-5)$$

其中  $m$  为整数,  $a_1 \neq 0$ ,  $a_i (i=1, 2, \cdots)$  为 0 到 9 之间的整数, 且

$$|x - x^*| \leq \frac{1}{2} \times 10^{m-n} \quad (1-6)$$

则称近似值  $x^*$  有  $n$  位有效数字.

例如,  $x = 0.003\ 400 \pm \frac{1}{2} \times 10^{-5}$  表示近似值 0.003 400 准确到小数点后第 5 位, 有 3 位有效数字.

上面的讨论表明, 可以用有效数字位数来刻画误差限. 形如式 (1-5) 的数, 当  $m$  一定时, 其有效数字位数  $n$  越大, 则误差限越小. 例如, 若  $x^* = 1\ 452.046$  是具有 7 位有效数字的近似值, 则它的误差限为

$$|x - x^*| \leq \frac{1}{2} \times 10^{-3}$$

又若  $x^* = 1\ 452.0$  是具有 5 位有效数字的近似值, 则其误差限为  $\frac{1}{2} \times 10^{-1}$ .

下面的定理给出了相对误差限与有效数字的关系.

**定理 1.1** 若  $x$  的近似值  $x^* = \pm 0.a_1a_2\cdots a_n \times 10^m (a_1 \neq 0)$  有  $n$  位有效数字, 则  $\frac{1}{2a_1} \times 10^{-n+1}$  为其相对误差限. 反之, 若  $x^*$  的相对误差限  $\varepsilon_r$  满足

$$\varepsilon_r \leq \frac{1}{2(a_1 + 1)} \times 10^{-n+1}$$

则  $x^*$  至少具有  $n$  位有效数字.

[证明] 由式 (1-6)

$$|e(x^*)| = |x - x^*| \leq \frac{1}{2} \times 10^{m-n}$$

从而有

$$|e_r(x^*)| = \left| \frac{e(x^*)}{x^*} \right| \leq \frac{\frac{1}{2} \times 10^{m-n}}{0. a_1 a_2 \cdots a_n \times 10^m} \leq \frac{1}{2a_1} \times 10^{-n+1}$$

所以  $\frac{1}{2a_1} \times 10^{-n+1}$  是  $x^*$  的相对误差限.

若  $\varepsilon_r \leq \frac{1}{2(a_1 + 1)} \times 10^{-n+1}$ , 由式(1-4)

$$\begin{aligned} |e(x^*)| &= |x^* e_r(x^*)| \leq 0. a_1 \cdots a_n \cdots \times 10^m \varepsilon_r \\ &\leq (a_1 + 1) \times 10^{m-1} \times \frac{1}{2(a_1 + 1)} \times 10^{-n+1} = \frac{1}{2} \times 10^{m-n} \end{aligned}$$

由式(1-6),  $x^*$  至少有  $n$  位有效数字.

定理 1.1 表明, 由有效数字位数可以求出相对误差限. 如  $x^* = 2.72$  是  $x = e$  的具有 3 位有效数字的近似值, 故其相对误差限为

$$\varepsilon_r = \frac{1}{2 \times 2} \times 10^{-3+1} = 0.25 \times 10^{-2}$$

## § 1.3 数值计算中误差的传播

### 1.3.1 基本运算中的误差估计

本节中所讨论的基本运算是四则运算与一些常用函数的计算.

由微分学, 当自变量的改变量(误差)很小时, 函数的微分作为函数的改变量的主要线性部分可以近似函数的改变量, 故利用微分运算公式可导出误差运算公式.

设数值计算中求得的解与参量(原始数据)  $x_1, x_2, \cdots, x_n$  有关, 记为

$$y = f(x_1, x_2, \cdots, x_n) \quad (1-7)$$

参量的误差必定引起解的误差. 设  $x_1, x_2, \cdots, x_n$  的近似值分别为  $x_1^*, x_2^*, \cdots, x_n^*$ , 相应的解为

$$y^* = f(x_1^*, x_2^*, \cdots, x_n^*) \quad (1-8)$$

假定  $f$  在点  $(x_1^*, \cdots, x_n^*)$  处可微, 则当数据误差较小时, 解的绝对误差为

$$\begin{aligned} e(y^*) &= y - y^* = f(x_1, \cdots, x_n) - f(x_1^*, \cdots, x_n^*) \\ &\approx df(x_1^*, \cdots, x_n^*) \end{aligned}$$



$$\begin{aligned}
 &= \sum_{i=1}^n \frac{\partial f(x_1^*, \dots, x_n^*)}{\partial x_i} (x_i - x_i^*) \\
 &= \sum_{i=1}^n \frac{\partial f(x_1^*, \dots, x_n^*)}{\partial x_i} e(x_i^*) \quad (1-9)
 \end{aligned}$$

其相对误差为

$$\begin{aligned}
 e_r(y^*) &= \frac{e(y^*)}{y^*} \approx d(\ln f) \\
 &= \sum_{i=1}^n \frac{\partial f(x_1^*, \dots, x_n^*)}{\partial x_i} \frac{e(x_i^*)}{f(x_1^*, \dots, x_n^*)} \\
 &= \sum_{i=1}^n \frac{\partial f(x_1^*, \dots, x_n^*)}{\partial x_i} \frac{x_i^*}{f(x_1^*, \dots, x_n^*)} e_r(x_i^*) \quad (1-10)
 \end{aligned}$$

特别地,由式(1-9)和式(1-10)可得和、差、积、商之误差公式

$$\begin{cases} e(x_1 \pm x_2) = e(x_1) \pm e(x_2) \\ e_r(x_1 \pm x_2) = \frac{x_1}{x_1 \pm x_2} e_r(x_1) \pm \frac{x_2}{x_1 \pm x_2} e_r(x_2) \end{cases} \quad (1-11)$$

$$\begin{cases} e(x_1 x_2) \approx x_2 e(x_1) + x_1 e(x_2) \\ e_r(x_1 x_2) \approx e_r(x_1) + e_r(x_2) \end{cases} \quad (1-12)$$

$$\begin{cases} e\left(\frac{x_1}{x_2}\right) \approx \frac{1}{x_2} e(x_1) - \frac{x_1}{x_2^2} e(x_2) \\ e_r\left(\frac{x_1}{x_2}\right) \approx e_r(x_1) - e_r(x_2) \end{cases} \quad (1-13)$$

式(1-11)~式(1-13)表明,和、差之误差为误差之和、差,积、商之相对误差为相对误差之和、差.由以上各式还可得出

$$|e(x_1 \pm x_2)| = |e(x_1) \pm e(x_2)| \leq |e(x_1)| + |e(x_2)| \quad (1-14)$$

$$|e_r(x_1 x_2)| \approx |e_r(x_1) + e_r(x_2)| \leq |e_r(x_1)| + |e_r(x_2)| \quad (1-15)$$

$$\left|e_r\left(\frac{x_1}{x_2}\right)\right| \approx |e_r(x_1) - e_r(x_2)| \leq |e_r(x_1)| + |e_r(x_2)| \quad (1-16)$$

因此,和、差的误差限不超过各数的误差限之和,积、商的相对误差限不超过各数的相对误差限之和.

**例 1** 设  $y = x^n$ , 求  $y$  的相对误差与  $x$  的相对误差之间的关系.

[解] 由式(1-10)得

$$e_r(y) = d(\ln x^n) = nd(\ln x) = ne_r(x)$$

所以  $x^n$  的相对误差是  $x$  的相对误差的  $n$  倍. 特别地,  $\sqrt{x}$  的相对误差是  $x$  的相对误差的一半.

**例 2** 假定运算中数据都准确到两位小数,试求  $x^* = 1.21 \times 3.65 - 9.81$  的绝对误差限和相对误差限,计算结果有几位有效数字?

[解] 由式(1-11)和式(1-12)得

$$e(x^*) = 3.65 \times e(1.21) + 1.21 \times e(3.65) - e(9.81)$$

因为式中数据都准确到两位小数,即其误差限均为  $\frac{1}{2} \times 10^{-2}$ ,故有

$$\begin{aligned} |e(x^*)| &\leq 3.65 \times |e(1.21)| + 1.21 \times |e(3.65)| + |e(9.81)| \\ &\leq (3.65 + 1.21 + 1) \times \frac{1}{2} \times 10^{-2} = 0.0293 \end{aligned}$$

$$|e_r(x^*)| = \frac{|e(x^*)|}{|x^*|} \leq \frac{0.0293}{5.3935} \approx 0.0054$$

所以,  $x^*$  的绝对误差限为 0.0293, 相对误差限为 0.0054, 计算结果有两位有效数字.

### 1.3.2 算法的数值稳定性

计算一个数学问题,需要预先设计好由已知数据计算问题结果的运算顺序,这就是算法.例如,计算积分值

$$I_n = \int_0^1 \frac{x^n}{x+5} dx \quad (n = 0, 1, 2, \dots)$$

由关系式

$$I_n + 5I_{n-1} = \int_0^1 \frac{x^n}{x+5} dx + \int_0^1 \frac{5x^{n-1}}{x+5} dx = \int_0^1 x^{n-1} dx = \frac{1}{n}$$

且

$$\frac{1}{6(n+1)} < I_n < \frac{1}{5(n+1)}$$

可设计如下两种算法:

算法 I 取  $I_0 = \int_0^1 \frac{1}{x+5} dx = \ln 1.2$ , 按公式

$$I_n = \frac{1}{n} - 5I_{n-1} \quad (n = 1, 2, \dots) \quad (1-17)$$

依次计算  $I_1, I_2, \dots$  的近似值.

算法 II 取  $I_n^* \approx \frac{1}{2} \left[ \frac{1}{6(n+1)} + \frac{1}{5(n+1)} \right]$ , 按公式

$$I_{k-1} = \frac{1}{5} \left( \frac{1}{k} - I_k \right) \quad (k = n, n-1, \dots, 1) \quad (1-18)$$

依次计算  $I_{n-1}, I_{n-2}, \dots, I_0$  的近似值.

分别取

$$I_0^* = 0.182\ 321\ 55$$

$$I_{14}^* = \frac{1}{2} \left( \frac{1}{6 \times 15} + \frac{1}{5 \times 15} \right) \approx 0.012\ 222\ 22$$

算法 I, II 的计算结果见表 1-1.

表 1-1

| $n$ | $I_n$ (按算法 I 计算) | $I_n$ (按算法 II 计算) |
|-----|------------------|-------------------|
| 0   | 0.182 321 55     | 0.182 321 55      |
| 1   | 0.088 392 25     | 0.088 392 22      |
| 2   | 0.058 038 75     | 0.058 038 92      |
| 3   | 0.043 139 58     | 0.043 138 73      |
| 4   | 0.034 302 08     | 0.034 306 33      |
| 5   | 0.028 489 58     | 0.028 468 35      |
| 6   | 0.024 218 75     | 0.024 324 91      |
| 7   | 0.021 763 39     | 0.021 232 60      |
| 8   | 0.016 183 05     | 0.018 836 99      |
| 9   | 0.030 195 88     | 0.016 926 17      |
| 10  | -0.050 979 41    | 0.015 369 14      |
| 11  | 0.345 806 12     | 0.014 063 39      |
| 12  | -0.645 697 26    | 0.013 016 36      |
| 13  | 8.305 409 38     | 0.011 841 27      |
| 14  | -41.455 618 31   | 0.012 222 22      |

由表 1-1 中结果可见,按算法 I 得到  $I_{10}^* < 0$ ,这显然是错的,因为对任意  $n \geq 0$ ,均有

$$I_n > \frac{1}{6(n+1)} > 0$$

而按算法 II 计算,尽管  $I_{14}^*$  取值精度不高,其误差限

$$\varepsilon = \frac{1}{2} \left( \frac{1}{75} - \frac{1}{90} \right) \approx 0.001\ 1$$

但递推计算得到的  $I_0^*$  却有 8 位有效数字. 为什么会出现这样的现象? 下面的分析说明,这是舍入误差在计算过程中传播所引起的后果.

设  $I_0^*$  有误差  $e_0$ . 假设计算过程中不产生新的舍入误差,则由式(1-17)得

$$e_n = I_n - I_n^* = -5I_{n-1} + 5I_{n-1}^* = -5e_{n-1} \quad (n = 1, 2, \dots)$$

从而有

$$e_n = (-5)^n e_0 \quad (1-19)$$

即原始数据  $I_0^*$  的误差  $e_0$  经公式(1-17)计算一次, 误差就扩大到 5 倍, 因而由算法 I 计算出的  $I_n^*$  的误差应是  $e_0$  的  $5^n$  倍. 在上例中,  $I_0^*$  有 8 位有效数字, 故

$$\varepsilon_0 = \frac{1}{2} \times 10^{-8}$$

由式(1-19)得

$$\varepsilon_9 = 5^9 \varepsilon_0 = \frac{10^9 \times 10^{-8}}{2^{10}} = \frac{10}{1\,024} > \frac{1}{2} \times 10^{-2}$$

而  $I_9^* = 0.030\,195\,88$ , 这说明  $I_9^*$  已无一位有效数字.

完全类似地, 按算法 II 从  $I_k$  计算  $I_{k-1}$ , 应有

$$e_{k-1} = -\frac{1}{5}e_k$$

从而有

$$e_0 = \left(-\frac{1}{5}\right)^n e_n$$

因此, 从  $I_{14}^*$  出发计算到  $I_0^*$  时, 其误差  $e_0$  已缩小到  $\frac{e_{14}}{5^{14}}$ .

上述事实说明, 对于同一数学问题, 使用的算法不同, 效果也大不相同. 称计算过程中舍入误差不增长的算法具有数值稳定性, 否则就是数值不稳定的. 在上例中, 算法 II 具有数值稳定性, 而算法 I 则是数值不稳定的. 显然, 只有选用数值稳定性好的算法, 才能求得较准确的结果.

## § 1.4 数值计算中应注意的问题

由于误差的影响, 计算过程中可能出现一些坏现象, 应尽量注意以下几个方面.

### 1. 避免两个相近的数相减

由式(1-11), 两数之差  $u = x - y$  的相对误差为

$$e_r(u) = e_r(x - y) = \frac{e(x) - e(y)}{x - y}$$

当  $x$  与  $y$  很接近时,  $u$  的相对误差很大, 有效数字位数将严重丢失. 例如, 求  $x = \sqrt{1 + 10^{-7}} - 1$  的近似值时, 如果运算过程中取 8 位有效数字, 则得  $x \approx 0.5 \times 10^{-7}$ , 只有一位有效数字. 为了避免这种情形出现, 常常改变计算公式. 当然也可

以通过在计算过程中多保留几位有效数字来改善计算效果,但不如前者有效. 如把  $x = \sqrt{1 + 10^{-7}} - 1$  变换成以下公式计算

$$x = \frac{10^{-7}}{\sqrt{1 + 10^{-7}} + 1}$$

计算过程仍保留 8 位有效数字, 则得  $x \approx 0.499\,999\,99 \times 10^{-7}$ , 结果仍有 8 位有效数字. 又如

**例 3** 利用四位数学用表求  $x = 1 - \cos 2^\circ$  的近似值.

[解] 由查表得  $\cos 2^\circ \approx 0.999\,4$ , 于是

$$x = 1 - \cos 2^\circ \approx 1 - 0.999\,4 = 0.000\,6 = x^*$$

且

$$|x - x^*| = |\cos 2^\circ - 0.999\,4| \leq \frac{1}{2} \times 10^{-4}$$

故近似值  $x^* = 0.000\,6$  有一位有效数字.

若改用公式

$$1 - \cos 2^\circ = 2\sin^2 1^\circ$$

计算, 查表得  $\sin 1^\circ \approx 0.017\,5$ , 于是

$$x = 2\sin^2 1^\circ \approx 2 \times (0.017\,5)^2 = 0.612\,5 \times 10^{-3} = x^*$$

此时

$$|e(x^*)| = |4\sin 1^\circ e(\sin 1^\circ)| \leq 4 \times 0.017\,5 \times \frac{1}{2} \times 10^{-4} < \frac{1}{2} \times 10^{-5}$$

故  $x^* = 0.612\,5 \times 10^{-3}$  至少有 2 位有效数字.

一般地, 当  $x$  接近于零时, 应作变换

$$1 - \cos x = 2\sin^2 \frac{x}{2}, \quad \frac{1 - \cos x}{\sin x} = \frac{\sin x}{1 + \cos x}$$

当  $x$  充分大时, 应作变换

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

$$\frac{1}{x} - \frac{1}{x+1} = \frac{1}{x(x+1)}$$

等等.

## 2. 避免大数“吃”小数的现象

计算机在进行运算时, 首先要把参加运算的数对阶, 即把两数都写成绝对值小于 1 而阶码相同的数. 如  $a = 10^9 + 1$ , 必须改写成

$$a = 0.1 \times 10^{10} + 0.000\,000\,000\,1 \times 10^{10}$$

如果计算机只能表示 8 位小数, 则算出  $a = 0.1 \times 10^{10}$ , 大数“吃”了小数. 这种情

形有时是允许的,有时则可能得出错误的结果.

#### 例 4 求二次方程

$$x^2 - (10^9 + 1)x + 10^9 = 0$$

的根.

[解] 利用因式分解容易求出,此方程的两个根为  $x_1 = 10^9, x_2 = 1$ ,但若用求根公式,则得

$$x = \frac{10^9 + 1 \pm \sqrt{(10^9 + 1)^2 - 4 \times 10^9}}{2}$$

若用 8 位小数的计算机运算,由于对阶,有

$$10^9 + 1 \approx 10^9$$

$$\sqrt{(10^9 + 1)^2 - 4 \times 10^9} \approx 10^9$$

这样求得  $x_1 = 10^9, x_2 = 0$ , 结果显然是错的. 为避免这种情形出现,也可采用改变计算公式的方法. 如将式

$$x_2 = \frac{10^9 + 1 - \sqrt{(10^9 + 1)^2 - 4 \times 10^9}}{2}$$

改变成

$$x_2 = \frac{2 \times 10^9}{10^9 + 1 + \sqrt{(10^9 + 1)^2 - 4 \times 10^9}}$$

则有

$$x_2 \approx \frac{2 \times 10^9}{10^9 + 10^9} = 1$$

此结果是准确的.

#### 3. 避免除数的绝对值远小于被除数的绝对值

由式(1-13)

$$e\left(\frac{x}{y}\right) = \frac{ye(x) - xe(y)}{y^2}$$

故当  $|y| \ll |x|$  时,舍入误差可能增大很多.

#### 4. 要简化计算,减少运算次数,提高效率

求一个问题的数值解往往有多种算法,不同的算法需要不同的计算量,而计算量的大小会影响误差的积累. 例如,若选用级数

$$\ln 2 = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \cdots + \frac{(-1)^{n-1}}{n} + \cdots$$

的前  $n$  项部分和来计算  $\ln 2$  的近似值,截断误差限为  $\frac{1}{n+1}$ . 如果要求误差小于  $10^{-5}$ ,则  $n \geq 10^5$ ,即要取前十万项求和. 这样做不仅计算量大,而且舍入误差的积累将使有效数字丢失严重. 如果利用级数

$$\ln \frac{1+x}{1-x} = 2x \left( 1 + \frac{1}{3}x^2 + \frac{1}{5}x^4 + \cdots + \frac{x^{2n}}{2n+1} + \cdots \right)$$

来计算, 当  $x = \frac{1}{3}$  时, 有

$$\ln 2 = \frac{2}{3} \left[ 1 + \frac{1}{3 \times 9} + \frac{1}{5 \times 9^2} + \cdots + \frac{1}{(2n+1) \times 9^n} + \cdots \right]$$

取前 5 项之和作为近似值, 产生的截断误差为

$$\begin{aligned} e &= \frac{2}{3} \left( \frac{1}{11 \times 9^5} + \frac{1}{13 \times 9^6} + \frac{1}{15 \times 9^7} + \cdots \right) \\ &< \frac{2}{3} \times \frac{1}{11 \times 9^5} \left( 1 + \frac{1}{9} + \frac{1}{9^2} + \cdots \right) \\ &= \frac{2}{3 \times 11 \times 9^5} \times \frac{1}{1 - \frac{1}{9}} \\ &= \frac{1}{12 \times 11 \times 9^4} < \frac{1}{10^5} \end{aligned}$$

显然, 第二种算法比第一种算法有效.

又如计算多项式

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \quad (1-20)$$

的值. 若直接按式(1-20)计算, 共需作  $\frac{n(n+1)}{2}$  次乘法与  $n$  次加法. 若按秦九韶算法

$$\begin{cases} u_n = a_n \\ u_k = x u_{k+1} + a_k \quad (k = n-1, n-2, \cdots, 1, 0) \\ P_n(x) = u_0 \end{cases} \quad (1-21)$$

计算, 即将式(1-20)改写成如下形式

$$P_n(x) = a_0 + x \{ a_1 + x [ \cdots x (a_{n-2} + x (a_{n-1} + a_n x)) \cdots ] \} \quad (1-22)$$

则只需作  $n$  次乘法和  $n$  次加法.

由以上例子可以看出选择合适的公式, 简化计算十分重要.

### 5. 选用数值稳定性好的算法

## 评 注

本章介绍了数值计算方法和误差的基本概念, 以及误差在近似计算中的传播规律. 误差在数值计算中的危害是十分严重的, 若未能控制误差的传播与积累, 误差会将精确值完全淹没, 得到完全错误的结果. 因此, 误差的分析及其控制

是一个非常重要的问题,本章讨论了选用和设计算法应遵循的若干原则,这对防止误差的传播和积累、确定方法的稳定性以及判别计算结果的可靠性有很大的帮助. 实际计算中,误差分析是一个困难而复杂的问题,尤其对大型科学计算的误差估计仍无定量的估计方法. 在数值计算方法中,一般注重误差的定性分析,即只讨论各种算法的数值稳定性,只要是数值稳定的,就不再进行舍入误差估计;截断误差将针对各种问题结合具体算法讨论.

附录 A 中简单介绍了本课程所需的通用计算机软件 MATLAB 的部分命令,如何更加深入和熟练地运用 MATLAB 进行数值计算,以及开发利用 MATLAB 工具箱解决大型科学计算与仿真实验问题,有待于读者自己的摸索与实验,在以后的各章中也会有更详细的计算实例. 读者也可参阅有关 MATLAB 方面的书籍或上网搜寻相关知识.

## 习 题 一

1. 按四舍五入原则,求下列各数的具有 4 位有效数字的近似值:

$$168.957, 3.000\ 45, 73.225\ 0, 0.001\ 526\ 32$$

2. 设下列各数均为经过四舍五入后得到的近似值,试求各数的绝对误差限和相对误差限:

$$a = 3\ 580, b = 0.004\ 76, c = 2\ 958 \times 10^{-2}, d = 0.143\ 0 \times 10^8$$

3. 已知  $a = 1.203\ 1, b = 0.978$  是经过四舍五入后得到的近似值,问  $a + b, a \times b$  有几位有效数字?

4. 设  $x > 0, x$  的相对误差为  $\delta$ , 求  $\ln x$  的绝对误差.

5. 求  $\sqrt{2}$  的近似值  $x^*$ , 使其相对误差不超过 0.1%.

6. 要使  $\sqrt{20}$  的近似值有小于 0.1% 的相对误差, 要取几位有效数字?

7. 正方形的边长约 100 cm, 问测量边长时误差应多大, 才能保证面积的误差不超过  $1\ \text{cm}^2$ ?

8. 计算球的体积, 为使其相对误差限为 1%, 问测量半径  $R$  时, 相对误差最大为多少?

9. 设  $s = \frac{1}{2}gt^2$ , 假定  $g$  是准确的, 而对  $t$  的测量有  $\pm 0.1\ \text{s}$  的误差, 试证当  $t$  增大时,  $s$  的绝对误差增大而相对误差却减小.

10. 已知  $\sqrt{783} \approx 27.983$  有 5 位有效数字, 试求方程

$$x^2 - 56x + 1 = 0$$

的两个根, 使它们至少有 4 位有效数字.



11. 设  $I_n = e^{-1} \int_0^1 x^n e^x dx$ , 求证:

(1)  $I_n = 1 - nI_{n-1} \quad (n=0, 1, 2, \dots)$

(2) 利用(1)中的公式正向递推计算时误差逐步增大;反向递推计算时误差逐步减小.

12. 试改变下列表达式,使计算结果比较精确:

(1)  $\frac{1}{1+2x} - \frac{1-x}{1+x}, |x| \ll 1$

(2)  $\sqrt{x + \frac{1}{x}} - \sqrt{x - \frac{1}{x}}, |x| \gg 1$

(3)  $\frac{\tan x - \sin x}{x^2}, |x| \ll 1 \text{ 且 } x \neq 0$

## 数值实验

1. 利用循环语句,计算数列  $\sqrt{5}, \sqrt{\sqrt{5}}, \sqrt{\sqrt{\sqrt{5}}}, \dots$  的极限,要求误差小于  $10^{-8}$ .

2. 利用级数

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

计算无理数  $\pi$  的近似值. 由于交错级数的部分和数列  $S_n$  在级数的和上下摆动,截断误差将小于第一个被舍弃的项  $|a_{n+1}|$  的绝对值. 试分析:若要使截断误差小于  $10^{-4}$  或  $10^{-8}$ ,应取多少项求和? 并分别计算  $\pi$  的近似值.

3. 在同一坐标系下,利用 plot 函数画出函数

$$y = \sin x, y_n = \sum_{i=0}^n (-1)^i \frac{x^{2i+1}}{(2i+1)!}, \quad (n = 2, 5, 10)$$

的图形,并加标注说明各条曲线的含义.

4. 分别将区间  $[-10, 10]$  分为 100, 200, 400 等份,利用 mesh 或 surf 命令画出二元函数

$$z = e^{-|x|} + \cos(x + y) + \frac{1}{x^2 + y^2 + 1}$$

的三维图形.

## 第二章 解线性方程组的直接方法

在科学研究和工程计算中,经常会遇到求解线性方程组的问题. 这类问题一方面来源于广泛的实际应用,如电网络分析、电磁场数值计算、结构设计问题等;另一方面来源于数学问题本身,如数据的插值与拟合、用差分方法或有限元法求解微分方程与积分方程等. 因此,线性方程组的数值解法是数值计算的重要内容.

设  $n$  阶线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \cdots \cdots \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases} \quad (2-1)$$

其矩阵形式为

$$Ax = b \quad (2-2)$$

其中

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

如果线性方程组(2-1)的系数矩阵  $A$  的行列式不为零,即  $\det A \neq 0$ ,则该方程组有唯一解. 由克拉默(Cramer)法则,其解为

$$x_i = \frac{\det A_i}{\det A} \quad (i = 1, 2, \cdots, n)$$

其中  $A_i$  为用方程组(2-1)的右端向量  $b$  代替  $A$  中第  $i$  列向量所得的矩阵. 这种方法需要计算  $n+1$  个  $n$  阶行列式并作  $n$  次除法,而每个  $n$  阶行列式计算需作  $(n-1) \times n!$  次乘法,计算量十分惊人. 如  $n=30$  时,就需作约  $2.38 \times 10^{35}$  次乘法. 可见 Cramer 法则在理论上是绝对正确的,但当  $n$  较大时,在实际计算中却是不可行的.

解线性方程组的方法大致可分为两类:直接方法和迭代法. 直接方法是指假设计算过程中不产生舍入误差,经过有限次运算可求得方程组的精确解的方法. 迭代法是从解的某个近似值出发,通过构造一个无穷序列去逼近精确解的方法.

一般地,有限步内得不到精确解.

本章介绍几种常用的解线性方程组的直接方法及有关问题. 在以下文中,如不作特别说明,均假设方程组(2-1)的系数矩阵非奇异,方程组存在唯一解.

## § 2.1 消 去 法

### 2.1.1 Gauss 消去法

如果线性方程组的系数矩阵是三角形矩阵,则方程组极易求解. 解线性方程组的直接方法的基本思想是将方程组(2-1)变形为等价的三角形方程组,然后求解. Gauss 消去法是最基本和最简单的直接方法,它由消元过程和回代过程构成. Gauss 消去法的基本思想和过程是:将方程组逐列逐行消去变量,转化为等价的上三角形方程组,此过程称为消元过程. 然后按着方程组相反顺序求解上三角形方程组,得原方程组的解,此过程称为回代过程.

为统一起见,将方程组(2-1)改写成

$$\begin{cases} a_{11}^{(1)}x_1 + a_{12}^{(1)}x_2 + \cdots + a_{1n}^{(1)}x_n = b_1^{(1)} \\ a_{21}^{(1)}x_1 + a_{22}^{(1)}x_2 + \cdots + a_{2n}^{(1)}x_n = b_2^{(1)} \\ \dots\dots\dots \\ a_{n1}^{(1)}x_1 + a_{n2}^{(1)}x_2 + \cdots + a_{nn}^{(1)}x_n = b_n^{(1)} \end{cases} \quad (2-3)$$

简记为  $A^{(1)}x = b^{(1)}$ , 其中  $A^{(1)} = A, b^{(1)} = b$ .

一般地,求解  $n$  阶方程组(2-1)的 Gauss 消去法的步骤如下:

**消元过程**

第一步:设  $a_{11}^{(1)} \neq 0$ , 记  $l_{i1} = a_{i1}^{(1)} / a_{11}^{(1)} (i=2, 3, \dots, n)$ , 将方程组(2-3)中第  $i$  个方程减去第 1 个方程乘以  $l_{i1} (i=2, 3, \dots, n)$ , 完成第一次消元,得(2-3)的同解方程组

$$\begin{cases} a_{11}^{(1)}x_1 + a_{12}^{(1)}x_2 + \cdots + a_{1n}^{(1)}x_n = b_1^{(1)} \\ a_{22}^{(2)}x_2 + \cdots + a_{2n}^{(2)}x_n = b_2^{(2)} \\ \dots\dots\dots \\ a_{n2}^{(2)}x_2 + \cdots + a_{nn}^{(2)}x_n = b_n^{(2)} \end{cases} \quad (2-4)$$

其中  $a_{ij}^{(2)} = a_{ij}^{(1)} - l_{i1}a_{1j}^{(1)}, b_i^{(2)} = b_i^{(1)} - l_{i1}b_1^{(1)} (i, j=2, 3, \dots, n)$ . 方程组(2-4)简记为  $A^{(2)}x = b^{(2)}$ .

第二步:设  $a_{22}^{(2)} \neq 0$ , 记  $l_{i2} = a_{i2}^{(2)} / a_{22}^{(2)} (i=3, \dots, n)$ . 将方程组(2-4)中第  $i$  个方程减去第 2 个方程乘以  $l_{i2} (i=3, \dots, n)$ , 完成第二次消元.

第  $k$  步:设第  $k-1$  次消元完成后得方程组(2-3)的同解方程组为

$$\left\{ \begin{array}{l} a_{11}^{(1)} x_1 + a_{12}^{(1)} x_2 + \cdots + a_{1k}^{(1)} x_k + \cdots + a_{1n}^{(1)} x_n = b_1^{(1)} \\ a_{22}^{(2)} x_2 + \cdots + a_{2k}^{(2)} x_k + \cdots + a_{2n}^{(2)} x_n = b_2^{(2)} \\ \cdots \cdots \cdots \\ a_{kk}^{(k)} x_k + \cdots + a_{kn}^{(k)} x_n = b_k^{(k)} \\ \cdots \cdots \cdots \\ a_{nk}^{(k)} x_k + \cdots + a_{nn}^{(k)} x_n = b_n^{(k)} \end{array} \right. \quad (2-5)$$

简记为  $A^{(k)} x = b^{(k)}$ . 设  $a_{kk}^{(k)} \neq 0$ , 记  $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$  ( $i = k+1, \cdots, n$ ). 将式(2-5)中第  $i$  个方程减去第  $k$  个方程乘以  $l_{ik}$  ( $i = k+1, \cdots, n$ ), 完成第  $k$  次消元, 得同解方程组

$$\left\{ \begin{array}{l} a_{11}^{(1)} x_1 + a_{12}^{(1)} x_2 + \cdots + a_{1k}^{(1)} x_k + a_{1k+1}^{(1)} x_{k+1} + \cdots + a_{1n}^{(1)} x_n = b_1^{(1)} \\ a_{22}^{(2)} x_2 + \cdots + a_{2k}^{(2)} x_k + a_{2k+1}^{(2)} x_{k+1} + \cdots + a_{2n}^{(2)} x_n = b_2^{(2)} \\ \cdots \cdots \cdots \\ a_{kk}^{(k)} x_k + a_{k,k+1}^{(k)} x_{k+1} + \cdots + a_{kn}^{(k)} x_n = b_k^{(k)} \\ a_{k+1,k+1}^{(k+1)} x_{k+1} + \cdots + a_{k+1,n}^{(k+1)} x_n = b_{k+1}^{(k+1)} \\ \cdots \cdots \cdots \\ a_{n,k+1}^{(k+1)} x_{k+1} + \cdots + a_{nn}^{(k+1)} x_n = b_n^{(k+1)} \end{array} \right. \quad (2-6)$$

其中  $a_{ij}^{(k+1)} = a_{ij}^{(k)} - l_{ik} a_{kj}^{(k)}$ ,  $b_i^{(k+1)} = b_i^{(k)} - l_{ik} b_k^{(k)}$  ( $i, j = k+1, \cdots, n$ ). 按上述作法, 完成  $n-1$  次消元后, 方程组(2-3)化成同解的上三角形方程组

$$\left\{ \begin{array}{l} a_{11}^{(1)} x_1 + a_{12}^{(1)} x_2 + a_{13}^{(1)} x_3 + \cdots + a_{1n}^{(1)} x_n = b_1^{(1)} \\ a_{22}^{(2)} x_2 + a_{23}^{(2)} x_3 + \cdots + a_{2n}^{(2)} x_n = b_2^{(2)} \\ a_{33}^{(3)} x_3 + \cdots + a_{3n}^{(3)} x_n = b_3^{(3)} \\ \cdots \cdots \cdots \\ a_{nn}^{(n)} x_n = b_n^{(n)} \end{array} \right. \quad (2-7)$$

简记为  $A^{(n)} x = b^{(n)}$

### 回代过程

按变量的逆序逐步回代得方程组(2-3)的解.

$$\left\{ \begin{array}{l} x_n = b_n^{(n)} / a_{nn}^{(n)} \\ x_k = (b_k^{(k)} - \sum_{l=k+1}^n a_{kl}^{(k)} x_l) / a_{kk}^{(k)} \quad (k = n-1, n-2, \cdots, 1) \end{array} \right. \quad (2-8)$$

### 算法 2.1

1. 输入  $A = (a_{ij})$ ,  $b = (b_1, \cdots, b_n)^T$ ,  $n$ , 置  $k=1$ .

2. 若  $a_{kk} \neq 0$ , 转 3; 否则输出失败信息, 停机.
3. 对  $i = k + 1, \dots, n$ , 置  $a_{ik}/a_{kk} \Rightarrow a_{ik}, b_i - a_{ik}b_k \Rightarrow b_i$ ;  
对  $j = k + 1, \dots, n$ , 置  $a_{ij} - a_{ik}a_{kj} \Rightarrow a_{ij}$ .
4. 若  $k = n - 1$ , 转 5; 否则  $k + 1 \Rightarrow k$ , 转 2.
5. 若  $a_{nn} = 0$ , 输出失败信息, 停机; 否则, 置  $b_n/a_{nn} \Rightarrow b_n$ .
6. 对  $k = n - 1, n - 2, \dots, 1$ , 置  $(b_k - \sum_{l=k+1}^n a_{kl}b_l)/a_{kk} \Rightarrow b_k$ .
7. 输出  $x = b$ , 停机.

### Gauss 消去法的计算量

由于计算机作乘除运算所需时间远大于作加减运算所需时间, 故我们只讨论乘除运算量.

由消去法步骤知, 在进行第  $k$  次消元时, 需作除法  $n - k$  次, 乘法  $(n - k)(n - k + 1)$  次, 故消元过程中乘除运算总量为

$$\text{乘法次数} \quad \sum_{k=1}^{n-1} (n - k)(n - k + 1) = \frac{n}{3}(n^2 - 1)$$

$$\text{除法次数} \quad \sum_{k=1}^{n-1} (n - k) = \frac{n}{2}(n - 1)$$

由式(2-8), 在回代过程中, 计算  $x_k$  需要  $n - k + 1$  次乘除法, 整个回代过程需要乘除运算的总量为

$$\sum_{k=1}^n (n - k + 1) = \frac{n}{2}(n + 1)$$

所以, Gauss 消去法的乘除总运算量为

$$\begin{aligned} N &= \frac{n}{3}(n^2 - 1) + \frac{n}{2}(n - 1) + \frac{n}{2}(n + 1) \\ &= \frac{n^3}{3} + n^2 - \frac{n}{3} \end{aligned}$$

由上式容易求出, 用 Gauss 消去法求解 30 阶线性方程组时, 共需乘除运算的次数为

$$\frac{30^3}{3} + 30^2 - \frac{30}{3} = 9\,890$$

远少于用 Cramer 法则求解所需的乘除运算量.

### 2.1.2 主元素消去法

Gauss 消去法虽然简单易行, 但计算过程要求  $a_{kk}^{(k)}$  (称为主元素) 均不为零, 否则 Gauss 消去法不能进行. 另一方面, 有时主元素  $a_{kk}^{(k)} \neq 0$ , 但  $|a_{kk}^{(k)}|$  很小, 用其作除数, 会使

$$|l_{ik}^{(k)}| = \left| \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \right|$$

很大,导致舍入误差的严重扩大以及计算结果的不正确.

### 例 1 求解方程组

$$\begin{cases} 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \\ 2.0x_1 + 4.5x_2 + 0.36x_3 = 0.020 \\ 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \end{cases}$$

其中所有系数均有 2 位有效数字.

[解] 为减小误差,计算过程中保留 3 位有效数字. 按 Gauss 消去法求解, 第一次消元得同解方程组

$$\begin{cases} 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \\ 0.100x_2 - 12.0x_3 = -24.0 \\ -10.0x_2 - 24.5x_3 = -59.0 \end{cases}$$

第二次消元得

$$\begin{cases} 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \\ 0.100x_2 - 12.0x_3 = -24.0 \\ -1220x_3 = -2460 \end{cases}$$

回代得解

$$x_3 = 2.02, \quad x_2 = 2.40, \quad x_1 = -5.80$$

容易验证,该方程组的准确解为

$$x_1 = -2.60, \quad x_2 = 1.00, \quad x_3 = 2.00$$

显然两者相差很大. 但若在解方程组前,先把方程的次序交换一下,将方程组改写成

$$\begin{cases} 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \\ 2.0x_1 + 4.5x_2 + 0.36x_3 = 0.020 \\ 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \end{cases}$$

再用 Gauss 消去法求解,消元后得同解方程组

$$\begin{cases} 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \\ 4.12x_2 - 2.24x_3 = -0.364 \\ 2.99x_3 = 5.99 \end{cases}$$

回代得解

$$x_3 = 2.00, \quad x_2 = 1.00, \quad x_1 = -2.60$$

与准确解相同.

产生上述现象的原因在于舍入误差. 因为按原方程组的方程顺序进行消元时, 主元  $a_{11}^{(1)} = 0.50$ ,  $a_{22}^{(2)} = 0.100$  都比较小, 以它们为除数就增长了舍入误差, 从而导致计算结果不准确. 为了在计算过程中抑制舍入误差的增长, 应尽量避免小主元的出现. 如例 1 中第二种解法, 通过交换方程次序, 选取绝对值大的元素作主元. 基于这种想法导出了主元素消去法.

### 1. 列主元素法

为简便起见, 用方程组 (2-1) 的增广矩阵

$$[A, b] = \begin{bmatrix} a_{11} & \cdots & a_{1n} & b_1 \\ \vdots & & \vdots & \vdots \\ a_{n1} & \cdots & a_{nn} & b_n \end{bmatrix} \quad (2-9)$$

表示方程组, 并直接在增广矩阵上运算.

列主元素法的基本思想是在每次消元前, 在要消去未知数的系数中找到绝对值最大的系数作为主元, 通过方程对换将其换到对角线上, 然后进行消元. 其具体步骤如下:

第一步: 首先在矩阵 (2-9) 的第 1 列中选取绝对值最大的元, 比如为  $a_{i_1 1}$ , 则  $|a_{i_1 1}| = \max_{1 \leq i \leq n} |a_{i1}|$ . 将 (2-9) 中第 1 行与第  $i_1$  行互换. 为方便起见, 记行互换后的增广矩阵为  $[A^{(1)}, b^{(1)}]$ , 然后进行第一次消元, 得矩阵

$$[A^{(2)}, b^{(2)}] = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & \vdots & & \vdots & \vdots \\ & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{bmatrix}$$

第二步: 在矩阵  $[A^{(2)}, b^{(2)}]$  的第 2 列中选主元, 比如  $a_{i_2 2}^{(2)}$ , 使  $|a_{i_2 2}^{(2)}| = \max_{2 \leq i \leq n} |a_{i2}^{(2)}|$ . 将矩阵  $[A^{(2)}, b^{(2)}]$  的第 2 行与第  $i_2$  行互换, 再进行第二次消元, 得矩阵  $[A^{(3)}, b^{(3)}]$ .

第  $k$  步: 在矩阵  $[A^{(k)}, b^{(k)}]$  的第  $k$  列中选主元, 如  $a_{i_k k}^{(k)}$ , 使  $|a_{i_k k}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|$ . 将  $[A^{(k)}, b^{(k)}]$  的第  $k$  行与第  $i_k$  行互换, 进行第  $k$  次消元.

如此经过  $n-1$  步, 增广矩阵 (2-9) 被化成上三角形矩阵, 最后由回代过程求解.

在上述过程中, 主元是按列选取的, 列主元素法由此得名. 例 1 中的第二种解法就是按列主元素法进行的.

### 2. 全主元素法

如果不是按列选主元, 而是在全体待选系数  $a_{ij}^{(k)}$  ( $i, j = k, k+1, \dots, n$ ) 中选取主元, 则得到全主元素法, 其计算过程如下:

第一步:在全体系数  $a_{ij} (i, j = 1, \dots, n)$  中选取绝对值最大的元作为主元,并通过行与列的互换把它换到  $a_{11}^{(1)}$  的位置,然后进行第一次消元,得到矩阵  $[A^{(2)}, b^{(2)}]$ .

第  $k$  步:在矩阵  $A^{(k)}$  的右下方  $n - k + 1$  阶子矩阵的所有元素  $a_{ij}^{(k)} (i, j = k, k + 1, \dots, n)$  中,选取绝对值最大的元作为主元,并通过行与列的互换将它换到  $a_{kk}^{(k)}$  的位置,然后进行第  $k$  次消元.

经过  $n - 1$  次消元后,得到与方程组 (2-1) 同解的上三角形方程组,再由回代过程求解.

**例 2** 用主元素法求解线性方程组

$$\begin{cases} x_1 + x_2 + x_3 = 6 \\ 12x_1 - 3x_2 + 3x_3 = 15 \\ -18x_1 + 3x_2 - x_3 = -15 \end{cases}$$

计算过程保留三位小数.

[解] 按列主元素法,求解过程如下:

$$\begin{aligned} & \left[ \begin{array}{cccc} 1 & 1 & 1 & 6 \\ 12 & -3 & 3 & 15 \\ -18 & 3 & -1 & -15 \end{array} \right] \xrightarrow{\text{第一行与第三行互换}} \left[ \begin{array}{cccc} -18 & 3 & -1 & -15 \\ 12 & -3 & 3 & 15 \\ 1 & 1 & 1 & 6 \end{array} \right] \\ & \xrightarrow{\text{第一次消元}} \left[ \begin{array}{cccc} -18 & 3 & -1 & -15 \\ 0 & -1 & 2.333 & 5 \\ 0 & 1.167 & 0.944 & 5.167 \end{array} \right] \\ & \xrightarrow{\text{第二,三行互换}} \left[ \begin{array}{cccc} -18 & 3 & -1 & -15 \\ 0 & 1.167 & 0.944 & 5.167 \\ 0 & -1 & 2.333 & 5 \end{array} \right] \\ & \xrightarrow{\text{第二次消元}} \left[ \begin{array}{cccc} -18 & 3 & -1 & -15 \\ 0 & 1.167 & 0.944 & 5.167 \\ 0 & 0 & 3.142 & 9.428 \end{array} \right] \end{aligned}$$

由回代过程得解

$$x_3 = 3.001 \quad x_2 = 2.000 \quad x_1 = 1.000$$

按全主元素法,求解过程如下:

$$\left[ \begin{array}{cccc} 1 & 1 & 1 & 6 \\ 12 & -3 & 3 & 15 \\ -18 & 3 & -1 & -15 \end{array} \right] \xrightarrow{\text{第一,三行互换}} \left[ \begin{array}{cccc} -18 & 3 & -1 & -15 \\ 12 & -3 & 3 & 15 \\ 1 & 1 & 1 & 6 \end{array} \right]$$



$$\begin{array}{l}
 \xrightarrow{\text{第一次消元}} \left[ \begin{array}{cccc} \boxed{-18} & 3 & -1 & -15 \\ 0 & -1 & \boxed{2.333} & 5 \\ 0 & 1.167 & 0.944 & 5.167 \end{array} \right] \\
 \xrightarrow{\text{第二,三列互换}} \left[ \begin{array}{cccc} \boxed{-18} & -1 & 3 & -15 \\ 0 & \boxed{2.333} & -1 & 5 \\ 0 & 0.944 & 1.167 & 5.167 \end{array} \right] \\
 \xrightarrow{\text{第二次消元}} \left[ \begin{array}{cccc} \boxed{-18} & -1 & 3 & -15 \\ 0 & \boxed{2.333} & -1 & 5 \\ 0 & 0 & 1.572 & 3.144 \end{array} \right]
 \end{array}$$

由回代过程得解

$$x_2 = 2.000 \quad x_3 = 3.000 \quad x_1 = 1.000$$

例 2 的计算结果表明,全主元素法的精度优于列主元素法,这是由于全主元素是在全体系数中选主元,故它对控制舍入误差十分有效.但全主元素法在计算过程中,需同时作行与列的互换,因而程序比较复杂,计算时间较长.列主元素法的精度虽稍低于全主元素法,但其计算简单,工作量大为减少,且计算经验与理论分析均表明,它与全主元素法同样具有良好的数值稳定性,故列主元素法是求解中小型稠密线性方程组的最好方法之一.

## § 2.2 直接三角分解法

### 2.2.1 Gauss 消去法的矩阵形式

如果用增广矩阵  $[A, b]$  表示线性方程组 (2-1), Gauss 消去法的消元过程可以通过用一串初等矩阵左乘增广矩阵表示.例如,第一次消元等价于用初等矩阵

$$L_1 = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -l_{21} & 1 & 0 & \cdots & 0 \\ -l_{31} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -l_{n1} & 0 & 0 & \cdots & 1 \end{bmatrix} \quad (2-10)$$

左乘矩阵  $[A^{(1)}, b^{(1)}] = [A, b]$ , 其中  $l_{i1} = a_{i1}^{(1)} / a_{11}^{(1)} (i=2, 3, \cdots, n)$ , 即

$$[A^{(2)}, b^{(2)}] = L_1 [A^{(1)}, b^{(1)}]$$

一般地,第  $k$  次消元等价于用初等矩阵

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1\ k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{bmatrix} \quad (2-11)$$

左乘矩阵  $[A^{(k)}, b^{(k)}]$ , 其中  $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$  ( $i = k+1, \dots, n$ ), 经过  $n-1$  次消元后得到

$$\begin{aligned} [A^{(n)}, b^{(n)}] &= \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & & \ddots & \vdots & \vdots \\ & & & a_{nn}^{(n)} & b_n^{(n)} \end{bmatrix} \\ &= L_{n-1} [A^{(n-1)}, b^{(n-1)}] \\ &= L_{n-1} L_{n-2} \cdots L_1 [A^{(1)}, b^{(1)}] \end{aligned}$$

因为  $L_k$  ( $k=1, 2, \dots, n-1$ ) 均为非奇异阵, 故它们的逆矩阵存在. 容易求出

$$L_k^{-1} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{k+1\ k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & & 1 \end{bmatrix} \quad (2-12)$$

令

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ & & & & 1 \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn-1} & 1 \end{bmatrix} \quad (2-13)$$

于是有

$$\begin{aligned} [A, b] &= [A^{(1)}, b^{(1)}] \\ &= L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} [A^{(n)}, b^{(n)}] \end{aligned}$$

即

$$[A, b] = [LA^{(n)}, Lb^{(n)}] \quad (2-14)$$

这说明,消元过程实际上是把系数矩阵  $A$  分解成单位下三角形矩阵与上三角形矩阵的乘积的过程,其中  $L$  为单位下三角形矩阵,  $U$  为上三角形矩阵

$$L = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn-1} & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ & u_{22} & u_{23} & \cdots & u_{2n} \\ & & \ddots & & \vdots \\ & & & & u_{nn} \end{bmatrix}$$

上述分解称为杜利特尔 (Doolittle) 分解,也称为 LU 分解.

当系数矩阵进行三角分解后,求解方程组  $Ax = b$  的问题就变得十分容易,它等价于求解两个三角形方程组  $Ly = b$  和  $Ux = y$ . 因此,解线性方程组问题可转化为矩阵的三角分解问题.

### 2.2.2 矩阵的直接三角分解

Gauss 消去法要求主元素  $a_{kk}^{(k)} \neq 0 (k = 1, \cdots, n-1)$ ,将这一条件与矩阵  $A$  自身的性质联系起来,有

**定理 2.1** 设  $A$  为  $n$  阶方阵,若  $A$  的顺序主子式  $A_i (i = 1, 2, \cdots, n-1)$  均不为零,则矩阵  $A$  存在唯一的 Doolittle 分解.

[证明] 当  $j=1$  时,因为  $A_1 = a_{11} \neq 0$ ,由 2.2.1 中的讨论,存在矩阵

$$L_1 = \begin{bmatrix} 1 & & & \\ -l_{21} & 1 & & \\ \vdots & & \ddots & \\ -l_{n1} & & & 1 \end{bmatrix}$$

其中  $l_{i1} = a_{i1}/a_{11} (i = 2, 3, \cdots, n)$ ,使得

$$A^{(2)} = L_1 A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ & \vdots & & \vdots \\ & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{bmatrix}$$

由行列式的性质易得,  $A^{(2)}$  的各阶顺序主子式  $A_i^{(2)} (i=1, 2, \dots, n-1)$  满足

$$\det A_i^{(2)} = \det A_i \neq 0$$

假定结论对  $j=k-1$  成立, 即

$$A^{(k)} = L_{k-1} \cdots L_1 A = \begin{bmatrix} a_{11} & a_{12} & \cdots & \cdots & a_{1n} \\ & a_{22}^{(2)} & \cdots & \cdots & a_{2n}^{(2)} \\ & & \ddots & & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & & \vdots & & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{bmatrix}$$

且  $\det A_i^{(k)} \neq 0 (i=1, 2, \dots, n-1)$ , 因为

$$\det A_k^{(k)} = \det A_{k-1}^{(k)} \times a_{kk}^{(k)} \neq 0$$

于是有  $a_{kk}^{(k)} \neq 0$ , 故存在矩阵

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{bmatrix}$$

其中  $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)} (i=k+1, \dots, n)$ , 使得

$$A^{(k+1)} = L_k A^{(k)} = \begin{bmatrix} a_{11} & a_{12} & \cdots & \cdots & a_{1n} \\ & a_{22}^{(2)} & \cdots & \cdots & a_{2n}^{(2)} \\ & & \ddots & & \vdots \\ & & & a_{k+1,k+1}^{(k+1)} & \cdots & a_{k+1,n}^{(k+1)} \\ & & & \vdots & & \vdots \\ & & & a_{n,k+1}^{(k+1)} & \cdots & a_{nn}^{(k+1)} \end{bmatrix}$$

且  $\det A_i^{(k+1)} \neq 0 (i=1, 2, \dots, n-1)$ .

按归纳法原理, 存在初等矩阵  $L_1, \dots, L_{n-1}$ , 使得

$$A^{(n)} = L_{n-1} \cdots L_2 L_1 A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ & a_{22}^{(2)} & a_{23}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & a_{33}^{(3)} & \cdots & a_{3n}^{(3)} \\ & & & \ddots & \vdots \\ & & & & a_{nn}^{(n)} \end{bmatrix}$$

由式(2-14)得

$$A = LA^{(n)} = LU$$

其中  $L = L_1^{-1}L_2^{-1}\cdots L_{n-1}^{-1}$  为单位下三角形矩阵,  $U = A^{(n)}$  为上三角形矩阵. 存在性得证. 唯一性. 设矩阵  $A$  有两种 Doolittle 分解

$$A = LU = \tilde{L}\tilde{U} \quad (2-15)$$

当  $A$  非奇异时,  $L, \tilde{L}, U, \tilde{U}$  均为非奇异阵. 于是由式(2-15), 有

$$\tilde{L}^{-1}L = \tilde{U}U^{-1}$$

由于单位下(上)三角形矩阵的逆仍是单位下(上)三角形矩阵, 上(下)三角形矩阵的乘积仍为上(下)三角形矩阵, 故有

$$\tilde{L}^{-1}L = \tilde{U}U^{-1} = I$$

所以  $L = \tilde{L}, U = \tilde{U}$ .

下面讨论如何对  $A$  进行 LU 分解.

两个矩阵相等就是它们的对应元素都相等, 比较  $A$  与  $LU$  的对应元素, 即可得出  $L, U$  中元素的计算公式.  $A = LU$ , 即

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix} = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \\ & & & u_{nn} \end{bmatrix}$$

由矩阵的乘法规则, 得

$$a_{1j} = u_{1j} \quad (j = 1, 2, \cdots, n)$$

$$a_{ij} = \begin{cases} \sum_{k=1}^{i-1} l_{ik}u_{kj} + u_{ij}, & j \geq i \\ \sum_{k=1}^j l_{ik}u_{kj}, & j < i \end{cases} \quad (i = 2, \cdots, n)$$

由此可得计算  $l_{ij}$  和  $u_{ij}$  的公式

$$\begin{cases} u_{1j} = a_{1j} & (j = 1, 2, \cdots, n) \\ u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik}u_{kj} & (i = 2, \cdots, n; j = i, \cdots, n) \\ l_{ij} = (a_{ij} - \sum_{k=1}^{j-1} l_{ik}u_{kj})/u_{jj} & (j = 1, 2, \cdots, n-1; i = j+1, \cdots, n) \end{cases}$$

计算过程应按第 1 行,第 1 列,第 2 行,第 2 列,……的顺序进行. 具体步骤如下:

1. 计算  $U$  的第 1 行,  $L$  的第 1 列

$$u_{1j} = a_{1j} \quad (j = 1, 2, \dots, n)$$

$$l_{i1} = a_{i1}/u_{11} \quad (i = 2, 3, \dots, n)$$

2. 计算  $U$  的第  $r$  行,  $L$  的第  $r$  列 ( $r = 2, \dots, n$ )

$$u_{rj} = a_{rj} - \sum_{k=1}^{r-1} l_{rk} u_{kj} \quad (j = r, r+1, \dots, n)$$

$$l_{ir} = (a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr})/u_{rr} \quad (i = r+1, \dots, n)$$

实际计算时,常将  $u_{rj}$  存放在  $a_{rj}$  的位置上,将  $l_{ir}$  存放在  $a_{ir}$  的位置上,按表 2-1 所示的紧凑格式及顺序进行.

表 2-1

|                  |                  |                  |     |                  |
|------------------|------------------|------------------|-----|------------------|
| $(a_{11})u_{11}$ | $(a_{12})u_{12}$ | $(a_{13})u_{13}$ | ... | $(a_{1n})u_{1n}$ |
| $(a_{21})l_{21}$ | $(a_{22})u_{22}$ | $(a_{23})u_{23}$ | ... | $(a_{2n})u_{2n}$ |
| $(a_{31})l_{31}$ | $(a_{32})l_{32}$ | $(a_{33})u_{33}$ | ... | $(a_{3n})u_{3n}$ |
| $\vdots$         | $\vdots$         | $\vdots$         |     | $\vdots$         |
| $(a_{n1})l_{n1}$ | $(a_{n2})l_{n2}$ | $(a_{n3})l_{n3}$ | ... | $(a_{nn})u_{nn}$ |

例 3 求矩阵

$$A = \begin{bmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{bmatrix}$$

的三角分解.

[解] 按式(2-16)及表 2-1,得表 2-2

表 2-2

|                       |                           |                                 |
|-----------------------|---------------------------|---------------------------------|
| (2)2                  | (2)2                      | (3)3                            |
| $(4)\frac{4}{2}=2$    | $(7)7-2\times 2=3$        | $(7)7-2\times 3=1$              |
| $(-2)\frac{-2}{2}=-1$ | $(4)[4-(-1)\times 2]/3=2$ | $(5)5-2\times 1-(-1)\times 3=6$ |

所以

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 2 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 2 & 2 & 3 \\ 0 & 3 & 1 \\ 0 & 0 & 6 \end{bmatrix}$$

$$A = LU$$

### 2.2.3 直接三角分解法

如果线性方程组  $Ax = b$  的系数矩阵已进行三角分解,  $A = LU$ , 则解方程组  $Ax = b$  等价于求解两个三角形方程组  $Ly = b$ ,  $Ux = y$ , 即由

$$Ly = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & & 1 & & \\ \vdots & & & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \quad (2-17)$$

可求出

$$y_k = \begin{cases} b_1 & (k=1) \\ b_k - \sum_{j=1}^{k-1} l_{kj} y_j & (k=2, \cdots, n) \end{cases} \quad (2-18)$$

再由

$$Ux = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad (2-19)$$

解得

$$x_k = \begin{cases} y_n / u_{nn} & (k=n) \\ (y_k - \sum_{j=k+1}^n u_{kj} x_j) / u_{kk} & (k=n-1, \cdots, 1) \end{cases} \quad (2-20)$$

容易看出, 式(2-18)和式(2-16)的运算规律相同, 故在利用三角分解求解方程组时, 只需把右端向量  $b$  列在表 2-1 的最后一列, 按  $u_{kj}$  的计算方法即可求出  $y_k$ .

表 2-3 是求解线性方程组的紧凑格式, 其计算顺序与计算方法与三角分解相同. 按表 2-3 计算后, 只需按式(2-20)即可求出方程组的解.

表 2-3

|                  |                  |                  |     |                  |            |
|------------------|------------------|------------------|-----|------------------|------------|
| $(a_{11})u_{11}$ | $(a_{12})u_{12}$ | $(a_{13})u_{13}$ | ... | $(a_{1n})u_{1n}$ | $(b_1)y_1$ |
| $(a_{21})l_{21}$ | $(a_{22})u_{22}$ | $(a_{23})u_{23}$ | ... | $(a_{2n})u_{2n}$ | $(b_2)y_2$ |
| $(a_{31})l_{31}$ | $(a_{32})l_{32}$ | $(a_{33})u_{33}$ | ... | $(a_{3n})u_{3n}$ | $(b_3)y_3$ |
| $\vdots$         | $\vdots$         | $\vdots$         |     | $\vdots$         | $\vdots$   |
| $(a_{n1})l_{n1}$ | $(a_{n2})l_{n2}$ | $(a_{n3})l_{n3}$ | ... | $(a_{nn})u_{nn}$ | $(b_n)y_n$ |

例 4 用紧凑格式解线性方程组

$$\begin{cases} 3x_1 + 2x_2 + 5x_3 = 6 \\ -x_1 + 4x_2 + 3x_3 = 5 \\ x_1 - x_2 + 3x_3 = 1 \end{cases}$$

[解] 按表 2-3 计算

|                      |                                                                    |                                        |                                        |
|----------------------|--------------------------------------------------------------------|----------------------------------------|----------------------------------------|
| (3)3                 | (2)2                                                               | (5)5                                   | (6)6                                   |
| $(-1) - \frac{1}{3}$ | $(4)4 + \frac{2}{3} = \frac{14}{3}$                                | $(3)3 + \frac{5}{3} = \frac{14}{3}$    | $(5)5 + 2 = 7$                         |
| $(1)\frac{1}{3}$     | $(-1)\left(-1 - \frac{2}{3}\right) / \frac{14}{3} = -\frac{5}{14}$ | $(3)3 + \frac{5}{3} - \frac{5}{3} = 3$ | $(1)1 + \frac{5}{2} - 2 = \frac{3}{2}$ |

所以

$$L = \begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{3} & 1 & 0 \\ \frac{1}{3} & -\frac{5}{14} & 1 \end{bmatrix} \quad y = \begin{bmatrix} 6 \\ 7 \\ \frac{3}{2} \end{bmatrix} \quad U = \begin{bmatrix} 3 & 2 & 5 \\ 0 & \frac{14}{3} & \frac{14}{3} \\ 0 & 0 & 3 \end{bmatrix}$$

解方程组

$$Ux = \begin{bmatrix} 3 & 2 & 5 \\ 0 & \frac{14}{3} & \frac{14}{3} \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 7 \\ \frac{3}{2} \end{bmatrix}$$

得原方程组的解

$$x_3 = \frac{1}{2}, \quad x_2 = 1, \quad x_1 = \frac{1}{2}.$$

#### 2.2.4 列主元素的三角分解法

从直接三角分解公式(2-16)知,当  $u_{jj}=0 (j=1, \dots, n-1)$  时,计算将中断;



或当  $|u_{jj}| (j=1, \dots, n-1)$  很小时, 按公式(2-16)计算会使舍入误差扩散. 但若矩阵  $A$  非奇异, 可增加列选主元过程, 即先进行一系列的行交换, 保证 LU 分解能够实现. 从而有如下定理.

**定理 2.2** 设矩阵  $A$  非奇异, 则存在置换矩阵  $P$ , 使得  $PA$  有唯一的 Doolittle 分解其中  $L$  是单位下三角形矩阵,  $U$  是上三角形矩阵, 且  $|l_{ij}| \leq 1$ .

证明从略.

在实际计算中, 行交换与分解计算总是交替进行的. 假设按表 2-3 的分解计算已完成了  $k-1$  步, 即有表 2-4

表 2-4

|                  |                  |                              |                  |                          |
|------------------|------------------|------------------------------|------------------|--------------------------|
| $(a_{11})u_{11}$ | $(a_{12})u_{12}$ | ...                          | $(a_{1n})u_{1n}$ | $(b_1)y_1$               |
| $(a_{21})l_{21}$ | $(a_{12})u_{22}$ | ...                          | $(a_{2n})u_{2n}$ | $(b_2)y_2$               |
| $\vdots$         | $\vdots$         |                              | $\vdots$         | $\vdots$                 |
|                  |                  | $(a_{k-1\ k-1})u_{k-1\ k-1}$ | ...              | $(a_{k-1\ n})u_{k-1\ n}$ |
|                  |                  |                              |                  | $(b_{k-1})y_{k-1}$       |
| $(a_{k1})l_{k1}$ | $(a_{k2})l_{k2}$ | ...                          | $(a_{kk})$       | ...                      |
|                  |                  | $(a_{k\ k-1})l_{k\ k-1}$     | $(a_{kn})$       | $(b_k)$                  |
| $\vdots$         | $\vdots$         | $\vdots$                     | $\vdots$         | $\vdots$                 |
| $(a_{n1})l_{n1}$ | $(a_{n2})l_{n2}$ | ...                          | $(a_{nk})$       | ...                      |
|                  |                  | $(a_{n\ k-1})l_{n\ k-1}$     | $(a_{nn})$       | $(b_n)$                  |

下面开始第  $k$  步分解, 为避免用零或绝对值很小的数作除数, 由公式(2-16)引入量

$$s_i = a_{ik} - \sum_{j=1}^{k-1} l_{ij}u_{jk} \quad (i = k, k+1, \dots, n)$$

于是  $s_k$  即为公式(2-16)中的  $u_{kk}$ . 比较  $|s_i| (i = k, k+1, \dots, n)$  的大小, 设

$$|s_{i_k}| = \max_{k \leq i \leq n} |s_i|$$

取  $s_{i_k}$  作为第  $k$  次的主元素  $u_{kk}$ , 交换表 2-4 中的第  $k$  行与第  $i_k$  行, 然后再作第  $k$  步分解. 结合紧凑格式表 2-4 将  $u_{ij}, l_{ij}$  的值存入原  $a_{ij}$  的存储单元, 将  $y_k, x_k$  的值先后存入  $b_k$  的存储单元, 得具体算法如下:

### 算法 2.2

1. 输入  $A = (a_{ij})_{n \times n}, b = (b_i)_{n \times 1}$ .
2. 对于  $k=1, 2, \dots, n-1$  进行消元.

对于  $i = k, k+1, \dots, n$ , 计算  $s_i = a_{ik} - \sum_{j=1}^{k-1} l_{ij}u_{jk}$ , 若  $|s_{i_k}| = \max_{k \leq i \leq n} |s_i|$ , 则

对于  $j=1, 2, \dots, n$ , 交换  $a_{kj} \leftrightarrow a_{i_k j}, b_k \leftrightarrow b_{i_k}$ ;

对于  $j = k, k+1, \dots, n$ , 置  $a_{kj} - \sum_{r=1}^{k-1} l_{kr} u_{rj} = u_{kj} \Rightarrow a_{kj}, b_k - \sum_{r=1}^{k-1} l_{kr} b_r = y_k \Rightarrow b_k$ ;

对于  $i = k+1, \dots, n$ , 置  $(a_{ik} - \sum_{r=1}^{k-1} l_{ir} u_{rk}) / u_{kk} = l_{ik} \Rightarrow a_{ik}$ .

3. 解方程组  $Ux = y$ . 置  $b_n / u_{nn} = x_n \Rightarrow b_n$ ,

对于  $k = n-1, \dots, 1$ , 置  $(y_k - \sum_{r=k+1}^n l_{kr} y_r) / u_{kk} = x_k \Rightarrow b_k$ .

4. 输出  $x = b$ , 停机.

算法 2.2 中仍保留  $u_{ij}, l_{ij}$ , 仅为了体现算法结构的清晰, 编程时一般直接用

$a_{ij}$ , 且当  $r_2 < r_1$  时,  $\sum_{r=r_1}^{r_2} (\quad) = 0$ .

**例 5** 用列主元素三角分解法求解方程组

$$\begin{cases} 2x_2 + x_4 = 0 \\ 2x_1 + 2x_2 + 3x_3 + 2x_4 = -2 \\ 4x_1 - 3x_2 + x_4 = -7 \\ 6x_1 + x_2 - 6x_3 - 5x_4 = 6 \end{cases}$$

[解] 对增广矩阵  $[A, b]$  进行分解计算

$$[A, b] = \left[ \begin{array}{ccccc|c} 0 & 2 & 0 & 1 & 0 & s_1 = 0 \\ 2 & 2 & 3 & 2 & -2 & s_2 = 2 \\ 4 & -3 & 0 & 1 & -7 & s_3 = 4 \\ 6 & 1 & -6 & -5 & 6 & s_4 = 6 \end{array} \right]$$

$$\xrightarrow[\text{交换第 1, 4 行}]{\text{主元素 } s_4 = 6} \left[ \begin{array}{ccccc|c} 6 & 1 & -6 & -5 & 6 \\ 2 & 2 & 3 & 2 & -2 \\ 4 & -3 & 0 & 1 & -7 \\ 0 & 2 & 0 & 1 & 0 \end{array} \right]$$

$$\xrightarrow{\text{计算 } u_{ij}, l_{ij}} \left[ \begin{array}{ccccc|c} 6 & 1 & -6 & -5 & 6 \\ \frac{1}{3} & \left[ \begin{array}{cccc} 2 & 3 & 2 & -2 \end{array} \right] & s_2 = 2 - \frac{1}{3} = \frac{5}{3} \\ \frac{2}{3} & \left[ \begin{array}{cccc} -3 & 0 & 1 & -7 \end{array} \right] & s_3 = -3 - \frac{2}{3} = -\frac{11}{3} \\ 0 & \left[ \begin{array}{cccc} 2 & 0 & 1 & 0 \end{array} \right] & s_4 = 2 - 0 = 2 \end{array} \right]$$

$$\begin{array}{l}
 \xrightarrow{\substack{\text{主元素 } s_3 = -\frac{11}{3} \\ \text{交换第 2,3 行}}} \left[ \begin{array}{ccccc} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -3 & 0 & 1 & -7 \\ \frac{1}{3} & 2 & 3 & 2 & -2 \\ 0 & 2 & 0 & 1 & 0 \end{array} \right] \\
 \\
 \xrightarrow{\text{计算 } u_{2j}, l_{22}, y_2} \left[ \begin{array}{ccccc} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11 \\ \frac{1}{3} & -\frac{5}{11} & 3 & 2 & -2 \\ 0 & -\frac{6}{11} & 0 & 1 & 0 \end{array} \right] \quad \begin{array}{l} s_3 = 3 - \left( -2 - \frac{20}{11} \right) = \frac{75}{11} \\ s_4 = 0 - \left( 0 - \frac{24}{11} \right) = \frac{24}{11} \end{array} \\
 \\
 \xrightarrow{\substack{\text{主元素 } s_3 = \frac{75}{11} \\ \text{计算 } u_{3j}, l_{33}, y_3}} \left[ \begin{array}{ccccc} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11 \\ \frac{1}{3} & -\frac{5}{11} & \frac{75}{11} & \frac{62}{11} & -9 \\ 0 & -\frac{6}{11} & \frac{8}{25} & 1 & 0 \end{array} \right] \\
 \\
 \xrightarrow{\text{计算 } u_{44}, y_4} \left[ \begin{array}{ccccc} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11 \\ \frac{1}{3} & -\frac{5}{11} & \frac{75}{11} & \frac{62}{11} & -9 \\ 0 & -\frac{6}{11} & \frac{8}{25} & \frac{39}{25} & -\frac{78}{25} \end{array} \right]
 \end{array}$$

解上三角形方程组

$$\begin{cases} 6x_1 + x_2 - 6x_3 - 5x_4 = 6 \\ -\frac{11}{3}x_2 + 4x_3 + \frac{13}{3}x_4 = -11 \\ \frac{75}{11}x_3 + \frac{62}{11}x_4 = -9 \\ \frac{39}{25}x_4 = -\frac{78}{25} \end{cases}$$

回代得原方程组的解

$$x_4 = -2, \quad x_3 = \frac{1}{3}, \quad x_2 = 1, \quad x_1 = -\frac{1}{2}$$

直接三角分解法计算量也是 $\frac{n^3}{3}$ 数量级. 由于计算出 $u_{ij}, l_{ij}$ 和 $y_k$ 之后, $a_{ij}, b_i$ 就不再有用, 故在编程计算时, 可将 $L, U$ 和 $y$ 存放在 $A, b$ 的存储单元中, 回代时 $x$ 取代 $y$ , 整个计算过程无需增加新的存储单元.

由于系数矩阵的三角分解与右端项无关. 因此, 直接三角分解法尤其适合求解系数矩阵相同, 而右端项不同的一系列方程组, 如求矩阵的逆矩阵、用反幂法求矩阵的特征值与特征向量等.

也可将矩阵 $A$ 分解成一个下三角形矩阵 $L$ 与一个单位上三角形矩阵 $U$ 的乘积, 矩阵的这种分解称为克劳特(Crout)分解. 由此引出的解线性方程组的方法也称为直接三角分解法.

在 MATLAB 中, 对于小型方程组, 可在命令窗口直接键入

```
>> x = A\b
```

得到用列主元素法求解方程组 $Ax = b$ 的解. 库函数 $\text{lu}()$ 执行矩阵的列主元素分解, 调用格式为

```
>> [L,U,P] = lu(A)    % 对矩阵 A 进行列主元素分解, P 为置换矩阵,
                        % 即 PA = LU, 则 A = P-1LU.
>> [L1,U1] = lu(A)    % 对矩阵 A 进行列主元素分解, 即 PA = LU1, 则
                        % A = (P-1L)U1 = L1U1.
```

如例 5 中先输入系数矩阵和右端项:

```
>> A = [0 2 0 1; 2 2 3 2; 4 -3 0 1; 6 1 -6 -5];
>> b = [0; -2; -7; 6];
```

再用左除命令得方程组 $Ax = b$ 的解:

```
>> x = (A\b)'          % 其中()'实现矩阵的转置运算.
x = -0.500 0   1.000 0   0.333 3   -2.000 0
```

对 $A$ 进行列主元素分解

```
>> [L,U,P] = lu(A)
L = 1.000 0           0           0           0
    0.666 7    1.000 0           0           0
    0.333 3   -0.454 5    1.000 0           0
           0   -0.545 5    0.320 0    1.000 0
```

$$\begin{aligned}
 U &= \begin{bmatrix} 6.000 & 0 & 1.000 & 0 & -6.000 & 0 & -5.000 & 0 \\ & 0 & -3.666 & 7 & 4.000 & 0 & 4.333 & 3 \\ & & 0 & 0 & 6.818 & 2 & 5.636 & 4 \\ & & & 0 & 0 & 0 & 1.560 & 0 \end{bmatrix} \\
 P &= \begin{bmatrix} 0 & 0 & 0 & 1 \\ & 0 & 0 & 1 & 0 \\ & 0 & 1 & 0 & 0 \\ & 1 & 0 & 0 & 0 \end{bmatrix}
 \end{aligned}$$

矩阵  $P$  左乘  $A$  实现第 1,4 行置换,第 2,3 行置换,与例 5 中的计算过程一致. 而

$$\begin{aligned}
 &>> [L1, U1] = lu(A) \\
 L1 &= \begin{bmatrix} & 0 & -0.545 & 5 & 0.320 & 0 & 1.000 & 0 \\ & 0.333 & 3 & -0.454 & 5 & 1.000 & 0 & 0 \\ & 0.666 & 7 & 1.000 & 0 & 0 & 0 & 0 \\ & 1.000 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\
 U1 &= \begin{bmatrix} 6.000 & 0 & 1.000 & 0 & -6.000 & 0 & -5.000 & 0 \\ & 0 & -3.666 & 7 & 4.000 & 0 & 4.333 & 3 \\ & & 0 & 0 & 6.818 & 2 & 5.636 & 4 \\ & & & 0 & 0 & 0 & 1.560 & 0 \end{bmatrix}
 \end{aligned}$$

其中  $L1$  并非单位下三角形矩阵,这是由于按列选主元素三角分解后又乘  $P^{-1}$  的缘故.

## § 2.3 特殊矩阵的三角分解法

在工程实际计算中,如三次样条插值或用差分法求解常微分方程边值问题,导出的线性方程组的系数矩阵  $A$  常常是稀疏的三对角形矩阵或  $A$  是对称正定阵,这使得  $A$  的三角分解也具有更简洁的形式,从而导出一些特殊的算法.

### 2.3.1 解三对角方程组的追赶法

方程组

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{bmatrix} \quad (2-21)$$

称为三对角方程组,其中系数矩阵  $A$  是一种特殊的系数矩阵,它的非零元素集

中分布在主对角线及其相邻的两条次对角线上,称为三对角矩阵.

如果  $A$  满足三角分解的条件,可将  $A$  分解成  $LU$ ,按 Doolittle 分解法不难证明如下定理.

**定理 2.3** 设方程组(2-21)中的系数矩阵  $A$  满足条件

$$\begin{cases} |b_1| > |c_1| > 0 \\ |b_i| \geq |a_i| + |c_i| \quad (a_i c_i \neq 0, \text{且 } i = 2, 3, \dots, n-1) \\ |b_n| > |a_n| > 0 \end{cases}$$

则  $A$  可唯一分解为

$$A = LU = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & 1 & & \\ & & \ddots & \ddots & \\ & & & l_n & 1 \end{bmatrix} \begin{bmatrix} u_1 & c_1 & & & \\ & u_2 & c_2 & & \\ & & u_3 & c_3 & \\ & & & \ddots & \ddots \\ & & & & \ddots & c_{n-1} \\ & & & & & u_n \end{bmatrix} \quad (2-22)$$

其中  $c_i (i = 1, 2, \dots, n-1)$  由原矩阵给出,且

$$\begin{cases} u_1 = b_1 \\ l_i = a_i / u_{i-1} \quad (i = 2, 3, \dots, n) \\ u_i = b_i - c_{i-1} l_i \end{cases} \quad (2-23)$$

当矩阵  $A$  三角分解后,解线性方程组(2-21)化为求解方程组  $Ly = d$  和  $Ux = y$ . 解  $Ly = d$ , 得

$$\begin{cases} y_1 = d_1 \\ y_k = d_k - l_k y_{k-1} \quad (k = 2, 3, \dots, n) \end{cases} \quad (2-24)$$

再解  $Ux = y$ , 得三对角方程组(2-21)的解

$$\begin{cases} x_n = y_n / u_n \\ x_k = (y_k - c_k x_{k+1}) / u_k \quad (k = n-1, n-2, \dots, 1) \end{cases} \quad (2-25)$$

称上述求解三对角方程组的方法为追赶法.

### 算法 2.3

1. 输入  $a = (a_2, \dots, a_n)$ ,  $b = (b_1, b_2, \dots, b_n)$ ,  $c = (c_1, c_2, \dots, c_{n-1})$ ,  $d = (d_1, d_2, \dots, d_n)$ ,  $n$ .
2. 对于  $i = 2, 3, \dots, n$ , 置  $a_i / b_{i-1} = l_i \Rightarrow a_i, b_i - c_{i-1} a_i = u_i \Rightarrow b_i, d_i - a_i d_{i-1} = y_i \Rightarrow d_i$ .
3. 置  $d_n / b_n = x_n \Rightarrow d_n$ .
4. 对于  $i = n-1, n-2, \dots, 1$ , 置  $(d_i - c_i d_{i+1}) / b_i = x_i \Rightarrow d_i$ .
5. 输出  $d = x$ , 停机.

追赶法是由于矩阵中出现了大量零元素,从而使得计算公式简化,计算量大为减少,其乘除法的运算量为  $5n - 4$ .

### 2.3.2 平方根法 (Cholesky 分解法)

**定理 2.4** 设  $A$  是对称正定矩阵,则存在唯一的非奇异下三角形矩阵  $L$ ,使得

$$A = LL^T \quad (2-26)$$

且  $L$  的对角元素皆为正数.

[证明] 存在性. 因为  $A$  对称正定,其各阶顺序主子式均大于零,由定理 2.1,矩阵  $A$  存在 Doolittle 分解,即

$$A = \tilde{L}U$$

其中  $\tilde{L}$  为单位下三角形矩阵,  $U$  为上三角形矩阵. 令  $D = \text{diag}(u_{11}, \dots, u_{nn})$ ,  $P = D^{-1}U$ , 则  $P$  为单位上三角形矩阵, 且  $A = \tilde{L}DP$ .

因为  $A$  对称,故有

$$A^T = P^T D \tilde{L}^T = A = \tilde{L}DP$$

由 Doolittle 分解的唯一性,得  $P^T = \tilde{L}$ , 即

$$A = P^T DP$$

对任意非零向量  $x, y = P^{-1}x \neq 0$ , 于是,由  $A$  正定得

$$\begin{aligned} x^T D x &= x^T (P^T)^{-1} A P^{-1} x \\ &= (P^{-1}x)^T A P^{-1} x = y^T A y > 0 \end{aligned}$$

即  $D$  正定,所以  $D$  的对角元素均为正数. 令

$$\tilde{D} = \text{diag}(\sqrt{u_{11}}, \dots, \sqrt{u_{nn}})$$

则

$$A = P^T \tilde{D}^T \tilde{D} P = (\tilde{D}P)^T \tilde{D}P = LL^T$$

其中  $L = (\tilde{D}P)^T$  为非奇异下三角形矩阵,且对角元素皆为正数.

唯一性. 假设存在非奇异下三角形矩阵  $G \neq L$ , 其对角元素皆为正数,且使得

$$A = LL^T = GG^T$$

于是有

$$L^T (G^T)^{-1} = L^{-1} L L^T (G^T)^{-1} = L^{-1} G G^T (G^T)^{-1} = L^{-1} G$$

因为  $L^T (G^T)^{-1}$  是上三角形矩阵,  $L^{-1} G$  是下三角形矩阵,故由上式必得

$$L^T (G^T)^{-1} = L^{-1} G = I$$

即  $G = L$ , 与假设矛盾.

矩阵的这种分解称为 Cholesky 分解. 用比较法可以导出  $L$  的计算公式. 设

$$L = \begin{bmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix}$$

比较  $A$  与  $LL^T$  的相应元素, 可得

$$\begin{cases} l_{kk} = (a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2)^{1/2} & (k = 1, 2, \cdots, n) \\ l_{ik} = (a_{ik} - \sum_{j=1}^{k-1} l_{ij}l_{kj})/l_{kk} & (i = k+1, \cdots, n) \end{cases} \quad (2-27)$$

计算顺序是按列进行, 即

$$l_{11} \rightarrow l_{i1} (i = 2, 3, \cdots, n) \rightarrow l_{22} \rightarrow l_{i2} (i = 3, \cdots, n) \rightarrow \cdots$$

当矩阵  $A$  完成 Cholesky 分解后, 求解方程组  $Ax = b$  就转化为依次求解方程组

$$Ly = b, \quad L^T x = y$$

它们的解分别为

$$y_k = (b_k - \sum_{j=1}^{k-1} l_{kj}y_j)/l_{kk} \quad (k = 1, 2, \cdots, n) \quad (2-28)$$

$$x_k = (y_k - \sum_{j=k+1}^n l_{jk}x_j)/l_{kk} \quad (k = n, n-1, \cdots, 1) \quad (2-29)$$

求解线性方程组的上述方法称为平方根法, 也称为 Cholesky 分解法. 这种方法无需选主元, 计算过程也是稳定的. 由于  $A$  的对称性, 平方根法的乘除运算量为  $n^3/6$  数量级, 约是 Gauss 消去法的一半. 但这种方法在求  $L$  时需作  $n$  次开方运算, 这样又增加了计算量.

### 2.3.3 改进的平方根法 ( $LDL^T$ 法)

定理 2.4 的证明过程表明, 对称正定矩阵  $A$  又可以作如下分解:

$$A = LDL^T$$

其中  $L$  为单位下三角形矩阵,  $D$  为对角阵. 记

$$L = \begin{bmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & 1 \end{bmatrix}$$



$$D = \text{diag}(d_1, \dots, d_n)$$

由比较法可以导出  $LDL^T$  分解的计算公式:

对  $k = 1, 2, \dots, n$

$$\begin{cases} d_k = a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2 d_j \\ l_{ik} = (a_{ik} - \sum_{j=1}^{k-1} l_{ij} d_j l_{kj}) / d_k \quad (i = k+1, \dots, n) \end{cases} \quad (2-30)$$

计算顺序如下

$$d_1 \rightarrow l_{i1} (i = 2, 3, \dots, n) \rightarrow d_2 \rightarrow l_{i2} (i = 3, \dots, n) \rightarrow d_3 \rightarrow \dots$$

为避免重复计算, 引进辅助量  $u_{ik} = l_{ik} d_k (k = 1, 2, \dots, n, i = k+1, \dots, n)$ , 则式 (2-30) 可改写成

$$\begin{cases} d_k = a_{kk} - \sum_{j=1}^{k-1} u_{kj} l_{kj} & (k = 1, 2, \dots, n) \\ u_{ik} = a_{ik} - \sum_{j=1}^{k-1} u_{ij} l_{kj} & (i = k+1, \dots, n) \\ l_{ik} = u_{ik} / d_k \end{cases} \quad (2-31)$$

按式 (2-31) 进行  $LDL^T$  分解, 乘除运算量与 Cholesky 分解相当, 且避免了开方运算.

矩阵  $A$  作  $LDL^T$  分解后, 解方程组  $Ax = b$  可分两步进行: 先解方程组  $Ly = b$ , 再由  $L^T x = D^{-1} y$  求  $x$ . 具体计算公式为

$$\begin{cases} y_1 = b_1 \\ y_k = b_k - \sum_{j=1}^{k-1} l_{kj} y_j & (k = 2, \dots, n) \\ x_n = y_n / d_n \\ x_k = \frac{y_k}{d_k} - \sum_{j=k+1}^n l_{jk} x_j & (k = n-1, \dots, 1) \end{cases} \quad (2-32)$$

求解线性方程组的这一方法称为改进平方根法, 也叫  $LDL^T$  法.

**例 6** 用改进平方根法求解方程组

$$\begin{cases} x_1 + 2x_2 + x_3 = 4 \\ 2x_1 + 5x_2 = 7 \\ x_1 + 14x_3 = 15 \end{cases}$$

[解] 容易验证, 系数矩阵

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 0 \\ 1 & 0 & 14 \end{bmatrix}$$

为对称正定阵. 按式(2-31)计算分解式, 得

$$d_1 = 1$$

$$u_{21} = 2, \quad u_{31} = 1$$

$$l_{21} = 2, \quad l_{31} = 1$$

$$d_2 = 5 - 2 \times 2 = 1$$

$$u_{32} = 0 - 1 \times 2 = -2, \quad l_{32} = -2$$

$$d_3 = 14 - 1 \times 1 - (-2) \times (-2) = 9$$

按式(2-32)计算, 得

$$y_1 = 4, \quad y_2 = 7 - 2 \times 4 = -1$$

$$y_3 = 15 - 1 \times 4 - (-2) \times (-1) = 9$$

$$x_3 = 1, \quad x_2 = -1 - (-2) \times 1 = 1$$

$$x_1 = 4 - 2 \times 1 - 1 \times 1 = 1$$

所以, 方程组的解为

$$x_1 = 1, \quad x_2 = 1, \quad x_3 = 1$$

平方根法与改进的平方根法不仅计算量仅是 Gauss 消去法的一半, 其数值稳定性良好, 是求解中小型稠密对称正定线性方程组的好方法.

MATLAB 中可用命令  $L = \text{chol}(A)$  计算矩阵  $A$  的 Cholesky 分解阵  $L$ .

#### 2.3.4 分块三角分解法

在求解微分方程数值解时, 如果用差分法或有限元法离散微分方程, 导出的线性方程组往往是稀疏的, 即系数矩阵中绝大多数元素是零, 而且具有分块结构. 这些稀疏性和分块性, 对于我们提高求解方程组的效率是很有帮助的. 作为一个例子, 我们讨论如下的分块三对角方程组

$$\begin{bmatrix} B_1 & C_1 & & & \\ A_2 & B_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & A_{n-1} & B_{n-1} & C_{n-1} \\ & & & A_n & B_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{n-1} \\ b_n \end{bmatrix}$$

在此假设所有的块矩阵  $A_i, B_i, C_i$  都是  $q \times q$  的矩阵,  $x_i$  和  $b_i$  都属于  $\mathbf{R}^q$ , 方程组为  $nq$  阶的.

首先考虑系数矩阵的 LU 分解. 设

$$A^{(k)} = \begin{bmatrix} B_1 & C_1 & & & \\ A_2 & B_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & A_{k-1} & B_{k-1} & C_{k-1} \\ & & & A_k & B_k \end{bmatrix}$$

那么当  $A^{(1)}, A^{(2)}, \dots, A^{(n)}$  非奇异时, 有

$$A = \begin{bmatrix} B_1 & C_1 & & & \\ A_2 & B_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & A_{n-1} & B_{n-1} & C_{n-1} \\ & & & A_n & B_n \end{bmatrix} = \begin{bmatrix} I & & & & \\ L_2 & I & & & \\ & \ddots & \ddots & & \\ & & L_{n-1} & I & \\ & & & L_n & I \end{bmatrix} \begin{bmatrix} U_1 & C_1 & & & \\ & U_2 & C_2 & & \\ & & \ddots & \ddots & \\ & & & U_{n-1} & C_{n-1} \\ & & & & U_n \end{bmatrix}$$

其中  $L_i, U_i$  按如下计算过程得到

$$U_1 = B_1$$

对于  $i=2, 3, \dots, n$ ,

由  $L_i U_{i-1} = A_i$ , 解得

$$L_i = A_i U_{i-1}^{-1},$$

$$U_i = B_i - L_i C_{i-1}$$

$A^{(1)}, A^{(2)}, \dots, A^{(n)}$  非奇异保证了  $U_i$  的可逆性, 使得  $A$  的 LU 分解可以进行. 随后未知向量  $x$  可以通过一追一赶得到

$$\begin{cases} y_1 = b_1 \\ y_i = b_i - L_{i-1} y_{i-1} \quad (i = 2, 3, \dots, n) \\ \begin{cases} \text{由 } U_n x_n = y_n \text{ 解得 } x_n \\ \text{再由 } U_i x_i = y_i - C_i x_{i+1} \text{ 解得 } x_i \quad (i = n-1, n-2, \dots, 1) \end{cases} \end{cases}$$

上述的追赶过程需要求解以块矩阵  $U_i$  为系数的方程组, 可以通过消元法或对  $U_i$  进行分解求解方程组.

稀疏线性方程组的求解是一个复杂而困难的问题, 随着未知变量排序的不同, 矩阵的分块结构也不同, 具体的分解过程也会不一样. 有需要的读者可参阅相关文献. 利用稀疏性和分块结构直接分解求解方程组只是一种选择, 更多的时

候我们将采用第三章介绍的迭代法.

## § 2.4 误差分析

由于不同的算法计算效果不同,因此必须进行误差分析.向量和矩阵的范数及矩阵的条件数在研究用数值方法所求得的线性方程组解的误差分析中起着十分重要的作用.

### 2.4.1 向量和矩阵的范数

#### 向量的范数

向量的范数是衡量向量大小的度量概念,它可以用来刻画  $\mathbf{R}^n$  中向量序列的收敛性.

一般地,定义范数如下:

**定义 2.1** 设对任意向量  $\mathbf{x} \in \mathbf{R}^n$ ,按一定的规则有一实数与之对应,记为  $\|\mathbf{x}\|$ ,若  $\|\mathbf{x}\|$  满足

- (1)  $\|\mathbf{x}\| \geq 0$ , 且  $\|\mathbf{x}\| = 0$  当且仅当  $\mathbf{x} = \mathbf{0}$ ;
- (2) 对任意实数  $\alpha$ , 都有  $\|\alpha\mathbf{x}\| = |\alpha| \|\mathbf{x}\|$ ;
- (3) 对任意  $\mathbf{x}, \mathbf{y} \in \mathbf{R}^n$ , 都有

$$\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$$

则称  $\|\mathbf{x}\|$  为向量  $\mathbf{x}$  的范数.

容易验证,向量的模符合以上三个条件,因而它是  $\mathbf{R}^n$  中的一种范数.设向量  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ ,按定义 2.1,向量  $\mathbf{x}$  的范数的具体形式可以有多种,常用的有以下三种:

- (1) 2-范数(向量的模)

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} \quad (2-33)$$

- (2) 1-范数

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| \quad (2-34)$$

- (3)  $\infty$ -范数

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} \{|x_i|\} \quad (2-35)$$

下面以  $\infty$ -范数为例进行验证.满足条件(1),(2)是显然的.由实数绝对值的性质,对任意实数  $x, y$ , 都有  $|x + y| \leq |x| + |y|$ . 故对任意向量  $\mathbf{x}, \mathbf{y} \in \mathbf{R}^n$ ,  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T, \mathbf{y} = (y_1, y_2, \dots, y_n)^T$ , 有

$$\|\mathbf{x} + \mathbf{y}\|_\infty = \max_{1 \leq i \leq n} \{|x_i + y_i|\} \leq \max_{1 \leq i \leq n} \{|x_i| + |y_i|\}$$

$$\leq \max_{1 \leq i \leq n} \{ |x_i| \} + \max_{1 \leq i \leq n} \{ |y_i| \} = \|x\|_{\infty} + \|y\|_{\infty}$$

所以条件(3)成立.

如果  $\mathbf{R}^n$  中有两个范数  $\|\cdot\|$  和  $\|\cdot\|'$ , 存在实数  $m, M > 0$ , 使得对任意  $n$  维向量  $x$ , 都有

$$m\|x\| \leq \|x\|' \leq M\|x\|$$

则称这两种范数是等价的. 不难证明, 1-范数, 2-范数和  $\infty$ -范数是等价的. 例如, 由式(2-33)与式(2-35), 对任意  $x \in \mathbf{R}^n$ , 有

$$\|x\|_{\infty} = \max_{1 \leq i \leq n} \{ |x_i| \} \leq \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \|x\|_2$$

设  $\|x\|_{\infty} = |x_j| = \max_{1 \leq i \leq n} \{ |x_i| \}$ , 则

$$\|x\|_{\infty} = |x_j| \geq \sqrt{\frac{x_1^2 + x_2^2 + \cdots + x_n^2}{n}} = \frac{\|x\|_2}{\sqrt{n}}$$

所以, 2-范数与  $\infty$ -范数是等价的. 事实上,  $\mathbf{R}^n$  中任意两种范数都是等价的.

今后如果不作说明,  $\|\cdot\|$  是指任意一种向量范数.

### 矩阵的范数

类似于向量范数的定义, 可以定义  $n \times n$  阶矩阵的范数.

**定义 2.2** 对任意  $n$  阶方阵  $A$ , 按一定的规则有一实数与之对应, 记为  $\|A\|$ . 若  $\|A\|$  满足

- (1)  $\|A\| \geq 0$ , 且  $\|A\| = 0$  当且仅当  $A = O$ ;
- (2) 对任意实数  $\alpha$ , 都有  $\|\alpha A\| = |\alpha| \|A\|$ ;
- (3) 对任意两个  $n$  阶方阵  $A, B$ , 都有

$$\|A + B\| \leq \|A\| + \|B\|$$

- (4)  $\|AB\| \leq \|A\| \|B\|$  (相容性条件)

则称  $\|A\|$  为矩阵  $A$  的范数.

这里(1)~(3)是向量范数定义的直接推广, 条件(4)则使矩阵范数在数值计算中使用更为方便.

**例 7** 若对任意  $n$  阶方阵  $A = (a_{ij})$ , 定义

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

试证  $\|A\|_1$  为矩阵  $A$  的范数.

[证明] (1) 显然  $\|A\|_1 \geq 0$ , 且  $\|A\|_1 = 0$ , 即

$$\max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}| = 0$$

当且仅当对所有  $j = 1, 2, \cdots, n$ , 有

$$\sum_{i=1}^n |a_{ij}| = 0$$

于是,对任意  $1 \leq i, j \leq n$ , 有

$$|a_{ij}| = 0$$

所以  $A = O$ .

(2) 对任意实数  $\alpha$ ,  $\alpha A = (\alpha a_{ij})$ , 于是

$$\begin{aligned}\|\alpha A\|_1 &= \max_{1 \leq j \leq n} \sum_{i=1}^n |\alpha a_{ij}| = \max_{1 \leq j \leq n} |\alpha| \sum_{i=1}^n |a_{ij}| \\ &= |\alpha| \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}| = |\alpha| \|A\|_1\end{aligned}$$

(3) 对任意两个  $n$  阶方阵  $A = (a_{ij})$ ,  $B = (b_{ij})$

$$\begin{aligned}\|A + B\|_1 &= \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij} + b_{ij}| \leq \max_{1 \leq j \leq n} \sum_{i=1}^n (|a_{ij}| + |b_{ij}|) \\ &\leq \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}| + \max_{1 \leq j \leq n} \sum_{i=1}^n |b_{ij}| = \|A\|_1 + \|B\|_1\end{aligned}$$

$$\begin{aligned}(4) \quad \|AB\|_1 &= \max_{1 \leq j \leq n} \sum_{i=1}^n \left| \sum_{k=1}^n a_{ik} b_{kj} \right| \leq \max_{1 \leq j \leq n} \sum_{i=1}^n \left( \sum_{k=1}^n |a_{ik}| |b_{kj}| \right) \\ &= \max_{1 \leq j \leq n} \sum_{k=1}^n \left( \sum_{i=1}^n |a_{ik}| \right) |b_{kj}| \leq \max_{1 \leq j \leq n} \sum_{k=1}^n \left( \max_{1 \leq i \leq n} \sum_{i=1}^n |a_{ik}| \right) |b_{kj}| \\ &= \left( \max_{1 \leq k \leq n} \sum_{i=1}^n |a_{ik}| \right) \max_{1 \leq j \leq n} \sum_{k=1}^n |b_{kj}| = \|A\|_1 \|B\|_1\end{aligned}$$

所以  $\|A\|_1$  是  $A$  的矩阵范数.

**定理 2.5** 设  $A$  为  $n$  阶方阵,  $\|\cdot\|$  是  $\mathbf{R}^n$  中的向量范数, 则

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

是一种矩阵范数, 称其为由向量范数  $\|\cdot\|$  诱导出的矩阵范数, 也称算子范数.

[证明] 设  $A = (a_{ij})$  为任意  $n$  阶方阵,  $x$  为任意  $n$  维非零向量. 因为

$$\frac{\|Ax\|}{\|x\|} = \left\| A \frac{x}{\|x\|} \right\|$$

$\frac{x}{\|x\|}$  为单位向量, 故

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \max_{\|x\|=1} \|Ax\| \quad (2-36)$$

(1) 显然  $\|A\| \geq 0$ , 若  $A = O$ , 则

$$\|A\| = \max_{\|x\|=1} \|Ax\| = 0$$

反之, 若  $\|A\| = 0$ , 即对任意非零向量, 有  $\|Ax\| = 0$ , 由定义 2.1 之(1), 得  $Ax = 0$ , 所以  $A = O$ .

(2) 对任意实数  $\alpha$ , 由式(2-36)及定义 2.1 有

$$\begin{aligned}\|\alpha A\| &= \max_{\|x\|=1} \|\alpha Ax\| = \max_{\|x\|=1} |\alpha| \|Ax\| \\ &= |\alpha| \max_{\|x\|=1} \|Ax\| = |\alpha| \|A\|\end{aligned}$$

(3) 对任意两个  $n$  阶方阵  $A$  和  $B$

$$\begin{aligned}\|A+B\| &= \max_{\|x\|=1} \|(A+B)x\| = \max_{\|x\|=1} \|Ax+Bx\| \\ &\leq \max_{\|x\|=1} (\|Ax\| + \|Bx\|) \\ &\leq \max_{\|x\|=1} \|Ax\| + \max_{\|x\|=1} \|Bx\| \\ &= \|A\| + \|B\|\end{aligned}$$

(4) 由式(2-36), 对任意  $n$  维非零向量  $x$ , 有

$$\frac{\|Ax\|}{\|x\|} \leq \|A\|$$

即

$$\|Ax\| \leq \|A\| \|x\|$$

于是

$$\begin{aligned}\|AB\| &= \max_{\|x\|=1} \|(AB)x\| = \max_{\|x\|=1} \|A(Bx)\| \\ &\leq \max_{\|x\|=1} \|A\| \|Bx\| \leq \max_{\|x\|=1} \|A\| \|B\| \|x\| \\ &= \|A\| \|B\|\end{aligned}$$

所以式(2-36)是矩阵  $A$  的范数.

由定理 2.5 的证明知, 由向量范数诱导出的矩阵范数除满足定义 2.2 中的(1)~(4)外, 还满足性质

(5) 对任意  $n$  维向量  $x$ , 都有

$$\|Ax\| \leq \|A\| \|x\| \quad (2-37)$$

这一性质称为矩阵范数与向量范数的相容性, 在误差分析中经常会用到.

常用的矩阵范数有四种, 前三种是由向量的 1-范数, 2-范数,  $\infty$ -范数诱导出的矩阵范数. 设  $A = (a_{ij})$  为  $n$  阶方阵:

(1) 1-范数

$$\|A\|_1 = \max_{\|x\|_1=1} \|Ax\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|$$

(2)  $\infty$ -范数

$$\|A\|_\infty = \max_{\|x\|_\infty=1} \|Ax\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$$

(3) 2-范数

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sqrt{\lambda_1}$$

其中  $\lambda_1$  是矩阵  $A^T A$  的最大特征值. 证明可参看[3].

#### (4) $F$ -范数

$$\|A\|_F = \left( \sum_{i=1}^n \sum_{j=1}^n a_{ij}^2 \right)^{1/2}$$

矩阵的  $F$ -范数可以认为是向量 2-范数的直接推广, 即将  $n$  阶方阵看作  $n^2$  维向量所得. 可以证明, 矩阵的  $F$ -范数与向量的 2-范数相容, 但  $F$ -范数不是由向量范数诱导出的矩阵范数. 因为  $\|I\|_F = \sqrt{n}$ , 而对任意一种由向量范数诱导出的矩阵范数  $\|\cdot\|$ , 都有

$$\|I\| = \max_{\|x\|=1} \|Ix\| = \max_{\|x\|=1} \|x\| = 1$$

如果将矩阵范数看作  $\mathbf{R}^{n^2}$  空间上的向量范数, 则由向量范数的等价性可得矩阵范数的等价性.

矩阵的误差可用矩阵范数表示. 设  $A^*$  是  $A$  的近似矩阵,  $\|A - A^*\|$ ,  $\|A - A^*\| / \|A\|$  分别称为  $A^*$  的关于范数  $\|\cdot\|$  的绝对误差与相对误差.

### 2.4.2 方程组的状态与条件数

一个实际问题化为数学问题, 初始数据往往会有误差, 即有扰动, 从而使计算结果产生误差, 因此需要研究扰动对解的影响.

**例 8** 容易看出, 方程组

$$\begin{cases} x_1 + x_2 = 2 \\ x_1 + 1.000\,01x_2 = 2 \end{cases}$$

的解为  $x_1 = 2, x_2 = 0$ . 而方程组

$$\begin{cases} x_1 + x_2 = 2 \\ x_1 + 1.000\,01x_2 = 2.000\,01 \end{cases}$$

的解为  $x_1 = 1, x_2 = 1$ .

比较这两个方程组可以看出, 它们只是右端项有微小的差别, 最大相对误差为  $\frac{1}{2} \times 10^{-5}$ , 但它们的解却大不相同, 解分量的相对误差至少为  $\frac{1}{2}$ .

当一个方程组, 由于系数矩阵或右端项的微小扰动, 而引起解发生巨大变化时, 称该方程组是“病态”的. 为了定量刻画方程组“病态”的程度, 下面对方程组  $Ax = b$  就系数矩阵或右端项分别有扰动的两种情形进行讨论.

首先考察右端项  $b$  的扰动对解的影响. 设  $b$  有扰动  $\delta b$ , 相应的解  $x$  的扰动记为  $\delta x$ , 即

$$A(x + \delta x) = b + \delta b$$

由  $Ax = b$ , 得  $A\delta x = \delta b$ , 从而有



$$\delta x = A^{-1} \delta b$$

两边取范数,得

$$\|\delta x\| = \|A^{-1} \delta b\| \leq \|A^{-1}\| \|\delta b\|$$

又因为

$$\|x\| \geq \frac{\|Ax\|}{\|A\|} = \frac{\|b\|}{\|A\|}$$

所以

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\|A^{-1}\| \|\delta b\|}{\frac{\|b\|}{\|A\|}} = \frac{\|A\| \|A^{-1}\| \|\delta b\|}{\|b\|} \quad (2-38)$$

式(2-38)表明,当右端项有扰动时,解的相对误差不超过右端项的相对误差的  $\|A\| \|A^{-1}\|$  倍.

如果右端项无扰动,系数矩阵  $A$  有扰动  $\delta A$ ,相应的解  $x$  的扰动仍记为  $\delta x$ ,则

$$(A + \delta A)(x + \delta x) = b$$

消去  $Ax = b$ ,得

$$A\delta x + \delta A(x + \delta x) = 0$$

因而有

$$\begin{aligned} \|\delta x\| &= \|A^{-1} \delta A(x + \delta x)\| \\ &\leq \|A^{-1}\| \|\delta A\| (\|x\| + \|\delta x\|) \end{aligned}$$

如果  $\delta A$  充分小,使得  $\|A^{-1}\| \|\delta A\| < 1$ ,则由上式得

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\|A^{-1}\| \|\delta A\|}{1 - \|A^{-1}\| \|\delta A\|} = \frac{\|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}}{1 - \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}} \quad (2-39)$$

式(2-39)表明,当系数矩阵有扰动时,解的扰动仍与  $\|A\| \|A^{-1}\|$  有关.  $\|A\| \|A^{-1}\|$  越大,解的扰动也越大.

一般地,若系数矩阵  $A$  有扰动  $\delta A$ ,右端项有扰动  $\delta b$ ,相应的解  $x$  有扰动  $\delta x$ ,且  $\|A^{-1}\| \|\delta A\| < 1$ ,则有

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\|A\| \|A^{-1}\|}{1 - \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}} \left( \frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right)$$

综合上述各式可以得出,当系数矩阵或右端项有扰动时,数  $\|A\| \|A^{-1}\|$  控制了解的扰动程度.也就是说,数  $\|A\| \|A^{-1}\|$  可以反映方程组的状态.

**定义 2.3** 对非奇异矩阵  $A$ ,称数  $\|A\| \|A^{-1}\|$  为矩阵  $A$  的条件数,记为

$$\text{cond}(A) = \|A\| \|A^{-1}\|$$

引入条件数后,式(2-38)和式(2-39)可改写为

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}$$

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\text{cond}(A) \frac{\|\delta A\|}{\|A\|}}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}}$$

因此,系数矩阵  $A$  的条件数刻画了方程组  $Ax = b$  的“病态”程度,条件数越大,“病态”越严重.

矩阵的条件数与所取范数有关,最常用的条件数有

$$\text{cond}(A)_{\infty} = \|A\|_{\infty} \|A^{-1}\|_{\infty} \quad (2-40)$$

$$\text{cond}(A)_2 = \|A\|_2 \|A^{-1}\|_2 = \sqrt{\frac{\lambda_1}{\lambda_n}} \quad (2-41)$$

其中  $\lambda_1, \lambda_n$  分别为矩阵的  $A^T A$  的最大特征值和最小特征值,故  $\text{cond}(A)_2$  又称为谱条件数. 特别地,如果  $A$  为实对称矩阵,  $\lambda_i (i=1, 2, \dots, n)$  为  $A$  的特征值,且  $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$ , 则

$$\text{cond}(A)_2 = \frac{|\lambda_1|}{|\lambda_n|}$$

例 8 中,方程组的系数矩阵为

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-5} \end{bmatrix}$$

其逆矩阵为

$$A^{-1} = \begin{bmatrix} 1 + 10^5 & -10^5 \\ -10^5 & 10^5 \end{bmatrix}$$

于是有

$$\text{cond}(A)_{\infty} = \|A\|_{\infty} \|A^{-1}\|_{\infty} = (2 + 10^{-5})(2 \times 10^5 + 1) > 4 \times 10^5$$

条件数很大,解的相对误差至少是右端项相对误差的  $4 \times 10^5$  倍,所以方程组是“病态”的.

由条件数的定义及矩阵范数的性质,易得

- (1) 对任意  $n$  阶方阵  $A$ , 都有  $\text{cond}(A) \geq 1$ .
- (2) 对任意非零实数  $\alpha$ , 有  $\text{cond}(\alpha A) = \text{cond}(A)$ .
- (3) 若  $A$  为正交矩阵, 则  $\text{cond}(A)_2 = 1$ .

## 例 9 已知方程组

$$\begin{cases} x_1 + \frac{1}{2}x_2 + \frac{1}{3}x_3 = 1 \\ \frac{1}{2}x_1 + \frac{1}{3}x_2 + \frac{1}{4}x_3 = 0 \\ \frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{5}x_3 = 0 \end{cases}$$

的解为  $x_1 = 9, x_2 = -36, x_3 = 30$ . 若把系数取成 2 位有效数字的小数, 得扰动方程组

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.50x_1 + 0.33x_2 + 0.25x_3 = 0 \\ 0.33x_1 + 0.25x_2 + 0.20x_3 = 0 \end{cases}$$

求解扰动方程组, 并分析所得结果.

[解] 用 Gauss 消去法求解, 第一次消元得

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.08x_2 + 0.085x_3 = -0.50 \\ 0.085x_2 + 0.0911x_3 = -0.33 \end{cases}$$

第二次消元得

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.08x_2 + 0.085x_3 = -0.50 \\ 7.875 \times 10^{-4}x_3 = 0.20125 \end{cases}$$

回代得解

$$x_3 = 255.55556, \quad x_2 = -277.77778, \quad x_1 = 55.55556$$

比较两个方程组可以看出, 它们的右端项相同, 系数矩阵中元的最大相对误差仅为 0.01, 而得到的解却严重失真. 这表明原方程组“病态”严重. 事实上, 原方程组的系数矩阵为

$$A = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}$$

其逆矩阵为

$$A^{-1} = \begin{bmatrix} 9 & -36 & 30 \\ -36 & 192 & -180 \\ 30 & -180 & 180 \end{bmatrix}$$

于是有

$$\text{cond}(A)_{\infty} = \|A\|_{\infty} \|A^{-1}\|_{\infty} = \frac{11}{6} \times 408 = 748$$

条件数很大,所以方程组“病态”严重.

原方程组的一般形式为

$$\begin{cases} x_1 + \frac{1}{2}x_2 + \frac{1}{3}x_3 + \cdots + \frac{1}{n}x_n = b_1 \\ \frac{1}{2}x_1 + \frac{1}{3}x_2 + \frac{1}{4}x_3 + \cdots + \frac{1}{n+1}x_n = b_2 \\ \dots\dots\dots \\ \frac{1}{n}x_1 + \frac{1}{n+1}x_2 + \frac{1}{n+2}x_3 + \cdots + \frac{1}{2n-1}x_n = b_n \end{cases}$$

是典型的“病态”方程组,称为 Hilbert 方程组, $n$  越大,“病态”越严重. 如  $n=6$  时,  $\text{cond}(A)_{\infty} = 29 \times 10^6$ ;  $n=10$  时,  $\text{cond}(A)_{\infty} = 35 \times 10^{12}$ . 严重“病态”的方程组,即使用主元素法求解也是数值不稳定的.

由于条件数需要求矩阵的逆阵,因而计算有一定的困难. 但在下列情况下,方程组往往是“病态”的:

- (1) 在用主元素法时出现小主元;
- (2) 系数矩阵中有行(或列)近似线性相关;
- (3) 系数矩阵的元素的值数量级相差很大.

对于病态方程组,通常采用双精度计算,可适当减轻病态矩阵的影响. 还可对方程组进行平衡预处理,即选择非奇异对角阵  $D, C$ ,使得求解线性方程组  $Ax = b$  的问题转化为求解等价线性方程组  $DAC(C^{-1}x) = Db$ ,而矩阵  $DAC$  的条件数得以改善. 例如,取  $C = I$  为单位阵,  $D$  的对角元素  $d_i$  满足

$$\frac{1}{d_i} = \max_{1 \leq j \leq n} |a_{ij}|, \quad i = 1, 2, \dots, n$$

令  $\tilde{A} = DA$ ,  $\tilde{b} = Db$ , 线性方程组  $\tilde{A}x = \tilde{b}$  的条件数往往会得到改善,解的精度能提高.

例 10 设方程  $Ax = b$  为

$$\begin{bmatrix} 1 & 10^6 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 10^6 \\ 2 \end{bmatrix}$$

计算条件数  $\text{cond}(A)_{\infty}$ , 并进行平衡预处理,使等价方程  $\tilde{A}x = \tilde{b}$  的条件数  $\text{cond}(\tilde{A})_{\infty}$  得到改善.

[解] 由于

$$A = \begin{bmatrix} 1 & 10^6 \\ 1 & 1 \end{bmatrix}, \quad A^{-1} = \frac{1}{10^6 - 1} \begin{bmatrix} -1 & 10^6 \\ 1 & -1 \end{bmatrix}$$

于是

$$\text{cond}(A)_\infty = \frac{(1 + 10^6)^2}{10^6 - 1} \approx 10^6$$

方程组  $Ax = b$  是“病态”的.

取  $D = \begin{bmatrix} 10^{-6} & 0 \\ 0 & 1 \end{bmatrix}$ , 原方程组的等价方程组  $DAx = Db$  的系数矩阵  $\tilde{A}$  及  $\tilde{A}^{-1}$  为

$$\tilde{A} = \begin{bmatrix} 10^{-6} & 1 \\ 1 & 1 \end{bmatrix}, \quad \tilde{A}^{-1} = \frac{1}{1 - 10^{-6}} \begin{bmatrix} -1 & 1 \\ 1 & -10^{-6} \end{bmatrix}$$

则

$$\text{cond}(\tilde{A})_\infty = \frac{2^2}{1 - 10^{-6}} \approx 4$$

条件数得到极大改善, 等价方程组  $\tilde{A}x = \tilde{b}$  是“良态”的. 对  $\tilde{A}x = \tilde{b}$  用列主元素法得近似解

$$x_1 = 1, \quad x_2 = 1$$

有些病态方程组, 选用合适的迭代法求解, 也能得到高精度的解. 参阅[3], [4], [18].

### 2.4.3 误差分析

在求得方程组  $Ax = b$  的一个近似解  $x^*$  后, 通常用残量(余量)  $r = b - Ax^*$  或其模  $\|r\|$  的大小来检验近似解  $x^*$  精确程度.

**定理 2.6** 设  $x$  和  $x^*$  分别是方程组  $Ax = b$  的准确解和近似解,  $r$  为  $x^*$  的残量, 则

$$\frac{\|x - x^*\|}{\|x\|} \leq \text{cond}(A) \frac{\|r\|}{\|b\|} \quad (2-42)$$

[证明] 因为

$$\begin{aligned} \|b\| &= \|Ax\| \leq \|A\| \|x\| \\ \|x - x^*\| &= \|A^{-1}(b - Ax^*)\| = \|A^{-1}r\| \leq \|A^{-1}\| \|r\| \end{aligned}$$

所以

$$\frac{\|x - x^*\|}{\|x\|} \leq \frac{\|A^{-1}\| \|r\|}{\frac{\|b\|}{\|A\|}} = \|A\| \|A^{-1}\| \frac{\|r\|}{\|b\|} = \text{cond}(A) \frac{\|r\|}{\|b\|}$$

由于残量  $r$  的计算很容易, 当方程组是“良态”时,  $r$  是检验近似解  $x^*$  精确

程度的一个简单实用的指标. 但对于“病态”严重的方程组, 即使  $\|r\|$  很小, 仍可能近似解  $x^*$  与精确解  $x$  有很大的差别. 如例 8 中, 方程组

$$\begin{cases} x_1 + x_2 = 2 \\ x_1 + 1.000\ 01x_2 = 2 \end{cases}$$

的准确解为  $x_1 = 2, x_2 = 0$ . 若以  $x_1 = 1, x_2 = 1$  作为它的近似解, 其残量

$$r = (2, 2)^T - (2, 2.000\ 01)^T = (0, -10^{-5})^T$$

很小, 但解的误差

$$x - x^* = (2, 0)^T - (1, 1)^T = (1, -1)^T$$

却不小.

由此可见, 利用残量  $r$  的大小来检验近似解  $x^*$  的精确程度仅适用于“良态”方程组, 对于“病态”方程组是不可靠的.

矩阵的模和条件数的计算有一定的困难, MATLAB 提供了函数 `norm()` 计算矩阵和向量的模、提供函数 `cond()` 计算矩阵的条件数. 例如, 在输入向量  $x$  和矩阵  $A$  后, 键入命令

```
>> norm1 = norm(x, 1)
>> norm2 = norm(A, 2)
>> cond2 = cond(A, 2)
>> condinf = cond(A, inf)
```

分别得到向量  $x$  的 1-范数、矩阵  $A$  的 2-范数及 2-范数、 $\infty$ -范数意义下的条件数. `norm(A)` 和 `cond(A)` 默认为是  $\|A\|_2$  和  $\text{cond}(A)_2$ .

## § 2.5 超定线性方程组的最小二乘解

如果方程组 (2-1) 中方程个数多于变量的个数, 称之为超定线性方程组. 这种方程组一般无精确解, 只能在各种规定的意义下求它的广义解.

设有超定线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \dots\dots\dots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases} \quad (2-43)$$

其矩阵形式仍记为  $Ax = b$ , 其中  $A$  为  $m \times n$  阶矩阵,  $x \in \mathbf{R}^n, b \in \mathbf{R}^m (m > n)$ .

考虑方程组 (2-43) 在下述意义下求解: 求  $x$ , 使得由它产生的每个方程的偏差  $\left(b_i - \sum_{j=1}^n a_{ij}x_j\right)$  的平方和最小, 即求问题

$$\min F(\mathbf{x}) = \sum_{i=1}^m \left( b_i - \sum_{j=1}^n a_{ij}x_j \right)^2 = \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2^2 \quad (2-44)$$

的解. 问题(2-44)称为线性最小二乘问题, 其解称为问题(2-43)的最小二乘解.

由多元微分学, 问题(2-44)的解必定是多元函数  $F(\mathbf{x})$  的驻点. 因为

$$\begin{aligned} \frac{\partial F}{\partial x_k} &= \sum_{i=1}^m 2 \left( b_i - \sum_{j=1}^n a_{ij}x_j \right) (-a_{ik}) \\ &= -2 \sum_{i=1}^m a_{ik} \left( b_i - \sum_{j=1}^n a_{ij}x_j \right) \quad (k = 1, 2, \dots, n) \end{aligned}$$

因此, 求解问题(2-44)可转化为求解方程组

$$\sum_{i=1}^m a_{ik} \left( b_i - \sum_{j=1}^n a_{ij}x_j \right) = 0 \quad (k = 1, 2, \dots, n)$$

即

$$\begin{cases} \sum_{i=1}^m a_{i1} \left( \sum_{j=1}^n a_{ij}x_j \right) = \sum_{i=1}^m a_{i1}b_i \\ \sum_{i=1}^m a_{i2} \left( \sum_{j=1}^n a_{ij}x_j \right) = \sum_{i=1}^m a_{i2}b_i \\ \dots\dots\dots \\ \sum_{i=1}^m a_{in} \left( \sum_{j=1}^n a_{ij}x_j \right) = \sum_{i=1}^m a_{in}b_i \end{cases} \quad (2-45)$$

为简便起见, 记  $\mathbf{a}_k = (a_{1k}, a_{2k}, \dots, a_{mk})^T$ , 则

$$\begin{aligned} \sum_{i=1}^m a_{ik} \left( \sum_{j=1}^n a_{ij}x_j \right) &= \sum_{j=1}^n \left( \sum_{i=1}^m a_{ik}a_{ij} \right) x_j = \sum_{j=1}^n \mathbf{a}_k^T \mathbf{a}_j x_j \\ \sum_{i=1}^m a_{ik}b_i &= \mathbf{a}_k^T \mathbf{b} \end{aligned} \quad (k = 1, 2, \dots, n)$$

于是方程组(2-45)可简记为

$$\begin{cases} \sum_{j=1}^n \mathbf{a}_1^T \mathbf{a}_j x_j = \mathbf{a}_1^T \mathbf{b} \\ \sum_{j=1}^n \mathbf{a}_2^T \mathbf{a}_j x_j = \mathbf{a}_2^T \mathbf{b} \\ \dots\dots\dots \\ \sum_{j=1}^n \mathbf{a}_n^T \mathbf{a}_j x_j = \mathbf{a}_n^T \mathbf{b} \end{cases}$$

用方程组(2-43)的系数矩阵表示, 则为

$$\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b} \quad (2-46)$$

这是一个  $n$  阶线性方程组,称为超定问题(2-43)的正则方程组或正规方程组. 如果  $A$  为列满秩,即  $\text{rank}(A) = n$ ,则  $A^T A$  是一个对称正定矩阵,因而正则方程组(2-46)存在唯一解,其解即为超定线性方程组(2-43)的最小二乘解.

**例 11** 求方程组

$$\begin{cases} x_1 - x_2 = 5 \\ -x_1 + 2x_2 = -4 \\ 2x_1 - 3x_2 = 10 \end{cases}$$

的最小二乘解.

[解] 因为系数矩阵为

$$A = \begin{bmatrix} 1 & -1 \\ -1 & 2 \\ 2 & -3 \end{bmatrix}$$

于是有

$$A^T A = \begin{bmatrix} 1 & -1 & 2 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \\ 2 & -3 \end{bmatrix} = \begin{bmatrix} 6 & -9 \\ -9 & 14 \end{bmatrix}$$

$$A^T b = \begin{bmatrix} 1 & -1 & 2 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 5 \\ -4 \\ 10 \end{bmatrix} = \begin{bmatrix} 29 \\ -43 \end{bmatrix}$$

正则方程组为

$$\begin{cases} 6x_1 - 9x_2 = 29 \\ -9x_1 + 14x_2 = -43 \end{cases}$$

其解为  $x_1 = \frac{19}{3}$ ,  $x_2 = 1$ . 这就是所求超定线性方程组的最小二乘解.

一般地,求解方程组(2-46)可用平方根法或  $LDL^T$  方法. 因此,求解超定线性方程组的最小二乘解的步骤为:

- (1) 计算  $M = A^T A$ ,  $f = A^T b$ ;
- (2) 对  $M$  作 Cholesky 分解  $M = LL^T$  (或  $LDL^T$  分解  $M = LDL^T$ );
- (3) 求解方程组  $Ly = f$ ,  $L^T x = y$  (或  $Lz = f$ ,  $y = D^{-1}z$ ,  $L^T x = y$ ).

这一方法简单易行,计算量约为  $\frac{1}{2}n^2 \left( m + \frac{1}{3}n \right)$ ,特别当  $m \gg n$  时,问题(2-46)的规模远小于问题(2-43). 虽然其数值稳定性较差,现在仍经常使用.

利用矩阵的 2-范数的正交不变性及矩阵  $M$  的正交三角分解(也称 QR 分解)提出的正交化方法,具有良好的数值稳定性,计算结果比正规化方法精确.



因此,这一方法应用范围较广. 但它的运算量比较大,约为  $n^2\left(m - \frac{n}{3}\right)$ , 详细内容可参阅[5].

多数超定线性方程组的最小二乘解可直接使用 MATLAB 中的库函数 `lsqr()` 求得. 如可简单地求得例 11 中的超定方程组的最小二乘解

```
>> A = [1 -1; -1 2; 2 -3];
>> b = [5; -4; 10];
>> x = lsqr(A, b)
x = 6.333 3
    1.000 0
```

## § 2.6 应用实例

### 2.6.1 平面桁架结构系

平面桁架是由刚性元件通过结点互相联结而组成的力学结构,它通常出现在桥梁结构和其他需要力学支撑的结构中. 图 2-1 是一个简单的静力桁架结构,其中刚性元件( $m=5$ )通过结点  $A, B, C, D$  相连.

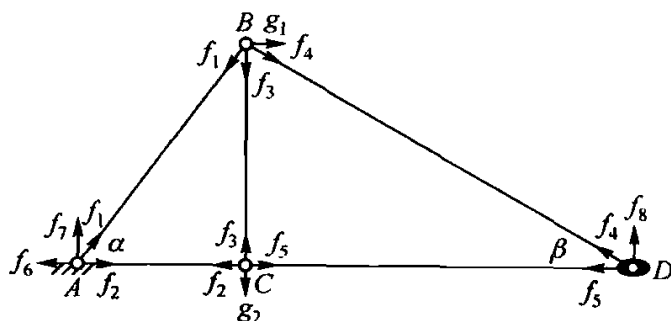


图 2-1

五个刚性元件的内力为  $\{f_1, f_2, \dots, f_5\}$ , 它们都处理为压力, 如果解是负的, 表示该力为张力. 桁架的左边由固定结点  $A$  支撑, 右边由滑轮  $D$  支撑,  $f_6, f_7, f_8$  是外部支撑力,  $g_1, g_2$  是外部负荷. 给定负荷时, 如何确定静力平衡时的五个内力及三个外部支撑力? 由于在静力平衡时, 每个结点处的水平方向合力与垂直方向的合力为零, 那么有

$$\text{结点 } A \begin{cases} f_1 \cos \alpha + f_2 - f_6 = 0 \\ f_1 \sin \alpha + f_7 = 0 \end{cases}$$

$$\text{结点 } B \begin{cases} -f_1 \cos \alpha + f_4 \cos \beta + g_1 = 0 \\ -f_1 \sin \alpha - f_3 - f_4 \sin \beta = 0 \end{cases}$$

$$\text{结点 } C \begin{cases} -f_2 + f_5 = 0 \\ f_3 - g_2 = 0 \end{cases}$$

$$\text{结点 } D \begin{cases} -f_4 \cos \beta - f_5 = 0 \\ f_4 \sin \beta + f_6 = 0 \end{cases}$$

设  $f$  表示未知力向量, 上述方程组可用矩阵表示为

$$\begin{bmatrix} \cos \alpha & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ \sin \alpha & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ -\cos \alpha & 0 & 0 & \cos \beta & 0 & 0 & 0 & 0 \\ -\sin \alpha & 0 & -1 & -\sin \beta & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\cos \beta & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sin \beta & 0 & 0 & 0 & 1 \end{bmatrix} f = \begin{bmatrix} 0 \\ 0 \\ -g_1 \\ 0 \\ 0 \\ g_2 \\ 0 \\ 0 \end{bmatrix}$$

若取  $\alpha = \frac{\pi}{3}, \beta = \frac{\pi}{6}$ , 外部负荷  $g_1 = 250 \text{ N}, g_2 = 1\,500 \text{ N}$ . 采用列主元素法或直接用矩阵左除运算  $A \setminus b$ , 得各节点的内力如下:

$$f = (-1\,174,483.3, 1\,500, -966.5, 483.3, 103.8, 1\,017,483.3)^T$$

### 2.6.2 化学吸收过程

化学吸收过程一般是由一系列的一种物质渗入另一种相互接触的物质而组成, 如蒸馏、浓缩和吸收过程. 液体的吸收过程可以用来从燃烧气体中除去二氧化硫  $\text{SO}_2$ . 图 2-2 显示了具有  $n=5$  步的吸收过程. 液体从顶端以流速  $L$  注入, 气体由底部以流速  $G$  导入. 设  $x_k$  和  $y_k$  分别表示在第  $k$  步被吸收于液体和气体中的某成分的量, 且有

$$y_k = ax_k + b \quad (1 \leq k \leq n)$$

设  $x_f$  表示液体中吸收的某成分的量,  $y_f$  表示气体中吸收的某成分的量,  $H$  表示液体的量. 经简化假设, 利用每一步中的分子平衡式, 得到吸收过程的下列方程组:

$$\tau = \frac{dx_1}{dt} = K(y_f - b) - (1 + \delta)x_1 + x_2$$

$$\tau = \frac{dx_2}{dt} = \delta x_1 - (1 + \delta)x_2 + x_3$$

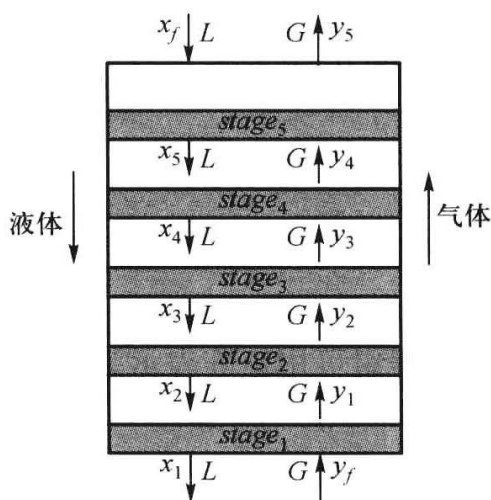


图 2-2

$$\tau = \frac{dx_3}{dt} = \delta x_2 - (1 + \delta)x_3 + x_4$$

$$\tau = \frac{dx_4}{dt} = \delta x_3 - (1 + \delta)x_4 + x_5$$

$$\tau = \frac{dx_5}{dt} = \delta x_4 - (1 + \delta)x_5 + x_f$$

其中  $\tau = H/L$  是在每一步液体的滞留时间,  $\delta = aG/L$  是除去因子,  $K = G/L$  是气体与液体之比. 为分析稳定态时的情况 (即常数解), 设  $\frac{dx_k}{dt} = 0 (1 \leq k \leq 5)$ ,  $x$  表示液体总量, 上述方程组可表示为线性方程组:

$$Ax = \begin{bmatrix} -(1+\delta) & 1 & 0 & 0 & 0 \\ \delta & -(1+\delta) & 1 & 0 & 0 \\ 0 & \delta & -(1+\delta) & 1 & 0 \\ 0 & 0 & \delta & -(1+\delta) & 1 \\ 0 & 0 & 0 & \delta & -(1+\delta) \end{bmatrix} x = \begin{bmatrix} K(b - y_f) \\ 0 \\ 0 \\ 0 \\ -x_f \end{bmatrix}$$

其中系数矩阵  $A$  是一个三对角矩阵. 不失一般性, 设吸收过程的参数为  $a = 0.72, b = 0, K = 1.63, x_f = 0.01, y_f = 0.06$ . 注意到系数矩阵中非零元素只有三个值, 用追赶法编写 MATLAB 程序如下:

```
>> n = 5; a = 0.72; b = 0.0; K = 1.63; xf = 0.01; yf = 0.06; delta = a * K;
>> A = zeros(n, n); c = zeros(n, 1);
>> c(1) = K * (b - yf); c(n) = -xf;
>> for i = 1:n - 1
A(i, i + 1) = 1; A(i + 1, i) = -delta / (1 + delta);
```

```

A(i+1,i+1) = -(1+delta) - A(i+1,i);
end;
>> for i=2:n c(i) = c(i) - A(i,i-1) * c(i-1); end;
>> x(n) = -c(n)/delta/(1+delta);
>> for k=n-1:-1:1 x(k) = (c(k) - x(k+1))/A(k,k); end;
>> y = a * x + b;

```

计算结果如下:

```

A =
    -2.173 6    1.000 0         0         0         0
     1.173 6   -2.173 6     1.000 0         0         0
         0     1.173 6   -2.173 6     1.000 0         0
         0         0     1.173 6   -2.173 6     1.000 0
         0         0         0     1.173 6   -2.173 6

```

分解后的矩阵  $L, U$  仍存入矩阵  $A$  的存储单元, 得

```

A =
    -2.173 6     1.         0         0         0
   -0.539 9   -1.633 7     1.         0         0
         0   -0.718 4   -1.455 2     1.         0
         0         0   -0.806 5   -1.367 1     1.
         0         0         0   -0.858 4   -1.315 2

```

各个阶段吸收于液体中的某成分的量:

```

x = 0.075 4  0.066 2  0.055 3  0.042 5  0.027 6

```

各个阶段吸收于气体中的某成分的量:

```

y = 0.054 3  0.047 6  0.039 8  0.030 6  0.019 9

```

## 评 注

Gauss 消元法是解线性方程组直接法的基础. 若系数矩阵  $A$  非奇异, 当  $|a_{kk}^{(k)}|$  很小或系数矩阵顺序主子式为零时, 采用列主元素法, 往往可得到较好的结果. 求解具有相同系数矩阵的一系列方程组时, 用直接三角分解法更为简便; 对于对称正定的系数矩阵, Cholesky 方法是有效的方法; 当系数矩阵是严格对角占优的三对角阵时, 追赶法是既快又数值稳定的常用方法, 若系数矩阵是分块三对角阵, 也有相应的追赶法.

直接法是目前求解中小规模线性方程组 ( $n < 1\,000$ ) 最常用的方法, 它的计

算量为  $O(n^3)$ . 直接法一般适用于系数矩阵稠密(即矩阵的绝大多数元素都不为零)而又不具有特殊结构的线性方程组. 如果系数矩阵具有某种特殊形式(如矩阵中非零元素占很小百分比的稀疏矩阵、正定矩阵、对称矩阵和反对称矩阵等), 无论在存储上还是在计算上都有专门的技术和方法, 详细可参阅[16 ~ 18].

## 习 题 二

1. 分别用 Gauss 消去法、列主元素法和全主元素法解下列方程组, 计算过程保留 3 位小数:

$$(1) \begin{cases} 2x_1 + 6x_2 - 4x_3 = 4 \\ x_1 + 4x_2 - 5x_3 = 3 \\ 6x_1 - x_2 + 18x_3 = 2 \end{cases}$$

$$(2) \begin{cases} 2x_1 + x_2 + 2x_3 = 6 \\ 4x_1 + 3x_2 + x_3 = 11 \\ 6x_1 + x_2 + 5x_3 = 13 \end{cases}$$

2. 用三角分解法求解题 1 中的方程组.

3. 用紧凑格式解下列方程组, 并写出  $L, U$  矩阵:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 4 & 9 & 16 \\ 1 & 8 & 27 & 64 \\ 1 & 16 & 81 & 256 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 10 \\ 44 \\ 190 \end{bmatrix}$$

4. 若

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & -l_{k+1,k} & & 1 & \\ & -l_{k+2,k} & & & \ddots \\ & \vdots & & & & \ddots \\ & -l_{nk} & & & & & 1 \end{bmatrix} \quad (k = 1, 2, \dots, n-1)$$

求证:(1)

$$L_k^{-1} = 2I - L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{k+1\ k} & 1 & \\ & & l_{k+2\ k} & & 1 \\ & & \vdots & & & \ddots \\ & & l_{n\ k} & & & & 1 \end{bmatrix}$$

(2)

$$L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = \begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{bmatrix}$$

5. 用三角分解法求下列矩阵的逆矩阵:

$$(1) \begin{bmatrix} 1 & 1 & -1 \\ 2 & 1 & 0 \\ 1 & -1 & 0 \end{bmatrix} \quad (2) \begin{bmatrix} 2 & 2 & 3 \\ 1 & -1 & 0 \\ -1 & 2 & 1 \end{bmatrix}$$

6. 分别用平方根法和改进平方根法求解方程组

$$\begin{bmatrix} 1 & 2 & 1 & -3 \\ 2 & 5 & 0 & -5 \\ 1 & 0 & 14 & 1 \\ -3 & -5 & 1 & 15 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 16 \\ 8 \end{bmatrix}$$

7. 设有方程组  $Ax = b$ , 其中

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & a \\ 0 & a & 2 \end{bmatrix}, \quad x = (x_1, x_2, x_3)^T, \quad b = (3, 3, 1)^T$$

(1) 求出  $A$  能进行 Cholesky 分解, 即  $A = LL^T$  (其中  $L$  为下三角矩阵) 的  $a$  的取值范围;

(2) 取  $a = 1$ , 对矩阵  $A$  进行 Cholesky 分解, 并用平方根法求解上述方程组, 计算过程保留 2 位小数.

8. 用追赶法解下列方程组:

$$(1) \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 2.5 \end{bmatrix}$$

$$(2) \begin{bmatrix} 136.01 & 90.860 & 0 & 0 \\ 90.860 & 98.810 & -67.590 & 0 \\ 0 & -67.590 & 132.01 & 46.260 \\ 0 & 0 & 46.260 & 177.17 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -33.254 \\ 49.709 \\ 28.067 \\ -7.3244 \end{bmatrix}$$

9. 证明矩阵的  $F$ -范数与向量的 2-范数相容.

10. 已知

$$A = \begin{bmatrix} 6 & 13 & -17 \\ 13 & 29 & -38 \\ -17 & -38 & 50 \end{bmatrix}$$

求  $\text{cond}(A)_1$  及  $\text{cond}(A)_\infty$ , 并说明方程组  $Ax = b$  是否病态.

11. 分别用 Gauss 消去法与列主元素法求解线性方程组.

$$\begin{cases} 1.003x_1 + 58.09x_2 = 68.12 \\ 321.8x_1 + 5.550x_2 = 377.3 \end{cases}$$

计算过程取 4 位有效数字. 比较计算结果, 哪个算法的精度高?

12. 已知方程组

$$\begin{cases} x_1 + 0.99x_2 = 1 \\ 0.99x_1 + 0.98x_2 = 1 \end{cases}$$

的解为  $x_1 = 100, x_2 = -100$

(1) 计算系数矩阵的条件数.

(2) 取  $x_1^* = (1, 0)^T, x_2^* = (100.5, -99.5)^T$ , 分别计算残量  $r_i = b - Ax_i^*$  ( $i=1, 2$ ). 本题的计算结果说明了什么?

13. 求超定方程组

$$\begin{cases} 2x_1 + 4x_2 = 11 \\ 3x_1 - 5x_2 = 3 \\ x_1 + 2x_2 = 6 \\ 2x_1 + x_2 = 7 \end{cases}$$

的最小二乘解.

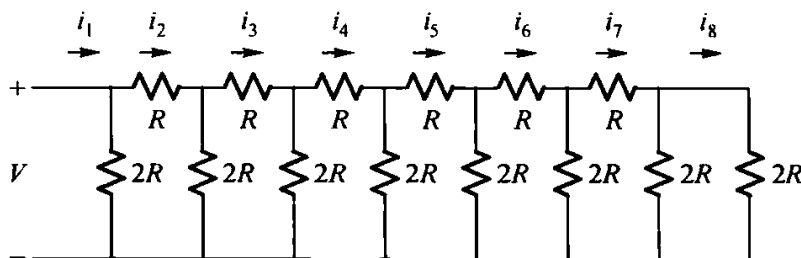
## 数值实验

1. 试用 MATLAB 软件编程实现矩阵的列主元素三角分解, 并求 Pascal 矩阵

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 3 & 6 & 10 & 15 \\ 1 & 4 & 10 & 20 & 35 \\ 1 & 5 & 15 & 35 & 70 \end{bmatrix}$$

的逆阵  $A^{-1}$ , 用左除命令  $A \setminus E$  检验你的结果.

2. 试用 MATLAB 软件编程实现追赶法求解三对角方程组的算法, 并考虑如下梯形电阻电路问题:



其中电路中的各个电流  $\{i_1, i_2, \dots, i_8\}$  满足下列线性方程组:

$$\begin{aligned} 2i_1 - 2i_2 &= V/R \\ -2i_1 + 5i_2 - 2i_3 &= 0 \\ -2i_2 + 5i_3 - 2i_4 &= 0 \\ -2i_3 + 5i_4 - 2i_5 &= 0 \\ -2i_4 + 5i_5 - 2i_6 &= 0 \\ -2i_5 + 5i_6 - 2i_7 &= 0 \\ -2i_6 + 5i_7 - 2i_8 &= 0 \\ -2i_7 + 5i_8 &= 0 \end{aligned}$$

设  $V = 220 \text{ V}$ ,  $R = 27 \Omega$ , 求各段电路的电流.

3. 方程组的性态和矩阵条件数的试验. 设有线性方程组  $Ax = b$ , 其中系数矩阵  $A = (a_{ij})_{n \times n}$  的元素分别为  $a_{ij} = x_i^{j-1}$  ( $x_i = 1 + 0.1i$ ,  $i, j = 1, 2, \dots, n$ ) 或  $a_{ij} = \frac{1}{i+j-1}$  ( $i, j = 1, 2, \dots, n$ ), 右端向量  $b = \left( \sum_{j=1}^n a_{1j}, \sum_{j=1}^n a_{2j}, \dots, \sum_{j=1}^n a_{nj} \right)^T$ . 利用 MATLAB 中的库函数,

- (1) 取  $n = 5, 10, 20$ , 分别求出系数矩阵的 2-条件数, 判别它们是否是病态



阵? 随着  $n$  的增大, 矩阵性态的变化如何?

(2) 分别取  $n = 5, 10, 20$ , 解上述两个线性方程组  $Ax = b$ , 并将求得的解与精确解  $x = (1, 1, \dots, 1)^T$  作比较, 说明了什么?

(3) 取  $n = 10$ , 对系数矩阵中的  $a_{22}$  和  $a_{nn}$  增加一个扰动  $10^{-8}$ , 求解方程组  $(A + \delta A)x = b$ , 解的变化如何?

4. 已知 Wilson 矩阵

$$A = \begin{bmatrix} 10 & 7 & 8 & 7 \\ 7 & 5 & 6 & 5 \\ 8 & 6 & 10 & 9 \\ 7 & 5 & 9 & 10 \end{bmatrix}.$$

右端项  $b = (32, 23, 33, 31)^T$ , 则方程组  $Ax = b$  有精确解  $x = (1, 1, \dots, 1)^T$ . 用 MATLAB 中的库函数,

(1) 求  $\det(A)$ ,  $\text{cond}(A)$ , 及  $A$  的所有特征值.

(2) 令

$$A + \delta A = \begin{bmatrix} 10 & 7.2 & 8.1 & 6.9 \\ 7.08 & 5.07 & 6.02 & 5 \\ 8.2 & 5.89 & 9.96 & 9.01 \\ 6.98 & 5.04 & 8.97 & 9.98 \end{bmatrix}$$

解扰动方程组  $(A + \delta A)(x + \delta x) = b$ , 并求出向量  $\delta x$  和  $\|\delta x\|_2$ , 从理论结果式

(2-39) 和实际计算结果两方面分析方程组  $Ax = b$  的解的相对误差  $\frac{\|\delta x\|_2}{\|x\|_2}$  与

$A$  的相对误差  $\frac{\|\delta A\|_2}{\|A\|_2}$  的关系.

(3) 利用函数  $\text{rand}(n)$ , 重新产生扰动矩阵  $\delta A$ , 使其各元素的绝对值不超过  $\frac{1}{2} \times 10^{-4}$ , 重复(2).

## 第三章 解线性方程组的迭代法

直接方法比较适用于中小型方程组. 对高阶方程组, 即使系数矩阵是稀疏的, 在运算中也很难保持其稀疏性, 因而有存储量大, 程序复杂等不足. 迭代法则能保持矩阵的稀疏性, 具有计算简单, 编制程序容易的优点, 并在许多情况下收敛较快. 故能有效地解一些高阶方程组.

### § 3.1 迭代法概述

迭代法的基本思想是构造一串收敛到解的序列, 即建立一种从已有近似解计算新的近似解的规则, 由不同的计算规则得到不同的迭代法. 本章着重介绍单步定常线性迭代法.

#### 3.1.1 迭代法的一般形式

对线性方程组

$$Ax = b \quad (3-1)$$

其中  $A = (a_{ij})$  为  $n \times n$  阶非奇异矩阵,  $b = (b_1, \dots, b_n)^T$ . 构造其形如

$$x = Mx + g \quad (3-2)$$

的同解方程组, 其中  $M$  为  $n$  阶方阵,  $g \in \mathbf{R}^n$ . 任取初始向量  $x^{(0)} \in \mathbf{R}^n$ , 代入迭代公式

$$x^{(k+1)} = Mx^{(k)} + g \quad (k = 0, 1, 2, \dots) \quad (3-3)$$

产生向量序列  $\{x^{(k)}\}$ , 当  $k$  充分大时, 以  $x^{(k)}$  作为方程组 (3-1) 的近似解, 这就是求解线性方程组的单步定常线性迭代法. 式 (3-3) 中的  $M$  称为迭代矩阵.

为研究迭代法的收敛性, 必须先介绍向量序列与矩阵序列收敛的概念.

#### 3.1.2 向量序列与矩阵序列的收敛性

由于  $\mathbf{R}^n$  中的向量可与  $\mathbf{R}^n$  中的点建立一一对应关系, 由点列的收敛概念及向量范数的等价性, 可得到向量序列的收敛概念.

**定义 3.1** 设  $\{x^{(k)}\}$  为  $\mathbf{R}^n$  中的向量序列,  $x \in \mathbf{R}^n$ , 如果

$$\lim_{k \rightarrow \infty} \|x^{(k)} - x\| = 0 \quad (3-4)$$

其中  $\|\cdot\|$  为向量范数, 则称序列  $\{\mathbf{x}^{(k)}\}$  收敛于  $\mathbf{x}$ , 记为

$$\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x} \quad (3-5)$$

$n$  维点列收敛的一种等价描述是其对应坐标序列均收敛, 向量序列也有类似的结果.

**定理 3.1**  $\mathbf{R}^n$  中的向量序列  $\{\mathbf{x}^{(k)}\}$  收敛于  $\mathbf{R}^n$  中的向量  $\mathbf{x}$  当且仅当

$$\lim_{k \rightarrow \infty} x_i^{(k)} = x_i \quad (i = 1, 2, \dots, n)$$

其中  $\mathbf{x}^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$ ,  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ .

[证明] 由定义 3.1,  $\{\mathbf{x}^{(k)}\}$  收敛于  $\mathbf{x}$ , 即

$$\lim_{k \rightarrow \infty} \|\mathbf{x}^{(k)} - \mathbf{x}\|_{\infty} = 0$$

而对任意  $1 \leq i \leq n$ , 有

$$0 \leq |x_i^{(k)} - x_i| \leq \max_{1 \leq j \leq n} |x_j^{(k)} - x_j| = \|\mathbf{x}^{(k)} - \mathbf{x}\|_{\infty}$$

由极限存在准则得

$$\lim_{k \rightarrow \infty} |x_i^{(k)} - x_i| = 0 \quad (i = 1, 2, \dots, n)$$

即

$$\lim_{k \rightarrow \infty} x_i^{(k)} = x_i \quad (i = 1, 2, \dots, n)$$

完全类似地, 可以定义矩阵序列的收敛.

**定义 3.2** 设  $\{A^{(k)}\}$  为  $n$  阶方阵序列,  $A$  为  $n$  阶方阵. 如果

$$\lim_{k \rightarrow \infty} \|A^{(k)} - A\| = 0 \quad (3-6)$$

其中  $\|\cdot\|$  为矩阵范数, 则称序列  $\{A^{(k)}\}$  收敛于矩阵  $A$ , 记为

$$\lim_{k \rightarrow \infty} A^{(k)} = A \quad (3-7)$$

**定理 3.2** 设  $A^{(k)} = (a_{ij}^{(k)})$  ( $k = 1, 2, \dots$ ),  $A = (a_{ij})$  均为  $n$  阶方阵, 则矩阵序列  $\{A^{(k)}\}$  收敛于矩阵  $A$  的充要条件为

$$\lim_{k \rightarrow \infty} a_{ij}^{(k)} = a_{ij} \quad (i, j = 1, 2, \dots, n)$$

证明留给读者.

定理 3.1 和 3.2 表明, 向量序列和矩阵序列的收敛可以归结为对应分量或对应元素序列的收敛.

由以上讨论容易看出, 若按式 (3-3) 产生的向量序列  $\{\mathbf{x}^{(k)}\}$  收敛于向量  $\mathbf{x}$ , 则有

$$\mathbf{x} = \lim_{k \rightarrow \infty} \mathbf{x}^{(k+1)} = \lim_{k \rightarrow \infty} [M\mathbf{x}^{(k)} + \mathbf{g}] = M\mathbf{x} + \mathbf{g}$$

即  $\mathbf{x}$  是方程组 (3-1) 的解.

## § 3.2 几种基本的迭代法

### 3.2.1 雅可比 (Jacobi) 迭代法

设方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \cdots \cdots \cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases} \quad (3-8a)$$

的系数矩阵  $A$  非奇异, 且  $a_{ii} \neq 0 (i=1, 2, \cdots, n)$ . 将方程组 (3-8a) 变形为

$$\begin{cases} x_1 = b_{12}x_2 + b_{13}x_3 + \cdots + b_{1n}x_n + g_1 \\ x_2 = b_{21}x_1 + b_{23}x_3 + \cdots + b_{2n}x_n + g_2 \\ \cdots \cdots \cdots \\ x_n = b_{n1}x_1 + b_{n2}x_2 + \cdots + b_{nn-1}x_{n-1} + g_n \end{cases} \quad (3-8b)$$

其中  $b_{ij} = -\frac{a_{ij}}{a_{ii}} (i \neq j, i, j=1, 2, \cdots, n)$ ,  $g_i = \frac{b_i}{a_{ii}} (i=1, 2, \cdots, n)$ . 记对角矩阵

$$D = \text{diag}(a_{11}, a_{22}, \cdots, a_{nn}), \quad D^{-1} = \text{diag}\left(\frac{1}{a_{11}}, \frac{1}{a_{22}}, \cdots, \frac{1}{a_{nn}}\right)$$

则

$$B = \begin{bmatrix} 0 & b_{12} & \cdots & b_{1n-1} & b_{1n} \\ b_{21} & 0 & \cdots & b_{2n-1} & b_{2n} \\ \vdots & \vdots & & \vdots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn-1} & 0 \end{bmatrix} = I - D^{-1}A, \quad g = \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_n \end{bmatrix} = D^{-1}b \quad (3-9)$$

方程组 (3-8b) 可简记为

$$x = Bx + g \quad (3-10)$$

将初始向量  $x^{(0)}$  代入迭代公式

$$x^{(k+1)} = Bx^{(k)} + g \quad (k=0, 1, \cdots) \quad (3-11)$$

产生向量序列  $\{x^{(k)}\}$ , 公式 (3-11) 所给出的迭代法称为 Jacobi 迭代法, 又称简单迭代法. 称式 (3-11) 为 Jacobi 迭代法的迭代公式,  $B$  为其迭代矩阵.

Jacobi 迭代法的计算过程如下:

#### 算法 3.1

1. 输入  $A = (a_{ij})$ ,  $b = (b_1, \cdots, b_n)$ , 维数  $n$ ,  $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \cdots, x_n^{(0)})$ ,  $\varepsilon$ , 最

大容许迭代次数  $N$ .

2. 置  $k = 1$

3. 对  $i = 1, 2, \dots, n$

$$x_i = \left( b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(0)} \right) / a_{ii}$$

4. 若  $\|x - x^{(0)}\| < \varepsilon$ , 输出  $x$ , 停机; 否则转 5.

5. 若  $k < N$ , 置  $k + 1 \Rightarrow k, x_i \Rightarrow x_i^{(0)} (i = 1, 2, \dots, n)$ , 转 3; 否则, 输出失败信息, 停机.

**例 1** 用 Jacobi 迭代法求解线性方程组

$$\begin{cases} 10x_1 - x_2 - 2x_3 = 72 \\ -x_1 + 10x_2 - 2x_3 = 83 \\ -x_1 - x_2 + 5x_3 = 42 \end{cases}$$

[解] 由 Jacobi 迭代法的计算公式(3-11), 有

$$\begin{cases} x_1^{(k+1)} = 0.1x_2^{(k)} + 0.2x_3^{(k)} + 7.2 \\ x_2^{(k+1)} = 0.1x_1^{(k)} + 0.2x_3^{(k)} + 8.3 \\ x_3^{(k+1)} = 0.2x_1^{(k)} + 0.2x_2^{(k)} + 8.4 \end{cases}$$

取  $x^{(0)} = (0, 0, 0)^T$ , 代入上式得

$$x_1^{(1)} = 7.2, \quad x_2^{(1)} = 8.3, \quad x_3^{(1)} = 8.4$$

$$x_1^{(2)} = 0.1 \times 8.3 + 0.2 \times 8.4 + 7.2 = 9.71$$

$$x_2^{(2)} = 0.1 \times 7.2 + 0.2 \times 8.4 + 8.3 = 10.70$$

$$x_3^{(2)} = 0.2 \times 7.2 + 0.2 \times 8.3 + 8.4 = 11.50$$

如此继续下去, 迭代结果见表 3-1.

表 3-1

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$ | $x_3^{(k)}$ |
|-----|-------------|-------------|-------------|
| 0   | 0.000 0     | 0.000 0     | 0.000 0     |
| 1   | 7.200 0     | 8.300 0     | 8.400 0     |
| 2   | 9.710 0     | 10.700 0    | 11.500 0    |
| 3   | 10.570 0    | 11.570 0    | 12.482 0    |
| 4   | 10.853 5    | 11.853 4    | 12.828 2    |

续表

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$ | $x_3^{(k)}$ |
|-----|-------------|-------------|-------------|
| 5   | 10.951 0    | 11.951 0    | 12.941 4    |
| 6   | 10.983 4    | 11.983 4    | 12.950 4    |
| 7   | 10.994 4    | 11.998 1    | 12.993 4    |
| 8   | 10.998 1    | 11.994 1    | 12.997 8    |
| 9   | 10.999 4    | 11.999 4    | 12.999 2    |

迭代 9 次, 得近似解  $\mathbf{x}^{(9)} = (10.999\ 4, 11.999\ 4, 12.999\ 2)^T$ , 容易验证, 方程组的精确解为  $\mathbf{x} = (11, 12, 13)^T$ , 从表 3.1 可以看出, 当迭代次数增加时, 迭代结果越来越接近精确解.

### 3.2.2 高斯 - 赛德尔 (Gauss-Seidel) 迭代法

在 Jacobi 迭代法中, 计算  $x_i^{(k+1)}$  时, 分量  $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{i-1}^{(k+1)}$  已经算出, 可以考虑用新的分量  $x_l^{(k+1)} (l = 1, 2, \dots, i-1)$  代替旧的分量  $x_l^{(k)} (l = 1, 2, \dots, i-1)$ , 这就是 Gauss-Seidel 迭代法. Gauss-Seidel 迭代法的计算公式为

$$\begin{cases} x_1^{(k+1)} = b_{12}x_2 + b_{13}x_3 + \dots + b_{1n}x_n + g_1 \\ x_2^{(k+1)} = b_{21}x_1^{(k+1)} + b_{23}x_3 + \dots + b_{2n}x_n + g_2 \\ \dots\dots\dots \\ x_n^{(k+1)} = b_{n1}x_1^{(k+1)} + b_{n2}x_2^{(k+1)} + \dots + b_{nn}x_n^{(k+1)} + g_n \end{cases} \quad (3-12)$$

而且近似向量  $\mathbf{x}^{(k)}$  和  $\mathbf{x}^{(k+1)}$  可以共用一组存储空间.

若将系数矩阵  $\mathbf{A}$  表示为

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}$$

其中

$$\mathbf{D} = \text{diag}(a_{11}, a_{22}, \dots, a_{nn})$$

$$\mathbf{L} = \begin{bmatrix} 0 & & & & \\ -a_{21} & 0 & & & \\ -a_{31} & -a_{32} & \ddots & & \\ \vdots & \vdots & & \ddots & \\ -a_{n1} & -a_{n2} & \dots & -a_{nn-1} & 0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 0 & -a_{12} & -a_{13} & \dots & -a_{1n} \\ & 0 & -a_{23} & \dots & -a_{2n} \\ & & \ddots & & \vdots \\ & & & & -a_{n-1\ n} \\ & & & & 0 \end{bmatrix}$$

并由式(3-9), 则迭代公式(3-12)可以用矩阵表示为

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\mathbf{L}\mathbf{x}^{(k+1)} + \mathbf{D}^{-1}\mathbf{U}\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b} \quad (3-13)$$

移项得

$$(I - D^{-1}L)x^{(k+1)} = D^{-1}Ux^{(k)} + D^{-1}b$$

因为  $|I - D^{-1}L| = 1$ , 所以矩阵  $I - D^{-1}L$  可逆, 且  $(I - D^{-1}L)^{-1} = [D^{-1}(D - L)]^{-1} = (D - L)^{-1}D$ , 故

$$\begin{aligned} x^{(k+1)} &= (I - D^{-1}L)^{-1}[D^{-1}Ux^{(k)} + D^{-1}b] \\ &= (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b \end{aligned} \quad (3-14)$$

式(3-14)就是 Gauss-Seidel 迭代法的矩阵表示, 矩阵  $M = (D - L)^{-1}U$  为 Gauss-Seidel 迭代法的迭代矩阵.

### 算法 3.2

1. 输入  $A = (a_{ij})$ ,  $b = (b_1, \dots, b_n)$ ,  $x^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)})$  维数  $n$ ,  $\varepsilon$ , 最大容许迭代次数  $N$ .

2. 置  $k = 1$ .

3. 计算:

$$\begin{aligned} x_1 &= \left( b_1 - \sum_{j=2}^n a_{1j}x_j^{(0)} \right) / a_{11} \\ x_i &= \left( b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j^{(0)} \right) / a_{ii} \quad (i = 2, 3, \dots, n-1) \\ x_n &= \left( b_n - \sum_{j=1}^{n-1} a_{nj}x_j \right) / a_{nn} \end{aligned}$$

4. 若  $\|x - x^{(0)}\| < \varepsilon$ , 输出  $x = (x_1, \dots, x_n)$ , 停机; 否则转 5.

5. 若  $k < N$ , 置  $k+1 \Rightarrow k$ ,  $x_i \Rightarrow x_i^{(0)} (i = 1, 2, \dots, n)$ , 转 3; 否则输出失败信息, 停机.

用 Gauss-Seidel 迭代法解例 1. 仍取  $x^{(0)} = (0, 0, 0)^T$ , 按式(3-13)计算得

$$x_1^{(1)} = \frac{1}{10}(x_2^{(0)} + 2x_3^{(0)} + b_1) = \frac{1}{10} \times 72 = 7.2000$$

$$\begin{aligned} x_2^{(1)} &= \frac{1}{10}(x_1^{(1)} + 2x_3^{(0)} + b_2) \\ &= \frac{1}{10}(7.2000 + 83) = 9.0200 \end{aligned}$$

$$\begin{aligned} x_3^{(1)} &= \frac{1}{5}(x_1^{(1)} + x_2^{(1)} + b_3) \\ &= \frac{1}{5}(7.2000 + 9.0200 + 42) = 11.6440 \end{aligned}$$

如此继续下去, 计算结果见表 3-2.

表 3-2

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$ | $x_3^{(k)}$ |
|-----|-------------|-------------|-------------|
| 0   | 0.000 0     | 0.000 0     | 0.000 0     |
| 1   | 7.200 0     | 9.020 0     | 11.644 0    |
| 2   | 10.430 8    | 11.671 9    | 12.820 5    |
| 3   | 10.931 3    | 11.957 2    | 12.977 8    |
| 4   | 10.991 3    | 11.994 7    | 12.997 2    |
| 5   | 10.998 9    | 11.999 3    | 12.999 6    |
| 6   | 10.999 9    | 11.999 9    | 13.000 0    |

计算结果表明,用 Gauss-Seidel 迭代法求解例 1 中的方程组比 Jacobi 迭代法效果好,迭代 5 次所得到的结果与例 1 中迭代 9 次所得的结果相仿.事实上,对有些问题 Gauss-Seidel 迭代法确实比 Jacobi 迭代法收敛得快,但也有 Gauss-Seidel 迭代比 Jacobi 迭代收敛得慢,甚至还有 Jacobi 迭代收敛, Gauss-Seidel 迭代发散的情形.

### 3.2.3 松弛 (Successive Over Relaxation) 迭代法

为了加速迭代向量序列  $\{x^{(k)}\}$  的收敛速度,我们在 Gauss-Seidel 迭代法的基础上,引入参数,可以得到一种新的迭代法.

由式(3-13), Gauss-Seidel 迭代法可以改写为

$$\begin{aligned} x^{(k+1)} &= D^{-1}(Lx^{(k+1)} + Ux^{(k)} + b) \\ &= x^{(k)} + D^{-1}(Lx^{(k+1)} + Ux^{(k)} + b - Dx^{(k)}) \end{aligned}$$

将上式等号右端的第二项看作修正项,在其前面乘以参数因子  $\omega$ ,即得到松弛法(简称 SOR 法)

$$\begin{aligned} x^{(k+1)} &= x^{(k)} + \omega D^{-1}(Lx^{(k+1)} + Ux^{(k)} + b - Dx^{(k)}) \\ &= (1 - \omega)x^{(k)} + \omega D^{-1}Lx^{(k+1)} + \omega D^{-1}Ux^{(k)} + \omega D^{-1}b \quad (3-15) \end{aligned}$$

因为  $|I - \omega D^{-1}L| = 1$ , 所以矩阵  $(I - \omega D^{-1}L)$  可逆,且

$$(I - \omega D^{-1}L)^{-1} = [D^{-1}(D - \omega L)]^{-1} = (D - \omega L)^{-1}D$$

代入式(3-15)可得

$$x^{(k+1)} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]x^{(k)} + (D - \omega L)^{-1}\omega b \quad (3-16)$$

松弛法的迭代矩阵

$$M = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]$$



参数因子  $\omega$  称为松弛因子. 当  $\omega < 1$  时, 方法称为低松弛法; 当  $\omega = 1$  时, 方法即为 Gauss-Seidel 迭代法; 当  $\omega > 1$  时, 方法称为超松弛法.

由式(3-15)知松弛法的计算公式为

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left( b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) \quad (i = 1, 2, \dots, n) \quad (3-17)$$

具体算法如下:

### 算法 3.3

1. 输入  $A = (a_{ij})$ ,  $b = (b_1, \dots, b_n)$ ,  $x^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)})$ , 参数  $\omega$ , 误差限  $\varepsilon$ , 最大容许迭代次数  $N$ .

2. 置  $k = 1$ .

3. 计算:

$$x_1 = (1 - \omega)x_1^{(0)} + \omega \left( b_1 - \sum_{j=2}^n a_{1j}x_j^{(0)} \right) / a_{11}$$

$$x_i = (1 - \omega)x_i^{(0)} + \omega \left( b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j^{(0)} \right) / a_{ii} \quad (i = 2, \dots, n-1)$$

$$x_n = (1 - \omega)x_n^{(0)} + \omega \left( b_n - \sum_{j=1}^{n-1} a_{nj}x_j \right) / a_{nn}$$

4. 若  $\|x - x^{(0)}\| < \varepsilon$ , 输出  $x = (x_1, \dots, x_n)$ , 停机, 否则转 5.

5. 若  $k < N$ , 置  $k+1 \Rightarrow k$ ,  $x_i \Rightarrow x_i^{(0)} (i = 1, 2, \dots, n)$ , 转 3; 否则输出失败信息, 停机.

利用 MATLAB 中矩阵计算的方便性, 可编写如下函数实现松弛法迭代计算:

```
function [x, k] = sor(A, b, omega, tol)    % 将函数存为 sor.m 文件
    D = diag(diag(A));                  % 取出的对角元, 构造对角阵 D
    L = D - tril(A);                     % L 为 A 的单位下三角形矩阵
    U = D - triu(A);                     % U 为 A 的单位上三角形矩阵
    x = zeros(size(b));                  % x^(0) = (0, 0, ..., 0)^T
    xk = zeros(100, size(b));
    for k = 1:100
        x = (D - omega * L) \ (omega * b + (1 - omega) * D * x + omega * U * x);
                                                % 迭代求 x^(k)
        error = norm(b - A * x) / norm(b); % 2-范数下的残差相对误差
        if (error < tol) break;
    end
end
```

**例 2** 取  $\mathbf{x}^{(0)} = (0, 0, 0)^T$ , 分别用 Gauss-Seidel 迭代法和松弛法 ( $\omega = 1.28$ ) 求解方程组

$$\begin{cases} 2x_1 - x_2 &= 1 \\ x_1 + 2x_2 - x_3 &= 0 \\ -x_2 + 2x_3 &= 1.8 \end{cases}$$

[解] 在 MATLAB 中, 输入  $A$  和  $b$ , 取 2-范数下的残差相对误差小于  $10^{-4}$  作为迭代停止标准. 令  $\omega = 1$  和  $1.28$ , 调用上述函数

```
>> sor(A, b, 1, 0.0001);
>> sor(A, b, 1.28, 0.0001);
```

得计算结果见表 3-3:

表 3-3

| $k$ | Gauss-Seidel 迭代法 $\mathbf{x}^{(k)}$ |         |         | $\omega = 1.28$ 松弛法 $\mathbf{x}^{(k)}$ |         |         |
|-----|-------------------------------------|---------|---------|----------------------------------------|---------|---------|
| 1   | 0.500 0                             | 0.250 0 | 1.025 0 | 0.640 0                                | 0.409 6 | 1.414 1 |
| 2   | 0.625 0                             | 0.825 0 | 1.312 5 | 0.722 9                                | 1.253 0 | 1.558 0 |
| 3   | 0.912 5                             | 1.112 5 | 1.456 3 | 1.239 5                                | 1.439 6 | 1.637 1 |
| 4   | 1.056 2                             | 1.256 3 | 1.528 1 | 1.214 2                                | 1.421 8 | 1.603 6 |
| 5   | 1.128 1                             | 1.328 1 | 1.564 1 | 1.209 9                                | 1.402 5 | 1.600 6 |
| 6   | 1.182 0                             | 1.382 0 | 1.591 0 | 1.198 8                                | 1.399 0 | 1.599 2 |
| 7   | 1.164 1                             | 1.364 1 | 1.582 0 | 1.199 7                                | 1.399 5 | 1.599 9 |
| 8   | 1.191 0                             | 1.391 0 | 1.595 5 | 1.199 8                                | 1.400 0 | 1.600 0 |
| 9   | 1.195 5                             | 1.395 5 | 1.597 8 |                                        |         |         |
| 10  | 1.197 8                             | 1.397 8 | 1.598 9 |                                        |         |         |
| 11  | 1.198 9                             | 1.398 9 | 1.599 4 |                                        |         |         |
| 12  | 1.199 4                             | 1.399 4 | 1.599 7 |                                        |         |         |
| 13  | 1.199 7                             | 1.399 7 | 1.599 9 |                                        |         |         |

由计算结果知松弛法迭代 8 次所得结果比 Gauss-Seidel 迭代法迭代 13 次的结果还要精确, 松弛因子  $\omega = 1.28$  加速收敛效果明显.

松弛因子的选取对收敛速度影响极大, 但目前尚无可供实用的计算最佳松弛因子的方法. 实际计算时, 通常是根据系数矩阵的性质及实际计算经验, 通过试算来确定松弛因子的值. 特别当  $A$  为对称正定的三对角阵时, 有

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(B)^2}}$$

其中  $\rho(B)$  为 Jacobi 迭代矩阵的谱半径.

为提高计算效率,改善收敛性,又产生了对称超松弛法,简称 SSOR 法. 对称超松弛法实质上是将  $L, U$  同等看待,连续两次使用 SOR 迭代公式得到. 它的优点是对某些使用 SOR 法不收敛的问题,可以构造出收敛的 SSOR 法;松弛法对松弛因子十分敏感,SSOR 法则不敏感.

### § 3.3 迭代法的收敛条件

#### 3.3.1 矩阵的谱半径

迭代法的收敛性与迭代矩阵的特征值有关.

**定义 3.3** 设  $A$  为  $n$  阶方阵,  $\lambda_i (i=1, 2, \dots, n)$  为  $A$  的特征值,称特征值模的最大值为矩阵  $A$  的谱半径,记为

$$\rho(A) = \max_{1 \leq i \leq n} \{ |\lambda_i| \} \quad (3-18)$$

$(\lambda_1, \lambda_2, \dots, \lambda_n)$  称为矩阵  $A$  的谱.

由特征值的定义容易得出,矩阵

$$A^k = \underbrace{AA \cdots A}_{k \uparrow}$$

的谱是  $(\lambda_1^k, \lambda_2^k, \dots, \lambda_n^k) (k=1, 2, \dots)$ , 因而

$$\rho(A^k) = [\rho(A)]^k \quad (3-19)$$

矩阵的谱半径与范数有以下关系.

**定理 3.3** 设  $A$  为任意  $n$  阶方阵,  $\|\cdot\|$  为任意由向量范数诱导出的矩阵范数,则

$$\rho(A) \leq \|A\|$$

[证明] 对  $A$  的任一特征值  $\lambda_i$  及相应的特征向量  $u_i$ , 都有

$$|\lambda_i| \|u_i\| = \|\lambda_i u_i\| = \|A u_i\| \leq \|A\| \|u_i\|$$

因为  $u_i$  为非零向量, 于是有

$$|\lambda_i| \leq \|A\|$$

由  $\lambda_i$  的任意性即得

$$\rho(A) \leq \|A\|$$

**定理 3.4** 设  $A$  为  $n$  阶方阵, 则对任意正数  $\varepsilon$ , 存在一种矩阵范数  $\|\cdot\|$ , 使得

$$\|A\| \leq \rho(A) + \varepsilon$$

证明参看[1].

对任意  $n$  阶方阵  $A$ , 一般不存在矩阵范数  $\|\cdot\|$ , 使得  $\rho(A) = \|A\|$ . 但若  $A$  为对称矩阵, 则有  $\rho(A) = \|A\|_2$ .

下面的结论对建立迭代法的收敛条件十分重要.

**定理 3.5** 设  $A$  为  $n$  阶方阵, 则

$$\lim_{k \rightarrow \infty} A^k = 0$$

的充要条件为  $\rho(A) < 1$ .

[证明] 必要性. 若

$$\lim_{k \rightarrow \infty} A^k = 0$$

由定义 3.2

$$\lim_{k \rightarrow \infty} \|A^k\| = 0$$

而

$$0 \leq \rho(A^k) = [\rho(A)]^k \leq \|A^k\|$$

于是由极限存在准则, 有

$$\lim_{k \rightarrow \infty} [\rho(A)]^k = 0$$

所以  $\rho(A) < 1$ .

充分性. 若  $\rho(A) < 1$ , 取  $\varepsilon = \frac{1 - \rho(A)}{2} > 0$ , 由定理 3.4, 存在一种矩阵范数  $\|\cdot\|$ , 使得

$$\|A\| \leq \rho(A) + \varepsilon = \frac{1 + \rho(A)}{2} < 1$$

而  $\|A^k\| \leq \|A\|^k$ , 于是

$$\lim_{k \rightarrow \infty} \|A^k\| = \lim_{k \rightarrow \infty} \|A\|^k = 0$$

所以

$$\lim_{k \rightarrow \infty} A^k = 0$$

### 3.3.2 迭代法的收敛条件

**定理 3.6** 对任意初始向量  $x^{(0)}$  和右端项  $g$ , 由迭代格式

$$x^{(k+1)} = Mx^{(k)} + g \quad (k = 0, 1, 2, \dots) \quad (3-20)$$

产生的向量序列  $\{x^{(k)}\}$  收敛的充要条件是

$$\rho(M) < 1$$

[证明] 设存在  $n$  维向量  $x^*$ , 使得

$$\lim_{k \rightarrow \infty} x^{(k)} = x^*$$

则  $x^*$  满足

$$\mathbf{x}^* = M\mathbf{x}^* + \mathbf{g}$$

由迭代公式(3-20),有

$$\begin{aligned}\mathbf{x}^{(k)} - \mathbf{x}^* &= M\mathbf{x}^{(k-1)} + \mathbf{g} - M\mathbf{x}^* - \mathbf{g} = M(\mathbf{x}^{(k-1)} - \mathbf{x}^*) \\ &= M^2(\mathbf{x}^{(k-2)} - \mathbf{x}^*) = M^k(\mathbf{x}^{(0)} - \mathbf{x}^*)\end{aligned}$$

于是有

$$\lim_{k \rightarrow \infty} M^k(\mathbf{x}^{(0)} - \mathbf{x}^*) = \lim_{k \rightarrow \infty} (\mathbf{x}^{(k)} - \mathbf{x}^*) = 0$$

因为  $\mathbf{x}^{(0)}$  为任意  $n$  维向量,因此上式成立必须

$$\lim_{k \rightarrow \infty} M^k = 0$$

由定理 3.5

$$\rho(M) < 1$$

反之,若  $\rho(M) < 1$ ,则  $\lambda = 1$  不是  $M$  的特征值,因而有  $|I - M| \neq 0$ ,于是对任意  $n$  维向量  $\mathbf{g}$ ,方程组  $(I - M)\mathbf{x} = \mathbf{g}$  有唯一解,记为  $\mathbf{x}^*$ ,即

$$\mathbf{x}^* = M\mathbf{x}^* + \mathbf{g}$$

并且

$$\lim_{k \rightarrow \infty} M^k = 0$$

又因为

$$\mathbf{x}^{(k)} - \mathbf{x}^* = M(\mathbf{x}^{(k-1)} - \mathbf{x}^*) = M^k(\mathbf{x}^{(0)} - \mathbf{x}^*)$$

所以,对任意初始向量  $\mathbf{x}^{(0)}$ ,都有

$$\lim_{k \rightarrow \infty} (\mathbf{x}^{(k)} - \mathbf{x}^*) = \lim_{k \rightarrow \infty} M^k(\mathbf{x}^{(0)} - \mathbf{x}^*) = 0$$

即由迭代公式(3-20)产生的向量序列  $\{\mathbf{x}^{(k)}\}$  收敛.

定理 3.6 表明,迭代法收敛与否只决定于迭代矩阵的谱半径,与初始向量及方程组的右端项无关.对同一方程组,由于不同的迭代法迭代矩阵不同,因此可能出现有的方法收敛,有的方法发散的情形.

**例 3** 讨论用 Jacobi 迭代法与 Gauss-Seidel 迭代法求解方程组

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 1 \\ x_1 + x_2 + x_3 = 2 \\ 2x_1 + 2x_2 + x_3 = 3 \end{cases}$$

的收敛性.

[解] 由定理 3.6,迭代法是否收敛等价于迭代矩阵的谱半径是否小于 1,故应先求迭代矩阵.因为

$$A = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} \quad D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$L = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & -2 & 0 \end{bmatrix} \quad U = \begin{bmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

Jacobi 迭代法的迭代矩阵为

$$B = I - D^{-1}A = \begin{bmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{bmatrix}$$

其特征方程为

$$|\lambda I - B| = \begin{vmatrix} \lambda & 2 & -2 \\ 1 & \lambda & 1 \\ 2 & 2 & \lambda \end{vmatrix} = \lambda^3 = 0$$

因此有  $\lambda_1 = \lambda_2 = \lambda_3 = 0$ , 于是  $\rho(B) = 0 < 1$ , 所以 Jacobi 迭代法收敛.

如果用 Gauss-Seidel 迭代, 由

$$D - L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 2 & 1 \end{bmatrix}$$

容易求出

$$(D - L)^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix}$$

于是迭代矩阵为

$$M = (D - L)^{-1}U = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 & 2 \\ 0 & 2 & -3 \\ 0 & 0 & 2 \end{bmatrix}$$

其特征方程为

$$|\lambda I - M| = \begin{vmatrix} \lambda & 2 & -2 \\ 0 & \lambda - 2 & 3 \\ 0 & 0 & \lambda - 2 \end{vmatrix} = \lambda(\lambda - 2)^2 = 0$$

特征值为  $\lambda_1 = 0, \lambda_2 = \lambda_3 = 2$ , 故  $\rho(M) = 2 > 1$ , 所以, Gauss-Seidel 迭代法发散.

由定理 3.4 及定理 3.6, 得

**推论 1** 设迭代格式  $x^{(k+1)} = Mx^{(k)} + g$ , 若  $\|M\| < 1$ , 则向量序列  $\{x^{(k)}\}$  收敛.

**推论 2** 松弛法收敛的必要条件是  $0 < \omega < 2$ .

[证明] 设松弛法的迭代矩阵  $M$  有特征值  $\lambda_1, \lambda_2, \dots, \lambda_n$ . 因为

$$|\det(M)| = |\lambda_1 \lambda_2 \cdots \lambda_n| \leq [\rho(M)]^n$$

由定理 3.6, 松弛法收敛必有

$$|\det(\mathbf{M})| < 1$$

又因为

$$\begin{aligned} |\det(\mathbf{M})| &= |(\mathbf{D} - \omega\mathbf{L})^{-1}| |(1 - \omega)\mathbf{D} + \omega\mathbf{U}| \\ |(\mathbf{D} - \omega\mathbf{L})^{-1}| &= \frac{1}{a_{11}a_{22}\cdots a_{nn}} \\ |(1 - \omega)\mathbf{D} + \omega\mathbf{U}| &= (1 - \omega)^n a_{11}a_{22}\cdots a_{nn} \end{aligned}$$

于是有

$$|\det(\mathbf{M})| = |(1 - \omega)^n| < 1$$

所以  $0 < \omega < 2$ .

例 3 也说明了  $0 < \omega < 2$  确实只是松弛法收敛的必要条件, 而非充分条件, 因为 Gauss-Seidel 迭代法即为  $\omega = 1$  的情形.

定理 3.6 虽然给出了判别迭代法收敛的充要条件, 但实际使用时很不方便. 因为求逆矩阵和特征值的难度并不亚于用直接方法求解线性方程组. 推论 1 与推论 2 使用起来方便得多, 但它们仅分别给出收敛的充分条件与必要条件, 许多情形下, 不能起作用. 如例 3 中, 两个方法均有  $\|\mathbf{B}\|_{\infty} > 1$ ,  $\|\mathbf{M}\|_{\infty} > 1$ , 由推论 1 无法判别收敛性. 下面对一些特殊的系数矩阵给出几个常用的判敛条件.

**定义 3.4** 若  $n$  阶方阵  $\mathbf{A} = (a_{ij})$  满足

$$|a_{ii}| \geq \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad (i = 1, 2, \dots, n) \quad (3-21)$$

且至少有一个  $i$  值, 使上式中不等号严格成立, 则称  $\mathbf{A}$  为弱对角占优阵. 若对所有  $i$ , 式(3-21)中不等号均严格成立, 则称  $\mathbf{A}$  为严格对角占优阵.

**定义 3.5** 如果矩阵  $\mathbf{A}$  不能通过行的互换和相应列的互换成为形式

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{O} & \mathbf{A}_{22} \end{bmatrix}$$

其中  $\mathbf{A}_{11}, \mathbf{A}_{22}$  为方阵, 则称  $\mathbf{A}$  是不可约的.

如例 2, 例 3 的系数矩阵是不可约的. 矩阵

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 0 \\ -1 & 3 & 0 \\ 4 & 1 & 2 \end{bmatrix}$$

是可约的. 一般地, 设  $\mathbf{A} = (a_{ij})$  为  $n$  阶方阵 ( $n \geq 2$ ), 若存在非空集  $I \subset \{1, 2, \dots, n\}$ , 使得当  $i \in I$  而  $j \notin I$  时有  $a_{ij} = 0$ , 则  $\mathbf{A}$  是可约阵.

几个常用的收敛条件.

设有线性方程组  $\mathbf{Ax} = \mathbf{b}$ , 下列结论成立:

1. 若  $A$  为严格对角占优阵或不可约弱对角占优阵, 则 Jacobi 迭代法和 Gauss-Seidel 迭代法均收敛.

2. 若  $A$  为严格对角占优阵,  $0 < \omega \leq 1$ , 则松弛法收敛.

3. 若  $A$  为对称正定矩阵,  $0 < \omega < 2$ , 则松弛法收敛. 即若  $A$  是对称正定矩阵, 则松弛法收敛的充要条件为  $0 < \omega < 2$ .

如例 1 中, 系数矩阵

$$A = \begin{bmatrix} 10 & -1 & -2 \\ -1 & 10 & -2 \\ -1 & -1 & 5 \end{bmatrix}$$

为严格对角占优阵, 故 Jacobi 迭代法与 Gauss-Seidel 迭代均收敛. 例 2 中, 系数矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

非严格对角占优, 但  $A$  为对称正定矩阵,  $\omega = 1.4$  故松弛法收敛.

上述结论的证明可参看[1, 3].

**例 4** 设有方程组  $Ax = b$ , 其中

$$A = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & 1 \end{bmatrix}$$

讨论用三种迭代法求解的收敛性.

[解] 因为  $A$  为对称矩阵, 且其各阶主子式皆大于零, 故  $A$  为对称正定矩阵. 由判别条件 3, Gauss-Seidel 迭代法与松弛法 ( $0 < \omega < 2$ ) 均收敛.  $A$  不是弱对角占优阵, 故不能用条件 1 判别 Jacobi 迭代的收敛性. 容易算出, Jacobi 迭代法的迭代矩阵为

$$B = I - D^{-1}A = \begin{bmatrix} 0 & -\frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & 0 & -\frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{2} & 0 \end{bmatrix}$$

其特征方程



$$\begin{aligned}
 |\lambda I - B| &= \begin{vmatrix} \lambda & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \lambda & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \lambda \end{vmatrix} = \lambda^3 + \frac{1}{4} - \frac{3}{4}\lambda \\
 &= \left(\lambda - \frac{1}{2}\right)^2 (\lambda + 1) = 0
 \end{aligned}$$

有根  $\lambda_1 = \lambda_2 = \frac{1}{2}$ ,  $\lambda_3 = -1$ , 因而  $\rho(B) = 1$ . 由定理 3.6, Jacobi 迭代法不收敛.

值得指出的是, 改变方程组中方程的次序, 即将系数矩阵作行交换, 会改变迭代法的收敛性. 例如若方程组  $Ax = b$  的系数矩阵为

$$A = \begin{bmatrix} 3 & -10 \\ 9 & -4 \end{bmatrix}$$

Jacobi 迭代与 Gauss-Seidel 迭代的迭代矩阵分别为

$$B = \begin{bmatrix} 0 & -\frac{10}{3} \\ -\frac{9}{4} & 0 \end{bmatrix} \quad M = \begin{bmatrix} 0 & \frac{10}{3} \\ 0 & \frac{15}{2} \end{bmatrix}$$

它们的谱半径为  $\rho(B) = \frac{\sqrt{30}}{2}$ ,  $\rho(M) = \frac{15}{2}$ . 由定理 3.6, 这两种迭代法均不收敛.

但若交换两个方程的次序, 得原方程组的同解方程组  $A'x = b'$ , 其中

$$A' = \begin{bmatrix} 9 & -4 \\ 3 & -10 \end{bmatrix}$$

显然,  $A'$  是严格对角占优阵, 因而对方程组  $A'x = b'$  用 Jacobi 迭代与 Gauss-Seidel 迭代求解均收敛.

### 3.3.3 误差估计

**定理 3.7** 设有迭代格式  $x^{(k+1)} = Mx^{(k)} + g$ , 若  $\|M\| < 1$ ,  $\{x^{(k)}\}$  收敛于  $x^*$ , 则有误差估计式

$$\|x^{(k)} - x^*\| \leq \frac{\|M\|^k}{1 - \|M\|} \|x^{(1)} - x^{(0)}\| \quad (3-22)$$

[证明] 因为  $\rho(M) \leq \|M\| < 1$ , 故  $|I - M| \neq 0$ , 于是  $(I - M)^{-1}$  存在, 方程组  $x = Mx + g$  有唯一解  $x^*$ , 且  $x^* = (I - M)^{-1}g$ . 从而有

$$\begin{aligned}
 x^{(k)} - x^* &= M(x^{(k-1)} - x^*) = M^k(x^{(0)} - x^*) \\
 &= M^k[x^{(0)} - (I - M)^{-1}g]
 \end{aligned}$$

$$\begin{aligned}
 &= \mathbf{M}^k (\mathbf{I} - \mathbf{M})^{-1} [(\mathbf{I} - \mathbf{M}) \mathbf{x}^{(0)} - \mathbf{g}] \\
 &= \mathbf{M}^k (\mathbf{I} - \mathbf{M})^{-1} (\mathbf{x}^{(0)} - \mathbf{x}^{(1)})
 \end{aligned}$$

取范数得

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \|\mathbf{M}\|^k \|(\mathbf{I} - \mathbf{M})^{-1}\| \|\mathbf{x}^{(0)} - \mathbf{x}^{(1)}\| \quad (3-23)$$

又因为  $(\mathbf{I} - \mathbf{M})(\mathbf{I} - \mathbf{M})^{-1} = \mathbf{I}$ , 于是

$$(\mathbf{I} - \mathbf{M})^{-1} = \mathbf{I} + \mathbf{M}(\mathbf{I} - \mathbf{M})^{-1}$$

取范数得

$$\begin{aligned}
 \|(\mathbf{I} - \mathbf{M})^{-1}\| &= \|\mathbf{I} + \mathbf{M}(\mathbf{I} - \mathbf{M})^{-1}\| \\
 &\leq \|\mathbf{I}\| + \|\mathbf{M}\| \|(\mathbf{I} - \mathbf{M})^{-1}\|
 \end{aligned}$$

移项得

$$\|(\mathbf{I} - \mathbf{M})^{-1}\| \leq \frac{1}{1 - \|\mathbf{M}\|} \quad (3-24)$$

将此式代入式(3-23), 即得

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \frac{\|\mathbf{M}\|^k}{1 - \|\mathbf{M}\|} \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|$$

有了误差估计式(3-22), 根据事先给定的精度  $\varepsilon$ , 可以估算出迭代的次数  $k$ :

$$k \geq \frac{\ln \frac{\varepsilon(1 - \|\mathbf{M}\|)}{\|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|}}{\ln \|\mathbf{M}\|} \quad (3-25)$$

如例 1 中, 若要求各分量的误差绝对值不超过  $10^{-4}$ , 则由

$$\mathbf{M} = \begin{bmatrix} 0 & 0.1 & 0.2 \\ 0.1 & 0 & 0.2 \\ 0.2 & 0.2 & 0 \end{bmatrix}, \quad \mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{x}^{(1)} = \begin{bmatrix} 7.2 \\ 8.3 \\ 8.4 \end{bmatrix}$$

有  $\|\mathbf{M}\|_{\infty} = 0.4$ ,  $\|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|_{\infty} = 8.4$ , 代入式(3-25)得

$$k \geq \frac{\ln \frac{10^{-4} \times (1 - 0.4)}{8.4}}{\ln 0.4} = 12.932$$

所以需要迭代 13 次才能保证各分量误差绝对值不超过  $10^{-4}$ .

实际计算时, 常常采取事后估计误差的方法, 即利用相邻两次迭代值之差是否达到精度作为停机准则. 因而下面的结论比定理 3.7 更实用.

**定理 3.8** 在定理 3.7 的条件下, 有

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \frac{\|\mathbf{M}\|}{1 - \|\mathbf{M}\|} \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\| \quad (3-26)$$

[证明] 因为

$$\mathbf{x}^{(k)} - \mathbf{x}^* = \mathbf{M}(\mathbf{x}^{(k-1)} - \mathbf{x}^*)$$

$$\begin{aligned}
 &= M[x^{(k-1)} - (I - M)^{-1}g] \\
 &= M(I - M)^{-1}[x^{(k-1)} - Mx^{(k-1)} - g] \\
 &= M(I - M)^{-1}(x^{(k-1)} - x^{(k)})
 \end{aligned}$$

所以

$$\begin{aligned}
 \|x^{(k)} - x^*\| &\leq \|M\| \|(I - M)^{-1}\| \|x^{(k-1)} - x^{(k)}\| \\
 &\leq \frac{\|M\|}{1 - \|M\|} \|x^{(k)} - x^{(k-1)}\|
 \end{aligned}$$

由定理 3.8, 为使  $\|x^{(k)} - x^*\| < \varepsilon$ , 只要

$$\|x^{(k)} - x^{(k-1)}\| \leq \frac{1 - \|M\|}{\|M\|} \varepsilon$$

即可. 实际计算时, 当  $\|M\|$  不太接近 1 时, 可用  $\|x^{(k)} - x^{(k-1)}\| < \varepsilon$  作为停机准则,  $x^{(k)}$  即为满足精度之近似解.

### § 3.4 最速下降法与共轭梯度法

最速下降法与共轭梯度法是基于变分法原理的迭代法, 等价于求一个二次函数的极值, 也是求解大型稀疏对称正定方程组的最有效方法之一.

#### 3.4.1 最速下降法

当方程组 (3-1) 的系数矩阵  $A$  对称正定时, 任取  $x \in \mathbb{R}^n$ , 构造二次函数

$$\begin{aligned}
 \varphi(x) &= \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle \\
 &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j - \sum_{j=1}^n b_j x_j
 \end{aligned} \tag{3-27}$$

利用  $A$  的对称性, 容易证明  $\varphi$  具有如下性质:

- (1)  $\varphi$  的梯度  $\nabla \varphi(x) = Ax - b$ ;
- (2) 对于任意的  $x, y \in \mathbb{R}^n$ , 实数  $\alpha \in \mathbb{R}$ , 有

$$\begin{aligned}
 \varphi(x + \alpha y) &= \frac{1}{2} \langle A(x + \alpha y), (x + \alpha y) \rangle - \langle b, (x + \alpha y) \rangle \\
 &= \varphi(x) + \alpha \langle Ax - b, y \rangle + \frac{\alpha^2}{2} \langle Ay, y \rangle;
 \end{aligned}$$

- (3) 若  $x^* = A^{-1}b$  是方程组 (3-1) 的解, 则

$$\varphi(x^*) = -\frac{1}{2} \langle b, x^* \rangle = -\frac{1}{2} \langle Ax^*, x^* \rangle$$

且对于任意的  $x \in \mathbb{R}^n$ , 有

$$\begin{aligned}\varphi(x) - \varphi(x^*) &= \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle + \frac{1}{2} \langle b, x^* \rangle \\ &= \frac{1}{2} \langle A(x - x^*), (x - x^*) \rangle\end{aligned}$$

**定理 3.9** 设  $A$  对称正定, 则  $x^*$  是方程组  $Ax = b$  的解的充要条件是  $x^*$  满足

$$\varphi(x^*) = \min_{x \in \mathbb{R}^n} \varphi(x)$$

[证明] 必要性. 设  $Ax^* = b$ , 则由性质(3)及  $A$  的对称正定性有

$$\varphi(x) - \varphi(x^*) = \frac{1}{2} \langle A(x - x^*), (x - x^*) \rangle \geq 0$$

即  $\varphi(x^*) = \min_{x \in \mathbb{R}^n} \varphi(x)$ .

充分性. 设  $\varphi(x^*) = \min_{x \in \mathbb{R}^n} \varphi(x)$ , 则对于任意的  $y \in \mathbb{R}^n$  有

$$\left. \frac{d\varphi(x^* + \alpha y)}{d\alpha} \right|_{\alpha=0} = 0$$

由性质(2)得

$$\langle Ax^* - b, y \rangle + \alpha \langle Ay, y \rangle \Big|_{\alpha=0} = 0$$

即对于任意的  $y \in \mathbb{R}^n$ , 有  $\langle Ax^* - b, y \rangle = 0$ , 故必有  $Ax^* = b$ .

定理 3.9 将求解方程组(3-1)的问题转化为求函数  $\varphi(x)$  的极小点问题, 即求向量  $x^* \in \mathbb{R}^n$ , 使得

$$\varphi(x^*) = \min_{x \in \mathbb{R}^n} \varphi(x) \quad (3-28)$$

由微积分知识, 一个多元函数沿其负梯度方向函数值下降最快, 因此可以每次迭代时, 从点  $x^{(k)}$  出发, 沿着函数在该点处的负梯度方向求一个近似点  $x^{(k+1)}$ , 使得  $\varphi(x^{(k+1)})$  在此方向达到极小值, 这就是最速下降法.

最速下降法具体计算过程如下: 设从  $x^{(0)}$  出发, 已迭代计算至  $x^{(k)}$ ,  $\varphi(x)$  在  $x^{(k)}$  处的负梯度方向  $r^{(k)} = -\nabla \varphi(x^{(k)}) = b - Ax^{(k)}$  (称为搜索方向), 沿此方向  $\varphi$  的函数值可表示为  $\varphi(x^{(k)} + \alpha r^{(k)})$ , 为求极小点令

$$\frac{d\varphi(x^{(k)} + \alpha r^{(k)})}{d\alpha} = \langle Ax^{(k)} - b, r^{(k)} \rangle + \alpha \langle Ar^{(k)}, r^{(k)} \rangle = 0$$

得

$$\alpha_k = \frac{\langle r^{(k)}, r^{(k)} \rangle}{\langle Ar^{(k)}, r^{(k)} \rangle}$$

于是  $x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}$  ( $k=1, 2, \dots$ ), 且序列  $\{x^{(k)}\}$  满足  $\varphi(x^{(k+1)}) < \varphi(x^{(k)})$  ( $k=1, 2, \dots$ ).

可以证明,从任意初始点  $\mathbf{x}^{(0)}$  出发,最速下降法所得序列  $\{\mathbf{x}^{(k)}\}$  均收敛于问题(3-28)的解,即收敛于方程组(3-1)的解. 其收敛速度取决于比值  $\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n}$ , 其中  $\lambda_1, \lambda_n$  分别是  $A$  的最大、最小特征值. 当  $\lambda_1 \gg \lambda_n$  时,最速下降法收敛缓慢. 因此实际中很少使用.

### 3.4.2 共轭梯度法 (Conjugate Gradient method)

在最速下降法中,每一步迭代的搜索方向为负梯度方向  $\mathbf{r}^{(k)}$ ,它只能保证  $\varphi(\mathbf{x})$  局部下降最快,从整体来看并非最优. 共轭梯度法仍然是一维搜索法,但不再沿负梯度方向  $\mathbf{r}^{(1)}, \mathbf{r}^{(2)}, \dots, \mathbf{r}^{(k)}$  搜索,而是沿一组关于  $A$  共轭的方向  $\mathbf{d}^{(1)}, \mathbf{d}^{(2)}, \dots, \mathbf{d}^{(k)}$  搜索.

**定义 3.6** 设  $A$  对称正定,若  $\mathbf{R}^n$  中的向量组  $\{\mathbf{d}^{(0)}, \mathbf{d}^{(1)}, \dots, \mathbf{d}^{(k)}\}$  满足

$$\langle \mathbf{d}^{(i)}, A\mathbf{d}^{(j)} \rangle = 0, \quad i \neq j$$

则称它是  $\mathbf{R}^n$  中的一个  $A$ -共轭向量组,或称  $A$ -正交向量组.

共轭梯度法算法如下:设从  $\mathbf{x}^{(0)}$  出发,已迭代计算至  $\mathbf{x}^{(k)}$ ,选取搜索方向

$$\mathbf{d}^{(k)} = -\nabla \varphi(\mathbf{x}^{(k)}) + \beta_{k-1} \mathbf{d}^{(k-1)} = \mathbf{b} - A\mathbf{x}^{(k)} + \beta_{k-1} \mathbf{d}^{(k-1)}$$

由  $\mathbf{d}^{(k)}$  与  $\mathbf{d}^{(k-1)}$  关于  $A$ -共轭,即  $\langle \mathbf{d}^{(k)}, A\mathbf{d}^{(k-1)} \rangle = 0$  得

$$\beta_{k-1} = -\frac{\langle \mathbf{b} - A\mathbf{x}^{(k)}, A\mathbf{d}^{(k-1)} \rangle}{\langle \mathbf{d}^{(k-1)}, A\mathbf{d}^{(k-1)} \rangle} \quad (3-29)$$

从而搜索方向

$$\mathbf{d}^{(k)} = (\mathbf{b} - A\mathbf{x}^{(k)}) - \frac{\langle \mathbf{b} - A\mathbf{x}^{(k)}, A\mathbf{d}^{(k-1)} \rangle}{\langle \mathbf{d}^{(k-1)}, A\mathbf{d}^{(k-1)} \rangle} \mathbf{d}^{(k-1)} \quad (3-30)$$

再沿  $\mathbf{d}^{(k)}$  方向求  $\varphi(\mathbf{x})$  的极小点  $\mathbf{x}^{(k+1)}$ ,即

$$\varphi(\mathbf{x}^{(k+1)}) = \min_{\lambda > 0} \varphi(\mathbf{x}^{(k)} + \lambda \mathbf{d}^{(k)})$$

经计算可得

$$\lambda_k = \frac{\langle \mathbf{b} - A\mathbf{x}^{(k)}, \mathbf{d}^{(k)} \rangle}{\langle \mathbf{d}^{(k)}, A\mathbf{d}^{(k)} \rangle} \quad (3-31)$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \lambda_k \mathbf{d}^{(k)} \quad (3-32)$$

由此得方程组(3-1)的近似解序列  $\{\mathbf{x}^{(k)}\}$ .

#### 算法 3.4

1. 输入  $A = (a_{ij}), \mathbf{b} = (b_1, b_2, \dots, b_n)^T$ , 及初始向量  $\mathbf{x}^{(0)}$ , 误差限  $\varepsilon$ .
2. 计算  $\mathbf{d}^{(0)} = \mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}, \lambda_0 = \langle \mathbf{r}^{(0)}, \mathbf{d}^{(0)} \rangle / \langle \mathbf{d}^{(0)}, A\mathbf{d}^{(0)} \rangle, \mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \lambda_0 \mathbf{d}^{(0)}$ .
3. 对于  $k = 1, 2, \dots$ , 计算

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$$

$$\beta_{k-1} = -\langle \mathbf{r}^{(k)}, A\mathbf{d}^{(k-1)} \rangle / \langle \mathbf{d}^{(k-1)}, A\mathbf{d}^{(k-1)} \rangle, \quad \mathbf{d}^{(k)} = \mathbf{r}^{(k)} + \beta_{k-1}\mathbf{d}^{(k-1)}$$

$$\lambda_k = \langle \mathbf{r}^{(k)}, \mathbf{d}^{(k)} \rangle / \langle \mathbf{d}^{(k)}, A\mathbf{d}^{(k)} \rangle, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \lambda_k \mathbf{d}^{(k)}$$

4. 若  $\mathbf{r}^{(k)} = 0$  或  $\|\mathbf{r}^{(k)}\| < \varepsilon$  则终止计算, 输出  $\mathbf{x}^* = \mathbf{x}^{(k)}$ . 否则转 3.

**例 5** 用共轭梯度法求解例 2 中的方程组.

[解] 例 2 中方程组的系数矩阵和右端项为

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 1.8 \end{bmatrix}$$

精度要求  $10^{-8}$ . 取初始向量  $\mathbf{x}^{(0)} = (0, 0, 0)^T$ , 利用 MATLAB 计算得

$$\mathbf{d}^{(0)} = \mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)} = (1.000, 0, 1.800)^T$$

$$\lambda_0 = \langle \mathbf{r}^{(0)}, \mathbf{d}^{(0)} \rangle / \langle \mathbf{d}^{(0)}, A\mathbf{d}^{(0)} \rangle = 0.5000$$

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \lambda_0 \mathbf{d}^{(0)} = (0.5000, 0, 0.9000)^T$$

$$\mathbf{r}^{(1)} = \mathbf{b} - A\mathbf{x}^{(1)} = (0, 1.4000, 0)^T$$

$$\beta_0 = -\langle \mathbf{r}^{(1)}, A\mathbf{d}^{(0)} \rangle / \langle \mathbf{d}^{(0)}, A\mathbf{d}^{(0)} \rangle = 0.4623$$

$$\mathbf{d}^{(1)} = \mathbf{r}^{(1)} + \beta_0 \mathbf{d}^{(0)} = (0.4623, 1.4000, 0.8321)^T$$

$$\lambda_1 = \langle \mathbf{r}^{(1)}, \mathbf{d}^{(1)} \rangle / \langle \mathbf{d}^{(1)}, A\mathbf{d}^{(1)} \rangle = 0.9298$$

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \lambda_1 \mathbf{d}^{(1)} = (0.9298, 1.3018, 1.6737)^T$$

$$\mathbf{r}^{(2)} = \mathbf{b} - A\mathbf{x}^{(2)} = (0.4421, -0.0000, 0.2456)^T$$

$$\beta_1 = -\langle \mathbf{r}^{(2)}, A\mathbf{d}^{(1)} \rangle / \langle \mathbf{d}^{(1)}, A\mathbf{d}^{(1)} \rangle = 0.1305$$

$$\mathbf{d}^{(2)} = \mathbf{r}^{(2)} + \beta_1 \mathbf{d}^{(1)} = (0.5024, 0.1827, -0.1370)^T$$

$$\lambda_2 = \langle \mathbf{r}^{(2)}, \mathbf{d}^{(2)} \rangle / \langle \mathbf{d}^{(2)}, A\mathbf{d}^{(2)} \rangle = 0.5377$$

$$\mathbf{x}^{(3)} = \mathbf{x}^{(2)} + \lambda_2 \mathbf{d}^{(2)} = (1.2000, 1.4000, 1.6000)^T$$

$$\mathbf{r}^{(3)} = \mathbf{b} - A\mathbf{x}^{(3)} = 10^{-15} \times (0.0000, -0.2220, 0.2220)^T$$

显然迭代到第 3 步即有  $\|\mathbf{r}^{(3)}\|_2 < 10^{-8}$ , 满足迭代收敛条件, 终止迭代. 因此有

$$\mathbf{x}^* = \mathbf{x}^{(3)} = (1.2000, 1.4000, 1.6000)^T$$

如果利用  $\{\mathbf{d}^{(k)}\}$  的  $A$ -共轭性, 式(3-29), 式(3-31)可简化为

$$\beta_{k-1} = \frac{\langle \mathbf{r}^{(k)}, \mathbf{r}^{(k)} \rangle}{\langle \mathbf{r}^{(k-1)}, \mathbf{r}^{(k-1)} \rangle}, \quad \lambda_k = \frac{\langle \mathbf{r}^{(k)}, \mathbf{r}^{(k)} \rangle}{\langle \mathbf{d}^{(k)}, A\mathbf{d}^{(k)} \rangle} \quad (3-33)$$

例 5 从  $\mathbf{x}^{(0)}$  出发, 用共轭梯度法迭代了 3 步就得到了准确解. 理论上可以证明, 若计算过程是精确的, 则对于任何初值  $\mathbf{x}^{(0)}$ , 用共轭梯度法求解问题(3-28)至多进行  $n$  步就能得到精确解. 因此共轭梯度法实质上是一种直接方法. 但在实际计算中, 由于舍入误差的出现, 破坏了它的有限终止性, 所以我们将它作为迭代法使用.

共轭梯度法的优点是存储量小,计算简便. 它能充分利用矩阵  $A$  的稀疏性, 计算时只需存储  $A$  中的非零元素. 共轭梯度法具有超线性收敛性. 如果系数矩阵  $A$  的特征值比较均匀地分布在一个很长的区间内, 共轭梯度法可能收敛得比较缓慢.

对于非对称线性方程组  $Ax = b$ , 可用共轭梯度法求解等价的对称正定方程组  $A^T Ax = A^T b$ , 收敛效果也是不错的.

如果方程组  $Ax = b$  的条件数比较大, 无论是 Jacobi 迭代法、松弛法, 还是共轭梯度法收敛都很缓慢, 为了改善收敛速度, 一般设法降低系数矩阵的条件数, 即预处理技术. 一种方法是在方程组两边同乘一个非奇异矩阵  $P^{-1}$ , 方程组等价地写为

$$P^{-1}Ax = P^{-1}b$$

使得条件数  $\text{cond}(P^{-1}A) \ll \text{cond}(A)$ . 例如, 在共轭梯度法求解  $Ax = b$  时, 若  $A = LL^T$ , 可将问题转化为求解同解方程组

$$L^{-1}AL^{-T}y = L^{-1}b, \quad x = L^{-T}y$$

用共轭梯度法求解方程组  $L^{-1}AL^{-T}y = L^{-1}b$ , 往往能改善收敛效果.

### § 3.5 应用实例

#### 静电场问题

设平面上有一对带有相同电量的正、负点电荷, 在它们的周围形成了一个电场, 这就是电偶极子. 偶极子周围的电位势  $u(x, y)$  满足 Poisson 方程

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = -\frac{\rho}{\varepsilon_p}$$

其中参数  $\varepsilon_p$  是偶极子周围介质的电介常数,  $\rho(x, y)$  是电荷密度. 考虑矩形区域  $[0, 1] \times [0, 1]$  上势函数的数值解. 设点  $(x_1, y_1) = (0.4, 0.4)$  处有一正电荷, 点  $(x_2, y_2) = (0.6, 0.6)$  处有一负电荷, 带电量分别为

$$\rho(0.4, 0.4) = -\rho(0.6, 0.6) = 10^{-8} \text{ coul/m}^2,$$

偶极子周围是空气, 参数  $\varepsilon_p = 8.9 \times 10^{-12}$ , 边界条件  $u(0, y) = u(1, y) = u(x, 0) = u(x, 1) = 0$ .

将区间  $[0, 1]$  分成  $n+1$  等份, 则  $\Delta x = \Delta y = h = 2/(n+1)$ , 用二阶中心差商离散偏微分方程中的二阶偏导数, 并以  $u_{ij}$  表示  $u(i\Delta x, j\Delta y)$  的计算近似值, 得差分方程组

$$u_{i+1j} + u_{i-1j} - 4u_{ij} + u_{ij+1} + u_{ij-1} = h^2 \left( -\frac{\rho}{\varepsilon_p} \right)_{ij} \quad (1 \leq i, j \leq n)$$

这是  $n^2$  阶的线性方程组,未知向量  $\mathbf{u} = [u_{11}, u_{12}, \dots, u_{nn}]^T$ , 系数矩阵  $A$  是  $n^2 \times n^2$  的稀疏矩阵,具有如下形式

$$A = \begin{bmatrix} A_1 & I & & & \\ I & A_2 & I & & \\ & I & \ddots & \ddots & \\ & & \ddots & A_{n-1} & I \\ & & & I & A_n \end{bmatrix}_{n^2 \times n^2}$$

其中  $I$  是  $n$  阶的单位阵,

$$A_i = \begin{bmatrix} -4 & 1 & & & \\ 1 & -4 & 1 & & \\ & 1 & -4 & \ddots & \\ & & \ddots & \ddots & 1 \\ & & & 1 & -4 \end{bmatrix}_{n \times n} \quad (i = 1, 2, \dots, n)$$

由于系数矩阵中有大量的零元素,考虑到存储空间和运算量,采用 SOR 迭代法求解线性方程组. 经计算,最佳松弛因子  $\omega = 1.811$ , 迭代误差精度取  $\varepsilon = 10^{-5}$ . 用 MATLAB 编程计算,若取粗网格  $n = 30$ ,得计算结果如图 3-1.

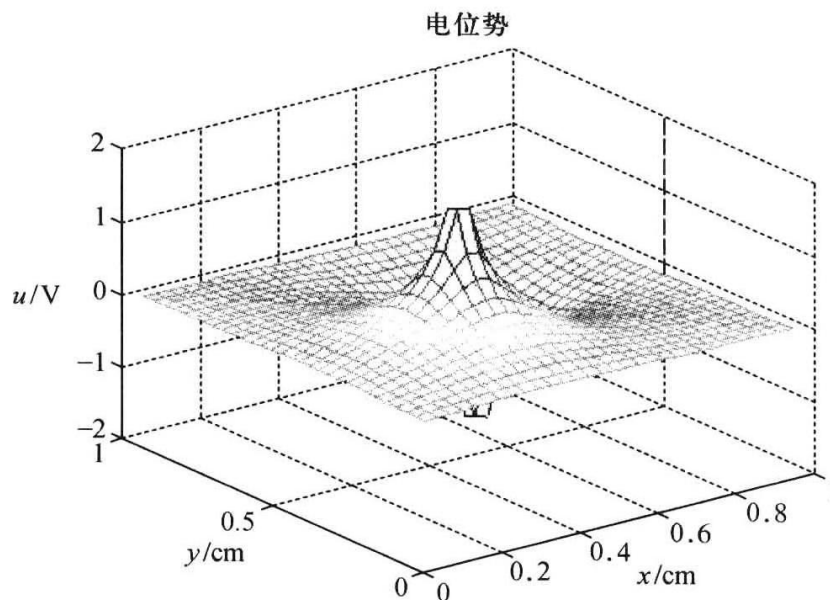


图 3-1

若取  $n = 100$ ,得数值解如图 3-2.



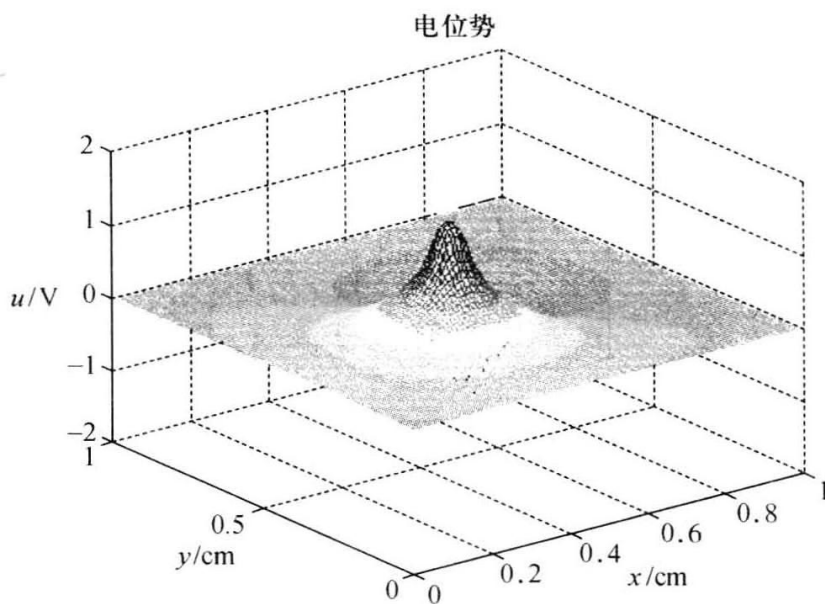


图 3-2

## 评 注

迭代法是求解大型线性方程组(特别是稀疏矩阵情形)的有效方法,具有存储空间小、程序简单等特点. Jacobi 法、Gauss-Seidel 法和 SOR 法是基本的迭代方法,它的理论成形于 19 世纪 50 年代,本章介绍的是最基本的内容. 迭代法的收敛性与系数矩阵的特性密切相关,计算中应选用收敛速度快的迭代法. 当迭代法收敛速度慢时,可通过整体校正、利用特征值外推和最小零偏差等方法加速收敛速度<sup>[18]</sup>.

共轭梯度法最初是针对对称方程组提出的方法,它具有超线性收敛性. 近 20 年来相关的研究得到了很大的发展,目前有关的理论和方法已经相当成熟. 大量的数值实验证实,当系数矩阵  $A$  的条件数较小,或  $A$  的特征值比较集中时,用共轭梯度法迭代几步就能得到高精度的解.

共轭梯度法与预处理技术相结合,往往会大大地改善收敛速度,得到令人满意的结果. 对于非对称矩阵,有计算效果不错的共轭梯度法方程残量法、共轭残量法、不完全分解预优共轭梯度法和分块 SSOR - CG 法等<sup>[18]</sup>. 此外,还有 Krylov 子空间上的共轭梯度型算法、基于 Arnoldi 迭代的广义最小残量法和利用 Lanczos 向量导出的双共轭梯度法等<sup>[18]</sup>. 对于大型稀疏矩阵,以 Galerkin 原理为基础建立的一类迭代法也是很有效的. 这些方法目前仍在发展中,但大量的数值实验表明这些算法是很有效的. 上述方法并没有绝对的好坏之分,方法的选择与具体问题有关.

对于偏微分方程离散后所得的大型代数方程组,系数矩阵经常是稀疏的,且很有规律,常常可将系数矩阵分成几个数组存储或将系数矩阵分块,将大型方程组降阶,再用直接法或迭代法求解较低阶的方程组,故上述各种方法都有相应特殊形式或分块形式.

MATLAB 软件中有基于 Krylov 子空间的迭代法、共轭梯度法及带预处理技术的共轭梯度法的计算工具.

### 习 题 三

1. 分别用 Jacobi 迭代法和 Gauss-Seidel 迭代法求解下列方程组:

$$(1) \begin{cases} 10x_1 - 2x_2 - x_3 = 3 \\ -2x_1 + 10x_2 - x_3 = 15 \\ -x_1 - 2x_2 + 5x_3 = 10 \end{cases} \quad (2) \begin{cases} 8x_1 - 3x_2 + 2x_3 = 20 \\ 4x_1 + 11x_2 - x_3 = 33 \\ 2x_1 + x_2 + 4x_3 = 12 \end{cases}$$

取初值  $\mathbf{x}^{(0)} = (0, 0, 0)^T$ , 计算结果精确到三位小数.

2. 分别用 Jacobi 迭代法, Gauss-Seidel 迭代法与 SOR 法 ( $\omega = 1.46$ ) 求解方程组

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

取初值  $\mathbf{x}^{(0)} = (1, 1, 1, 1)^T$ , 精确到  $10^{-3}$ .

3. 用 Jacobi 迭代和 Gauss-Seidel 迭代求解方程组

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 1 \\ x_1 + x_2 + x_3 = 1 \\ 2x_1 + 2x_2 + x_3 = 1 \end{cases}$$

取初值  $\mathbf{x}^{(0)} = (0, 0, 0)^T$ , 迭代 4 次, 并比较它们的计算结果.

4. 分别用 Gauss-Seidel 迭代法与 SOR 法 ( $\omega = 1.25$ ) 求解方程组

$$\begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 24 \\ 30 \\ -24 \end{bmatrix}$$

取  $\mathbf{x}^{(0)} = (1, 1, 1)^T$ , 迭代 7 次, 并比较它们的计算结果.

## 5. 若用 Jacobi 迭代法求解方程组

$$\begin{bmatrix} a & -1 & -3 \\ -1 & a & -2 \\ 3 & -2 & a \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

讨论实数  $a$  与收敛性的关系.

## 6. 求证用 Jacobi 迭代法求解方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 = b_1 \\ a_{21}x_1 + a_{22}x_2 = b_2 \end{cases} \quad (a_{11}a_{22} \neq 0)$$

迭代收敛的充要条件是

$$\left| \frac{a_{12}a_{21}}{a_{11}a_{22}} \right| < 1$$

## 7. 设有方程组

$$\begin{cases} 10x_1 + 4x_2 + 4x_3 = 13 \\ 4x_1 + 10x_2 + 8x_3 = 11 \\ 4x_1 + 8x_2 + 10x_3 = 25 \end{cases}$$

(1) 分别写出 Jacobi 迭代法, Gauss-Seidel 迭代法及 SOR 法 ( $\omega = 1.35$ ) 的计算公式及迭代矩阵;

(2) 对任意初值, 各迭代是否收敛? 说明理由.

## 8. 设有方程组

$$\begin{bmatrix} 1.2 & -3.6 & -12 \\ -10 & 9 & 0.5 \\ 1 & -4 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

试写出收敛的迭代式, 并说明理由.

## 9. 设有方程组

$$\begin{bmatrix} 1 & a & a \\ 4a & 1 & 0 \\ a & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

(1) 分别写出 Jacobi 迭代法和 Gauss-Seidel 迭代法的计算公式;

(2) 用迭代收敛的充要条件给出使这两种迭代法都收敛的  $a$  的取值范围.

## 10. 分别用 Jacobi 迭代法和 Gauss-Seidel 迭代法求解方程组

$$\begin{cases} 20x_1 + 2x_2 + 3x_3 = 24 \\ x_1 + 8x_2 + x_3 = 12 \\ 2x_1 - 3x_2 + 15x_3 = 30 \end{cases}$$

取  $\mathbf{x}^{(0)} = (0, 0, 0)^T$ , 问迭代是否收敛? 若收敛, 需要迭代多少次, 才能保证各分量误差绝对值小于  $10^{-6}$ ?

11. 证明严格对角占优阵是非奇异矩阵.

12. 证明对称矩阵

$$A = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}$$

当  $-\frac{1}{2} < a < 1$  时为正定矩阵, 且只有当  $-\frac{1}{2} < a < \frac{1}{2}$  时, 用 Jacobi 迭代法求解方程组  $A\mathbf{x} = \mathbf{b}$  才收敛.

13. 分别用最速下降法和共轭梯度法求解方程组

$$\begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 24 \\ 30 \\ -24 \end{bmatrix}$$

## 数值实验

1. 试分别用

(1) Jacobi 迭代法; (2) Gauss-Seidel 迭代法和; (3) 共轭梯度法解线性方程组

$$\begin{bmatrix} 10 & 1 & 2 & 3 & 4 \\ 1 & 9 & -1 & 2 & -3 \\ 2 & -1 & 7 & 3 & -5 \\ 3 & 2 & 3 & 12 & -1 \\ 4 & -3 & -5 & -1 & 15 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 12 \\ -27 \\ 14 \\ -17 \\ 12 \end{bmatrix}$$

迭代初始向量取  $\mathbf{x}^{(0)} = (0, 0, 0, 0, 0)^T$ .

2. 若仅保存系数矩阵  $A = \langle a_{ij} \rangle_{n \times n}$  的非零元, 用共轭梯度法求解  $n = 10^5$  阶的

$$\text{方程组 } A\mathbf{x} = \mathbf{b}, \text{ 其中 } a_{ij} = \begin{cases} 3, & i=j \\ -1, & |i-j|=1 \\ \frac{1}{2}, & i+j=n+1, i \neq \frac{n}{2}, \frac{n}{2}+1 \\ 0, & \text{其他} \end{cases} \quad (i, j=1, 2, \dots, n),$$

$\mathbf{b} = (2.5, 1.5, \dots, 1.5, 1.0, 1.0, 1.5, \dots, 1.5, 2.5)^T$ .

3. 设方程组  $Ax = b$ , 将矩阵  $A$  进行 LU 分解, 取  $P = L$  或用不同的预处理矩阵, 利用 MATLAB 命令 `cgs(A,b,[],[],P)`, `bicg(A,b,[],[],P)`, `bicgstab(A,b,[],[],P)` 迭代求解方程组  $P^{-1}Ax = P^{-1}b$ , 并说明数值实验的结果.

## 第四章 矩阵特征值与特征向量的计算

工程实践中有多种振动问题,如桥梁或建筑物的振动,机械机件的振动,飞机机翼的颤动等.这些问题的求解常常归纳为求矩阵的特征值问题.另外,一些稳定性分析问题及相关分析问题也可以转化为求矩阵特征值与特征向量的问题.

本章仅对实矩阵进行讨论.

### § 4.1 幂法和反幂法

#### 4.1.1 幂法

幂法主要用于求矩阵的按模最大的特征值与相应的特征向量.它是通过迭代产生向量序列,由此计算特征值和特征向量.

设  $n \times n$  阶实矩阵  $A$  的特征值  $\lambda_i (i=1, 2, \dots, n)$  满足

$$|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$$

且与  $\lambda_i (i=1, 2, \dots, n)$  相应的特征向量  $u_1, u_2, \dots, u_n$  线性无关.

给定初始向量  $x^{(0)} \neq 0$ , 由迭代公式

$$x^{(k+1)} = Ax^{(k)} \quad (k = 0, 1, 2, \dots) \quad (4-1)$$

产生向量序列  $\{x^{(k)}\}$ , 可以证明, 当  $k$  充分大时, 有  $\lambda_1 \approx x_i^{(k+1)} / x_i^{(k)}$ , 相应的特征向量为  $x^{(k+1)}$ .

为简便起见, 不妨设  $\|u_i\| = 1 (i=1, 2, \dots, n)$ . 因为  $u_i$  线性无关, 故必存在  $n$  个不全为零的数  $\alpha_i (i=1, 2, \dots, n)$ , 使得  $x^{(0)} = \sum_{i=1}^n \alpha_i u_i$ . 由式(4-1)得

$$x^{(k+1)} = Ax^{(k)} = A^{k+1}x^{(0)} = \sum_{i=1}^n A^{k+1}(\alpha_i u_i) = \sum_{i=1}^n \alpha_i \lambda_i^{k+1} u_i$$

即

$$x^{(k+1)} = \lambda_1^{k+1} \left[ \alpha_1 u_1 + \left( \frac{\lambda_2}{\lambda_1} \right)^{k+1} \alpha_2 u_2 + \dots + \left( \frac{\lambda_n}{\lambda_1} \right)^{k+1} \alpha_n u_n \right] \quad (4-2)$$

设  $\alpha_1 \neq 0$ , 由  $|\lambda_1| > |\lambda_i| (i=2, 3, \dots, n)$  得

$$\lim_{k \rightarrow \infty} \left( \frac{\lambda_i}{\lambda_1} \right)^{k+1} \alpha_i u_i = 0$$

于是

$$\lim_{k \rightarrow \infty} \sum_{i=2}^n \left( \frac{\lambda_i}{\lambda_1} \right)^{k+1} \alpha_i \mathbf{u}_i = 0$$

故只要  $k$  充分大, 就有

$$\mathbf{x}^{(k+1)} = \lambda_1^{k+1} \left[ \alpha_1 \mathbf{u}_1 + \sum_{i=2}^n \left( \frac{\lambda_i}{\lambda_1} \right)^{k+1} \alpha_i \mathbf{u}_i \right] \approx \lambda_1^{k+1} \alpha_1 \mathbf{u}_1 \quad (4-3)$$

因此, 可把  $\mathbf{x}^{(k+1)}$  作为与  $\lambda_1$  相应的特征向量的近似. 由式

$$\mathbf{x}^{(k+1)} \approx \lambda_1^{k+1} \alpha_1 \mathbf{u}_1, \quad \mathbf{x}^{(k)} \approx \lambda_1^k \alpha_1 \mathbf{u}_1$$

又可得出

$$\lambda_1 \approx \frac{x_i^{(k+1)}}{x_i^{(k)}} \quad (i = 1, 2, \dots, n) \quad (4-4)$$

其中  $x_i^{(k)}$  为  $\mathbf{x}^{(k)}$  的第  $i$  个分量. 按式(4-1)和式(4-4)计算矩阵  $A$  按模最大的特征值与相应的特征向量的方法称为幂法.

需要说明的是, 如果  $\mathbf{x}^{(0)}$  的选取恰恰使得  $\alpha_1 = 0$ , 幂法计算仍能进行. 因为计算过程中舍入误差的影响, 迭代若干次后, 必然会产生一个向量  $\mathbf{x}^{(k)}$ , 它在  $\mathbf{u}_1$  方向上的分量不为零, 这样, 以后的计算就满足所设条件. 另一方面, 因为  $\mathbf{x}^{(k)} \approx \lambda_1^k \alpha_1 \mathbf{u}_1$ , 计算过程中可能会出现溢出 ( $|\lambda_1| > 1$ ) 或成为 0 ( $|\lambda_1| < 1$ ) 的情形. 为避免这一情形的出现, 实际计算时每次迭代所求得的向量都要归一化. 因此, 幂法实际使用的计算公式是

$$\begin{cases} \mathbf{y}^{(k)} = \frac{\mathbf{x}^{(k)}}{x_r^{(k)}}, \quad |x_r^{(k)}| = \max_{1 \leq i \leq n} |x_i^{(k)}| & (k = 0, 1, 2, \dots) \\ \mathbf{x}^{(k+1)} = A\mathbf{y}^{(k)} \\ \lambda_1 \approx x_r^{(k+1)} \end{cases} \quad (4-5)$$

#### 算法 4.1

1. 输入  $A = (a_{ij})$ , 初始向量  $\mathbf{x} = (x_1, \dots, x_n)$ , 误差限  $\varepsilon$ , 最大迭代次数  $N$ .
2. 置  $k = 1, \mu = 0$ .
3. 求整数  $r$ , 使  $|x_r| = \max_{1 \leq i \leq n} |x_i|$ ,  $x_r \Rightarrow \alpha$ .
4. 计算

$$\mathbf{y} = \frac{\mathbf{x}}{\alpha}$$

$$\mathbf{x} = A\mathbf{y}$$

置  $x_r \Rightarrow \lambda$ .

5. 若  $|\lambda - \mu| < \varepsilon$ , 输出  $\lambda, \mathbf{x}$ , 停机; 否则, 转 6.
6. 若  $k < N$ , 置  $k + 1 \Rightarrow k, \lambda \Rightarrow \mu$ , 转 3; 否则, 输出失败信息, 停机.

例 1 用幂法求矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

的按模最大的特征值和相应的特征向量. 取  $\mathbf{x}^{(0)} = (0, 0, 1)^T$ , 要求误差不超过  $10^{-3}$ .

[解] 由算法 4.1

$$\mathbf{y}^{(0)} = \mathbf{x}^{(0)} = (0, 0, 1)^T$$

$$\mathbf{x}^{(1)} = A\mathbf{y}^{(0)} = (0, -1, 2)^T, \quad \alpha = 2$$

$$\mathbf{y}^{(1)} = \frac{\mathbf{x}^{(1)}}{\alpha} = (0, -0.5, 1)^T$$

$$\mathbf{x}^{(2)} = A\mathbf{y}^{(1)} = (0.5, -2, 2.5)^T, \quad \alpha = 2.5$$

如此继续下去, 计算结果见表 4-1.

表 4-1

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$  | $x_3^{(k)}$ | $\alpha$    | $y_1^{(k)}$ | $y_2^{(k)}$  | $y_3^{(k)}$ |
|-----|-------------|--------------|-------------|-------------|-------------|--------------|-------------|
| 0   | 0           | 0            | 1           | 1           | 0           | 0            | 1           |
| 1   | 0           | -1           | 2           | 2           | 0           | -0.5         | 1           |
| 2   | 0.5         | -2           | 2.5         | 2.5         | 0.2         | -0.8         | 1           |
| 3   | 1.2         | -2.6         | 2.8         | 2.8         | 0.428 571 4 | -0.928 571 4 | 1           |
| 4   | 1.785 714 2 | -2.857 142 8 | 2.928 571 4 | 2.928 571 4 | 0.609 756 0 | -0.975 609 7 | 1           |
| 5   | 2.195 121 8 | -2.951 394   | 2.975 609 7 | 2.975 609 7 | 0.737 704 8 | -0.991 861 9 | 1           |
| 6   | 2.467 271 7 | -2.983 723 8 | 2.991 861 9 | 2.991 861 9 | 0.824 660 9 | -0.997 279 9 | 1           |
| 7   | 2.646 601 8 | -2.994 559 8 | 2.997 279 9 | 2.997 279 9 | 0.883 001 2 | -0.999 092 4 | 1           |
| 8   | 2.765 094 8 | -2.998 184 8 | 2.999 092 4 | 2.999 092 4 | 0.921 977 2 | -0.999 697 3 | 1           |
| 9   | 2.843 651 7 | -2.999 394 6 | 2.999 697 3 | 2.999 697 3 |             |              |             |

因为

$$2.999\ 697\ 3 - 2.999\ 092\ 4 = 0.000\ 604\ 9 < 10^{-3}$$

所以  $\lambda_1 \approx 2.999\ 697\ 3$ , 相应的特征向量为  $\mathbf{u} \approx (2.843\ 651\ 7, -2.999\ 394\ 6, 2.999\ 697\ 3)^T$ , 事实上,  $A$  的特征值为  $\lambda_1 = 3, \lambda_2 = 2, \lambda_3 = 1$ , 与  $\lambda_1$  相应的特征向量为  $(1, -1, 1)^T$ .

幂法的收敛速度依赖于比值  $\left| \frac{\lambda_2}{\lambda_1} \right|$ , 称其为收敛因子, 比值越小, 收敛越快.

例 1 中, 收敛因子  $\left| \frac{\lambda_2}{\lambda_1} \right| = \frac{2}{3}$ , 当迭代进行 9 次时, 所求特征值之误差不超过



$10^{-3}$ . 读者可以用幂法自行计算矩阵

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{bmatrix}$$

的按模最大的特征值, 取  $\mathbf{x}^{(0)} = (1, 1, 1)^T$ , 计算结果将表明, 只要迭代 5 次即可求得误差不超过  $10^{-3}$  的近似值. 这是因为该矩阵的收敛因子  $\left| \frac{\lambda_2}{\lambda_1} \right| \approx 0.065$ , 因而收敛很快.

前面的讨论中, 我们假定  $|\lambda_1| > |\lambda_2|$ . 如果按模最大的特征值有多个, 即矩阵  $A$  的特征值分布满足

$$|\lambda_1| = |\lambda_2| = \cdots = |\lambda_m| > |\lambda_{m+1}| \geq \cdots \geq |\lambda_n| \quad (m \geq 2)$$

幂法是否仍然有效? 下面就几种特殊情况进行讨论:

(1)  $\lambda_1$  是  $m$  重根, 即  $\lambda_1 = \lambda_2 = \cdots = \lambda_m$ , 矩阵  $A$  仍有  $n$  个线性无关的特征向量. 此时, 式(4-2)变成

$$\mathbf{x}^{(k+1)} = \lambda_1^{k+1} \left[ \alpha_1 \mathbf{u}_1 + \cdots + \alpha_m \mathbf{u}_m + \left( \frac{\lambda_{m+1}}{\lambda_1} \right)^{k+1} \alpha_{m+1} \mathbf{u}_{m+1} + \cdots + \left( \frac{\lambda_n}{\lambda_1} \right)^{k+1} \alpha_n \mathbf{u}_n \right] \quad (4-6)$$

显然, 只要  $\alpha_1, \cdots, \alpha_m$  不全为零, 当  $k$  充分大时, 就有

$$\mathbf{x}^{(k+1)} \approx \lambda_1^{k+1} (\alpha_1 \mathbf{u}_1 + \cdots + \alpha_m \mathbf{u}_m)$$

因为  $\alpha_1 \mathbf{u}_1 + \cdots + \alpha_m \mathbf{u}_m$  也是矩阵  $A$  的相应于  $\lambda_1$  的特征向量, 因此仍有

$$\lambda_1 = \lambda_2 = \cdots = \lambda_m \approx \frac{x_i^{(k+1)}}{x_i^{(k)}}$$

$\mathbf{x}^{(k+1)}$  为相应的特征向量, 即对这种情况幂法仍然有效.

(2)  $\lambda_1 = -\lambda_2$ ,  $|\lambda_1| > |\lambda_3|$ , 且矩阵  $A$  有  $n$  个线性无关的特征向量. 式(4-2)变成

$$\mathbf{x}^{(k+1)} = \lambda_1^{k+1} \left[ \alpha_1 \mathbf{u}_1 + (-1)^{k+1} \alpha_2 \mathbf{u}_2 + \left( \frac{\lambda_3}{\lambda_1} \right)^{k+1} \alpha_3 \mathbf{u}_3 + \cdots + \left( \frac{\lambda_n}{\lambda_1} \right)^{k+1} \alpha_n \mathbf{u}_n \right] \quad (4-7)$$

分析上式可知,  $\{\mathbf{x}^{(k+1)}\}$  是个摆动序列, 当  $k$  充分大时, 有

$$\mathbf{x}^{(2k-1)} \approx \lambda_1^{2k-1} (\alpha_1 \mathbf{u}_1 - \alpha_2 \mathbf{u}_2)$$

$$\mathbf{x}^{(2k)} \approx \lambda_1^{2k} (\alpha_1 \mathbf{u}_1 + \alpha_2 \mathbf{u}_2)$$

因此有

$$\lambda_1^2 \approx \frac{x_i^{(k+2)}}{x_i^{(k)}}$$

即

$$\lambda_{1,2} \approx \pm \sqrt{x_i^{(k+2)} / x_i^{(k)}} \quad (4-8)$$

又由  $\mathbf{x}^{(k+1)} \approx \lambda_1^{k+1} [\alpha_1 \mathbf{u}_1 + (-1)^{k+1} \alpha_2 \mathbf{u}_2]$ ,  $\mathbf{x}^{(k)} \approx \lambda_1^k [\alpha_1 \mathbf{u}_1 + (-1)^k \alpha_2 \mathbf{u}_2]$  可导出

$$\begin{cases} \mathbf{x}^{(k+1)} + \lambda_1 \mathbf{x}^{(k)} \approx 2\lambda_1^{k+1} \alpha_1 \mathbf{u}_1 \\ \mathbf{x}^{(k+1)} - \lambda_1 \mathbf{x}^{(k)} \approx 2\lambda_1^{k+1} (-1)^{k+1} \alpha_2 \mathbf{u}_2 \end{cases} \quad (4-9)$$

故在这种情况下,仍可按幂法产生向量序列,式(4-8),式(4-9)为最大模特征值  $\lambda_1, \lambda_2$  与相应的特征向量的计算公式.

例如,矩阵  $A = \begin{bmatrix} 4 & -1 & 1 \\ 16 & -2 & -2 \\ 16 & -3 & -1 \end{bmatrix}$ , 若取  $\mathbf{x}^{(0)} = (1, 0, 0)^T$ , 用幂法计算得

$$\mathbf{x}^{(0)} = (1, 0, 0), \quad \mathbf{x}^{(1)} = (4, 16, 16)$$

$$\mathbf{x}^{(2)} = (16, 0, 0), \quad \mathbf{x}^{(3)} = (64, 256, 256)$$

$$\mathbf{x}^{(4)} = (256, 0, 0), \quad \mathbf{x}^{(5)} = (1024, 4096, 4096)$$

$$\mathbf{x}^{(6)} = (4096, 0, 0), \quad \mathbf{x}^{(7)} = (16384, 65536, 65536)$$

显然有  $\{\mathbf{x}^{(2k)}\}$  和  $\{\mathbf{x}^{(2k+1)}\}$  两个摆动的子列, 且  $\frac{x_i^{(k+2)}}{x_i^{(k)}} = 16 = \lambda_1^2$ . 事实上  $A$  有特征

值  $\lambda = 4, -4$  或  $1$ .

综上所述, 当  $A$  的特征值分布为  $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$  或  $\lambda_1 = \lambda_2 = \dots = \lambda_m, |\lambda_1| > |\lambda_{m+1}| \geq \dots \geq |\lambda_n|$  时, 用幂法可以计算出  $\lambda_1$  及相应的特征向量. 如果按式(4-1)迭代所得向量序列  $\{\mathbf{x}^{(k)}\}$  呈有规律的摆动, 则可能为  $\lambda_1 = -\lambda_2$  的情形. 否则, 应考虑用别的方法求解. 此外, 当矩阵  $A$  无  $n$  个线性无关的特征量时, 幂法收敛很慢, 亦应考虑改用其他方法.

幂法计算简便易行, 它是求大型稀疏矩阵按模最大特征值的常用方法.

### 4.1.2 幂法的加速

因为幂法的收敛速度是线性的, 而且依赖于收敛因子  $|\lambda_2/\lambda_1|$ , 当收敛因子接近于 1 时, 幂法收敛很慢. 幂法的加速有许多方法, 下面介绍其中的两种.

#### (一) 原点移位法

容易看出, 矩阵  $A$  与  $A - \lambda_0 I$  的特征值有以下关系: 若  $\lambda_i$  是  $A$  的特征值, 则  $\lambda_i - \lambda_0$  就是  $A - \lambda_0 I$  的特征值, 而且相应的特征向量不变. 如果对矩阵  $A - \lambda_0 I$  按式(4-1)计算, 则有

$$\begin{aligned} \mathbf{x}^{(k+1)} &= (A - \lambda_0 I) \mathbf{x}^{(k)} \\ &= (\lambda_1 - \lambda_0)^{k+1} \left[ \alpha_1 \mathbf{u}_1 + \left( \frac{\lambda_2 - \lambda_0}{\lambda_1 - \lambda_0} \right)^{k+1} \alpha_2 \mathbf{u}_2 + \dots + \left( \frac{\lambda_n - \lambda_0}{\lambda_1 - \lambda_0} \right)^{k+1} \alpha_n \mathbf{u}_n \right] \end{aligned}$$

适当地选取  $\lambda_0$ , 使得  $|\lambda_1 - \lambda_0| > |\lambda_i - \lambda_0|$ , 且

$$\left| \frac{\lambda_i - \lambda_0}{\lambda_1 - \lambda_0} \right| < \left| \frac{\lambda_2}{\lambda_1} \right| \quad (i = 2, 3, \dots, n)$$

这样,用幂法计算  $A - \lambda_0 I$  的最大模特征值  $\lambda_1 - \lambda_0$  及相应特征向量的收敛速度比对  $A$  用幂法计算要快. 这种加速收敛的方法称为原点移位法.

原点移位法使用简便,不足之处在于  $\lambda_0$  的选取十分困难,通常需要使用者对特征值的分布有一大概的了解,才能粗略地估计  $\lambda_0$ ,并通过计算不断进行修改. 但在一些简单情形,  $\lambda_0$  是可以估计的. 如当矩阵  $A$  的特征值满足  $\lambda_1 > \lambda_2 \geq \lambda_3 \geq \dots \geq \lambda_n > 0$  (或  $\lambda_1 < \lambda_2 \leq \lambda_3 \leq \dots \leq \lambda_n < 0$ ) 时,取  $\lambda_0 = \frac{1}{2}(\lambda_2 + \lambda_n)$ ,则有

$$|\lambda_i - \lambda_0| \leq |\lambda_2 - \lambda_0| < |\lambda_1 - \lambda_0| \quad (i = 2, 3, \dots, n)$$

且

$$\frac{\lambda_2 - \lambda_0}{\lambda_1 - \lambda_0} = \frac{\lambda_2 - \lambda_n}{2\lambda_1 - \lambda_2 - \lambda_n} = \frac{\lambda_2 - \lambda_n}{\lambda_1 - \lambda_n + \lambda_1 - \lambda_2} < \frac{\lambda_2 - \lambda_n}{\lambda_1 - \lambda_n} < \frac{\lambda_2}{\lambda_1}$$

因此,用原点移位法求  $\lambda_1$  可使收敛速度加快.

**例 2** 取  $\lambda_0 = 2.9$ , 用原点移位法求矩阵

$$A = \begin{bmatrix} -4 & 14 & 0 \\ -5 & 13 & 0 \\ -1 & 0 & 2.8 \end{bmatrix}$$

的按模最大的特征值,要求误差不超过  $10^{-4}$ .

[解]

$$A - \lambda_0 I = \begin{bmatrix} -6.9 & 14 & 0 \\ -5 & 10.1 & 0 \\ -1 & 0 & -0.1 \end{bmatrix}$$

取  $x^{(0)} = (1, 1, 1)^T$ , 按算法 4.1 进行计算,结果见表 4-2.

表 4-2

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$ | $x_3^{(k)}$  | $\alpha_k$  | $y_1^{(k)}$ | $y_2^{(k)}$ | $y_3^{(k)}$  |
|-----|-------------|-------------|--------------|-------------|-------------|-------------|--------------|
| 0   | 1           | 1           | 1            | 1           | 1           | 1           | 1            |
| 1   | 7.1         | 5.1         | -1.1         | 7.1         | 1           | 0.718 309 8 | -0.154 929 5 |
| 2   | 3.156 337 2 | 2.254 929   | -0.984 507 1 | 3.156 337 2 | 1           | 0.714 413 2 | -0.311 914 4 |
| 3   | 3.101 785   | 2.215 573 3 | -0.968 808 6 | 3.101 785   | 1           | 0.714 289 7 | -0.312 339   |
| 4   | 3.100 056 8 | 2.214 326   | -0.968 766 1 | 3.100 056 8 | 1           | 0.714 285 6 | -0.312 499 4 |
| 5   | 3.099 998 4 | 2.214 284 6 | -0.968 750 1 | 3.099 998 4 | 1           |             |              |

$$|\alpha_3 - \alpha_4| = 0.000\,058\,4 < 10^{-4}$$

所以,矩阵  $A$  的按模最大的特征值为

$$\lambda_1 \approx 3.099\,998\,4 + 2.9 = 5.999\,998\,4$$

相应的特征向量为  $x \approx (3.099\,998\,4, 2.214\,284\,6, -0.968\,750\,1)^T$ .

不难求出,  $A$  的特征值为  $\lambda_1 = 6, \lambda_2 = 3, \lambda_3 = 2.8$ , 若对  $A$  直接用幂法, 则收敛因子  $|\lambda_2/\lambda_1| = \frac{1}{2}$ , 而用原点移位法, 则有

$$\left| \frac{\lambda_2 - \lambda_0}{\lambda_1 - \lambda_0} \right| = \frac{0.1}{3.1} = \frac{1}{31}$$

因此收敛速度明显加快.

### (二) 幂法的艾特肯 (Aitken) 加速

如果序列  $\{a_k\}$  线性收敛到  $a$ , 即

$$\lim_{k \rightarrow \infty} \frac{a_{k+1} - a}{a_k - a} = c \neq 0$$

则当  $k$  充分大时, 有

$$\frac{a_{k+2} - a}{a_{k+1} - a} \approx \frac{a_{k+1} - a}{a_k - a}$$

由此可解出

$$a \approx \frac{a_{k+2}a_k - a_{k+1}^2}{a_{k+2} - 2a_{k+1} + a_k} = a_k - \frac{(a_{k+1} - a_k)^2}{a_{k+2} - 2a_{k+1} + a_k} = \hat{a}_k \quad (4-10)$$

序列  $\{\hat{a}_k\}$  比  $\{a_k\}$  更快地收敛到  $a$ , 这就是 Aitken 加速法. 将这一方法用于幂法所产生的序列  $\{a_k\}$ , 可加快幂法的收敛速度.

#### 算法 4.2

(1) 输入  $A = (a_{ij})$ , 初始向量  $x = (x_1, \dots, x_n)$ , 误差限  $\varepsilon$ , 最大迭代次数  $N$ .

(2) 置  $k=1, \alpha_0=0, \alpha_1=0, \lambda_0=1$ .

(3) 求整数  $r$ , 使  $|x_r| = \max_{1 \leq i \leq n} |x_i|$ ,  $x_r \Rightarrow \alpha$ .

(4) 计算

$$y = \frac{x}{\alpha}$$

$$x = Ay$$

置  $x_r \Rightarrow \alpha_2$

(5) 计算

$$\lambda = \alpha_0 - \frac{(\alpha_1 - \alpha_0)^2}{\alpha_2 - 2\alpha_1 + \alpha_0}$$

(6) 若  $|\lambda - \lambda_0| < \varepsilon$ , 输出  $\lambda, x$ , 停机; 否则转 7.

(7) 若  $k < N$ , 置  $\alpha_1 \Rightarrow \alpha_0, \alpha_2 \Rightarrow \alpha_1, \lambda \Rightarrow \lambda_0, k+1 \Rightarrow k$ , 转 3; 否则, 输出失败信息, 停机.

例 3 用 Aitken 加速法求矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

的按模最大的特征值与相应的特征向量. 取  $x^{(0)} = (0, 0, 1)^T$ .

[解] 按算法 4.2 计算, 结果见表 4-3.

表 4-3

| $k$ | $x_1^{(k)}$ | $x_2^{(k)}$  | $x_3^{(k)}$ | $\alpha_k$  | $\lambda^{(k)}$ |
|-----|-------------|--------------|-------------|-------------|-----------------|
| 1   | 0           | -1           | 2           | 2           |                 |
| 2   | 0.5         | -2           | 2.5         | 2.5         |                 |
| 3   | 1.2         | -2.6         | 2.8         | 2.8         | 3.25            |
| 4   | 1.785 714 2 | -2.857 142 8 | 2.928 571 4 | 2.928 571 4 | 3.024 999 9     |
| 5   | 2.195 121 8 | -2.951 394   | 2.975 609 7 | 2.975 609 7 | 3.002 747 2     |
| 6   | 2.467 271 7 | -2.983 723 8 | 2.991 861 9 | 2.991 861 9 | 3.000 441 6     |

与例 1 的计算结果比较, 显然, Aitken 加速法的收敛速度比幂法快.

算法 4.2 是用幂法迭代一步, 加速一次, 通常也采用幂法迭代两步或三步, 加速一次的方法.

### 4.1.3 反幂法

反幂法是计算矩阵按模最小的特征值及特征向量的方法, 也是修正特征值、求相应特征向量的最有效的方法.

设  $A$  为  $n \times n$  阶非奇异矩阵,  $\lambda, u$  为  $A$  的特征值与相应的特征向量, 则  $A^{-1}$  的特征值是  $A$  的特征值的倒数, 而相应的特征向量不变, 即

$$A^{-1}u = \frac{1}{\lambda}u \quad (4-11)$$

因此, 若对矩阵  $A^{-1}$  用幂法, 即可计算出  $A^{-1}$  的按模最大的特征值, 其倒数恰为  $A$  的按模最小的特征值. 这就是反幂法的基本思想.

因为  $A^{-1}$  的计算比较麻烦, 而且往往不能保持矩阵  $A$  的一些好性质 (如稀疏性), 因此, 反幂法在实际运算时以求解方程组

$$Ax^{(k+1)} = x^{(k)} \quad (4-12)$$

代替幂法迭代

$$\mathbf{x}^{(k+1)} = \mathbf{A}^{-1} \mathbf{x}^{(k)}$$

求得  $\mathbf{x}^{(k+1)}$ , 每迭代一次要解一个线性方程组 (4-12). 由于矩阵在迭代过程中不变, 故可对  $\mathbf{A}$  先进行三角分解, 这样, 每次迭代只要解两个三角形方程组.

反幂法计算的主要步骤如下:

1. 对  $\mathbf{A}$  进行三角分解  $\mathbf{A} = \mathbf{LU}$ .

2. 求整数  $r$ , 使得  $|x_r^{(k)}| = \max_{1 \leq i \leq n} |x_i^{(k)}|$ ,  $\alpha = x_r^{(k)}$ , 计算

$$\mathbf{y}^{(k)} = \frac{\mathbf{x}^{(k)}}{\alpha}$$

3. 解方程组

$$\mathbf{Lz} = \mathbf{y}^{(k)}$$

$$\mathbf{Ux}^{(k+1)} = \mathbf{z}$$

$$\lambda_n \approx \frac{1}{x_r^{(k+1)}}$$

反幂法的收敛因子为  $\left| \frac{\lambda_n}{\lambda_{n-1}} \right|$ , 收敛因子越小, 收敛越快.

用带原点移位的反幂法来修正特征值, 并求相应的特征向量是非常有效的. 设已知  $\mathbf{A}$  的一个特征值  $\lambda$  的近似值为  $\lambda^*$ , 因为  $\lambda^*$  接近  $\lambda$ , 一般应有

$$0 < |\lambda - \lambda^*| \ll |\lambda_i - \lambda^*| \quad (\lambda_i \neq \lambda)$$

故  $\lambda - \lambda^*$  是矩阵  $\mathbf{A} - \lambda^* \mathbf{I}$  的按模最小的特征值, 且由上式可知, 比值  $|\lambda - \lambda^*| / |\lambda_i - \lambda^*|$  ( $\lambda_i \neq \lambda$ ) 较小. 因此, 对  $\mathbf{A} - \lambda^* \mathbf{I}$  用反幂法求  $\lambda - \lambda^*$  一般收敛很快, 通常只要经过二、三次迭代就能达到较高的精度.

### 算法 4.3

(1) 输入  $\mathbf{A} = (a_{ij})$ , 近似值  $\lambda^*$ , 初始向量  $\mathbf{x} = (x_1, \dots, x_n)$ , 误差限  $\varepsilon$ , 最大容许迭代次数  $N$ .

(2) 置  $k = 1, \mu = 1$ .

(3) 作三角分解  $(\mathbf{A} - \lambda^* \mathbf{I}) = \mathbf{LU}$ .

(4) 求整数  $r$ , 使得  $|x_r| = \max_{1 \leq i \leq n} |x_i|$ ,  $x_r \Rightarrow \alpha$ .

(5) 计算

$$\mathbf{y} = \frac{\mathbf{x}}{\alpha}$$

$$\mathbf{Lz} = \mathbf{y}$$

$$\mathbf{Ux} = \mathbf{z}$$

$$\text{置 } x_r \Rightarrow \beta$$

(6) 若  $\left| \frac{1}{\beta} - \frac{1}{\mu} \right| < \varepsilon$ , 则置  $\lambda = \lambda^* + \frac{1}{\beta}$ , 输出  $\lambda, x$ , 停机; 否则转 7.

(7) 若  $k < N$ , 则置  $k+1 \Rightarrow k, \beta \Rightarrow \mu$ , 转 4; 否则输出失败信息, 停机.

说明: 如果  $\lambda^* = 0$ , 则算法 4.3 求出  $A$  的按模最小的特征值与特征向量.

例 4 用反幂法求矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

接近 2.93 的特征值, 并求相应的特征向量, 取  $x^{(0)} = (0, 0, 1)^T$ .

[解] 对  $A - 2.93I$  作三角分解得

$$\begin{aligned} A - 2.93I &= \begin{bmatrix} -0.93 & -1 & 0 \\ 0 & -0.93 & -1 \\ 0 & -1 & -0.93 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & \frac{1}{0.93} & 1 \end{bmatrix} \begin{bmatrix} -0.93 & -1 & 0 \\ 0 & -0.93 & -1 \\ 0 & 0 & -0.93 + \frac{1}{0.93} \end{bmatrix} \end{aligned}$$

按算法 4.3 计算的数值结果见表 4-4.

表 4-4

| $k$ | $z_1^{(k)}$ | $z_2^{(k)}$  | $z_3^{(k)}$ | $x_1^{(k)}$ | $x_2^{(k)}$  | $x_3^{(k)}$ | $\alpha_k$  | $\lambda$   |
|-----|-------------|--------------|-------------|-------------|--------------|-------------|-------------|-------------|
| 1   | 0           | 0            | 1           | 7.959 058 6 | -7.401 924 5 | 6.883 789 8 | 7.959 059 5 | 3.075 268 8 |
| 2   | 1           | -0.93        | 1.864 9     | 12.692 313  | -12.803 851  | 12.837 581  | 12.837 581  | 3.008 787 8 |
| 3   | 0.988 684 1 | -0.997 372 5 | 2.072 443 6 | 14.278 436  | -14.267 629  | 14.266 268  | 14.278 436  | 3.000 095 4 |

迭代 3 次, 即得  $\lambda \approx 3.000\ 095\ 4$ , 与准确值  $\lambda = 3$  的误差小于  $10^{-4}$ , 相应的特征向量为

$$u \approx \frac{x^{(3)}}{\alpha_3} = (1, -0.999\ 243\ 1, 0.999\ 147\ 8)^T$$

与准确值  $u = (1, -1, 1)^T$  比较, 残量为  $\|r\|_{\infty} < 0.001$ .

## § 4.2 雅可比 (Jacobi) 方法

Jacobi 方法是求实对称矩阵的全部特征值及相应的特征向量的一种方法,

它基于以下两个结论:

(1) 任意实对称矩阵  $A$  可以通过正交相似变换化成对角型, 即存在正交矩阵  $Q$ , 使得

$$Q^T A Q = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

其中  $\lambda_i (i=1, 2, \dots, n)$  是  $A$  的特征值,  $Q$  中各列即为相应的特征向量.

(2) 在正交相似变换下, 矩阵元素的平方和不变. 设  $A = (a_{ij})_{n \times n}$ ,  $Q$  为正交矩阵, 记  $B = Q^T A Q = (b_{ij})_{n \times n}$ , 则

$$\sum_{i,j=1}^n a_{ij}^2 = \sum_{i,j=1}^n b_{ij}^2 \quad (4-13)$$

Jacobi 方法的基本思想是通过一次正交变换, 将  $A$  中一对非零的非对角元素化成零, 并且使得非对角元素的平方和减小. 反复进行上述过程, 使变换后的矩阵的非对角元素的平方和趋于零, 从而使该矩阵近似为对角矩阵, 得到全部特征值和特征向量.

### 4.2.1 矩阵的旋转变换

设  $A$  为  $n$  阶实对称矩阵, 考虑矩阵

$$V_{ij}(\varphi) = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & \cos \varphi & \cdots & \sin \varphi \\ & & & \vdots & 1 & \\ & & & & \ddots & \\ & & & & & 1 \\ & & & -\sin \varphi & \cdots & \cos \varphi \\ & & & & & & 1 \\ & & & & & & & \ddots \\ & & & & & & & & 1 \end{bmatrix} \quad \begin{matrix} \\ \\ \\ i \\ \\ \\ j \\ \\ \end{matrix} \quad (4-14)$$

它是将  $n$  阶单位阵中对角线上第  $i$  和第  $j$  个元换成  $\cos \varphi$ , 非对角元  $v_{ij}$  和  $v_{ji}$  分别换成  $\sin \varphi$  和  $-\sin \varphi$  而得到的, 容易验证,  $V_{ij}(\varphi)$  是正交矩阵, 若记

$$A^{(1)} = V_{ij} A V_{ij}^T = (a_{ij}^{(1)})$$

则有



$$\begin{cases} a_{ii}^{(1)} = a_{ii} \cos^2 \varphi + a_{jj} \sin^2 \varphi + a_{ij} \sin 2\varphi \\ a_{jj}^{(1)} = a_{ii} \sin^2 \varphi + a_{jj} \cos^2 \varphi - a_{ij} \sin 2\varphi \\ a_{ik}^{(1)} = a_{ki}^{(1)} = a_{ik} \cos \varphi + a_{jk} \sin \varphi & (k \neq i, j) \\ a_{jk}^{(1)} = a_{kj}^{(1)} = -a_{ik} \sin \varphi + a_{jk} \cos \varphi \\ a_{kl}^{(1)} = a_{lk}^{(1)} = a_{kl} & (k, l \neq i, j) \\ a_{ij}^{(1)} = a_{ji}^{(1)} = \frac{1}{2}(a_{jj} - a_{ii}) \sin 2\varphi + a_{ij} \cos 2\varphi \end{cases} \quad (4-15)$$

如果  $a_{ij} \neq 0$ , 取  $\varphi$  使得

$$\tan 2\varphi = 2a_{ij}/(a_{ii} - a_{jj}) \left( |\varphi| < \frac{\pi}{4} \right)$$

则有  $a_{ij}^{(1)} = a_{ji}^{(1)} = 0$ , 这样, 就得到一个使  $A$  中非零的非对角元素  $a_{ij} = a_{ji}$  变成零的正交相似变换. 从几何上看,  $V_{ij}$  相当于将  $n$  维空间中的  $Ox_i$  轴与  $Ox_j$  轴在  $x_i Ox_j$  平面中作一旋转, 故  $V_{ij}$  称为旋转变换矩阵.

对  $A^{(1)}$  重复上述过程, 可得  $A^{(2)}$ , 如此继续下去, 得到一个矩阵序列  $\{A^{(k)}\}$ . 可以证明, 虽然这种变换不一定能使矩阵中非对角元中零元素的个数单调增加, 但可以保证非对角元的平方和递减, 我们以  $A$  与  $A^{(1)}$  为例进行讨论.

$$\text{设 } E(A) = \sum_{k \neq l} a_{kl}^2, E(A^{(1)}) = \sum_{k \neq l} a_{kl}^{(1)2}, \text{ 由式(4-15)}$$

$$\begin{aligned} a_{kl} &= a_{kl}^{(1)} & (k, l \neq i, j) \\ a_{ki}^{(1)} &= a_{ik}^{(1)} = a_{ik} \cos \varphi + a_{jk} \sin \varphi \\ a_{jk}^{(1)} &= a_{kj}^{(1)} = -a_{ik} \sin \varphi + a_{jk} \cos \varphi \end{aligned}$$

可得

$$\begin{aligned} E(A^{(1)}) &= \sum_{k \neq l \neq i, j} (a_{kl}^{(1)})^2 + 2 \sum_{k \neq i, j} [(a_{ik}^{(1)})^2 + (a_{jk}^{(1)})^2] \\ &= \sum_{k \neq l \neq i, j} a_{kl}^2 + 2 \sum_{k \neq i, j} (a_{ik}^2 + a_{jk}^2) \\ &= E(A) - 2a_{ij}^2 < E(A) \end{aligned} \quad (4-16)$$

式(4-16)表明, 在上述旋转变换下, 非对角元的平方和严格单调递减, 因而由式(4-13)即得, 对角元的平方和单调递增. 正是利用这一点, 导出了 Jacobi 方法.

## 4.2.2 Jacobi 方法

通过一系列旋转相似变换将  $A$  变成  $A^{(k+1)}$ , 求得  $A$  的全部特征值与特征向量的方法称为 Jacobi 方法. 如果在对  $A$  作相似变换的过程中, 每一步都选绝对值最大的非对角元素  $a_{ij}^{(k)}$ , 以此确定旋转矩阵, 这种方法称为古典 Jacobi 方法. 其计算过程如下:

(1) 令  $k=0, A^{(k)} = A$ .

(2) 求整数  $i, j$ , 使得  $|a_{ij}^{(k)}| = \max_{1 \leq l, m \leq n, l \neq m} |a_{lm}^{(k)}|$ .

(3) 计算旋转矩阵

$$a = \cot 2\varphi = \frac{a_{ii}^{(k)} - a_{jj}^{(k)}}{2a_{ij}^{(k)}}$$

$$b = \tan \varphi = \operatorname{sign}(a) (\sqrt{a^2 + 1} - |a|)$$

$$c = \cos \varphi = \frac{1}{\sqrt{1 + b^2}}, d = \sin \varphi = bc$$

$$V^{(k)} = V_{ij}(\varphi)$$

(4) 计算  $A^{(k+1)}$

$$a_{ii}^{(k+1)} = c^2 a_{ii}^{(k)} + d^2 a_{jj}^{(k)} + 2cd a_{ij}^{(k)}$$

$$a_{jj}^{(k+1)} = d^2 a_{ii}^{(k)} + c^2 a_{jj}^{(k)} - 2cd a_{ij}^{(k)}$$

$$a_{il}^{(k+1)} = a_{li}^{(k+1)} = c a_{il}^{(k)} + d a_{jl}^{(k)} \quad (l = 1, 2, \dots, n, l \neq i, j)$$

$$a_{jl}^{(k+1)} = a_{lj}^{(k+1)} = -d a_{il}^{(k)} + c a_{jl}^{(k)}$$

$$a_{lm}^{(k+1)} = a_{ml}^{(k+1)} = a_{lm}^{(k)} \quad (l, m = 1, 2, \dots, n, l, m \neq i, j)$$

$$\text{置 } a_{ij}^{(k+1)} = a_{ji}^{(k+1)} = 0$$

(5) 计算  $E(A^{(k+1)}) = \sum_{l \neq m} (a_{lm}^{(k+1)})^2$ .

(6) 若  $E(A^{(k+1)}) < \varepsilon$ , 则  $a_{11}^{(k+1)}, a_{22}^{(k+1)}, \dots, a_{nn}^{(k+1)}$  为特征值,  $Q = V^{(0)} V^{(1)} \dots V^{(k)}$  的各列为相应的特征向量; 否则  $k+1 \Rightarrow k$  返回 2, 重复上述过程.

一般地, Jacobi 法不能在有限步内将  $A$  化成对角阵, 但有以下收敛性结果.

**定理 4.1** 设  $A$  为  $n$  阶实对称阵, 对  $A$  用古典 Jacobi 法得到序列  $\{A^{(k)}\}$ , 其中  $A^{(0)} = A$ , 则

$$\lim_{k \rightarrow \infty} E(A^{(k)}) = 0$$

即古典 Jacobi 法收敛.

[证明] 由 Jacobi 法计算过程

$$|a_{ij}^{(k)}| = \max_{l \neq m} |a_{lm}^{(k)}|$$

$$\text{故有} \quad (a_{ij}^{(k)})^2 \geq \frac{1}{n(n-1)} E(A^{(k)}) \quad (4-17)$$

另一方面, 由计算  $A^{(k+1)}$  的公式可以得出

$$\begin{cases} (a_{ii}^{(k+1)})^2 + (a_{jj}^{(k+1)})^2 = (a_{ii}^{(k)})^2 + (a_{jj}^{(k)})^2 + 2(a_{ij}^{(k)})^2 \\ (a_{il}^{(k+1)})^2 + (a_{jl}^{(k+1)})^2 = (a_{il}^{(k)})^2 + (a_{jl}^{(k)})^2, \quad l \neq i, j \end{cases}$$

于是有  $E(A^{(k+1)}) = E(A^{(k)}) - 2(a_{ij}^{(k)})^2$ , 代入式(4-17)得

$$E(A^{(k+1)}) \leq \left(1 - \frac{2}{n(n-1)}\right) E(A^{(k)}) \leq \left[1 - \frac{2}{n(n-1)}\right]^{k+1} E(A)$$

因为  $1 - \frac{2}{n(n-1)} < 1$ , 所以

$$\lim_{k \rightarrow \infty} E(A^{(k)}) = \lim_{k \rightarrow \infty} \left[1 - \frac{2}{n(n-1)}\right]^k E(A) = 0$$

定理 4.7 表明, 古典 Jacobi 法是收敛的, 进一步分析还可以得出 Jacobi 法收敛较快. 另外, 这种方法对舍入误差有较强的稳定性, 因而解的精度高, 且所求得特征向量正交性很好. 它的不足之处是运算量大, 且不能保持矩阵的特殊形状 (如稀疏性), 因此 Jacobi 法是求中小型稠密实对称矩阵的全部特征值与特征向量的较好方法.

**例 5** 用 Jacobi 方法求矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

的特征值.

[解] 首先取  $i=1, j=2$ , 因为  $\cot 2\varphi = 0$ , 故有  $\varphi = \frac{\pi}{4}$ , 于是  $\cos \varphi = \sin \varphi = \frac{1}{\sqrt{2}}$ , 由式 (4-14)

$$V^{(0)} = V_{12}(\varphi) = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} A^{(1)} &= V^{(0)} A^{(0)} V^{(0)T} \\ &= \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & -\frac{1}{\sqrt{2}} \\ 0 & 3 & -\frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 2 \end{bmatrix} \end{aligned}$$

再取  $i=1, j=3, \tan 2\varphi = \sqrt{2}, \cos \varphi = 0.888\ 073\ 8, \sin \varphi = 0.459\ 700\ 7$ , 则

$$V^{(1)} = \begin{bmatrix} 0.888\ 07 & 0 & 0.459\ 70 \\ 0 & 1 & 0 \\ -0.459\ 70 & 0 & 0.888\ 07 \end{bmatrix}$$

$$A^{(2)} = V^{(1)} A^{(1)} V^{(1)T} = \begin{bmatrix} 0.633\ 97 & -0.325\ 06 & 0 \\ -0.325\ 06 & 3 & -0.627\ 96 \\ 0 & -0.627\ 96 & 2.366\ 03 \end{bmatrix}$$

下面应取  $i=2, j=3$ , 重复上述过程. 如此继续下去, 可得

$$A^{(5)} = \begin{bmatrix} 0.585\ 79 & 0.002\ 038\ 3 & 0 \\ 0.002\ 038\ 3 & 3.414\ 01 & 0.016\ 758 \\ 0 & 0.016\ 758 & 2.000\ 20 \end{bmatrix}$$

$$\begin{aligned} E(A^{(5)}) &= 2 \times [(0.002\ 038\ 3)^2 + (0.016\ 758)^2] \\ &= 5.699\ 7 \times 10^{-4} \approx 0.000\ 569\ 97 \end{aligned}$$

所以  $A$  的特征值为

$$\lambda_1 \approx 3.414\ 01, \quad \lambda_2 \approx 2.000\ 20, \quad \lambda_3 \approx 0.585\ 79$$

与其准确值

$$\lambda_1 = 2 + \sqrt{2} \approx 3.414\ 213\ 6, \quad \lambda_2 = 2, \quad \lambda_3 = 2 - \sqrt{2} \approx 0.585\ 786\ 4$$

比较, 最大误差为 0.000 203 6

古典 Jacobi 法在计算每个旋转矩阵  $V^{(k)}$  前, 都需要对  $\frac{1}{2}n(n-1)$  个非对角元素进行比较, 从中找出绝对值最大的元素. 为减少运算量, 常用的一种改进方法是: 取定正序列  $\{\varepsilon_k\}$ ,  $\varepsilon_k \rightarrow 0 (k \rightarrow \infty)$ , 以  $\varepsilon_k$  为限, 逐行检查非对角元, 若  $|a_{ij}| < \varepsilon_k$  就跳过, 否则为  $V_{ij}(\varphi)$  消去元  $a_{ij}$  和  $a_{ji}$ , 反复进行上述过程, 直到所有非对角元的绝对值均小于  $\varepsilon_k$ , 再以  $\varepsilon_{k+1}$  为限, 进行第  $k+1$  轮循环消元. 当  $\varepsilon_k$  充分小时, 所得到的矩阵的对角元即为  $A$  的全部特征值.

## § 4.3 QR 方法

### 4.3.1 基本 QR 方法

20 世纪 60 年代出现的 QR 算法是目前计算一般中小型矩阵的全部特征值与特征向量的最有效的方法, MATLAB 中的库函数 eig 就是采用这种方法. 这里仅讨论实矩阵, 并假定矩阵非奇异. 因为否则, 矩阵  $A - \alpha I$  ( $\alpha$  不是  $A$  的特征值) 必定是非奇异的, 而由  $A - \alpha I$  的特征值与特征向量容易得到  $A$  的特征值与特征向量.

因为任一非奇异实矩阵都可以分解成一个正交矩阵  $Q$  和一个上三角形矩阵  $R$  的乘积,而且当  $R$  的对角元符号取定时,分解是唯一的. 基本 QR 方法的基本思想是利用矩阵的 QR 分解,通过迭代格式

$$\begin{cases} A^{(k)} = Q_k R_k \\ A^{(k+1)} = R_k Q_k \end{cases} \quad (k = 1, 2, \dots) \quad (4-18)$$

将  $A = A^{(1)}$  化成相似的上三角形矩阵(或分块上三角形矩阵),从而求出矩阵  $A$  的全部特征值与特征向量.

由式(4-18),  $A = A^{(1)} = Q_1 R_1$ , 即  $Q_1^{-1} A = R_1$ , 于是

$$A^{(2)} = R_1 Q_1 = Q_1^{-1} A Q_1$$

即  $A^{(2)}$  与  $A$  相似. 同理可得,  $A^{(k)} \sim A (k=2, 3, \dots)$ . 故它们有相同的特征值.

可以证明,在一定条件下,基本 QR 方法产生的矩阵序列  $\{A^{(k)}\}$  “基本”收敛于一个上三角形矩阵(或分块上三角形矩阵),即主对角线(或主对角线子块)及其以下元素均收敛,主对角线(或主对角线子块)以上元素可以不收敛. 特别地,如果  $A$  是实对称阵,则  $\{A^{(k)}\}$  “基本”收敛于对角矩阵.

因为上三角形矩阵的主对角元(或分块上三角形矩阵中,主对角子块的特征值)即为该矩阵的特征值,故当  $k$  充分大时,  $A^{(k)}$  的主对角元(或主对角子块的特征值)就可以作为  $A$  的特征值的近似.

基本 QR 方法的主要运算是矩阵作 QR 分解,分解的方法有多种,下面以 Schmit 正交化方法为例说明.

设  $A$  为  $n$  阶非奇异实矩阵,记为

$$A = [a_1 a_2 \cdots a_n]$$

其中  $a_j = (a_{1j}, a_{2j}, \dots, a_{nj})^T (j=1, 2, \dots, n)$ , 取

$$b_1 = a_1 / \|a_1\|, \quad b'_2 = a_2 - \langle a_2, b_1 \rangle b_1$$

显然  $b_1 \perp b'_2$ , 再取  $b_2 = b'_2$ , 则

$$\|b_1\| = \|b_2\| = 1, \quad \langle b_1, b_2 \rangle = 0$$

一般地,取

$$b'_k = a_k - \sum_{i=1}^{k-1} \langle a_k, b_i \rangle b_i \quad (4-19)$$

$$b_k = b'_k / \|b'_k\| \quad (k=2, 3, \dots, n)$$

则向量组  $b_1, b_2, \dots, b_n$  正交, 且  $\|b_k\| = 1 (k=1, 2, \dots, n)$ . 式(4-19)可改写成

$$a_k = \langle a_k, b_1 \rangle b_1 + \cdots + \langle a_k, b_{k-1} \rangle b_{k-1} + \|b'_k\| b_k$$

于是

$$A = [a_1 a_2 \cdots a_n] = [b_1 b_2 \cdots b_n] \begin{bmatrix} \|a_1\| & \langle a_2, b_1 \rangle & \cdots & \langle a_n, b_1 \rangle \\ & \|b'_2\| & \cdots & \langle a_n, b_2 \rangle \\ & & \ddots & \vdots \\ & & & \|b'_{n-1}\| & \langle a_n, b_{n-1} \rangle \\ & & & & \|b'_n\| \end{bmatrix}$$

$$= QR$$

这就是用 Schmidt 正交化方法对矩阵进行 QR 分解的过程.

例 6 用 Schmidt 正交化方法将矩阵

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

进行 QR 分解.

$$[\text{解}] \quad a_1 = (2, -1, 0)^T, a_2 = (-1, 2, -1)^T, a_3 = (0, -1, 2)^T.$$

因而  $b_1 = a_1 / \|a_1\|_2 = \left( \frac{2}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, 0 \right)^T$ , 按式(4-19)计算

$$b'_2 = a_2 - \langle a_2, b_1 \rangle b_1$$

$$= (-1, 2, -1)^T + \frac{4}{\sqrt{5}} \left( \frac{2}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, 0 \right)^T = \left( \frac{3}{5}, \frac{6}{5}, -1 \right)^T$$

$$b_2 = b'_2 / \|b'_2\|_2 = \left( \frac{3}{\sqrt{70}}, \frac{6}{\sqrt{70}}, -\frac{5}{\sqrt{70}} \right)^T$$

$$b'_3 = a_3 - \langle a_3, b_1 \rangle b_1 - \langle a_3, b_2 \rangle b_2$$

$$= (0, -1, 2)^T - \frac{1}{\sqrt{5}} \left( \frac{2}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, 0 \right)^T + \frac{16}{\sqrt{70}} \left( \frac{3}{\sqrt{70}}, \frac{6}{\sqrt{70}}, -\frac{5}{\sqrt{70}} \right)^T$$

$$= \left( \frac{2}{7}, \frac{4}{7}, \frac{6}{7} \right)^T$$

$$b_3 = b'_3 / \|b'_3\|_2 = \left( \frac{1}{\sqrt{14}}, \frac{1}{\sqrt{14}}, \frac{3}{\sqrt{14}} \right)^T$$

所以

$$A = \begin{bmatrix} 2/\sqrt{5} & 3/\sqrt{70} & 1/\sqrt{14} \\ -1/\sqrt{5} & 6/\sqrt{70} & 2/\sqrt{14} \\ 0 & -5/\sqrt{70} & 3/\sqrt{14} \end{bmatrix} \begin{bmatrix} \sqrt{5} & -4/\sqrt{5} & 1/\sqrt{5} \\ 0 & \sqrt{70}/5 & -16/\sqrt{70} \\ 0 & 0 & 2\sqrt{40}/7 \end{bmatrix}$$

基本 QR 方法每次迭代都需作一次 QR 分解与矩阵乘法, 计算量大, 而且收敛速度慢. 因此实际使用的 QR 方法是先用一系列相似变换将  $A$  化成拟上三角形矩阵(称为上 Hessenberg 矩阵), 然后对此矩阵用基本 QR 方法. 因为拟上三角

形矩阵中具有较多的零元素,这样做可望减少运算量.化  $A$  为相似的似上三角形矩阵的方法有多种,本节介绍其中的一种:Householder 变换.

### 4.3.2 豪斯霍尔德 (Householder) 变换

设向量  $w = (w_1, \dots, w_n)^T$  满足  $\|w\|_2 = \sqrt{w_1^2 + \dots + w_n^2} = 1$ , 则称

$$H = I - 2ww^T$$

$$= \begin{bmatrix} 1 - 2w_1^2 & -2w_1w_2 & \cdots & -2w_1w_n \\ -2w_2w_1 & 1 - 2w_2^2 & \cdots & -2w_2w_n \\ \vdots & \vdots & \ddots & \vdots \\ -2w_nw_1 & -2w_nw_2 & \cdots & 1 - 2w_n^2 \end{bmatrix} \quad (4-20)$$

为 Householder 矩阵或反射矩阵. 容易证明, Householder 矩阵具有以下性质:

- (1)  $H$  是实对称的正交矩阵, 即  $H^{-1} = H^T = H$ ;
- (2)  $\det(H) = -1$ ;
- (3)  $H$  仅有两个不等的特征值  $\pm 1$ , 其中  $1$  是  $n-1$  重特征值,  $-1$  是单重特征值,  $w$  为其相应的特征向量;
- (4) 对任意的  $x \in (\text{span}\{w\})^\perp$ ,  $\alpha \in \mathbf{R}$ , 有

$$H(x + \alpha w) = x - \alpha w \quad (4-21)$$

从图 4-1 可以看出,  $x + \alpha w$  与  $H(x + \alpha w)$  关于超平面  $(\text{span}\{w\})^\perp$  对称, 反射变换的名称由此而来.

**定理 4.2** 设  $x, y$  为  $\mathbf{R}^n$  中任意非零向量, 且  $\|y\|_2 = 1$ , 则存在 Householder 矩阵  $H$ , 使得

$$Hx = \pm \|x\|_2 y \quad (4-22)$$

[证明] 由性质 4 容易得出, 对任意  $x \in \mathbf{R}^n$ ,  $w$  与  $x - Hx$  平行. 故要使式 (4-22) 成立, 应取

$$w = \frac{x - (\pm \|x\|_2 y)}{\|x \mp \|x\|_2 y\|_2} \quad (4-23)$$

令  $H = I - 2ww^T$ , 于是

$$\begin{aligned} Hx &= (I - 2ww^T)x \\ &= x - 2 \frac{x \mp \|x\|_2 y}{\|x \mp \|x\|_2 y\|_2^2} (x^T \mp \|x\|_2 y^T)x \end{aligned} \quad (4-24)$$

由 2-范数的定义

$$\begin{aligned} \|x \mp \|x\|_2 y\|_2^2 &= (x \mp \|x\|_2 y)^T (x \mp \|x\|_2 y) \\ &= x^T x \mp 2 \|x\|_2 y^T x + \|x\|_2^2 \\ &= 2(x^T \mp \|x\|_2 y^T)x \end{aligned}$$

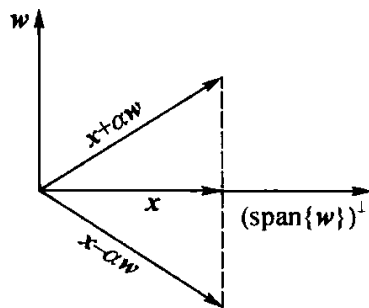


图 4-1

将上式代入式(4-24)即得

$$Hx = \pm \|x\|_2 y$$

定理 4.2 表明,对任一非零向量  $x$ ,都可以构造一个 Householder 变换,它将  $x$  变成事先给定的单位向量的倍数.特别地,若取  $y = e_i$ ,则  $x$  经过 Householder 变换后可变成只有一个分量不为零.

实际计算时,若  $x = (x_1, \dots, x_n)^T \approx e_i$ ,则  $x - \|x\|_2 e_i \text{sign}(x_i) \approx 0$ ,即  $\|x - \|x\|_2 e_i \text{sign}(x_i)\|_2 \ll 1$ ,从而在计算  $w$  时会产生较大的误差,为此我们总取

$$w = \frac{x + \|x\|_2 e_i \text{sign}(x_i)}{\|x + \|x\|_2 e_i \text{sign}(x_i)\|_2} \quad (4-25)$$

使得 Householder 矩阵  $H = I - 2ww^T$  将  $x$  变成与  $e_i$  共线的向量,即

$$Hx = -\text{sign}(x_i) \|x\|_2 e_i$$

### 4.3.3 化一般矩阵为拟上三角形矩阵

称形如

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1,n-1} & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2,n-1} & h_{2n} \\ & h_{32} & h_{33} & \cdots & h_{3,n-1} & h_{3n} \\ & & & \ddots & \vdots & \vdots \\ & & & & h_{n-1,n-1} & \\ & & & & h_{n,n-1} & h_{nn} \end{bmatrix} \quad (4-26)$$

的矩阵为拟上三角形矩阵,也称为上海森堡(Hessenberg)阵.如果次对角线元  $h_{i,i-1} (i=2,3,\dots,n)$  全不为零,则称该矩阵为不可约的上 Hessenberg 矩阵.下面讨论如何利用 Householder 变换将一般矩阵  $A$  相似变换成(4-26)的形式.

首先,选取 Householder 矩阵  $H_1$ ,使得经  $H_1$  相似变换后的矩阵  $H_1 A H_1$  的第 1 列中有尽可能多的零元素.为此,应取  $H_1$  为如下形式

$$H_1 = \left[ \begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & \tilde{H}_1 & \\ 0 & & & \end{array} \right]$$

其中  $\tilde{H}_1$  为  $n-1$  阶 Householder 矩阵.于是有

$$H_1 A H_1 = \begin{bmatrix} a_{11} & a_2^T \tilde{H}_1 \\ \tilde{H}_1 a_1 & \tilde{H}_1 A_{22} \tilde{H}_1 \end{bmatrix}$$



其中

$$\mathbf{a}_1 = (a_{21}, a_{31}, \dots, a_{n1})^T, \mathbf{a}_2 = (a_{12}, a_{13}, \dots, a_{1n})^T$$

$$\mathbf{A}_{22} = \begin{bmatrix} a_{22} & a_{23} & \cdots & a_{2n} \\ a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & & \vdots \\ a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$$

由定理 4.2, 只要取  $\tilde{\mathbf{H}}_1$ , 使得

$$\tilde{\mathbf{H}}_1 \mathbf{a}_1 = \alpha \underbrace{(1, 0, \dots, 0)}_{n-1 \text{ 个元}}^T \quad (4-27)$$

就会使得变换后的矩阵  $\mathbf{H}_1 \mathbf{A} \mathbf{H}_1$  的第 1 列出现  $n-2$  个零元. 完全类似地, 可构造如下形式的 Householder 矩阵

$$\mathbf{H}_2 = \left[ \begin{array}{cc|ccc} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \hline 0 & 0 & & & \\ \vdots & \vdots & & \tilde{\mathbf{H}}_2 & \\ 0 & 0 & & & \end{array} \right]$$

使得

$$\mathbf{H}_2 \mathbf{H}_1 \mathbf{A} \mathbf{H}_1 \mathbf{H}_2 = \begin{bmatrix} * & * & * & \cdots & * \\ * & * & * & \cdots & * \\ 0 & * & * & \cdots & * \\ 0 & 0 & * & \cdots & * \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & * & \cdots & * \end{bmatrix}$$

如此进行  $n-2$  次, 可以构造  $n-2$  个 Householder 矩阵  $\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_{n-2}$  使得

$$\mathbf{H}_{n-2} \cdots \mathbf{H}_2 \mathbf{H}_1 \mathbf{A} \mathbf{H}_1 \mathbf{H}_2 \cdots \mathbf{H}_{n-2} = \mathbf{H}$$

其中  $\mathbf{H}$  为形如式(4-26)的上 Hessenberg 矩阵. 特别地, 如果  $\mathbf{A}$  为实对称矩阵, 则经过上述正交相似变换后所得到的矩阵  $\mathbf{H}$  为三对角阵.

**例 7** 用 Householder 变换将矩阵

$$\mathbf{A} = \begin{bmatrix} 5 & -2 & 2\sqrt{2} & -3\sqrt{2} \\ 1 & 0 & 5\sqrt{2}/2 & -\sqrt{2}/2 \\ 0 & -\sqrt{2} & 1 & 0 \\ 0 & \sqrt{2} & -4 & -1 \end{bmatrix}$$

化成拟上三角形矩阵.

[解] 因为  $\mathbf{a}_1 = (1, 0, 0)^T$ , 由式(4-27)得  $\mathbf{H}_1 = \mathbf{I}$ . 记

$$\mathbf{A}_{22}^{(1)} = \begin{bmatrix} 0 & 5\sqrt{2}/2 & -\sqrt{2}/2 \\ -\sqrt{2} & 1 & 0 \\ \sqrt{2} & -4 & -1 \end{bmatrix}$$

为使 Householder 矩阵  $\tilde{\mathbf{H}}_2$  满足  $\tilde{\mathbf{H}}_2 \begin{bmatrix} -\sqrt{2} \\ \sqrt{2} \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ , 由式(4-25), 取

$$\mathbf{w}' = (-\sqrt{2}, \sqrt{2})^T - 2(1, 0)^T = (-\sqrt{2} - 2, \sqrt{2})^T$$

$$\mathbf{w} = \mathbf{w}' / \|\mathbf{w}'\|_2 = \left( -\frac{\sqrt{2} + \sqrt{2}}{2}, \frac{\sqrt{2}}{2\sqrt{2} + \sqrt{2}} \right)^T$$

$$\tilde{\mathbf{H}}_2 = \mathbf{I} - 2\mathbf{w}\mathbf{w}^T$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 2 \begin{bmatrix} \frac{2 + \sqrt{2}}{4} & -\frac{\sqrt{2}}{4} \\ -\frac{\sqrt{2}}{4} & \frac{2 - \sqrt{2}}{4} \end{bmatrix}$$

$$= \begin{bmatrix} -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$\mathbf{H}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

于是有

$$\mathbf{H} = \mathbf{H}_2 \mathbf{H}_1 \mathbf{A} \mathbf{H}_1 \mathbf{H}_2$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} 5 & -2 & 2\sqrt{2} & -3\sqrt{2} \\ 1 & 0 & 5\sqrt{2}/2 & -\frac{\sqrt{2}}{2} \\ 0 & -\sqrt{2} & 1 & 0 \\ 0 & \sqrt{2} & -4 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$= \begin{bmatrix} 5 & -2 & -5 & -1 \\ 1 & 0 & -3 & 2 \\ 0 & 2 & 2 & -3 \\ 0 & 0 & 1 & -2 \end{bmatrix}$$

#### 4.3.4 拟上三角形矩阵的 QR 分解

因为拟上三角矩阵  $H$  的特殊形状, 通常用  $n-1$  个旋转变换 (又称 Givens 变换) 可以将它化成上三角矩阵, 从而得到  $H$  的 QR 分解式. 具体步骤为:

设  $h_{21} \neq 0$  (否则进行下一步), 取旋转矩阵

$$V_{21} = \begin{bmatrix} \cos \varphi_1 & \sin \varphi_1 & 0 & \cdots & 0 \\ -\sin \varphi_1 & \cos \varphi_1 & 0 & \cdots & 0 \\ & & 1 & \cdots & 0 \\ & & & \ddots & \vdots \\ & & & & 1 \end{bmatrix}$$

其中  $\cos \varphi_1 = \frac{h_{11}}{r_1}$ ,  $\sin \varphi_1 = \frac{h_{21}}{r_1}$ ,  $r_1 = \sqrt{h_{11}^2 + h_{21}^2}$ , 则必有

$$V_{21}H = \begin{bmatrix} r_1 & h_{12}^{(2)} & h_{13}^{(2)} & \cdots & h_{1n-1}^{(2)} & h_{1n}^{(2)} \\ 0 & h_{22}^{(2)} & h_{23}^{(2)} & \cdots & h_{2n-1}^{(2)} & h_{2n}^{(2)} \\ 0 & h_{32}^{(2)} & h_{33}^{(2)} & \cdots & h_{3n-1}^{(2)} & h_{3n}^{(2)} \\ & & \ddots & & \vdots & \vdots \\ & & & & h_{nn-1}^{(2)} & h_{nn}^{(2)} \end{bmatrix} = H^{(2)}$$

设  $h_{32}^{(2)} \neq 0$  (否则进行下一步), 再取

$$V_{32} = \begin{bmatrix} 1 & 0 & 0 & & & \\ & \cos \varphi_2 & \sin \varphi_2 & & & \\ & -\sin \varphi_2 & \cos \varphi_2 & & & \\ & & & 1 & & \\ & & & & \ddots & \\ & & & & & 1 \end{bmatrix}$$

其中  $\cos \varphi_2 = \frac{h_{22}^{(2)}}{r_2}$ ,  $\sin \varphi_2 = \frac{h_{32}^{(2)}}{r_2}$ ,  $r_2 = \sqrt{(h_{22}^{(2)})^2 + (h_{32}^{(2)})^2}$ , 则

$$V_{32}H^{(2)} = \begin{bmatrix} r_1 & h_{12}^{(3)} & h_{13}^{(3)} & \cdots & h_{1\ n-1}^{(3)} & h_{1n}^{(3)} \\ & r_2 & h_{23}^{(3)} & \cdots & h_{2\ n-1}^{(3)} & h_{2n}^{(3)} \\ & & h_{33}^{(3)} & \cdots & h_{3\ n-1}^{(3)} & h_{3n}^{(3)} \\ & & h_{43}^{(3)} & \cdots & h_{4\ n-1}^{(3)} & h_{4n}^{(3)} \\ & & & \ddots & \vdots & \vdots \\ & & & & h_{n\ n-1}^{(3)} & h_{nn}^{(3)} \end{bmatrix} = H^{(3)}$$

假设上述过程已经进行了  $k-1$  步,有

$$H^{(k)} = V_{k\ k-1}H^{(k-1)}$$

$$= \begin{bmatrix} r_1 & \cdots & h_{1\ k-1}^{(k)} & h_{1k}^{(k)} & \cdots & h_{1\ n-1}^{(k)} & h_{1n}^{(k)} \\ & \ddots & & \vdots & & \vdots & \vdots \\ & & r_{k-1} & h_{k-1\ k}^{(k)} & \cdots & h_{k-1\ n-1}^{(k)} & h_{k-1\ n}^{(k)} \\ \hline & & & h_{kk}^{(k)} & \cdots & h_{k\ n-1}^{(k)} & h_{kn}^{(k)} \\ & & & h_{k+1\ k}^{(k)} & \cdots & h_{k+1\ n-1}^{(k)} & h_{k+1\ n}^{(k)} \\ & & & & \ddots & \vdots & \vdots \\ & & & & & h_{n\ n-1}^{(k)} & h_{nn}^{(k)} \end{bmatrix}$$

设  $h_{k+1\ k}^{(k)} \neq 0$ , 取

$$V_{k+1\ k} = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 1 & & & & \\ & & & \cos \varphi_k & \sin \varphi_k & & \\ & & & -\sin \varphi_k & \cos \varphi_k & & \\ & & & & & 1 & \\ & & & & & & \ddots \\ & & & & & & & 1 \end{bmatrix} \begin{matrix} \cdots k \\ \cdots k+1 \end{matrix} \quad (4-28)$$

其中  $\cos \varphi_k = h_{kk}^{(k)} / r_k$ ,  $\sin \varphi_k = h_{k+1\ k}^{(k)} / r_k$ ,  $r_k = \sqrt{(h_{kk}^{(k)})^2 + (h_{k+1\ k}^{(k)})^2}$ , 于是

$$V_{k+1} H^{(k)} = \begin{bmatrix} r_1 & \cdots & h_{1k}^{(k+1)} & h_{1\ k+1}^{(k+1)} & \cdots & h_{1\ n-1}^{(k+1)} & h_{1n}^{(k+1)} \\ & \ddots & \vdots & \vdots & & \vdots & \vdots \\ & & r_k & h_{k\ k+1}^{(k+1)} & \cdots & h_{k\ n-1}^{(k+1)} & h_{kn}^{(k+1)} \\ \cdots & \cdots & \cdots & h_{k+1\ k+1}^{(k+1)} & \cdots & h_{k+1\ n-1}^{(k+1)} & h_{k+1\ n}^{(k+1)} \\ & & & h_{k+2\ k+1}^{(k+1)} & \cdots & h_{k+2\ n-1}^{(k+1)} & h_{k+2\ n}^{(k+1)} \\ & & & & \ddots & \vdots & \vdots \\ & & & & & h_{n\ n-1}^{(k+1)} & h_{nn}^{(k+1)} \end{bmatrix} = H^{(k+1)}$$

因此,最多作  $n-1$  次旋转变换,即得

$$\begin{aligned} H^{(n)} &= V_{n\ n-1} V_{n-1\ n-2} \cdots V_{21} H \\ &= \begin{bmatrix} r_1 & h_{12}^{(n)} & h_{13}^{(n)} & \cdots & h_{1n}^{(n)} \\ & r_2 & h_{23}^{(n)} & \cdots & h_{2n}^{(n)} \\ & & \ddots & & \vdots \\ & & & & r_n \end{bmatrix} = R \end{aligned}$$

因为  $V_{ii-1}$  ( $i=2,3,\cdots,n$ ) 均为正交矩阵,故

$$H = V_{21}^T V_{32}^T \cdots V_{n\ n-1}^T R = QR$$

其中  $Q = V_{21}^T V_{32}^T \cdots V_{n\ n-1}^T$  仍为正交矩阵. 容易算出完成这一过程的运算量约为  $4n^2$ , 比一般矩阵的 QR 分解的运算量  $O(n^3)$  少了一个数量级.

不难证明,  $\tilde{H} = QR$  仍是拟上三角矩阵,于是可以按上述步骤一直迭代下去,这样得到的 QR 方法的运算量比基本 QR 方法大为减少. 需要说明的是,我们通常用 QR 方法计算特征值,然后用反幂法求其相应的特征向量.

**例 8** 用 QR 方法求矩阵

$$A = \begin{bmatrix} 5 & -3 & 2 \\ 6 & -4 & 4 \\ 4 & -4 & 5 \end{bmatrix}$$

的全部特征值.

[解] 首先将  $A$  化成拟上三角形矩阵. 取

$$w' = (6, 4)^T + \sqrt{6^2 + 4^2} (1, 0)^T = (6 + 2\sqrt{13}, 4)^T$$

$$w = w' / \|w'\|_2 = (0.957\ 092, 0.289\ 784)^T$$

$$I - 2ww^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 2 \begin{bmatrix} 0.916\ 025 & 0.277\ 350 \\ 0.277\ 350 & 0.083\ 974\ 7 \end{bmatrix}$$

$$= \begin{bmatrix} -0.832\ 050 & -0.554\ 700 \\ -0.554\ 700 & 0.832\ 050 \end{bmatrix}$$

于是

$$\begin{aligned} H_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & -0.832\ 050 & -0.554\ 700 \\ 0 & -0.554\ 700 & 0.832\ 050 \end{bmatrix} \\ H &= H_1 A H_1 = \begin{bmatrix} 5 & 1.386\ 750 & 3.328\ 200 \\ -7.211\ 102 & -1.230\ 768 & -8.153\ 840 \\ 0 & -0.153\ 846 & 2.230\ 767 \end{bmatrix} \end{aligned}$$

$H$  即为与  $A$  相似的拟上三角形矩阵. 下面将  $H$  进行 QR 分解, 记  $H^{(1)} = H$ , 按式 (4-28)

$$\begin{aligned} r_1 &= \sqrt{5^2 + (-7.211\ 102)^2} = 8.774\ 964 \\ \cos \varphi_1 &= \frac{5}{r_1} = 0.569\ 803, \quad \sin \varphi_1 = -0.821\ 781 \end{aligned}$$

取

$$V_{21} = \begin{bmatrix} 0.569\ 803 & -0.821\ 781 & 0 \\ 0.821\ 781 & 0.569\ 803 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

于是

$$V_{21} H^{(1)} = \begin{bmatrix} 8.774\ 964 & 1.801\ 596 & 8.597\ 089 \\ 0 & 0.438\ 310 & -1.911\ 030 \\ 0 & -0.153\ 846 & 2.230\ 767 \end{bmatrix}$$

再取

$$\begin{aligned} r_2 &= \sqrt{(0.438\ 310)^2 + (-0.153\ 846)^2} = 0.464\ 526 \\ \cos \varphi_2 &= \frac{0.438\ 310}{r_2} = 0.943\ 564, \quad \sin \varphi_2 = \frac{-0.153\ 846}{r_2} = -0.331\ 189 \end{aligned}$$

$$V_{32} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.943\ 564 & -0.331\ 189 \\ 0 & 0.331\ 189 & 0.943\ 564 \end{bmatrix}$$

于是

$$V_{32} V_{21} H^{(1)} = \begin{bmatrix} 8.774\ 964 & 1.801\ 596 & 8.597\ 089 \\ 0 & 0.464\ 526 & -2.541\ 982 \\ 0 & 0 & 1.471\ 953 \end{bmatrix} = R_1$$

$$Q_1 = V_{21}^T V_{32}^T = \begin{bmatrix} 0.569\ 803 & 0.775\ 403 & 0.272\ 165 \\ -0.821\ 781 & 0.537\ 643 & 0.188\ 712 \\ 0 & -0.331\ 189 & 0.943\ 564 \end{bmatrix}$$

第一次迭代得

$$H^{(2)} = R_1 Q_1 = \begin{bmatrix} 3.519\ 482 & 4.925\ 491 & 10.840\ 117 \\ -0.381\ 739 & 1.091\ 627 & -2.310\ 653 \\ 0 & -0.487\ 495 & 1.388\ 883 \end{bmatrix}$$

重复上述过程,迭代 11 次得

$$H^{(12)} = \begin{bmatrix} 2.992\ 032 & -1.003\ 853 & 12.013\ 392 \\ -0.007\ 496 & 2.004\ 695 & 1.941\ 971 \\ 0 & -0.000\ 325 & 0.999\ 895 \end{bmatrix}$$

所以  $A$  的特征值为

$$\lambda_1 \approx 2.992\ 032, \quad \lambda_2 \approx 2.004\ 695, \quad \lambda_3 \approx 0.999\ 895$$

(准确值为  $\lambda_1 = 3, \lambda_2 = 2, \lambda_3 = 1$ ), 此时,  $H^{(12)}$  的下三角非对角元的最大模为 0.007 496, 故 QR 方法“基本”收敛速度较慢.

#### 4.3.5 带原点移位的 QR 方法

理论分析和实际计算均表明, QR 方法产生的矩阵序列  $\{H^{(k)}\}$  的右下角对角元素  $h_{nn}^{(k)}$  最先与  $A$  的特征值接近. 可以证明, 若矩阵  $A$  的特征值满足

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_{n-1}| > |\lambda_n|$$

则  $\{H^{(k)}\}$  的右下角对角元  $h_{nn}^{(k)} \rightarrow \lambda_n (k \rightarrow \infty)$ , 且收敛速度是线性的, 速率为  $\left| \frac{\lambda_n}{\lambda_{n-1}} \right|$ . 于是考虑引入原点移位的技巧来加快收敛速度, 即取位移  $\mu$ , 使其满足

$$|\lambda_1 - \mu| \geq |\lambda_2 - \mu| \geq \cdots \geq |\lambda_{n-1} - \mu| > |\lambda_n - \mu|$$

且

$$\left| \frac{\lambda_n - \mu}{\lambda_{n-1} - \mu} \right| \ll 1$$

这样, 对  $(H - \mu I)$  用 QR 方法就可以加快收敛速度, 这就是带原点移位的 QR 方法. 其具体步骤为:

- (1) 用 Householder 变换将矩阵  $A$  化成拟上三角形矩阵  $H$ ;
- (2) 对  $k = 1, 2, \cdots$  取位移  $\mu_k$ , 将  $H^{(k)} - \mu_k I$  进行 QR 分解

$$H^{(k)} - \mu_k I = Q_k R_k$$

- (3)  $H^{(k+1)} = R_k Q_k + \mu_k I$

关于每次迭代时位移  $\mu_k$  的取法及 QR 方法的迭代准则, 可参看[3].

至今为止, QR 方法仍是求解实矩阵特征值问题的最有效的数值方法, 它几

乎可以适用于求所有矩阵的全部特征值和特征向量. 基本 QR 方法收敛较慢, 其收敛速度是线性的. 带原点移位的 QR 方法具有二次收敛性, 它主要用于特征值全为实数的矩阵特征值计算.

如果  $A$  为实对称矩阵, 在用 QR 方法求其特征值时, 充分利用对称性, 又引出了对称 QR 方法. 这是目前求解中小型稠密实对称矩阵的特征值与特征向量的最有效方法.

对步隐式 QR 方法的基本思想是: 先把原点移位量扩充到复数域, 每次迭代作两次原点移位的 QR 变换, 然后把它们结合在一起, 转化成实数运算. 此方法适用于求实矩阵的复数特征值, 计算效率很高, 在某些条件下可以达到三阶收敛性.

MATLAB 软件中提供了有关特征值计算和矩阵分解相关的函数, 例如

```
>> eig(A)           % 仅计算 A 的所有特征值.
>> [V,D] = eig(A)   % 对角阵 D 表示了 A 的所有特征值,
                     % V 的第 j 列为 D 的第 j 行的对角元 d(j) 所对应的特
                     % 征向量.
>> [Q,R] = qr(A)    % 求出 A 的 QR 分解.
>> [P,H] = hess(A)  % H 为 A 的上 Hessenberg 阵, P 为相应的变换阵.
```

## § 4.4 应用实例

### 化学吸收过程

继续考虑第二章的化学吸收过程. 由于在每一步中, 液体吸收某成分的总量满足一阶常微分方程组, 这一微分方程组可用矩阵形式表示为:

$$\frac{dx}{dt} = \frac{1}{\tau} \begin{bmatrix} -(1+\delta) & 1 & 0 & 0 & 0 \\ \delta & -(1+\delta) & 1 & 0 & 0 \\ 0 & \delta & -(1+\delta) & 1 & 0 \\ 0 & 0 & \delta & -(1+\delta) & 1 \\ 0 & 0 & 0 & \delta & -(1+\delta) \end{bmatrix} x + \frac{1}{\tau} \begin{bmatrix} K(b - y_f) \\ 0 \\ 0 \\ 0 \\ -x_f \end{bmatrix}$$

在前面的讨论中, 我们设导数向量  $\frac{dx}{dt} = 0$ , 通过求解线性方程组, 获得稳定态时



的吸收量. 但这一静态分析不能提供关于吸收过程的有价值的信息. 为对整个吸收过程有一个全面的了解, 我们需求解系数矩阵  $A$  的特征值. 设  $D = (\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5)^T$  和  $U = [u_1, u_2, u_3, u_4, u_5]$  分别表示矩阵  $A$  的特征值和特征向量,  $x_s$  表示稳定解. 如果特征值已知, 系统输入  $x_f, y_f$  是常数, 则解可以表示为

$$x(t) = \sum_{k=1}^5 c_k \exp(\lambda_k t) u_k + x_s$$

其中向量  $C = (c_1, c_2, c_3, c_4, c_5)^T$  依赖于初值条件  $x(0)$ . 上述表达式的前五项称为系统的自然模, 是解的瞬时部分. 特别地, 令  $t=0$ , 即得向量  $C$  满足的线性方程组

$$UC = x(0) - x_s$$

设第  $k$  个特征根被表示为

$$\lambda_k = a_k + ib_k \quad (k = 1, 2, 3, 4, 5)$$

$a_k, b_k$  分别是实部和虚部. 当时间增加时, 要使解的瞬时部分趋于零, 必须对于所有的  $k$  有  $a_k < 0$ . 即所有的特征值都必须位于复平面的左半平面内. 进一步地, 系统的解  $x(t)$  趋于  $x_s$  的速度也由特征值确定. 若仅关心解的瞬时部分以怎样的速率衰变为初始值的  $p\%$ . 注意到第  $k$  个自然模以速率  $\exp(a_k)$  趋于零, 并假设解衰变到初值的  $p/100$ , 解得第  $k$  个自然模衰变时间

$$T_k = \frac{\ln(0.01p)}{a_k}, \quad 1 \leq k \leq 5$$

在整个吸收过程中, 解趋于它的稳定解的速度由衰变时间的最大值  $T_{\max} = \{T_k\}$  决定.

设  $\tau = 1.86$  min, 其他参数如上, 并设瞬时衰变比例是  $p = 1\%$ , 利用  $QR$  方法或直接调用库函数, 计算过程和结果如下:

```
>> eig(a)'      % 计算特征值
-2.177 408 33   -0.159 795 97   -0.586 167 63
-1.168 602 15   -1.751 036 67
>> [V D] = eig(a) % 计算特征向量
V =
-0.244 145 595 3 -0.244 145 595 3 -0.419 328 689 5 -0.483 685 113 4
-0.419 328 689 5  0.458 109 788 2 -0.458 109 788 2 -0.454 270 596 6
-0.000 000 000 0  0.454 270 596 6 -0.573 058 541 3 -0.573 058 541 3
-0.000 000 000 0  0.567 652 849 1  0.000 000 000  0.537 637 647 4
-0.537 637 647 4  0.533 131 972 2  0.000 000 000 0 -0.533 131 972 1
```

```

-0.336 270 752 0   0.336 270 752 0   0.577 556 902 4 -0.666 197 383 7
0.577 556 902 4

>> for i = 1:5      % 计算衰变时间
T(i) = log(0.01)/d(i,i);
end;
T = 2.114 977 76   28.819 063 06   7.856 404 78   3.940 751 08
2.629 967 87

```

由于上述特征值全是负的,且第二个特征值的模最小,相应的衰变时间最长,在时刻  $T(2) = 28.82 \text{ min}$  之后,瞬时部分衰变到初始值的 1%. 若给出初始值  $\mathbf{x}(0)$ , 解线性方程组  $UC = \mathbf{x}(0) - \mathbf{x}_s$ , 得向量  $C$ , 从而得解

$$\mathbf{x}(t) = \sum_{k=1}^5 c_k \exp(\lambda_k t) \mathbf{u}_k + \mathbf{x}_s$$

## 评 注

幂法是求矩阵最大模的特征值和相应特征向量的有效而又简单的方法,特别适用于大型矩阵或稀疏矩阵,也是计算矩阵谱半径的有效方法. 但它的收敛速度是线性的,令人不太满意,一般用原点位移法或 Aitken 外推加速技术加速收敛. 反幂法是计算矩阵最小模的特征值及特征向量的有效方法,它的计算过程需要求解线性方程组,一般采用直接三角分解法. 结合原点位移法,反幂法可以修正某一指定的特征值及特征向量. 若知道矩阵特征值的大致分布情况,带位移的反幂法可以求出每一个特征值及其特征向量.

Jacobi 方法是一个经典方法,大多数情况下它与对称矩阵的 QR 方法相比,没有多大的优势,但当矩阵几乎接近对角形时,仍是很有效的方法,而且它便于并行计算,近来又引起人们的关注.

QR 方法是至今最有效的求矩阵特征值的方法,它可以求出全部特征值与特征向量. 当矩阵  $A$  是对称矩阵时,可以找到正交阵  $Q$  使得  $Q^T A Q = T$  为三对角阵,使得存储空间和计算量都大为降低,从而 QR 方法的计算公式可以简单得多<sup>[18]</sup>. 一般情况下,QR 算法的收敛速度依赖于特征值的比,且是一阶收敛的. 带原点位移的 QR 算法具有二阶精度,对于对称矩阵,适当选择位移量,可以达到三阶收敛. 此外还有利用实 Schur 分解所获得的不变子空间,求矩阵特征值和特征向量的方法<sup>[19]</sup>.

## 习 题 四

1. 用幂法计算下列矩阵的按模最大的特征值及相应的特征向量, 取  $x^{(0)} = (1, 0, 0)^T$ :

$$(1) \begin{bmatrix} -4 & 14 & 0 \\ -5 & 13 & 0 \\ -1 & 0 & 2 \end{bmatrix} \quad (2) \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix}$$

2. 用带 Aitken 加速的幂法计算第 1 题中矩阵的最大模特征值及相应的特征向量, 要求结果有四位有效数字.

3. 求矩阵

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{bmatrix}$$

的接近 9.6 的特征值及相应的特征向量.

4. 用反幂法计算矩阵

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{bmatrix}$$

相应于特征值 1.267 9 的特征向量.

5. 已知矩阵

$$A = \begin{bmatrix} 6 & -12 & 6 \\ -21 & -3 & 24 \\ -12 & -12 & 51 \end{bmatrix}$$

(1) 用反幂法求  $A$  的按模最小的特征值, 迭代 2 次, 计算中取 4 位有效数字.

(2) 以 (1) 的计算结果为近似值, 再用反幂法改进这个特征值, 并求相应的特征向量.

6. 设矩阵  $A$  的特征值满足

$$\lambda_1 > \lambda_2 \geq \lambda_3 \geq \cdots \geq \lambda_n$$

求证当取  $\lambda_0 = \frac{\lambda_2 + \lambda_n}{2}$  时, 用原点移位法求  $\lambda_1$  的收敛速度最快.

7. 设  $A$  为  $n$  阶实对称矩阵. 对任意非零向量  $x \in \mathbb{R}^n$ , 称  $\frac{\langle Ax, x \rangle}{\langle x, x \rangle} = \frac{x^T A x}{x^T x}$  为  $A$  的 Rayleigh 商, 求证:

$$\min_{x \neq 0} \frac{\langle Ax, x \rangle}{\langle x, x \rangle} = \min \lambda(A), \quad \max_{x \neq 0} \frac{\langle Ax, x \rangle}{\langle x, x \rangle} = \max \lambda(A)$$

8. 用 Jacobi 方法求下列矩阵的全部特征值:

$$(1) \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 6 \end{bmatrix} \quad (2) \begin{bmatrix} 2 & -1 & 0 \\ -1 & -2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

9. 用 Householder 变换将下列矩阵化成相似的拟上三角形矩阵:

$$(1) \begin{bmatrix} -4 & -3 & -7 \\ 2 & 3 & 2 \\ 4 & 2 & 7 \end{bmatrix} \quad (2) \begin{bmatrix} 4 & 3 & 0 & 4 \\ 3 & 5 & -1 & 2 \\ 0 & -1 & 2 & 1 \\ 4 & 2 & 1 & -1 \end{bmatrix}$$

10. 设  $\lambda$  是实对称矩阵  $A$  的特征值, 相应的特征向量为  $u$ , 且  $\|u\|_2 = 1$ , 若存在正交矩阵  $P$ , 使得  $Pu = e_1 = (1, 0, \dots, 0)^T$ , 求证矩阵  $B = PAP^T$  的第 1 行和第 1 列除  $\lambda$  外, 其余元素均为零.

11. 用 QR 方法求下列矩阵的全部特征值:

$$(1) \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{bmatrix}; \quad (2) \begin{bmatrix} 3 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

## 数值实验

$$1. \text{ 设 } A = \begin{bmatrix} 6 & -4 & 18 \\ 20 & -6 & -6 \\ 22 & -22 & 11 \end{bmatrix}, \text{ 取 } x^{(0)} = (1, 1, 1)^T, \text{ 求 } A \text{ 的按模最大的特征}$$

值和特征向量, 要求误差小于  $10^{-6}$ .

(1) 直接用幂法计算, 出现什么现象, 说明原因;

(2) 用归一化的幂法计算.

$$2. \text{ 设 } A = \begin{bmatrix} 12 & 6 & -6 \\ 6 & 16 & 2 \\ -6 & 2 & 16 \end{bmatrix}, \text{ 取 } x^{(0)} = (1, 1, 1)^T, \text{ 先用幂法迭代 3 次, 得到 } A$$

的按模最大特征值的近似值, 取  $\lambda^*$  为其整数部分, 再用反幂法计算  $A$  的按模最大特征值的更精确的近似值, 要求误差小于  $10^{-10}$ .

3. 根据习题 7 介绍的 Rayleigh 商方法, 对于数值实验题 2 中的矩阵  $A$ , 用归一化的幂法所得向量  $\{x^{(k)}\}$  的 Rayleigh 商作为  $A$  的按模最大特征值的近似值, 将计算结果与归一化的幂法得到的结果进行比较, 并说明原因.

## 4. 已知矩阵

$$\begin{bmatrix} 190 & 66 & -84 & 30 \\ 66 & 303 & 42 & -36 \\ 336 & -168 & 147 & -112 \\ 30 & -36 & 28 & 291 \end{bmatrix}$$

- (1) 用幂法求矩阵的按模最大的特征值与特征向量；
- (2) 用反幂法求矩阵的按模最小的特征值与特征向量；
- (3) 用 QR 方法求矩阵的所有特征值与特征向量,并用函数  $[V, D] = \text{eig}(A)$  检验你的结果.

## 第五章 插 值 法

生产实践中常常出现这样的问题:给出一批离散样点,要求作出一条通过这些点的光滑曲线,以便满足设计要求或进行加工.反映在数学上,即已知函数在一些点上的值,寻求它的分析表达式.因为由函数的表格形式不能直接得出表中未列点处的函数值,也不便于研究函数的性质.此外,有些函数虽有表达式,但因式子复杂,不易计算其值和进行理论分析,也需要构造一个简单函数来近似它.

解决这种问题的方法有两类:一类是给出函数  $f(x)$  的一些样点值,选定一个便于计算的函数形式,如多项式、分式线性函数及三角多项式等,要求它通过已知样点,由此确定函数  $\varphi(x)$  作为  $f(x)$  的近似.这就是插值法.另一类方法在选定近似函数的形式后,不要求近似函数过已知样点,只要求在某种意义下它在这些点上的总偏差最小.这类方法称为曲线(数据)拟合法.

本章主要讨论构造插值多项式的几种常用方法及其误差.

### § 5.1 拉格朗日 (Lagrange) 插值

#### 5.1.1 多项式插值

用插值法求函数的近似表达式时,首先要选定近似函数的形式.可供选择的函数很多,常用的是多项式函数.因为多项式函数计算简便,只需用加、减、乘等运算,而且其导数与积分仍为多项式.

用多项式作为研究插值的工具,称为代数插值,其基本问题是:已知函数  $f(x)$  在区间  $[a, b]$  上  $n+1$  个不同点  $x_0, x_1, \dots, x_n$  处的函数值  $y_i = f(x_i)$  ( $i = 0, 1, \dots, n$ ), 求一个至多  $n$  次的多项式

$$\varphi_n(x) = a_0 + a_1x + \dots + a_nx^n \quad (5-1)$$

使其在给定点处与  $f(x)$  同值,即满足插值条件

$$\varphi_n(x_i) = f(x_i) = y_i \quad (i = 0, 1, \dots, n) \quad (5-2)$$

$\varphi_n(x)$  称为插值多项式,  $x_i$  ( $i = 0, 1, \dots, n$ ) 称为插值节点,简称节点,  $[a, b]$  称为插值区间.从几何上看(如图 5-1 所示),  $n$  次多项式插值就是过  $n+1$  个点  $(x_i, f(x_i))$  ( $i = 0, 1, \dots, n$ ), 作一条多项式曲线  $y = \varphi_n(x)$  近似曲线  $y = f(x)$ .

$n$  次多项式(5-1)有  $n+1$  个待定系数,由插值条件式(5-2)恰好给出  $n+1$  个方程

$$\begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0 \\ a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_n x_1^n = y_1 \\ \cdots \cdots \cdots \\ a_0 + a_1 x_n + a_2 x_n^2 + \cdots + a_n x_n^n = y_n \end{cases} \quad (5-3)$$

记此方程组的系数矩阵为  $A$ , 则

$$\det A = \begin{vmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{vmatrix}$$

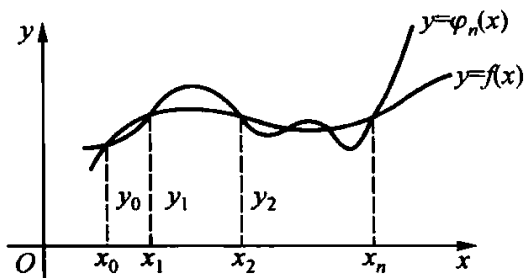


图 5-1

是范德蒙德 (Vandermonde) 行列式. 当  $x_0, x_1, \cdots, x_n$  互不相同时, 此行列式值不为零. 因此, 方程组(5-3)有唯一解. 这表明, 只要  $n+1$  个节点互不相同, 满足插值要求式(5-2)的插值多项式(5-1)是存在唯一的.

### 5.1.2 插值多项式的误差估计

插值多项式与被插函数之间的差

$$R_n(x) = f(x) - \varphi_n(x)$$

称为截断误差, 又称为插值余项.

假定  $f(x)$  在区间  $[a, b]$  上  $n+1$  次连续可导, 对  $[a, b]$  上任意点  $x$ , 且  $x \neq x_i$  ( $i=0, 1, \cdots, n$ ), 构造辅助函数

$$\psi(t) = f(t) - \varphi_n(t) - \frac{R_n(x)}{\omega_{n+1}(x)} \omega_{n+1}(t) \quad (5-4)$$

其中  $\omega_{n+1}(t) = \prod_{j=0}^n (t - x_j)$ . 显然

$$\psi(x) = f(x) - \varphi_n(x) - \frac{R_n(x)}{\omega_{n+1}(x)} \omega_{n+1}(x) = 0$$

又由插值条件式(5-2),  $R_n(x_i) = 0$  ( $i=0, 1, \cdots, n$ ), 故函数  $\psi(t)$  在区间  $[a, b]$  内至少有  $n+2$  个零点  $x, x_0, x_1, \cdots, x_n$ . 由罗尔 (Rolle) 定理, 函数

$$\psi'(t) = f'(t) - \varphi'_n(t) - \frac{R_n(x)}{\omega_{n+1}(x)} \omega'_{n+1}(t)$$

在  $(a, b)$  内至少有  $n+1$  个零点. 反复使用 Rolle 定理, 可以得出

$$\psi^{(n+1)}(t) = f^{(n+1)}(t) - \varphi_n^{(n+1)}(t) - \frac{R_n(x)}{\omega_{n+1}(x)} \omega_{n+1}^{(n+1)}(t)$$

在  $(a, b)$  内至少有一个零点, 设为  $\xi$ , 即

$$\psi^{(n+1)}(\xi) = 0$$

因为  $\varphi_n(t)$  为至多  $n$  次多项式,  $\omega_{n+1}(t)$  为最高次项系数为 1 的  $n+1$  次多项式, 因而

$$\varphi_n^{(n+1)}(t) \equiv 0, \quad \omega_{n+1}^{(n+1)}(t) = (n+1)!$$

于是有

$$\psi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - \frac{R_n(x)}{\omega_{n+1}(x)}(n+1)! = 0$$

所以

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-x_0)(x-x_1)\cdots(x-x_n)$$

当  $x = x_i (i=0, 1, \cdots, n)$  时, 上式自然成立, 因此, 上式对  $[a, b]$  上的任意点都成立. 这就是插值多项式的误差估计.

**定理 5.1** 设  $x_0, x_1, \cdots, x_n$  是区间  $[a, b]$  上的互异节点,  $\varphi_n(x)$  是过这组节点的  $n$  次插值多项式. 如果  $f(x)$  在  $[a, b]$  上  $n+1$  次连续可导, 则对  $[a, b]$  内任意点  $x$ , 插值余项为

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \quad \xi \in (a, b) \quad (5-5)$$

通过解方程组 (5-3) 来确定插值多项式, 计算量较大. 下面介绍几种常用的确定插值多项式的方法.

### 5.1.3 Lagrange 插值多项式

先讨论只有两个节点  $x_0, x_1 (n=1)$  的插值多项式. 由前所述, 插值多项式应设为  $\varphi_1(x) = a_0 + a_1x$ , 且满足插值条件

$$\varphi_1(x_0) = a_0 + a_1x_0 = y_0 = f(x_0)$$

$$\varphi_1(x_1) = a_0 + a_1x_1 = y_1 = f(x_1)$$

解此方程组得

$$a_0 = \frac{y_1x_0 - y_0x_1}{x_0 - x_1}, \quad a_1 = \frac{y_0 - y_1}{x_0 - x_1}$$

所以, 两个节点的一次插值多项式为

$$\varphi_1(x) = \frac{y_1x_0 - y_0x_1}{x_0 - x_1} + \frac{y_0 - y_1}{x_0 - x_1}x \quad (5-6)$$

如图 5-2 所示, 这是用过两点  $(x_0, y_0), (x_1, y_1)$  的直线  $y = \varphi_1(x)$  近似曲线  $y = f(x)$ , 故这种插值又称为线性插值.

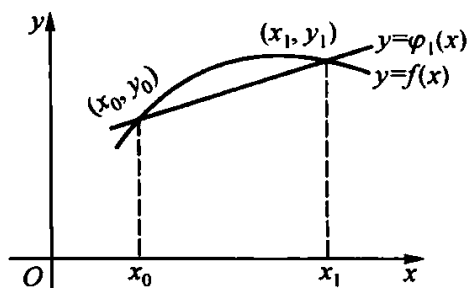


图 5-2



如果将式(5-6)改写成以下形式

$$\varphi_1(x) = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0} \quad (5-7)$$

式(5-7)中,  $\varphi_1(x)$  被表成两个线性函数的线性组合. 记

$$l_0(x) = \frac{x - x_1}{x_0 - x_1}, \quad l_1(x) = \frac{x - x_0}{x_1 - x_0}$$

显然, 它们满足

$$l_0(x_0) = 1, \quad l_0(x_1) = 0$$

$$l_1(x_0) = 0, \quad l_1(x_1) = 1$$

即  $l_i(x) (i=0, 1)$  在对应的插值点  $x_i$  处取值为 1, 在其他点处取值为 0. 不难想象, 以对应点处的函数值为系数对它们作线性组合所得的函数, 不仅仍是线性的, 且必定满足插值条件. 由此得到启发, 当节点增多到  $n+1$  个时, 可以先构造  $n$  次多项式  $l_i(x) (i=0, 1, \dots, n)$ , 它们满足

$$l_i(x_j) = \begin{cases} 0, & j \neq i \\ 1, & j = i \end{cases} \quad (5-8)$$

然后以对应点处的函数值为系数作线性组合, 即得所要求的插值多项式. 下面推导  $l_i(x) (i=0, 1, \dots, n)$  的表达式.

由式(5-8), 多项式  $l_i(x)$  有  $n$  个根  $x_j (j=0, 1, \dots, n, j \neq i)$ , 且  $l_i(x_i) = 1$ , 故它必定是以下形式

$$\begin{aligned} l_i(x) &= \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)} \\ &= \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \quad (i = 0, 1, \dots, n) \end{aligned} \quad (5-9)$$

这些函数称为 Lagrange 插值基函数. 利用它们立即得出插值问题的解

$$\varphi_n(x) = \sum_{i=0}^n y_i l_i(x) = \sum_{i=0}^n y_i \left( \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} \right) \quad (5-10a)$$

事实上, 因为每个插值基函数  $l_i(x) (i=0, 1, \dots, n)$  都是  $n$  次多项式, 故  $\varphi_n(x)$  是至多  $n$  次多项式. 由式(5-8)又得

$$\varphi_n(x_k) = \sum_{i=0}^n y_i l_i(x_k) = y_k \quad (k = 0, 1, \dots, n)$$

即  $\varphi_n(x)$  满足插值条件式(5-2).

式(5-10a)称为  $n$  次 Lagrange 插值多项式. 为了以后便于区别, 常用  $L_n(x)$  代替  $\varphi_n(x)$ , 以突出表示这是由 Lagrange 插值所得到的插值多项式, 即

$$L_n(x) = \sum_{i=0}^n y_i l_i(x) \quad (5-10b)$$

由前面讨论的结果,  $n+1$  个节点的  $n$  次 Lagrange 插值多项式存在唯一, 式 (5-5) 为插值余项. 式 (5-10b) 形式对称, 容易编制程序. 算法 5.1 用于计算非节点处的函数近似值.

### 算法 5.1

(1) 输入  $x, x_i, y_i (i=0, 1, \dots, n)$ .

(2) 对  $i=0, 1, \dots, n, l_i = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$ .

(3)  $L = \sum_{i=0}^n y_i l_i$ .

(4) 输出  $L \approx f(x)$ , 停机.

因为  $\omega_{n+1}(x) = \prod_{j=0}^n (x - x_j)$ , 求导可得  $\omega'_{n+1}(x_i) = \prod_{\substack{j=0 \\ j \neq i}}^n (x_i - x_j)$ , 于是式

(5-10b) 可改写成

$$L_n(x) = \sum_{i=0}^n \frac{\omega_{n+1}(x)}{(x - x_i) \omega'_{n+1}(x_i)} y_i \quad (5-11)$$

特别地, 当  $n=1$  时, 两点一次 Lagrange 插值多项式为

$$L_1(x) = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1$$

当  $n=2$  时, 即给定三个节点  $x_0, x_1, x_2$  处的函数值  $y_0, y_1, y_2$ , 三个插值基函数为

$$l_0(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)}$$

$$l_1(x) = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)}$$

$$l_2(x) = \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

二次 Lagrange 插值多项式为

$$L_2(x) = y_0 l_0(x) + y_1 l_1(x) + y_2 l_2(x)$$

如图 5-3 所示, 这是一条过点  $(x_i, y_i)$  ( $i=0, 1, 2$ ) 的抛物线. 这说明用二次插值多项式近似函数, 其几何意义是用过曲线上三点的抛物线近似曲线, 故三点二次插值又称为抛物线插值.

例 1 已知函数  $y = \ln x$  的函数表如下:

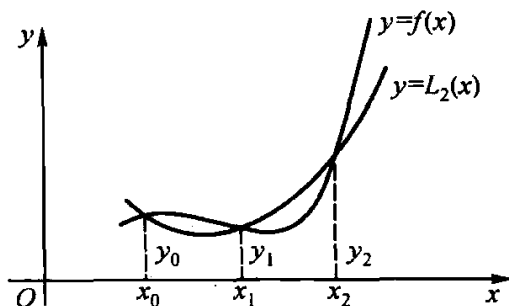


图 5-3

| $x$         | 10      | 11      | 12      | 13      | 14      |
|-------------|---------|---------|---------|---------|---------|
| $y = \ln x$ | 2.302 6 | 2.397 9 | 2.484 9 | 2.564 9 | 2.639 1 |

分别用 Lagrange 线性插值和抛物线插值求  $\ln 11.5$  的近似值,并估计截断误差.

[解] 线性插值. 取两个节点  $x_0 = 11, x_1 = 12$ , 插值基函数为

$$l_0(x) = \frac{x - x_1}{x_0 - x_1} = -(x - 12), \quad l_1(x) = \frac{x - x_0}{x_1 - x_0} = x - 11$$

由式(5-7)

$$L_1(x) = -2.397\,9(x - 12) + 2.484\,9(x - 11)$$

将  $x = 11.5$  代入, 即得

$$\ln 11.5 \approx L_1(11.5) = 2.397\,9 \times 0.5 + 2.484\,9 \times 0.5 = 2.441\,4$$

按式(5-5)

$$R_1(x) = \frac{(\ln x)''}{2!} \xi (x - 11)(x - 12)$$

因为  $(\ln x)'' = -\frac{1}{x^2}$ ,  $\xi$  在 11 与 12 之间, 故

$$|(\ln x)''_\xi| = \frac{1}{\xi^2} \leq \frac{1}{11^2} = 0.008\,264\,5$$

于是

$$|R_1(11.5)| \leq \frac{1}{2} \times 0.008\,264\,5 \times 0.5 \times 0.5 = 1.033\,06 \times 10^{-3}$$

抛物线插值. 取  $x_0 = 11, x_1 = 12, x_2 = 13$ , 插值多项式为

$$\begin{aligned} L_2(x) &= 2.397\,9 \frac{(x-12)(x-13)}{(11-12)(11-13)} + 2.484\,9 \frac{(x-11)(x-13)}{(12-11)(12-13)} + \\ &\quad 2.564\,9 \frac{(x-11)(x-12)}{(13-11)(13-12)} \\ &= 1.198\,95(x-12)(x-13) - 2.484\,9(x-11)(x-13) + \\ &\quad 1.282\,45(x-11)(x-12). \end{aligned}$$

所以

$$\begin{aligned} \ln 11.5 &\approx L_2(11.5) \\ &= 1.198\,95 \times (-0.5) \times (-1.5) - 2.484\,9 \times 0.5 \times (-1.5) + \\ &\quad 1.282\,45 \times 0.5 \times (-0.5) \\ &= 2.442\,275 \end{aligned}$$

因为  $(\ln x)''' = \frac{2}{x^3}$ , 于是

$$\max_{11 \leq x \leq 13} |(\ln x)'''| \leq \frac{2}{11^3} = 0.150\,3 \times 10^{-2}$$

因此用抛物线插值计算的误差为

$$\begin{aligned} |R_2(11.5)| &= \frac{|(\ln x)'''|}{3!} |(11.5 - 11)(11.5 - 12)(11.5 - 13)| \\ &\leq \frac{1}{6} \times 0.1503 \times 10^{-2} \times 0.5 \times 0.5 \times 1.5 \\ &= 9.3938 \times 10^{-5} \end{aligned}$$

查表可得  $\ln 11.5 = 2.442347$ .

**例 2** 设  $x_0, x_1, \dots, x_n$  为  $n+1$  个互异节点,  $l_i(x) (i=0, 1, \dots, n)$  为这组节点上的 Lagrange 插值基函数, 试证:  $\sum_{i=0}^n x_i^k l_i(x) \equiv x^k (k=0, 1, \dots, n)$ .

[证明] 由上式左端的表达式, 设  $f_k(x) = x^k (k=0, 1, \dots, n)$ , 则  $f_k(x)$  对于这  $n+1$  个节点的 Lagrange 插值函数为

$$L_{k,n}(x) = \sum_{i=0}^n x_i^k l_i(x) \quad (k=0, 1, \dots, n) \quad (5-12)$$

对于任意  $x$ , 由于  $f_k^{(m)}(x) \equiv 0 (m > k)$ , 则插值函数  $L_{k,n}(x)$  的余项为

$$R_{k,n}(x) = f_k(x) - L_{k,n}(x) = \frac{f_k^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) \equiv 0 \quad (k=0, 1, \dots, n)$$

所以

$$L_{k,n}(x) = \sum_{i=0}^n x_i^k l_i(x) \equiv f_k(x) = x^k \quad (k=0, 1, \dots, n)$$

下面的 MATLAB 函数实现了 Lagrange 插值运算:

```
function y = lagrange(xi, yi, x)
n = length(xi); m = length(x); y = 0;
for i = 1:n
    t = ones(1, length(x));
    for j = 1:n
        if j ~= i, t = t * (x - xi(j)) / (xi(i) - xi(j));
        end
    end
    y = y + t * yi(i);
end
```

其中  $xi, yi$  是插值数据;  $x$  为待计算函数值的点组成的数组, 也可是符号变量, 得到插值函数表达式. 例如:

```
>> xi = [11 12 13]; yi = [2.3979 2.4849 2.5649]; symx = sym('x');
>> y = lagrange(xi, yi, symx)
```

得到插值多项式:

$$y = -23\,979/20\,000 * (-x + 12) * (x - 13) - 24\,849/10\,000 * (x - 11) * (x - 13) + 25\,649/10\,000 * (1/2 * x - 11/2) * (x - 12)$$

与例 1 中的  $L_2(x)$  一致. 若构造  $x$  的更细密的求值点:

$$>> xx = 11:0.1:13;$$

输入命令:

$$>> yy = \text{lagrange}(xi, yi, xx)$$

即可得到函数在相应点处的一组近似值  $yy$ .

## § 5.2 牛顿 (Newton) 插值

如果将图 5-1 中直线用点斜式方程表示, 即把线性插值公式改写成以下形式

$$\varphi_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0) \quad (5-13)$$

由此导出插值多项式的另一种表示形式——牛顿插值公式. 在导出 Newton 公式前, 先介绍公式表示中所需要用到的差商、差分概念及性质.

### 5.2.1 差商

**定义 5.1** 设有函数  $f(x)$ ,  $x_0, x_1, x_2, \dots$  为一系列互不相等的点, 称  $\frac{f(x_i) - f(x_j)}{x_i - x_j} (i \neq j)$  为  $f(x)$  关于点  $x_i, x_j$  的一阶差商 (也称均差), 记为  $f[x_i, x_j]$ , 即

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

类似于高阶导数的定义, 称一阶差商的差商

$$\frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

为  $f(x)$  关于点  $x_i, x_j, x_k$  的二阶差商, 记为  $f[x_i, x_j, x_k]$ . 一般地, 称

$$\frac{f[x_0, x_1, \dots, x_{k-1}] - f[x_1, x_2, \dots, x_k]}{x_0 - x_k}$$

为  $f(x)$  关于点  $x_0, x_1, \dots, x_k$  的  $k$  阶差商, 记为

$$f[x_0, x_1, \dots, x_k] = \frac{f[x_0, x_1, \dots, x_{k-1}] - f[x_1, x_2, \dots, x_k]}{x_0 - x_k} \quad (5-14)$$

容易证明,差商具有下述性质:

(1) 各阶差商均具有线性性,即若  $f(x) = a\varphi(x) + b\psi(x)$ , 则对任意正整数  $k$ , 都有

$$f[x_0, x_1, \dots, x_k] = a\varphi[x_0, x_1, \dots, x_k] + b\psi[x_0, x_1, \dots, x_k] \quad (5-15)$$

(2)  $k$  阶差商  $f[x_0, x_1, \dots, x_k]$  可表示成  $f(x_0), f(x_1), \dots, f(x_k)$  的线性组合

$$f[x_0, x_1, \dots, x_k] = \sum_{i=0}^k \frac{f(x_i)}{\omega'_{k+1}(x_i)} \quad (5-16)$$

其中  $\omega'_{k+1}(x_i) = \prod_{\substack{j=0 \\ j \neq i}}^n (x_i - x_j)$ .

$k=1$  是显然的. 当  $k=2$  时

$$\begin{aligned} f[x_0, x_1, x_2] &= \frac{f[x_0, x_1] - f[x_1, x_2]}{x_0 - x_2} \\ &= \frac{1}{x_0 - x_2} \left[ \frac{f(x_0) - f(x_1)}{x_0 - x_1} - \frac{f(x_1) - f(x_2)}{x_1 - x_2} \right] \\ &= \frac{f(x_0)}{(x_0 - x_1)(x_0 - x_2)} + \frac{f(x_1)}{(x_1 - x_0)(x_1 - x_2)} + \frac{f(x_2)}{(x_2 - x_0)(x_2 - x_1)} \end{aligned}$$

一般情形可用数学归纳法证明.

(3) 各阶差商均具有对称性,即改变节点的位置,差商值不变. 如

$$f[x_i, x_j] = f[x_j, x_i]$$

$$f[x_i, x_j, x_k] = f[x_i, x_k, x_j] = f[x_j, x_i, x_k]$$

(4) 若  $f(x)$  是  $n$  次多项式, 则一阶差商  $f[x, x_i]$  是  $n-1$  次多项式.

事实上, 如果  $f(x)$  是  $n$  次多项式, 则  $P(x) = f(x) - f(x_i)$  也是  $n$  次多项式, 且  $P(x_i) = 0$ . 于是  $P(x)$  可分解为

$$P(x) = (x - x_i)P_{n-1}(x)$$

其中  $P_{n-1}(x)$  为  $n-1$  次多项式. 所以

$$f[x, x_i] = \frac{f(x) - f(x_i)}{x - x_i} = \frac{(x - x_i)P_{n-1}(x)}{x - x_i} = P_{n-1}(x)$$

为  $n-1$  次多项式.

计算各阶差商, 可按表 5-1 进行.

表 5-1

| $x_i$ | $f(x_i)$ | 一阶差商          | 二阶差商               | 三阶差商 | 四阶差商 | ... |
|-------|----------|---------------|--------------------|------|------|-----|
| $x_0$ | $f(x_0)$ |               |                    |      |      |     |
| $x_1$ | $f(x_1)$ | $f[x_0, x_1]$ |                    |      |      |     |
| $x_2$ | $f(x_2)$ | $f[x_1, x_2]$ | $f[x_0, x_1, x_2]$ |      |      |     |

续表

| $x_i$ | $f(x_i)$ | 一阶差商          | 二阶差商               | 三阶差商                    | 四阶差商                         | ... |
|-------|----------|---------------|--------------------|-------------------------|------------------------------|-----|
| $x_3$ | $f(x_3)$ | $f[x_2, x_3]$ | $f[x_1, x_2, x_3]$ | $f[x_0, x_1, x_2, x_3]$ |                              |     |
| $x_4$ | $f(x_4)$ | $f[x_3, x_4]$ | $f[x_2, x_3, x_4]$ | $f[x_1, x_2, x_3, x_4]$ | $f[x_0, x_1, x_2, x_3, x_4]$ |     |
| $x_5$ | $f(x_5)$ | $f[x_4, x_5]$ | $f[x_3, x_4, x_5]$ | $f[x_2, x_3, x_4, x_5]$ | $f[x_1, x_2, x_3, x_4, x_5]$ |     |

### 5.2.2 Newton 插值公式

按定义 5.1 线性插值公式(5-13)可表示成

$$\varphi_1(x) = f(x_0) + (x - x_0)f[x_0, x_1] \quad (5-17)$$

式(5-17)称为一次 Newton 插值多项式. 一般地, 由各阶差商的定义, 依次可得

$$f(x) = f(x_0) + (x - x_0)f[x, x_0]$$

$$f[x, x_0] = f[x_0, x_1] + (x - x_1)f[x, x_0, x_1]$$

$$f[x, x_0, x_1] = f[x_0, x_1, x_2] + (x - x_2)f[x, x_0, x_1, x_2]$$

.....

$$f[x, x_0, \dots, x_{n-1}] = f[x_0, x_1, \dots, x_n] + (x - x_n)f[x, x_0, \dots, x_n]$$

将以上各式分别乘以  $1, (x - x_0), (x - x_0)(x - x_1), \dots, (x - x_0)(x - x_1) \cdots (x - x_{n-1})$ , 然后相加并消去两边相等的部分, 即得

$$\begin{aligned} f(x) &= f(x_0) + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots \\ &\quad + (x - x_0)(x - x_1) \cdots (x - x_{n-1})f[x_0, x_1, \dots, x_n] \\ &\quad + (x - x_0)(x - x_1) \cdots (x - x_n)f[x, x_0, x_1, \dots, x_n] \end{aligned} \quad (5-18)$$

记

$$\begin{aligned} N_n(x) &= f(x_0) + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots \\ &\quad + (x - x_0)(x - x_1) \cdots (x - x_{n-1})f[x_0, x_1, \dots, x_n] \end{aligned} \quad (5-19)$$

$$\begin{aligned} R_n(x) &= (x - x_0)(x - x_1) \cdots (x - x_n)f[x, x_0, x_1, \dots, x_n] \\ &= \hat{\omega}_{n+1}(x)f[x, x_0, x_1, \dots, x_n] \end{aligned} \quad (5-20)$$

则

$$f(x) = N_n(x) + R_n(x)$$

显然,  $N_n(x)$  是至多  $n$  次的多项式. 而由

$$R_n(x_i) = \omega_{n+1}(x_i)f[x_i, x_0, x_1, \dots, x_n] = 0 \quad (i = 0, 1, \dots, n)$$

得  $f(x_i) = N_n(x_i) \quad (i = 0, 1, \dots, n)$ . 这表明  $N_n(x)$  满足插值条件式(5-2), 因而它是  $f(x)$  的  $n$  次插值多项式. 这种形式的插值多项式称为 Newton 插值多项式.

Newton 插值的优点是:每增加一个节点,插值多项式只增加一项,即

$$N_{n+1}(x) = N_n(x) + (x - x_0)(x - x_1) \cdots (x - x_n)f[x_0, x_1, \cdots, x_{n+1}]$$

因此便于递推运算. 而且 Newton 插值的计算量小于 Lagrange 插值.

由插值多项式的唯一性可知,  $n$  次 Newton 插值多项式与  $n$  次 Lagrange 插值多项式是相等的, 即  $N_n(x) = L_n(x)$ , 它们只是表示形式不同. 因此 Newton 余项与 Lagrange 余项也是相等的, 即

$$R_n(x) = \omega_{n+1}(x)f[x, x_0, x_1, \cdots, x_n] = \frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x), \quad \xi \in (a, b)$$

由此可得差商与导数的关系

$$f[x_0, x_1, \cdots, x_n] = \frac{f^{(n)}(\xi)}{n!} \quad (5-21)$$

其中  $\xi \in (\alpha, \beta)$ ,  $\alpha = \min_{0 \leq i \leq n} \{x_i\}$ ,  $\beta = \max_{0 \leq i \leq n} \{x_i\}$ .

Newton 插值多项式可按表 5-2 计算.

表 5-2

| $x_i$    | $y_i = f(x_i)$ | 一阶差商              | 二阶差商                       | ...      | $n$ 阶差商                    |                               |
|----------|----------------|-------------------|----------------------------|----------|----------------------------|-------------------------------|
| $x_0$    | $y_0$          |                   |                            |          |                            | 1                             |
| $x_1$    | $y_1$          | $f[x_0, x_1]$     |                            |          |                            | $x - x_0$                     |
| $x_2$    | $y_2$          | $f[x_1, x_2]$     | $f[x_0, x_1, x_2]$         |          |                            | $\prod_{j=0}^1 (x - x_j)$     |
| $x_3$    | $y_3$          | $f[x_2, x_3]$     | $f[x_1, x_2, x_3]$         |          |                            | $\prod_{j=0}^2 (x - x_j)$     |
| $\vdots$ | $\vdots$       | $\vdots$          | $\vdots$                   | $\vdots$ |                            | $\vdots$                      |
| $x_n$    | $y_n$          | $f[x_{n-1}, x_n]$ | $f[x_{n-2}, x_{n-1}, x_n]$ |          | $f[x_0, x_1, \cdots, x_n]$ | $\prod_{j=0}^{n-1} (x - x_j)$ |

$n$  次 Newton 插值多项式  $N_n(x)$  为表 5-2 中对角线上的差商值与右端乘子积的和. 注意, 此时我们将  $y_0 = f(x_0)$  看作  $f(x)$  关于点  $x_0$  的零阶差商.

算法 5.2 给出了用 Newton 插值计算非节点处函数近似值的过程.

### 算法 5.2

(1) 输入数据  $x, x_i, y_i (i=0, 1, \cdots, n)$ .

(2) 置  $f_k = y_k (k=0, 1, \cdots, n)$ .

(3) 计算各阶差商. 对  $i=1, 2, \cdots, n$

$$\frac{f_{k-1} - f_k}{x_{k-i} - x_k} \Rightarrow f_k \quad (k = n, n-1, \cdots, i)$$



$$\begin{aligned}
 (4) \quad p &= f_0 + \sum_{k=0}^n f_k \left[ \prod_{j=0}^{k-1} (x - x_j) \right] \\
 &= \{ [ (f_n(x - x_{n-1}) + f_{n-1})(x - x_{n-2}) + f_{n-2} ] (x - x_{n-3}) + \cdots \} (x - x_0) + f_0
 \end{aligned}$$

(5) 输出  $p \approx f(x)$ , 停机.

例 3 用 Newton 插值公式计算例 1 中的  $\ln 11.5$ .

[解] 如果仍取点  $x_0 = 11, x_0 = 12, x_0 = 13$  作抛物线插值, 按表 5-2 计算, 结果如下:

| $x_i$ | $y_i = \ln x_i$ | 一阶差商    | 二阶差商     |                    |
|-------|-----------------|---------|----------|--------------------|
| 11    | 2.397 9         |         |          | 1                  |
| 12    | 2.484 9         | 0.087 0 |          | $x - 11$           |
| 13    | 2.564 9         | 0.080 0 | -0.003 5 | $(x - 11)(x - 12)$ |

$$N_2(x) = 2.397 9 + 0.087 0(x - 11) - 0.003 5(x - 11)(x - 12)$$

所以

$$\begin{aligned}
 \ln 11.5 &\approx N_2(11.5) \\
 &= 2.397 9 + 0.087 0 \times 0.5 + 0.003 5 \times 0.5 \times 0.5 \\
 &= 2.442 275
 \end{aligned}$$

若加节点  $x = 10, 14, \ln 10 = 2.302 6, \ln 14 = 2.639 1$ , 用  $\ln x$  的四次 Newton 插值多项式近似, 则按表 5-2 计算结果如下:

| $x_i$ | $y_i = \ln x_i$ | 一阶差商    | 二阶差商      | 三阶差商     | 四阶差商       |                             |
|-------|-----------------|---------|-----------|----------|------------|-----------------------------|
| 10    | 2.302 6         |         |           |          |            | 1                           |
| 11    | 2.397 9         | 0.095 3 |           |          |            | $x - 10$                    |
| 12    | 2.484 9         | 0.087 0 | -0.004 15 |          |            | $(x - 10)(x - 11)$          |
| 13    | 2.564 9         | 0.080 0 | -0.003 50 | 0.000 22 |            | $\prod_{k=10}^{12} (x - k)$ |
| 14    | 2.639 1         | 0.074 2 | -0.002 90 | 0.000 20 | -0.000 005 | $\prod_{k=10}^{13} (x - k)$ |

所以

$$\begin{aligned}
 \ln 11.5 &\approx N_4(11.5) \\
 &= 2.302 6 + 0.095 3 \times 1.5 - 0.004 15 \times 1.5 \times 0.5 + 0.000 22 \times 1.5 \times \\
 &\quad 0.5 \times (-0.5) - 0.000 005 \times 1.5 \times 0.5 \times (-0.5) \times (-1.5) \\
 &= 2.442 352 2
 \end{aligned}$$

### 5.2.3 差分

当节点等距时,即相邻两个节点之差(称为步长)为常数,Newton 插值公式的形式会更简单.此时关于节点间函数的平均变化率(差商)可用函数值之差(差分)来表示.

**定义 5.2** 设有等距节点  $x_k = x_0 + kh$  ( $k = 0, 1, \dots, n$ ), 步长  $h$  为常数,  $f_k = f(x_k)$ . 称相邻两个节点  $x_k, x_{k+1}$  处的函数值的增量  $f_{k+1} - f_k$  ( $k = 0, 1, \dots, n-1$ ) 为函数  $f(x)$  在点  $x_k$  处以  $h$  为步长的一阶差分,记为  $\Delta f_k$ ,即

$$\Delta f_k = f_{k+1} - f_k \quad (k = 0, 1, \dots, n) \quad (5-22)$$

类似地,定义差分的差分为高阶差分.如二阶差分为

$$\Delta^2 f_k = \Delta f_{k+1} - \Delta f_k \quad (k = 0, 1, \dots, n-2)$$

一般地, $m$  阶差分为

$$\Delta^m f_k = \Delta^{m-1} f_{k+1} - \Delta^{m-1} f_k \quad (m = 2, 3, \dots) \quad (5-23)$$

计算各阶差分,也可按表 5-3 进行.

表 5-3

| $x_k$    | $f_k = f(x_k)$ | $\Delta f_k$ | $\Delta^2 f_k$ | $\Delta^3 f_k$ | $\Delta^4 f_k$ | ... |
|----------|----------------|--------------|----------------|----------------|----------------|-----|
| $x_0$    | $f_0$          |              |                |                |                |     |
| $x_1$    | $f_1$          | $\Delta f_0$ |                |                |                |     |
| $x_2$    | $f_2$          | $\Delta f_1$ | $\Delta^2 f_0$ |                |                |     |
| $x_3$    | $f_3$          | $\Delta f_2$ | $\Delta^2 f_1$ | $\Delta^3 f_0$ |                |     |
| $x_4$    | $f_4$          | $\Delta f_3$ | $\Delta^2 f_2$ | $\Delta^3 f_1$ | $\Delta^4 f_0$ |     |
| $\vdots$ | $\vdots$       | $\vdots$     | $\vdots$       | $\vdots$       | $\vdots$       |     |

由式(5-22)和式(5-23)定义的各阶差分又称为向前差分.常用的差分还有两种:

$$\nabla f_k = f_k - f_{k-1} \quad (5-24)$$

称为  $f(x)$  在  $x_k$  处以  $h$  为步长的向后差分.

$$\delta f_k = f\left(x_k + \frac{h}{2}\right) - f\left(x_k - \frac{h}{2}\right) \quad (5-25)$$

称为  $f(x)$  在  $x_k$  处以  $h$  为步长的中心差分.一般地, $m$  阶向后差分与  $m$  阶中心差分公式为

$$\nabla^m f_k = \nabla^{m-1} f_k - \nabla^{m-1} f_{k-1} \quad (5-26)$$

$$\delta^m f_k = \delta^{m-1} f_{k+\frac{1}{2}} - \delta^{m-1} f_{k-\frac{1}{2}} \quad (5-27)$$

计算时,可分别按表 5-4 和表 5-5 排列.

表 5-4

| $x_k$ | $f_k = f(x_k)$ | $\nabla f_k$ | $\nabla^2 f_k$ | $\nabla^3 f_k$ | $\nabla^4 f_k$ | ... |
|-------|----------------|--------------|----------------|----------------|----------------|-----|
| $x_0$ | $f_0$          |              |                |                |                |     |
| $x_1$ | $f_1$          | $\nabla f_1$ |                |                |                |     |
| $x_2$ | $f_2$          | $\nabla f_2$ | $\nabla^2 f_2$ |                |                |     |
| $x_3$ | $f_3$          | $\nabla f_3$ | $\nabla^2 f_3$ | $\nabla^3 f_3$ |                |     |
| $x_4$ | $f_4$          | $\nabla f_4$ | $\nabla^2 f_4$ | $\nabla^3 f_4$ | $\nabla^4 f_4$ |     |

表 5-5

| $x_k$ | $f_k = f(x_k)$ | $\delta$                  | $\delta^2$     | $\delta^3$                  | $\delta^4$     | ... |
|-------|----------------|---------------------------|----------------|-----------------------------|----------------|-----|
| $x_0$ | $f_0$          |                           |                |                             |                |     |
| $x_1$ | $f_1$          | $\delta f_{\frac{1}{2}}$  |                |                             |                |     |
| $x_2$ | $f_2$          | $\delta f_{1\frac{1}{2}}$ | $\delta^2 f_1$ |                             |                |     |
| $x_3$ | $f_3$          | $\delta f_{2\frac{1}{2}}$ | $\delta^2 f_2$ | $\delta^3 f_{1\frac{1}{2}}$ |                |     |
| $x_4$ | $f_4$          | $\delta f_{3\frac{1}{2}}$ | $\delta^2 f_3$ | $\delta^3 f_{2\frac{1}{2}}$ | $\delta^4 f_2$ |     |

由差分表 5-3 ~ 表 5-5 可以看出, 差分值所在的列数为差分的阶数. 要确定某个差分值是哪个点的差分, 对向前差分则为左上斜线上函数值对应的自变量值; 向后差分则为差分值所在行的自变量值; 对中心差分, 只要将左上斜线上函数值所对应的自变量值加上  $\frac{m}{2}h$  即得, 其中  $m$  为差分的阶数,  $h$  为步长.

例如, 按例 1 中所给出的  $y = \ln x$  的函数表, 可列出  $h = 1$  的差分表如下:

| $x_i$ | $f_i = \ln x_i$ | 一阶差分    | 二阶差分     | 三阶差分     | 四阶差分     |
|-------|-----------------|---------|----------|----------|----------|
| 10    | 2.302 6         |         |          |          |          |
| 11    | 2.397 9         | 0.095 3 |          |          |          |
| 12    | 2.484 9         | 0.087 0 | -0.008 3 |          |          |
| 13    | 2.564 9         | 0.080 0 | -0.007 0 | -0.001 3 |          |
| 14    | 2.639 1         | 0.074 2 | -0.005 8 | 0.001 2  | -0.000 1 |

表中值 -0.007 0 左上斜线函数值对应的自变量值为 11, 所在行的自变量值为 13,  $m = 2, h = 1$ , 故

$$-0.007\ 0 = \Delta^2 \ln 11 = \nabla^2 \ln 13 = \delta^2 \ln 12$$

同理可得

$$-0.001\ 3 = \Delta^3 \ln 10 = \nabla^3 \ln 13 = \delta^3 \ln 11.5$$

差分具有以下性质:

(1) 各阶差分均具有线性性, 即若  $f(x) = a\varphi(x) + b\psi(x)$ , 则对任意正整数  $m$ , 都有

$$\Delta^m f_k = a\Delta^m \varphi_k + b\Delta^m \psi_k \quad (5-28)$$

(2) 各阶差分均可表成函数值的线性组合, 例如

$$\Delta^2 f_k = \Delta f_{k+1} - \Delta f_k = f_{k+2} - 2f_{k+1} + f_k$$

$$\Delta^3 f_k = \Delta^2 f_{k+1} - \Delta^2 f_k = f_{k+3} - 3f_{k+2} + 3f_{k+1} - f_k$$

$$\nabla^2 f_k = \nabla f_k - \nabla f_{k-1} = f_k - 2f_{k-1} + f_{k-2}$$

一般地

$$\Delta^m f_k = \sum_{j=0}^m (-1)^j \binom{m}{j} f_{k+m-j} \quad (5-29)$$

$$\nabla^m f_k = \sum_{j=0}^m (-1)^j \binom{m}{j} f_{k-j} \quad (5-30)$$

其中  $\binom{m}{j} = C_m^j = \frac{m!}{j!(m-j)!}$  为二项展开式的系数.

(3) 各种差分之间可以互化, 这由差分表即可看出. 向后差分与中心差分化成向前差分的公式如下:

$$\nabla^m f_k = \Delta^m f_{k-m} \quad (5-31)$$

$$\delta^m f_k = \Delta^m f_{k-\frac{m}{2}} \quad (5-32)$$

例如,  $\nabla^2 f_4 = \nabla f_4 - \nabla f_3 = f_4 - 2f_3 + f_2 = \Delta^2 f_2 = \Delta^2 f_{4-2}$ .

(4) 可用差分表示差商. 例如

$$f[x_k, x_{k+1}] = \frac{f(x_k) - f(x_{k+1})}{x_k - x_{k+1}} = \frac{\Delta f_k}{h} = \frac{\nabla f_{k+1}}{h}$$

$$f[x_k, x_{k+1}, x_{k+2}] = \frac{f[x_k, x_{k+1}] - f[x_{k+1}, x_{k+2}]}{x_k - x_{k+2}} = \frac{\Delta^2 f_k}{2h^2} = \frac{\nabla^2 f_{k+2}}{2h^2}$$

一般地, 有

$$f[x_k, x_{k+1}, \dots, x_{k+m}] = \frac{1}{m! h^m} \Delta^m f_k = \frac{1}{m! h^m} \nabla^m f_{k+m} \quad (m = 1, 2, \dots) \quad (5-33)$$

结合式(5-21), 可得差分与导数的关系

$$\Delta^m f_k = m! h^m f[x_k, x_{k+1}, \dots, x_{k+m}] = h^m f^{(m)}(\xi), \quad x_k < \xi < x_{k+m} \quad (5-34)$$

### 5.2.4 等距节点插值公式

如果插值节点是等距的, 则插值公式可用差分表示. 设已知节点  $x_k = x_0 + kh$

( $k=0,1,2,\cdots,n$ ), 将式(5-33)代入插值公式(5-19), 得

$$\begin{aligned} N_n(x) &= f(x_0) + f[x_0, x_1](x - x_0) + \cdots \\ &\quad + f[x_0, x_1, \cdots, x_n](x - x_0)(x - x_1)\cdots(x - x_{n-1}) \\ &= f_0 + \frac{1}{h}\Delta f_0(x - x_0) + \frac{\Delta^2 f_0}{2!h^2}(x - x_0)(x - x_1) + \cdots + \\ &\quad \frac{\Delta^n f_0}{n!h^n}(x - x_0)(x - x_1) + \cdots + (x - x_{n-1}) \end{aligned} \quad (5-35)$$

若令  $x = x_0 + th$ , 则上式又可变形为

$$\begin{aligned} N_n(x_0 + th) &= f_0 + t\Delta f_0 + \frac{t(t-1)}{2!}\Delta^2 f_0 + \cdots \\ &\quad + \frac{t(t-1)\cdots(t-n+1)}{n!}\Delta^n f_0 \end{aligned} \quad (5-36)$$

式(5-36)称为 Newton 向前插值公式, 其余项为

$$\begin{aligned} R_n(x) &= \frac{f^{n+1}(\xi)}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n) \\ &= \frac{t(t-1)\cdots(t-n)}{(n+1)!}h^{n+1}f^{n+1}(\xi), \quad \xi \in (x_0, x_n) \end{aligned} \quad (5-37)$$

完全类似地, 也可以用向后差分表示 Newton 插值公式. 令  $x = x_n + th$ ,  $x \in [x_0, x_n]$ , 则有

$$\begin{aligned} N_n(x_n + th) &= f_n + t\nabla f_n + \frac{t(t+1)}{2!}\nabla^2 f_n + \cdots \\ &\quad + \frac{t(t+1)\cdots(t+n-1)}{n!}\nabla^n f_n \end{aligned} \quad (5-38)$$

式(5-38)称为 Newton 向后插值公式, 其余项为

$$R_n(x) = \frac{t(t+1)\cdots(t+n)}{(n+1)!}h^{n+1}f^{(n+1)}(\xi), \quad \xi \in (x_0, x_n) \quad (5-39)$$

Newton 向前、向后插值公式均是 Newton 插值公式在等距节点时的变形. 实际计算时, 也可列表进行. 将表 5-6 中对角线上的差分值与对应行右端因子乘积求和即得 Newton 向前插值公式, 而 Newton 向后插值公式则为最后的节点所在行的各阶差分值与对应列下端因子乘积之和.

表 5-6

| $x_i$ | $f_i = f(x_i)$ | 一阶差分                      | 二阶差分 | 三阶差分 | ... | $n$ 阶差分 |     |
|-------|----------------|---------------------------|------|------|-----|---------|-----|
| $x_0$ | $f_0$          |                           |      |      |     |         | 1   |
| $x_1$ | $f_1$          | $\Delta f_0 (\nabla f_1)$ |      |      |     |         | $t$ |

续表

| $x_i$    | $f_i = f(x_i)$ | 一阶差分                         | 二阶差分                             | 三阶差分                               | ...      | $n$ 阶差分                                |                                        |
|----------|----------------|------------------------------|----------------------------------|------------------------------------|----------|----------------------------------------|----------------------------------------|
| $x_2$    | $f_2$          | $\Delta f_1(\nabla f_2)$     | $\Delta^2 f_0(\nabla^2 f_2)$     |                                    |          |                                        | $\frac{t(t-1)}{2}$                     |
| $x_3$    | $f_3$          | $\Delta f_2(\nabla f_3)$     | $\Delta^2 f_1(\nabla^2 f_3)$     | $\Delta^3 f_0(\nabla^3 f_3)$       |          |                                        | $\frac{1}{3!} \prod_{j=0}^2 (t-j)$     |
| $\vdots$ | $\vdots$       | $\vdots$                     | $\vdots$                         | $\vdots$                           | $\ddots$ |                                        | $\vdots$                               |
| $x_n$    | $f_n$          | $\Delta f_{n-1}(\nabla f_n)$ | $\Delta^2 f_{n-2}(\nabla^2 f_n)$ | $\Delta^3 f_{n-3}(\nabla^3 f_n)$   |          | $\Delta^n f_0(\nabla^n f_n)$           | $\frac{1}{n!} \prod_{j=0}^{n-1} (t-j)$ |
|          | 1              | $t$                          | $\frac{t(t+1)}{2}$               | $\frac{1}{3!} \prod_{j=0}^2 (t+j)$ | ...      | $\frac{1}{n!} \prod_{j=0}^{n-1} (t+j)$ |                                        |

**例 4** 已知  $f(x) = \sin x$  的函数表如下, 分别用 Newton 向前、向后插值公式求  $\sin 0.578\ 91$  的近似值.

| $x$      | 0.4      | 0.5      | 0.6      | 0.7      |
|----------|----------|----------|----------|----------|
| $\sin x$ | 0.389 42 | 0.479 43 | 0.564 64 | 0.644 22 |

[解] 取  $x_0 = 0.4, x_1 = 0.5, x_2 = 0.6, x_3 = 0.7$ , 按表 5-6 计算得

| $x_i$ | $f_i = \sin x_i$ | 一阶差分     | 二阶差分               | 三阶差分                     |                          |
|-------|------------------|----------|--------------------|--------------------------|--------------------------|
| 0.4   | 0.389 42         |          |                    |                          | 1                        |
| 0.5   | 0.479 43         | 0.090 01 |                    |                          | $t$                      |
| 0.6   | 0.564 64         | 0.085 21 | -0.004 80          |                          | $\frac{1}{2}t(t-1)$      |
| 0.7   | 0.644 22         | 0.079 58 | -0.005 63          | -0.000 83                | $\frac{t}{3!}(t-1)(t-2)$ |
|       | 1                | $t$      | $\frac{t}{2}(t+1)$ | $\frac{t}{3!}(t+1)(t+2)$ |                          |

Newton 向前插值公式为

$$N_3(x_0 + th) = 0.389\ 42 + 0.090\ 01t - \frac{1}{2} \times 0.004\ 80t(t-1) - \frac{1}{6} \times 0.000\ 83t(t-1)(t-2)$$

将  $t = \frac{x - x_0}{h} = (0.578\ 91 - 0.4)/0.1 = 1.789\ 1$  代入上式得

$$\begin{aligned}\sin 0.578\ 91 &\approx N_3(0.578\ 91) \\ &= 0.389\ 42 + 0.090\ 01 \times 1.789\ 1 - \frac{1}{2} \times 0.004\ 80 \times 1.789\ 1 \times 0.789\ 1 \\ &\quad + \frac{1}{6} \times 0.000\ 83 \times 1.789\ 1 \times 0.789\ 1 \times 0.210\ 9 \\ &= 0.547\ 11\end{aligned}$$

由式(5-37),误差为

$$\begin{aligned}&|R_3(0.578\ 91)| \\ &= \left| \frac{0.1^4}{4!} \times 1.789\ 1 \times 0.789\ 1 \times (-0.210\ 9) \times (-1.210\ 9) \sin \xi \right| \\ &< 2 \times 10^{-6}\end{aligned}$$

Newton 向后插值公式为

$$\begin{aligned}N_3(x_4 + th) &= 0.644\ 22 + 0.079\ 58t \\ &\quad - \frac{0.005\ 63}{2}t(t+1) - \frac{0.000\ 83}{3!}t(t+1)(t+2)\end{aligned}$$

将  $t = \frac{x - x_4}{h} = -1.210\ 9$  代入上式得

$$\begin{aligned}\sin 0.578\ 91 &\approx 0.644\ 22 - 0.079\ 58 \times 1.210\ 9 \\ &\quad - \frac{1}{2} \times 0.005\ 63 \times (-1.210\ 9) \times (-0.210\ 9) \\ &\quad - \frac{1}{3!} \times 0.000\ 83 \times (-1.210\ 9) \times (-0.210\ 9) \times 0.789\ 1 \\ &= 0.547\ 11\end{aligned}$$

查表可得  $\sin 0.578\ 91 = 0.547\ 111\ 8$ .

如果取  $x_0 = 0.4, x_1 = 0.5, x_2 = 0.6$ , 用二阶 Newton 向后插值公式, 则得

$$N_2(x) = 0.564\ 64 + 0.085\ 21t - \frac{1}{2} \times 0.004\ 80t(t+1)$$

将  $t = \frac{x - x_2}{h} = -0.210\ 9$  代入上式得

$$\begin{aligned}\sin 0.578\ 91 &\approx 0.564\ 64 + 0.085\ 21 \times (-0.210\ 9) \\ &\quad - 0.002\ 40 \times (-0.210\ 9) \times 0.789\ 1 \\ &= 0.547\ 07\end{aligned}$$

其误差为

$$|R_2(5.789\ 1)| = \frac{0.1^3}{3!} |(-0.210\ 9) \times 0.789\ 1 \times 1.789\ 1 \cos \xi|$$

$$\leq 0.5 \times 10^{-4}$$

为便于递推计算,当插值点  $x$  接近数据表头时,一般用向前插值公式,而当插值点  $x$  接近数据表尾时,则采用向后插值公式.

### § 5.3 分段线性插值

在代数插值中,为了提高插值多项式对函数的逼近程度,常常增加节点个数,即提高多项式的次数,但这样做往往不能达到预想的结果. 例如,函数  $f(x) = \frac{1}{1+x^2}$ , 如果在区间  $[-5, 5]$  上取 11 个等距节点  $x_k = -5 + k (k = 0, 1, \dots, 10)$ , 由 Lagrange 插值公式可得到  $f(x)$  的 10 次插值多项式  $L_{10}(x)$ . 如图 5-4 所示,  $L_{10}(x)$  仅在区间中部能较好地逼近函数  $f(x)$ , 在其他部位差异较大,而且越接近端点,逼近效果越差. 可以证明,当节点无限加密时,  $L_n(x)$  也只能在很小范围内收敛. 这一现象称为 Runge 现象,它表明通过增加节点来提高逼近程度是不宜的. 因而一般不采用高次多项式插值.

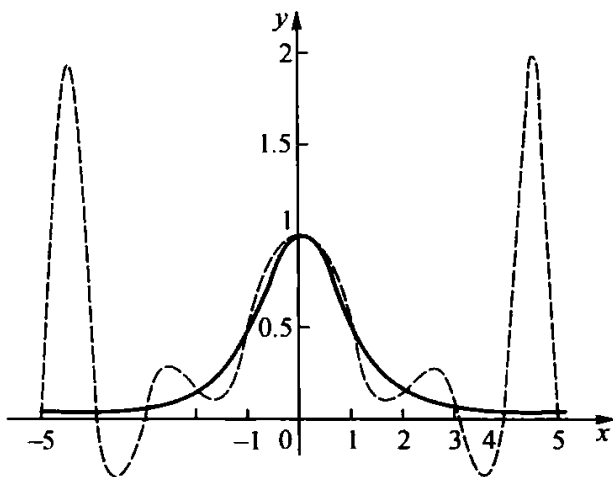


图 5-4

直观上容易想象,如果不用多项式曲线,而是将曲线  $y = f(x)$  的两个相邻的点用线段连接(见图 5-5),这样得到的折线必定能较好地近似曲线. 而且只要  $f(x)$  连续,节点越密,近似程度越好. 由此得到启发,为提高精度,在加密节点时,可以把节点分成若干段,分段用低次多项式近似函数,这就是分段插值的思想. 用折线近似曲线,相当于分段用线性插值,称为分段线性插值.

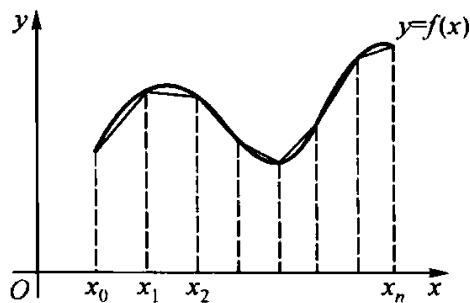


图 5-5



设在区间  $[a, b]$  上给定  $n+1$  个节点  $a = x_0 < x_1 < \cdots < x_{n-1} < x_n = b$  及节点上的函数值  $y_i = f(x_i)$  ( $i = 0, 1, \cdots, n$ ), 作一个插值函数  $\varphi(x)$ , 使其满足

(1)  $\varphi(x_i) = y_i$  ( $i = 0, 1, \cdots, n$ );

(2) 在每个小区间  $[x_i, x_{i+1}]$  ( $i = 0, 1, \cdots, n-1$ ) 上,  $\varphi(x)$  是线性函数, 则称函数  $\varphi(x)$  为  $[a, b]$  上关于数据  $(x_i, y_i)$  ( $i = 0, 1, \cdots, n$ ) 的分段线性插值函数.

由 Lagrange 线性插值公式容易写出  $\varphi(x)$  的分段表达式

$$\varphi(x) = \frac{x - x_{i+1}}{x_i - x_{i+1}} y_i + \frac{x - x_i}{x_{i+1} - x_i} y_{i+1}, \quad x_i \leq x \leq x_{i+1} \quad (i = 0, 1, \cdots, n-1) \quad (5-40)$$

也可以通过构造基函数的方法来求  $\varphi(x)$ .

首先构造一组基函数  $l_i(x)$  ( $i = 0, 1, \cdots, n$ ), 每个  $l_i(x)$  满足:

(1)  $l_i(x_j) = \begin{cases} 0, & j \neq i \\ 1, & j = i \end{cases} \quad (i, j = 0, 1, \cdots, n);$

(2)  $l_i(x)$  在每个小区间  $[x_j, x_{j+1}]$  ( $j = 0, 1, \cdots, n-1$ ) 上是线性函数.

这组函数称为分段线性插值基函数, 其图形如图 5-6 所示.

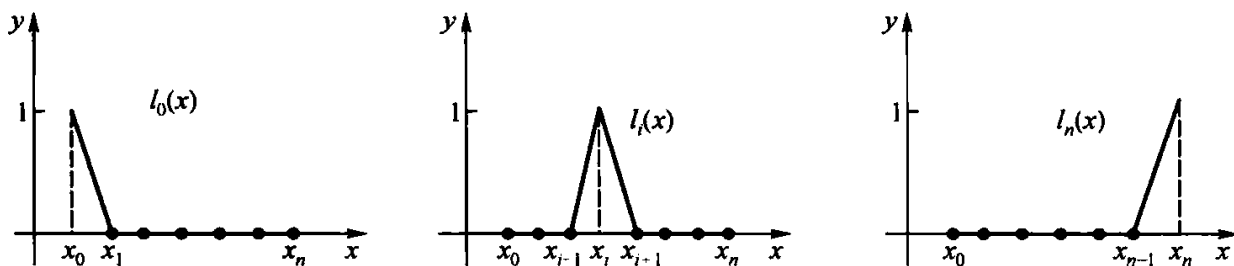


图 5-6

由图 5-6 可直接写出  $l_i(x)$  的表达式如下:

$$l_0(x) = \begin{cases} \frac{x - x_1}{x_0 - x_1}, & x \in [x_0, x_1] \\ 0, & x \in (x_1, x_n] \end{cases}$$

$$l_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}}, & x \in [x_{i-1}, x_i] \\ \frac{x - x_{i+1}}{x_i - x_{i+1}}, & x \in (x_i, x_{i+1}] \\ 0, & x \in [x_0, x_{i-1}) \cup (x_{i+1}, x_n] \end{cases} \quad (i = 1, 2, \cdots, n-1)$$

(5-41)

$$l_n(x) = \begin{cases} \frac{x - x_{n-1}}{x_n - x_{n-1}}, & x \in [x_{n-1}, x_n] \\ 0, & x \in [x_0, x_{n-1}) \end{cases}$$

类似于 Lagrange 插值多项式的构造, 函数

$$\varphi(x) = \sum_{i=0}^n y_i l_i(x) \quad (5-42)$$

就是所求的分段线插值函数. 它与式(5-40)表示同一个函数.

**例 5** 已知函数  $y = f(x) = \frac{1}{1+x^2}$ , 在  $[0, 5]$  上取等距节点  $x_i = 0 + i (i = 0, 1, \dots, 5)$ . 求分段线性插值函数, 并由此计算  $f(4.5)$  的近似值.

[解] 节点处函数值如下表

| $x_i$                     | 0        | 1        | 2        | 3        | 4        | 5        |
|---------------------------|----------|----------|----------|----------|----------|----------|
| $y_i = \frac{1}{1+x_i^2}$ | 1.000 00 | 0.500 00 | 0.200 00 | 0.100 00 | 0.058 82 | 0.038 46 |

由式(5-40), 在区间  $[4, 5]$  上分段线性插值函数为

$$\begin{aligned} \varphi(x) &= \frac{x-5}{4-5} \times 0.058\ 82 + \frac{x-4}{5-4} \times 0.038\ 46 \\ &= 0.038\ 46(x-4) - 0.058\ 82(x-5) \end{aligned}$$

将  $x = 4.5$  代入, 得

$$f(4.5) \approx \varphi(4.5) = 0.038\ 46 \times 0.5 + 0.058\ 82 \times 0.5 = 0.048\ 64$$

与精确值  $f(4.5) = 0.047\ 06$  比较, 结果是比较令人满意的.

分段线性插值的误差估计如下.

**定理 5.2** 如果  $f(x)$  在  $[a, b]$  上二阶连续可微, 则分段线性插值函数  $\varphi(x)$  的余项有以下估计

$$|R(x)| = |f(x) - \varphi(x)| \leq \frac{h^2}{8} M \quad (5-43)$$

其中  $h = \max_{0 \leq i \leq n-1} (x_{i+1} - x_i)$ ,  $M = \max_{a \leq x \leq b} |f''(x)|$ .

[证明] 因为在每个小区间  $[x_i, x_{i+1}] (i = 0, 1, \dots, n-1)$  上,  $\varphi(x)$  是  $f(x)$  的线性插值, 故由式(5-5), 对任意  $x \in [x_i, x_{i+1}]$  有

$$R(x) = f(x) - \varphi(x) = \frac{f''(\xi)}{2} (x - x_i)(x - x_{i+1}), \quad \xi \in (x_i, x_{i+1})$$

又因为

$$(x - x_i)(x - x_{i+1}) = \left(x - \frac{x_i + x_{i+1}}{2}\right)^2 - \frac{(x_i - x_{i+1})^2}{4} \leq 0$$

因而

$$\max_{x_i \leq x \leq x_{i+1}} |(x - x_i)(x - x_{i+1})| = \frac{(x_i - x_{i+1})^2}{4}$$

于是

$$|R(x)| \leq \frac{|f''(\xi)|}{2} \frac{(x_i - x_{i+1})^2}{4} \leq \frac{(x_i - x_{i+1})^2}{8} \max_{x_i \leq x \leq x_{i+1}} |f''(x)|$$

所以,对任意  $x \in [a, b]$ , 都有

$$|R(x)| \leq \max_{0 \leq i \leq n-1} \left\{ \frac{(x_i - x_{i+1})^2}{8} \max_{x_i \leq x \leq x_{i+1}} |f''(x)| \right\} \leq \frac{h^2}{8} M$$

分段线插值简便易行,定理 5.2 还表明,当节点加密时,分段线性插值的误差变小,收敛性有保证. 另一方面,在分段线性插值中,每个小区间上的插值函数只依赖于本段的节点值,因而每个节点只影响到节点邻近的一、二个小区间,计算过程中数据误差基本上不扩大,从而保证了节点数增加时插值过程的稳定性. 但分段线性插值函数仅在  $[a, b]$  上连续,一般地,在节点处插值函数不可微,这就不能满足有些工程技术问题的光滑度要求.

## § 5.4 埃尔米特 (Hermite) 插值

如果对插值函数,不仅要求它在节点处与函数同值,而且要求它与函数有相同的一阶、二阶甚至更高阶的导数值,这就是 Hermite 插值问题. 本节主要讨论在节点处插值函数与函数的值及一阶导数值均相等的 Hermite 插值.

### 5.4.1 Hermite 插值

设已知函数  $y = f(x)$  在  $n+1$  个互异节点  $x_0, x_1, \dots, x_n$  上的函数值  $y_i = f(x_i)$  ( $i = 0, 1, \dots, n$ ) 和导数值  $y'_i = f'(x_i)$  ( $i = 0, 1, \dots, n$ ), 要求一个至多  $2n+1$  次的多项式  $H(x)$ , 使得

$$H(x_i) = y_i, \quad H'(x_i) = y'_i \quad (i = 0, 1, \dots, n) \quad (5-44)$$

满足条件(5-44)的多项式  $H(x)$  称为 Hermite 插值多项式.

我们仍采用构造插值基函数的方法来求 Hermite 插值多项式. 可以设想,如果有两组函数  $h_i(x), H_i(x)$  ( $i = 0, 1, \dots, n$ ), 它们满足:

(1)  $h_i(x), H_i(x)$  ( $i = 0, 1, \dots, n$ ) 都是至多  $2n+1$  次多项式;

$$(2) h_i(x_j) = \begin{cases} 0, & j \neq i \\ 1, & j = i \end{cases}, h'_i(x_j) = 0 \quad (j = 0, 1, \dots, n) \quad (5-45)$$

$$H_i(x_j) = 0, \quad H'_i(x_j) = \begin{cases} 0, & j \neq i \\ 1, & j = i \end{cases} \quad (j = 0, 1, \dots, n)$$

则多项式

$$H(x) = \sum_{i=0}^n [y_i h_i(x) + y'_i H_i(x)]$$

必定满足插值条件式(5-44), 且次数不超过  $2n+1$ . 按条件式(5-45),  $h_i(x)$  在  $x_j (j \neq i)$  处函数值与导数值均为 0, 故它们应含因子  $(x-x_j)^2 (j \neq i)$ , 因此可以设为

$$h_i(x) = [a + b(x-x_i)] l_i^2(x) \quad (5-46)$$

其中  $l_i(x) = \omega_{n+1}(x)/(x-x_i)\omega'_{n+1}(x_i)$  为 Lagrange 插值基函数. 由条件式(5-45),  $h_i(x)$  还应满足

$$h_i(x_i) = 1, \quad h'_i(x_i) = 0$$

代入式(5-46), 得

$$h_i(x_i) = a l_i^2(x_i) = a = 1$$

$$\begin{aligned} h'_i(x_i) &= b l_i^2(x_i) + 2[a + b(x_i - x_i)] l_i(x_i) l'_i(x_i) \\ &= b + 2a l'_i(x_i) = 0 \end{aligned}$$

其解为  $b = -2l'_i(x_i)$ . 所以

$$h_i(x) = [1 - 2(x-x_i)l'_i(x_i)] l_i^2(x) \quad (i = 0, 1, \dots, n) \quad (5-47)$$

同理, 由于  $H_i(x)$  在  $x_j (j \neq i)$  处的函数值与导数值均为 0, 而  $H_i(x_i) = 0$ , 故可设

$$H_i(x) = c(x-x_i) l_i^2(x)$$

代入条件式(5-45)得

$$H'_i(x_i) = c l_i^2(x_i) = 1$$

于是  $c = 1$ , 因此

$$H_i(x) = (x-x_i) l_i^2(x) \quad (i = 0, 1, \dots, n) \quad (5-48)$$

所以 Hermite 插值多项式为

$$\begin{aligned} H(x) &= \sum_{i=0}^n [y_i h_i(x) + y'_i H_i(x)] \\ &= \sum_{i=0}^n \{ [1 - 2(x-x_i)l'_i(x_i)] l_i^2(x) y_i + (x-x_i) l_i^2(x) y'_i \} \quad (5-49) \end{aligned}$$

特别地, 当  $n=1$  时, 有

$$h_0(x) = \left( 1 + 2 \frac{x-x_0}{x_1-x_0} \right) \left( \frac{x-x_1}{x_0-x_1} \right)^2$$

$$h_1(x) = \left( 1 + 2 \frac{x-x_1}{x_0-x_1} \right) \left( \frac{x-x_0}{x_1-x_0} \right)^2$$

$$H_0(x) = (x-x_0) \left( \frac{x-x_1}{x_0-x_1} \right)^2$$

$$H_1(x) = (x - x_1) \left( \frac{x - x_0}{x_1 - x_0} \right)^2$$

所以,两个节点的三次 Hermite 插值多项式为

$$\begin{aligned} H(x) = & \left( 1 + 2 \frac{x - x_0}{x_1 - x_0} \right) \left( \frac{x - x_1}{x_0 - x_1} \right)^2 y_0 + \left( 1 + 2 \frac{x - x_1}{x_0 - x_1} \right) \left( \frac{x - x_0}{x_1 - x_0} \right)^2 y_1 + \\ & (x - x_0) \left( \frac{x - x_1}{x_0 - x_1} \right)^2 y'_0 + (x - x_1) \left( \frac{x - x_0}{x_1 - x_0} \right)^2 y'_1 \end{aligned} \quad (5-50)$$

### 5.4.2 误差估计

与 Lagrange 插值的误差估计相类似,可导出 Hermite 插值的误差估计.

**定理 5.3** 设  $x_0, x_1, \dots, x_n$  为区间  $[a, b]$  上的互异节点,  $H(x)$  为  $f(x)$  的过这组节点的  $2n+1$  次 Hermite 插值多项式. 如果  $f(x)$  在  $[a, b]$  上  $2n+2$  次连续可导, 则对任意  $x \in [a, b]$ , 插值余项为

$$R(x) = f(x) - H(x) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \omega_{n+1}^2(x), \quad \xi \in (a, b) \quad (5-51)$$

请读者仿定理 5.1 的导出自行证明此定理, 可令辅助函数为

$$\psi(t) = f(t) - H(t) - \frac{R(x)}{\omega_{n+1}^2(x)} \omega_{n+1}^2(t) \quad (5-52)$$

特别地, 三次 Hermite 插值余项为

$$R(x) = \frac{f^{(4)}(\xi)}{4!} (x - x_0)^2 (x - x_1)^2 \quad (5-53)$$

**定理 5.4** 设  $x_0, x_1, \dots, x_n$  为区间  $[a, b]$  上的互异节点,  $f(x) \in C^{(1)}[a, b]$ , 则满足插值条件式(5-44)的  $2n+1$  次 Hermite 多项式是唯一的.

[证明] 假设  $H(x)$  与  $\bar{H}(x)$  是满足相同插值要求的  $2n+1$  次 Hermite 多项式, 容易看出,  $\bar{H}(x)$  也是  $H(x)$  的  $2n+1$  次 Hermite 插值多项式. 由余项公式(5-51)

$$H(x) - \bar{H}(x) = \frac{H^{(2n+2)}(\xi)}{(2n+2)!} \omega_{n+1}^2(x) \quad (5-54)$$

因为  $H(x)$  为至多  $2n+1$  次多项式, 故  $H^{(2n+2)}(x) \equiv 0$ . 于是  $H(x) - \bar{H}(x) \equiv 0$ , 所以 Hermite 插值多项式是唯一的.

由此导出以下结论:

(1) 至多  $2n+1$  次的多项式在任意  $n+1$  个互异节点上的 Hermite 多项式就是其自身.

$$(2) \sum_{i=0}^n h_i(x) = 1$$

事实上,令  $f(x) = 1$ , 则有  $f(x_i) = 1, f'(x_i) = 0 (i = 0, 1, \dots, n)$ . 显然  $H(x) = \sum_{i=0}^n h_i(x)$  满足这组插值条件, 由定理 5.4 即得结论.

**例 6** 按下表求 Hermite 插值多项式

|        |   |   |
|--------|---|---|
| $x_i$  | 0 | 1 |
| $y_i$  | 0 | 1 |
| $y'_i$ | 3 | 9 |

[解] 将表中数据代入式(5-50), 即得 Hermite 插值多项式

$$\begin{aligned}
 H(x) &= \left(1 + 2 \frac{x-0}{1-0}\right) \left(\frac{x-1}{0-1}\right)^2 \times 0 + \left(1 + 2 \frac{x-1}{0-1}\right) \left(\frac{x-0}{1-0}\right)^2 \times 1 \\
 &\quad + (x-0) \left(\frac{x-1}{0-1}\right)^2 \times 3 + (x-1) \left(\frac{x-0}{1-0}\right)^2 \times 9 \\
 &= (3-2x)x^2 + 3x(x-1)^2 + 9x^2(x-1) \\
 &= 10x^3 - 12x^2 + 3x
 \end{aligned}$$

### 5.4.3 分段三次 Hermite 插值

当节点较多时, 为避免多项式次数过高而引起非节点处的偏离过大, 仍采用分段插值的方法.

给定函数表

|        |        |   |        |   |        |   |     |   |        |
|--------|--------|---|--------|---|--------|---|-----|---|--------|
| $x_i$  | $x_0$  | < | $x_1$  | < | $x_2$  | < | ... | < | $x_n$  |
| $y_i$  | $y_0$  |   | $y_1$  |   | $y_2$  |   | ... |   | $y_n$  |
| $y'_i$ | $y'_0$ |   | $y'_1$ |   | $y'_2$ |   | ... |   | $y'_n$ |

若把节点两两分段, 在每一小段上作三次 Hermite 插值, 就得到一个分段三次 Hermite 插值函数  $H(x)$ , 它满足:

(1)  $H(x_i) = y_i, H'(x_i) = y'_i \quad (i = 0, 1, \dots, n)$ ;

(2) 在每个小区间  $[x_i, x_{i+1}] (i = 0, 1, \dots, n-1)$  上,  $H(x)$  是三次多项式.

由式(5-50)可直接写出分段三次 Hermite 插值函数的分段表达式

$$H(x) = \left(1 + 2 \frac{x-x_i}{x_{i+1}-x_i}\right) \left(\frac{x-x_{i+1}}{x_i-x_{i+1}}\right)^2 y_i + \left(1 + 2 \frac{x-x_{i+1}}{x_i-x_{i+1}}\right) \left(\frac{x-x_i}{x_{i+1}-x_i}\right)^2 y_{i+1} +$$

$$\begin{aligned} & (x - x_i) \left( \frac{x - x_{i+1}}{x_i - x_{i+1}} \right)^2 y'_i + (x - x_{i+1}) \left( \frac{x - x_i}{x_{i+1} - x_i} \right)^2 y'_{i+1}, \\ & x \in [x_i, x_{i+1}] \quad (i = 0, 1, \dots, n-1) \end{aligned} \quad (5-55)$$

也可以通过构造基函数给出分段三次 Hermite 插值函数的表达式. 参照分段线性插值与 Hermite 插值基函数公式(5-47)和式(5-48), 可得出分段三次 Hermite 插值的基函数为

$$\begin{aligned} h_0(x) &= \begin{cases} \left( 1 + 2 \frac{x - x_0}{x_1 - x_0} \right) \left( \frac{x - x_1}{x_0 - x_1} \right)^2, & x \in [x_0, x_1] \\ 0, & x \in [x_1, x_n] \end{cases} \\ h_i(x) &= \begin{cases} \left( 1 + 2 \frac{x - x_i}{x_{i-1} - x_i} \right) \left( \frac{x - x_{i-1}}{x_i - x_{i-1}} \right)^2, & x \in [x_{i-1}, x_i] \\ \left( 1 + 2 \frac{x - x_i}{x_{i+1} - x_i} \right) \left( \frac{x - x_{i+1}}{x_i - x_{i+1}} \right)^2, & x \in (x_i, x_{i+1}] \quad (i = 1, \dots, n-1) \\ 0, & x \in [x_0, x_{i-1}) \cup (x_{i+1}, x_n] \end{cases} \end{aligned} \quad (5-56)$$

$$\begin{aligned} h_n(x) &= \begin{cases} \left( 1 + 2 \frac{x - x_n}{x_{n-1} - x_n} \right) \left( \frac{x - x_{n-1}}{x_n - x_{n-1}} \right)^2, & x \in [x_{n-1}, x_n] \\ 0, & x \in [x_0, x_{n-1}) \end{cases} \\ H_0(x) &= \begin{cases} (x - x_0) \left( \frac{x - x_1}{x_0 - x_1} \right)^2, & x \in [x_0, x_1] \\ 0, & x \in (x_1, x_n] \end{cases} \\ H_i(x) &= \begin{cases} (x - x_i) \left( \frac{x - x_{i-1}}{x_i - x_{i-1}} \right)^2, & x \in (x_{i-1}, x_i] \\ (x - x_i) \left( \frac{x - x_{i+1}}{x_i - x_{i+1}} \right)^2, & x \in (x_i, x_{i+1}] \quad (i = 1, \dots, n-1) \\ 0, & x \in [x_0, x_{i-1}) \cup (x_{i+1}, x_n] \end{cases} \quad (5-57) \\ H_n(x) &= \begin{cases} (x - x_n) \left( \frac{x - x_{n-1}}{x_n - x_{n-1}} \right)^2, & x \in [x_{n-1}, x_n] \\ 0, & x \in [x_0, x_{n-1}) \end{cases} \end{aligned}$$

分段三次 Hermite 插值函数为

$$H(x) = \sum_{i=0}^n [y_i h_i(x) + y'_i H_i(x)] \quad (5-58)$$

由余项公式(5-53)可以导出, 分段三次 Hermite 插值的误差有如下估计

$$|R(x)| = |f(x) - H(x)| \leq \frac{h^4}{384} \max_{a \leq x \leq b} |f^{(4)}(x)| \quad (5-59)$$

其中  $h = \max_{0 \leq i \leq n-1} (x_{i+1} - x_i)$

分段三次 Hermite 插值函数是插值区间上的光滑函数, 它与函数  $f(x)$  在节点处密合程度较好.

#### 5.4.4 Hermite 插值的一般形式

Hermite 插值问题的一般形式是: 已知函数  $f(x)$  在  $n+1$  个互异节点  $x_0, x_1, \dots, x_n$  处的函数值  $y_i = f(x_i)$  ( $i = 0, 1, \dots, n$ ) 及某些节点上的导数值  $y'_{i_k} = f'(x_{i_k})$  ( $k = 0, 1, \dots, m, m \leq n, 0 \leq i_k \leq n$ ), 求一个至多  $n+m+1$  次的多项式  $H(x)$ , 使得

$$\begin{aligned} H(x_i) &= y_i \quad (i = 0, 1, \dots, n) \\ H'(x_{i_k}) &= y'_{i_k} \quad (k = 0, 1, \dots, m) \end{aligned} \quad (5-60)$$

与前面的讨论类似, 可以证明满足条件 (5-60) 的 Hermite 插值多项式是存在唯一的, 且有

**定理 5.5** 设  $x_0, x_1, \dots, x_n$  为区间  $[a, b]$  上的互异节点,  $H(x)$  为  $f(x)$  的满足条件式 (5-60) 的 Hermite 插值多项式. 若  $f(x) \in C^{(n+m+2)}[a, b]$ , 则对  $\forall x \in [a, b]$ , 有

$$R(x) = f(x) - H(x) = \frac{f^{(n+m+2)}(\xi)}{(m+n+2)!} \omega_{n+1}(x) \prod_{k=0}^m (x - x_{i_k}) \quad (5-61)$$

[证明] 对  $\forall x \in [a, b], x \neq x_i (i = 0, 1, \dots, n)$ , 令

$$\psi(t) = f(t) - H(t) - \frac{R(x)}{\omega_{n+1}(x) \prod_{k=0}^m (x - x_{i_k})} \omega_{n+1}(t) \prod_{k=0}^m (t - x_{i_k})$$

显然,  $\psi(t) \in C^{(n+m+2)}[a, b]$  且  $\psi(x) = 0, \psi(x_i) = 0 (i = 0, 1, \dots, n)$ . 由罗尔定理,  $\psi'(t)$  在  $[a, b]$  上至少有  $n+1$  个异于节点  $x_i (i = 0, 1, \dots, n)$  与  $x$  的零点. 由插值条件  $f'(x_{i_k}) = H'(x_{i_k})$  可得

$$\begin{aligned} \psi'(x_{i_l}) &= f'(x_{i_l}) - H'(x_{i_l}) - \frac{R(x)}{\omega_{n+1}(x) \prod_{k=0}^m (x - x_{i_k})} \\ &\quad \left[ \omega'_{n+1}(x_{i_l}) \prod_{k=0}^m (x_{i_l} - x_{i_k}) + \omega_{n+1}(x_{i_l}) \prod_{\substack{k=0 \\ k \neq l}}^m (x_{i_l} - x_{i_k}) \right] \\ &= 0 \quad (l = 0, 1, \dots, m) \end{aligned}$$

因此  $\psi'(t)$  在  $[a, b]$  上至少有  $n+m+2$  个零点. 反复用罗尔定理可得出,



$\psi^{(n+m+2)}(t)$  在  $(a, b)$  上至少有一个零点, 即存在  $\xi \in (a, b)$ , 使得

$$\psi^{(n+m+2)}(\xi) = f^{(n+m+2)}(\xi) - \frac{R(x)}{\omega_{n+1}(x) \prod_{k=0}^m (x - x_{i_k})} (n+m+2)! = 0$$

于是

$$R(x) = \frac{f^{(n+m+2)}(\xi)}{(n+m+2)!} \omega_{n+1}(x) \prod_{k=0}^m (x - x_{i_k}), \quad \xi \in (a, b)$$

当  $x = x_i (i=0, 1, \dots, n)$  时, 上式自然成立. 所以, 对任意  $x \in [a, b]$ , 式(5-61)都成立.

一般形式的 Hermite 插值多项式的求法仍可采用构造基函数的方法, 但用以下方法计算往往更简便. 设

$$H(x) = \varphi_n(x) + P_m(x) \omega_{n+1}(x)$$

其中  $\varphi_n(x)$  为  $f(x)$  的满足条件  $\varphi_n(x_i) = y_i (i=0, 1, \dots, n)$  的  $n$  次插值多项式,

$\omega_{n+1}(x) = \prod_{j=0}^n (x - x_j)$ ,  $P_m(x)$  为待定的  $m$  次多项式, 显然

$$H(x_i) = \varphi_n(x_i) + P_m(x_i) \omega_{n+1}(x_i) = y_i$$

于是由导数值插值条件, 有

$$H'(x_{i_k}) = \varphi'_n(x_{i_k}) + P'_m(x_{i_k}) \omega'_{n+1}(x_{i_k}) = y'_{i_k}$$

由此确定  $P_m(x)$ , 即可求得 Hermite 插值多项式, 这种方法称为降阶法.

**例 7** 按下表求 Hermite 插值多项式

|        |   |   |   |
|--------|---|---|---|
| $x_i$  | 0 | 1 | 2 |
| $y_i$  | 0 | 1 | 1 |
| $y'_i$ | 0 | 1 |   |

[解] 解法一. 容易求出满足函数值插值条件的插值多项式为

$$\varphi_2(x) = x - \frac{1}{2}x(x-1) = \frac{1}{2}x(3-x)$$

设

$$H(x) = \varphi_2(x) + (a+bx)x(x-1)(x-2)$$

求导得

$$\begin{aligned} H'(x) &= \frac{3}{2} - x + bx(x-1)(x-2) \\ &\quad + (a+bx)[(x-1)(x-2) + x(x-2) + x(x-1)] \end{aligned}$$

代入导数值插值条件得

$$H'(0) = \frac{3}{2} + 2a = 0, \quad H'(1) = \frac{1}{2} + \left(b - \frac{3}{4}\right)(-1) = 1$$

求解得  $a = -\frac{3}{4}, b = \frac{1}{4}$ , 故所求 Hermite 插值多项式为

$$H(x) = \frac{1}{2}x(3-x) + \left(\frac{1}{4}x - \frac{3}{4}\right)x(x-1)(x-2) = \frac{x^2}{4}(x-3)^2$$

解法二. 因为  $x=0$  为二阶零点, 故可直接设插值多项式为

$$H(x) = x^2(ax^2 + bx + c)$$

代入插值条件  $H(1) = H(2) = H'(1) = 1$ , 得方程组

$$a + b + c = 1, \quad 16a + 8b + 4c = 1, \quad 4a + 3b + 2c = 1$$

其解为  $a = \frac{1}{4}, b = -\frac{3}{2}, c = \frac{9}{4}$ , 所求插值多项式为

$$H(x) = x^2\left(\frac{x^2}{4} - \frac{3}{2}x + \frac{9}{4}\right) = \frac{x^2}{4}(x-3)^2$$

## § 5.5 样条插值

许多工程技术中提出的计算问题对插值函数的光滑性有较高要求, 如飞机的机翼外形, 内燃机的进、排气门的凸轮曲线, 都要求曲线具有较高的光滑程度, 不仅要连续, 而且要有连续的曲率, 即二阶导数连续. 这就导致了样条插值的产生.

### 5.5.1 样条函数的概念

所谓样条(spline)本来是工程设计中使用的一种绘图工具, 它是富有弹性的细木条或细金属条. 绘图员利用它把一些已知点连接成一条光滑曲线(称为样条曲线), 并使连接点处有连续的曲率. 三次样条插值就是由此抽象出来的.

数学上将具有一定光滑性的分段多项式称为样条函数. 具体地说, 给定区间  $[a, b]$  的一个分划

$$\Delta: a = x_0 < x_1 < \cdots < x_{n-1} < x_n = b$$

如果函数  $S(x)$  满足:

- (1) 在每个小区间  $[x_i, x_{i+1}]$  ( $i=0, 1, \cdots, n-1$ ) 上  $S(x)$  是  $m$  次多项式;
- (2)  $S(x)$  在  $[a, b]$  上具有  $m-1$  阶连续导数,

则称  $S(x)$  为关于分划  $\Delta$  的  $m$  次样条函数, 其图形为  $m$  次样条曲线.

显然, 折线是一次样条曲线. 绘图员用“样条”绘出的图形为三次样条曲线, 这也是最常用的样条函数.

### 5.5.2 三次样条插值

利用样条函数进行插值,即取插值函数为样条函数,称为样条插值.例如,分段线性插值是一次样条插值.本节只介绍三次样条插值,即已知函数  $y=f(x)$  在区间  $[a,b]$  上的  $n+1$  个节点

$$a = x_0 < x_1 < \cdots < x_n = b$$

上的值  $y_i=f(x_i)$  ( $i=0,1,\cdots,n$ ),求插值函数  $S(x)$ ,使得

$$(1) S(x_i)=y_i \quad (i=0,1,\cdots,n);$$

(2) 在每个小区间  $[x_j, x_{j+1}]$  ( $j=0,1,\cdots,n-1$ ) 上  $S(x)$  是三次多项式,记为  $S_j(x)$ ;

$$(3) S(x) \text{ 在 } [a,b] \text{ 上二阶连续可微,}$$

则函数  $S(x)$  称为  $f(x)$  的三次样条插值函数.

可以证明,在一定的边条件下,三次样条插值问题的解是存在唯一的.下面讨论三次样条插值函数的求法.

由于  $S(x)$  在每个小区间上都是三次多项式,故共有  $4n$  个参数.为简化计算,取节点上的导数值或二阶导数值为参数,来导出三次样条插值函数的表达式.

#### 1. 以节点处的二阶导数值为参数的三次样条插值函数

设  $S''(x_i)=M_i$  ( $i=0,1,\cdots,n$ ). 因为在小区间  $[x_j, x_{j+1}]$  上  $S(x)=S_j(x)$  是三次多项式,故  $S_j''(x)$  为线性函数,由 Lagrange 插值公式得

$$S_j''(x) = M_{j+1} \frac{x-x_j}{h_j} - M_j \frac{x-x_{j+1}}{h_j} \quad (5-62)$$

其中  $h_j=x_{j+1}-x_j$ . 将上式积分两次,并代入插值条件  $S_j(x_j)=y_j, S_j(x_{j+1})=y_{j+1}$ ,得

$$\begin{aligned} S_j(x) = & M_{j+1} \frac{(x-x_j)^3}{6h_j} - M_j \frac{(x-x_{j+1})^3}{6h_j} + \left( y_{j+1} - \frac{M_{j+1}h_j^2}{6} \right) \frac{x-x_j}{h_j} \\ & - \left( y_j - \frac{M_jh_j^2}{6} \right) \frac{x-x_{j+1}}{h_j} \quad (j=0,1,\cdots,n-1) \end{aligned} \quad (5-63)$$

求导得

$$\begin{aligned} S_j'(x) = & \frac{M_{j+1}}{2h_j}(x-x_j)^2 - \frac{M_j}{2h_j}(x-x_{j+1})^2 + \frac{1}{h_j} \left( y_{j+1} - \frac{M_{j+1}h_j^2}{6} \right) - \frac{1}{h_j} \left( y_j - \frac{M_jh_j^2}{6} \right) \\ = & \frac{M_{j+1}}{2h_j}(x-x_j)^2 - \frac{M_j}{2h_j}(x-x_{j+1})^2 + \frac{y_{j+1}-y_j}{h_j} - \frac{h_j}{6}(M_{j+1}-M_j) \\ & (j=0,1,\cdots,n-1) \end{aligned} \quad (5-64)$$

因为  $S(x)$  在节点处一阶导数连续,故应有

$$S'(x_j + 0) = S'_j(x_j + 0) = S'(x_j - 0) = S'_{j-1}(x_j - 0) \quad (j = 1, 2, \dots, n-1)$$

即

$$-\frac{h_j}{2}M_j + \frac{y_{j+1} - y_j}{h_j} - \frac{h_j}{6}(M_{j+1} - M_j) = \frac{h_{j-1}}{2}M_j + \frac{y_j - y_{j-1}}{h_{j-1}} - \frac{h_{j-1}}{6}(M_j - M_{j-1})$$

化简得

$$\frac{h_{j-1}}{6}M_{j-1} + \frac{h_{j-1} + h_j}{3}M_j + \frac{h_j}{6}M_{j+1} = \frac{y_{j+1} - y_j}{h_j} - \frac{y_j - y_{j-1}}{h_{j-1}} \quad (j = 1, \dots, n-1) \quad (5-65)$$

令

$$\begin{aligned} \alpha_j &= \frac{h_{j-1}}{h_{j-1} + h_j} \\ \beta_j &= 1 - \alpha_j = \frac{h_j}{h_{j-1} + h_j} \\ c_j &= 6 \left( \frac{y_{j+1} - y_j}{h_j} - \frac{y_j - y_{j-1}}{h_{j-1}} \right) / (h_{j-1} + h_j) \end{aligned} \quad (5-66)$$

则式(5-65)改写成

$$\alpha_j M_{j-1} + 2M_j + \beta_j M_{j+1} = c_j \quad (j = 1, 2, \dots, n-1) \quad (5-67)$$

式(5-67)给出了含  $n+1$  个参数  $M_0, M_1, \dots, M_n$  的  $n-1$  个方程,称为三次样条的  $M$  关系式,或按其力学意义,称为三弯矩方程. 为完全确定这些参数,需要根据问题的具体情况,在区间的两个端点处给出条件,称为边界条件. 常用的边界条件有以下三种:

(1) 给定两端点处的导数值  $S'(a) = y'_0, S'(b) = y'_n$ . 特别地,当  $y'_0 = y'_n = 0$  时,样条曲线在端点处呈水平状态.

(2) 给定两端点处的二阶导数值  $S''(a) = y''_0, S''(b) = y''_n$ . 特别地,当  $y''_0 = y''_n = 0$  时,称为自然边界条件.

(3) 如果  $f(x)$  是以  $b-a$  为周期的周期函数,则  $S(x)$  也应是具有同样周期的周期函数,在端点处需满足

$$S'(a+0) = S'(b-0), \quad S''(a+0) = S''(b-0)$$

将这三种边界条件中的任一种与三弯矩方程(5-67)联立,即可求出参数  $M_i (i=0, 1, \dots, n)$ .

如果问题要求  $S(x)$  满足边界条件(1),由式(5-64)得

$$S'(a) = y'_0 = -\frac{h_0}{2}M_0 + \frac{y_1 - y_0}{h_0} - \frac{h_0}{6}(M_1 - M_0)$$

$$S'(b) = y'_n = \frac{h_{n-1}}{2}M_n + \frac{y_n - y_{n-1}}{h_{n-1}} - \frac{h_{n-1}}{6}(M_n - M_{n-1})$$

化简得

$$\begin{aligned} 2M_0 + M_1 &= \frac{6}{h_0} \left( \frac{y_1 - y_0}{h_0} - y'_0 \right) \\ M_{n-1} + 2M_n &= \frac{6}{h_{n-1}} \left( y'_n - \frac{y_n - y_{n-1}}{h_{n-1}} \right) \end{aligned} \quad (5-68)$$

式(5-68)和式(5-67)联立,即得关于参数  $M_i (i=0, 1, \dots, n)$  的  $n+1$  阶线性方程组,其矩阵形式为

$$\begin{bmatrix} 2 & \beta_0 & & & \\ \alpha_1 & 2 & \beta_1 & & \\ & \alpha_2 & 2 & \beta_2 & \\ & & \ddots & \ddots & \ddots \\ & & & \alpha_{n-1} & 2 & \beta_{n-1} \\ & & & & \alpha_n & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix}$$

其中  $\beta_0 = 1, \alpha_n = 1, c_0 = \frac{6}{h_0} \left( \frac{y_1 - y_0}{h_0} - y'_0 \right), c_n = \frac{6}{h_{n-1}} \left( y'_n - \frac{y_n - y_{n-1}}{h_{n-1}} \right)$ . 这是三对角方程组,可用追赶法求解.

如果问题要求  $S(x)$  满足边界条件(2),即

$$M_0 = y''_0, \quad M_n = y''_n$$

此时方程组(5-67)中实际只有  $n-1$  个未知数,其矩阵形式的

$$\begin{bmatrix} 2 & \beta_1 & & & \\ \alpha_2 & 2 & \beta_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \alpha_{n-2} & 2 & \beta_{n-2} \\ & & & \alpha_{n-1} & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} c_1 - \alpha_1 y''_0 \\ c_2 \\ \vdots \\ c_{n-2} \\ c_{n-1} - \beta_{n-1} y''_n \end{bmatrix} \quad (5-69)$$

这仍是三对角方程组.

如果问题要求  $S(x)$  满足周期边界条件(3),将边界条件  $S'(x_0 + 0) = S'(x_n - 0), M_0 = M_n$  与方程组(5-67)联立,得到  $n$  阶方程组

$$\begin{bmatrix} 2 & \beta_1 & & & \alpha_1 \\ \alpha_2 & 2 & \beta_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \alpha_{n-1} & 2 & \beta_{n-1} \\ \beta_n & & & \alpha_n & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix} \quad (5-70)$$

其中  $\alpha_n = \frac{h_{n-1}}{h_0 + h_{n-1}}, \beta_n = \frac{h_0}{h_0 + h_{n-1}}, c_n = \frac{6}{h_0 + h_{n-1}} \left( \frac{y_1 - y_0}{h_0} - \frac{y_n - y_{n-1}}{h_{n-1}} \right)$ . 其求解过程

类似于求解三对角方程组的追赶法.

容易验证,这三种情形所得到的方程组的系数矩阵均为严格对角占优阵,因此方程组有唯一解.将解代入式(5-63),即得三次样条插值函数.

## 2. 以节点处的导数值为参数的三次样条插值函数

类似地,也可以节点处的导数值为参数来导出三次样条插值函数的表达式.

设  $S'(x_j) = m_j (j=0, 1, \dots, n)$ . 按分段三次 Hermite 插值公式(5-55),  $S(x)$  可表示成

$$S_j(x) = \left(1 + 2 \frac{x - x_j}{h_j}\right) \left(\frac{x - x_{j+1}}{h_j}\right)^2 y_j + \left(1 - 2 \frac{x - x_{j+1}}{h_j}\right) \left(\frac{x - x_j}{h_j}\right)^2 y_{j+1} + (x - x_j) \left(\frac{x - x_{j+1}}{h_j}\right)^2 m_j + (x - x_{j+1}) \left(\frac{x - x_j}{h_j}\right)^2 m_{j+1}, \quad x \in [x_j, x_{j+1}] \quad (5-71)$$

由二阶导数在节点处连续

$$S_j''(x_j + 0) = S_{j-1}''(x_j - 0) \quad (j = 1, 2, \dots, n-1)$$

可导出关于参数  $m_j$  的方程组

$$\beta_j m_{j-1} + 2m_j + \alpha_j m_{j+1} = d_j \quad (j = 1, 2, \dots, n-1) \quad (5-72)$$

其中

$$\begin{aligned} \alpha_j &= \frac{h_{j-1}}{h_{j-1} + h_j} \\ \beta_j &= 1 - \alpha_j = \frac{h_j}{h_{j-1} + h_j} \\ d_j &= 3 \left[ \frac{\beta_j}{h_{j-1}} (y_j - y_{j-1}) + \frac{\alpha_j}{h_j} (y_{j+1} - y_j) \right] \end{aligned} \quad (5-73)$$

式(5-72)称为三次样条的  $m$  关系式,或按其力学意义称为三转角方程. 方程组(5-72)含有  $n+1$  个未知数,  $n-1$  个方程,与对  $M$  关系式的讨论类似,增加边界条件(1),(2)后,可得关于参数  $m_j$  的三对角方程组;增加周期边界条件(3)后,得  $n$  阶方程组. 这些方程组的系数矩阵均非奇异,故方程组有唯一解. 解出参数  $m_j (j=0, 1, \dots, n)$  后代入式(5-71),即得三次样条插值函数.

**例 8** 已知函数表

|       |   |   |   |    |
|-------|---|---|---|----|
| $j$   | 0 | 1 | 2 | 3  |
| $x_j$ | 0 | 1 | 2 | 3  |
| $y_j$ | 0 | 2 | 3 | 16 |

求满足边界条件  $y'(0) = 1, y'(3) = 0$  的三次样条插值函数.

[解] 如果用三弯矩方程求解, 由已知  $h_j = 1 (j = 0, 1, 2)$ , 按式(5-66)和式(5-68)计算方程组的系数及右端项, 结果如下:

| $j$ | $\alpha_j$ | $\beta_j$ | $c_j$ |
|-----|------------|-----------|-------|
| 0   |            | 1         | 6     |
| 1   | 1/2        | 1/2       | -3    |
| 2   | 1/2        | 1/2       | 36    |
| 3   | 1          |           | -78   |

将上述数据代入式(5-67)和式(5-68)得方程组

$$\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1/2 & 2 & 1/2 & 0 \\ 0 & 1/2 & 2 & 1/2 \\ 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ M_2 \\ M_3 \end{bmatrix} = \begin{bmatrix} 6 \\ -3 \\ 36 \\ -78 \end{bmatrix}$$

其解为  $M_0 = \frac{28}{3}, M_1 = -\frac{38}{3}, M_2 = \frac{106}{3}, M_3 = \frac{170}{3}$ . 将此解代入式(5-63), 得

$$\begin{aligned} S_0(x) &= -\frac{38}{3} \times \frac{(x-0)^3}{6} - \frac{28}{3} \times \frac{(x-1)^3}{6} + \\ &\quad \left(2 + \frac{38}{3 \times 6}\right)(x-0) - \left(0 - \frac{28}{3 \times 6}\right)(x-1) \\ &= -\frac{19}{9}x^3 - \frac{14}{9}(x-1)^3 + \frac{37}{9}x + \frac{14}{9}(x-1) \\ &= \frac{x}{3}(-11x^2 + 14x + 3), \quad x \in [0, 1] \end{aligned}$$

$$S_1(x) = \frac{1}{3}(24x^3 - 91x^2 + 108x - 35), \quad x \in [1, 2]$$

$$S_2(x) = \frac{1}{3}(-46x^3 + 329x^2 - 732x + 525), \quad x \in [2, 3]$$

如果用三转角方程求解, 由式(5-73)

$$\alpha_1 = \alpha_2 = \beta_1 = \beta_2 = \frac{1}{2}$$

$$d_1 = 3\left[\frac{1}{2}(y_1 - y_0) + \frac{1}{2}(y_2 - y_1)\right] = \frac{9}{2}$$

$$d_2 = 3\left[\frac{1}{2}(y_2 - y_1) + \frac{1}{2}(y_3 - y_2)\right] = 21$$

将上述数据及  $m_0 = 1, m_3 = 0$  代入式(5-72), 得三转角方程

$$\frac{1}{2} + 2m_1 + \frac{1}{2}m_2 = \frac{9}{2}, \quad \frac{1}{2}m_1 + 2m_2 = 21$$

其解为  $m_1 = -\frac{2}{3}, m_2 = \frac{32}{3}$ . 将此解代入式(5-71), 即得三次样条插值函数的分段表示式

$$S(x) = \begin{cases} \frac{x}{3}(-11x^2 + 14x + 3), & 0 \leq x \leq 1 \\ \frac{1}{3}(24x^3 - 91x^2 + 108x - 35), & 1 \leq x \leq 2 \\ \frac{1}{3}(-46x^3 + 329x^2 - 732x + 525), & 2 \leq x \leq 3 \end{cases}$$

比较上面两种解法, 对第一种边界条件, 用三转角方程计算较简便.

小结计算三次样条插值函数的步骤为:

- (1) 根据给定的点  $(x_j, y_j)$  及相应的边界条件计算  $\alpha_j, \beta_j, c_j$  或  $d_j$ . 一般地, 对第一种边界条件用三转角方程, 对第二种边界条件用三弯矩方程较为简便;
- (2) 解方程组(5-67)或(5-72), 求出参数  $\{M_j\}$  或  $\{m_j\}$ ;
- (3) 将求得的参数代入式(5-63)或式(5-71), 即得三次样条插值函数  $S(x)$  的分段表示式.

三次样条插值函数有较好的收敛性.

**定理 5.6** 设  $f(x)$  在  $[a, b]$  上四次连续可微,  $S(x)$  为  $f(x)$  在  $[a, b]$  上的三次样条插值函数, 则

$$|f^{(l)}(x) - S^{(l)}(x)| \leq c_l h^{4-l} \max_{a \leq x \leq b} |f^{(4)}(x)|, \quad x \in [a, b] \quad (l = 0, 1, 2, 3) \quad (5-74)$$

其中  $h = \max_{0 \leq j \leq n-1} h_j, c_0 = \frac{5}{384}, c_1 = \frac{1}{24}, c_2 = \frac{3}{8}, c_3 = \frac{\beta + \beta^{-1}}{2}, \beta = \max_{0 \leq j \leq n-1} h_j / \min_{0 \leq j \leq n-1} h_j$  称为分划比.

证明略.

定理 5.6 表明, 当分划小区间的长度趋于零时,  $S(x)$  及其一至二阶导数分别一致收敛到  $f(x)$  及其一至二阶导数, 因而, 为提高精度只需加密分划节点, 不需要提高样条函数的次数. 由于样条插值有这样好的性质, 因此它应用十分广泛, 在外形设计及计算机辅助设计的许多领域中, 都是十分有效的数学工具.

简单地比较前面所述的几种插值方法, 它们各有优缺点. 多项式插值宜用较低次多项式, 例如  $n \leq b$ , 当次数较高时, 收敛性与稳定性均较差. 分段线性插值或分段低次插值具有较好的稳定性与收敛性, 且计算简便, 但光滑程度较差. 样条插值克服了这一缺点, 并具有良好的收敛性, 但其计算复杂, 稳定性不如分段



插值. 进一步的讨论可参看有关资料.

MATLAB 软件中提供了若干个插值函数, 如一维插值函数 `interp1()`、多项式拟合函数 `polyfit()`、三次样条插值 `csape()` 等. 一维插值函数 `interp1()` 的调用格式为

```
>> yy = interp1( xi, yi, xx, 'method' )
```

其中 `xi, yi` 表示插值数据, 是两个数组; `xx` 为待计算函数值的点组成的数组. 插值方法一般默认为 'linear', 采用分段线性插值; 也可选 'cubic' 或 'pchip' 采用分段三次 Hermite 插值、'spline' 采用分段三次样条插值.

**例 9** 设数据取自函数  $f(x) = (x^2 - 5x + 3)e^{-2x} \sin x$ , 试根据生成的数据进行分段线性、三次 Hermite 插值和三次样条插值, 作出近似曲线.

[解] 根据给出的函数, 直接生成插值数据:

```
>> xi = 0:0.2:1; yi = (xi.^2 - 5 * xi + 3) . * exp( -2 * xi) . * sin( xi );
```

为得到较光滑的曲线, 构造更细密的计算点:

```
>> xx = 0:0.04:1;
```

进行插值计算, 并作图:

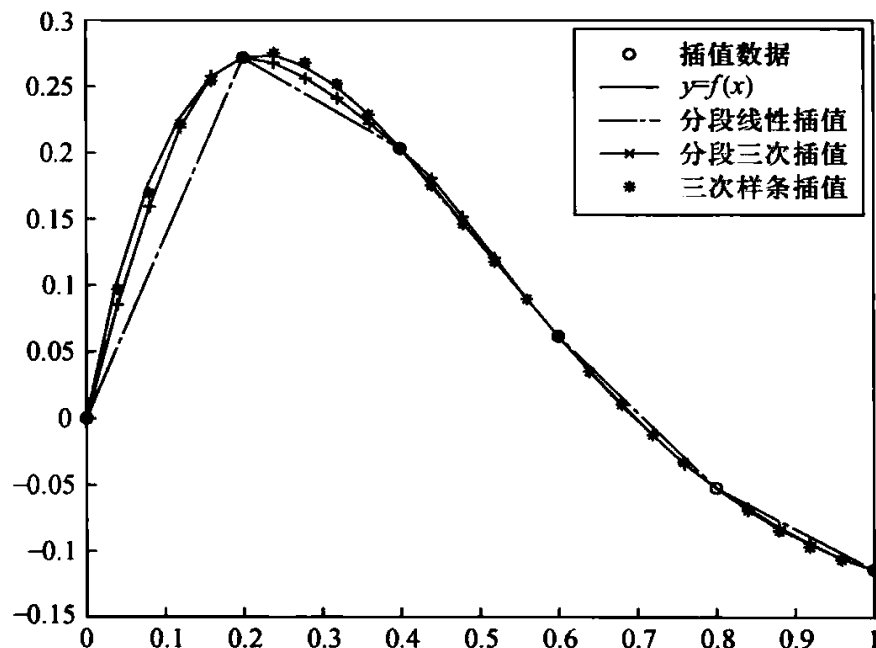


图 5-7

```
>> y0 = (xx.^2 - 5 * xx + 3) . * exp( -2 * xx) . * sin( xx );
>> y1 = interp1( xi, yi, xx );
>> y2 = interp1( xi, yi, xx, 'cubic' );
```

```
>> y3 = interp1 (xi, yi, xx, 'spline');
>> plot (xi, yi, 'o', xx, y0, 'k-', xx, y1, 'r-', xx, y2, 'b-', xx, y3, 'm-')
>> legend('插值数据', 'y = f(x)', '分段线性插值', '分段三次插值', '三次样条插值')
```

由计算结果知分段线性插值的近似效果较粗糙, 采用 'cubic' 和 'spline' 的近似曲线更接近于原函数曲线. 事实上, 应用样条插值得到的近似值十分逼近精确值.

## § 5.6 二维插值

在科学研究和生产实际问题中, 经常遇到二维或更高维的插值问题, 如数字地形的模拟、飞机、汽车外形的设计等. 本节将模仿一维插值函数方法, 讨论矩形区域上的简单插值.

二维插值的一般提法是: 给定了二元函数  $u(x, y)$  的一组离散值

$$u_{ij} = u(x_i, y_j) \quad (i = 0, 1, \dots, n; j = 0, 1, \dots, m)$$

要求构造关于  $x$  为  $n$  次、 $y$  为  $m$  次的二元多项式

$$P_{n,m}(x, y) = \sum_{i=0}^n \sum_{j=0}^m a_{ij} x^i y^j \quad (5-75)$$

满足插值条件

$$P_{n,m}(x_i, y_j) = u_{ij} = u(x_i, y_j) \quad (i = 0, 1, \dots, n; j = 0, 1, \dots, m) \quad (5-76)$$

我们称二元多项式  $P_{n,m}(x, y)$  为插值多项式,  $(x_i, y_j) (i = 0, 1, \dots, n; j = 0, 1, \dots, m)$  为插值节点.

### 5.6.1 Lagrange 插值

设平面矩形区域  $D = \{(x, y) \mid a \leq x \leq b, c \leq y \leq d\}$ , 相应  $x, y$  的剖分为

$$\Delta_x: a = x_0 < x_1 < \dots < x_n = b$$

$$\Delta_y: c = y_0 < y_1 < \dots < y_m = d$$

则

$$\Delta = \Delta_x \times \Delta_y = \{(x_i, y_j) : 0 \leq i \leq n, 0 \leq j \leq m\} \quad (5-77)$$

构成  $D$  的一个剖分节点.

根据一维 Lagrange 插值多项式的方法, 若关于  $x$  为  $n$  次多项式  $l_i(x)$  和关于  $y$  为  $m$  次的多项式  $\tilde{l}_j(y)$  分别是关于节点  $\Delta_x$  和  $\Delta_y$  的 Lagrange 插值基函数, 那么关于  $x$  为  $n$  次、 $y$  为  $m$  次的二元多项式  $l_{ij}(x, y) = l_i(x) \tilde{l}_j(y)$  满足

$$l_{ij}(x_s, y_t) = l_i(x_s) \tilde{l}_j(y_t) = \delta_{is} \delta_{jt} = \begin{cases} 1, & s = i, t = j \\ 0, & s \neq i \text{ 或 } t \neq j \end{cases} \\ (i, s = 0, 1, \dots, n; j, t = 0, 1, \dots, m)$$

且

$$P_{n,m}(x, y) = \sum_{i=0}^n \sum_{j=0}^m u(x_i, y_j) l_{ij}(x, y) \quad (5-78)$$

就是满足插值条件(5-76)的关于  $x$  为  $n$  次、 $y$  为  $m$  次的二元插值多项式。

可以证明:满足插值条件(5-76)的形如式(5-78)的插值多项式是唯一的,而且当  $u(x, y)$  关于  $x$  有  $n+1$  阶连续导数,关于  $y$  有  $m+1$  阶连续导数,则有

$$\max_{\substack{a \leq x \leq b \\ c \leq y \leq d}} |u(x, y) - P_{n,m}(x, y)| = O\left\{ \max_{\substack{1 \leq i \leq n \\ 1 \leq j \leq m}} (h_i^{n+1}, h_j^{m+1}) \right\} \quad (5-79)$$

详细可参阅[24]。

若给定的节点很多,直接求一个二元插值多项式,必然使得多项式次数很高,而高次多项式容易产生 Runge 现象,近似效果并不一定好。因此经常采用分片低次二元多项式插值,即将区域分成若干个小矩形或小三角形区域,然后在每一小区域上求一个次数较低的二元插值多项式,再将它们在小区域的边界上连接起来。

### 5.6.2 分片双线性插值

分片双线性插值是一维分段线性插值的直接推广。在矩形区域  $D$  上

$$P_{1,1}(x, y) = \sum_{i=0}^n \sum_{j=0}^m u(x_i, y_j) \varphi_{ij}(x, y) \quad (5-80)$$

其中  $\varphi_{ij}(x, y) = l_i(x) \tilde{l}_j(y)$ ,  $l_i(x)$ ,  $\tilde{l}_j(y)$ ,  $(0 \leq i \leq n, 0 \leq j \leq m)$  由式(5-41)给出。

若  $D$  上典型小区域  $A_{ij}$  的顶点为

$$\{(x_i, y_j), (x_{i+1}, y_j), (x_i, y_{j+1}), (x_{i+1}, y_{j+1})\}, \quad 0 \leq i \leq n-1, \quad 0 \leq j \leq m-1$$

类似于式(5-40),分片插值多项式  $P_{1,1}(x, y)$  在小区域  $A_{ij}$  上的具体形式为

$$P_{1,1}(x, y) = u_{ij} \frac{(x - x_{i+1})(y - y_{j+1})}{(x_i - x_{i+1})(y_j - y_{j+1})} + u_{i+1,j} \frac{(x - x_i)(y - y_{j+1})}{(x_{i+1} - x_i)(y_j - y_{j+1})} + \\ u_{ij+1} \frac{(x - x_{i+1})(y - y_j)}{(x_i - x_{i+1})(y_{j+1} - y_j)} + u_{i+1,j+1} \frac{(x - x_i)(y - y_j)}{(x_{i+1} - x_i)(y_{j+1} - y_j)}, \quad (5-81) \\ (x, y) \in A_{ij}, \quad 0 \leq i \leq n-1, \quad 0 \leq j \leq m-1$$

由于在各个小矩形区域的边界上分别  $x$  固定或  $y$  固定,分片双线性插值是一条直线,相邻小矩形区域上的分片插值在共有的边界上有两个公共的端点,从而是两条重合的直线。因此分片双线性插值在连接处是连续的,但不能保证光

滑性.

### 5.6.3 分片双三次 Hermite 插值

如果希望分片插值多项式在连接处有连续的偏导数,我们考虑带有导数插值条件的分片双三次 Hermite 插值.

设平面矩形区域  $D = \{(x, y) \mid a \leq x \leq b, c \leq y \leq d\}$  的剖分如(5-77),在节点  $(x_i, y_j)$  处给出数据

$$\begin{aligned} u_{ij} &= u(x_i, y_j), \quad u'_{x,ij} = \frac{\partial u(x_i, y_j)}{\partial x}, \quad u'_{y,ij} = \frac{\partial u(x_i, y_j)}{\partial y}, \\ u''_{xy,ij} &= \frac{\partial^2 u(x_i, y_j)}{\partial x \partial y} \quad (0 \leq i \leq n, 0 \leq j \leq m) \end{aligned} \quad (5-82)$$

求矩形区域  $D$  上的分片双三次多项式  $H_{3,3}(x, y)$  满足

$$\begin{cases} H_{3,3}(x_i, y_j) = u_{ij} \\ \frac{\partial H_{3,3}(x_i, y_j)}{\partial x} = u'_{x,ij} \\ \frac{\partial H_{3,3}(x_i, y_j)}{\partial y} = u'_{y,ij} \\ \frac{\partial^2 H_{3,3}(x_i, y_j)}{\partial x \partial y} = u''_{xy,ij} \end{cases} \quad (0 \leq i \leq n, 0 \leq j \leq m) \quad (5-83)$$

满足条件(5-83)的多项式称为分片双三次 Hermite 插值.

利用式(5-56),式(5-57)给出的一维分段三次 Hermite 插值基函数  $\{h_i(x), H_i(x)\}$ , 可得分片双三次 Hermite 插值

$$\begin{aligned} H_{3,3}(x, y) &= \sum_{i=0}^n \sum_{j=0}^m u_{ij} h_i(x) \bar{h}_j(y) + \sum_{i=0}^n \sum_{j=0}^m u'_{x,ij} H_i(x) \bar{h}_j(y) + \\ &\sum_{i=0}^n \sum_{j=0}^m u'_{y,ij} h_i(x) \bar{H}_j(y) + \sum_{i=0}^n \sum_{j=0}^m u''_{xy,ij} H_i(x) \bar{H}_j(y) \end{aligned} \quad (5-84)$$

另外,常用的分片二维样条插值也有很好的光滑性,此处不再赘述.

MATLAB 中有方便的二维插值函数:如 `interp2()`, 其调用格式为:

```
>> uu = interp2(xi, yj, uij, xx, yy, 'method')
```

其中数组 `xi, yj` 分别矩形区域剖分节点的横坐标和纵坐标, 矩阵 `uij` 表示被插值函数在节点上的函数值, 数组 `xx, yy` 分别表示待计算近似值的横坐标和纵坐标, 'method' 是采用的方法, 可选 'nearest' 为最邻近插值; 'linear' 为双线性插值; 'cubic' 为双三次插值; 'spline' 为双三次样条插值; 缺省为双线性插值. 当数据 `xi, yj, uij` 不能构成矩阵时, 应当使用更一般的函数 `griddata()`, 其调用格式为:

```
>> uu = griddata( xi, yj, uij, xx, yy, 'method')
```

各种符号含义同上.

## § 5.7 快速傅里叶变换(FFT)

工程技术中常把复杂的波形分解成谐波的叠加,数学上则表示为用三角函数的叠加来近似周期函数,即三角函数插值.快速傅里叶变换是充分利用三角函数的周期性,去掉重复运算,以达到减少运算量的一种常用方法.

### 5.7.1 三角函数插值

三角函数插值的基本问题:设已知  $f(x)$  在区间  $[0, 2\pi]$  上  $N+1$  个等距节点  $x_l = \frac{2\pi}{N}l$  ( $l=0, 1, \dots, N$ ) 处的值为  $f\left(\frac{2\pi}{N}l\right) = f_l$ , 且  $f_0 = f_N$ , 求函数

$$P(x) = \sum_{k=0}^{N-1} c_k e^{ikx} \quad (5-85)$$

使其在节点处与  $f(x)$  同值,即满足插值条件

$$P(x_l) = f(x_l) = f_l \quad (l=0, 1, \dots, N) \quad (5-86)$$

显然,函数  $P(x)$  自然满足  $P(x_0) = P(x_N)$ , 这是因为  $e^{ikx}$  ( $k=0, 1, \dots, N-1$ ) 都以  $2\pi$  为周期. 因此,由插值条件式(5-86)可得  $N$  阶线性方程组

$$Ac = f \quad (5-87)$$

其中

$$A = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & e^{i\frac{2\pi}{N}} & e^{i2\frac{2\pi}{N}} & \cdots & e^{i(N-1)\frac{2\pi}{N}} \\ 1 & e^{i\frac{4\pi}{N}} & e^{i2\frac{4\pi}{N}} & \cdots & e^{i(N-1)\frac{4\pi}{N}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & e^{i\frac{(N-1)2\pi}{N}} & e^{i2\frac{(N-1)2\pi}{N}} & \cdots & e^{i(N-1)\frac{(N-1)2\pi}{N}} \end{bmatrix} \quad (5-88)$$

$$c = (c_0, c_1, \dots, c_{N-1})^T, \quad f = (f_0, f_1, \dots, f_{N-1})^T$$

记矩阵  $A$  的第  $j$  列向量为

$$a_j = (1, e^{ij\frac{2\pi}{N}}, e^{ij\frac{4\pi}{N}}, \dots, e^{ij\frac{(N-1)2\pi}{N}})^T \quad (j=0, 1, \dots, N-1)$$

由复向量的内积运算,有

$$\begin{aligned} (a_j, a_k) &= \sum_{l=0}^{N-1} e^{ij\frac{2l\pi}{N}} \overline{e^{ik\frac{2l\pi}{N}}} \\ &= \sum_{l=0}^{N-1} e^{i(j-k)\frac{2l\pi}{N}} = \begin{cases} N, & j=k \\ 0, & j \neq k \end{cases} \end{aligned} \quad (5-89)$$

因此,矩阵  $A$  的列向量为正交向量组,故方程组(5-87)有解. 以向量  $\bar{a}_k^T$  左乘方程组(5-87),有

$$\bar{a}_k^T A c = N c_k = \bar{a}_k^T f = \sum_{l=0}^{N-1} f_l e^{-ik \frac{2l\pi}{N}}$$

由此可得方程组(5-87)的解

$$c_k = \frac{1}{N} \sum_{l=0}^{N-1} f_l e^{-ik \frac{2l\pi}{N}} \quad (k = 0, 1, \dots, N-1) \quad (5-90)$$

将式(5-90)代入式(5-85),即得函数  $f(x)$  的三角插值函数. 通过式(5-90)由数据  $\{f_l\}_{l=0}^{N-1}$  求系数  $\{c_k\}_{k=0}^{N-1}$  的过程称为  $f(x)$  的有限离散傅里叶变换,简称 DFT (discrete Fourier Transform).

反之,由式(5-87)和式(5-88)容易得出

$$f_l = \sum_{k=0}^{N-1} c_k e^{ik \frac{2l\pi}{N}} \quad (l = 0, 1, \dots, N-1) \quad (5-91)$$

按式(5-91)由数据  $\{c_k\}_{k=0}^{N-1}$  求出  $\{f_l\}_{l=0}^{N-1}$  的过程称为有限离散傅里叶反变换.

无论是有限离散傅里叶变换还是其反变换,都只是作复数加法与乘法运算,计算是简便易行的. 但当  $N$  较大时,由于每求一个  $c_k$  就要作  $N$  次复数乘法与  $N-1$  次加法运算,整个过程需作  $N^2$  次乘法,  $N(N-1)$  次加法与  $N$  次除法,故计算量十分可观. 事实上,由三角函数的周期性,式(5-90)与式(5-91)的计算中有许多是相同的,如果加以合并,就可以减少运算次数,特别是乘法运算次数. 基于这一想法,提出了快速傅里叶变换.

### 5.7.2 快速傅里叶变换(FFT)

取  $N = 2^m$ ,  $m$  为正整数,令  $w = e^{-i\frac{2\pi}{N}}$ ,  $a_l = \frac{f_l}{N}$ , 则式(5-90)可改写成

$$c_k = \frac{1}{N} \sum_{l=0}^{N-1} f_l e^{-ik \frac{2l\pi}{N}} = \sum_{l=0}^{N-1} a_l w^{kl} \quad (k = 0, 1, \dots, N-1) \quad (5-92)$$

快速傅里叶变换(简称 FFT)的基本思想是:将下标  $k, l$  用  $m$  位二进制数表示,设

$$\begin{aligned} k &= 2^{m-1} k_{m-1} + 2^{m-2} k_{m-2} + \dots + 2k_1 + k_0 \\ l &= 2^{m-1} l_{m-1} + 2^{m-2} l_{m-2} + \dots + 2l_1 + l_0 \end{aligned} \quad (k_j, l_j = 0, 1, j = 0, 1, \dots, m-1) \quad (5-93)$$

分别记  $k = (k_{m-1}, k_{m-2}, \dots, k_0)$ ,  $l = (l_{m-1}, l_{m-2}, \dots, l_0)$ ,  $c_k = c(k_{m-1}, k_{m-2}, \dots, k_0)$ ,  $a_l = a(l_{m-1}, l_{m-2}, \dots, l_0)$ , 则式(5-92)可表示成

$$\begin{aligned}
c_k &= c(k_{m-1}, k_{m-2}, \dots, k_0) = \sum_{l=0}^{N-1} a_l w^{kl} \\
&= \sum_{l_0=0}^1 \cdots \sum_{l_{m-2}=0}^1 \sum_{l_{m-1}=0}^1 a(l_{m-1}, l_{m-2}, \dots, l_0) w^{(2^{m-1}k_{m-1} + \cdots + 2k_1 + k_0)(2^{m-1}l_{m-1} + \cdots + 2l_1 + l_0)} \\
&= \sum_{l_0=0}^1 \cdots \sum_{l_{m-2}=0}^1 \sum_{l_{m-1}=0}^1 a(l_{m-1}, l_{m-2}, \dots, l_0) w^{2^{2m-2}k_{m-1}l_{m-1} + \cdots + 2^m(k_{m-1}l_1 + \cdots + k_1l_{m-1}) + \cdots + 2(k_1l_0 + k_0l_1) + k_0l_0}
\end{aligned}$$

由三角函数的周期性, 有  $w^{2^m} = w^{2^{m+j}} = 1 (j=1, 2, \dots, m-2)$ , 从而有

$$\begin{aligned}
c_k &= c(k_{m-1}, k_{m-2}, \dots, k_0) \\
&= \sum_{l_0=0}^1 \cdots \sum_{l_{m-2}=0}^1 \sum_{l_{m-1}=0}^1 a(l_{m-1}, l_{m-2}, \dots, l_0) w^{2^{m-1}(k_{m-1}l_0 + \cdots + k_0l_{m-1}) + \cdots + 2(k_1l_0 + k_0l_1) + k_0l_0} \\
&= \sum_{l_0=0}^1 \cdots \sum_{l_{m-2}=0}^1 \sum_{l_{m-1}=0}^1 a(l_{m-1}, \dots, l_0) w^{2^{m-1}k_0l_{m-1} + (2^{m-1}k_1 + 2^{m-2}k_0)l_{m-2} + \cdots + (2^{m-1}k_{m-1} + \cdots + 2k_1 + k_0)l_0} \\
&= \sum_{l_0=0}^1 \cdots \left[ \sum_{l_{m-2}=0}^1 \left( \sum_{l_{m-1}=0}^1 a(l_{m-1}, \dots, l_0) w^{2^{m-1}k_0l_{m-1}} \right) w^{(2^{m-1}k_1 + 2^{m-2}k_0)l_{m-2}} \right] \cdots w^{(2^{m-1}k_{m-1} + \cdots + 2k_1 + k_0)l_0}
\end{aligned} \tag{5-94}$$

式(5-94)按括号由里向外层层计算最后得到  $\{c_k\}_{k=0}^{N-1}$ . 为简明起见, 下面以  $m=3$ , 即  $N=8$  为例说明.

首先, 将  $k, l (k, l=0, 1, \dots, 7)$  表成二进制数, 设

$$k = 2^2k_2 + 2k_1 + k_0 = (k_2, k_1, k_0)$$

$$l = 2^2l_2 + 2l_1 + l_0 = (l_2, l_1, l_0)$$

并记  $c_k = c(k_2, k_1, k_0)$ ,  $a_l = a(l_2, l_1, l_0)$ , 则式(5-92)可表示成

$$\begin{aligned}
c(k_2, k_1, k_0) &= \sum_{l=0}^7 a_l w^{kl} \\
&= \sum_{l_0=0}^1 \sum_{l_1=0}^1 \sum_{l_2=0}^1 a(l_2, l_1, l_0) w^{(2^2k_2 + 2k_1 + k_0)(2^2l_2 + 2l_1 + l_0)} \\
&= \sum_{l_0=0}^1 \sum_{l_1=0}^1 \sum_{l_2=0}^1 a(l_2, l_1, l_0) w^{2^4k_2l_2 + 2^3(k_2l_1 + k_1l_2) + 2^2(k_2l_0 + k_1l_1 + k_0l_2) + 2(k_1l_0 + k_0l_1) + k_0l_0}
\end{aligned}$$

因为  $w = e^{-i\frac{2\pi}{8}}$ , 故  $w^{2^3} = w^{2^4} = 1$ . 于是

$$\begin{aligned}
c_k &= c(k_2, k_1, k_0) \\
&= \sum_{l_0=0}^1 \sum_{l_1=0}^1 \sum_{l_2=0}^1 a(l_2, l_1, l_0) w^{2^2(k_2l_0 + k_1l_1 + k_0l_2) + 2(k_1l_0 + k_0l_1) + k_0l_0} \\
&= \sum_{l_0=0}^1 \left[ \sum_{l_1=0}^1 \left( \sum_{l_2=0}^1 a(l_2, l_1, l_0) w^{2^2k_0l_2} \right) w^{(2^2k_1 + 2k_0)l_1} \right] w^{(2^2k_2 + 2k_1 + k_0)l_0} \tag{5-95}
\end{aligned}$$

对式(5-95)按括号层层求和,并记

$$\begin{aligned} A_1(l_0, l_1, k_0) &= \sum_{l_2=0}^1 a(l_2, l_1, l_0) w^{(2^2 k_0 l_2)} \\ &= a(0, l_1, l_0) + a(1, l_1, l_0) w^{(k_0, 0, 0)} \end{aligned} \quad (5-96)$$

$$\begin{aligned} A_2(l_0, k_1, k_0) &= \sum_{l_1=0}^1 A_1(l_0, l_1, k_0) w^{(2^2 k_1 + 2k_0) l_1} \\ &= A_1(l_0, 0, k_0) + A_1(l_0, 1, k_0) w^{(k_1, k_0, 0)} \end{aligned} \quad (5-97)$$

$$\begin{aligned} A_3(k_2, k_1, k_0) &= \sum_{l_1=0}^1 A_2(l_0, k_1, k_0) w^{(2^2 k_2 + 2k_1 + k_0) l_0} \\ &= A_2(0, k_1, k_0) + A_2(1, k_1, k_0) w^{(k_2, k_1, k_0)} \end{aligned} \quad (5-98)$$

从而有  $c_k = A_3(k_2, k_1, k_0)$ , 这就是 FFT 的计算过程. 为便于估计 FFT 的计算量, 我们作如下分析. 在式(5-96)中, 当  $k_0$  分别取 0, 1 时,  $w^{(k_0, 0, 0)} = w^{4k_0}$  分别为  $w^0 = 1, w^4 = -1$ , 因而计算全部 8 项  $A_1(l_0, l_1, k_0)$  至多需做 4 次乘法、8 次加减法. 在式(5-97)中, 注意到  $w^{4+j} = -w^j$ , 当  $k_1, k_0$  分别取遍 0, 1 时  $w^{(k_1, k_0, 0)} = w^{4k_1 + 2k_0}$  分别为  $w^0 = 1, w^2, w^4 = -1, w^6 = -w^2$ , 故只需先算出  $w^2$ , 计算全部 8 项  $A_2(l_0, k_1, k_0)$  需再做 4 次乘法、8 次加减法. 同样在式(5-98)中, 当  $k_2, k_1, k_0$  分别取遍 0, 1 时,  $w^{(k_2, k_1, k_0)} = w^{4k_2 + 2k_1 + k_0}$  分别为  $w^0 = -w^4 = 1, w = -w^5, w^2 = -w^6, w^3 = -w^7$ , 先算出  $w, w^2, w^3$ , 计算全部 8 项  $c_k = A_3(k_2, k_1, k_0)$  只需再做 4 次乘法、8 次加减法. 因而计算全部  $\{c_k\}_{k=0}^7$  的总计算量为 12 次复数乘法和 24 次加减法, 而 DFT 的计算量为 64 次乘法和 56 次加法. 类似地, 对于一般的  $N = 2^m$ , 用 FFT 法计算  $\{c_k\}_{k=0}^{N-1}$  的计算量为  $\frac{N \log_2 N}{2}$  次复数乘法和  $N \log_2 N$  次加法. 当  $N$  较大时, 节省的计算量是相当可观的. 例如, 当  $N = 2^{10}$  时, FFT 法的乘法计算量仅为 DFT 法的乘法计算量的  $\frac{1}{200}$ .

MATLAB 中的函数  $c = \text{fft}(f)$  可以实现快速傅里叶变换,  $f = \text{ifft}(c)$  可进行反变换.

## § 5.8 应用实例

### 5.8.1 一维数据的插值

利用 lagrange 插值、分段线性和三次样条插值近似来自极坐标曲线  $\rho =$



$5\sin\frac{\theta}{3}$  的数据, 并作出平滑的曲线.

由于曲线有自交点, 且  $x$  与  $y$  的关系非一一对应的, 我们选  $\theta$  为自变量, 分别对  $x(\theta)$  和  $y(\theta)$  进行插值, 然后再在  $xOy$  平面上作图.

首先根据函数可直接生成插值数据, 为使曲线较光滑, 再生成更细密的待计算的插值点, 画出图 5-8(a) 所示的原曲线图.

```
>> theta = 0:0.5:3 * pi; r2 = 5 * sin(theta/3); xi = r2. * cos(theta); yi = r2. * sin(theta);
>> theta2 = 0:0.05:3 * pi;
>> x0 = 5 * sin(theta2/3). * cos(theta2); y0 = 5 * sin(theta2/3). * sin(theta2);
>> subplot(2,2,1); plot(xi,yi,'.',x0,y0,'-');
```

或用命令 `polar(theta,r2)` 可直接作出极坐标图. 然后可直接调用函数 `interp1()` 进行各种插值计算, 计算结果如图 5-8(b)~(d) 所示.

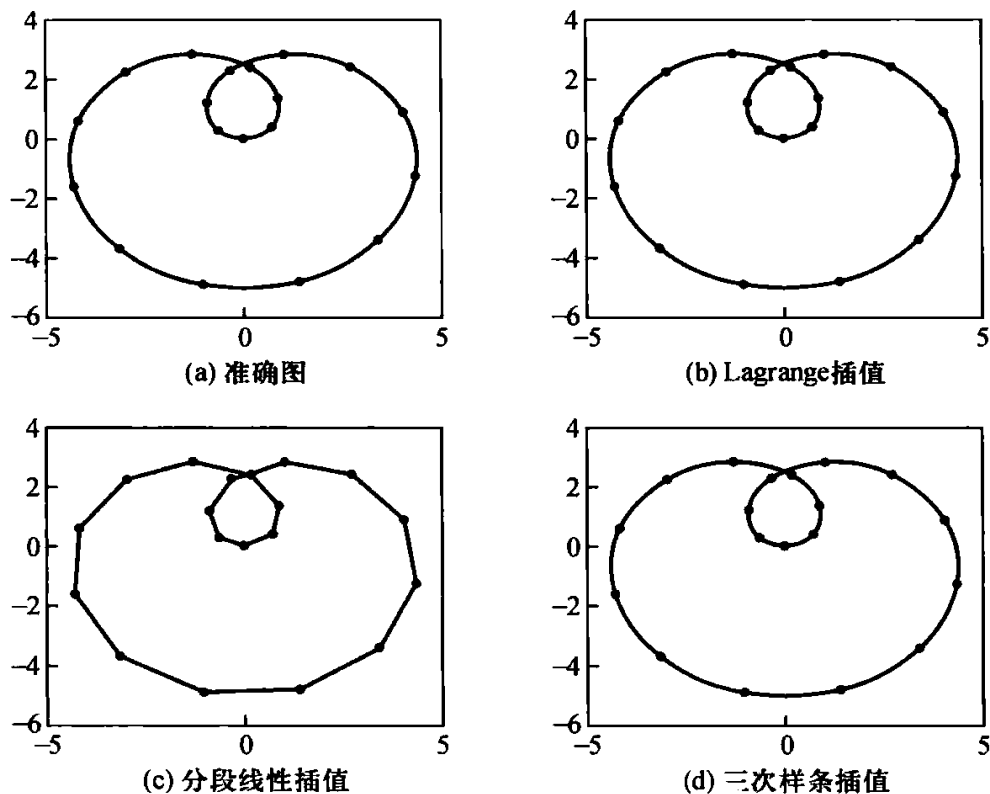


图 5-8

```
>> x1 = lagrange(theta,xi,theta2); y1 = lagrange(theta,yi,theta2);
>> x2 = interp1(theta,xi,theta2,'linear'); y2 = interp1(theta,yi,theta2,'linear');
>> x3 = interp1(theta,xi,theta2,'spline'); y3 = interp1(theta,yi,theta2,'spline');
>> subplot(2,2,2); plot(xi,yi,'.',x1,y1,'-');
```

```
>> subplot(2,2,3);plot(xi,yi,'.',x2,y2,'-')
>> subplot(2,2,4);plot(xi,yi,'.',x3,y3,'-')
```

从图形可见 Lagrange 插值和三次样条插值的效果较理想,分段线性插值需要更多的数据才能有较好的近似效果.

### 5.8.2 水库底部地形的近似模拟

建造水力发电站时,常要求建设水坝,以便拦截洪水,有效地利用水资源,从而在山底或河道中形成大型水库.为准确地估算水库的容量和水坝闸门所受到的压力,要求对水库底部地形做详细的勘探并能较准确地计算底部的海拔深度,进一步可利用第七章的数值积分方法计算水库的容量及水坝所受到的压力.

为方便起见,我们假设水库是矩形的,并建立  $10 \times 10$  的网格,测量得各网格点处的海拔深度列于表 5-7 中,试对表 5-7 中数据进行水库底部曲面进行二维插值,并画出水库底部地形图.

表 5-7

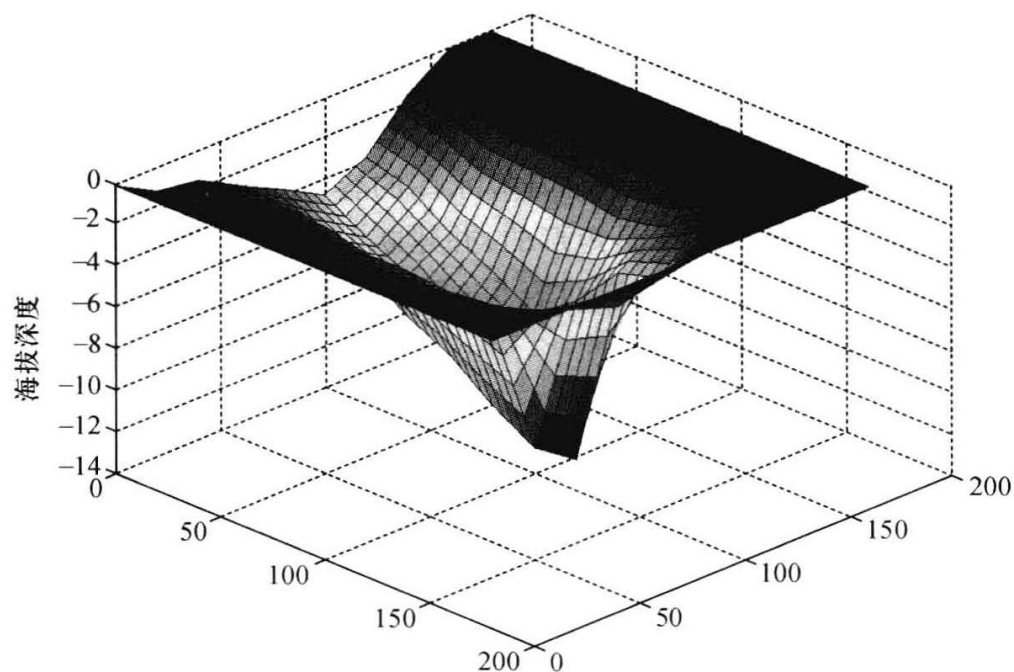
| $x \backslash y$ | 0.0  | 20.0 | 40.0 | 60.0 | 80.0 | 100.0 | 120.0 | 140.0 | 160.0 | 180.0 |
|------------------|------|------|------|------|------|-------|-------|-------|-------|-------|
| 0.0              | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   |
| 20.0             | -1.1 | -1.1 | -1.2 | -1.2 | -1.4 | -1.4  | -1.3  | -1.2  | -1.0  | 0.0   |
| 40.0             | -1.4 | -1.6 | -1.8 | -1.8 | -2.2 | -2.2  | -1.8  | -1.6  | -1.1  | 0.0   |
| 60.0             | -2.5 | -2.7 | -2.8 | -3.3 | -3.3 | -3.5  | -4.2  | -2.8  | -1.8  | 0.0   |
| 80.0             | -3.5 | -4.6 | -5.8 | -7.2 | -8.2 | -9.8  | -11.0 | -3.2  | -2.5  | 0.0   |
| 100.0            | -4.6 | -5.8 | -6.8 | -8.6 | -9.8 | -10.6 | -12.4 | -3.6  | -2.0  | 0.0   |
| 120.0            | -3.6 | -4.2 | -5.6 | -6.6 | -7.8 | -6.4  | -4.2  | -3.6  | -1.1  | 0.0   |
| 140.0            | -1.4 | -1.8 | -2.5 | -2.6 | -2.8 | -3.2  | -2.8  | -1.6  | 0.0   | 0.0   |
| 160.0            | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   |
| 180.0            | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   |

首先输入插值数据

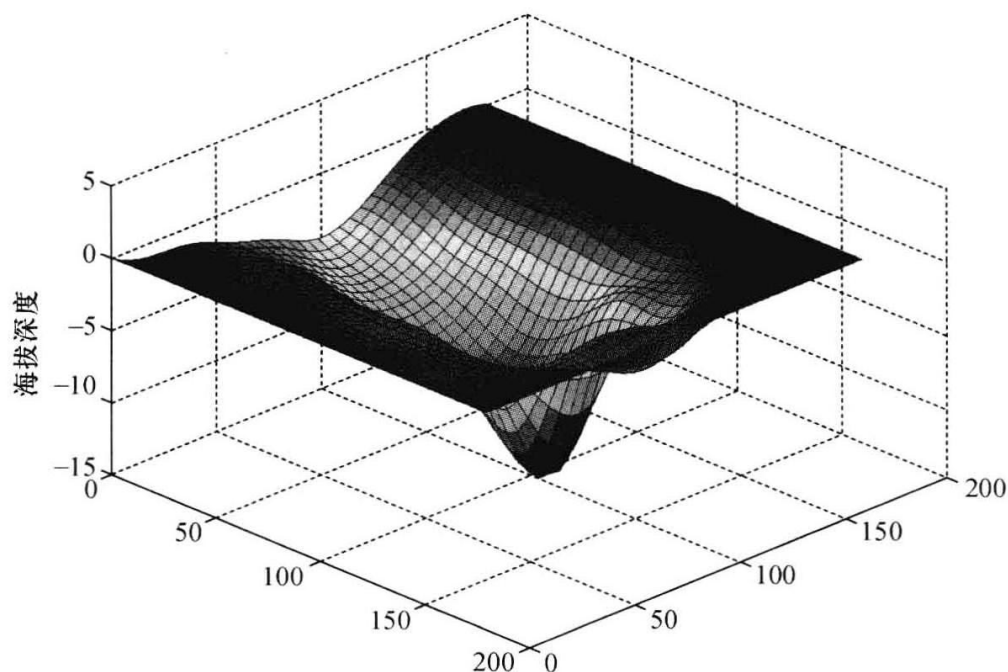
```
>> x = 0.0:20:180.0; y = 0.0:20:180;
>> z = [0 0 0 0 0 0 0 0 0 0; -1.1 -1.1 -1.2 -1.2 -1.4 -1.4
-1.3 -1.2 -1.0 0.0; -1.4 -1.6 -1.8 ..... -3.2 -2.8 -1.6
0.0 0.0; 0 0 0 0 0 0 0 0 0 0; 0 0 0 0 0 0 0 0 0 0];
```

由于表 5-7 中海拔深度数据  $z$  正好构成一个  $10 \times 10$  的矩阵,可直接调用 `interp2()` 函数,此处分别采用分片双线性插值和三次样条插值,计算结果见图 5-9.

```
>> [xx,yy] = meshgrid(0:5:180,0:5:180);  
>> zz1 = interp2(x,y,z,xx,yy,'bilinear');surf(xx,yy,zz1);view(45,40)  
>> zz3 = interp2(x,y,z,xx,yy,'spline');surf(xx,yy,zz3);view(45,40)
```



(a) 双线性插值



(b) 三次样条插值

图 5-9

由图可见,三次样条曲面是光滑的,而双线性插值曲面连续但不光滑.如果勘测数据不完全,不能用函数 `interp2()`,应该调用函数 `griddata()`.例如,仅测得如下的数据(见表 5-8):

表 5-8

| $x \backslash y$ | 0.0  | 20.0 | 40.0 | 60.0 | 80.0 | 100.0 | 120.0 | 140.0 | 160.0 | 180.0 |
|------------------|------|------|------|------|------|-------|-------|-------|-------|-------|
| 0.0              | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   |
| 20.0             | ×    | ×    | -1.2 | ×    | ×    | -1.4  | ×     | -1.2  | ×     | 0.0   |
| 40.0             | -1.4 | ×    | ×    | -1.8 | ×    | -2.2  | -1.8  | ×     | -1.1  | ×     |
| 60.0             | ×    | -2.7 | ×    | -3.3 | ×    | ×     | -4.2  | -2.8  | ×     | ×     |
| 80.0             | -3.5 | ×    | -5.8 | ×    | -8.2 | -9.8  | -11.0 | ×     | ×     | ×     |
| 100.0            | ×    | -5.8 | ×    | -8.6 | -9.8 | -10.6 | -12.4 | ×     | -2.0  | ×     |
| 120.0            | -3.6 | ×    | -5.6 | ×    | -7.8 | -6.4  | ×     | -3.6  | ×     | 0.0   |
| 140.0            | -1.4 | ×    | -2.5 | ×    | -2.8 | -3.2  | -2.8  | ×     | ×     | 0.0   |
| 160.0            | ×    | ×    | ×    | ×    | ×    | ×     | ×     | ×     | ×     | ×     |
| 180.0            | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   |

那么空间点  $(x_i, y_i, z_i)$  按行按列依次输入数组如下:

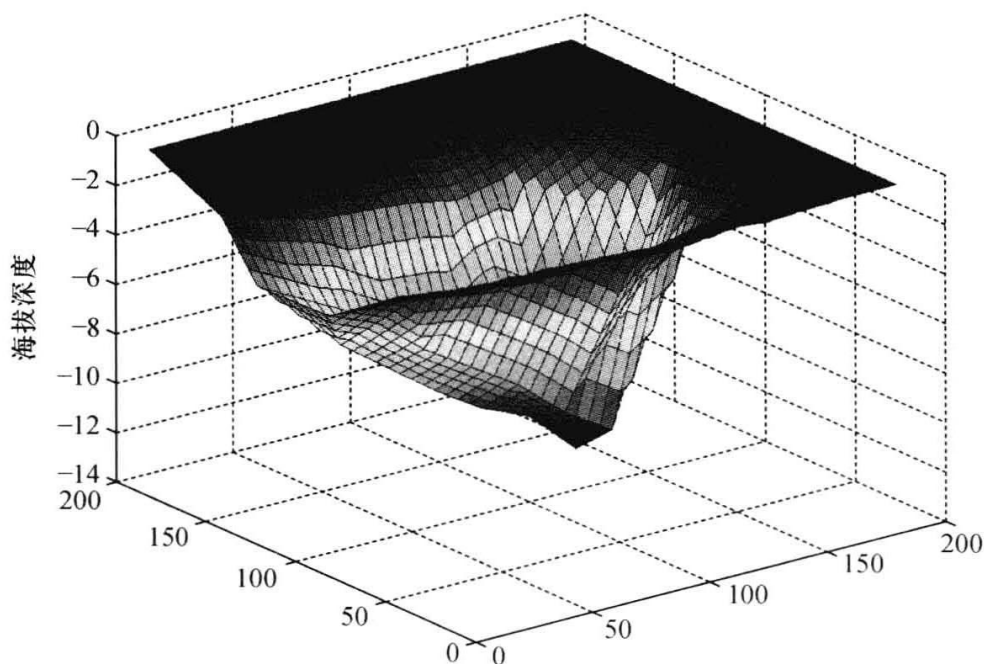
```
>> xi = [0.0 20.0 40.0 60.0 80.0 100.0 120.0 140.0 160.0 180.0 40.0
100.0 140.0 180.0 0.0 60.0 100.0 120.0 160.0 20.0 60.0 120.0 140.0 0.0
40.0 80.0 100.0 120.0 20.0 60.0 80.0 100.0 120.0 160.0 0.0 40.0 80.0
100.0 140.0 180.0 0.0 40.0 80.0 100.0 120.0 180.0 0.0 20.0 40.0 60.0
80.0 100.0 120.0 140.0 160.0 180.0];
>> yi = [0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 20.0 20.0 20.0 20.0
40.0 40.0 40.0 40.0 40.0 60.0 60.0 60.0 60.0 80.0 80.0 80.0 80.0
100.0 100.0 100.0 100.0 100.0 100.0 100.0 120.0 120.0 120.0 120.0 120.0
120.0 140.0 140.0 140.0 140.0 140.0 140.0 180.0 180.0 180.0 180.0
180.0 180.0 180.0 180.0 180.0 180.0];
>> zi = [0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -1.2 -1.4 -1.2 0.0
-1.4 -1.8 -2.2 -1.8 -1.1 -2.7 -3.3 -4.2 -2.8 -3.5 -5.8
-8.2 -9.8 -11.0 -5.8 -8.6 -9.8 -10.6 -12.4 -2.0 -3.6 -5.6
-7.8 -6.4 -3.6 0.0 -1.4 -2.5 -2.8 -3.2 -2.8 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0];
```

再进行分片三角形双线性和双三次插值并作图,结果见图 5-10.

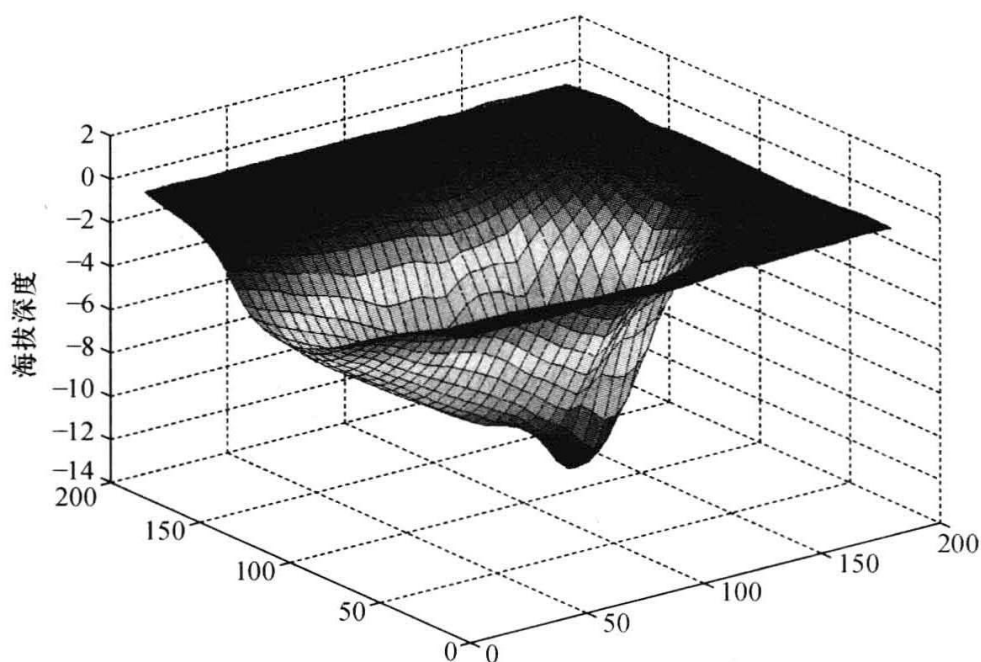
```
>> zz1 = griddata( xi, yi, zi, xx, yy, 'linear' ); surf( xx, yy, zz1 );
```

```
>> zz3 = griddata( xi, yi, zi, xx, yy, 'cubic' ); surf( xx, yy, zz3 );
```

三维插值或其他高维插值可调用函数 `interp3()` 和 `interpn()` 实现.



(a) 三角形双线性插值



(b) 三角形双三次插值

图 5-10

## 评 注

函数插值是数值积分和数值微分的基础,也是微分方程数值解法的常用工具.本章讨论了常用的 Lagrange 插值公式、Newton 插值公式和 Hermite 插值公式,给出了各插值公式的误差表达式.若被插值函数充分光滑,由插值误差表达式可以给出插值近似值的误差限.由于高次插值的 Runge 现象,实际计算中很少采用高次插值,而分段低次插值却受到重视,如分段线性插值、分段三次 Hermite 插值和三次样条插值.只要插值节点足够稠密,分段低次插值是能够达到总体精度要求的.

样条插值的理论和方法自 20 世纪 60 年代以来有了飞速的发展.本章仅介绍了三弯矩算法和三转角算法.分段三次样条插值不仅是光滑的,而且函数及其一、二阶导数均具有一致收敛性.至于更便于计算的 B-样条函数,读者可参阅 [22].

二维及多元函数的插值有着广泛的应用领域.利用构造基函数的方法很容易将一维插值方法推广到多维的情形.常用的方法是分片双一次插值、分片双三次 Hermite 插值和双三次样条插值.在飞机、轮船等的外形设计中常用的是 Coons 曲面插值.详细可参阅 [23].

## 习 题 五

1. 已知函数表如下:

|                 |     |   |   |
|-----------------|-----|---|---|
| $x_i$           | -1  | 0 | 1 |
| $y_i = 2^{x_i}$ | 0.5 | 1 | 2 |

(1) 利用线性插值计算  $2^{0.3}$  的近似值并估计误差;

(2) 利用二次插值计算  $2^{0.3}$  的近似值并估计误差.

2. 已知函数表如下:

|       |          |          |          |          |
|-------|----------|----------|----------|----------|
| $x_i$ | 0.527    | 0.727    | 0.807    | 0.927    |
| $y_i$ | 0.010 75 | 0.012 19 | 0.011 88 | 0.014 26 |

用二次插值求  $y(0.7)$  的近似值.

3. 已知函数表如下:

|          |    |   |   |    |
|----------|----|---|---|----|
| $x_i$    | 1  | 3 | 4 | 6  |
| $f(x_i)$ | -7 | 5 | 8 | 14 |

试求其 3 阶 Lagrange 插值多项式,并以此计算  $f(2)$  的近似值.

4. 已知函数表如下:

|                        |          |          |          |          |          |
|------------------------|----------|----------|----------|----------|----------|
| $x_i$                  | 0.40     | 0.55     | 0.65     | 0.80     | 0.90     |
| $y_i = \text{sh } x_i$ | 0.410 75 | 0.578 15 | 0.696 75 | 0.888 11 | 1.026 52 |

利用四次插值计算  $\text{sh } 0.569$  的近似值,并估计误差.

5. 已知函数表如下:

|                    |           |           |           |           |           |
|--------------------|-----------|-----------|-----------|-----------|-----------|
| $x_i$              | 2.0       | 2.1       | 2.2       | 2.3       | 2.4       |
| $y_i = \sqrt{x_i}$ | 1.414 214 | 1.449 138 | 1.483 240 | 1.516 575 | 1.549 193 |

试用多项式插值计算  $\sqrt{2.15}$  的近似值,要求误差不超过  $\frac{1}{2} \times 10^{-5}$ .

6. 已知  $f(x) = e^{x^2-1}$  的函数表如下:

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $x_i$    | 1        | 1.1      | 1.2      | 1.3      | 1.4      |
| $f(x_i)$ | 1.000 00 | 1.233 68 | 1.552 71 | 1.993 72 | 2.611 70 |

- (1) 利用四次 Lagrange 插值计算  $f(1.25)$  的近似值,并估计误差;
- (2) 求  $f(x)$  的四次 Newton 插值多项式,并计算  $f(1.25)$  的近似值;
- (3) 增加节点  $x_5 = 1.5, f(x_5) = 3.490\ 34$ , 求  $f(x)$  的五次 Newton 插值多项式,并计算  $f(1.25)$  的近似值.

7. 设  $x_0, x_1, \dots, x_n$  为  $n+1$  个互异节点,  $l_j(x) (j=0, 1, \dots, n)$  为这组节点上的  $n$  次 Lagrange 插值基函数,试证:

$$(1) \sum_{j=0}^n (x_j - x)^k l_j(x) \equiv 0 \quad (k = 1, \dots, n);$$

(2) 设  $P(x)$  是任意一个最高次项系数为 1 的  $n+1$  次多项式,则

$$P(x) - \sum_{j=0}^n P(x_j) l_j(x) = \omega_{n+1}(x) = \prod_{j=0}^n (x - x_j)$$

8. 已知函数表如下:

|          |           |           |           |           |
|----------|-----------|-----------|-----------|-----------|
| $x_i$    | 1. 615    | 1. 634    | 1. 702    | 1. 828    |
| $f(x_i)$ | 2. 414 50 | 2. 462 59 | 2. 652 71 | 3. 030 35 |

试求其 3 阶 Newton 插值多项式,并以此计算  $f(1.682)$  的近似值.

9. 设有函数  $\varphi(x) = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}t^2} dt$  的函数表如下

|              |            |            |            |            |
|--------------|------------|------------|------------|------------|
| $x$          | 0. 10      | 0. 11      | 0. 12      | 0. 13      |
| $\varphi(x)$ | 0. 539 828 | 0. 543 795 | 0. 547 758 | 0. 551 717 |

试给出  $\varphi(0.1236)$  的值,要求截断误差不超过  $\frac{1}{2} \times 10^{-5}$ . 如果要求结果有 6 位有效数字,  $\varphi(0.1236)$  为多少.

10. 已知函数表如下:

|          |   |   |   |    |
|----------|---|---|---|----|
| $x_i$    | 1 | 2 | 3 | 4  |
| $f(x_i)$ | 3 | 5 | 9 | 15 |

分别用 Newton 向前、向后插值公式计算  $f(1.5)$ ,  $f(3.7)$  的近似值.

11. 已知  $y = \cos x$  的函数表如下:

|                  |           |           |           |           |           |           |           |
|------------------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $x_i$            | 0. 0      | 0. 1      | 0. 2      | 0. 3      | 0. 4      | 0. 5      | 0. 6      |
| $y_i = \cos x_i$ | 1. 000 00 | 0. 995 00 | 0. 980 07 | 0. 955 34 | 0. 921 06 | 0. 877 58 | 0. 825 34 |

分别用 4 阶 Newton 向前、向后插值公式计算  $\cos 0.048$  和  $\cos 0.575$  的近似值.

12. 在  $[0, \pi/2]$  上给出  $f(x) = \sin x$  的等距节点函数表,为保证用分段线性插值求非节点处的正弦值时,其截断误差不超过  $10^{-6}$ ,函数表的步长至少应取多大? 若改用分段二次插值求非节点处的正弦值,步长应取多大?



13. 设  $f(x) = x^7 + 5x^5 + 1$ , 求差商:

(1)  $f[2^0, 2^1, \dots, 2^7]$ ; (2)  $f[2^0, 2^1, \dots, 2^k]$  ( $k \geq 8$ )

14. 设  $f(x)$  是一个  $n$  次多项式,  $f(x) = \sum_{k=0}^n a_k x^k$  ( $a_n \neq 0$ ), 试证:

$$f[x_0, x_1, \dots, x_m, x] = \begin{cases} n - m - 1 \text{ 次多项式,} & m \leq n - 1 \\ a_n, & m = n - 1 \\ 0, & m > n - 1 \end{cases}$$

15. 设  $f(x) = 3xe^x - e^{2x}$ , 节点  $x_0 = 1, x_1 = 1.05$ , 求  $f(x)$  的 3 次 Hermite 插值多项式及  $f(1.03)$  的近似值, 并估计误差.

16. 给定函数表

|           |   |   |    |
|-----------|---|---|----|
| $x_i$     | 0 | 1 | 2  |
| $f(x_i)$  | 1 | 2 | 1  |
| $f'(x_i)$ |   | 0 | -1 |

且已知  $f(x)$  在  $[0, 2]$  上 5 阶连续可导, 求  $f(x)$  的 4 次 Hermite 插值多项式  $H_4(x)$ , 并写出余项  $f(x) - H_4(x)$  的表示式.

17. 已知函数表如下:

|       |   |   |   |   |
|-------|---|---|---|---|
| $x_i$ | 0 | 1 | 2 | 3 |
| $y_i$ | 0 | 0 | 0 | 0 |

边界条件为

(1)  $m_0 = 1, m_3 = 0$

(2)  $M_0 = 1, M_3 = 0$

分别求三次样条插值函数.

18. 已知函数表如下:

|       |         |         |         |         |         |
|-------|---------|---------|---------|---------|---------|
| $x_i$ | 0.25    | 0.30    | 0.39    | 0.45    | 0.53    |
| $y_i$ | 0.500 0 | 0.547 7 | 0.624 5 | 0.672 8 | 0.278 0 |

求自然边界条件下的三次样条插值函数.

19. 已知函数表如下:

|          |   |   |   |
|----------|---|---|---|
| $x_i$    | 1 | 2 | 3 |
| $f(x_i)$ | 2 | 4 | 8 |

求满足边界条件  $f'(1) = 1.386\ 3, f'(3) = 5.545\ 2$  的三次样条插值函数.

20. 已知  $f(x) = e^{-x}$ , 取节点  $x_0 = 0, x_1 = 0.2, x_2 = 0.6, x_3 = 1.0$

(1) 求  $f(x)$  在自然边界条件下的三次样条插值函数;

(2) 求  $f(x)$  满足边界条件  $f'(0) = -1, f'(1) = -0.367\ 9$  的三次样条插值函数;

(3) 利用样条插值函数计算  $\int_0^1 f(x) dx$  的近似值.

21. 设  $f_l = (l-2)^2 (l=0, 1, 2, 3)$ , 分别用 DFT 法和 FFT 法求  $\{c_k\}_{k=0}^3$ , 并比较它们的计算量.

22. 试对  $N = 2^4 = 16$  给出快速傅里叶变换的计算公式及流程图.

## 数值实验

1. 试编写 MATLAB 函数实现 Newton 插值, 要求能输出插值多项式. 对函数  $f(x) = \frac{1}{1+4x^2}$  在区间  $[-5, 5]$  上实现 10 次多项式插值.

(1) 输出插值多项式;

(2) 在区间  $[-5, 5]$  内均匀插入 99 个节点, 计算这些节点上函数  $f(x)$  的近似值, 并在同一张图中画出原函数和插值多项式的图形;

(3) 观察龙格现象, 计算插值函数在各节点上的误差, 并画出误差图.

2. 试编写 MATLAB 函数分别实现分段线性插值、分段三次 Hermite 插值和三次样条插值. 将区间  $[-5, 5]$  分成 10 等份, 对上述函数  $f(x) = \frac{1}{1+4x^2}$  调用你的程序, 画出相应的插值函数图形, 计算误差比较各种插值的精确度.

3. 已知直升机旋转机翼外形轮廓线上某些点的数据:

| $k$ | $x_k$ | $y_k$ | $k$ | $x_k$ | $y_k$ |
|-----|-------|-------|-----|-------|-------|
| 1   | 520   | 0     | 3   | 156.6 | -36   |
| 2   | 280   | -30   | 4   | 78    | -35   |

续表

| $k$ | $x_k$ | $y_k$  | $k$ | $x_k$ | $y_k$ |
|-----|-------|--------|-----|-------|-------|
| 5   | 39.62 | -28.44 | 10  | 78    | 35    |
| 6   | 3.1   | -9.4   | 11  | 156.6 | 36    |
| 7   | 0     | 0      | 12  | 280   | 30    |
| 8   | 3.1   | 9.4    | 13  | 520   | 0     |
| 9   | 39.62 | 28.44  |     |       |       |

以  $k$  为自变量,用表中 13 个节点上的数据,分别作出  $x(k), y(k)$  的样条插值,平均增加节点至 121 个,在  $xOy$  平面上绘制出较光滑的机翼外形曲线图.

4. 试分别编写矩形网格上的双线性插值和三次样条插值,对平面区域  $D = [-3, 3] \times [-2, 2]$  上函数  $f(x, y) = (x^2 - 2x)e^{-x^2 - y^2 - xy}$ , 先以 0.5 步长产生较粗糙的网格,生成插值数据,再以 0.2 为步长产生细密的网格,调用你的函数作双线性插值和三次样条插值,计算函数的近似值,画出近似曲面图形. 进一步将近似值与精确值比较,求出各网格点上的绝对误差,画出误差图形.

## 第六章 函数逼近

曲线拟合又称函数逼近,是求近似函数的又一类数值方法.它不要求近似函数在节点处与函数同值,即不要求近似曲线过已知点,只要求它尽可能反映给定数据点的基本趋势,在某种意义下与函数最“逼近”.下面我们先举例说明.

**例 1** 给定一组实验数据如下:

| $i$   | 1   | 2   | 3   | 4   |
|-------|-----|-----|-----|-----|
| $x_i$ | 2   | 4   | 6   | 8   |
| $y_i$ | 1.1 | 2.8 | 4.9 | 7.2 |

求  $x, y$  的函数关系.

[解] 先作草图.如图 6-1 所示,这些点的分布接近一条直线,因此可设想,  $y$  为  $x$  的一次函数. 设

$$y = a_1 x + a_0 \quad (6-1)$$

从图 6-1 不难看出,无论  $a_0, a_1$  取何值,直线都不可能同时过全部数据点.怎样选取  $a_0, a_1$ ,才能使直线(6-1)“最好”地反映数据点的基本趋势?首先要建立好坏的标准.

假设  $a_0, a_1$  已经确定,  $y_i^* = a_1 x_i + a_0$  ( $i = 1, 2, 3, 4$ ) 为由近似函数求得的近似值,它与观测值  $y_i$  之差

$$\delta_i = y_i - y_i^* = y_i - a_1 x_i - a_0 \quad (i = 1, 2, 3, 4) \quad (6-2)$$

称为残差.显然,残差的大小可作为衡量近似函数好坏的标准.常用的准则有以下三种:

(1) 使残差的绝对值之和最小,即

$$\sum_i |\delta_i| = \min$$

(2) 使残差的最大绝对值最小,即

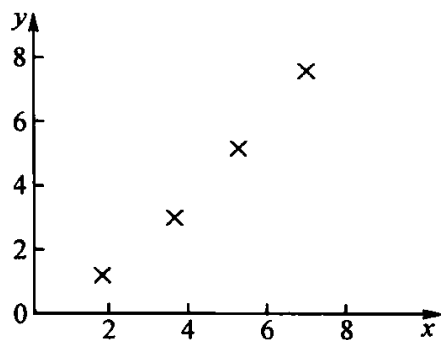


图 6-1

$$\max_i |\delta_i| = \min$$

(3) 使残差的平方和最小, 即

$$\sum_i \delta_i^2 = \min$$

准则(1)的提出很自然, 也合理, 但实际使用不便. 按准则(2)求近似函数的方法称为函数的最佳一致逼近. 按准则(3)确定参数, 求得近似函数的方法称为最佳平方逼近, 也称曲线拟合(或数据拟合)的最小二乘法. 它的计算比较简便, 是实践中常用的一种函数逼近方法.

## § 6.1 数据拟合的最小二乘法

数据拟合的最小二乘问题是: 根据给定的数据组  $(x_i, y_i) (i = 1, 2, \dots, n)$ , 选取近似函数形式, 即给定函数类  $H$ , 求函数  $\varphi(x) \in H$ , 使得

$$\sum_{i=1}^n \delta_i^2 = \sum_{i=1}^n [y_i - \varphi(x_i)]^2$$

为最小, 即

$$\sum_{i=1}^n [y_i - \varphi(x_i)]^2 = \min_{\psi \in H} \sum_{i=1}^n [y_i - \psi(x_i)]^2$$

这种求近似函数的方法称为数据拟合的最小二乘法, 函数  $\varphi(x)$  称为这组数据的最小二乘函数. 通常取  $H$  为一些比较简单函数的集合, 如低次多项式, 指数函数等. 例 1 中取  $H$  为一次多项式集合.

### 6.1.1 多项式拟合

对给定的数据组  $(x_i, y_i) (i = 1, 2, \dots, n)$ , 求一个  $m$  次多项式 ( $m < n$ )

$$P_m(x) = a_0 + a_1x + \dots + a_mx^m \quad (6-3)$$

使得

$$\sum_{i=1}^n \delta_i^2 = \sum_{i=1}^n [y_i - P_m(x_i)]^2 = F(a_0, a_1, \dots, a_m) \quad (6-4)$$

为最小, 即选取参数  $a_i (i = 0, 1, \dots, m)$ , 使得

$$F(a_0, a_1, \dots, a_m) = \sum_{i=1}^n [y_i - P_m(x_i)]^2 = \min_{\psi \in H} \sum_{i=1}^n [y_i - \psi(x_i)]^2$$

其中  $H$  为至多  $m$  次多项式集合. 这就是数据的多项式拟合,  $P_m(x)$  称为这组数据的最小二乘  $m$  次拟合多项式.

与第二章 § 6 求超定线性方程组的最小二乘解的方法相同, 由多元函数取极值的必要条件, 得方程组

$$\frac{\partial F}{\partial a_j} = -2 \sum_{i=1}^n [y_i - \sum_{k=0}^m a_k x_i^k] x_i^j = 0 \quad (j = 0, 1, \dots, m)$$

移项得

$$\sum_{k=0}^m a_k \left( \sum_{i=1}^n x_i^{k+j} \right) = \sum_{i=1}^n y_i x_i^j \quad (j = 0, 1, \dots, m)$$

即

$$\begin{cases} na_0 + a_1 \sum_{i=1}^n x_i + a_2 \sum_{i=1}^n x_i^2 + \dots + a_m \sum_{i=1}^n x_i^m = \sum_{i=1}^n y_i \\ a_0 \sum_{i=1}^n x_i + a_1 \sum_{i=1}^n x_i^2 + a_2 \sum_{i=1}^n x_i^3 + \dots + a_m \sum_{i=1}^n x_i^{m+1} = \sum_{i=1}^n y_i x_i \\ \dots\dots\dots \\ a_0 \sum_{i=1}^n x_i^m + a_1 \sum_{i=1}^n x_i^{m+1} + a_2 \sum_{i=1}^n x_i^{m+2} + \dots + a_m \sum_{i=1}^n x_i^{2m} = \sum_{i=1}^n y_i x_i^m \end{cases} \quad (6-5)$$

这是最小二乘拟合多项式的系数  $a_k$  ( $k = 0, 1, \dots, m$ ) 应满足的方程组, 称为正则方程组或法方程组. 由函数组  $\{1, x, x^2, \dots, x^m\}$  的线性无关性可以证明, 方程组 (6-5) 存在唯一解, 且解所对应的多项式 (6-3) 必定是已给数据组  $(x_i, y_i)$  ( $i = 1, 2, \dots, n$ ) 的最小二乘  $m$  次拟合多项式.

如图 6-1 表明, 可用一次多项式 (6-1) 拟合例 1 中数据组所给出的函数关系. 将数据代入正则方程组 (6-5) 得

$$\begin{cases} 4a_0 + 20a_1 = 16 \\ 20a_0 + 120a_1 = 100.4 \end{cases}$$

其解为  $a_0 = -1.1, a_1 = 1.02$ , 所以

$$y = 1.02x - 1.1$$

就是所给数据组的最小二乘拟合直线.

### 例 2 求数据表

| $i$   | 1        | 2       | 3       | 4       | 5       | 6       | 7       | 8       | 9       |
|-------|----------|---------|---------|---------|---------|---------|---------|---------|---------|
| $x_i$ | -1       | -0.75   | -0.5    | -0.25   | 0       | 0.25    | 0.5     | 0.75    | 1       |
| $y_i$ | -0.220 9 | 0.329 5 | 0.882 6 | 1.439 2 | 2.000 3 | 2.564 5 | 3.133 4 | 3.760 1 | 4.283 6 |

的最小二乘二次拟合多项式.

[解] 设二次拟合多项式为  $P_2(x) = a_0 + a_1x + a_2x^2$ , 将数据表代入式 (6-5), 得此问题的正则方程组

$$\begin{cases} 9a_0 + 0 + 3.75a_2 = 18.1723 \\ 0 + 3.75a_1 + 0 = 8.4842 \\ 3.75a_0 + 0 + 2.7656a_2 = 7.6173 \end{cases}$$

其解为  $a_0 = 2.0034$ ,  $a_1 = 2.2625$ ,  $a_2 = 0.0378$ . 所以此数据组的最小二乘二次拟合多项式为

$$P_2(x) = 2.0034 + 2.2625x + 0.0378x^2$$

### 6.1.2 指数拟合

如果数据点  $(x_i, y_i) (i = 1, 2, \dots, n)$  的分布近似指数曲线, 则可考虑用指数函数

$$y = be^{ax} \quad (6-6)$$

去拟合数据, 按最小二乘原理,  $a, b$  的选取应使得

$$F(a, b) = \sum_{i=1}^n (y_i - be^{ax_i})^2$$

为最小. 由此导出的正则方程组是关于参数  $a, b$  的非线性方程组, 称其为非线性最小二乘问题, 而多项式拟合中正则方程组为线性方程组, 称为线性最小二乘问题. 非线性最小二乘问题的求解比较复杂. 但若对式 (6-6) 两端取对数, 相当于作变换  $\tilde{y} = \ln y$ , 则有

$$\ln y = ax + \ln b$$

上式右端是线性函数. 因此, 如果数据点  $(x_i, y_i) (i = 1, 2, \dots, n)$  的分布近似于指数曲线, 则数据组  $(x_i, \ln y_i) (i = 1, 2, \dots, n)$  的分布就近似于直线. 故可以先求出数据组  $(x_i, \ln y_i)$  的最小二乘拟合直线  $\tilde{y} = a_0 + a_1 x$ , 取指数即得数据组  $(x_i, y_i) (i = 1, 2, \dots, n)$  的最小二乘拟合指数  $y = e^{a_0 + a_1 x} = e^{a_0} e^{a_1 x}$ .

**例 3** 设一发射源的发射强度公式形为

$$I = I_0 e^{-\alpha t}$$

现测得  $I$  与  $t$  的数据如下表:

| $t_i$ | 0.2  | 0.3  | 0.4  | 0.5  | 0.6  | 0.7  | 0.8  |
|-------|------|------|------|------|------|------|------|
| $I_i$ | 3.16 | 2.38 | 1.75 | 1.34 | 1.00 | 0.74 | 0.56 |

试用最小二乘法确定  $I_0$  与  $\alpha$ .

[解] 先求数据表

| $t_i$     | 0.2     | 0.3     | 0.4     | 0.5     | 0.6     | 0.7      | 0.8      |
|-----------|---------|---------|---------|---------|---------|----------|----------|
| $\ln I_i$ | 1.150 6 | 0.867 1 | 0.559 6 | 0.292 7 | 0.000 0 | -0.301 1 | -0.579 8 |

的最小二乘拟合直线. 将此表数据代入式(6-5), 得正则方程组

$$\begin{cases} 7a_0 + 3.5a_1 = 1.989\ 1 \\ 3.5a_0 + 2.03a_1 = 0.185\ 8 \end{cases}$$

其解为  $a_0 = 1.73, a_1 = -2.89$ . 所以

$$I_0 = e^{a_0} = 5.64, \quad \alpha = -a_1 = 2.89$$

发射强度公式近似为

$$I = 5.64e^{-2.89t}$$

### 6.1.3 分式线性拟合

如果数据点  $(x_i, y_i) (i = 1, 2, \dots, n)$  的分布近似于分式线性函数  $y = \frac{1}{ax + b}$  的图像, 怎样求这种形式的最小二乘拟合函数? 与指数拟合相类似, 可先作变换

$$\tilde{y} = \frac{1}{y} = ax + b$$

将问题线性化, 按数据组  $\left(x_i, \frac{1}{y_i}\right) (i = 1, 2, \dots, n)$  求其最小二乘一次拟合多项式

$\tilde{y} = ax + b$ , 然后取倒数即得数据组  $(x_i, y_i) (i = 1, 2, \dots, n)$  的最小二乘拟合函数.

若取拟合函数的形式为  $y = \frac{x}{ax + b}$ , 则应先作倒数变换

$$\tilde{y} = \frac{1}{y} = \frac{ax + b}{x} = a + \frac{b}{x}$$

同时令  $t = \frac{1}{x}$ , 于是  $\tilde{y} = a + bt$  为线性拟合问题, 然后按数据组

$$(t_i, \tilde{y}_i) = \left(\frac{1}{x_i}, \frac{1}{y_i}\right) \quad (i = 1, 2, \dots, n)$$

求其最小二乘拟合直线  $\tilde{y} = a + bt = a + \frac{b}{x}$ , 再取倒数得所求拟合函数

$$y = \frac{1}{\tilde{y}} = \frac{1}{a + \frac{b}{x}} = \frac{x}{ax + b}$$

综上所述, 求数据组的最小二乘拟合函数的步骤为:

(1) 由给定数据确定近似函数的表达形式, 一般可通过描点观察或经验估计得到;



(2) 按最小二乘原则确定表达式中的参数, 即由偏差平方和最小导出正则方程组, 求解得参数. 值得提出的是: 一些简单的非线性最小二乘问题通常需先作变换将问题化为线性最小二乘问题再求解.

两点说明:

(1) 在指数拟合中, 先作变量代换对新变量求最小二乘拟合函数, 然后再还原所得的近似函数与直接对原变量按最小二乘原则求得的拟合函数是不同的. 但由于实际计算时, 人们关心的是问题的简化, 就把这两者的区别忽略了.

(2) 实际问题中, 由于各点的观测数据精度不同, 常常引入加权方差, 即确定参数的准则为使得

$$\sum_{i=1}^n \omega_i \delta_i^2$$

最小, 其中  $\omega_i (i=1, 2, \dots, n)$  为加权系数.

#### 6.1.4 线性最小二乘法的一般形式

一般地, 设给定数据组  $(x_i, y_i) (i=1, 2, \dots, n)$ ,  $\varphi_0(x), \varphi_1(x), \dots, \varphi_m(x)$  为已知的一组  $[a, b]$  上线性无关的函数, 选取近似函数为

$$\varphi(x) = a_0 \varphi_0(x) + a_1 \varphi_1(x) + \dots + a_m \varphi_m(x) \quad (6-7)$$

使得

$$\begin{aligned} \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)]^2 &= \sum_{i=1}^n \omega_i \left[ y_i - \sum_{k=0}^m a_k \varphi_k(x_i) \right]^2 \\ &= \min_{\psi \in H} \sum_{i=1}^n \omega_i [y_i - \psi(x_i)]^2 \end{aligned} \quad (6-8)$$

其中  $\omega_i > 0 (i=1, 2, \dots, n)$  为权系数,  $H$  为  $\varphi_0(x), \varphi_1(x), \dots, \varphi_m(x)$  的线性组合的全体, 这就是线性最小二乘法的一般形式. 特别地, 取  $\varphi_k(x) = x^k (k=0, 1, \dots, m)$ , 就是多项式拟合.

与多项式拟合的讨论相类似, 上述问题的正则方程组为

$$\sum_{i=1}^n \omega_i [y_i - \varphi(x_i)] \varphi_j(x_i) = 0 \quad (j=0, 1, \dots, m)$$

$$\text{即} \quad \sum_{k=0}^m a_k \left[ \sum_{i=1}^n \omega_i \varphi_k(x_i) \varphi_j(x_i) \right] = \sum_{i=1}^n \omega_i y_i \varphi_j(x_i) \quad (j=0, 1, \dots, m) \quad (6-9)$$

如果记  $(\varphi_k, \varphi_j) = \sum_{i=1}^n \omega_i \varphi_k(x_i) \varphi_j(x_i)$ ,  $(y, \varphi_j) = \sum_{i=1}^n \omega_i y_i \varphi_j(x_i)$ , 方程组 (6-9) 可表示成矩阵形式

$$\begin{bmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_m) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_m) \\ \vdots & \vdots & & \vdots \\ (\varphi_m, \varphi_0) & (\varphi_m, \varphi_1) & \cdots & (\varphi_m, \varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (y, \varphi_0) \\ (y, \varphi_1) \\ \vdots \\ (y, \varphi_m) \end{bmatrix} \quad (6-10)$$

由  $\varphi_k(x) (k=0, 1, \dots, m)$  线性无关可导出式(6-10)的系数矩阵非奇异, 从而保证了方程组(6-9)的解存在、唯一.

**定理 6.1** 设  $a_0, a_1, \dots, a_m$  为方程组(6-9)的解, 则函数  $\varphi(x) = \sum_{k=0}^m a_k \varphi_k(x)$  满足关系式(6-8), 即它是数据组  $(x_i, y_i) (i=1, 2, \dots, n)$  的最小二乘函数.

[证明] 对任意  $\psi(x) = \sum_{k=0}^m c_k \varphi_k(x) \in H$ , 有

$$\begin{aligned} \sum_{i=1}^n \omega_i [y_i - \psi(x_i)]^2 &= \sum_{i=1}^n \omega_i [y_i - \varphi(x_i) + \varphi(x_i) - \psi(x_i)]^2 \\ &= \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)]^2 + 2 \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)] \\ &\quad \times [\varphi(x_i) - \psi(x_i)] + \sum_{i=1}^n \omega_i [\varphi(x_i) - \psi(x_i)]^2 \end{aligned}$$

由式(6-9)可得

$$\begin{aligned} &\sum_{i=1}^n \omega_i [y_i - \varphi(x_i)] [\varphi(x_i) - \psi(x_i)] \\ &= \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)] \left[ \sum_{k=0}^m (a_k - c_k) \varphi_k(x_i) \right] \\ &= \sum_{k=0}^m (a_k - c_k) \left[ \sum_{i=1}^n \omega_i (y_i - \varphi(x_i)) \varphi_k(x_i) \right] = 0 \end{aligned}$$

从而有

$$\begin{aligned} \sum_{i=1}^n \omega_i [y_i - \psi(x_i)]^2 &= \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)]^2 + \sum_{i=1}^n \omega_i [\varphi(x_i) - \psi(x_i)]^2 \\ &\geq \sum_{i=1}^n \omega_i [y_i - \varphi(x_i)]^2 \end{aligned}$$

即

$$\sum_{i=1}^n \omega_i [y_i - \varphi(x_i)]^2 = \min_{\psi \in H} \sum_{i=1}^n \omega_i [y_i - \psi(x_i)]^2$$

所以  $\varphi(x)$  是数据组  $(x_i, y_i) (i=1, 2, \dots, n)$  的最小二乘函数.

## 例 4 已知数据表

|     |          |          |          |          |         |         |         |         |
|-----|----------|----------|----------|----------|---------|---------|---------|---------|
| $x$ | -0.980 0 | -0.730 0 | -0.480 0 | -0.230 0 | 0.020 0 | 0.270 0 | 0.520 0 | 0.770 0 |
| $y$ | 5.238 9  | 3.256 4  | 1.542 7  | 0.966 2  | 1.629 8 | 2.405 7 | 2.400 4 | 2.279 8 |

由函数  $\varphi_0(x) = e^{x^2}$ ,  $\varphi_1(x) = \sin 5x$ ,  $\varphi_2(x) = \ln(x + 1.5)$  线性组合而成, 求最小二乘拟合函数  $\varphi(x) = ae^{x^2} + b\sin 5x + c\ln(x + 1.5)$ .

[解] 显然函数组  $\varphi_0(x)$ ,  $\varphi_1(x)$ ,  $\varphi_2(x)$  线性无关, 计算它们在各节点处的值

$$\begin{aligned}\varphi_0(x_j) &= [2.612\ 7 \quad 1.703\ 9 \quad 1.259\ 1 \quad 1.054\ 3 \quad 1.000\ 4 \\ &\quad 1.075\ 6 \quad 1.310\ 5 \quad 1.809\ 2], \\ \varphi_1(x_j) &= [0.982\ 5 \quad 0.486\ 8 \quad -0.675\ 5 \quad -0.912\ 8 \quad 0.099\ 8 \\ &\quad 0.975\ 7 \quad 0.515\ 5 \quad -0.650\ 6], \\ \varphi_2(x_j) &= [-0.653\ 9 \quad -0.261\ 4 \quad 0.019\ 8 \quad 0.239\ 0 \quad 0.418\ 7 \\ &\quad 0.571\ 0 \quad 0.703\ 1 \quad 0.819\ 8],\end{aligned}$$

按式(6-10)计算得正则方程组

$$\begin{bmatrix} 19.575\ 0 & 2.231\ 3 & 1.560\ 7 \\ 2.231\ 3 & 4.142\ 6 & -0.573\ 2 \\ 1.560\ 7 & -0.573\ 2 & 2.221\ 2 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 33.685\ 9 \\ 7.072\ 2 \\ 1.597\ 2 \end{bmatrix}$$

解正则方程组得  $a = 0.650\ 3$ ,  $b = 0.785\ 4$ ,  $c = -0.237\ 8$ , 从而最小二乘拟合函数为

$$\varphi(x) = 0.650\ 3e^{x^2} + 0.785\ 4\sin(5x) - 0.237\ 8\ln(x + 1.5)$$

此题若用 MATLAB 计算, 直接利用左除命令, MATLAB 将对超定方程组求最小二乘解, 得到同样的结果. 具体过程如下:

```
>> x = [-0.980 0 -0.730 0 -0.480 0 -0.230 0 0.020 0 0.270 0
0.520 0 0.770 0]';
>> y = [5.238 9 3.256 4 1.542 7 0.966 2 1.629 8 2.405 7 2.400 4
2.279 8]';
>> A = [exp(x.^2) sin(5 * x) log(x + 1.5)]; A = A' * A;
>> abc = (A \ y)'
abc = [0.650 3 0.785 4 -0.237 8]
```

最小二乘法的正则方程组一般是病态的, 当  $m$  较大时更是如此, 这使得按上述过程求解时误差较大. 如果适当地选取  $\varphi_k(x)$  ( $k = 0, 1, \dots, m$ ), 使得

$$\begin{aligned}
 (\varphi_k, \varphi_k) &= \sum_{i=1}^n \omega_i \varphi_k^2(x_i) > 0 \quad (k = 0, 1, \dots, m) \\
 (\varphi_k, \varphi_j) &= 0 \quad (k \neq j)
 \end{aligned} \tag{6-11}$$

则方程组(6-10)的系数矩阵成对角形, 正则方程组(6-9)极易解出, 其解为

$$a_k = \frac{(\gamma, \varphi_k)}{(\varphi_k, \varphi_k)} = \frac{\sum_{i=1}^n \omega_i y_i \varphi_k(x_i)}{\sum_{i=1}^n \omega_i \varphi_k^2(x_i)} \quad (k = 0, 1, \dots, m) \tag{6-12}$$

最小二乘函数为

$$\varphi(x) = \sum_{k=0}^m a_k \varphi_k(x) = \sum_{k=0}^m \frac{(\gamma, \varphi_k)}{(\varphi_k, \varphi_k)} \varphi_k(x) \tag{6-13}$$

**定义 6.1** 称满足条件式(6-11)的函数族  $\varphi_0(x), \varphi_1(x), \dots, \varphi_m(x)$  为以  $\{\omega_i\} (i=1, 2, \dots, n)$  为权关于点集  $\{x_1, x_2, \dots, x_n\}$  的正交函数族.

类似于定理 6.4 的证明, 可导出下列多项式系

$$\begin{cases} \varphi_0(x) = 1 \\ \varphi_1(x) = x - \alpha_1 \\ \varphi_k(x) = (x - \alpha_k) \varphi_{k-1}(x) - \beta_k \varphi_{k-2}(x) \quad (k = 2, 3, \dots, m) \end{cases} \tag{6-14}$$

是以  $\omega_i (i=1, 2, \dots, n)$  为权关于点集  $\{x_1, x_2, \dots, x_n\}$  的正交函数族, 其中

$$\alpha_k = \frac{(x \varphi_{k-1}, \varphi_{k-1})}{(\varphi_{k-1}, \varphi_{k-1})} = \frac{\sum_{i=1}^n \omega_i x_i \varphi_{k-1}^2(x_i)}{\sum_{i=1}^n \omega_i \varphi_{k-1}^2(x_i)} \quad (k = 1, 2, \dots, m) \tag{6-15}$$

$$\beta_k = \frac{(\varphi_{k-1}, \varphi_{k-1})}{(\varphi_{k-2}, \varphi_{k-2})} = \frac{\sum_{i=1}^n \omega_i \varphi_{k-1}^2(x_i)}{\sum_{i=1}^n \omega_i \varphi_{k-2}^2(x_i)} \quad (k = 2, 3, \dots, m)$$

于是, 求数据组  $(x_i, y_i) (i=1, 2, \dots, n)$  带权  $\omega_i (i=1, 2, \dots, n)$  的最小二乘拟合多项式可按以下过程进行:

(1) 按式(6-14)和式(6-15)构造正交函数族,  $\{\varphi_0(x), \varphi_1(x), \dots, \varphi_m(x)\}$ ;

(2) 由式(6-12)计算出正则方程组(6-9)的解;

(3) 按式(6-13)写出最小二乘  $m$  次拟合多项式.

**例 5** 利用正交函数族求例 2 所给数据表的最小二乘二次拟合多项式.

[解] 按式(6-14), 式(6-15)计算, 得

$$\varphi_0(x) = 1$$

$$\alpha_1 = \frac{(x\varphi_0, \varphi_0)}{(\varphi_0, \varphi_0)} = \frac{\sum_{i=1}^9 x_i}{\sum_{i=1}^9 1} = \frac{0}{9} = 0$$

$$\varphi_1(x) = x$$

$$\alpha_2 = \frac{(x\varphi_1, \varphi_1)}{(\varphi_1, \varphi_1)} = \frac{\sum_{i=1}^9 x_i^3}{\sum_{i=1}^9 x_i^2} = 0$$

$$\beta_2 = \frac{(\varphi_1, \varphi_2)}{(\varphi_0, \varphi_0)} = \frac{\sum_{i=1}^9 x_i^2}{\sum_{i=1}^9 1} = \frac{3.75}{9} = 0.416\ 67$$

$$\varphi_2(x) = x^2 - 0.416\ 67$$

由式(6-12)

$$a_0 = \frac{(\gamma, \varphi_0)}{(\varphi_0, \varphi_0)} = \frac{\sum_{i=1}^9 y_i}{\sum_{i=1}^9 1} = \frac{18.172\ 3}{9} = 2.019\ 14$$

$$a_1 = \frac{(\gamma, \varphi_1)}{(\varphi_1, \varphi_1)} = \frac{\sum_{i=1}^9 y_i x_i}{\sum_{i=1}^9 x_i^2} = \frac{8.484\ 2}{3.75} = 2.262\ 5$$

$$a_2 = \frac{(\gamma, \varphi_2)}{(\varphi_2, \varphi_2)} = \frac{\sum_{i=1}^9 y_i (x_i^2 - 0.416\ 67)}{\sum_{i=1}^9 (x_i^2 - 0.416\ 67)^2} = \frac{0.045\ 45}{1.203\ 1} = 0.037\ 8$$

将上述结果代入式(6-13),得最小二乘二次拟合多项式为

$$\begin{aligned}\varphi(x) &= a_0\varphi_0(x) + a_1\varphi_1(x) + a_2\varphi_2(x) \\ &= 2.019\ 14 + 2.262\ 5x + 0.037\ 8(x^2 - 0.416\ 67) \\ &= 2.003\ 4 + 2.262\ 5x + 0.037\ 8x^2\end{aligned}$$

这与例2的计算结果相同.

MATLAB 中的函数 polyfit() 可直接实现最小二乘多项式拟合,其调用格式为

$$\gg p = \text{polyfit}(x, y, n)$$

其中  $x, y$  为拟合数据,  $n$  为拟和多项式的最高次数,  $p$  是一个向量, 各分量表示拟合多项式按幂次由高次到低次的系数. 如例 5 用 MATLAB 计算过程如下

```
>> x = -1:0.25:1;
>> y = [-0.220 9 0.329 5 0.882 6 1.439 2 2.000 3 2.564 5 3.133 4
3.760 1 4.283 6];
>> p = polyfit(x, y, 2)
p = [0.037 851 082 251 08 2.262 446 666 666 67 2.003 373 160 173 16]
```

得到结果与例 2、例 5 结果一致.

## § 6.2 正交多项式

如前所述, 利用正交函数系可以简化最小二乘法的求解, 并提高解的精度. 正交多项式系, 由于其计算简便, 是函数逼近的重要工具.

### 6.2.1 基本概念

**定义 6.2** 如果函数系  $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x), \dots$  满足

$$(\varphi_j, \varphi_k) = \int_a^b \omega(x) \varphi_j(x) \varphi_k(x) dx = \begin{cases} 0, & j \neq k \\ \alpha_k > 0, & j = k \end{cases} \quad (j, k = 0, 1, 2, \dots)$$

则称此函数系为区间  $[a, b]$  上关于权函数  $\omega(x)$  的正交函数系. 特别地, 若  $\alpha_k = 1 (k = 0, 1, 2, \dots)$ , 则称其为标准正交函数系.

例如, 三角函数系  $\{1, \cos x, \sin x, \cos 2x, \sin 2x, \dots, \cos nx, \sin nx, \dots\}$  就是区间  $[-\pi, \pi]$  上关于权函数  $\omega(x) \equiv 1$  的正交函数系.

正交函数系具有以下性质.

**定理 6.2** 区间  $[a, b]$  上关于权函数  $\omega(x)$  的正交函数系  $\varphi_0, \varphi_1, \dots, \varphi_n$  必定是线性无关的.

[证明] 用反证法. 假设  $\varphi_0, \varphi_1, \dots, \varphi_n$  线性相关, 即存在不全为零的实数  $c_0, c_1, \dots, c_n$ , 使得

$$c_0 \varphi_0(x) + c_1 \varphi_1(x) + \dots + c_n \varphi_n(x) \equiv 0, \quad x \in [a, b]$$

不妨设  $c_i \neq 0$ , 以  $\omega(x) \varphi_i(x)$  乘上式后, 在区间  $[a, b]$  上积分得

$$c_0 (\varphi_0, \varphi_i) + c_1 (\varphi_1, \varphi_i) + \dots + c_n (\varphi_n, \varphi_i) = c_i (\varphi_i, \varphi_i) = 0$$

因为  $(\varphi_i, \varphi_i) > 0$ , 故  $c_i = 0$ , 导出矛盾. 所以  $\varphi_0, \varphi_1, \dots, \varphi_n$  在  $[a, b]$  上线性无关.

如果正交函数系  $\{\varphi_i(x)\}$  中函数均为代数多项式, 则称其为正交多项式系. 由定理 6.2, 正交多项式系是线性无关的.

**定理 6.3** 设  $\varphi_k(x) (k = 0, 1, 2, \dots)$  是最高次项系数不为零的  $k$  次多项式,

则  $\{\varphi_k(x)\}$  是  $[a, b]$  上关于权函数  $\omega(x)$  的正交多项式系的充要条件是对任意至多  $k-1$  次的多项式  $Q_{k-1}(x)$ , 均有

$$(\varphi_k, Q_{k-1}) = \int_a^b \omega(x) \varphi_k(x) Q_{k-1}(x) dx = 0 \quad (k = 1, 2, \dots)$$

[证明] 充分性. 若对任意至多  $k-1$  次多项式  $Q_{k-1}(x)$ , 都有

$$\int_a^b \omega(x) \varphi_k(x) Q_{k-1}(x) dx = 0 \quad (k = 1, 2, \dots)$$

特别地, 对多项式  $\varphi_j(x) (j=0, 1, \dots, k-1)$ , 有

$$\int_a^b \omega(x) \varphi_k(x) \varphi_j(x) dx = 0$$

即对任意  $k \neq j$ , 都有  $(\varphi_k, \varphi_j) = 0$ . 另一方面, 由  $\varphi_k(x)$  的最高次项系数不为零可得  $\varphi_k^2(x) \geq 0$ , 且  $\varphi_k^2(x) \neq 0 (k=0, 1, \dots)$ , 根据定理积分性质有

$$(\varphi_k, \varphi_k) = \int_a^b \omega(x) \varphi_k^2(x) dx > 0$$

所以,  $\{\varphi_k(x)\}$  为正交多项式系.

必要性. 设  $\{\varphi_k(x)\}$  是  $[a, b]$  上关于权函数  $\omega(x)$  的正交多项式系, 由定理 6.2,  $\varphi_0(x), \dots, \varphi_{k-1}(x)$  在  $[a, b]$  上线性无关, 因而任一至多  $k-1$  次多项式  $Q_{k-1}(x)$  均能表示成它们的线性组合. 设

$$Q_{k-1}(x) = \sum_{j=0}^{k-1} b_j \varphi_j(x)$$

于是

$$\begin{aligned} \int_a^b \omega(x) \varphi_k(x) Q_{k-1}(x) dx &= \int_a^b \omega(x) \varphi_k(x) \left[ \sum_{j=0}^{k-1} b_j \varphi_j(x) \right] dx \\ &= \sum_{j=0}^{k-1} b_j \int_a^b \omega(x) \varphi_k(x) \varphi_j(x) dx = 0 \end{aligned}$$

### 6.2.2 格拉姆-施密特 (Gram-Schmidt) 方法

构造正交多项式的一般方法由以下定理给出.

**定理 6.4** 按以下方式定义的多项式集合  $\{\varphi_0, \varphi_1, \dots, \varphi_n\}$  是区间  $[a, b]$  上关于权函数  $\omega(x) \geq 0 (\omega(x) \neq 0)$  的正交函数族.

$$\varphi_0(x) = 1$$

$$\varphi_1(x) = x - \alpha_1 \quad (6-16)$$

$$\varphi_k(x) = (x - \alpha_k) \varphi_{k-1}(x) - \beta_k \varphi_{k-2}(x) \quad (k = 2, 3, \dots, n)$$

其中

$$\alpha_k = \frac{(\varphi_k, \varphi_{k-1})}{(\varphi_{k-1}, \varphi_{k-1})} = \frac{\int_a^b \omega(x) x \varphi_{k-1}^2(x) dx}{\int_a^b \omega(x) \varphi_{k-1}^2(x) dx} \quad (k = 1, 2, \dots, n)$$

$$\beta_k = \frac{(\varphi_{k-1}, \varphi_{k-1})}{(\varphi_{k-2}, \varphi_{k-2})} = \frac{\int_a^b \omega(x) \varphi_{k-1}^2(x) dx}{\int_a^b \omega(x) \varphi_{k-2}^2(x) dx} \quad (k = 2, 3, \dots, n)$$

[证明] 显然,  $\varphi_k(x)$  ( $k=0, 1, \dots, n$ ) 为  $k$  次多项式, 且  $k$  次项系数为 1.

因为  $\varphi_k(x) \not\equiv 0$  ( $k=0, 1, \dots, n$ ),  $\omega(x) \geq 0$ ,  $\omega(x) \not\equiv 0$ , 由定积分性质

$$(\varphi_k, \varphi_k) = \int_a^b \omega(x) \varphi_k^2(x) dx > 0$$

下面用数学归纳法证明  $(\varphi_j, \varphi_k) = 0$  ( $j \neq k$ ). 由对称性, 只需证  $(\varphi_j, \varphi_k) = 0$  ( $j < k$ ,  $k=1, 2, \dots, n$ ). 当  $k=1$  时

$$\begin{aligned} (\varphi_0, \varphi_1) &= \int_a^b \omega(x) \varphi_0(x) \varphi_1(x) dx \\ &= \int_a^b \omega(x) (x - \alpha_1) dx \\ &= \int_a^b x \omega(x) dx - \alpha_1 \int_a^b \omega(x) dx \\ &= (x \varphi_0, \varphi_0) - \frac{(x \varphi_0, \varphi_0)}{(\varphi_0, \varphi_0)} (\varphi_0, \varphi_0) = 0 \end{aligned}$$

结论成立.

假设对  $k=m-1$  ( $1 \leq m \leq n$ ) 结论成立, 即  $(\varphi_j, \varphi_{m-1}) = 0$  ( $j=0, 1, \dots, m-1$ ), 下证  $(\varphi_j, \varphi_m) = 0$  ( $j=0, 1, \dots, m-1$ ). 因为

$$\begin{aligned} (\varphi_{m-1}, \varphi_m) &= \int_a^b \omega(x) \varphi_{m-1}(x) [(x - \alpha_m) \varphi_{m-1}(x) - \beta_m \varphi_{m-2}(x)] dx \\ &= (x \varphi_{m-1}, \varphi_{m-1}) - \alpha_m (\varphi_{m-1}, \varphi_{m-1}) - \beta_m (\varphi_{m-1}, \varphi_{m-2}) \\ (\varphi_{m-2}, \varphi_m) &= \int_a^b \omega(x) \varphi_{m-2}(x) [(x - \alpha_m) \varphi_{m-1}(x) - \beta_m \varphi_{m-2}(x)] dx \\ &= (x \varphi_{m-2}, \varphi_{m-1}) - \alpha_m (\varphi_{m-2}, \varphi_{m-1}) - \beta_m (\varphi_{m-2}, \varphi_{m-2}) \end{aligned}$$

对  $j < m-2$ , 有

$$(\varphi_j, \varphi_m) = \int_a^b x \omega(x) \varphi_j(x) \varphi_{m-1}(x) dx - \alpha_m (\varphi_j, \varphi_{m-1}) - \beta_m (\varphi_j, \varphi_{m-2})$$

由归纳法假设

$$(\varphi_{m-1}, \varphi_{m-2}) = 0, (\varphi_j, \varphi_{m-1}) = (\varphi_j, \varphi_{m-2}) = 0 \quad (j = 0, 1, \dots, m-3)$$

另一方面, 由式(6-16)

$$x \varphi_j(x) = \varphi_{j+1}(x) + \alpha_{j+1} \varphi_j(x) + \beta_{j+1} \varphi_{j-1}(x)$$

于是

$$(\varphi_{m-1}, \varphi_m) = 0$$

$$(\varphi_{m-2}, \varphi_m) = (x \varphi_{m-2}, \varphi_{m-1}) - (\varphi_{m-1}, \varphi_{m-1})$$



$$\begin{aligned}
&= (\alpha_{m-1}\varphi_{m-2} + \beta_{m-1}\varphi_{m-3}, \varphi_{m-1}) = 0 \\
(\varphi_j, \varphi_m) &= (\varphi_j, (x - \alpha_m)\varphi_{m-1} - \beta_m\varphi_{m-2}) \\
&= (x\varphi_j, \varphi_{m-1}) - \alpha_m(\varphi_j, \varphi_{m-1}) - \beta_m(\varphi_j, \varphi_{m-2}) \\
&= (\varphi_{j+1} + \alpha_{j+1}\varphi_j + \beta_{j+1}\varphi_{j-1}, \varphi_{m-1}) = 0 \quad (j = 1, 2, \dots, m-3) \\
(\varphi_0, \varphi_m) &= (\varphi_0, (x - \alpha_m)\varphi_{m-1} - \beta_m\varphi_{m-2}) \\
&= (x\varphi_0, \varphi_{m-1}) - \alpha_m(\varphi_0, \varphi_{m-1}) - \beta_m(\varphi_0, \varphi_{m-2}) \\
&= (\varphi_1 + \alpha_1\varphi_0, \varphi_{m-1}) = 0
\end{aligned}$$

因此对  $k = m$  结论成立. 由归纳法原理, 对一切自然数  $n$  结论均成立.

按式(6-16)构造正交多项式的方法称为 Gram-Schmidt 方法.

下面介绍几种常用的正交多项式.

### 6.2.3 常用的正交多项式

#### (一) 勒让德 (Legendre) 多项式

Legendre 多项式的一般表示式为

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2 - 1)^n] \quad (n = 0, 1, 2, \dots) \quad (6-17)$$

具体表达为

$$P_0(x) = 1$$

$$P_1(x) = x$$

$$P_2(x) = \frac{1}{2}(3x^2 - 1)$$

$$P_3(x) = \frac{1}{2}(5x^3 - 3x)$$

$$P_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3)$$

.....

$$P_n(x) = \sum_{k=0}^{\left[\frac{n}{2}\right]} \frac{(-1)^k (2n-2k)!}{2^n k! (n-k)! (n-2k)!} x^{n-2k} \quad (n = 0, 1, 2, \dots) \quad (6-18)$$

Legendre 多项式有以下性质:

(1)  $\{P_n(x)\}$  是区间  $[-1, 1]$  上关于权函数  $\omega(x) \equiv 1$  的正交函数系, 且

$$(P_n, P_m) = \int_{-1}^1 P_n(x) P_m(x) dx = \begin{cases} 0, & n \neq m \\ \frac{2}{2n+1}, & n = m \end{cases} \quad (6-19)$$

(2) Legendre 多项式满足递推公式

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x) \quad (n=1,2,\dots) \quad (6-20)$$

通过变数变换由 Legendre 多项式(6-18)可以得到在任意区间 $[a,b]$ 上关于权函数 $\omega(x) \equiv 1$ 的正交多项式系.

令 $x = \frac{b+a}{2} + \frac{b-a}{2}t$ , 当 $x$ 在区间 $[a,b]$ 上变化时, 对应的 $t$ 在 $[-1,1]$ 上变化, 故

$$\bar{P}_n(x) = P_n(t) = P_n\left(\frac{2x - (b+a)}{b-a}\right) \quad (n=0,1,\dots) \quad (6-21)$$

为 $[a,b]$ 上的正交多项式. 例如, 区间 $[0,1]$ 上的正交多项式为

$$\bar{P}_0(x) = P_0(2x-1) = 1$$

$$\bar{P}_1(x) = P_1(2x-1) = 2x-1$$

$$\bar{P}_2(x) = \frac{1}{2}[3(2x-1)^2 - 1] = 6x^2 - 6x + 1$$

$$\bar{P}_3(x) = \frac{1}{2}[5(2x-1)^3 - 3(2x-1)] = 20x^3 - 30x^2 + 12x - 1$$

.....

## (二) 第一类切比雪夫(Chebyshev)多项式

第一类 Chebyshev 多项式的一般表示式为

$$T_n(x) = \cos(n \arccos x) \quad (-1 \leq x \leq 1, n=0,1,2,\dots) \quad (6-22)$$

令 $x = \cos \theta$ , 当 $x$ 在 $[-1,1]$ 上变化时, 对应的 $\theta$ 在 $[0,\pi]$ 上变化, 式(6-22)可改写成

$$T_n(x) = \cos n\theta$$

具体表达为

$$T_0(x) = \cos 0 = 1$$

$$T_1(x) = \cos \theta = x$$

$$T_2(x) = \cos 2\theta = 2\cos^2 \theta - 1 = 2x^2 - 1$$

$$T_3(x) = \cos 3\theta = 4x^3 - 3x$$

$$T_4(x) = \cos 4\theta = 8x^4 - 8x^2 + 1$$

.....

$$T_n(x) = \frac{n}{2} \sum_{k=0}^{\left[\frac{n}{2}\right]} (-1)^k \frac{(n-k-1)!}{k!(n-2k)!} (2x)^{n-2k} \quad (6-23)$$

由式(6-23)容易看出,  $T_n(x)$ 是首项系数为 $2^{n-1}$ 的 $n$ 次多项式.

第一类 Chebyshev 多项式有以下性质:

(1)  $\{T_n(x)\}$  是区间  $[-1, 1]$  上关于权函数  $\omega(x) = (1-x^2)^{-\frac{1}{2}}$  的正交多项式系, 且

$$(T_n, T_m) = \int_{-1}^1 \frac{T_n(x) T_m(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0, & n \neq m \\ \frac{\pi}{2}, & n = m \neq 0 \\ \pi, & n = m = 0 \end{cases} \quad (6-24)$$

(2) 第一类 Chebyshev 多项式满足递推公式

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad (n = 1, 2, \dots) \quad (6-25)$$

(3)  $T_n(x)$  在  $[-1, 1]$  上有  $n$  个互异零点

$$x_i = \cos \frac{2i-1}{2n} \pi \quad (i = 1, 2, \dots, n)$$

(4)  $T_n(x)$  在  $[-1, 1]$  上有  $n+1$  个极值点

$$x_k = \cos \frac{k\pi}{n} \quad (k = 0, 1, \dots, n).$$

且  $|T_n(x_k)| = 1 \quad (k = 0, 1, \dots, n)$

性质(3)和性质(4)由余弦函数性质即得.

### (三) 拉盖尔 (Laguerre) 多项式

Laguerre 多项式定义为

$$L_n(x) = e^x \frac{d^n}{dx^n} (x^n e^{-x}) \quad (n = 0, 1, 2, \dots) \quad (6-26)$$

其具体表达式为

$$\begin{aligned} L_0(x) &= 1 \\ L_1(x) &= -x + 1 \\ L_2(x) &= x^2 - 4x + 2 \\ L_3(x) &= -x^3 + 9x^2 - 18x + 6 \\ &\dots\dots\dots \end{aligned}$$

$$L_n(x) = \sum_{k=0}^n (-1)^k \binom{n}{k} \frac{n!}{k!} x^k \quad (n = 1, 2, \dots)$$

Laguerre 多项式有以下性质

(1)  $\{L_n(x)\}$  是区间  $[0, +\infty)$  上关于权函数  $\omega(x) = e^{-x}$  的正交多项式系, 且

$$(L_n, L_m) = \int_0^{+\infty} e^{-x} L_n(x) L_m(x) dx = \begin{cases} 0, & n \neq m \\ (n!)^2, & n = m \end{cases} \quad (6-27)$$

(2) Laguerre 多项式满足递推公式

$$L_{n+1}(x) = (2n+1-x)L_n(x) - x^2 L_{n-1}(x) \quad (n = 1, 2, \dots) \quad (6-28)$$

## § 6.3 函数的最佳平方逼近

有些函数由于表达式比较复杂而不易计算和研究,也需要用简单函数去近似它,这是函数逼近所研究的问题. 本节讨论如何用最小二乘法求连续函数的近似函数.

**定义 6.3** 设函数  $f(x)$  在区间  $[a, b]$  上连续,  $\varphi_i(x) (i = 0, 1, \dots, m)$  为定义在  $[a, b]$  上的一组线性无关的连续函数.  $H = \text{Span}\{\varphi_0, \varphi_1, \dots, \varphi_m\}$ . 如果函数

$$\varphi(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \dots + a_m\varphi_m(x) \quad (6-29)$$

使得

$$\begin{aligned} F(a_0, a_1, \dots, a_m) &= \int_a^b \omega(x) \left[ f(x) - \sum_{k=0}^m a_k \varphi_k(x) \right]^2 dx \\ &= \min_{\psi \in H} \int_a^b \omega(x) [f(x) - \psi(x)]^2 dx \end{aligned} \quad (6-30)$$

其中  $\omega(x)$  为权函数, 则称  $\varphi(x)$  为函数  $f(x)$  在  $H$  中关于权函数  $\omega(x)$  的最佳平方逼近函数.

特别地, 如果  $\varphi_k(x)$  为  $k$  次多项式 ( $k = 0, 1, \dots, m$ ), 则  $\varphi(x)$  称为  $f(x)$  在  $[a, b]$  上关于权函数  $\omega(x)$  的  $m$  次最佳平方逼近多项式或最小二乘逼近多项式.

在具体问题中, 权函数  $\omega(x)$  是给定的, 如果没有特别指明, 就表示  $\omega(x) \equiv 1$ .

与离散的情形类似, 由多元函数取极值的必要条件

$$\frac{\partial F}{\partial a_j} = -2 \int_a^b \omega(x) \left[ f(x) - \sum_{k=0}^m a_k \varphi_k(x) \right] \varphi_j(x) dx = 0 \quad (j = 0, 1, \dots, m) \quad (6-31)$$

得正则方程组

$$\sum_{k=0}^m a_k \int_a^b \omega(x) \varphi_k(x) \varphi_j(x) dx = \int_a^b \omega(x) f(x) \varphi_j(x) dx \quad (j = 0, 1, \dots, m)$$

改写成矩阵形式为

$$\begin{bmatrix} (\varphi_0, \varphi_0) & (\varphi_0, \varphi_1) & \cdots & (\varphi_0, \varphi_m) \\ (\varphi_1, \varphi_0) & (\varphi_1, \varphi_1) & \cdots & (\varphi_1, \varphi_m) \\ \vdots & \vdots & & \vdots \\ (\varphi_m, \varphi_0) & (\varphi_m, \varphi_1) & \cdots & (\varphi_m, \varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (f, \varphi_0) \\ (f, \varphi_1) \\ \vdots \\ (f, \varphi_m) \end{bmatrix} \quad (6-32)$$

其中  $(\varphi, \psi) = \int_a^b \omega(x) \varphi(x) \psi(x) dx$ . 由  $\varphi_0, \varphi_1, \dots, \varphi_m$  线性无关, 可导出方程组 (6-32) 的系数矩阵非奇异, 故解存在、唯一.

**定理 6.5** 设  $f(x)$  在  $[a, b]$  上连续,  $\varphi(x) = \sum_{k=0}^m a_k \varphi_k(x)$  是  $f(x)$  在  $H$  中关于权函数  $\omega(x)$  的最佳平方逼近函数的充要条件是

$$(f - \varphi, \varphi_j) = 0 \quad (j = 0, 1, \dots, m) \quad (6-33)$$

[证明] 必要性. 设  $\varphi(x)$  是  $f(x)$  在  $H$  中关于权函数  $\omega(x)$  的最佳平方逼近函数, 且存在  $i, 0 \leq i \leq m$ , 使得

$$(f - \varphi, \varphi_i) = r \neq 0$$

令

$$\psi(x) = \varphi(x) + \frac{r}{(\varphi_i, \varphi_i)} \varphi_i = \sum_{\substack{k=0 \\ k \neq i}}^m a_k \varphi_k(x) + \left[ a_i + \frac{r}{(\varphi_i, \varphi_i)} \right] \varphi_i(x)$$

显然,  $\psi(x) \in H$ , 且

$$\begin{aligned} & \int_a^b \omega(x) [f(x) - \psi(x)]^2 dx \\ &= (f - \psi, f - \psi) \\ &= (f - \varphi, f - \varphi) - 2 \left( f - \varphi, \frac{r}{(\varphi_i, \varphi_i)} \varphi_i \right) + \frac{r^2}{(\varphi_i, \varphi_i)^2} (\varphi_i, \varphi_i) \\ &= (f - \varphi, f - \varphi) - \frac{r^2}{(\varphi_i, \varphi_i)} < (f - \varphi, f - \varphi) \end{aligned}$$

这与  $\varphi(x)$  是最佳平方逼近函数矛盾, 故必要性成立.

充分性. 设式 (6-33) 成立, 则对任意  $\psi(x) \in H$ , 记  $\psi = \sum_{k=0}^m b_k \varphi_k$ , 都有

$$\begin{aligned} (f - \psi, f - \psi) &= (f - \varphi + \varphi - \psi, f - \varphi + \varphi - \psi) \\ &= (f - \varphi, f - \varphi) + 2(f - \varphi, \varphi - \psi) + (\varphi - \psi, \varphi - \psi) \end{aligned}$$

因为

$$\begin{aligned} (f - \varphi, \varphi - \psi) &= \left( f - \varphi, \sum_{k=0}^m (a_k - b_k) \varphi_k \right) \\ &= \sum_{k=0}^m (a_k - b_k) (f - \varphi, \varphi_k) = 0 \\ (\varphi - \psi, \varphi - \psi) &= \int_a^b \omega(x) [\varphi(x) - \psi(x)]^2 dx \geq 0 \end{aligned}$$

所以

$$(f - \psi, f - \psi) \geq (f - \varphi, f - \varphi)$$

故  $\varphi(x)$  是  $f(x)$  在  $H$  中关于权  $\omega(x)$  的最佳平方逼近函数.

由定理 6.5, 将方程组 (6-32) 的解代入式 (6-29) 所得的函数  $\varphi(x)$  必定是  $f(x)$  在  $H$  中关于权  $\omega(x)$  的最佳平方逼近函数, 且是唯一的.

如果取  $\varphi_k(x)$  为  $k$  次多项式, 且  $\varphi_0(x), \varphi_1(x), \dots, \varphi_m(x)$  为  $[a, b]$  上关于权

函数  $\omega(x)$  的正交多项式系, 则求函数  $f(x)$  的  $m$  次最佳平方逼近多项式就归结为求解正则方程组

$$\begin{bmatrix} (\varphi_0, \varphi_0) & & & \\ & (\varphi_1, \varphi_1) & & \\ & & \ddots & \\ & & & (\varphi_m, \varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (f, \varphi_0) \\ (f, \varphi_1) \\ \vdots \\ (f, \varphi_m) \end{bmatrix} \quad (6-34)$$

容易得出, 方程组 (6-34) 的解为

$$a_k = \frac{(f, \varphi_k)}{(\varphi_k, \varphi_k)} \quad (k = 0, 1, \dots, m) \quad (6-35)$$

所以,  $f(x)$  在  $[a, b]$  上的  $m$  次最佳平方逼近多项式为

$$\varphi(x) = \sum_{k=0}^m \frac{(f, \varphi_k)}{(\varphi_k, \varphi_k)} \varphi_k(x) \quad (6-36)$$

**例 6** 求  $f(x) = \sin \pi x$  在  $[0, 1]$  上的二次最佳平方逼近多项式.

[解] 若取  $\varphi_0(x) = 1, \varphi_1(x) = x, \varphi_2(x) = x^2$ , 容易求出此问题的正则方程组为三元 Hilbert 方程组, 故先按式 (6-16) 构造正交多项式.

$$\varphi_0(x) = 1$$

$$\alpha_1 = \frac{(x\varphi_0, \varphi_0)}{(\varphi_0, \varphi_0)} = \frac{\int_0^1 x dx}{\int_0^1 1 dx} = \frac{1}{2}$$

$$\varphi_1(x) = x - \frac{1}{2}$$

$$\alpha_2 = \frac{(x\varphi_1, \varphi_1)}{(\varphi_1, \varphi_1)} = \frac{\int_0^1 x \left(x - \frac{1}{2}\right)^2 dx}{\int_0^1 \left(x - \frac{1}{2}\right)^2 dx} = \frac{1}{2}$$

$$\beta_2 = \frac{(\varphi_1, \varphi_1)}{(\varphi_0, \varphi_0)} = \frac{\int_0^1 \left(x - \frac{1}{2}\right)^2 dx}{\int_0^1 dx} = \frac{1}{12}$$

$$\varphi_2(x) = \left(x - \frac{1}{2}\right)^2 - \frac{1}{12} = x^2 - x + \frac{1}{6}$$

于是

$$(\varphi_0, \varphi_0) = 1$$

$$(\varphi_1, \varphi_1) = \int_0^1 \left(x - \frac{1}{2}\right)^2 dx = \frac{1}{12}$$

$$(\varphi_2, \varphi_2) = \int_0^1 \left(x^2 - x + \frac{1}{6}\right)^2 dx = \frac{1}{180}$$

$$(f, \varphi_0) = \int_0^1 \sin \pi x dx = \frac{2}{\pi}$$

$$(f, \varphi_1) = \int_0^1 \left(x - \frac{1}{2}\right) \sin \pi x dx = 0$$

$$(f, \varphi_2) = \int_0^1 \left(x^2 - x + \frac{1}{6}\right) \sin \pi x dx = \frac{\pi^2 - 12}{3\pi^3}$$

所以,  $f(x) = \sin \pi x$  在  $[0, 1]$  上的二次最佳平方逼近多项式为

$$\begin{aligned} \varphi(x) &= \frac{(f, \varphi_0)}{(\varphi_0, \varphi_0)} \varphi_0(x) + \frac{(f, \varphi_1)}{(\varphi_1, \varphi_1)} \varphi_1(x) + \frac{(f, \varphi_2)}{(\varphi_2, \varphi_2)} \varphi_2(x) \\ &= \frac{2}{\pi} + \frac{\frac{\pi^2 - 12}{3\pi^3}}{\frac{1}{180}} \left(x^2 - x + \frac{1}{6}\right) \\ &= \frac{60\pi^2 - 720}{\pi^3} \left(x^2 - x + \frac{1}{6}\right) + \frac{2}{\pi} \\ &\approx -4.1225x^2 + 4.1225x - 0.05047 \end{aligned}$$

**例 7** 求  $f(x) = e^x$  在  $[-1, 1]$  上的三次最佳平方逼近多项式.

[解] Legendre 多项式是  $[-1, 1]$  上的正交多项式系, 故可取

$$\varphi_0(x) = P_0(x) = 1$$

$$\varphi_1(x) = P_1(x) = x$$

$$\varphi_2(x) = P_2(x) = \frac{1}{2}(3x^2 - 1)$$

$$\varphi_3(x) = P_3(x) = \frac{1}{2}(5x^3 - 3x)$$

由式(6-9)

$$(\varphi_k, \varphi_k) = \frac{2}{2k+1} \quad (k = 0, 1, 2, 3)$$

$$(f, \varphi_0) = \int_{-1}^1 e^x dx = e - e^{-1} \approx 2.3504$$

$$(f, \varphi_1) = \int_{-1}^1 x e^x dx = 2e^{-1} \approx 0.7358$$

$$(f, \varphi_2) = \int_{-1}^1 \frac{1}{2}(3x^2 - 1)e^x dx = e - 7e^{-1} \approx 0.1431$$

$$(f, \varphi_3) = \int_{-1}^1 \frac{1}{2}(5x^3 - 3x)e^x dx = -5e + 37e^{-1} \approx 0.02013$$

于是正则方程组之解为

$$a_0 = \frac{1}{2}(f, \varphi_0) \approx 1.175\,2, \quad a_1 = \frac{3}{2}(f, \varphi_1) \approx 1.103\,7$$

$$a_2 = \frac{5}{2}(f, \varphi_2) \approx 0.357\,8, \quad a_3 = \frac{7}{2}(f, \varphi_3) \approx 0.070\,64$$

所以函数  $f(x) = e^x$  在  $[-1, 1]$  上的三次最佳平方逼近多项式为

$$\begin{aligned} \varphi(x) &= 1.175\,2\varphi_0(x) + 1.103\,7\varphi_1(x) + 0.357\,8\varphi_2(x) + 0.070\,46\varphi_3(x) \\ &= 0.176\,2x^3 + 0.536\,7x^2 + 0.998\,0x + 0.996\,3 \end{aligned}$$

如果求  $f(x) = e^x$  在  $[0, 1]$  上的二次最佳平方逼近多项式, 则可用变数变换, 从 Legendre 多项式得出  $[0, 1]$  上的正交多项式  $\bar{P}_n(x) = P_n(2x - 1)$ . 取

$$\varphi_0(x) = \bar{P}_0(x) = 1$$

$$\varphi_1(x) = \bar{P}_1(x) = 2x - 1$$

$$\varphi_2(x) = \bar{P}_2(x) = 6x^2 - 6x + 1$$

于是

$$(\varphi_0, \varphi_0) = \int_0^1 dx = 1$$

$$(\varphi_1, \varphi_1) = \int_0^1 (2x - 1)^2 dx = \frac{1}{3}$$

$$(\varphi_2, \varphi_2) = \int_0^1 (6x^2 - 6x + 1)^2 dx = \frac{1}{5}$$

$$(f, \varphi_0) = \int_0^1 e^x dx = e - 1 \approx 1.718\,3$$

$$(f, \varphi_1) = \int_0^1 e^x (2x - 1) dx = 3 - e \approx 0.281\,7$$

$$(f, \varphi_2) = \int_0^1 e^x (6x^2 - 6x + 1) dx = 7e - 19 \approx 0.027\,97$$

由式(6-36)得  $f(x) = e^x$  在  $[0, 1]$  上的二次最佳平方逼近多项式为

$$\begin{aligned} \varphi(x) &= 1.718\,3\varphi_0(x) + 0.845\,1\varphi_1(x) + 0.139\,9\varphi_2(x) \\ &= 0.839\,4x^2 + 0.850\,8x + 1.013\,1 \end{aligned}$$

## § 6.4 应用实例

### 6.4.1 线性最小二乘拟合问题

由于噪音干扰, 函数值  $f(x) = 0.123\,5e^{-0.213\,8x} + 0.546\,7e^{-0.178\,5x} \cdot \sin(1.257\,4x)$ ,  $x \in (0, 10)$  的获取有一定的误差, 试用多项式拟合这组数据.



我们用 MATLAB 函数解决该问题. 首先利用 rand( ) 函数, 产生带有一定误差的拟合数据:

```
>> x=0:0.2:10;
>> y=0.123 5 * exp( -0.213 8 * x) +0.546 7 * exp( -0.178 5 * x) . * sin
(1.257 4 * x) +0.1 * (rand(1,51) -0.5);
```

再计算精确值, 并将拟合数据与函数显示于图 6-2 中:

```
>> y0=0.123 5 * exp( -0.213 8 * x) +0.546 7 * exp( -0.178 5 * x) . *
sin(1.257 4 * x);
```

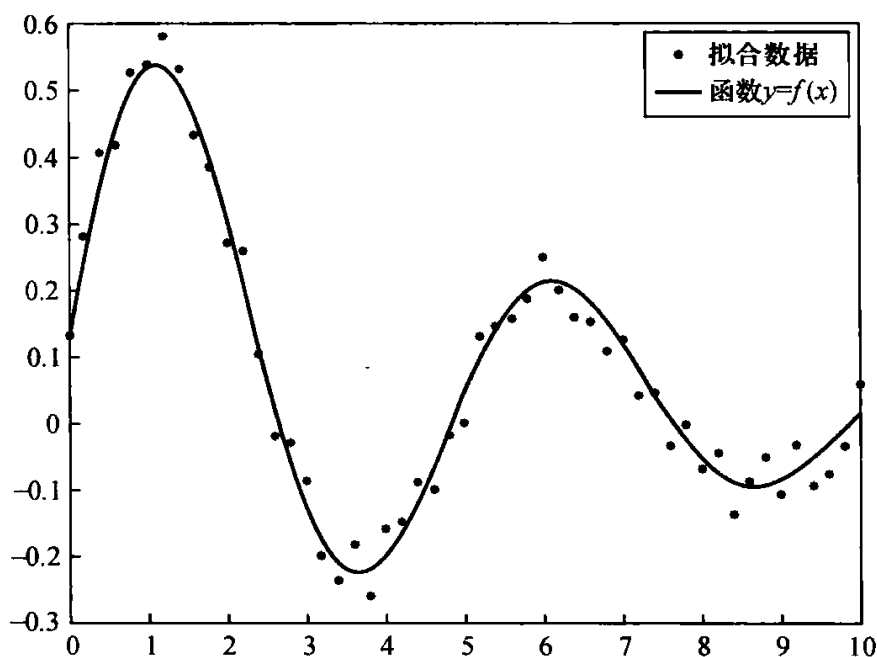


图 6-2

下面就这组数据进行 3 次、5 次、7 次和 9 次多项式拟合:

```
>> p3 = polyfit(x,y,3); p5 = polyfit(x,y,5); p7 = polyfit(x,y,7); p9 = polyfit
(x,y,9);
```

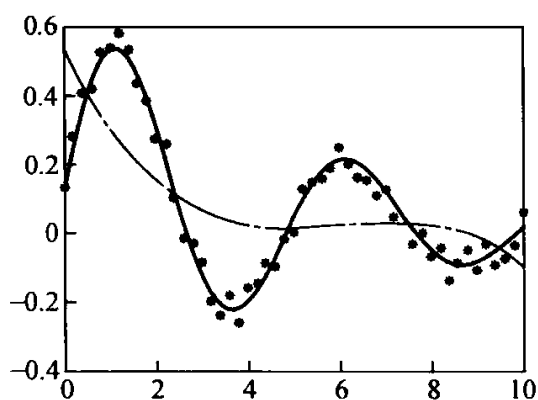
由命令 vpa( ) 显示各拟合多项式:

```
>> vpa(poly2sym(p3),6)
ans = -.262 872e-2 * x^3 + .479 786e-1 * x^2 - .281 000 * x + .545 488
>> vpa(poly2sym(p5),6)
ans = .107 083e-2 * x^5 - .274 462e-1 * x^4 + .247 724 * x^3 - .913 889 *
x^2 + 1.109 43 * x + .114 425
>> vpa(poly2sym(p7),6)
```

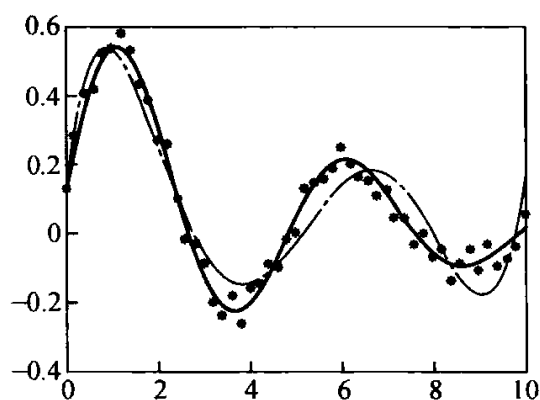
```
ans = -.473 482e-4 * x^7 + .153 976e-2 * x^6 - .182 857e-1 * x^5 +
.903 545e-1 * x^4 - .106 059 * x^3 - .445 388 * x^2 + .921 980 * x +
.110 266
>> vpa(poly2sym(p9),6)
ans = .213 933e-5 * x^9 - .845 363e-4 * x^8 + .129 198e-2 * x^7 - .918 972e
-2 * x^6
+.258 383e-1 * x^5 + .157 738e-1 * x^4 - .147 448 * x^3 - .168 718 * x^2
+.689 413 * x + .140 306
```

再调用命令 `polyval(*,*)` 计算拟合多项式在各节点上的值, 拟合效果见图6-3.

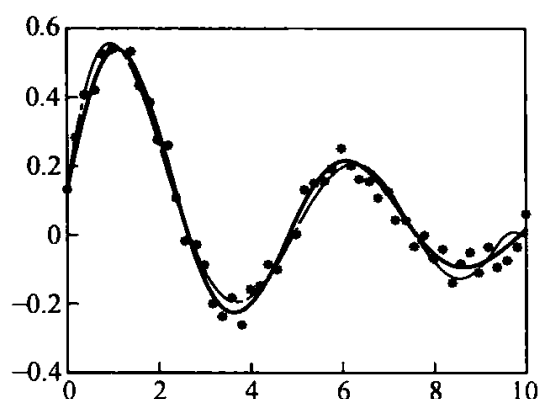
```
>> yp3 = polyval(p3,x); yp5 = polyval(p5,x); yp7 = polyval(p7,x); yp9 =
polyval(p9,x);
>> subplot(2,2,1); plot(x,y,'.',x,y0,'-',x,yp3,'-.')
>> subplot(2,2,2); plot(x,y,'.',x,y0,'-',x,yp5,'-.')
>> subplot(2,2,3); plot(x,y,'.',x,y0,'-',x,yp7,'-.')
>> subplot(2,2,4); plot(x,y,'.',x,y0,'-',x,yp9,'-')
```



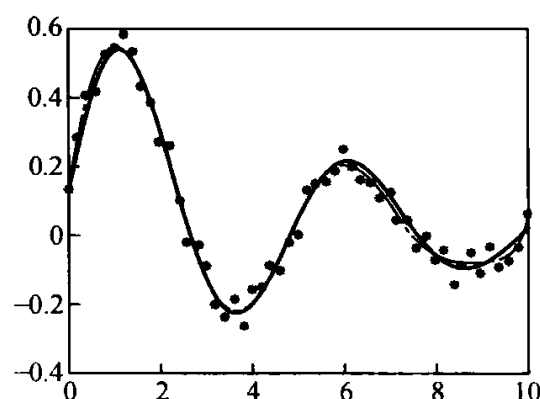
(a) 3次多项式拟合



(b) 5次多项式拟合



(c) 7次多项式拟合



(d) 9次多项式拟合

图 6-3

由图可见,3次、5次多项式拟合的效果并不理想,适当提高拟合多项式的次数,拟合效果得到明显的改善.但高次多项式最小二乘拟合产生的法方程组是病态的,会使舍入误差积累,影响数值解的精确度.如此例中用9次多项式拟合时,MATLAB软件会发出病态警告,我们从图6-3(d)中发现9次多项式拟合的整体效果是非常不错的,但拟合曲线的右端与原函数图形有较大差别,这就是由于问题的病态影响了解的精确度.读者可进行更高次多项式拟合实验.

### 6.4.2 非线性最小二乘拟合

实际问题中常遇到非线性最小二乘拟合问题,即拟合函数是待定参数的非线性函数,而且不能方便地将非线性问题转化为线性拟合问题,那么只能按最小二乘的原理,利用求极小值的方法建立关于待定参数是非线性的法方程组,再用求非线性方程组的方法求解法方程组,从而得到非线性最小二乘拟合函数.

例如,以前面的函数

$$f(x) = 0.1235e^{-0.2138x} + 0.5467e^{-0.1785x} \cdot \sin(1.2574x), \quad x \in (0, 10)$$

为例,产生拟合数据

```
>> x = 0 : 0.2 : 10;
>> y = 0.1235 * exp(-0.2138 * x) + 0.5467 * exp(-0.1785 * x) *
sin(1.2574 * x);
```

求形为  $f(x) = ae^{bx} + ce^{dx} \cdot \sin(fx)$ ,  $x \in (0, 10)$  的拟合函数. 这是非线性最小二乘拟合问题,问题归结为求  $a, b, c, d, f$ , 使得

$$I = \sum_i [ae^{bx_i} + ce^{dx_i} \cdot \sin(fx_i) - y_i]^2$$

达到极小值. 按最小二乘原理,对  $I$  关于  $a, b, c, d, f$  求偏导数得到法方程组

$$\begin{cases} \frac{\partial I}{\partial a} = \sum_i 2[ae^{bx_i} + ce^{dx_i} \sin(fx_i)] \cdot e^{bx_i} = 0 \\ \frac{\partial I}{\partial b} = \sum_i 2[ae^{bx_i} + ce^{dx_i} \sin(fx_i)] \cdot ax_i e^{bx_i} = 0 \\ \frac{\partial I}{\partial c} = \sum_i 2[ae^{bx_i} + ce^{dx_i} \sin(fx_i)] \cdot e^{dx_i} \sin(fx_i) = 0 \\ \frac{\partial I}{\partial d} = \sum_i 2[ae^{bx_i} + ce^{dx_i} \sin(fx_i)] \cdot cx_i e^{dx_i} \sin(fx_i) = 0 \\ \frac{\partial I}{\partial f} = \sum_i 2[ae^{bx_i} + ce^{dx_i} \sin(fx_i)] \cdot cx_i e^{dx_i} \cos(fx_i) = 0 \end{cases}$$

这是关于  $a, b, c, d, f$  的非线性方程组,如果不借助计算机,不能得到  $a, b, c, d, f$  的确切值. 我们当然可以利用 MATLAB 编程求解法方程组,得到拟合函数,但这样太麻烦,可直接调用最优化工具箱中的最小二乘曲线拟合函数 `lsqcurvefit()`

或求解无约束最优化函数 `fminsearch()` 得到拟合函数。

首先需要将函数  $f(x) = ae^{bx} + ce^{dx} \cdot \sin(fx)$  设为 MATLAB 中函数：

```
>> fx = inline('a(1) * exp(a(2) * x) + a(3) * exp(a(4) * x) * sin(a(5) * x)', 'a', 'x');
```

其中数组 `a` 是  $1 \times 5$  的向量, 表示待定参数. 然后调用函数 `lsqcurvefit()` 对前面的数据 `x, y` 求解拟合参数:

```
>> [aa, min] = lsqcurvefit(fx, [1, 1, 1, 1, 1], x, y); aa, min
Optimization terminated successfully:
Relative function value changing by less than OPTIONS.TolFun
aa = 0.1232 -0.2132 0.5472 -0.1787 1.2575
min = 5.9117e-007
```

由计算结果知, 计算值 `ans` 已非常接近精确值  $a = [0.1235 \quad -0.2138 \quad 0.5467 \quad -0.1785 \quad 1.2574]$ , 而且在该组参数下目标函数达到最小值  $\min = 5.9117e-007$ . 若希望提高精确度, 则可修改最优化函数的选项, 具体如下:

```
>> g = optimset; g.TolFun = 1e-20; g.Tolx = 1e-10;
>> [aa, min] = lsqcurvefit(fx, [1, 1, 1, 1, 1], x, y, [], [], g); aa, min
Optimization terminated successfully:
Relative function value changing by less than OPTIONS.TolFun
aa = 0.123 499 999 967 53 -0.213 799 999 939 32 0.546 700 000
049 16
-0.178 500 000 023 14 1.257 400 000 005 35
min = 5.937 363 642 829 464e-021
```

显然得到的参数已是非常精确了。

若得到的拟合数据有误差, 得到的将是对数据 `x, y` 的最小二乘拟合函数. 例如, 拟合数据带有噪音干扰:

```
>> x = 0:0.2:10;
>> y = 0.123 5 * exp(-0.213 8 * x) + 0.546 7 * exp(-0.178 5 * x) *
sin(1.257 4 * x) + 0.000 1 * (rand(1, 51) - 0.5);
```

同样调用函数 `lsqcurvefit()` 得到:

```
>> [aa, min] = lsqcurvefit(fx, [1, 1, 1, 1, 1], x, y, [], [], g); aa, min
Maximum number of function evaluations exceeded;
increase options.MaxFunEvals
aa = 0.123 530 738 876 30 -0.213 843 734 237 87 0.546 684 876 431 72
-0.178 486 782 701 81 1.257 401 745 397 35
```

$$\min = 3.219\ 200\ 874\ 165\ 107e-008$$

将得到的参数值  $aa$  与精确值进行比较,我们发现计算结果仅有四位有效数字.事实上,由于拟合数据本身有误差,已不可能再提高精确度了.

函数  $fminsearch()$  的功能和调用格式与  $lsqcurvefit()$  基本一致.

## 评 注

函数逼近是数学的一个经典分支,在科学与工程计算中有着广泛的应用.本章讨论了离散数据的线性最小二乘拟合、函数的最佳平方逼近的基本理论与基本方法,这些问题实际上是求连续函数子空间上的极值.由于最小二乘拟合和最佳平方逼近最后都归结为求解正则方程组,而正则方程组往往是病态的,这使得正交函数族有了特别重要的意义.本章介绍了正交多项式的基本性质和构造方法.正交多项式分离散型和连续型两种,离散型正交多项式适用于最小二乘拟合,连续型正交多项式适用于最佳平方逼近和数值积分中的高斯型求积公式.正交多项式因自变量定义域和权函数的不同而不同,使用时应注意.

## 习 题 六

1. 已知数据表如下:

|       |        |        |        |        |        |        |        |        |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|
| $x_i$ | 1.36   | 1.49   | 1.73   | 1.81   | 1.95   | 2.16   | 2.28   | 2.48   |
| $y_i$ | 14.094 | 15.069 | 16.844 | 17.378 | 18.435 | 19.949 | 20.963 | 22.495 |

试求其最小二乘一次拟合多项式.

2. 已知数据表如下:

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| $x_i$ | 1  | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| $y_i$ | 10 | 5 | 4 | 2 | 1 | 1 | 2 | 3 | 4  |

试求其最小二乘二次拟合多项式.

3. 某平炉的生产记录见下表:

| $i$   | 1   | 2   | 3   | 4   | 5   |
|-------|-----|-----|-----|-----|-----|
| $x_i$ | 165 | 123 | 150 | 121 | 141 |
| $y_i$ | 187 | 126 | 172 | 125 | 148 |

表中  $i$  为实验次数,  $x_i$  为全部炉料熔化完毕时钢液的含碳量,  $y_i$  为熔毕至出钢所需的冶炼时间, 求钢液含碳量与冶炼时间的函数关系.

4. 用最小二乘法求一形如  $y = a + bx^2$  ( $a, b$  为常数) 的经验公式拟合下列数据:

| $x_i$ | 19   | 25   | 31   | 38   | 44   |
|-------|------|------|------|------|------|
| $y_i$ | 19.0 | 32.3 | 49.0 | 73.3 | 97.8 |

5. 用最小二乘法求一形如  $y = ax + bx^3$  ( $a, b$  为常数) 的经验公式拟合以下数据:

| $x_i$ | -3     | -2    | -1    | 0     | 1    | 2    | 3     |
|-------|--------|-------|-------|-------|------|------|-------|
| $y_i$ | -28.49 | -8.99 | -1.51 | 0.001 | 1.47 | 9.02 | 28.42 |

6. 用最小二乘法求一形如  $y = ae^{bx}$  的经验公式拟合下列数据:

| $x_i$ | 1    | 2    | 3    | 4    | 5    | 6    | 7     | 8     |
|-------|------|------|------|------|------|------|-------|-------|
| $y_i$ | 15.3 | 20.5 | 27.4 | 36.6 | 49.1 | 65.6 | 87.87 | 117.6 |

7. 某化学反应中, 测得生成物的质量浓度  $y$  与时间  $t$  的关系见下表:

| $t_i$ | 1    | 2    | 3    | 4    | 6    | 8    | 10    | 12    | 14    | 16    |
|-------|------|------|------|------|------|------|-------|-------|-------|-------|
| $y_i$ | 4.00 | 6.41 | 8.01 | 8.79 | 9.53 | 9.86 | 10.33 | 10.42 | 10.53 | 10.61 |

试用最小乘法求一形如  $y = \frac{t}{at + b}$  的经验公式拟合以上数据.

8. 用形如  $y = ae^{\frac{b}{t}}$  的经验公式拟合第 7 题中数据, 并与第 7 题的结果比较 (残差平方和的大小描述拟合的好坏).

9. 设  $\{\varphi_k(x)\} (k=0,1,2,\dots)$  是区间  $[0,1]$  上权函数为  $\omega(x)=x$  的最高项系数为 1 的正交多项式系, 其中  $\varphi_0(x)=1$ , 试求  $\int_0^1 x\varphi_k(x)dx$  和  $\varphi_2(x)$ .

10. 证明函数系  $\{\varphi_0, \varphi_1, \dots, \varphi_n\}$  线性无关的充要条件是 Gram 行列式  $G(\varphi_0, \varphi_1, \dots, \varphi_n) \neq 0$ .

11. 利用正交函数族, 求下列函数的最小二乘三次拟合多项式:

(1)  $f(x) = \sin x, x_k = 0.2k, \omega_k = 1 \quad (k=0,1,\dots,5)$

(2)  $f(x) = \ln(1+x), x_k = 0.2, \omega_k = 1 \quad (k=0,1,\dots,5)$

(3)  $f(x) = e^{-x}, x_k = 0.2k, \omega_k = e^{0.2k} \quad (k=0,1,\dots,5)$

12. 求  $f(x) = \cos x, x \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$  在  $H = \text{Span}\{1, x^2, x^4\}$  中的最佳平方逼近多项式.

13. 求下列函数在指定区间上关于  $f(x)$  的一次最佳平方逼近多项式:

(1)  $f(x) = \sqrt{x}, [0,1]$

(2)  $f(x) = e^x, [0,1]$

(3)  $f(x) = \sin \frac{\pi}{2}x, [0,1]$

14. 对第 13 题中的函数和区间, 求二次最佳平方逼近多项式.

15. 利用 Legendre 多项式, 求  $f(x) = \sin x$  在  $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$  上的三次最佳平方逼近多项式  $P_3(x)$ , 并计算均方误差  $\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} [\sin x - P_3(x)]^2 dx$ .

## 数值实验

1. 研究误差函数  $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$  的拟合问题, 先利用 MATLAB 中的函数  $\text{erf}()$  产生 11 个数据点  $x_k = \frac{k-1}{10}, y_k = \text{erf}(x_k) (k=1,2,\dots,11)$ .

(1) 试分别用最高次为 1 至 10 的多项式进行最小二乘拟合, 比较各阶拟合多项式在数据点处与精确值  $\text{erf}(x)$  的误差, 最大误差与拟合多项式的最高次数的关系如何?

(2) 由于函数  $\text{erf}(x)$  是奇函数, 即  $\text{erf}(-x) = -\text{erf}(x)$ , 选择仅含有  $x$  的奇次幂项的多项式进行拟合, 即

$$\text{erf}(x) \approx a_1 x + a_2 x^3 + \dots + a_n x^{2n-1} \quad (n=1,2,\dots,10)$$

再分析误差与多项式最高次数之间的关系.

(3) 注意到函数  $\operatorname{erf}(x)$  当  $x \rightarrow \infty$  时, 有  $\operatorname{erf}(x) \rightarrow 1$ , 而多项式拟合函数是无界的, 因而多项式并不是合适的拟合函数. 令  $z = \frac{1}{1+x}$ , 选择拟合函数

$$\operatorname{erf}(x) \approx a_1 + e^{-x^2} (a_2 + a_3 z + a_4 z^2 + a_5 z^3)$$

求出拟合函数, 并分析误差情况.

2. 炼钢厂出钢时所用的盛钢水的钢包, 在使用过程中由于钢液及炉渣对包衬耐火材料的侵蚀, 使其容积不断增大. 经试验, 钢包的容积与相应的使用次数的数据列表如下:

| 使用次数 $x$ | 容积 $y$ | 使用次数 $x$ | 容积 $y$ |
|----------|--------|----------|--------|
| 2        | 106.42 | 11       | 110.59 |
| 3        | 108.26 | 12       | 110.60 |
| 5        | 109.58 | 14       | 110.72 |
| 6        | 109.50 | 16       | 110.90 |
| 7        | 109.86 | 17       | 110.76 |
| 9        | 110.00 | 19       | 111.10 |
| 10       | 109.93 | 20       | 111.30 |

选用双曲线  $\frac{1}{y} = a + b \frac{1}{x}$  对数据进行拟合, 用最小二乘法求出拟合函数, 作出拟合曲线图.

3. 设有一组实测数据:

| $x_i$ | 0.1     | 0.2     | 0.3     | 0.4     | 0.5     | 0.6     | 0.7     | 0.8     | 0.9     | 1.0     |
|-------|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| $y_i$ | 2.324 0 | 2.647 0 | 2.970 7 | 3.288 5 | 3.600 8 | 3.909 0 | 4.214 7 | 4.519 1 | 4.823 2 | 5.127 5 |

假设该组数据可能满足的函数为  $f(x) = ax + bx^2 e^{-cx} + d$ , 试求出最小二乘拟合函数的参数  $a, b, c, d$ , 并作图显示结果.



## 第七章 数值微分与数值积分

微积分学已经给出求函数导数及区间上积分的许多方法,但这些方法对一些实际问题往往难以实行.如实际问题常常只给出函数在一些离散点上的值,而无解析表达式;或虽有函数的解析表达式,却因其原函数不能用初等函数表示,或形式复杂,难以计算.因此需要建立利用离散点上函数的信息求函数导数及积分近似值的方法,通常称为数值微分与数值积分法.

### § 7.1 数值微分

由导数的定义

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

容易想到,当  $h$  充分小时,可用差商近似导数,这是最简单的数值微分公式.

#### 7.1.1 差商型求导公式

利用差商求导数的近似公式通常有以下几种:

(1) 向前差商公式

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (7-1)$$

(2) 向后差商公式

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \quad (7-2)$$

(3) 中心差商公式

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (7-3)$$

由泰勒(Taylor)公式很容易得到它们的余项公式

$$f'(x) - \frac{f(x+h) - f(x)}{h} = -\frac{f''(x + \theta_1 h)}{2}h = O(h)$$

$$f'(x) - \frac{f(x) - f(x-h)}{h} = \frac{f''(x - \theta_2 h)}{2}h = O(h)$$

$$f'(x) - \frac{f(x+h) - f(x-h)}{2h} = -\frac{f^{(3)}(x+\theta_1 h) + f^{(3)}(x-\theta_2 h)}{12} h^2 = O(h^2) \quad (7-4)$$

其中  $0 < \theta_1, \theta_2 < 1$ . 由式(7-4)可以看出, 用差商近似导数, 其精确度与步长  $h$  有关,  $h$  越小近似程度越高. 中心差商公式的余项与  $h^2$  同阶, 应比向前、向后差商公式的精确度高. 实际计算时, 如果  $h$  取得过小, 又会因有效数字损失而导致误差增大. 以  $f(x) = e^x$  为例的计算结果说明了上述事实(见表 7-1).

表 7-1

| $h$       | $x$       | $y = e^x$   | 向前差商 $D_+$ | 中心差商 $D$  | $f'(1.15) - D_+$ | $f'(1.15) - D$        |
|-----------|-----------|-------------|------------|-----------|------------------|-----------------------|
| $10^{-2}$ | 1.14      | 3.126 768 4 |            |           |                  |                       |
|           | 1.15      | 3.158 192 9 | 3.174 03   | 3.158 245 | -0.015 837 1     | -0.000 052 1          |
|           | 1.16      | 3.189 933 3 |            |           |                  |                       |
| $10^{-3}$ | 1.149     | 3.155 036 3 |            |           |                  |                       |
|           | 1.15      | 3.158 192 9 | 3.159 776  | 3.158 2   | -0.001 583 1     | $-7.1 \times 10^{-6}$ |
|           | 1.151     | 3.161 352 7 |            |           |                  |                       |
| $10^{-4}$ | 1.149 9   | 3.157 877 1 |            |           |                  |                       |
|           | 1.15      | 3.158 192 9 | 3.158 38   | 3.158     | -0.000 187 1     | 0.000 192 9           |
|           | 1.150 1   | 3.158 508 7 |            |           |                  |                       |
| $10^{-6}$ | 1.149 999 | 3.158 189 7 |            |           |                  |                       |
|           | 1.15      | 3.158 192 9 | 3.2        | 3.2       | -0.041 8         | -0.041 8              |
|           | 1.150 001 | 3.158 196 1 |            |           |                  |                       |

**例 1** 设  $f(x) = e^x$ , 分别取步长  $h = 10^{-2}, 10^{-3}, 10^{-4}, 10^{-6}$ , 用向前差商公式和中心差商公式计算  $f'(1.15)$ .

[解] 按式(7-1)和式(7-3), 计算结果如下:

表 7-1 结果说明, 当步长由  $10^{-2}$  缩小到  $10^{-4}$  时, 误差在减小. 进而缩小到  $10^{-6}$  时, 由于有效数字的丢失, 误差反而增大. 当步长不太小时, 中心差商公式的精度优于向前差商公式.

从几何上看, 如图 7-1 所示, 向前、向后、中心差商公式分别是以弦  $BC, AB, AC$  的斜率近似曲线  $y = f(x)$  在点  $B$  处的切线斜率. 一般地,  $AC$  的斜率与点  $B$  处切线的斜率更接近.

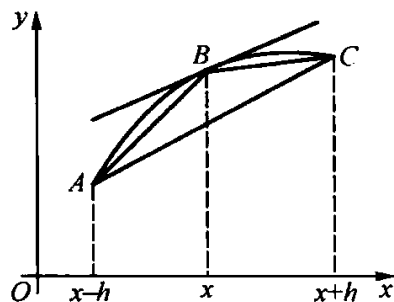


图 7-1

### 7.1.2 插值型求导公式

已知函数在一些离散点上的值时,可用插值多项式近似函数,因此可望用插值多项式的导数作为函数导数的近似值.

设已知函数  $f(x)$  在  $[a, b]$  内  $n+1$  个节点  $x_i (i=0, 1, \dots, n)$  处的函数值  $y_i = f(x_i) (i=0, 1, \dots, n)$ , 其插值多项式为  $\varphi_n(x)$ , 用  $\varphi_n(x)$  的导数近似函数导数, 即

$$f^{(k)}(x) \approx \varphi_n^{(k)}(x) \quad (k=1, 2, \dots) \quad (7-5)$$

式(7-5)称为插值型求导公式, 这是求数值微分的常用方法.

由定理 5.1, 只要  $f(x)$  在  $[a, b]$  内  $n+1$  阶可导, 插值余项为

$$R_n(x) = f(x) - \varphi_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) \quad (7-6)$$

将上式两端对  $x$  求导, 得

$$\begin{aligned} R'_n(x) &= f'(x) - \varphi'_n(x) = \frac{d}{dx} \left[ \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) \right] \\ &= \frac{\omega_{n+1}(x)}{(n+1)!} \frac{d}{dx} [f^{(n+1)}(\xi)] + \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x) \end{aligned} \quad (7-7)$$

这就是  $k=1$  时插值型求导公式(7-5)的截断误差.

对任意的  $x \in [a, b]$ , 因为  $\xi$  未知, 故由式(7-7)很难估计误差, 但若取  $x = x_i (i=0, 1, \dots, n)$ , 则因  $\omega_{n+1}(x_i) = 0$ , 误差就容易估计. 由式(7-7)有

$$\begin{aligned} R'_n(x_i) &= f'(x_i) - \varphi'_n(x_i) \\ &= \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega'_{n+1}(x_i) \\ &= \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{\substack{j=0 \\ j \neq i}}^n (x_i - x_j) \quad (i=0, 1, \dots, n) \end{aligned} \quad (7-8)$$

因此, 插值型求导公式通常用于求节点处导数的近似值.

下面给出等距节点情况下, 求一阶导数的几个常用公式.

1. 两点公式 ( $n=1$ ) 过节点  $x_0, x_1 = x_0 + h$  的 Lagrange 插值多项式为

$$L_1(x) = \frac{x - x_1}{-h} f(x_0) + \frac{x - x_0}{h} f(x_1)$$

故有

$$\begin{cases} f'(x_0) \approx L'_1(x_0) = \frac{f(x_1) - f(x_0)}{h} \\ f'(x_1) \approx L'_1(x_1) = \frac{f(x_1) - f(x_0)}{h} \end{cases} \quad (7-9)$$

截断误差为

$$\begin{cases} R'_1(x_0) = -\frac{h}{2}f''(\xi_0) \\ R'_1(x_1) = \frac{h}{2}f''(\xi_1) \end{cases} \quad (\xi_0, \xi_1 \in (x_0, x_1)) \quad (7-10)$$

2. 三点公式 过节点  $x_i = x_0 + ih (i = 0, 1, 2)$ ,  $f(x)$  的 Lagrange 插值多项式为

$$L_2(x) = \frac{(x-x_1)(x-x_2)}{2h^2}f(x_0) + \frac{(x-x_0)(x-x_2)}{-h^2}f(x_1) + \frac{(x-x_0)(x-x_1)}{2h^2}f(x_2)$$

求导得

$$L'_2(x) = \frac{2x-x_1-x_2}{2h^2}f(x_0) - \frac{2x-x_0-x_2}{h^2}f(x_1) + \frac{2x-x_0-x_1}{2h^2}f(x_2)$$

分别代入  $x_0, x_1, x_2$  得三点公式

$$\begin{cases} f'(x_0) \approx \frac{1}{2h}[-3f(x_0) + 4f(x_1) - f(x_2)] \\ f'(x_1) \approx \frac{1}{2h}[-f(x_0) + f(x_2)] \\ f'(x_2) \approx \frac{1}{2h}[f(x_0) - 4f(x_1) + 3f(x_2)] \end{cases} \quad (7-11)$$

截断误差为

$$\begin{cases} R'_2(x_0) = \frac{h^2}{3}f^{(3)}(\xi_0) \\ R'_2(x_1) = -\frac{h^2}{6}f^{(3)}(\xi_1), \xi_i \in (x_0, x_2) \quad (i = 0, 1, 2) \\ R'_2(x_2) = \frac{h^2}{3}f^{(3)}(\xi_2) \end{cases} \quad (7-12)$$

求二阶导数的三点公式为

$$f''(x_i) \approx L''_2(x_i) = \frac{1}{h^2}[f(x_0) - 2f(x_1) + f(x_2)] \quad (i = 0, 1, 2) \quad (7-13)$$

### 7.1.3 利用样条函数求数值微分

由第五章 §5 的讨论, 当步长  $h \rightarrow 0$  时, 三次样条函数  $S(x)$  及其一、二阶导数均一致收敛于被插值函数  $f(x)$  及其一、二阶导数, 故用样条函数的导数近似函数导数

$$f^{(k)}(x) \approx S^{(k)}(x) \quad (k = 1, 2)$$

不仅可靠性好,而且可以计算非节点处导数的近似值. 其截断误差为

$$f^{(k)}(x) - S^{(k)}(x) = O(h^{4-k})$$

以三转角关系式为例

$$S(x) = \left(1 + 2 \frac{x - x_i}{h_i}\right) \frac{(x - x_{i+1})^2}{h_i^2} f(x_i) + \left(1 - 2 \frac{x - x_{i+1}}{h_i}\right) \frac{(x - x_i)^2}{h_i^2} f(x_{i+1}) + \\ (x - x_i) \frac{(x - x_{i+1})^2}{h_i^2} m_i + (x - x_{i+1}) \frac{(x - x_i)^2}{h_i^2} m_{i+1} \quad (x_i \leq x \leq x_{i+1})$$

对  $x$  求导得

$$f'(x) \approx S'(x) = \frac{6}{h_i^3} [(x - x_{i+1})^2 + h_i(x - x_{i+1})] f(x_i) + \\ \frac{6}{h_i^3} [h_i(x - x_i) - (x - x_i)^2] f(x_{i+1}) + \\ \frac{1}{h_i^2} [3(x - x_{i+1})^2 + 2h_i(x - x_{i+1})] m_i + \\ \frac{1}{h_i^2} [3(x - x_i)^2 - 2h_i(x - x_i)] m_{i+1} \\ (x_i \leq x \leq x_{i+1}) \quad (7-14)$$

特别地,  $f'(x_i) \approx m_i (i=0, 1, \dots, n)$ ,  $m_i$  由三转角方程求得.

$$f''(x) \approx S''(x) = \frac{6}{h_i^3} [2(x - x_{i+1}) + h_i] f(x_i) + \frac{6}{h_i^3} [h_i - 2(x - x_i)] f(x_{i+1}) + \\ \frac{2}{h_i^2} [3(x - x_{i+1}) + h_i] m_i + \frac{2}{h_i^2} [3(x - x_i) - h_i] m_{i+1} \\ (x_i \leq x \leq x_{i+1}) \quad (7-15)$$

MATLAB 中的函数 `diff()` 可以直接求给定函数的各阶导数,其调用格式为

`>> y = diff(fx, x, n);` 或 `>> y = diff(fx, x);`

其中 `fx` 为预先设定的函数, `x` 为自变量, 它们应设定为符号变量, `n` 为求导的阶数, 若 `n` 省略则自动求一阶导数. 至于数值微分, 读者可根据本节介绍的各种公式编写 MATLAB 函数计算. 由于数值微分往往涉及较小的数做除数, 会影响计算结果的精确度, 所以一般利用样条函数求数值微分.

## § 7.2 牛顿 - 科茨 (Newton-Cotes) 求积公式

### 7.2.1 数值积分的基本思想

定积分的定义表明, 函数  $f(x)$  在  $[a, b]$  上的积分值是和式的极限

$$I(f) = \int_a^b f(x) dx = \lim_{\lambda \rightarrow 0} \sum_{i=1}^n f(\xi_i) \Delta x_i$$

其中  $\lambda = \max_{1 \leq i \leq n} \Delta x_i$ , 积分和式  $\sum_{i=1}^n f(\xi_i) \Delta x_i$  是一些点上函数值的线性组合. 构造数值积分方法的基本思想就是用被积函数在积分区间  $[a, b]$  上的某些节点  $x_k$  ( $k = 0, 1, \dots, n$ ) 处的函数值的线性组合作为定积分的近似值, 即

$$I(f) = \int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k) \quad (7-16)$$

其中  $\{A_k\}$  称为求积系数, 它只与节点  $\{x_k\}$  有关, 与被积函数  $f(x)$  无关.

由节点  $\{x_k\}$  和求积系数  $\{A_k\}$  的不同取法, 可以得到不同的求积公式. 最直接自然的一种想法是用  $f(x)$  在  $[a, b]$  上的插值多项式  $\varphi_n(x)$  代替  $f(x)$ , 以  $\varphi_n(x)$  在区间  $[a, b]$  上的积分值作为所求积分  $I(f)$  的近似值, 即

$$I(f) \approx \int_a^b \varphi_n(x) dx$$

这样得到的求积公式称为插值型求积公式. 通常采用 Lagrange 插值.

设  $[a, b]$  上有  $n+1$  个互异节点  $x_0, x_1, \dots, x_n$ ,  $f(x)$  的  $n$  次 Lagrange 插值多项式为

$$L_n(x) = \sum_{k=0}^n l_k(x) f(x_k)$$

其中  $l_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}$ , 插值型求积公式为

$$I(f) \approx \int_a^b L_n(x) dx = \sum_{k=0}^n \left( \int_a^b l_k(x) dx \right) f(x_k) \quad (7-17)$$

显然, 求积系数  $A_k = \int_a^b l_k(x) dx$  ( $k = 0, 1, \dots, n$ ) 仅与节点  $\{x_k\}$  有关, 与被积函数  $f(x)$  的具体形式无关. 求积公式 (7-17) 的截断误差为

$$\begin{aligned} R_n(f) &= \int_a^b f(x) dx - \int_a^b L_n(x) dx \\ &= \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) dx \end{aligned} \quad (7-18)$$

式 (7-18) 表明, 插值型求积公式的截断误差恰为插值余项在  $[a, b]$  上的定积分值.

## 7.2.2 Newton-Cotes 公式

如果取等距节点  $\{x_k\}$ , 则插值型求积公式更简便易算.

设  $x_k = a + kh$  ( $k = 0, 1, \dots, n$ ),  $h = \frac{b-a}{n}$ , 由式 (7-17)

$$\begin{aligned}
 A_k &= \int_a^b l_k(x) dx = \int_a^b \left( \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} \right) dx \\
 &= \int_a^b \left[ \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - a - jh}{(k - j)h} \right] dx
 \end{aligned}$$

令  $x = a + th$ , 则有

$$\begin{aligned}
 A_k &= \int_0^n \left[ \prod_{\substack{j=0 \\ j \neq k}}^n \frac{(t - j)h}{(k - j)h} \right] h dt \\
 &= \frac{(-1)^{n-k} h}{k!(n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt \\
 &= \frac{(-1)^{n-k} (b - a)}{k!(n-k)! n} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt
 \end{aligned}$$

令

$$C_k^{(n)} = \frac{(-1)^{n-k}}{k!(n-k)! n} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt \quad (7-19)$$

则  $A_k = (b - a) C_k^{(n)}$ , 求积公式(7-17)可简化为

$$I(f) = \int_a^b f(x) dx \approx (b - a) \sum_{k=0}^n C_k^{(n)} f(x_k) \quad (7-20)$$

式(7-20)称为  $n$  阶 Newton-Cotes 公式, 简记为 N - C 公式,  $\{C_k^{(n)}\}$  称为 Cotes 系数.

N - C 公式的截断误差为

$$\begin{aligned}
 R_n(f) &= \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j) dx \\
 &= \frac{h^{n+2}}{(n+1)!} \int_0^n f^{(n+1)}(\xi) \prod_{j=0}^n (t - j) dt
 \end{aligned} \quad (7-21)$$

因为  $C_k^{(n)}$  仅与等分区间数  $n$  有关, 与积分区间及被积函数均无关, 故可事先造好 Cotes 系数表(见表 7-2), 这样使用 N - C 公式计算定积分近似值更加方便.

例如,  $n=4$  时, N - C 公式为

$$\begin{aligned}
 I(f) &= \int_a^b f(x) dx \\
 &\approx \frac{b-a}{90} \left[ 7f(a) + 32f\left(a + \frac{b-a}{4}\right) + 12f\left(\frac{a+b}{2}\right) + \right. \\
 &\quad \left. 32f\left(a + \frac{3(b-a)}{4}\right) + 7f(b) \right]
 \end{aligned} \quad (7-22)$$

式(7-22)是常用的求积公式, 称为 Cotes 公式.

表 7-2

| $n$ | $C_k^{(n)}$      |                 |                  |                  |                 |                  |                  |
|-----|------------------|-----------------|------------------|------------------|-----------------|------------------|------------------|
| 1   | $\frac{1}{2}$    | $\frac{1}{2}$   |                  |                  |                 |                  |                  |
| 2   | $\frac{1}{6}$    | $\frac{4}{6}$   | $\frac{1}{6}$    |                  |                 |                  |                  |
| 3   | $\frac{1}{8}$    | $\frac{3}{8}$   | $\frac{3}{8}$    | $\frac{1}{8}$    |                 |                  |                  |
| 4   | $\frac{7}{90}$   | $\frac{16}{45}$ | $\frac{2}{15}$   | $\frac{16}{45}$  | $\frac{7}{90}$  |                  |                  |
| 5   | $\frac{19}{288}$ | $\frac{25}{96}$ | $\frac{25}{144}$ | $\frac{25}{144}$ | $\frac{25}{96}$ | $\frac{19}{288}$ |                  |
| 6   | $\frac{41}{840}$ | $\frac{9}{35}$  | $\frac{9}{280}$  | $\frac{34}{105}$ | $\frac{9}{280}$ | $\frac{9}{35}$   | $\frac{41}{840}$ |

特别地, 当  $n=1$  时,  $C_0^{(1)} = C_1^{(1)} = \frac{1}{2}$ , 因此有

$$I(f) = \int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)] \quad (7-23)$$

如图 7-2 所示, 这是以过点  $(a, f(a))$ ,  $(b, f(b))$  的直线代替曲线  $y=f(x)$ , 以梯形面积近似曲边梯形面积. 故式 (7-23) 又称为梯形公式.

当  $n=2$  时, N-C 公式为

$$I(f) = \int_a^b f(x) dx \approx \frac{b-a}{6} \left[ f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \quad (7-24)$$

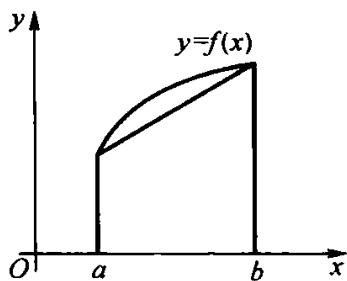


图 7-2

从几何上看, 这是以过曲线上三点  $(a, f(a))$ ,  $\left(\frac{a+b}{2}, f\left(\frac{a+b}{2}\right)\right)$ ,  $(b, f(b))$  的抛物线代替曲线  $y =$

$f(x)$  求曲边梯形面积的近似值, 因而此公式称为抛物线求积公式, 又称辛普森 (Simpson) 公式. 梯形公式与 Simpson 公式都是常用的求积公式.

### 7.2.3 误差估计

首先引入一种衡量数值积分公式近似程度的概念.

#### (一) 求积公式的代数精确度

**定义 7.1** 若当  $f(x)$  为任意次数不高于  $m$  的多项式时, 求积公式



$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k)$$

均精确成立,而对某个  $m+1$  次多项式,公式不精确成立,则称该求积公式具有  $m$  次代数精确度.

由定义容易看出,求积公式的代数精确度越高,使公式精确成立的多项式的次数越高,从而使用这个公式越可靠.因此,代数精确度在一定程度上确实能反映求积公式的近似程度.

不难验证,梯形公式具有一次代数精确度.因为若  $f(x)$  是一次多项式,则

$$R_1(x) = f(x) - L_1(x) = \frac{f''(\xi)}{2}(x-a)(x-b) = 0$$

从而有

$$\int_a^b f(x) dx = \int_a^b L_1(x) dx = \frac{b-a}{2}[f(a) + f(b)]$$

若取  $f(x) = x^2$ , 则

$$\int_a^b f(x) dx = \int_a^b x^2 dx = \frac{1}{3}(b^3 - a^3) \neq \frac{b-a}{2}(a^2 + b^2)$$

一般地,由  $n$  次插值多项式导出的求积公式至少有  $n$  次代数精确度.对 N-C 公式,更有以下结论.

**定理 7.1**  $2n$  阶 Newton-Cotes 公式至少具有  $2n+1$  次代数精确度.

[证明] 设  $P_{2n+1}(x) = a_{2n+1}x^{2n+1} + a_{2n}x^{2n} + \cdots + a_1x + a_0$  为  $2n+1$  次多项式,由式(7-21)

$$\begin{aligned} R_{2n}(P_{2n+1}) &= \int_a^b \frac{P_{2n+1}^{(2n+1)}(\xi)}{(2n+1)!} \prod_{j=0}^{2n} (x - x_j) dx \\ &= a_{2n+1} \int_a^b \prod_{j=0}^{2n} (x - x_j) dx \end{aligned}$$

令  $x = a + nh + th$  代入上式,得

$$\begin{aligned} R_{2n}(P_{2n+1}) &= a_{2n+1} h^{2n+2} \int_{-n}^n \prod_{j=0}^{2n} (t + n - j) dt \\ &= a_{2n+1} h^{2n+2} \int_{-n}^n (t+n)(t+n-1)\cdots(t+1)t(t-1)\cdots(t-n) dt \\ &= a_{2n+1} h^{2n+2} \int_{-n}^n t(t^2-1)(t^2-2^2)\cdots(t^2-n^2) dt \end{aligned}$$

上式中被积函数是奇函数,积分区间关于原点对称,故积分值为零,即

$$R_{2n}(P_{2n+1}) = 0$$

所以  $2n$  阶 N-C 公式至少有  $2n+1$  次代数精确度.

容易证明,求积公式(7-16)具有  $m$  次代数精确度的充要条件是它对  $f(x)$

$= 1, x, \dots, x^m$  都能准确成立, 但对  $f(x) = x^{m+1}$ , 式 (7-16) 不准确成立. 这一结论给出了判别一个求积公式的代数精确度的方法.

**例 2** 判别求积公式

$$\int_{-1}^1 f(x) dx \approx \frac{1}{9} [5f(\sqrt{0.6}) + 8f(0) + 5f(-\sqrt{0.6})] \quad (*)$$

的代数精确度.

[解] 记

$$I(f) = \int_{-1}^1 f(x) dx$$

$$\tilde{I}(f) = \frac{1}{9} [5f(\sqrt{0.6}) + 8f(0) + 5f(-\sqrt{0.6})]$$

因为

$$I(1) = \int_{-1}^1 dx = 2, \quad \tilde{I}(1) = \frac{1}{9}(5 + 8 + 5) = 2$$

$$I(x) = \int_{-1}^1 x dx = 0, \quad \tilde{I}(x) = \frac{1}{9}[5 \times \sqrt{0.6} + 8 \times 0 + 5 \times (-\sqrt{0.6})] = 0$$

$$I(x^2) = \int_{-1}^1 x^2 dx = \frac{2}{3}, \quad \tilde{I}(x^2) = \frac{1}{9}(5 \times 0.6 + 0 + 5 \times 0.6) = \frac{2}{3}$$

$$I(x^3) = \int_{-1}^1 x^3 dx = 0, \quad \tilde{I}(x^3) = \frac{1}{9}[5 \times (\sqrt{0.6})^3 + 0 + 5 \times (-\sqrt{0.6})^3] = 0$$

$$I(x^4) = \int_{-1}^1 x^4 dx = \frac{2}{5}, \quad \tilde{I}(x^4) = \frac{1}{9}(5 \times 0.36 + 0 + 5 \times 0.36) = \frac{2}{5}$$

$$I(x^5) = \int_{-1}^1 x^5 dx = 0, \quad \tilde{I}(x^5) = \frac{1}{9}[5 \times (\sqrt{0.6})^5 + 0 + 5 \times (-\sqrt{0.6})^5] = 0$$

$$I(x^6) = \int_{-1}^1 x^6 dx = \frac{2}{7}, \quad \tilde{I}(x^6) = \frac{1}{9}[5 \times 0.6^3 + 0 + 5 \times 0.6^3] = 0.24 \neq \frac{2}{7}$$

所以求积公式 (\*) 具有 5 次代数精确度.

利用余项公式及积分中值定理, 可以导出低阶 N - C 公式的误差估计.

## (二) 梯形公式和辛普森公式的误差估计

由式 (7-21), 梯形公式的截断误差为

$$R_1(f) = \int_a^b \frac{f''(\xi_x)}{2} (x-a)(x-b) dx, \quad \xi_x \in (a, b)$$

因为  $(x-a)(x-b)$  在区间  $[a, b]$  内不变号, 如果  $f(x)$  二阶连续可导, 则由反常积分中值定理, 存在  $\xi \in (a, b)$ , 使得

$$R_1(f) = \frac{f''(\xi)}{2} \int_a^b (x-a)(x-b) dx$$

$$\begin{aligned}
 &= \frac{f''(\xi)}{2} \left[ \frac{x^3}{3} - (a+b) \frac{x^2}{2} + abx \right]_a^b \\
 &= -\frac{f''(\xi)}{12} (b-a)^3
 \end{aligned}$$

**定理 7.2** 若  $f(x) \in C^{(2)}[a, b]$ , 则梯形公式的误差为

$$R_1(f) = \int_a^b f(x) dx - \frac{b-a}{2} [f(a) + f(b)] = -\frac{(b-a)^3}{12} f''(\xi), \quad \xi \in (a, b) \quad (7-25)$$

借助于  $f(x)$  的 Hermite 插值与定理 7.1, 可导出 Simpson 公式的误差估计.

**定理 7.3** 若  $f(x) \in C^{(4)}[a, b]$ , 则 Simpson 公式的误差为

$$\begin{aligned}
 R_2(f) &= \int_a^b f(x) dx - \frac{b-a}{6} \left[ f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \\
 &= -\frac{(b-a)^5}{2 \cdot 880} f^{(4)}(\xi) = -\frac{h^5}{90} f^{(4)}(\xi), \quad \xi \in (a, b)
 \end{aligned} \quad (7-26)$$

其中  $h = \frac{b-a}{2}$  为步长.

[证明] 作  $f(x)$  的三次插值多项式  $H_3(x)$ , 使其满足

$$\begin{aligned}
 H_3(a) &= f(a), \quad H_3(b) = f(b) \\
 H_3\left(\frac{a+b}{2}\right) &= f\left(\frac{a+b}{2}\right), \quad H'_3\left(\frac{a+b}{2}\right) = f'\left(\frac{a+b}{2}\right)
 \end{aligned}$$

由式(5-61), 插值余项为

$$R(x) = f(x) - H_3(x) = \frac{f^{(4)}(\xi_x)}{4!} (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b), \quad \xi_x \in (a, b)$$

定理 7.1 表明, Simpson 公式至少具有 3 次代数精确度, 故有

$$\begin{aligned}
 \int_a^b H_3(x) dx &= \frac{b-a}{6} \left[ H_3(a) + 4H_3\left(\frac{a+b}{2}\right) + H_3(b) \right] \\
 &= \frac{b-a}{6} \left[ f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]
 \end{aligned}$$

于是

$$\begin{aligned}
 R_2(f) &= \int_a^b f(x) dx - \frac{b-a}{6} \left[ f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right] \\
 &= \int_a^b [f(x) - H_3(x)] dx \\
 &= \int_a^b \frac{f^{(4)}(\xi_x)}{4!} (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) dx
 \end{aligned}$$

因为  $(x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b)$  在  $[a, b]$  内不变号,  $f(x) \in C^{(4)}[a, b]$ , 由反常积

分中值定理得

$$\begin{aligned} R_2(f) &= \frac{f^{(4)}(\xi)}{4!} \int_a^b (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) dx \\ &= -\frac{(b-a)^5}{2 \cdot 880} f^{(4)}(\xi) = -\frac{h^5}{90} f^{(4)}(\xi), \quad \xi \in (a, b) \end{aligned}$$

利用更复杂的运算可以导出 Cotes 公式的截断误差为

$$R_4(f) = -\frac{8}{945} h^7 f^{(6)}(\xi), \quad \xi \in (a, b) \quad (7-27)$$

其中  $h = \frac{b-a}{4}$ .

**例 3** 分别用梯形公式与 Simpson 公式计算积分

$$I = \int_0^1 e^x dx$$

的近似值,并估计截断误差.

[解] 由梯形公式(7-23)

$$I \approx \frac{1}{2}(e^0 + e^1) = 1.85914$$

截断误差为

$$|R_1| = \frac{1}{12} |f''(\xi)| \leq \frac{e}{12} = 0.22652$$

由 Simpson 公式(7-24)

$$I \approx \frac{1}{6}(e^0 + 4e^{\frac{1}{2}} + e^1) = 1.71886$$

截断误差为

$$|R_2| = \frac{1}{2 \cdot 880} |f^{(4)}(\xi)| \leq \frac{e}{2 \cdot 880} \approx 0.00094$$

计算结果表明,用 Simpson 公式计算的结果明显优于梯形公式的计算结果.

### § 7.3 复化求积公式

前面导出的误差估计式表明,用 N-C 公式计算积分近似值时,步长越小,截断误差越小.但缩小步长等于增加节点数,亦即提高插值多项式的次数,Runge 现象表明,这样做并不一定能提高精度.理论上已经证明,当  $n \rightarrow \infty$  时, N-C 公式所求得的近似值不一定收敛于积分的准确值,而且随着  $n$  的增大, N-C 公式是不稳定的.因此,实际中不常用高阶 N-C 公式,为提高计算精度,可考虑对被积函数用分段低次多项式插值,由此导出复化求积公式.

### 7.3.1 复化梯形公式

用分段线性插值函数近似被积函数, 等于把积分区间分成若干小区间, 在每个小区间上以梯形面积近似曲边梯形面积, 即用梯形公式求小区间上积分的近似值. 如图 7-3 所示, 这样求得的近似值显然比用梯形公式计算精度高. 定积分存在定理表明, 只要被积函数连续, 当小区间长度趋于零时, 小梯形面积之和趋于曲边梯形面积的准确值, 即定积分的准确值.

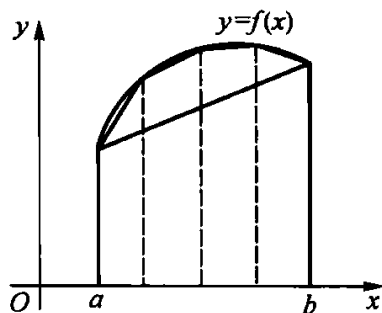


图 7-3

将积分区间  $[a, b]$  分成  $n$  等份, 记  $h = \frac{b-a}{n}$ ,  $x_k = a + kh$  ( $k = 0, 1, \dots, n$ ). 在每个小区间  $[x_k, x_{k+1}]$  ( $k = 0, 1, \dots, n-1$ ) 上用梯形公式并求和, 得

$$\int_a^b f(x) dx = \sum_{k=0}^{n-1} \int_{x_k}^{x_{k+1}} f(x) dx \approx \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})]$$

整理得

$$\int_a^b f(x) dx \approx \frac{h}{2} [f(a) + f(b) + 2 \sum_{k=1}^{n-1} f(x_k)] = T_n \quad (7-28)$$

式(7-28)称为复化梯形公式.

如果  $f(x) \in C^{(2)}[a, b]$ , 在小区间  $[x_k, x_{k+1}]$  上, 梯形公式的截断误差为

$$\int_{x_k}^{x_{k+1}} f(x) dx - \frac{h}{2} [f(x_k) + f(x_{k+1})] = -\frac{h^3}{12} f''(\xi_k), \quad \xi_k \in (x_k, x_{k+1})$$

因此

$$R_T(f) = \int_a^b f(x) dx - T_n = -\frac{h^3}{12} \sum_{k=0}^{n-1} f''(\xi_k)$$

因为  $f''(x)$  在  $[a, b]$  连续, 由介值定理, 存在  $\xi \in (a, b)$ , 使得

$$f''(\xi) = \frac{1}{n} \sum_{k=0}^{n-1} f''(\xi_k)$$

从而有

$$R_T(f) = \int_a^b f(x) dx - T_n = -\frac{h^3}{12} n f''(\xi) = -\frac{b-a}{12} h^2 f''(\xi), \quad \xi \in (a, b) \quad (7-29)$$

这就是复化梯形公式的截断误差.

下面简单讨论复化梯形公式的数值稳定性. 设计算函数值  $f(x_k)$  时产生误差为  $\delta_k$  ( $k = 0, 1, \dots, n$ ), 则用式(7-28)计算结果的误差为

$$\begin{aligned}
 |\delta| &= \frac{h}{2} \left| \delta_0 + \delta_n + 2 \sum_{k=1}^{n-1} \delta_k \right| \\
 &\leq nh \max_{0 \leq k \leq n} \{ |\delta_k| \} \\
 &= (b-a) \max_{0 \leq k \leq n} \{ |\delta_k| \}
 \end{aligned}$$

因此,无论  $n$  为多大,复化梯形公式是数值稳定的.

### 7.3.2 复化 Simpson 公式

如果用分段二次插值函数近似被积函数,即在小区间上用 Simpson 公式计算积分近似值,就导出复化 Simpson 公式.

将区间  $[a, b]$  分成  $n = 2m$  等份,分点为  $x_k = a + kh$  ( $k = 0, 1, \dots, n$ ),  $h = \frac{b-a}{n}$ , 在每个小区间  $[x_{2k-2}, x_{2k}]$  上,用 Simpson 公式求积分,则有

$$\begin{aligned}
 \int_{x_{2k-2}}^{x_{2k}} f(x) dx &\approx \frac{x_{2k} - x_{2k-2}}{6} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})] \\
 &= \frac{h}{3} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})]
 \end{aligned}$$

求和得

$$\begin{aligned}
 \int_a^b f(x) dx &= \sum_{k=1}^m \int_{x_{2k-2}}^{x_{2k}} f(x) dx \\
 &\approx \sum_{k=1}^m \frac{h}{3} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})]
 \end{aligned}$$

整理后得到

$$\int_a^b f(x) dx \approx \frac{h}{3} [f(a) + f(b) + 2 \sum_{k=1}^{m-1} f(x_{2k}) + 4 \sum_{k=1}^m f(x_{2k-1})] \quad (7-30)$$

式(7-30)称为复化 Simpson 公式.

如果  $f(x) \in C^{(4)}[a, b]$ , 由式(7-26)可得复化 Simpson 公式的截断误差为

$$\begin{aligned}
 R_s(f) &= \int_a^b f(x) dx - \frac{h}{3} [f(a) + f(b) + 2 \sum_{k=1}^{m-1} f(x_{2k}) + 4 \sum_{k=1}^m f(x_{2k-1})] \\
 &= \sum_{k=1}^m -\frac{(2h)^5}{2 \cdot 880} f^{(4)}(\xi_k), \quad \xi_k \in [x_{2k-2}, x_{2k}]
 \end{aligned}$$

因为  $f^{(4)}(x)$  连续, 故存在  $\xi \in (a, b)$ , 使得

$$f^{(4)}(\xi) = \frac{1}{m} \sum_{k=1}^m f^{(4)}(\xi_k)$$

代入上式得

$$R_s(f) = -\frac{(2h)^5}{2 \cdot 880} m f^{(4)}(\xi) = -\frac{b-a}{180} h^4 f^{(4)}(\xi), \quad \xi \in (a, b) \quad (7-31)$$

式(7-31)表明,步长  $h$  越小,截断误差越小.与复化梯形公式的分析相类似,可以证明,当  $n=2m \rightarrow \infty$  时,用复化 Simpson 公式所求得的近似值收敛于积分值,而且算法具有数值稳定性.

**例 4** 根据下面的函数表,计算积分  $I = \int_0^1 \frac{\sin x}{x} dx$  的近似值.

| $k$ | $x_k$ | $f(x_k) = \frac{\sin x_k}{x_k}$ | $k$ | $x_k$ | $f(x_k) = \frac{\sin x_k}{x_k}$ |
|-----|-------|---------------------------------|-----|-------|---------------------------------|
| 0   | 0     | 1.000 000                       | 5   | 0.625 | 0.936 156                       |
| 1   | 0.125 | 0.997 398                       | 6   | 0.75  | 0.908 852                       |
| 2   | 0.25  | 0.989 616                       | 7   | 0.875 | 0.877 193                       |
| 3   | 0.375 | 0.976 727                       | 8   | 1     | 0.841 471                       |
| 4   | 0.5   | 0.958 851                       |     |       |                                 |

[解] (1) 由复化梯形公式,有

$$I \approx \frac{1}{16} \left[ f(0) + f(1) + 2 \sum_{k=1}^7 f\left(\frac{k}{8}\right) \right] = 0.945 691$$

(2) 由复化 Simpson 公式,有

$$\begin{aligned} I &\approx \frac{1}{24} \left[ f(0) + f(1) + 2 \sum_{k=1}^3 f\left(\frac{k}{4}\right) + 4 \sum_{k=1}^4 f\left(\frac{2k-1}{8}\right) \right] \\ &= 0.946 084 \end{aligned}$$

与准确值  $I=0.946 083 1 \cdots$  比较,显然用复化 Simpson 公式计算精度较高.事实上,由误差公式(7-29)和式(7-31)有  $R_T(f) = O(h^2)$ ,  $R_S(f) = O(h^4)$ ,故当  $h$  比较小时,用复化 Simpson 公式计算误差较小.

由误差估计公式不仅可以计算所求近似值的误差,反之,亦可由给定的精度估计应取多大步长.

**例 5** 若用复化求积公式计算积分

$$I = \int_0^1 e^{-x} dx$$

的近似值,要求计算结果有四位有效数字,  $n$  应取多大?

[解] 因为当  $0 \leq x \leq 1$  时,有

$$0.3 < e^{-1} \leq e^{-x} \leq 1$$

于是

$$0.3 < \int_0^1 e^{-x} dx < 1$$

要求计算结果有四位有效数字,即要求误差不超过  $\frac{1}{2} \times 10^{-4}$ . 又因为

$$|f^{(k)}(x)| = e^{-x} \leq 1, \quad x \in [0, 1]$$

由式(7-29)

$$|R_r| = \frac{1}{12}h^2 |f''(\xi)| \leq \frac{h^2}{12} \leq \frac{1}{2} \times 10^{-4}$$

即

$$n^2 \geq \frac{1}{6} \times 10^4$$

开方得

$$n \geq 40.8$$

因此若用复化梯形公式求积分,  $n$  应等于 41 才能达到精度.

若用复化 Simpson 公式, 由式(7-31)

$$|R_s| = \frac{h^4}{180} |f^{(4)}(\xi)| \leq \frac{h^4}{180} \leq \frac{1}{2} \times 10^{-4}$$

即得  $n \geq 3.2$ . 故应取  $n = 4$ .

例 5 的计算结果表明, 为达到相同的精度, 用复化 Simpson 公式所需的计算量比复化梯形公式少, 这也说明了复化 Simpson 公式的精度较高.

### 7.3.3 逐次分半算法

如前所述, 复化求积公式的截断误差随着步长的缩小而减小, 而且由给定的精度可以预先确定步长. 但由于误差估计式中出现高阶导数, 使问题变得十分困难. 实际计算时, 通常是从某个步长  $h$  出发计算近似值, 然后在前次分割的基础上, 再将每个小区间等分, 即将积分区间逐次分半, 并以前后两次计算结果之差来估计误差, 直到求得满足精度的积分近似值. 这样既缩小了步长, 又保留了已有的节点, 减少了计算量. 这就是逐次分半法.

#### (一) 梯形公式的逐次分半算法

将区间  $[a, b]$  分成  $n = 2^m$  ( $m = 0, 1, 2, \dots$ ) 等份, 记  $h_m = \frac{b-a}{2^m}$ , 记

$$T_{2^m} = \frac{h_m}{2} \left[ f(a) + f(b) + 2 \sum_{k=1}^{2^m-1} f(a + kh_m) \right] \quad (m = 0, 1, 2, \dots)$$

(7-32)

称  $\{T_{2^m}\}$  为梯形值序列. 因为区间逐次分半, 第  $m-1$  次计算的节点是第  $m$  次计算时的偶数节点, 因而在计算  $T_{2^m}$  时只需计算新节点处的函数值. 由复化梯形公式(7-28), 有

$$T_{2^m} = \frac{h_m}{2} \left[ f(a) + f(b) + 2 \sum_{k=1}^{2^m-1} f(a + kh_m) \right]$$



$$\begin{aligned}
&= \frac{h_m}{2} \left[ f(a) + f(b) + 2 \sum_{k=1}^{2^{m-1}-1} f(a + 2kh_m) + 2 \sum_{k=1}^{2^{m-1}} f(a + (2k-1)h_m) \right] \\
&= \frac{h_m}{2} \left[ f(a) + f(b) + 2 \sum_{k=1}^{2^{m-1}-1} f(a + kh_{m-1}) + 2 \sum_{k=1}^{2^{m-1}} f(a + (2k-1)h_m) \right]
\end{aligned}$$

于是得到递推公式

$$T_{2^m} = \frac{1}{2} T_{2^{m-1}} + h_m \sum_{k=1}^{2^{m-1}} f(a + (2k-1)h_m) \quad (m = 1, 2, \dots) \quad (7-33)$$

式(7-33)称为梯形公式的逐次分半计算公式.

复化梯形公式的误差估计式表明,由式(7-33)所生成的序列  $\{T_{2^m}\}$  收敛于积分值  $\int_a^b f(x) dx$ , 其误差为

$$\begin{aligned}
R(f, T_{2^m}) &= \int_a^b f(x) dx - T_{2^m} \\
&= -\frac{b-a}{12} h_m^2 f''(\xi_m), \quad \xi_m \in (a, b)
\end{aligned}$$

同时有

$$R(f, T_{2^{m-1}}) = -\frac{b-a}{12} h_{m-1}^2 f''(\xi_{m-1}), \quad \xi_{m-1} \in (a, b)$$

将以上两式相减得

$$\begin{aligned}
T_{2^m} - T_{2^{m-1}} &= \frac{b-a}{12} [h_m^2 f''(\xi_m) - h_{m-1}^2 f''(\xi_{m-1})] \\
&= -\frac{b-a}{12} h_m^2 [4f''(\xi_{m-1}) - f''(\xi_m)]
\end{aligned}$$

由 3.1 段的讨论,  $f''(\xi_{m-1})$  与  $f''(\xi_m)$  分别是  $f''(x)$  在  $[a, b]$  上  $2^{m-1}$  个点与  $2^m$  个点处的算术平均值(每个小区间取一个点). 如果  $f(x) \in C^{(2)}[a, b]$ , 则当  $m$  较大时, 有  $f''(\xi_{m-1}) \approx f''(\xi_m)$ , 从而有

$$T_{2^m} - T_{2^{m-1}} \approx -3 \frac{b-a}{12} h_m^2 f''(\xi_m) = 3R(f, T_{2^m})$$

即

$$|R(f, T_{2^m})| \approx \frac{1}{3} |T_{2^m} - T_{2^{m-1}}|$$

因此,若事先给定误差限  $\varepsilon$ , 则当  $|T_{2^m} - T_{2^{m-1}}| < 3\varepsilon$  时就可停止计算, 并认为  $T_{2^m}$  是满足精度要求的近似值. 以计算所得的近似值来估计误差称为事后误差估计.

### 算法 7.1

1. 输入  $a, b, f(x), \varepsilon$ .

2. 置  $m = 1, h = \frac{b-a}{2}, T_0 = h[f(a) + f(b)]$ .

3. 置  $F = 0$ , 对  $k = 1, 2, \dots, 2^{m-1}$

$$F = F + f(a + (2k-1)h)$$

4.  $T = \frac{1}{2}T_0 + hF$ .

5. 若  $|T - T_0| < 3\varepsilon$ , 输出  $I \approx T$ , 停机; 否则  $m+1 \Rightarrow m, \frac{h}{2} \Rightarrow h, T \Rightarrow T_0$ , 转 3.

完全类似地, 可以导出

### (二) Simpson 公式的逐次分半算法

将区间  $[a, b]$  分成  $2^m$  等份,  $h_m = \frac{b-a}{2^m}$ , 记

$$S_{2^m} = \frac{h_m}{3} \left[ f(a) + f(b) + 2 \sum_{k=1}^{2^{m-1}-1} f(a + 2kh_m) + 4 \sum_{k=1}^{2^{m-1}} f(a + (2k-1)h_m) \right] \quad (7-34)$$

按式(7-34)计算所得的序列  $\{S_{2^m}\}$  称为 Simpson 序列. 当  $m \rightarrow \infty$  时,  $\{S_{2^m}\}$  收敛于  $\int_a^b f(x) dx$ .

由复化 Simpson 公式的误差式(7-31)可得

$$\begin{aligned} R(f, S_{2^m}) &= \int_a^b f(x) dx - S_{2^m} \\ &\approx \frac{1}{15} (S_{2^m} - S_{2^{m-1}}) \end{aligned}$$

因此, Simpson 公式的逐次分半算法以  $|S_{2^m} - S_{2^{m-1}}| < 15\varepsilon$  为停步准则, 此时  $I \approx S_{2^m}$  即为满足精度要求的积分近似值.

#### 算法 7.2

1. 输入  $a, b, f(x), \varepsilon$ .

2. 置  $F_1 = f(a) + f(b), F_2 = f\left(\frac{a+b}{2}\right), S_0 = \frac{b-a}{6} (F_1 + 4F_2) \quad m = 2,$

$$h = \frac{b-a}{4}.$$

3. 置  $F_3 = 0$ , 对  $k = 1, 2, \dots, 2^{m-1}, F_3 = F_3 + f(a + (2k-1)h)$ .

4.  $S = \frac{h}{3} (F_1 + 2F_2 + 4F_3)$ .

5. 若  $|S - S_0| < 15\varepsilon$ , 则输出  $I \approx S$ , 停机; 否则, 置  $m+1 \Rightarrow m, \frac{h}{2} \Rightarrow h, F_2 \Rightarrow F_3, S \Rightarrow S_0$ , 转 3.

**例 6** 用 Simpson 公式的逐次分半算法按例 4 所给数据表计算积分  $I = \int_0^1 \frac{\sin x}{x} dx$  的近似值, 要求误差不超过  $10^{-6}$ .

[解] 按式(7-34)计算

$$S_2 = \frac{1}{6}(1 + 0.841\,471 + 4 \times 0.958\,851) = 0.946\,146$$

$$S_4 = \frac{1}{12}[1 + 0.841\,471 + 2 \times 0.958\,851 + 4(0.989\,616 + 0.908\,852)] \\ = 0.946\,087$$

$$\frac{1}{15} |S_4 - S_2| = 3.93 \times 10^{-6} > 10^{-6}$$

$$S_8 = 0.946\,084$$

$$\frac{1}{15} |S_8 - S_4| = \frac{3}{15} \times 10^{-6}$$

所以

$$I \approx S_8 = 0.946\,084$$

在实际计算时, 常以  $|T_{2m} - T_{2m-1}| < \varepsilon$  与  $|S_{2m} - S_{2m-1}| < \varepsilon$  作为逐次分半算法的停步准则.

根据算法 7.2, 编写逐次分半 Simpson 算法的 MATLAB 函数如下:

```
function I = simpsonstep( fx, a, b, eps)           % fx 为设定的被积函数
h = b - a; c = (a + b)/2; m = 1; h = h/2;
f1 = feval( fx, a) + feval( fx, b); f2 = feval( fx, c); f3 = 0; % feval( ) 为求函数 fx 在各点的值
s0 = 0; s = h/6 * (f1 + 4 * f2);
while (abs(s - s0) >= 15 * eps)
    m = m + 1; h = h/2; n = 2^m; f2 = f2 + f3; s0 = s;
    x = a + h; 2 * h; b - h; f3 = sum( feval( fx, x) );
    s = h/3 * (f1 + 2 * f2 + 4 * f3);
end
I = s;
```

例如计算例 6 中的积分  $I = \int_0^1 \frac{\sin x}{x} dx$ , 要求误差不超过  $10^{-14}$ . 若只是简单地命令

```
>> fx = inline('sin(x)./x','x');
>> I = simpsonstep( fx, 0, 1, 1e-14);
```

将会导致计算  $fx(0)$  时出现除数为 0 的错误信息, 逐次分半算法将无法进行下去. 一种补救的方法是将积分下限由 0 修改为最小的浮点数, 另一种补救的方法是利用被积函数的性质将被积函数设置为如下的 M-函数:

```
function y = fx(x)
if x == 0 y = 1; else y = sin(x) ./ x;
end
```

然后调用函数 `simpsonstep( )`:

```
>> I = simpsonstep(@fx, 0, 1, 1e-14);
```

计算结果列于下表中.

| m  | n     | h                | I                    |
|----|-------|------------------|----------------------|
| 2  | 4     | 0.250 000 00     | 0.946 086 933 951 79 |
| 3  | 8     | 0.125 000 00     | 0.946 083 310 888 47 |
| 4  | 16    | 0.062 500 00     | 0.946 083 085 384 95 |
| 5  | 32    | 0.031 250 00     | 0.946 083 071 305 56 |
| 6  | 64    | 0.015 625 00     | 0.946 083 070 425 83 |
| 7  | 128   | 0.007 812 50     | 0.946 083 070 370 85 |
| 8  | 256   | 0.003 906 25     | 0.946 083 070 367 41 |
| 9  | 512   | 1.953 125 00e-3  | 0.946 083 070 367 20 |
| 10 | 1 024 | 9.765 625 0e-004 | 0.946 083 070 367 18 |
| 11 | 2 048 | 4.882 812 5e-004 | 0.946 083 070 367 18 |

MATLAB 也提供了基于复化梯形公式的函数 `trapz( )` 和基于自适应复化 simpson 公式的函数 `quad( )`、`quadl( )`, 它们的调用格式为

```
>> s = trapz(x, y); % x 为自变量的节点向量, y 为相应节点处函数
                    值向量.
>> s = quadl(fx, a, b, eps); % fx 为被积函数, a、b 为积分下、上限, eps 为误
                    差限.
```

注: 目前的 MATLAB 版本建议使用函数 `quadl( )`, 它的精度和计算效率均优于函数 `quad( )`.

## § 7.4 龙贝格 (Romberg) 求积公式

### 7.4.1 理查逊 (Richardson) 外推法

式(7-29)表明,复化梯形公式的截断误差为  $O(h^2)$ ,因而梯形值序列  $\{T_{2^m}\}$  收敛较慢.事实上,进一步分析可得出

$$R(f, T_n) = a_1 h^2 + a_2 h^4 + \cdots + a_k h^{2k} + \cdots \quad (7-35)$$

其中系数  $a_1, a_2, \dots, a_k, \dots$  均与  $h$  无关,式(7-35)称为欧拉-麦克劳林 (Euler-Maclaurin) 公式.从此式出发,利用加速收敛的技巧,可以构造出快速收敛的近似值序列, Richardson 外推法就是这样一种方法.

设用某种数值方法求量  $I$  的近似值,一般地,近似值是步长  $h$  的函数,记为  $I_1(h)$ ,相应的误差为

$$I - I_1(h) = a_1 h^{p_1} + a_2 h^{p_2} + \cdots + a_k h^{p_k} + \cdots \quad (7-36)$$

其中  $0 < p_1 < p_2 < \cdots < p_k < \cdots$ ,  $a_i (i=1, 2, \dots)$  与  $h$  无关.若用  $\alpha h$  代替式(7-36)中的  $h$ ,则得

$$\begin{aligned} I - I_1(\alpha h) &= a_1 (\alpha h)^{p_1} + a_2 (\alpha h)^{p_2} + \cdots + a_k (\alpha h)^{p_k} + \cdots \\ &= a_1 \alpha^{p_1} h^{p_1} + a_2 \alpha^{p_2} h^{p_2} + \cdots + a_k \alpha^{p_k} h^{p_k} + \cdots \end{aligned} \quad (7-37)$$

式(7-37)减去式(7-36)乘以  $\alpha^{p_1}$ ,得

$$\begin{aligned} I - I_1(\alpha h) - \alpha^{p_1} [I - I_1(h)] \\ = a_2 (\alpha^{p_2} - \alpha^{p_1}) h^{p_2} + a_3 (\alpha^{p_3} - \alpha^{p_1}) h^{p_3} + \cdots + a_k (\alpha^{p_k} - \alpha^{p_1}) h^{p_k} + \cdots \end{aligned}$$

取  $\alpha$  满足  $|\alpha| \neq 1$ , 以  $1 - \alpha^{p_1}$  除上式两端,得

$$I - \frac{I_1(\alpha h) - \alpha^{p_1} I_1(h)}{1 - \alpha^{p_1}} = b_2 h^{p_2} + b_3 h^{p_3} + \cdots + b_k h^{p_k} + \cdots \quad (7-38)$$

其中  $b_i = a_i (\alpha^{p_i} - \alpha^{p_1}) / (1 - \alpha^{p_1}) (i=2, 3, \dots)$  仍与  $h$  无关.令

$$I_2(h) = \frac{I_1(\alpha h) - \alpha^{p_1} I_1(h)}{1 - \alpha^{p_1}}$$

由式(7-38),以  $I_2(h)$  作为  $I$  的近似值,其误差至少为  $O(h^{p_2})$ ,因此  $I_2(h)$  收敛于  $I$  的速度比  $I_1(h)$  快.不断重复以上做法,可以得到一个函数序列

$$I_m(h) = \frac{I_{m-1}(\alpha h) - \alpha^{p_{m-1}} I_{m-1}(h)}{1 - \alpha^{p_{m-1}}} \quad (m = 2, 3, \dots)$$

以  $I_m(h)$  近似  $I$ ,误差为  $I - I_m(h) = O(h^{p_m})$ .随着  $m$  的增大,收敛速度越来越快,这就是 Richardson 外推法.将此方法用于梯形值序列  $\{T_{2^k}\}$ ,可以构造出一种收敛很快,计算简便的数值积分方法.

### 7.4.2 Romberg 求积公式

设  $I = \int_a^b f(x) dx$ , 记  $2^k$  等分区间  $[a, b]$  用复化梯形公式所求得的近似值为

$T_{2^k} = T_0^{(k)} = I_1\left(\frac{b-a}{2^k}\right)$  ( $k=0, 1, 2, \dots$ ). 由 Euler-Maclaurin 公式

$$R(f, T_0^{(k)}) = a_1\left(\frac{b-a}{2^k}\right)^2 + a_2\left(\frac{b-a}{2^k}\right)^4 + \dots + a_i\left(\frac{b-a}{2^k}\right)^{2i} + \dots$$

取  $\alpha = \frac{1}{2}$ , 由 Richardson 外推公式

$$I_2\left(\frac{b-a}{2^k}\right) = \frac{I_1\left(\frac{b-a}{2^{k+1}}\right) - \left(\frac{1}{2}\right)^2 I_1\left(\frac{b-a}{2^k}\right)}{1 - \left(\frac{1}{2}\right)^2} = \frac{4T_0^{(k+1)} - T_0^{(k)}}{3}$$

记  $I_2\left(\frac{b-a}{2^k}\right) = T_1^{(k)}$ , 则

$$T_1^{(k)} = \frac{4T_0^{(k+1)} - T_0^{(k)}}{3} \quad (k=0, 1, 2, \dots) \quad (7-39)$$

且有

$$R(f, T_1^{(k)}) = O\left(\left(\frac{b-a}{2^k}\right)^4\right)$$

事实上, 容易验证,  $\{T_1^{(k)}\}$  就是 Simpson 值序列  $\{S_{2^k}\}$ . 记  $h_k = \frac{b-a}{2^k}$ , 则由复化梯形公式有

$$\begin{aligned} 4T_0^{(k)} - T_0^{(k-1)} &= 2h_k[f(a) + f(b) + 2\sum_{i=1}^{2^{k-1}} f(a + ih_k)] - \\ &\quad \frac{h_{k-1}}{2}[f(a) + f(b) + 2\sum_{i=1}^{2^{k-1}-1} f(a + ih_{k-1})] \\ &= h_k[f(a) + f(b) + 4\sum_{j=1}^{2^{k-1}} f(a + (2j-1)h_k) + 2\sum_{j=1}^{2^{k-1}-1} f(a + 2jh_k)] \\ &= 3S_{2^k} \end{aligned}$$

于是

$$S_{2^k} = \frac{4T_0^{(k)} - T_0^{(k-1)}}{3} = T_1^{(k-1)}$$

重复 Richardson 公式, 可得

$$\begin{aligned}
 I_{m+1}\left(\frac{b-a}{2^k}\right) &= \frac{I_m\left(\frac{b-a}{2^{k+1}}\right) - \left(\frac{1}{2}\right)^{2m} I_m\left(\frac{b-a}{2^k}\right)}{1 - \left(\frac{1}{2}\right)^{2m}} \\
 &= \frac{4^m I_m\left(\frac{b-a}{2^{k+1}}\right) - I_m\left(\frac{b-a}{2^k}\right)}{4^m - 1}
 \end{aligned}$$

记  $T_m^{(k)} = I_{m+1}\left(\frac{b-a}{2^k}\right)$ , 则上式可记为

$$T_m^{(k)} = \frac{4^m T_{m-1}^{(k+1)} - T_{m-1}^{(k)}}{4^m - 1} \quad (m = 1, 2, \dots; k = 0, 1, 2, \dots) \quad (7-40)$$

且  $R(f, T_m^{(k)}) = O\left(\left(\frac{b-a}{2^k}\right)^{2m+2}\right)$ . 式(7-40)称为 Romberg 求积公式, 其中  $T_0^{(k)}$  由复化梯形公式求得. Romberg 公式的计算过程见表 7-3, 按行依次计算  $T_0^{(0)}, T_0^{(1)}, T_0^{(2)}, T_1^{(1)}, T_2^{(0)}, \dots$ .

表 7-3

| $k$ | $T_0^{(k)}$      | $T_1^{(k)}$   | $T_2^{(k)}$   | $T_3^{(k)}$   | ... |
|-----|------------------|---------------|---------------|---------------|-----|
| 0   | $T_0^{(0)}$<br>↓ |               |               |               |     |
| 1   | $T_0^{(1)}$      | → $T_1^{(0)}$ |               |               |     |
| 2   | $T_0^{(2)}$      | → $T_1^{(1)}$ | → $T_2^{(0)}$ |               |     |
| 3   | $T_0^{(3)}$      | → $T_1^{(2)}$ | → $T_2^{(1)}$ | → $T_3^{(0)}$ |     |

表 7-3 中第一列是梯形序列值,  $I - T_0^{(k)} = O\left(\left(\frac{b-a}{2^k}\right)^2\right)$ . 以后各列则是对前一列用 Richardson 外推算法得到, 误差为  $I - T_m^{(k)} = O\left(\left(\frac{b-a}{2^k}\right)^{2m+2}\right)$ , 因而,  $m$  越大, 近似值序列  $\{T_m^{(k)}\}$  的收敛速度越快.

按式(7-33)和式(7-40)计算, 以  $T_m^{(0)} (m = 0, 1, 2, \dots)$  作为积分近似值的算法称为 Romberg 方法, 其停步准则为  $|T_m^{(0)} - T_{m-1}^{(0)}| < \varepsilon$ , 则  $T_m^{(0)}$  即为满足精度的近似值.

### 算法 7.3

(1) 输入  $a, b, f(x), \varepsilon$ .

(2) 置  $h = b - a, T_0^{(0)} = \frac{h}{2} [f(a) + f(b)], k = 1$ .

(3) 计算

$$T_0^{(k)} = \frac{1}{2} \left[ T_0^{(k-1)} + h \sum_{i=1}^{2^{k-1}} f\left(a + \left(i - \frac{1}{2}\right)h\right) \right]$$

对  $j = 1, \dots, k$

$$T_j^{(k-j)} = \frac{4^j T_{j-1}^{(k-j+1)} - T_{j-1}^{(k-j)}}{4^j - 1}$$

(4) 若  $|T_k^{(0)} - T_{k-1}^{(0)}| < \varepsilon$ , 输出  $T_k^{(0)} \approx \int_a^b f(x) dx$  停机; 否则,  $\frac{h}{2} \Rightarrow h, k+1 \Rightarrow k$ ,

返回 3.

**例 7** 用 Romberg 方法计算积分  $I = \int_0^1 \frac{\sin x}{x} dx$  的近似值, 要求误差不超过  $\frac{1}{2} \times 10^{-6}$ .

[解] 根据例 4 中的函数表按式 (7-23), 式 (7-33) 和式 (7-40) 计算, 得

$$T_0^{(0)} = \frac{1}{2} [f(0) + f(1)] = 0.920\,735\,5$$

$$T_0^{(1)} = \frac{1}{2} T_0^{(0)} + \frac{1}{2} f(0.5) = 0.939\,793\,2$$

$$T_1^{(0)} = \frac{4T_0^{(1)} - T_0^{(0)}}{3} = 0.946\,145\,7$$

$$T_0^{(2)} = \frac{1}{2} T_0^{(1)} + \frac{1}{4} [f(0.25) + f(0.75)] = 0.944\,513\,6$$

$$T_1^{(1)} = \frac{4T_0^{(2)} - T_0^{(1)}}{3} = 0.946\,087\,0$$

$$T_2^{(0)} = \frac{16T_1^{(1)} - T_1^{(0)}}{15} = 0.946\,083\,0$$

$$\begin{aligned} T_0^{(3)} &= \frac{1}{2} T_0^{(2)} + \frac{1}{8} [f(0.125) + f(0.375) + f(0.625) + f(0.875)] \\ &= 0.945\,691\,0 \end{aligned}$$

$$T_1^{(2)} = \frac{4T_0^{(3)} - T_0^{(2)}}{3} = 0.946\,083\,5$$

$$T_2^{(1)} = \frac{16T_1^{(2)} - T_1^{(1)}}{15} = 0.946\,083\,2$$



$$T_3^{(0)} = \frac{64T_2^{(1)} - T_2^{(0)}}{63} = 0.946\,083\,2$$

因为

$$|T_3^{(0)} - T_2^{(0)}| = 2 \times 10^{-7} < \frac{1}{2} \times 10^{-6}$$

所以

$$I = \int_0^1 \frac{\sin x}{x} dx \approx 0.946\,083\,2$$

事实上,由例7的计算过程可以看出,与准确值  $I = 0.946\,083\,1\cdots$  比较,  $T_2^{(0)}$  已达到精度要求,此时需要计算4个点的函数值. 要达到相同的精度,用复化 Simpson 公式需计算8个点的函数值,而用复化梯形公式则需计算167个点的函数值,所以 Romberg 算法优于复化梯形方法与复化 Simpson 方法.

对于一种确定的求积公式,我们可以增加节点个数,缩小步长  $h$ ,得到较精确的近似值;另一方面,在节点个数一定的前提下,不同求积公式的精确程度也是不同的. 理论分析与实际计算均表明,由 Romberg 公式得到的近似值的精确度高于复化求积公式得到的近似值,复化 Simpson 公式的精确度又高于复化梯形公式. 由此得到启发,在节点数固定的前提下,可望通过适当地选取节点  $\{x_k\}$  与求积系数  $\{A_k\}$ ,提高数值积分方法的精确度.

## § 7.5 高斯 (Gauss) 型求积公式

### 7.5.1 一般理论

#### (一) Gauss 型求积公式

Gauss 型求积公式的基本思想是:在节点数  $n$  固定时,适当地选取节点  $\{x_k\}$  与求积系数  $\{A_k\}$ ,使求积公式具有最高的代数精确度.

由定义 7.1,求积公式

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=1}^n A_k f(x_k) \quad (7-41)$$

其中  $\rho(x) \geq 0$  为权函数,具有  $m$  次代数精确度,充要条件是对  $f(x) = x^l$  ( $l = 0, 1, \cdots, m$ ) 式(7-41)精确成立,即

$$\int_a^b \rho(x) x^l dx = \sum_{k=1}^n A_k x_k^l \quad (l = 0, 1, \cdots, m) \quad (7-42)$$

令

$$\mu_l = \int_a^b \rho(x) x^l dx \quad (l = 0, 1, \cdots, m) \quad (7-43)$$

将式(7-43)代入式(7-42),得节点  $\{x_k\}$  与求积系数  $\{A_k\}$  应满足方程组

$$\begin{cases} A_1 + A_2 + \cdots + A_n = \mu_0 \\ A_1 x_1 + A_2 x_2 + \cdots + A_n x_n = \mu_1 \\ \dots\dots\dots \\ A_1 x_1^m + A_2 x_2^m + \cdots + A_n x_n^m = \mu_m \end{cases} \quad (7-44)$$

式(7-44)含有  $2n$  个未知数,  $m+1$  个方程,可以证明,当  $m+1 \leq 2n$  时,方程组(7-44)有解. 这表明  $n$  个节点的求积公式的代数精确度可达到  $2n-1$ . 但若取  $2n$  次多项式

$$p_{2n}(x) = (x - x_1)^2 (x - x_2)^2 \cdots (x - x_n)^2$$

显然,对任意  $n$  个互异节点  $x_1, x_2, \dots, x_n$ , 都有  $P_{2n}(x_k) = 0 (k = 1, 2, \dots, n)$ , 从而

有  $\sum_{k=1}^n A_k P_{2n}(x_k) = 0$ . 然而

$$\int_a^b \rho(x) p_{2n}(x) dx > 0$$

因此,  $n$  个节点的求积公式的最高代数精确度为  $2n-1$ .

**定义 7.2** 如果一组节点  $x_1, x_2, \dots, x_n \in [a, b]$  能使求积公式

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=1}^n A_k f(x_k) \quad (7-41)$$

具有  $2n-1$  次代数精确度,则称这组节点  $\{x_k\}$  为 Gauss 点,公式(7-41)称为带权函数  $\rho(x)$  的 Gauss 型求积公式.

**例 8** 试确定求积公式

$$\int_{-1}^1 f(x) dx \approx af(-\sqrt{0.6}) + bf(0) + cf(\sqrt{0.6})$$

中的待定参数  $a, b$  和  $c$ ,使其代数精确度尽量高,并指出公式具有几次代数精确度,它是否是 Gauss 型求积公式.

[解] 记

$$I(f) = \int_a^b f(x) dx, \quad \tilde{I}(f) = af(-\sqrt{0.6}) + bf(0) + cf(\sqrt{0.6})$$

因为

$$I(1) = \int_{-1}^1 1 dx = 2, \quad \tilde{I}(1) = a + b + c$$

$$I(x) = \int_{-1}^1 x dx = 0, \quad \tilde{I}(x) = -\sqrt{0.6}a + \sqrt{0.6}c$$

$$I(x^2) = \int_{-1}^1 x^2 dx = \frac{2}{3}, \quad \tilde{I}(x^2) = 0.6a + 0.6c$$

要使求积公式具有尽可能高的代数精确度,  $a, b, c$  必须满足方程组

$$\begin{cases} a + b + c = 2 \\ -\sqrt{0.6}a + \sqrt{0.6}c = 0 \\ 0.6a + 0.6c = \frac{2}{3} \end{cases}$$

求解得  $a = c = \frac{5}{9}$ ,  $b = \frac{8}{9}$ , 所以当  $a = c = \frac{5}{9}$ ,  $b = \frac{8}{9}$  时, 求积公式的代数精确度最高.

又因为

$$I(x^3) = \int_{-1}^1 x^3 dx = 0, \quad \tilde{I}(x^3) = \frac{5}{9} [(-\sqrt{0.6})^3 + (\sqrt{0.6})^3] = 0$$

$$I(x^4) = \int_{-1}^1 x^4 dx = \frac{2}{5}, \quad \tilde{I}(x^4) = \frac{5}{9} [(-\sqrt{0.6})^4 + (\sqrt{0.6})^4] = 0.4$$

$$I(x^5) = \int_{-1}^1 x^5 dx = 0, \quad \tilde{I}(x^5) = \frac{5}{9} [(-\sqrt{0.6})^5 + (\sqrt{0.6})^5] = 0$$

$$I(x^6) = \int_{-1}^1 x^6 dx = \frac{2}{7}, \quad \tilde{I}(x^6) = \frac{5}{9} [(-\sqrt{0.6})^6 + (\sqrt{0.6})^6] = \frac{2 \cdot 16}{9} \neq \frac{2}{7}$$

故此求积公式具有 5 次代数精确度, 它是 Gauss 型求积公式.

如何确定  $\{x_k\}$  和  $\{A_k\}$ , 使得式 (7-41) 具有  $2n-1$  次代数精确度? 最直接的方法是解方程组 (7-44). 但因 (7-44) 是非线性方程组, 求解十分困难. 一般利用正交多项式来求出 Gauss 点与求积系数.

## (二) Gauss 点与正交多项式的关系

先举例说明.

**例 9** 确定  $x_1, x_2, A_1, A_2$ , 使求积公式

$$\int_{-1}^1 f(x) dx \approx A_1 f(x_1) + A_2 f(x_2) \quad (7-45)$$

具有 3 次代数精确度.

[解] 取  $f(x)$  为任意三次多项式, 利用多项式除法,  $f(x)$  一定能表成如下形式

$$f(x) = (a_0 + a_1 x)(x - x_1)(x - x_2) + (b_0 + b_1 x)$$

于是有

$$\begin{aligned} \int_{-1}^1 f(x) dx &= \int_{-1}^1 (a_0 + a_1 x)(x - x_1)(x - x_2) dx + \int_{-1}^1 (b_0 + b_1 x) dx \\ &\approx A_1(b_0 + b_1 x_1) + A_2(b_0 + b_1 x_2) \end{aligned}$$

如果求积公式 (7-45) 有 3 次代数精确度, 则

$$\int_{-1}^1 (b_0 + b_1 x) dx = A_1(b_0 + b_1 x_1) + A_2(b_0 + b_1 x_2)$$

比较以上两式, 即得

$$\int_{-1}^1 (a_0 + a_1 x)(x - x_1)(x - x_2) dx = 0$$

对任意实数  $a_0, a_1$  都成立. 特别地, 取  $a_0 = 1, a_1 = 0$  和  $a_0 = 0, a_1 = 1$ , 得方程组

$$\begin{cases} \int_{-1}^1 (x - x_1)(x - x_2) dx = 0 \\ \int_{-1}^1 x(x - x_1)(x - x_2) dx = 0 \end{cases}$$

即

$$\begin{cases} \frac{2}{3} + 2x_1x_2 = 0 \\ -\frac{2}{3}(x_1 + x_2) = 0 \end{cases}$$

由此解出  $x_1 = -x_2 = \frac{1}{\sqrt{3}}$ , 这就是 Gauss 点. 再取两个特殊的  $f(x)$  (至多为三次多项式), 连同 Gauss 点代入式(7-45), 可求出求积系数. 比如取  $f_1(x) \equiv 1, f_2(x) = x$  代入式(7-45), 令其精确成立, 即得

$$\begin{cases} \int_{-1}^1 dx = 2 = A_1 + A_2 \\ \int_{-1}^1 x dx = 0 = A_1 \frac{1}{\sqrt{3}} - A_2 \frac{1}{\sqrt{3}} \end{cases}$$

其解为  $A_1 = A_2 = 1$ , 所以求积公式

$$\int_{-1}^1 f(x) dx \approx f\left(\frac{1}{\sqrt{3}}\right) + f\left(-\frac{1}{\sqrt{3}}\right)$$

具有 3 次代数精确度.

回顾以上求解过程, 我们首先用多项式除法将三次多项式表成  $q(x)(x - x_1)(x - x_2) + r(x)$  的形式, 其中  $q(x), r(x)$  均为一次多项式. 利用  $r(x_1) = f(x_1), r(x_2) = f(x_2)$  且求积公式对  $r(x)$  精确成立, 导出  $x_1, x_2$  应满足的条件

$$\int_{-1}^1 q(x)(x - x_1)(x - x_2) dx = 0$$

令  $q(x)$  为特殊多项式代入上式, 求出 Gauss 点  $x_1, x_2$ . 然后再取  $f(x)$  为特殊多项式, 如  $f(x) = 1, x$ , 连同 Gauss 点代入求积公式, 就可解出求积系数  $A_1, A_2$ . 上述求解过程可直接推广到一般情况.

用  $n$  次多项式  $\omega_n(x) = \prod_{k=1}^n (x - x_k)$  除任意的次数不超过  $2n - 1$  的多项式  $p_{2n-1}(x)$ , 得

$$p_{2n-1}(x) = q(x)\omega_n(x) + r(x) \quad (7-46)$$

其中  $q(x), r(x)$  均为至多  $n - 1$  次多项式. 将式(7-46)代入求积公式(7-41),

由求积公式对  $r(x)$  精确成立, 得

$$\int_a^b \rho(x) q(x) \omega_n(x) dx = 0 \quad (7-47)$$

对任意至多  $n-1$  次多项式  $q(x)$  均成立, 式(7-47)称为正交条件. 特别地, 取  $q(x) = 1, x, \dots, x^{n-1}$ , 分别代入式(7-47), 得  $n$  元方程组, 其解即为 Gauss 点  $\{x_k\}$ .

**定理 7.4** 求积公式(7-41)的节点是 Gauss 点的充要条件是函数

$$\omega_n(x) = \prod_{k=1}^n (x - x_k)$$

与任意次数不超过  $n-1$  的多项式  $p(x)$  在  $[a, b]$  上关于权函数  $\rho(x)$  正交, 即

$$\int_a^b \rho(x) p(x) \omega_n(x) dx = 0$$

[证明] 必要性. 若  $x_1, \dots, x_n$  是 Gauss 点, 则求积公式

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=1}^n A_k f(x_k)$$

具有  $2n-1$  次代数精确度. 设  $p(x)$  为至多  $n-1$  次多项式, 那么  $p(x)\omega_n(x)$  为至多  $2n-1$  次多项式, 于是

$$\int_a^b \rho(x) p(x) \omega_n(x) dx = \sum_{k=1}^n A_k p(x_k) \omega_n(x_k) = 0$$

即  $\omega_n(x)$  与任意次数不超过  $n-1$  的多项式  $p(x)$  在区间  $[a, b]$  上关于权函数  $\rho(x)$  正交.

充分性. 设  $f(x)$  为任意的次数不超过  $2n-1$  的多项式, 用  $\omega_n(x)$  除  $f(x)$  得

$$f(x) = q(x)\omega_n(x) + r(x)$$

其中  $q(x), r(x)$  均为至多  $n-1$  次多项式, 且  $r(x_k) = f(x_k) (k=1, 2, \dots, n)$ .

取  $f_l(x) = x^l (l=0, 1, \dots, n-1)$  代入求积公式(7-41), 并令其精确成立, 得  $n$  阶线性方程组

$$\begin{cases} A_1 + A_2 + \dots + A_n = \mu_0 \\ A_1 x_1 + A_2 x_2 + \dots + A_n x_n = \mu_1 \\ \dots\dots\dots \\ A_1 x_1^{n-1} + A_2 x_2^{n-1} + \dots + A_n x_n^{n-1} = \mu_{n-1} \end{cases} \quad (7-48)$$

其中  $\mu_l = \int_a^b \rho(x) x^l dx (l=0, 1, \dots, n-1)$ . 其系数矩阵为 Vandermonde 行列式, 故方程组(7-48)有唯一解. 由于求积公式(7-41)是线性的, 以式(7-48)的解为求积系数, 式(7-41)对任意至多  $n-1$  次多项式均精确成立. 特别地, 有

$$\int_a^b \rho(x) r(x) dx = \sum_{k=1}^n A_k r(x_k)$$

从而有

$$\begin{aligned}\int_a^b \rho(x) f(x) dx &= \int_a^b \rho(x) q(x) \omega_n(x) dx + \int_a^b \rho(x) r(x) dx \\ &= \int_a^b \rho(x) r(x) dx = \sum_{k=1}^n A_k r(x_k) = \sum_{k=1}^n A_k f(x_k)\end{aligned}$$

所以,求积公式(7-41)具有  $2n-1$  次代数精确度,  $\{x_k\}$  为 Gauss 点.

由定理 7.4 及定理 6.3 可知,区间  $[a, b]$  上关于权函数  $\rho(x)$  的正交多项式系中的  $n$  次多项式的根恰恰就是 Gauss 点. 因此,求 Gauss 点等价于求  $[a, b]$  上关于权函数  $\rho(x)$  的  $n$  次正交多项式的  $n$  个实根. 如果取

$$p_k(x) = \frac{\omega_n(x)}{(x - x_k) \omega'_n(x_k)} = \prod_{\substack{j=1 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} \quad (k = 1, 2, \dots, n)$$

则  $p_k(x)$  均为  $n-1$  次多项式, 且

$$p_k(x_j) = \begin{cases} 1, & j = k \\ 0, & j \neq k \end{cases}$$

将  $p_k(x)$  代入式(7-41)即得

$$\int_a^b \rho(x) \frac{\omega_n(x)}{(x - x_k) \omega'_n(x_k)} dx = \sum_{i=1}^n A_i p_k(x_i) = A_k \quad (7-49)$$

式(7-49)即为计算 Gauss 型求积公式的求积系数的公式.

### (三) Gauss 型求积公式的误差

Gauss 型求积公式的截断误差有以下估计.

**定理 7.5** 设  $f(x)$  在  $[a, b]$  上  $2n$  阶连续可微,  $\rho(x) \geq 0$ , 则带权函数  $\rho(x)$  的 Gauss 型求积公式的余项为

$$\begin{aligned}R(f) &= \int_a^b \rho(x) f(x) dx - \sum_{k=1}^n A_k f(x_k) \\ &= \frac{f^{(2n)}(\xi)}{(2n)!} \int_a^b \rho(x) \omega_n^2(x) dx, \quad \xi \in (a, b)\end{aligned} \quad (7-50)$$

[证明] 设在节点  $x_1, x_2, \dots, x_n$  上  $f(x)$  的  $2n-1$  次 Hermite 插值多项式为  $H(x)$ , 由 Hermite 余项公式

$$f(x) - H(x) = \frac{f^{(2n)}(\xi_x)}{(2n)!} \omega_n^2(x), \quad \xi_x \in (a, b)$$

有

$$\begin{aligned}R(f) &= \int_a^b \rho(x) f(x) dx - \sum_{k=1}^n A_k f(x_k) \\ &= \int_a^b \rho(x) f(x) dx - \sum_{k=1}^n A_k H(x_k)\end{aligned}$$

$$\begin{aligned}
 &= \int_a^b \rho(x) f(x) dx - \int_a^b \rho(x) H(x) dx \\
 &= \int_a^b \frac{f^{(2n)}(\xi_x)}{(2n)!} \rho(x) \omega_n^2(x) dx
 \end{aligned}$$

因为  $\rho(x) \omega_n^2(x) \geq 0$ , 由反常积分中值定理, 存在  $\xi \in (a, b)$  使得

$$R(f) = \frac{f^{(2n)}(\xi)}{(2n)!} \int_a^b \rho(x) \omega_n^2(x) dx$$

进一步论证可以导出, 如果  $f(x) \in C[a, b]$ , 当  $n \rightarrow \infty$  时, Gauss 型求积公式收敛于积分准确值, 即

$$\int_a^b \rho(x) f(x) dx = \lim_{n \rightarrow \infty} \sum_{k=1}^n A_k f(x_k)$$

证明过程可看参考书[9].

用 Gauss 型求积公式计算积分近似值时, 公式中的 Gauss 点与求积系数都是预先给出的, 一般精度较高. 舍入误差主要来源于函数值  $f(x_k)$  的计算. 如果记  $f(x_k)$  的近似值为  $f^*(x_k)$ , 则

$$\begin{aligned}
 \left| \sum_{k=1}^n A_k f(x_k) - \sum_{k=1}^n A_k f^*(x_k) \right| &= \left| \sum_{k=1}^n A_k [f(x_k) - f^*(x_k)] \right| \\
 &\leq \max_{1 \leq k \leq n} |f(x_k) - f^*(x_k)| \sum_{k=1}^n |A_k|
 \end{aligned} \tag{7-51}$$

因为求积公式对  $f(x) \equiv 1$  和  $f(x) = p_k^2(x)$  精确成立, 故有

$$\int_a^b \rho(x) dx = \sum_{k=1}^n A_k, \quad \int_a^b \rho(x) p_k^2(x) dx = \sum_{l=1}^n A_l p_k^2(x_l) = A_k > 0$$

代入式(7-51)得

$$\left| \sum_{k=1}^n A_k f(x_k) - \sum_{k=1}^n A_k f^*(x_k) \right| \leq \int_a^b \rho(x) dx \max_{1 \leq k \leq n} |f(x_k) - f^*(x_k)| \tag{7-52}$$

所以 Gauss 型求积公式是数值稳定的.

### 7.5.2 几种常用的 Gauss 型求积公式

对不同的权函数  $\rho(x)$ , 选取不同的正交多项式系, 可以导出不同的 Gauss 型求积公式.

#### (一) 高斯-勒让德(Gauss-Legendre)求积公式

区间  $[-1, 1]$  上的 Legendre 多项式是关于权函数  $\rho(x) \equiv 1$  的正交多项式系, 其表达式为

$$P_0(x) = 1$$

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2 - 1)^n] \quad (n = 1, 2, \dots) \quad (7-53)$$

由 5.1 中的讨论,  $n$  次 Legendre 多项式的零点  $x_k (k = 1, 2, \dots, n)$  是带权函数  $\rho(x) \equiv 1$  的 Gauss 型求积公式的 Gauss 点, 求积系数为

$$A_k = \frac{2}{(1 - x_k^2) [P'_n(x_k)]^2} \quad (k = 1, 2, \dots, n) \quad (7-54)$$

按上述方法构造的 Gauss 型求积公式称为 Gauss-Legendre 求积公式.

例如, 当  $n=2$  时,  $P_2(x) = \frac{1}{2}(3x^2 - 1)$ , 其零点为  $x = \pm \frac{1}{\sqrt{3}}$ . 由式(7-54), 有

$$A_k = \frac{2}{\left[1 - \left(\pm \frac{1}{\sqrt{3}}\right)^2\right] \left(\pm \frac{3}{\sqrt{3}}\right)^2} = \frac{2}{\left(1 - \frac{1}{3}\right) \times 3} = 1 \quad (k = 1, 2)$$

所以两个节点的 Gauss-Legendre 公式为

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

表 7-4 给出了部分 Gauss-Legendre 求积公式的节点与求积系数值, 以便查用.

表 7-4

| $n$ | $x_k$                | $A_k$        | $n$ | $x_k$                | $A_k$        |
|-----|----------------------|--------------|-----|----------------------|--------------|
| 1   | 0                    | 2            | 5   | $\pm 0.906\ 179\ 85$ | 0.236 926 89 |
| 2   | $\pm 0.577\ 350\ 27$ | 1            |     | $\pm 0.538\ 469\ 31$ | 0.478 628 67 |
| 3   | $\pm 0.774\ 596\ 67$ | 0.555 555 56 |     | 0                    | 0.568 888 89 |
|     | 0                    | 0.888 888 89 | 6   | $\pm 0.932\ 469\ 51$ | 0.171 324 49 |
| 4   | $\pm 0.861\ 136\ 31$ | 0.347 854 85 |     | $\pm 0.661\ 209\ 39$ | 0.360 761 57 |
|     | $\pm 0.339\ 981\ 04$ | 0.652 145 15 |     | $\pm 0.238\ 619\ 19$ | 0.467 913 93 |

由式(7-50)及  $n$  次 Legendre 多项式  $P_n(x)$  与  $\omega_n(x)$  的关系, 容易得出 Gauss-Legendre 求积公式的截断误差为

$$\begin{aligned} R(f) &= \frac{f^{(2n)}(\xi)}{(2n)!} \int_{-1}^1 \omega_n^2(x) dx = \frac{f^{(2n)}(\xi)}{(2n)!} \frac{2^{2n} (n!)^4}{[(2n)!]^2} \int_{-1}^1 P_n^2(x) dx \\ &= \frac{2^{2n+1} (n!)^4}{(2n+1) [(2n)!]} f^{(2n)}(\xi), \quad \xi \in (-1, 1) \end{aligned} \quad (7-55)$$

计算函数  $f(x)$  在任意有限区间  $[a, b]$  上的积分值的 Gauss-Legendre 公式可



通过变量变换得到. 令  $x = \frac{a+b}{2} + \frac{b-a}{2}t$ , 则

$$\begin{aligned}\int_a^b f(x) dx &= \frac{b-a}{2} \int_{-1}^1 f\left(\frac{a+b}{2} + \frac{b-a}{2}t\right) dt \\ &= \frac{b-a}{2} \int_{-1}^1 F(t) dt \approx \frac{b-a}{2} \sum_{k=1}^n A_k F(t_k)\end{aligned}\quad (7-56)$$

其中  $\{t_k\}$  为  $n$  次 Legendre 多项式  $P_n(t)$  的零点,  $A_k$  按式 (7-54) 计算.

**例 10** 求积分  $I = \int_0^{\frac{\pi}{2}} \sin t dt$  的近似值.

[解] 作变换  $t = \frac{\pi}{4} + \frac{\pi}{4}x$ , 则

$$I = \int_0^{\frac{\pi}{2}} \sin t dt = \frac{\pi}{4} \int_{-1}^1 \sin \frac{\pi(x+1)}{4} dx$$

若用  $n=2$  的 Gauss-Legendre 求积公式计算, 由表 7-4,  $x_1 = -0.577\ 350\ 27$ ,  $x_2 = 0.577\ 350\ 27$ ,  $A_{1,2} = 1$ , 则

$$I \approx \frac{\pi}{4} \sin \left[ \frac{\pi}{4} (1 - 0.577\ 350\ 3) \right] + \frac{\pi}{4} \sin \left( \frac{\pi}{4} \times 1.577\ 350\ 3 \right) = 0.998\ 472\ 5$$

其误差为

$$\begin{aligned}|R(f)| &= \left| \frac{2^5 \times (2!)^4}{5 \times (4!)^3} f^{(4)}(\xi) \right| \\ &\leq \frac{2^9}{5 \times 24^3} \times \left( \frac{\pi}{4} \right)^5 = \frac{\pi^5}{10 \times 24^3} \approx 0.002\ 2\end{aligned}$$

若用  $n=3$  的 Gauss-Legendre 求积公式, 查表 7-4 得

$$\begin{aligned}I &\approx 0.555\ 555\ 6 f(-0.774\ 596\ 7) + 0.888\ 888\ 9 f(0) + \\ &\quad 0.555\ 555\ 6 f(0.774\ 596\ 7) \\ &= 0.555\ 555\ 6 \left[ \frac{\pi}{4} \sin \left[ \frac{\pi}{4} \times (1 - 0.774\ 596\ 7) \right] + \right. \\ &\quad \left. \frac{\pi}{4} \sin \left( \frac{\pi}{4} \times 1.774\ 596\ 7 \right) \right] + 0.888\ 888\ 9 \times \frac{\pi}{4} \times \sin \frac{\pi}{4} \\ &= 0.506\ 354\ 4 + 0.493\ 653\ 6 \\ &= 1.000\ 008\ 1\end{aligned}$$

其误差为

$$|R(f)| = \left| \frac{2^7 \times (3!)^4}{7 \times (6!)^3} f^{(6)}(\xi) \right| \leq \frac{2^7 \times 6^4}{7 \times 720^3} \times \left( \frac{\pi}{4} \right)^7 \approx 1.17 \times 10^{-5}$$

如果用三点的 Simpson 公式求积分近似值, 则有

$$I = \int_0^{\frac{\pi}{2}} \sin t dt \approx \frac{\pi}{12} \left( \sin 0 + 4 \sin \frac{\pi}{4} + \sin \frac{\pi}{2} \right)$$

$$= \frac{\pi}{12}(2\sqrt{2} + 1) \approx 1.002\,279\,9$$

$I$  的精确值为 1. 由例 9 结果可见, 当被积函数充分光滑, 在节点数相同时, Gauss 型求积公式比 N-C 公式的精度高.

## (二) 高斯 - 切比雪夫 (Gauss-Chebyshev) 求积公式

Chebyshev 多项式是  $[-1, 1]$  上关于权函数  $\rho(x) = \frac{1}{\sqrt{1-x^2}}$  的正交多项式系, 其表达式为

$$T_n(x) = \cos(n \arccos x) \quad (n = 0, 1, 2, \dots)$$

$n$  次 Chebyshev 多项式  $T_n(x)$  的零点为  $x_k = \cos \frac{2k-1}{2n}\pi$  ( $k = 1, 2, \dots, n$ ). 以  $x_k$  为节点, 求积系数为  $A_k = \frac{\pi}{n}$  ( $k = 1, 2, \dots, n$ ) 的带有权函数  $\rho(x) = \frac{1}{\sqrt{1-x^2}}$  的 Gauss 型求积公式

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{n} \sum_{k=1}^n f\left(\cos\left(\frac{2k-1}{2n}\pi\right)\right) \quad (7-57)$$

称为 Gauss-Chebyshev 求积公式. 其余项为

$$R(f) = \frac{\pi}{2^{2n-1}(2n)!} f^{(2n)}(\xi), \quad \xi \in (-1, 1) \quad (7-58)$$

## (三) 高斯 - 拉盖尔 (Gauss-Laguerre) 求积公式

Laguerre 多项式

$$L_n(x) = e^x \frac{d^n}{dx^n} (e^{-x} x^n) \quad (n = 0, 1, 2, \dots)$$

是区间  $[0, +\infty)$  上关于权函数  $\rho(x) = e^{-x}$  的正交多项式系, 以  $n$  次 Laguerre 多项式  $L_n(x)$  的零点  $\{x_k\}$  为 Gauss 点导出的 Gauss 型求积公式

$$\int_0^{+\infty} e^{-x} f(x) dx \approx \sum_{k=1}^n A_k f(x_k) \quad (7-59)$$

称为 Gauss-Laguerre 求积公式. 求积系数的计算公式为

$$A_k = \frac{(n!)^2}{x_k [L'_n(x_k)]^2} \quad (k = 1, 2, \dots, n) \quad (7-60)$$

求积公式 (7-59) 的截断误差为

$$R(f) = \frac{(n!)^2}{(2n)!} f^{(2n)}(\xi), \quad \xi \in (0, +\infty) \quad (7-61)$$

表 7-5 列出了部分 Gauss-Laguerre 求积公式的节点和求积系数值, 以供查用.

表 7-5

| $n$ | $x_k$        | $A_k$        | $A_k e^{x_k^2}$ |
|-----|--------------|--------------|-----------------|
| 2   | 0.585 786 44 | 0.853 553 39 | 1.533 326 03    |
|     | 3.414 213 56 | 0.146 446 61 | 4.450 957 34    |
| 3   | 0.415 774 56 | 0.711 093 01 | 1.077 692 86    |
|     | 2.294 280 36 | 0.278 517 73 | 2.762 142 96    |
|     | 6.289 945 08 | 0.010 389 26 | 5.601 094 63    |
| 4   | 0.322 547 69 | 0.603 154 10 | 0.832 739 12    |
|     | 1.745 761 10 | 0.357 418 69 | 2.048 102 44    |
|     | 4.536 620 30 | 0.038 887 91 | 3.631 146 31    |
|     | 9.395 070 91 | 0.000 539 29 | 6.487 145 08    |

利用 Gauss-Laguerre 求积公式也可以求积分  $\int_0^{+\infty} g(x) dx$  的近似值. 令  $f(x) = e^{-x} g(x)$ , 则

$$\int_0^{+\infty} g(x) dx = \int_0^{+\infty} e^{-x} f(x) dx \approx \sum_{k=1}^n A_k f(x_k) = \sum_{k=1}^n A_k e^{x_k} g(x_k) \quad (7-62)$$

而对积分  $\int_a^{+\infty} \varphi(t) dt$ , 只要作变换  $x = t - a$ , 就有

$$\int_a^{+\infty} \varphi(t) dt = \int_0^{+\infty} \varphi(x + a) dx = \int_0^{+\infty} g(x) dx$$

按式(7-62)即可计算其近似值.

#### (四) 高斯-埃尔米特 (Gauss-Hermite) 求积公式

求积公式

$$\int_{-\infty}^{+\infty} e^{-x^2} f(x) dx \approx \sum_{k=1}^n A_k f(x_k) \quad (7-63)$$

称为 Gauss-Hermite 求积公式, 其中节点  $x_k (k = 1, 2, \dots, n)$  为  $n$  次 Hermite 多项式

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} (e^{-x^2}) \quad (n = 0, 1, 2, \dots) \quad (7-64)$$

的  $n$  个零点, 求积系数为

$$A_k = \frac{2^{n+1} n! \sqrt{\pi}}{[H'_n(x_k)]^2} \quad (k = 1, 2, \dots, n) \quad (7-65)$$

因为 Hermite 多项式是  $(-\infty, +\infty)$  上以  $\rho(x) = e^{-x^2}$  为权函数的正交多项式系, 故公式(7-63)为 Gauss 型求积公式. 其截断误差为

$$R(f) = \frac{n! \sqrt{\pi}}{2^n (2n)!} f^{(2n)}(\xi), \quad \xi \in (-\infty, +\infty) \quad (7-66)$$

部分 Gauss-Hermite 求积公式的节点与求积系数见表 7-6.

表 7-6

| $n$ | $x_k$                | $A_k$        | $A_k e^{x_k^2}$ |
|-----|----------------------|--------------|-----------------|
| 2   | $\pm 0.707\ 106\ 78$ | 0.886 226 93 | 1.461 141 18    |
| 3   | 0                    | 1.181 635 90 | 1.181 635 90    |
|     | $\pm 1.224\ 744\ 87$ | 0.295 408 98 | 1.323 931 18    |
| 4   | $\pm 0.524\ 647\ 62$ | 0.804 914 09 | 1.059 964 48    |
|     | $\pm 1.650\ 680\ 12$ | 0.081 312 84 | 1.240 225 82    |
| 5   | 0                    | 0.945 308 72 | 0.945 308 72    |
|     | $\pm 0.958\ 572\ 46$ | 0.393 619 32 | 0.986 581 00    |
|     | $\pm 2.020\ 182\ 87$ | 0.019 953 24 | 1.181 488 63    |

对积分  $\int_{-\infty}^{+\infty} g(x) dx$ , 令  $f(x) = e^{x^2} g(x)$ , 则

$$\int_{-\infty}^{+\infty} g(x) dx = \int_{-\infty}^{+\infty} e^{-x^2} f(x) dx$$

故只要对  $e^{x^2} g(x)$  用 Gauss-Hermite 求积公式, 即可计算  $\int_{-\infty}^{+\infty} g(x) dx$  的近似值.

例 11 利用  $n=3$  的 Gauss-Hermite 求积公式计算积分

$$I = \int_0^{+\infty} e^{-x^2} \sqrt{1+x^2} dx$$

[解] 因积分  $\int_0^{+\infty} e^{-x^2} \sqrt{1+x^2} dx$  收敛,  $e^{-x^2} \sqrt{1+x^2}$  是偶函数, 故积分  $\int_{-\infty}^{+\infty} e^{-x^2} \sqrt{1+x^2} dx$  收敛, 且

$$\int_0^{+\infty} e^{-x^2} \sqrt{1+x^2} dx = \frac{1}{2} \int_{-\infty}^{+\infty} e^{-x^2} \sqrt{1+x^2} dx$$

查表 7-6 得

$$\int_{-\infty}^{+\infty} e^{-x^2} \sqrt{1+x^2} dx \approx 1.181\ 635\ 9 \times \sqrt{1+0} + 0.295\ 409\ 0 \times$$

$$\begin{aligned} & [\sqrt{1+(1.224\ 744\ 9)^2} + \sqrt{1+(-1.224\ 744\ 9)^2}] \\ &= 1.181\ 635\ 9 + 0.295\ 409\ 0 \times 3.162\ 277\ 7 \\ &= 2.115\ 801\ 2 \end{aligned}$$

所以

$$I = \int_0^{+\infty} e^{-x^2} \sqrt{1+x^2} dx \approx 1.057\,900\,6$$

综上所述,用 Gauss 型求积公式计算积分近似值的步骤为:

- (1) 把所求积分化成标准形式,即通过变换将积分区间化成  $[-1, 1]$ ,  $[0, +\infty)$  或  $(-\infty, +\infty)$ , 根据被积函数形式, 确定用哪种求积公式;
- (2) 选定  $n$ , 查表得节点  $\{x_k\}$  及求积系数  $\{A_k\}$ ;
- (3) 按求积公式计算积分近似值.

Gauss 型求积公式不仅精度高,而且可以计算反常积分,瑕积分亦可化成无穷积分求近似值. 这是前几节所介绍的 N-C 公式, Romberg 方法等无法解决的. 但用 Gauss 型求积公式很难根据事先给定的精度及误差公式确定节点数  $n$ , 并且当节点数  $n$  增加时, 节点  $\{x_k\}$  与求积系数  $\{A_k\}$  都会改变, 需重新查表按新公式计算, 前面计算过的结果不能继续使用, 因而增加了计算量.

## § 7.6 振荡函数的积分

在工程实际中,常常需要计算振荡函数的积分. 如果被积函数是一个变化激烈的振荡函数, 如

$$\int_a^b f(x) \cos \omega x dx, \int_a^b f(x) \sin \omega x dx \quad (7-67)$$

其中  $f(x)$  为非振荡函数,  $\omega$  为较大的正数. 利用前面给出的求积公式虽然也能计算这两个积分的近似值, 但为得到比较准确的近似值, 必须在每半个周期内都要取到几个节点. 否则, 计算效果可能很不理想. 例如, 用复化梯形公式和复化 Simpson 公式分别计算积分

$$I = \int_0^{\frac{5}{8}\pi} \sin 20x dx$$

的近似值. 如果取  $n = 10, h = \frac{\pi}{16}$ , 用复化梯形公式和复化 Simpson 公式分别求得近似值为

$$T_{10} = -0.040\,7, S_{10} = -0.119\,7$$

这与积分的准确值

$$I = \int_0^{\frac{5}{8}\pi} \sin 20x dx = \frac{1}{20} = 0.05$$

相差很大. 事实上, 如果用复化梯形公式计算此积分的近似值, 要求其误差不超过 0.001, 由误差公式

$$|R_n| = \frac{1}{12} \times \frac{5}{8} \pi h^2 \times 20^2 \sin \xi$$

$$\leq \frac{125\pi}{6}h^2 \leq 0.001$$

可以估计出  $h \leq 0.003\,908\,82$ , 即至少应取  $n = 503$  个节点. 这样, 节点数比较多, 求积的工作量十分可观.

仔细分析误差公式可以发现, 用复化求积公式计算振荡积分, 之所以要取较多的节点, 原因在于被积函数求导时将出现  $\omega^2$  或  $\omega^4$ , 当  $\omega$  较大时, 误差就大. 解决此问题的一个方法是用  $f(x)$  的插值函数  $\varphi(x)$  代入式 (7-68), 以积分

$$\int_a^b \varphi(x) \cos \omega x dx, \int_a^b \varphi(x) \sin \omega x dx \quad (7-68)$$

作为积分式 (7-67) 的近似. 特别地, 若用分段低次插值多项式  $\varphi(x)$  近似  $f(x)$ , 积分式 (7-68) 可以精确地求出.

例如, 将区间  $[a, b]$  分为  $n$  等份,  $h = \frac{b-a}{n}$ ,  $x_i = a + ih$  ( $i = 0, 1, \dots, n$ ). 取  $\varphi(x)$  为  $f(x)$  的分段线性插值函数, 则有

$$\begin{aligned} & \int_a^b \varphi(x) \cos \omega x dx \\ &= \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} \varphi(x) \cos \omega x dx \\ &= \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} \left\{ f(x_i) + \frac{1}{h} [f(x_{i+1}) - f(x_i)] (x - x_i) \right\} \cos \omega x dx \\ &= \sum_{i=0}^{n-1} \left\{ \frac{1}{\omega} \left[ f(x_i) - \frac{x_i}{h} (f(x_{i+1}) - f(x_i)) \right] (\sin \omega x_{i+1} - \sin \omega x_i) + \right. \\ & \quad \left. \frac{1}{\omega h} [f(x_{i+1}) - f(x_i)] \left( x \sin \omega x + \frac{1}{\omega} \cos \omega x \right) \right|_{x_i}^{x_{i+1}} \Big\} \\ &= \sum_{i=0}^{n-1} \left\{ \frac{1}{\omega} [f(x_{i+1}) \sin \omega x_{i+1} - f(x_i) \sin \omega x_i] - \right. \\ & \quad \frac{\sin \omega h}{\omega^2 h} [f(x_{i+1}) \sin \omega x_{i+1} - f(x_i) \sin \omega x_i] + \\ & \quad \left. \frac{2 \sin^2 \frac{\omega h}{2}}{\omega^2 h} [f(x_{i+1}) \cos \omega x_{i+1} + f(x_i) \cos \omega x_i] \right\} \\ &= \frac{1}{\omega} [f(x_n) \sin \omega x_n - f(x_0) \sin \omega x_0] - \frac{\sin \omega h}{\omega^2 h} [f(x_n) \sin \omega x_n - f(x_0) \sin \omega x_0] + \\ & \quad \frac{2}{\omega^2 h} \sin^2 \frac{\omega h}{2} [f(x_n) \cos \omega x_n + f(x_0) \cos \omega x_0] + \frac{4}{\omega^2 h} \sin^2 \frac{\omega h}{2} \sum_{i=1}^{n-1} f(x_i) \cos \omega x_i \end{aligned}$$

整理上式并令

$$\alpha = \frac{4}{\omega^2 h} \sin^2 \frac{\omega h}{2}, \quad \beta = \frac{2}{\omega^2 h} \sin \frac{\omega h}{2} \quad (7-69)$$

得

$$\begin{aligned} \int_a^b \varphi(x) \cos \omega x dx &= f(x_0) \left[ -\frac{1}{\omega} \sin \omega x_0 + \beta \sin \omega \left( x_0 + \frac{h}{2} \right) \right] + \\ &\quad f(x_n) \left[ \frac{1}{\omega} \sin \omega x_n - \beta \sin \omega \left( x_n - \frac{h}{2} \right) \right] + \\ &\quad \alpha \sum_{i=1}^{n-1} f(x_i) \cos \omega x_i \end{aligned} \quad (7-70)$$

于是得到求积公式

$$\begin{aligned} &\int_a^b f(x) \cos \omega x dx \\ &\approx \int_a^b \varphi(x) \cos \omega x dx \\ &= f(x_0) \left[ -\frac{1}{\omega} \sin \omega x_0 + \beta \sin \omega \left( x_0 + \frac{h}{2} \right) \right] + \\ &\quad f(x_n) \left[ \frac{1}{\omega} \sin \omega x_n - \beta \sin \omega \left( x_n - \frac{h}{2} \right) \right] + \alpha \sum_{i=1}^{n-1} f(x_i) \cos \omega x_i \end{aligned} \quad (7-71)$$

完全类似地有

$$\begin{aligned} \int_a^b f(x) \sin \omega x dx &\approx f(x_0) \left[ \frac{1}{\omega} \cos \omega x_0 - \beta \cos \omega \left( x_0 + \frac{h}{2} \right) \right] + \\ &\quad f(x_n) \left[ -\frac{1}{\omega} \cos \omega x_n + \beta \cos \omega \left( x_n - \frac{h}{2} \right) \right] + \alpha \sum_{i=1}^{n-1} f(x_i) \sin \omega x_i \end{aligned} \quad (7-72)$$

利用求积公式(7-72)计算积分

$$I = \int_0^{\frac{5}{8}\pi} \sin 20x dx$$

取  $n=10, h=\frac{\pi}{16}$ , 则有

$$\alpha = \frac{64}{400\pi} \sin^2 \frac{5\pi}{8} \approx 0.043\,471\,1$$

$$\beta = \frac{32}{400\pi} \sin \frac{5\pi}{8} \approx 0.023\,526\,4$$

$$I \approx \frac{1}{20} \cos 0 - \beta \cos \left( 20 \times \frac{\pi}{32} \right) - \frac{1}{20} \cos \left( 20 \times \frac{5}{8} \pi \right) + \beta \cos \left( 20 \times \frac{19}{32} \pi \right) +$$

$$\begin{aligned}
& \alpha \sum_{i=1}^9 \sin \left( 20 \times \frac{i\pi}{16} \right) \\
&= 0.05 - 0.023\,526\,4 \times (-0.382\,683) + 0.023\,526\,4 \times 0.923\,880 + \\
&\quad 0.043\,471\,1 \times (-0.707\,107) = 0.049\,999\,99
\end{aligned}$$

计算结果与精确值十分接近.

由分段线性插值的误差公式容易得到求积公式(7-71)和式(7-72)的误差为

$$\begin{aligned}
& \left| \int_a^b f(x) \cos \omega x dx - \int_a^b \varphi(x) \cos \omega x dx \right| \\
& \leq (b-a) \max_{a \leq x \leq b} |f(x) - \varphi(x)| \quad (7-73) \\
& \leq \frac{b-a}{8} h^2 \max_{a \leq x \leq b} |f''(x)|
\end{aligned}$$

$$\left| \int_a^b f(x) \sin \omega x dx - \int_a^b \varphi(x) \sin \omega x dx \right| \leq \frac{b-a}{8} h^2 \max_{a \leq x \leq b} |f''(x)| \quad (7-74)$$

需要指出的是:当  $\omega h = 2k\pi$  或  $\omega h = k\pi$ ,  $\omega x_0 = \left(j + \frac{1}{2}\right)\pi$  ( $k, j$  为正整数) 时, 求积公式(7-71)和式(7-72)的误差较大. 改进方法可参看[10].

## § 7.7 重积分的数值计算

计算重积分的一个最直接的思路是像计算定积分那样, 用多维插值多项式近似被积函数, 从而求得重积分的近似值. 但是由于重积分积分域和多维插值多项式的复杂性, 得到的近似求积公式将会繁杂得多, 甚至很难得到近似求积公式.

为简便起见, 我们考虑矩形区域

$$D = \{(x, y) \mid a \leq x \leq b, c \leq y \leq d\} \quad (\text{这里 } a, b, c, d \text{ 是常数})$$

上的二重积分

$$I = \iint_D f(x, y) dx dy \quad (7-75)$$

其中  $f(x, y)$  为  $D$  上的连续函数.

### 7.7.1 插值型求积公式

设矩形区域  $D = \{(x, y) \mid a \leq x \leq b, c \leq y \leq d\}$  的剖分为

$$\Delta_x: a = x_0 < x_1 < \cdots < x_n = b$$

$$\Delta_y: c = y_0 < y_1 < \cdots < y_m = d$$



由第五章公式(5-78)得插值多项式为

$$P_{n,m}(x,y) = \sum_{i=0}^n \sum_{j=0}^m f(x_i, y_j) l_i(x) \tilde{l}_j(y)$$

从而有

$$\begin{aligned} I &= \iint_D f(x,y) dx dy = \int_a^b dx \int_c^d f(x,y) dy \\ &\approx \int_a^b dx \int_c^d \left[ \sum_{i=0}^n \sum_{j=0}^m f(x_i, y_j) l_i(x) \tilde{l}_j(y) \right] dy \\ &= \sum_{i=0}^n \sum_{j=0}^m f(x_i, y_j) \int_a^b l_i(x) dx \int_c^d \tilde{l}_j(y) dy = \sum_{i=0}^n \sum_{j=0}^m A_{ij} f(x_i, y_j) \end{aligned} \quad (7-76)$$

公式(7-76)就是计算二重积分(7-75)的 Newton-Cotes 型求积公式,  $A_{ij} = \int_a^b l_i(x) dx \int_c^d \tilde{l}_j(y) dy$  为求积系数.

当  $n=m=1$  时, 则  $x_0=a, x_1=b, y_0=c, y_1=d, l_i(x), \tilde{l}_j(y)$  皆为一次插值基函数, 计算可得  $A_{ij} = \frac{1}{4}(b-a)(d-c)$ , 求积公式(7-76)为

$$\begin{aligned} I &= \int_a^b dx \int_c^d f(x,y) dy \\ &\approx \frac{1}{4}(b-a)(d-c)[f(a,c) + f(a,d) + f(b,c) + f(b,d)] \end{aligned}$$

称

$$T(f) = \frac{1}{4}(b-a)(d-c)[f(a,c) + f(a,d) + f(b,c) + f(b,d)] \quad (7-77)$$

为计算二重积分(7-75)的梯形公式.

当  $n=m=2$  时, 区间  $[a,b]$  和  $[c,d]$  均为二等分, 类似可得计算二重积分(7-75)的 Simpson 公式

$$\begin{aligned} S(f) &= \frac{(b-a)(d-c)}{36} \left[ f(a,c) + 4f\left(a, \frac{c+d}{2}\right) + f(a,d) + 4f\left(\frac{a+b}{2}, c\right) + \right. \\ &\quad \left. 16f\left(\frac{a+b}{2}, \frac{c+d}{2}\right) + 4f\left(\frac{a+b}{2}, d\right) + \right. \\ &\quad \left. f(b,c) + 4f\left(b, \frac{c+d}{2}\right) + f(b,d) \right] \end{aligned}$$

(7-78)

由于公式(7-77)和(7-78)实际上是关于  $x$  方向和  $y$  方向同时使用了定积分的梯形公式或 Simpson 公式,那么由定积分梯形公式和 Simpson 公式的截断误差(7-25)和(7-26),可以得到式(7-77)和(7-78)的截断误差分别为

$$\iint_D f(x, y) dx dy - T(f) = -\frac{(b-a)(d-c)}{12} \left[ (b-a)^2 \frac{\partial^2 f(\xi_1, \eta_1)}{\partial x^2} + (d-c)^2 \frac{\partial^2 f(\xi_2, \eta_2)}{\partial y^2} \right] \quad (7-79)$$

$$\iint_D f(x, y) dx dy - S(f) = -\frac{(b-a)(d-c)}{180} \left[ \frac{(b-a)^4}{16} \frac{\partial^4 f(\xi_1, \eta_1)}{\partial x^4} + \frac{(d-c)^4}{16} \frac{\partial^4 f(\xi_2, \eta_2)}{\partial y^4} \right] \quad (7-80)$$

其中  $(\xi_1, \eta_1), (\xi_2, \eta_2) \in D$ .

由误差估计式(7-79)和(7-80)知:求积公式(7-77)和(7-78)的截断误差与使用公式(7-77)和(7-78)的矩形的长和宽有关.长和宽越小,截断误差也越小.因此,为了提高精确度,考虑二重积分的复化求积公式.

### 7.7.2 复化求积公式

设矩形区域  $D = \{(x, y) \mid a \leq x \leq b, c \leq y \leq d\}$  关于  $x$  方向被  $n$  等分,关于  $y$  方向被  $m$  等分,即  $x, y$  方向的步长分别为  $h = \frac{b-a}{n}, k = \frac{d-c}{m}$ ,节点

$$x_i = a + ih, \quad i = 0, 1, \dots, n$$

$$y_j = c + jk, \quad j = 0, 1, \dots, m$$

那么积分

$$I = \iint_D f(x, y) dx dy = \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} \int_{x_i}^{x_{i+1}} dx \int_{y_j}^{y_{j+1}} f(x, y) dy$$

使用梯形公式(7-77)得

$$\begin{aligned} T_{n,m}(f) &= \frac{hk}{4} \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} [f(x_i, y_j) + f(x_i, y_{j+1}) + f(x_{i+1}, y_j) + f(x_{i+1}, y_{j+1})] \\ &= \frac{hk}{4} \sum_{i=0}^n \sum_{j=0}^m t_{ij} f(x_i, y_j) \end{aligned} \quad (7-81)$$

式(7-81)称为二重积分(7-75)的复化梯形公式,其中系数  $t_{ij}$  由表 7-7 给出.

表 7-7

| $j \backslash i$ | 0        | 1        | 2        | ... | $m-2$    | $m-1$    | $m$      |
|------------------|----------|----------|----------|-----|----------|----------|----------|
| 0                | 1        | 2        | 2        | ... | 2        | 2        | 1        |
| 1                | 2        | 4        | 4        | ... | 4        | 4        | 2        |
| 2                | 2        | 4        | 4        | ... | 4        | 4        | 2        |
| $\vdots$         | $\vdots$ | $\vdots$ | $\vdots$ |     | $\vdots$ | $\vdots$ | $\vdots$ |
| $n-2$            | 2        | 4        | 4        | ... | 4        | 4        | 2        |
| $n-1$            | 2        | 4        | 4        | ... | 4        | 4        | 2        |
| $n$              | 1        | 2        | 2        | ... | 2        | 2        | 1        |

若矩形的等分数  $n$  和  $m$  皆为偶数, 类似可得计算二重积分(7-75)的复化 Simpson 公式

$$\begin{aligned}
 S_{n,m}(f) &= \frac{hk}{9} \sum_{i=0}^{n/2-1} \sum_{j=0}^{m/2-1} [f(x_{2i}, y_{2j}) + 4f(x_{2i}, y_{2j+1}) + f(x_{2i}, y_{2j+2}) + \\
 &\quad 4f(x_{2i+1}, y_{2j}) + 16f(x_{2i+1}, y_{2j+1}) + 4f(x_{2i+1}, y_{2j+2}) + f(x_{2i+2}, y_{2j}) + \\
 &\quad 4f(x_{2i+2}, y_{2j+1}) + f(x_{2i+2}, y_{2j+2})] \\
 &= \frac{hk}{9} \sum_{i=0}^n \sum_{j=0}^m s_{ij} f(x_i, y_j) \quad (7-82)
 \end{aligned}$$

其中系数  $s_{ij}$  由表 7-8 给出.

表 7-8

| $j \backslash i$ | 0        | 1        | 2        | 3        | 4        | ... | $m-3$    | $m-2$    | $m-1$    | $m$      |
|------------------|----------|----------|----------|----------|----------|-----|----------|----------|----------|----------|
| 0                | 1        | 4        | 2        | 4        | 2        | ... | 4        | 2        | 4        | 1        |
| 1                | 4        | 16       | 8        | 16       | 8        | ... | 16       | 8        | 16       | 4        |
| 2                | 2        | 8        | 4        | 8        | 4        | ... | 8        | 4        | 8        | 2        |
| 3                | 4        | 16       | 8        | 16       | 8        | ... | 16       | 8        | 16       | 4        |
| 4                | 2        | 8        | 4        | 8        | 4        | ... | 8        | 4        | 8        | 2        |
| $\vdots$         | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |     | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $n-3$            | 4        | 16       | 8        | 16       | 8        | ... | 16       | 8        | 16       | 4        |

续表

| $i \backslash j$ | 0 | 1  | 2 | 3  | 4 | ... | $m-3$ | $m-2$ | $m-1$ | $m$ |
|------------------|---|----|---|----|---|-----|-------|-------|-------|-----|
| $n-2$            | 2 | 8  | 4 | 8  | 4 | ... | 8     | 4     | 8     | 2   |
| $n-1$            | 4 | 16 | 8 | 16 | 8 | ... | 16    | 8     | 16    | 4   |
| $n$              | 1 | 4  | 2 | 4  | 2 | ... | 4     | 2     | 4     | 1   |

例 12 取  $n = m = 4$ , 用复化 Simpson 公式计算二重积分  $I = \int_0^{0.2} dx \int_0^{0.5} e^{x-y} dy$ .

[解] 将积分域  $x, y$  方向均 4 等分, 则  $h = 0.05$ ,  $k = 0.125$ , 节点  $x_i = 0.05i, y_j = 0.125j (i, j = 0, 1, \dots, 4)$ . 各网格点上的系数  $s_{ij}$  如图 7-4 所示, 那么

$$\begin{aligned}
 I &= \int_0^{0.2} dx \int_0^{0.5} e^{x-y} dy \\
 &\approx S_{4,4} = \frac{0.125 \times 0.05}{9} \sum_{i=0}^4 \sum_{j=0}^4 s_{ij} e^{x_i - y_j} \\
 &= \frac{0.00625}{9} \sum_{i=0}^4 \sum_{j=0}^4 s_{ij} e^{0.05i - 0.125j} \\
 &\approx 0.0871152777778
 \end{aligned}$$

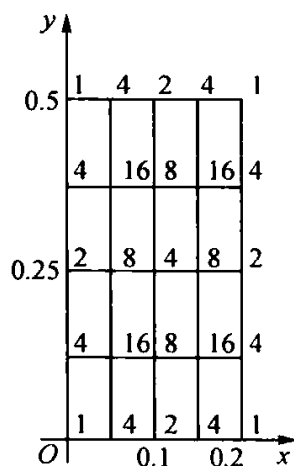


图 7-4

事实上准确值

$$\begin{aligned}
 I &= \int_0^{0.2} dx \int_0^{0.5} e^{x-y} dy = (e^{0.2} - 1)(1 - e^{-0.5}) \\
 &= 0.08711519719109\dots
 \end{aligned}$$

可见近似值  $S_{4,4}$  已有 5 位有效数字.

实际计算中, 将二重积分化为

$$I = \int_a^b dx \int_c^d f(x, y) dy = \int_a^b F(x) dx$$

其中  $F(x) = \int_c^d f(x, y) dy$ , 再分别对积分变量  $x$  在区间  $[a, b]$  上、 $y$  在区间  $[c, d]$  上使用 Gauss 型求积公式, 也可得到高精度的二重积分求积公式. 详细可参阅 [24].

MATLAB 软件提供的函数 `dblquad` ( ) 可直接计算矩形区域上的二重积分, 其调用形式为

```
>> I = dblquad(fxy, a, b, c, d, eps)
```

其中  $f_{xy}$  是设定的二元被积函数名,  $a, b, c, d$  是积分限,  $\text{eps}$  是误差限. 误差限  $\text{eps}$  可以缺省. MATLAB 软件中没有提供可计算一般二维区域上二重积分的函数, 但数值积分工具箱中的函数  $\text{gquad2dgggen}(\quad)$  可解决这一问题. 长方体上的三重积分函数为  $\text{triplequad}(\quad)$ , 其调用形式为

```
>> triplequad(fxyz,a,b,c,d,e,f,eps,@quadl)
```

读者可以自编程序实现特殊区域上的重积分数值计算.

## § 7.8 应用实例

### 7.8.1 RC 电路的电压

考虑图 7-5 由电阻器、电容器、电源和电压器组成的简单的 RC 电路.

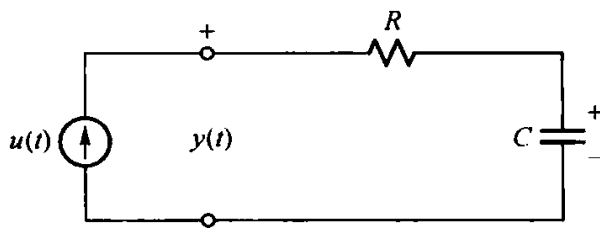


图 7-5

设导线内的电流为  $u(t)$ , 电压器两端的电压为  $y(t)$ . 设电压的总量经过整个环路下降到零, 由 Kirchhoff 电压原理, 知电压可表示为

$$y(t) = Ru(t) + x(t)$$

其中第一项表示电流经过电阻  $R$  电压的损耗, 第二项  $x(t)$  表示电流经过电容器时电压的损耗. 由于电容器两端电压的变化率正比于流经电容器的电流量, 当  $C$  表示为电容量时, 有

$$C \frac{dx(t)}{dt} = u(t)$$

设电容器是在时刻  $t=0$  时启动, 并具有初值  $x(0)$ . 对上述微分方程两端积分, 得电容器两端电压

$$x(t) = x(0) + \frac{1}{C} \int_0^t u(\tau) d\tau$$

从而电压器两端的电压为

$$y(t) = Ru(t) + x(0) + \frac{1}{C} \int_0^t u(\tau) d\tau$$

假设电流函数为  $u(t) = \frac{10te^{-t}}{1 + \cos^2 t}$  mA, 电阻  $R = 33 \Omega$ , 电容  $C = 470 \mu\text{F}$ , 电容的初值  $x(0) = 2 \text{ V}$ , 计算各时刻的电压值  $y(t)$ . 将时间区间  $[0, 10]$  分成 100 等份, 然后用复化梯形公式计算电压  $y(t)$  在各个时间节点  $t_i (i = 0, 1, 2, \dots, 100)$  上的近似值. 用 MATLAB 软件编程计算如下:

```
function rcint = rcint(n)
    T = 10; R = 33; C = 470.e - 6; m = 10; h1 = 10/n
    t = 0:h1:10; x = zeros(n + 1, 1); y = zeros(n + 1, 1);
    h2 = T * h1/m;
    f = inline('t. * exp(-t)./(100 * (1 + cos(t).^2))','t'); % 被积函数
    ft = f(t);
    x(1) = 2.0; y(1) = R * ft(1) + x(1);
    for i = 2:n + 1
        dx = (ft(i - 1) + ft(i))/2.0;
        for j = 1:m - 1 dx = dx + f(t(i - 1) + j * h2); end
        dx = dx * h/C;
        x(i) = x(i - 1) + dx;
        y(i) = R * ft(i) + x(i);
    end
    plot(t, y, 'k-');
    xlabel('t(sec)'); ylabel('y(t)(V)');
```

计算得电压  $y(t)$  变化显示如图 7-6 所示.

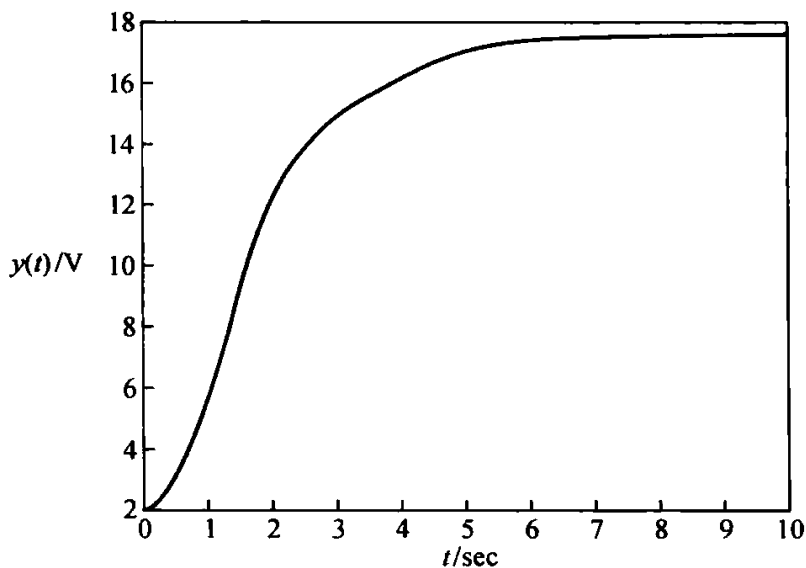


图 7-6 RC 电路电压图

由于当  $t \rightarrow \infty$  时,  $u(t) \rightarrow 0$ , 故  $y(t)$  的稳态值就是电容电压  $x(t)$  的稳态值, 此问题中稳态值  $x = 17.61\text{V}$ . 电压  $y(t)$  在各个时间节点  $t_i (i=0, 1, 2, \dots, 100)$  处的近似值如下:

$$y = 2.000\ 000\ 000\ 000\ 00 \quad 2.064\ 887\ 101\ 212\ 77 \quad 2.215\ 773\ 449\ 959\ 87 \quad \dots$$

$$17.610\ 558\ 304\ 190\ 03 \quad 17.611\ 279\ 128\ 185\ 24 \quad \dots \quad 17.613\ 756\ 833\ 928\ 92$$

根据计算结果,  $t = 10\text{s}$  时, 电压  $y(10) = 17.613\ 756\ 833\ 9$  已趋于稳定值.

### 7.8.2 利用数值积分求解积分方程

在工程具体问题中, 常会遇到如下形式的方程

$$y(x) = \int_a^b K(x, t) y(t) dt + f(x) \quad (7-83)$$

其中  $K(x, t)$  和  $f(x)$  是已知函数,  $y(x)$  是未知函数. 由于积分号下含有未知函数, 故称式(7-83)为积分方程. 积分方程的求解往往很困难, 利用数值积分法, 可求得近似解.

若对于区间  $[a, b]$  上的积分采用数值求积公式  $I(g) = \sum_{k=0}^n A_k g(x_k)$ , 其中  $x_k$  和  $A_k (k=0, 1, 2, \dots, n)$  分别为求积节点和求积系数. 那么上述积分方程中的积分项近似为

$$\int_a^b K(x, t) y(t) dt \approx \sum_{k=0}^n A_k K(x, x_k) y(x_k), \quad x \in [a, b]$$

从而积分方程可近似为

$$y(x) \approx \sum_{k=0}^n A_k K(x, x_k) y(x_k) + f(x), \quad x \in [a, b]$$

设  $\tilde{y}(x)$  为  $y(x)$  的近似函数,  $\tilde{y}_k$  表示  $y(x_k)$  的近似值. 由上述近似方程, 得

$$\tilde{y}(x) = \sum_{k=0}^n A_k K(x, x_k) \tilde{y}(x_k) + f(x), \quad x \in [a, b]$$

记  $K_{ik} = K(x_i, x_k)$ ,  $f_i = f(x_i) (i, k=0, 1, 2, \dots, n)$ , 得线性代数方程组

$$\tilde{y}_i = \sum_{k=0}^n K_{ik} \tilde{y}_k + f_i, \quad i = 0, 1, 2, \dots, n$$

若记

$$\mathbf{K} = \begin{bmatrix} A_0 K_{00} & A_1 K_{01} & \cdots & A_n K_{0n} \\ A_0 K_{10} & A_1 K_{11} & \cdots & A_n K_{1n} \\ \vdots & & & \vdots \\ A_0 K_{n0} & A_1 K_{n1} & \cdots & A_n K_{nn} \end{bmatrix}$$

$\mathbf{f} = (f_0, f_1, f_2, \dots, f_n)^T$ ,  $\tilde{\mathbf{y}} = (\tilde{y}_0, \tilde{y}_1, \dots, \tilde{y}_n)^T$ ,  $\mathbf{M} = \mathbf{I} - \mathbf{K}$ , 线性代数方程组可表为矩

阵形式  $M\bar{y} = f$ . 若系数矩阵  $M$  非奇异, 线性代数方程组存在唯一解  $\bar{y} = (\bar{y}_0, \bar{y}_1, \dots, \bar{y}_n)^T$ , 从而得积分方程的近似表达式.

例如, 分别用梯形求积公式和 Simpson 求积公式及其复化求积公式求解积分方程:

$$y(x) = \int_0^1 \left( \frac{1}{1+t} - x \right) y(t) dt + \frac{4x^3 + 5x^2 - 2x + 5}{8(x+1)^2}$$

[解] 1. 用梯形公式离散方程中的积分项, 即

$$\int_0^1 \left( \frac{1}{1+t} - x \right) y(t) dt \approx \frac{1}{2} \left[ (1-x)\bar{y}_0 + \left( \frac{1}{2} - x \right) \bar{y}_1 \right]$$

从而有

$$\bar{y}_i(x) = \frac{1}{2} \left[ (1-x)\bar{y}_0 + \left( \frac{1}{2} - x \right) \bar{y}_1 \right] + \frac{4x^3 + 5x^2 - 2x + 5}{8(x+1)^2}$$

分别令  $x=0, 1$ , 得方程组

$$\begin{cases} \bar{y}_0 = \frac{1}{2} \left( \bar{y}_0 + \frac{1}{2} \bar{y}_1 \right) + \frac{5}{8} \\ \bar{y}_1 = \frac{1}{2} \left( -\frac{1}{2} \bar{y}_1 \right) + \frac{3}{8} \end{cases}$$

求得解  $\bar{y}_0 = 0.3, \bar{y}_1 = 1.4$ , 代入  $\bar{y}_i(x)$  的表达式, 得

$$\bar{y}_i(x) = -\frac{7x^3 + 6x^2 - 9x - 28}{20(x+1)^2}$$

2. 若用 Simpson 公式求解积分方程, 那么

$$\int_0^1 \left( \frac{1}{1+t} - x \right) y(t) dt \approx \frac{1}{6} \left[ (1-x)\bar{y}_0 + 4\left(\frac{2}{3} - x\right)\bar{y}_1 + \left(\frac{1}{2} - x\right)\bar{y}_2 \right]$$

有

$$\bar{y}_i(x) = \frac{1}{6} \left[ (1-x)\bar{y}_0 + 4\left(\frac{2}{3} - x\right)\bar{y}_1 + \left(\frac{1}{2} - x\right)\bar{y}_2 \right] + \frac{4x^3 + 5x^2 - 2x + 5}{8(x+1)^2}$$

当  $x=0, \frac{1}{2}, 1$  时, 得线性方程组

$$\begin{cases} \bar{y}_0 = \frac{1}{6} \left( \bar{y}_0 + \frac{8}{3} \bar{y}_1 + \frac{1}{2} \bar{y}_2 \right) + \frac{5}{8} \\ \bar{y}_1 = \frac{1}{6} \left( \frac{1}{2} \bar{y}_0 + \frac{2}{3} \bar{y}_1 \right) + \frac{23}{72} \\ \bar{y}_2 = \frac{1}{6} \left( -\frac{4}{3} \bar{y}_1 - \frac{1}{2} \bar{y}_2 \right) + \frac{3}{8} \end{cases}$$

经求解得  $\bar{y}_0 = 1.017\ 845\ 117\ 8, \bar{y}_1 = 0.454\ 797\ 979\ 8, \bar{y}_2 = 0.252\ 861\ 952\ 9$ , 代入  $\bar{y}_i(x)$  的表达式, 得



$$\tilde{y}_s(x) = -\frac{4\,972x^3 + 5\,587x^2 - 3\,742x - 39\,997}{35\,640(x+1)^2}$$

此积分方程的精确解为  $y(x) = \frac{1}{(x+1)^2}$ , 两个近似函数与精确解形式上有很大差异, 近似函数与精确解的在区间  $[0, 1]$  上的曲线如图 7-7 所示, 两个近似函数的效果并不理想.

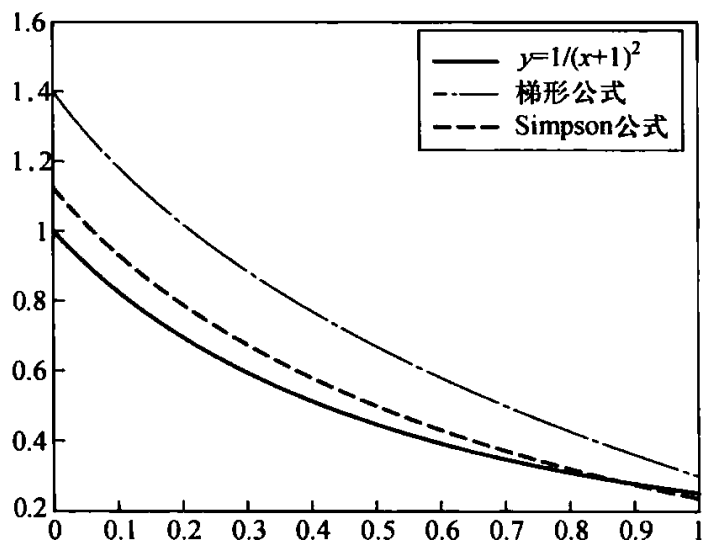


图 7-7

3. 如果取  $n=4$ , 采用复化梯形求积公式计算, 则有

$$\begin{aligned} \tilde{y}_4(x) = & \frac{1}{8} \left[ (1-x)\tilde{y}_0 + 2\left(\frac{4}{5}-x\right)\tilde{y}_1 + 2\left(\frac{2}{3}-x\right)\tilde{y}_2 \right. \\ & \left. + 2\left(\frac{4}{7}-x\right)\tilde{y}_3 + \left(\frac{1}{2}-x\right)\tilde{y}_4 \right] + \frac{4x^3 + 5x^2 - 2x + 5}{8(x+1)^2} \end{aligned}$$

分别取  $x=0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$ , 得线性方程组

$$\begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} = \begin{bmatrix} 8 & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{16} \\ \frac{3}{32} & \frac{11}{80} & \frac{5}{48} & \frac{9}{112} & \frac{3}{32} \\ \frac{1}{16} & \frac{3}{40} & \frac{1}{24} & \frac{1}{56} & 0 \\ \frac{3}{32} & \frac{1}{80} & -\frac{1}{48} & -\frac{5}{112} & -\frac{3}{32} \\ 0 & -\frac{1}{20} & -\frac{1}{12} & -\frac{3}{28} & -\frac{1}{16} \end{bmatrix} \begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} + \begin{bmatrix} \frac{5}{8} \\ \frac{39}{100} \\ \frac{23}{72} \\ \frac{16}{49} \\ \frac{3}{8} \end{bmatrix}$$

求解线性方程组得

$$\tilde{y} = \begin{bmatrix} 1.038\ 424\ 081\ 591\ 44 \\ 0.683\ 191\ 582\ 099\ 52 \\ 0.461\ 654\ 118\ 689\ 67 \\ 0.382\ 659\ 883\ 735\ 29 \\ 0.245\ 995\ 266\ 899\ 01 \end{bmatrix}$$

代入  $\tilde{y}_{i4}(x)$  的表达式, 得

$$\tilde{y}_{i4}(x) = \frac{-0.042\ 429x^3 - 0.046\ 434x^2 + 0.034\ 419x + 1.038\ 424}{(x+1)^2}$$

近似函数  $\tilde{y}_{i4}(x)$  与精确解仍有很大差别, 结果不理想.

4. 如果取  $n=4$ , 采用复化 Simpson 求积公式计算, 有

$$\begin{aligned} \tilde{y}_{s4}(x) = & \frac{1}{12} \left[ (1-x)\tilde{y}_0 + 4\left(\frac{4}{5}-x\right)\tilde{y}_1 + 2\left(\frac{2}{3}-x\right)\tilde{y}_2 \right. \\ & \left. + 4\left(\frac{4}{7}-x\right)\tilde{y}_3 + \left(\frac{1}{2}-x\right)\tilde{y}_4 \right] + \frac{4x^3 + 5x^2 - 2x + 5}{8(x+1)^2} \end{aligned}$$

分别取  $x=0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$ , 得如下的线性方程组

$$\begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} = \begin{bmatrix} \frac{1}{12} & \frac{4}{15} & \frac{1}{9} & \frac{4}{21} & \frac{1}{24} \\ \frac{1}{16} & \frac{11}{60} & \frac{5}{72} & \frac{3}{28} & \frac{1}{48} \\ \frac{1}{24} & \frac{1}{10} & \frac{1}{36} & \frac{1}{42} & 0 \\ \frac{1}{48} & \frac{1}{60} & -\frac{1}{72} & -\frac{5}{84} & -\frac{1}{48} \\ 0 & -\frac{1}{15} & -\frac{1}{18} & -\frac{1}{7} & -\frac{1}{24} \end{bmatrix} \begin{bmatrix} \tilde{y}_0 \\ \tilde{y}_1 \\ \tilde{y}_2 \\ \tilde{y}_3 \\ \tilde{y}_4 \end{bmatrix} + \begin{bmatrix} \frac{5}{8} \\ \frac{39}{100} \\ \frac{23}{72} \\ \frac{16}{49} \\ \frac{3}{8} \end{bmatrix}$$

求解线性方程组得

$$\tilde{y} = \begin{pmatrix} 1.001\ 780\ 540\ 130\ 38 \\ 0.641\ 414\ 181\ 527\ 15 \\ 0.445\ 492\ 267\ 368\ 35 \\ 0.327\ 212\ 076\ 565\ 57 \\ 0.250\ 315\ 105\ 717\ 43 \end{pmatrix}$$

将其代入  $\tilde{y}_{s4}(x)$  的表达式, 取 6 位小数位得

$$\tilde{y}_{s4}(x) = \frac{0.001\ 465x^3 + 0.001\ 150x^2 - 0.002\ 096x + 1.001\ 781}{(x+1)^2}$$

比较  $\tilde{y}_{s4}(x)$  与精确解,  $\tilde{y}_{s4}(x)$  的分子中各项系数误差已小于 0.0021. 由图 7-8

可见,近似函数  $\tilde{y}_4(x)$  与精确解的图形已完全重合,经计算近似解的误差小于 0.002,若要求更高的精度,可再增加节点.

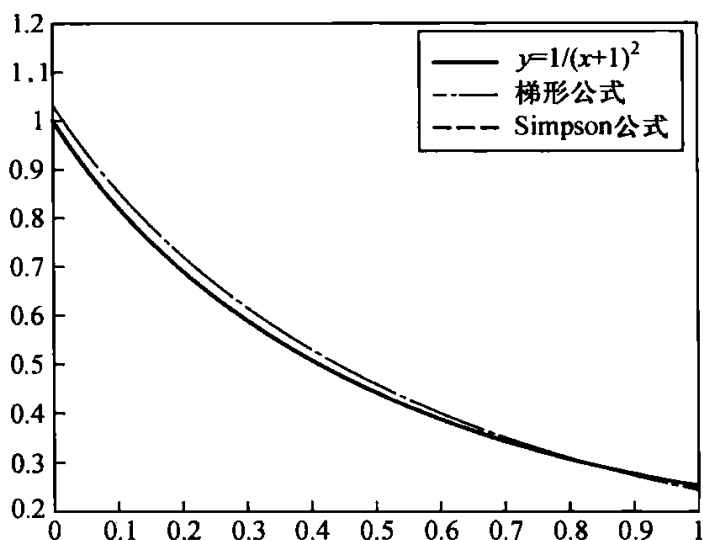


图 7-8

## 评 注

本章介绍了数值微分的最基本的差商型近似计算公式和插值型近似计算公式(含三次样条求导公式).数值微分公式还可由 Taylor 展开法和数值积分得到.若将 Taylor 展开式展开至待求的一阶或二阶导数,得到的数值微分公式与差商型公式一致;若将 Taylor 展开式展开至比待求导数高两阶的导数,并用待求导数的差商近似更高阶的导数,得到数值微分的隐式格式,且精度可提高到  $O(h^4)$ .利用微分与积分运算的互逆性,可借助数值积分公式计算数值微分.若采用 Simpson 求积公式,同样可得到具有精度  $O(h^4)$  的隐式格式.数值微分的隐式格式需要求解一个主对角严格占优的三对角线性方程组,用追赶法求解是数值稳定的.结合外推技术,也可大大地提高数值微分的精度.详细可参阅[24].

本章重点讨论了 Newton-Cotes 求积公式、Gauss 型求积公式和 Romberg 求积公式,它们都是以插值法为基础的. Newton-Cotes 求积公式中的节点是等距的,高阶 Newton-Cotes 求积公式( $n \geq 8$ )是数值不稳定的,通常使用低阶复化求积公式,实际计算中常采用逐次分半或自适应的梯形算法和 Simpson 算法. Gauss 型求积公式中的节点是正交多项式的零点,通常不是等距的,具有  $2n-1$  次代数精确度,且是数值稳定的.实际计算中低阶复化 Gauss 型求积公式也是很有效的. Richardson 外推是数值计算中提高计算精度的一种重要技术,由此产生的 Romberg 求积方法计算简单,精度较高,是实际计算中的常用方法.

本章对振荡函数的积分作了简单的讨论. 也可对被积函数中的非振荡部分用分段二次插值近似, 得到振荡函数积分的近似求积公式; 或先以被积函数中振荡部分的零点分割积分区间, 再在每两个相邻零点构成的子区间上, 运用数值积分公式. 本章未涉及奇异积分的计算. 奇异积分有时可通过变量置换消去奇点, 或通过区间截断和极限过程计算积分近似值, 无穷区间上的积分可利用 Gauss-Laguerre 求积公式和 Gauss-Hermite 求积公式计算.

## 习 题 七

1. 已知函数表如下:

| $x_i$    | 101       | 102       | 103       |
|----------|-----------|-----------|-----------|
| $f(x_i)$ | 10.049 88 | 10.099 50 | 10.148 89 |

用三点公式求  $f'(101)$ ,  $f'(102)$ .

2. 已知函数  $y = e^x$  的函数值如下:

| $x_i$           | 2.5      | 2.6      | 2.7      | 2.8      | 2.9      |
|-----------------|----------|----------|----------|----------|----------|
| $y_i = e^{x_i}$ | 12.182 5 | 13.463 7 | 14.879 7 | 16.444 6 | 18.174 1 |

取  $h = 0.1$ ,  $h = 0.2$ , 分别用三点公式计算  $x = 2.7$  处的一、二阶导数值.

3. 设  $f(x) = \frac{1}{(1+x)^2}$ , 取步长  $h = 0.1$ , 分别用二点和三点公式计算  $f'(0.5)$ , 并估计误差.

4. 分别用梯形公式和 Simpson 公式计算下列积分的近似值并估计误差:

$$(1) \int_0^1 e^{-x} dx$$

$$(2) \int_{0.1}^{0.5} e^{\frac{1}{x}} dx$$

5. 求 Newton-Cotes 系数  $C_k^{(3)}$  ( $k = 0, 1, 2, 3$ ).

6. 试证明  $n+1$  个节点的 Newton-Cotes 系数满足

$$\sum_{k=0}^n C_k^{(n)} = 1$$

7. 设  $f''(x) < 0$ ,  $x \in [a, b]$ , 试证明用梯形公式计算积分  $\int_a^b f(x) dx$  所得近似

值小于准确值,并说明其几何意义.

8. 求近似公式

$$\int_0^1 f(x) dx \approx \frac{1}{3} \left[ 2f\left(\frac{1}{4}\right) - f\left(\frac{1}{2}\right) + 2f\left(\frac{3}{4}\right) \right]$$

的代数精确度.

9. 分别用  $n=10$  的复化梯形公式和复化 Simpson 公式计算积分

$$I = \int_1^1 e^{-x^2} dx$$

的近似值,并估计误差.

10. 分别用复化梯形公式和复化 Simpson 公式计算下列积分的近似值:

$$(1) \int_0^1 \frac{x}{1+x^2} dx, n=8$$

$$(2) \int_0^1 \frac{\ln(1+x)}{(1+x)^2} dx, n=4$$

11. 若用复化梯形公式计算积分  $\int_0^1 e^{-x^2} dx$ , 问积分区间要等分为多少份才能保证计算结果有四位有效数字(假定计算过程无舍入误差)? 若用复化 Simpson 公式计算, 积分区间应等分为多少份?

12. 根据函数表

| $i$      | 0         | 1         | 2         | 3         | 4         | 5         | 6         | 7         | 8         |
|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| $x_i$    | 0         | 0.125     | 0.25      | 0.375     | 0.5       | 0.625     | 0.75      | 0.875     | 1         |
| $f(x_i)$ | 1.000 000 | 0.997 398 | 0.989 616 | 0.976 727 | 0.958 851 | 0.936 156 | 0.908 852 | 0.877 193 | 0.841 471 |

计算积分  $I = \int_0^1 f(x) dx$  的近似值,并估计误差.

13. 试用梯形公式的逐次分半算法计算下列积分:

$$(1) \int_0^\pi \frac{dx}{x + \cos x}$$

$$(2) \int_0^1 \frac{dx}{1+x^3}$$

要求误差不超过  $\varepsilon = 3 \times 10^{-4}$ .

14. 用 Romberg 方法计算积分

$$I = \int_0^4 \sqrt{1 + \cos^2 x} dx$$

的近似值,要求误差不超过  $10^{-2}$ .

## 15. 用 Romberg 方法计算积分

$$I = \int_0^1 e^{-x^2} dx$$

的近似值, 要求误差不超过  $10^{-4}$ .

## 16. 确定下列公式中的节点及求积系数, 使其具有尽可能高的代数精确度:

$$(1) \int_{-h}^h f(x) dx \approx A_1 f\left(-\frac{h}{2}\right) + A_2 f(0) + A_3 f\left(\frac{h}{2}\right)$$

$$(2) \int_0^2 f(x) dx \approx f(x_1) + f(x_2)$$

$$(3) \int_{-1}^1 f(x) dx \approx A_1 f(x_1) + A_2 f(0) + f(1)$$

17. 试证明区间  $[a, b]$  上的两点 Gauss 型求积公式为

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[ f\left(a + \left(\frac{1}{2} - \frac{\sqrt{3}}{6}\right)h\right) + f\left(a + \left(\frac{1}{2} + \frac{\sqrt{3}}{6}\right)h\right) \right]$$

并估计截断误差, 其中  $h = b - a$ .

18. 试确定  $x_1, x_2, A_1, A_2$ , 使求积公式

$$\int_0^1 \sqrt{x} f(x) dx \approx A_1 f(x_1) + A_2 f(x_2)$$

为 Gauss 型求积公式.

## 19. 判别下列求积公式中, 哪个是 Gauss 型求积公式:

$$(1) \int_{-1}^1 f(x) dx \approx \frac{2}{3} [f(-1) + f(0) + f(1)]$$

$$(2) \int_{-1}^1 f(x) dx \approx \frac{1}{3} [f(-1) + 4f(0) + f(1)]$$

$$(3) \int_0^2 f(x) dx \approx \frac{1}{9} [5f(1 - \sqrt{0.6}) + 8f(1) + 5f(1 + \sqrt{0.6})]$$

## 20. 证明: 若求积公式

$$\int_a^b f(x) dx \approx \sum_{k=1}^n A_k f(x_k)$$

的代数精确度不小于  $n-1$ , 则必有

$$A_k = \int_a^b l_k(x) dx \quad (k = 1, 2, \dots, n)$$

其中  $l_k(x)$  是以  $x_1, x_2, \dots, x_n$  为节点的 Lagrange 插值基函数.

## 21. 用三点及五点 Gauss-Legendre 求积公式计算积分

$$I = \int_0^1 \frac{2\sin(2x-1)}{3-2x} dx$$

## 22. 利用 Gauss-Chebyshev 求积公式计算积分

$$I = \int_{-1}^1 \frac{p_n(x)}{\sqrt{1-x^2}} dx$$

其中  $p_n(x)$  为  $n$  次多项式.

23. 分别用  $n=4$  的 Gauss-Laguerre 求积公式和 Gauss-Hermite 求积公式计算积分

$$I = \int_0^{+\infty} e^{-x^2} dx$$

的近似值, 并与准确值  $I = \frac{\sqrt{\pi}}{2}$  作比较.

24. 用求积公式(7-72)计算积分

$$I = \int_0^{\frac{3}{2}\pi} \cos 15x dx$$

取  $n=10, h = \frac{3\pi}{20}$ .

25. 若分区间  $[a, b]$  为  $N$  等份, 记  $h = \frac{b-a}{N}, x_k = a + kh (k=0, 1, \dots, N)$ , 用阶梯函数

$$\varphi(x) = f\left(x_k + \frac{h}{2}\right), \quad x_k \leq x < x_{k+1} \quad (k=0, 1, \dots, N-1)$$

近似函数  $f(x)$ , 试导出振荡积分  $\int_a^b f(x) \cos \omega x dx$  的求积公式.

26. 设  $n=m=4$ , 试分别用复化梯形公式和复化 Simpson 计算二重积分, 并将计算结果与精确值作比较:

$$(1) I = \int_{1.0}^{1.5} dx \int_{1.2}^{1.6} \ln(x+2y) dy; \quad (2) I = \int_0^{0.5} dx \int_1^{1.5} \frac{1}{1+xy} dy.$$

27. 设  $n=m=4$ , 试推导在  $x$  方向用复化梯形公式, 在  $y$  方向用复化 Simpson 公式的计算矩形区域上二重积分的近似求积公式, 并计算 26 题中的积分.

## 数值实验

1. 考纽螺线的形状像钟表的发条, 也称回旋曲线, 它在直角坐标系中的参数方程为

$$\begin{cases} x(s) = \int_0^s \cos \frac{1}{2} at^2 dt \\ y(s) = \int_0^s \sin \frac{1}{2} at^2 dt \end{cases}$$

曲线关于原点对称. 取  $a=1$ , 参数  $s$  的变化范围  $[-5, 5]$ , 容许误差限分别是

$10^{-6}$  和  $10^{-10}$ . 选取适当的节点个数, 利用数值积分方法计算曲线上点的坐标, 并画出曲线的图形.

2. 利用数值积分公式求解积分方程:

(1)  $y(x) = \int_0^1 (t-x)y(t)dt + e^{2x} + \left(\frac{e^2-1}{2}\right)x - \frac{e^2+1}{4}$ , 精确解是  $y(x) = e^{2x}$ ;

(2)  $\int_{-\pi}^{\pi} g(t) \ln [(\cos x - \cos t)^2 + (\sin x - \sin t)^2] dt = -2\cos x$ , 精确解是  $g(x) = 2\cos x$ .

分别用复化求积公式和高斯型求积公式离散积分项, 求解上述积分方程, 得到解函数的近似表达式, 分析计算结果, 要求误差小于  $10^{-6}$ . 绘出数值解和精确解的图形及误差图形.

3. 设  $n, m$  分别取 10, 20, 编写计算二重积分(7-75)的程序, 计算积分:

(1)  $I = \int_{-1}^1 dx \int_{-2}^2 e^{-\frac{x^2}{2}} \sin(x^2 + y^2) dy$ ; (2)  $I = \int_0^{\frac{\pi}{2}} dx \int_0^{\frac{\pi}{3}} \frac{\tan(x^2 + y^2)}{(1+x+y)} dy$ .



## 第八章 非线性方程及非线性方程组的解法

科学应用与工程实际中出现的问题常常可以归结为求解非线性方程或非线性方程组的问题,例如多项式求根问题.一般地,求方程

$$f(x) = 0 \quad (8-1)$$

的根,其中  $f(x)$  为非线性函数,即为非线性方程求解问题.

非线性方程的根可以是实数,也可以是复数,如果在  $x = a$  处,  $f(x)$  满足

$$\begin{aligned} f(a) &= 0, \quad f^{(l)}(a) = 0 \quad (l = 1, \dots, k-1) \\ f^{(k)}(a) &\neq 0 \quad (k = 1, 2, \dots) \end{aligned} \quad (8-2)$$

则称  $x = a$  是方程(8-1)的  $k$  重根或  $f(x)$  的  $k$  重零点,特别地,当  $k = 1$  时,  $x = a$  称为单根或单零点. 方程(8-1)有  $k$  重根  $x = a$  等价于函数  $f(x)$  可以分解成以下形式

$$f(x) = (x - a)^k \varphi(x) \quad (8-3)$$

其中  $\varphi(a) \neq 0$ .

非线性方程(组)的根能用解析式表示的极少,绝大多数需借助于数值方法求近似解. 本章主要介绍几种求解非线性方程的常用数值方法,最后一节简略地介绍求解非线性方程组的方法.

### § 8.1 对分区间法

设  $f(x)$  在区间  $[a, b]$  上连续,且有

$$f(a)f(b) < 0$$

则由连续函数介值定理,  $f(x)$  在  $(a, b)$  内必有零点,称  $(a, b)$  为方程(8-1)的有根区间. 不妨设  $f(a) < 0, f(b) > 0$ . 取  $x_0 = \frac{a+b}{2}$ , 若  $f(x_0) = 0$ , 则  $x = x_0$  就是方程(8-1)的解. 否则,若  $f(x_0) < 0$ , 取  $a_1 = x_0, b_1 = b$ ; 若  $f(x_0) > 0$ , 取  $a_1 = a, b_1 = x_0$ , 则有

$$[a_1, b_1] \subset [a, b], \quad b_1 - a_1 = \frac{b-a}{2}$$

且  $f(x)$  在  $[a_1, b_1]$  上连续, 满足  $f(a_1)f(b_1) < 0$ . 重复上述过程又可得到区间  $[a_2, b_2]$ , 满足  $[a_2, b_2] \subset [a_1, b_1]$ ,  $b_2 - a_2 = \frac{b_1 - a_1}{2}$ , 且  $f(a_2)f(b_2) < 0$ . 如此继续下去, 得到一个区间序列

$$[a, b] \supset [a_1, b_1] \supset [a_2, b_2] \supset \cdots \supset [a_n, b_n] \supset \cdots$$

满足

$$f(a_n)f(b_n) < 0, \quad b_n - a_n = \frac{b - a}{2^n}$$

因而  $[a_n, b_n]$  ( $n = 1, 2, \cdots$ ) 均为方程 (8-1) 的有根区间. 由区间套定理, 存在  $x^* \in [a, b]$ , 使得

$$\lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} b_n = x^*$$

且  $x = x^*$  是方程 (8-1) 的根. 若取  $x_n = \frac{a_n + b_n}{2}$  作为根  $x^*$  的近似值, 则误差为

$$|x^* - x_n| \leq \frac{b_n - a_n}{2} = \frac{b - a}{2^{n+1}} \quad (8-4)$$

按上述方法求非线性方程 (8-1) 的近似解称为对分区间法 (bisection). 式 (8-4) 表明, 只要  $f(x)$  连续, 对分区间法总是收敛的.

### 算法 8.1

- (1) 输入  $f(x)$ ,  $a, b$ , 误差限  $\varepsilon$ , 最大容许迭代次数  $N$ .
- (2) 置  $n = 0$ .
- (3)  $x_n = \frac{a + b}{2}$ .
- (4) 若  $f(x_n) = 0$  或  $\frac{b - a}{2} < \varepsilon$ , 则输出  $x^* = x_n$ , 停机; 否则转 5.
- (5) 若  $n < N$ , 转 6; 否则输出失败信息, 停机.
- (6) 若  $f(a)f(x_n) < 0$ , 则  $x_n \Rightarrow b$ ; 否则,  $x_n \Rightarrow a$ .
- (7) 置  $n + 1 \Rightarrow n$ , 转 3.

式 (8-4) 不仅可以估计对分区间法的误差, 而且可以由给定的误差限估计出对分区间的次数. 设要求近似解的误差不超过  $\varepsilon$ , 则由式 (8-4)

$$|x^* - x_n| \leq \frac{b - a}{2^{n+1}} \leq \varepsilon$$

可得

$$2^{n+1} \geq \frac{b - a}{\varepsilon}$$

从而有

$$n \geq \frac{\ln(b-a) - \ln \varepsilon}{\ln 2} - 1 \quad (8-5)$$

例 1 试用对分区间法求方程

$$f(x) = x^3 + 10x - 20 = 0 \quad (8-6)$$

的唯一实根, 要求误差不超过  $\frac{1}{2} \times 10^{-4}$ .

[解] 容易验证

$$f(1) = 1 + 10 - 20 < 0, \quad f(2) = 8 + 20 - 20 > 0$$

且  $f'(x) = 3x^2 + 10 > 0, \forall x \in (-\infty, +\infty)$ , 故  $f(x)$  在  $(-\infty, +\infty)$  上单调增加, 于是方程 (8-6) 有唯一实根, 且  $[1, 2]$  为有根区间. 为达到要求精度, 按式 (8-5), 对分区间次数为

$$n \geq \frac{\ln(2-1) - \ln\left(\frac{1}{2} \times 10^{-4}\right)}{\ln 2} - 1 = 13.28$$

故  $n = 14$ , 计算结果见 8-1 表.

表 8-1

| $n$ | $a_n$       | $b_n$       | $x_n$       | $f(x_n)$      |
|-----|-------------|-------------|-------------|---------------|
| 0   | 1           | 2           | 1.5         | -1.625        |
| 1   | 1.5         | 2           | 1.75        | 2.859 375     |
| 2   | 1.5         | 1.75        | 1.625       | 0.541 015 6   |
| 3   | 1.5         | 1.625       | 1.562 5     | -0.560 302 7  |
| 4   | 1.562 5     | 1.625       | 1.593 75    | -0.014 312 7  |
| 5   | 1.593 75    | 1.625       | 1.609 375   | 0.262 172 6   |
| 6   | 1.593 75    | 1.609 375   | 1.601 562 5 | 0.123 636 7   |
| 7   | 1.593 75    | 1.601 562 5 | 1.597 656 3 | 0.054 589 4   |
| 8   | 1.593 75    | 1.597 656 3 | 1.595 703 2 | 0.020 120 8   |
| 9   | 1.593 75    | 1.595 703 2 | 1.594 726 6 | 0.002 899 6   |
| 10  | 1.593 75    | 1.594 726 6 | 1.594 238 3 | -0.005 707 7  |
| 11  | 1.594 238 3 | 1.594 726 6 | 1.594 482 5 | -0.001 403 7  |
| 12  | 1.594 482 5 | 1.594 726 6 | 1.594 604 6 | 0.000 748 64  |
| 13  | 1.594 482 5 | 1.594 604 6 | 1.594 543 6 | -0.000 326 42 |
| 14  | 1.594 543 6 | 1.594 604 6 | 1.594 574 1 |               |

所以  $x_{14} = 1.594\ 574\ 1$  即为符合精度要求的解.

对分区间法的显著优点是计算简便,容易估计误差,但它收敛较慢,且只能用于求奇数重实根.

## § 8.2 简单迭代法

如果已知方程(8-1)的根  $x^*$  的一个近似值  $x_0$ ,通过构造一个递推公式

$$x_{n+1} = g(x_n) \quad (n = 0, 1, 2, \dots) \quad (8-7)$$

由此产生序列  $\{x_n\}$ ,以  $x_n$  逐次逼近  $x^*$ . 上述求非线性方程近似根的方法称为迭代法(单步),式(8-7)称为迭代格式, $g(x)$ 称为迭代函数, $\{x_n\}$ 称为迭代序列. 若  $\{x_n\}$  收敛于  $x^*$ ,则称迭代法收敛,否则称迭代法是发散的. 当  $g(x)$  连续时,若  $\lim_{n \rightarrow \infty} x_n = x^*$ ,则有  $x^* = g(x^*)$ ,称  $x^*$  为函数  $g(x)$  的不动点,上述方法又称为不动点迭代法. 迭代格式的构造不同即得出不同的迭代法,本节介绍一种最直接、简单的迭代法.

### 8.2.1 简单迭代法的一般形式及其几何意义

如果将方程(8-1)改写成以下等价形式

$$x = \varphi(x) \quad (8-8)$$

则方程(8-8)和(8-1)同解. 取定初始值  $x_0$ ,由迭代公式

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2, \dots) \quad (8-9)$$

产生迭代序列  $\{x_n\}$ . 显然,若  $\{x_n\}$  收敛于  $x^*$ , $\varphi(x)$  在  $x^*$  处连续,就有

$$x^* = \lim_{n \rightarrow \infty} x_{n+1} = \lim_{n \rightarrow \infty} \varphi(x_n) = \varphi(x^*)$$

即  $x^*$  是方程(8-8)的解,从而也是(8-1)的解. 故当  $n$  充分大时,可取  $x_n$  作为方程(8-1)的根的近似值. 按上述方法构造迭代格式(8-9)来求解方程(8-1),称为简单迭代法. 简单迭代法是一种不动点迭代法.

**例 2** 用简单迭代法求方程

$$f(x) = x^3 + 10x - 20 = 0$$

的根,要求精确到六位小数.

[解] 由例 1,所给方程在  $[1, 2]$  内有根,取  $x_0 = 1.5$ .

(1) 若化原方程为等价方程

$$x = x^3 + 11x - 20$$

按迭代格式

$$x_{n+1} = x_n^3 + 11x_n - 20 \quad (8-10)$$

计算,有

$$x_1 = -0.125, \quad x_2 = -21.376\,953, \quad x_3 = -10\,023.861, \dots$$

显然,按迭代格式(8-10)计算,所得迭代序列发散.

(2) 若化原方程为等价方程

$$x = \frac{20}{x^2 + 10}$$

按迭代格式

$$x_{n+1} = \frac{20}{x_n^2 + 10} \quad (8-11a)$$

计算,结果见表 8-2.

表 8-2

| $n$ | $x_n$       | $n$ | $x_n$       |
|-----|-------------|-----|-------------|
| 0   | 1.5         | 8   | 1.594 493 4 |
| 1   | 1.632 653 1 | 9   | 1.594 590 0 |
| 2   | 1.579 085 8 | 10  | 1.594 550 8 |
| 3   | 1.600 830 9 | 11  | 1.594 566 7 |
| 4   | 1.592 019 6 | 12  | 1.594 560 3 |
| 5   | 1.595 592 8 | 13  | 1.594 562 9 |
| 6   | 1.594 144 2 | 14  | 1.594 561 8 |
| 7   | 1.594 731 5 | 15  | 1.594 562 2 |

因为

$$|x_{15} - x_{14}| = 1.594\,562\,2 - 1.594\,561\,8 = 0.000\,000\,4 < \frac{1}{2} \times 10^{-6}$$

所以  $x_{15} = 1.594\,562$  即为方程满足精度要求的近似根.

(3) 若化原方程为等价方程

$$x = 2 - \frac{x^3}{10}$$

按迭代格式

$$x_{n+1} = 2 - \frac{x_n^3}{10} \quad (8-11b)$$

计算,结果见表 8-3.

表 8-3

| $n$ | $x_n$       | $n$ | $x_n$       |
|-----|-------------|-----|-------------|
| 0   | 1.5         | 8   | 1.584 093 0 |
| 1   | 1.662 5     | 9   | 1.602 495 5 |
| 2   | 1.540 500 6 | 10  | 1.588 480 5 |
| 3   | 1.634 417 3 | 11  | 1.599 183 5 |
| 4   | 1.563 394 8 | 12  | 1.591 026 8 |
| 5   | 1.617 874 5 | 13  | 1.597 252 9 |
| 6   | 1.576 518 5 | 14  | 1.592 506 2 |
| 7   | 1.608 170 4 | 15  | 1.596 128 3 |

$$|x_{15} - x_{14}| = 0.362\ 21 \times 10^{-2}$$

容易看出,按迭代格式(8-11b)计算,所得迭代序列也收敛,但收敛较慢,迭代15次尚未达到精度要求.

例2的计算结果表明,将一方程化为等价方程的方法很多,由此可以构造许多不同的迭代函数,得到多种迭代格式.而它们所产生的迭代序列则可能收敛,也可能发散,可能收敛很快,也可能收敛很慢.迭代法的收敛性取决于迭代函数在方程的根的邻近的性态.

从几何上看,简单迭代法是将求曲线  $y=f(x)$  的零点问题化为求曲线  $y=\varphi(x)$  与直线  $y=x$  的交点,迭代过程如图8-1所示,从初始点  $x_0$  出发,沿直线  $x=x_0$  走到曲线  $y=\varphi(x)$ ,得点  $(x_0, \varphi(x_0))$ ,再沿直线  $y=\varphi(x_0)$  走到直线  $y=x$ ,交点为  $(x_1, x_1)$ ,如此继续下去,越来越接近点  $(x^*, x^*)$ .

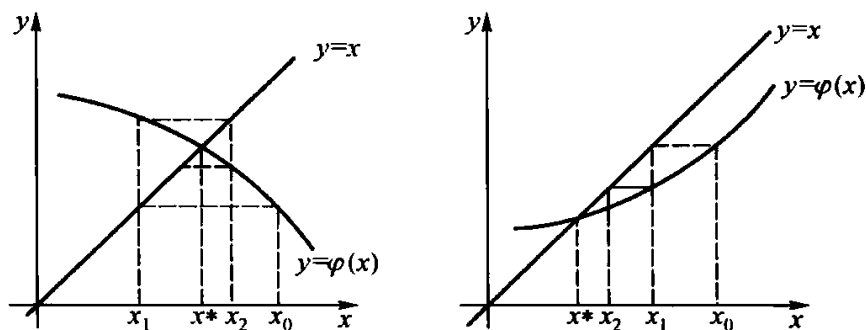


图 8-1

当然,迭代过程也可能出现图8-2所示的情况,此时点  $(x_n, x_n)$  越来越远离交点  $(x^*, x^*)$ ,迭代序列发散.

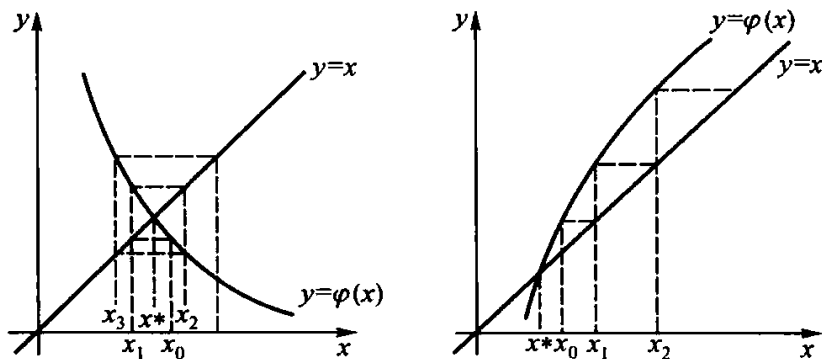


图 8-2

### 8.2.2 迭代法的收敛条件

**定理 8.1 (压缩映象原理)** 设函数  $\varphi(x)$  在区间  $[a, b]$  上满足条件:

- (1) 对任意  $x \in [a, b]$ , 都有  $a \leq \varphi(x) \leq b$ ;
- (2) 存在常数  $0 < L < 1$ , 使得对一切  $x, y \in [a, b]$ , 都有

$$|\varphi(x) - \varphi(y)| \leq L|x - y|$$

则方程  $x = \varphi(x)$  在  $[a, b]$  内有唯一的根  $x^*$ , 且对任何初值  $x_0 \in [a, b]$ , 迭代序列

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, \dots)$$

均收敛于  $x^*$ , 并有

$$|x^* - x_n| \leq \frac{L^n}{1 - L} |x_1 - x_0| \quad (8-12)$$

[证明] 由条件(2)易得  $\varphi(x)$  在  $[a, b]$  上连续. 令  $\psi(x) = x - \varphi(x)$ , 则  $\psi(x)$  在  $[a, b]$  上连续, 且

$$\psi(a) = a - \varphi(a) \leq 0, \quad \psi(b) = b - \varphi(b) \geq 0$$

由连续函数介值定理, 存在  $\xi \in [a, b]$ , 使得  $\psi(\xi) = 0$ , 即  $\xi = \varphi(\xi)$ , 所以方程  $x = \varphi(x)$  在  $[a, b]$  内有根.

假设方程  $x = \varphi(x)$  在  $[a, b]$  内有两个根  $x_1^* \neq x_2^*$ , 由条件(2), 有

$$|x_1^* - x_2^*| = |\varphi(x_1^*) - \varphi(x_2^*)| \leq L|x_1^* - x_2^*| < |x_1^* - x_2^*|$$

导出矛盾, 唯一性得证.

对任意  $x_0 \in [a, b]$ , 由迭代公式有

$$|x_n - x^*| = |\varphi(x_{n-1}) - \varphi(x^*)| \leq L|x_{n-1} - x^*|$$

依此类推, 即得

$$|x_n - x^*| \leq L^n |x_0 - x^*|$$

因为  $0 < L < 1$ , 所以

$$\lim_{n \rightarrow \infty} x_n = x^*$$

即对任意初值  $x_0 \in [a, b]$ , 迭代序列  $\{x_n\}$  均收敛到方程的根  $x^*$ .

类似地, 对任意正整数  $k$ , 有

$$\begin{aligned} |x_{k+1} - x_k| &= |\varphi(x_k) - \varphi(x_{k-1})| \\ &\leq L |x_k - x_{k-1}| \leq \cdots \\ &\leq L^k |x_1 - x_0| \end{aligned}$$

于是, 对任意正整数  $n, p$ , 有

$$\begin{aligned} |x_{n+p} - x_n| &\leq |x_{n+p} - x_{n+p-1}| + |x_{n+p-1} - x_{n+p-2}| + \cdots + |x_{n+1} - x_n| \\ &\leq L^{p-1} |x_{n+1} - x_n| + L^{p-2} |x_{n+1} - x_n| + \cdots + |x_{n+1} - x_n| \\ &= \frac{1 - L^p}{1 - L} |x_{n+1} - x_n| \\ &\leq \frac{1 - L^p}{1 - L} L^n |x_1 - x_0| \end{aligned} \quad (8-13)$$

令  $p \rightarrow \infty$ , 得

$$|x^* - x_n| \leq \frac{L^n}{1 - L} |x_1 - x_0|$$

定理 8.1 给出了判别不动点迭代法收敛的充分条件. 由于实际计算时, 常数  $L$  十分难求, 而我们所讨论的函数通常都是可导函数, 故实用的收敛条件是用导数的界得到的.

**定理 8.2** 设函数  $\varphi(x)$  在区间  $[a, b]$  上满足条件:

- (1) 对任意  $x \in [a, b]$ , 有  $\varphi(x) \in [a, b]$ ;
- (2) 存在常数  $0 < L < 1$ , 使得对任意  $x \in [a, b]$ , 都有  $|\varphi'(x)| \leq L$ .

则方程 (8-8) 在  $[a, b]$  上有唯一的根  $x^*$ , 且对任意初值  $x_0 \in [a, b]$ , 迭代序列

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2, \cdots)$$

均收敛于  $x^*$ , 并有

$$|x^* - x_n| \leq \frac{L^n}{1 - L} |x_1 - x_0|$$

[证明] 设  $x, y$  为  $[a, b]$  上任意两点, 由微分中值定理, 在  $x, y$  之间至少存在一点  $\xi$ , 使得

$$\varphi(x) - \varphi(y) = \varphi'(\xi)(x - y)$$

于是

$$\begin{aligned} |\varphi(x) - \varphi(y)| &= |\varphi'(\xi)(x - y)| \\ &\leq L |x - y| \end{aligned}$$

即  $\varphi(x)$  满足定理 8.1 的条件 (2), 故结论成立.

用定理 8.2 判别不动点迭代法的收敛性比定理 8.1 方便. 如例 2 中, 取



$\varphi_1(x) = \frac{20}{x^2 + 10}$ , 则有

$$\varphi'_1(x) = \frac{-40x}{(x^2 + 10)^2}$$

对  $\forall x \in [1, 2]$ , 有

$$|\varphi'_1(x)| = \frac{40x}{(x^2 + 10)^2} \leq \frac{80}{11^2} < 1$$

所以迭代收敛.

对  $\varphi_2(x) = 2 - \frac{x^3}{10}$ , 因为

$$\varphi'_2(x) = -\frac{3}{10}x^2$$

其绝对值在区间  $[1, 2]$  上的最大值超过 1, 故  $\varphi_2(x)$  在  $[1, 2]$  上不满足定理 8.2 的条件, 不能用定理 8.2 判定收敛性. 容易看出, 在小区间  $[1.5, 1.7]$  上, 有  $1.5 < \varphi_2(x) < 1.7$ , 且

$$|\varphi'_2(x)| = \frac{3}{10}x^2 \leq 0.867$$

因此  $\varphi_2(x)$  在小区间  $[1.5, 1.7]$  上满足定理 8.2 的条件, 故取  $x_0 = 1.5$ , 按迭代函数  $\varphi_2(x)$  所导出的迭代序列是收敛的.

误差估计式 (8-12) 表明, 常数  $L$  越小, 不动点迭代法收敛越快, 因而用简单迭代法求方程 (8-1) 的根时, 构造迭代函数  $\varphi(x)$  的原则是使  $|\varphi'(x)|$  在有根区间  $[a, b]$  上有尽可能小的上界. 如例 2 中迭代函数  $\varphi_1(x)$  所产生的迭代序列收敛比较快.

按式 (8-13), 当  $0 < L \leq 0.5$  时, 有

$$|x^* - x_n| \leq \frac{L}{1-L} |x_n - x_{n-1}| \leq |x_n - x_{n-1}|$$

故简单迭代法常以  $|x_n - x_{n-1}| < \varepsilon$  作为停步准则.

一般说来, 构造迭代函数使其在预先指定的较大区间上满足定理 8.2 的条件是比较困难的, 但若取  $x_0$  充分接近根  $x^*$ , 则收敛性的讨论可在根的邻近进行.

**定理 8.3** 如果函数  $\varphi(x)$  在  $x^*$  的一邻域  $O(x^*, \delta^*)$  内连续可微,  $x^*$  为方程 (8-8) 的根, 且  $|\varphi'(x^*)| < 1$ , 则存在正数  $\delta, \delta \leq \delta^*$ , 使得对任意  $x_0 \in [x^* - \delta, x^* + \delta]$ , 迭代序列

$$x_{n+1} = \varphi(x_n) \quad (n = 0, 1, 2, \dots)$$

收敛于  $x^*$ .

[证明] 因为  $\varphi'(x)$  在  $O(x^*, \delta^*)$  内连续, 且  $|\varphi'(x^*)| < 1$ , 故存在正数

$L < 1, \delta < \delta^*$ , 使得对  $\forall x \in [x^* - \delta, x^* + \delta]$ , 有

$$|\varphi'(x)| \leq L < 1$$

另一方面, 由  $\varphi(x^*) = x^*$ , 又有

$$|\varphi(x) - x^*| = |\varphi(x) - \varphi(x^*)| \leq L|x - x^*| < \delta$$

即  $\varphi(x) \in [x^* - \delta, x^* + \delta]$ . 根据定理 8.2, 迭代序列  $x_{n+1} = \varphi(x_n)$  收敛于  $x^*$ .

定理 8.3 给出了初值在根邻近时, 不动点迭代法收敛的充分条件, 称为局部收敛性定理. 这一定理表明, 只要构造迭代函数, 使其在根的邻近满足导数的绝对值有小于 1 的上界, 即可保证简单迭代法收敛. 事实上, 在用简单迭代法求解方程(8-1)时, 常常先用对分区间法求得较好的初值, 然后再进行迭代.

### 8.2.3 Steffensen 方法——简单迭代法的加速

#### (一) 收敛速度

**定义 8.1** 设序列  $\{x_n\}$  收敛于  $x^*$ . 如果存在正数  $r$  和  $a$ , 使得

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|^r} = a$$

成立, 则称序列  $\{x_n\}$  是  $r$  阶收敛的, 或称  $\{x_n\}$  的收敛阶为  $r$ .

显然, 收敛阶  $r$  的大小刻画了序列  $\{x_n\}$  的收敛速度,  $r$  越大, 收敛越快. 特别地, 若  $r=1$ , 称序列是线性收敛的; 若  $r>1$ , 称为超线性收敛;  $r=2$  为平方收敛.

关于迭代法的收敛速度有以下结果.

**定理 8.4** 设迭代函数  $\varphi(x)$  在  $x^*$  邻近有  $r$  阶连续导数 ( $r \geq 2$ ), 且  $x^* = \varphi(x^*)$ ,  $\varphi^{(k)}(x^*) = 0$  ( $k=1, \dots, r-1$ ),  $\varphi^{(r)}(x^*) \neq 0$ , 则迭代公式(8-9)所产生的迭代序列  $\{x_n\}$  是  $r$  阶收敛的. 若  $x^* = \varphi(x^*)$ ,  $0 < |\varphi'(x^*)| < 1$ , 则迭代序列  $\{x_n\}$  是线性收敛的.

[证明] 显然,  $\varphi(x)$  满足定理 8.3 的条件, 故存在  $\delta > 0$ , 使得对任意  $x_0 \in [x^* - \delta, x^* + \delta]$ , 迭代序列  $x_{n+1} = \varphi(x_n)$  收敛于  $x^*$ , 且  $x_n \in [x^* - \delta, x^* + \delta]$  ( $n=0, 1, 2, \dots$ ).

由  $\varphi(x)$  在  $x^*$  处的 Taylor 公式得

$$\begin{aligned} x_{n+1} = \varphi(x_n) &= \varphi(x^*) + \varphi'(x^*)(x_n - x^*) + \dots + \frac{\varphi^{(r-1)}(x^*)}{(r-1)!}(x_n - x^*)^{r-1} \\ &\quad + \frac{\varphi^{(r)}(\xi)}{r!}(x_n - x^*)^r \\ &= x^* + \frac{\varphi^{(r)}(\xi)}{r!}(x_n - x^*)^r, \quad \xi \text{ 在 } x^* \text{ 与 } x_n \text{ 之间} \end{aligned}$$

于是

$$\frac{x_{n+1} - x^*}{(x_n - x^*)^r} = \frac{\varphi^{(r)}(\xi)}{r!}$$

令  $n \rightarrow \infty$ , 因为  $x_n \rightarrow x^*$ , 故  $\xi \rightarrow x^*$ , 从而有

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|^r} = \frac{|\varphi^{(r)}(x^*)|}{r!} \neq 0$$

所以, 迭代序列  $\{x_n\}$  是  $r$  阶收敛的.

类似地, 当  $0 < |\varphi'(x^*)| < 1$  时, 由定理 8.3, 迭代序列  $\{x_n\}$  收敛于  $x^*$ , 且由微分中值定理

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|} = \lim_{n \rightarrow \infty} \frac{|\varphi(x_n) - \varphi(x^*)|}{|x_n - x^*|} = |\varphi'(x^*)| \neq 0$$

故  $\{x_n\}$  线性收敛于  $x^*$ .

按定理 8.4, 若  $\varphi'(x^*) \neq 0$ , 简单迭代法是线性收敛的. 为加快收敛速度, 可用 Aitken 加速, 由此导出 Steffensen 方法.

## (二) Steffensen 方法

第四章中已提到, 如果序列  $\{x_n\}$  线性收敛于  $x^*$ , 则按下式产生的新序列

$$\bar{x}_n = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n} \quad (8-14)$$

(称为序列  $\{x_n\}$  的 Aitken 序列) 比序列  $\{x_n\}$  更快地收敛于  $x^*$ . 将 Aitken 加速法与简单迭代格式(8-9)相结合, 得到 Steffensen 方法, 其迭代公式为

$$\begin{cases} y_n = \varphi(x_n) \\ z_n = \varphi(y_n) \\ x_{n+1} = x_n - \frac{(y_n - x_n)^2}{z_n - 2y_n + x_n} \end{cases} \quad (n = 0, 1, 2, \dots) \quad (8-15)$$

Steffensen 方程也是一种不动点迭代, 其迭代公式为

$$x_{n+1} = x_n - \frac{[\varphi(x_n) - x_n]^2}{\varphi(\varphi(x_n)) - 2\varphi(x_n) + x_n}$$

可以证明, 当  $\varphi'(x^*) \neq 1$  时, Steffensen 方法是至少二阶局部收敛的.

**例 3** 用 Steffensen 方法求方程  $f(x) = x^3 + 10x - 20 = 0$  的根, 取  $x_0 = 1.5$ ,  $\varepsilon = 10^{-6}$ .

[解] 取迭代函数为  $\varphi(x) = \frac{20}{x^2 + 10}$ , 按式(8-15)计算, 结果见下表.

| $n$ | $x_n$       | $y_n$       | $z_n$       |
|-----|-------------|-------------|-------------|
| 0   | 1.5         | 1.632 653 1 | 1.579 085 8 |
| 1   | 1.594 494 7 | 1.594 589 5 | 1.594 551 0 |
| 2   | 1.594 562 1 | 1.594 562 1 | 1.594 562 1 |

迭代 2 次, 就得到满足精度要求的解

$$x^* \approx x_2 = 1.594\ 562\ 1$$

可见 Steffensen 方法收敛很快.

## § 8.3 牛顿(Newton)法与弦截法

### 8.3.1 Newton 法

将非线性方程线性化, 以线性方程的解逐步逼近非线性方程的解, 这就是 Newton 法的基本思想.

设  $f(x)$  在其零点  $x^*$  邻近一阶连续可微, 且  $f'(x) \neq 0$ , 当  $x_0$  充分接近  $x^*$  时, 由 Taylor 公式有

$$f(x^*) \approx f(x_0) + f'(x_0)(x^* - x_0)$$

以方程

$$f(x_0) + f'(x_0)(x - x_0) = 0$$

近似方程(8-1), 其解

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

可作为方程(8-1)的近似解. 重复以上过程, 得迭代公式

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (n = 0, 1, 2, \dots) \quad (8-16)$$

按式(8-16)求方程(8-1)的近似解称为 Newton 法. Newton 法也是一种不动点迭代, 其迭代函数为

$$\varphi(x) = x - \frac{f(x)}{f'(x)}$$

如图 8-3 所示, 从几何上看,  $y = f(x_0) + f'(x_0)(x - x_0)$  为曲线  $y = f(x)$  过点  $(x_0, f(x_0))$  的切线,  $x_1$  为切线与  $x$  轴的交点,  $x_2$  则是曲线上点  $(x_1, f(x_1))$  处的切线与  $x$  轴的交点. 如此继续下去,  $x_{n+1}$  为曲线上点  $(x_n, f(x_n))$  处的切线与  $x$  轴的交点. 因此 Newton 法是以曲线的切线与  $x$  轴的交点作为曲线与  $x$  轴的交点的近似, 故 Newton 法又称为切线法.

#### 算法 8.2

(1) 输入  $f(x)$ , 初始值  $x_0$ , 误差限  $\varepsilon$ , 最大允许迭代次数  $N$ .

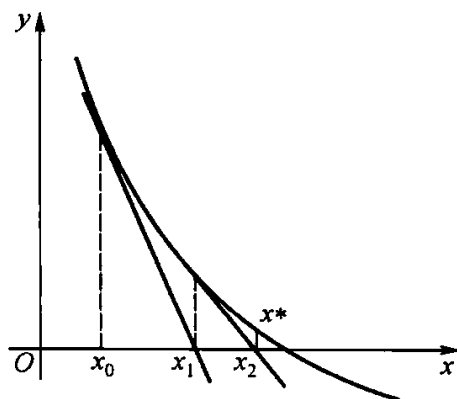


图 8-3

(2) 置  $n = 1$ .

(3) 计算

$$x = x_0 - \frac{f(x_0)}{f'(x_0)}$$

(4) 若  $|x - x_0| < \varepsilon$  或  $\frac{|x - x_0|}{|x|} < \varepsilon$ , 则输出  $x^* = x$ , 停机; 否则转 5.

(5) 若  $n < N$ , 则置  $n + 1 \Rightarrow n, x \Rightarrow x_0$ , 转 3; 否则输出失败信息, 停机.

说明: 在第 4 步中,  $\frac{|x - x_0|}{|x|} < \varepsilon$  为相对误差检验, 即相对误差达到精度也

可作为停机准则. 有时, Newton 法还以  $|f(x)| < \varepsilon$  作为停机准则.

**例 4** 用 Newton 法求方程  $f(x) = x^3 + 10x - 20 = 0$  的根, 取  $x_0 = 1.5$ .

[解] 因为  $f'(x) = 3x^2 + 10$ , 故 Newton 迭代公式为

$$x_{n+1} = x_n - \frac{x_n^3 + 10x_n - 20}{3x_n^2 + 10}$$

代入初值得

$$x_1 = 1.597\ 014\ 9, \quad x_2 = 1.594\ 563\ 7$$

$$x_3 = 1.594\ 562\ 1, \quad x_4 = 1.594\ 562\ 1$$

迭代 3 次所得近似解就准确到 8 位有效数字, 可见 Newton 法收敛很快.

**定理 8.5** 设函数  $f(x)$  在其零点  $x^*$  邻近二阶连续可微, 且  $f'(x^*) \neq 0$ , 则存在  $\delta > 0$ , 使得对任意  $x_0 \in [x^* - \delta, x^* + \delta]$ , Newton 法所产生的序列  $\{x_n\}$  至少二阶收敛于  $x^*$ .

[证明] 按式 (8-16), Newton 法的迭代函数为

$$\varphi(x) = x - \frac{f(x)}{f'(x)}$$

于是有

$$\varphi'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}$$

由已知  $f''(x)$  在  $x^*$  邻近连续, 因而  $\varphi'(x)$  在  $x^*$  邻近连续, 且

$$\varphi'(x^*) = \frac{f(x^*)f''(x^*)}{[f'(x^*)]^2} = 0$$

根据定理 8.4, Newton 法所产生的序列  $\{x_n\}$  至少二阶收敛于  $x^*$ .

定理 8.5 表明, 当初始值  $x_0$  充分接近方程 (8-1) 的单根  $x^*$  时, Newton 法的收敛速度较快, 但计算实践表明, 当初始值不够好时, Newton 法可能发散. 一般可由问题的实际背景来预测或由对分区间法求得较好的初始值. 还要指出的是, 当  $x^*$  为方程 (8-1) 的  $r$  重根时, Newton 法是线性收敛的, 且

$$\lim_{n \rightarrow \infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|} = \frac{r-1}{r}$$

显然,  $r$  越大收敛越慢. 为了改善其收敛性, 通常用以下两种方法:

(1) 如果重根  $x^*$  的重数  $r$  已知 ( $r \geq 2$ ), 则用迭代公式

$$x_{n+1} = x_n - r \frac{f(x_n)}{f'(x_n)} \quad (8-17)$$

(2) 如果  $x^*$  的重数未知, 则令  $\mu(x) = \frac{f(x)}{f'(x)}$ , 对  $\mu(x)$  用 Newton 法, 得迭代公式

$$x_{n+1} = x_n - \frac{f(x_n)f'(x_n)}{[f'(x_n)]^2 - f(x_n)f''(x_n)} \quad (8-18)$$

容易证明, 这两种方法都是至少二阶收敛的, 作为习题, 请读者自行证明.

**例 5** 取  $x_0 = 0.5$ , 求方程  $\sin x - x = 0$  的重根, 要求误差小于  $10^{-8}$ .

[解] 设  $f(x) = \sin x - x$ , 显然  $f(0) = 0$ , 且  $f'(0) = f''(0) = f'''(0) = 0$ ,  $f^{(4)}(0) = -1 \neq 0$ , 故是方程的 3 重根. 分别用 Newton 法、公式 (8-17) 和公式 (8-18) 计算, 结果列于下表:

| $n$      | Newton 法 $x_n$       | 公式 (8-17) $x_n$            | 公式 (8-18) $x_n$            |
|----------|----------------------|----------------------------|----------------------------|
| 0        | 0.5                  | 0.5                        | 0.5                        |
| 1        | 0.331 931 939 489 11 | -0.004 204 181 532 67      | 0.008 274 056 887 75       |
| 2        | 0.220 880 000 703 62 | 2.476 920 685 917 650e-009 | 3.776 252 810 970 548e-008 |
| 3        | 0.147 133 388 298 26 |                            | 1.502 191 775 200 419e-008 |
| 4        | 0.098 053 507 283 12 |                            | 1.502 191 775 200 419e-008 |
| $\vdots$ | $\vdots$             |                            |                            |
| 43       | 0.000 000 019 811 53 |                            |                            |
| 44       | 0.000 000 019 811 53 |                            |                            |

由计算结果可见, Newton 法对于重根的求解确实收敛缓慢, 而公式 (8-17) 和公式 (8-18) 的收敛速度要快得多.

Newton 法对于单根的求解具有收敛快、稳定性好、精度高等优点, 它是求解非线性方程的有效方法之一. 但它每次迭代需要计算函数值与导数值, 计算量较大. 而且当导数计算有困难时, Newton 法将无法进行.

### 8.3.2 弦截法

为避免计算导数,可用差商近似导数. 在 Newton 迭代公式(8-16)中,用差商  $\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$  代替导数  $f'(x_n)$ , 得到迭代公式

$$x_{n+1} = x_n - \frac{f(x_n)}{f(x_n) - f(x_{n-1})}(x_n - x_{n-1}) \quad (n = 1, 2, \dots) \quad (8-19)$$

按式(8-19)计算方程(8-1)的近似解称为弦截法.

如图 8-4 所示,过曲线上两点  $(x_{n-1}, f(x_{n-1}))$ ,  $(x_n, f(x_n))$  的直线为

$$y - f(x_n) = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}(x - x_n)$$

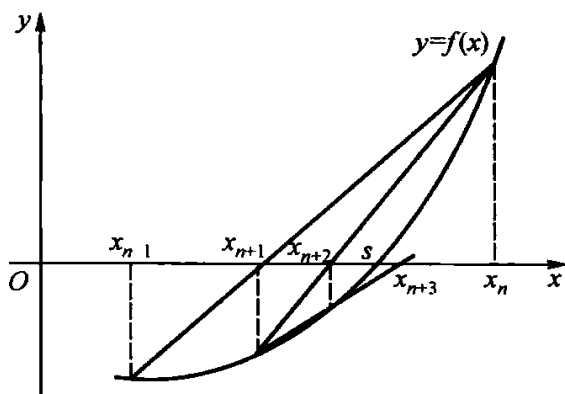


图 8-4

它与  $x$  轴的交点为

$$x = x_n - \frac{f(x_n)}{f(x_n) - f(x_{n-1})}(x_n - x_{n-1}) = x_{n+1}$$

于是,从几何上看,弦截法是以曲线上两点的割线与  $x$  轴的交点作为曲线与  $x$  轴的交点的近似,故弦截法又称为割线法. 另一方面,割线的函数表达式恰为函数  $f(x)$  的以  $x_{n-1}, x_n$  为节点的线性插值多项式,按式(8-19)求方程(8-1)的近似解等价于用  $f(x)$  的线性插值多项式的零点近似函数  $f(x)$  的零点,因此弦截法还称为线性插值法.

弦截法是两步法,用弦截法求非线性方程的解,必须给出两个初始值  $x_0, x_1$ , 通常取根所在区间的端点即可. 一般地,弦截法仍以  $|x_n - x_{n-1}| < \varepsilon$  为停步准则,也可以  $|f(x_n)| < \varepsilon$  为停步准则.

#### 算法 8.3

- (1) 输入  $f(x)$ , 初始值  $x_0, x_1$ , 误差限  $\varepsilon$ , 最大允许迭代次数  $N$ .
- (2) 置  $f_0 = f(x_0)$ ,  $f_1 = f(x_1)$ ,  $n = 1$ .

(3) 计算

$$x = x_1 - \frac{f_1}{f_1 - f_0}(x_1 - x_0)$$

(4) 若  $|x - x_1| < \varepsilon$ , 输出  $x^* = x$ , 停机; 否则转 5.

(5) 若  $n < N$ , 置  $n+1 \Rightarrow n, x_1 \Rightarrow x_0, x \Rightarrow x_1, f_1 \Rightarrow f_0, f(x) \Rightarrow f_1$ , 转 3; 否则, 输出失败信息, 停机.

弦截法不是不动点迭代, 前面的收敛性讨论不适用于它. 关于弦截法的收敛性有以下结果.

**定理 8.6** 设  $f(x)$  在  $x^*$  邻近二阶连续可微, 且  $f(x^*) = 0, f'(x^*) \neq 0$ , 则存在  $\delta > 0$ , 当  $x_0, x_1 \in [x^* - \delta, x^* + \delta]$  时, 由弦截法产生的序列  $\{x_n\}$  收敛于  $x^*$ , 且收敛阶至少为 1.618.

证明参看参考文献[9].

**例 6** 用弦截法求方程  $f(x) = x^3 + 10x - 20 = 0$  的根.

[解] 因为  $f(1.5) < 0, f(2) > 0, [1.5, 2]$  为有根区间. 取  $x_0 = 1.5, x_1 = 2$ , 按算法 8.3 计算, 结果见表 8-4.

表 8-4

| $n$ | $x_n$       | $f(x_n)$              |
|-----|-------------|-----------------------|
| 0   | 1.5         | -1.625                |
| 1   | 2           | 8                     |
| 2   | 1.584 415 6 | -0.178 370 2          |
| 3   | 1.593 479 5 | -0.019 078 6          |
| 4   | 1.594 565 1 | 0.000 052 56          |
| 5   | 1.594 562 1 | $-2.2 \times 10^{-7}$ |

计算结果表明, 迭代 5 次所得近似解精确到 8 位有效数字. 它的收敛速度虽低于 Newton 法, 但比例 2 所用的简单迭代法快.

## § 8.4 抛物线 (Müller) 法

如果考虑用  $f(x)$  的二次插值多项式的零点来近似  $f(x)$  的零点, 就导出抛物线法.

设已知方程 (8-1) 的根的三个近似值  $x_{n-2}, x_{n-1}, x_n$ , 以这三点为节点的  $f(x)$  的二次插值多项式为



$$P_2(x) = f(x_n) + f[x_n, x_{n-1}](x - x_n) + f[x_n, x_{n-1}, x_{n-2}](x - x_n)(x - x_{n-1}) \quad (8-20)$$

为简便起见,令

$$\begin{cases} a_n = f[x_n, x_{n-1}, x_{n-2}] \\ b_n = f[x_n, x_{n-1}] + f[x_n, x_{n-1}, x_{n-2}](x_n - x_{n-1}) \\ c_n = f(x_n) \end{cases} \quad (8-21)$$

则式(8-20)可改写为

$$P_2(x) = a_n(x - x_n)^2 + b_n(x - x_n) + c_n \quad (8-22)$$

其零点为

$$x = x_n + \frac{-b_n \pm \sqrt{b_n^2 - 4a_n c_n}}{2a_n} = x_n - \frac{2c_n}{b_n \pm \sqrt{b_n^2 - 4a_n c_n}} \quad (8-23)$$

按式(8-23),  $f(x)$  的二次插值多项式  $P_2(x)$  有两个零点, 取哪个作为新的近似根? 考虑到  $x_n$  已是方程(8-1)的近似根, 新的近似根自然应在  $x_n$  的邻近, 故选取新近似根的原则是使得  $|x - x_n|$  较小, 于是有

$$x_{n+1} = x_n - \frac{2c_n \operatorname{sgn}(b_n)}{|b_n| + \sqrt{b_n^2 - 4a_n c_n}} \quad (8-24)$$

按式(8-24)计算方程(8-1)的近似根称为抛物线法, 也称 Müller 方法, 二次插值法.

如图 8-5 所示,  $y = P_2(x)$  是过曲线上三点  $(x_i, f(x_i))$  ( $i = n-2, n-1, n$ ) 的抛物线, 故抛物线法的几何意义是以过曲线上三点的抛物线与  $x$  轴的交点作为曲线与  $x$  轴交点的近似.

抛物线法的计算步骤为:

(1) 给定精度  $\varepsilon > 0$ , 初始值  $x_0, x_1, x_2$ , 计算  $f_0, f_1, f_2$  (要求  $f_0, f_1, f_2$  不等).

(2) 计算

$$\begin{aligned} a &= f[x_2, x_1, x_0] \\ b &= f[x_2, x_1] + a(x_2 - x_1) \\ c_2 &= f_2 \\ x_3 &= x_2 - \frac{2c \operatorname{sgn}(b)}{|b| + \sqrt{b^2 - 4ac}} \\ f_3 &= f(x_3) \end{aligned}$$

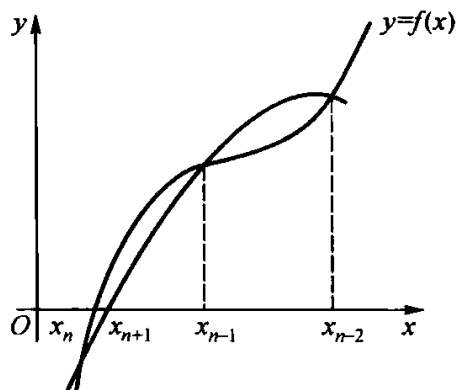


图 8-5

(3) 若  $|f_3| < \varepsilon$ , 则  $x^* = x$ ; 否则  $x_{i+1} \Rightarrow x_i, f_{i+1} \Rightarrow f_i (i=0, 1, 2)$ , 转 2.

**例 7** 用抛物线法求方程  $f(x) = x^3 + 10x - 20 = 0$  的根.

[解] 取  $x_0 = 1.5, x_1 = 1.75, x_2 = 2$ , 则  $f_0 = -1.625, f_1 = 2.859\ 375, f_2 = 8$ .  
按式(8-21)和式(8-24)计算, 得

$$a = f[2, 1.75, 1.5] = 5.25$$

$$b = f[2, 1.75] + a(2 - 1.75) = 21.875$$

$$c = 8$$

$$x_3 = 2 - \frac{2 \times 8}{21.875 + \sqrt{(21.875)^2 - 4 \times 5.25 \times 8}} = 1.594\ 900$$

重复上述过程, 迭代数次可得到满足精度要求的解.

可以证明, 如果  $f(x)$  在其零点  $x^*$  邻近三阶连续可微, 且初值  $x_0, x_1, x_2$  充分接近  $x^*$ , 则抛物线法是收敛的. 特别地, 若  $x^*$  是方程(8-1)的单根, 收敛阶为 1.84. 另一方面, 在收敛性的证明中虽然要求初始值充分接近根  $x^*$ , 但实际计算表明, 抛物线法对初值要求并不苛刻, 在初值不太好的情形下常常也能收敛. 它的缺点是程序较复杂, 并且在计算实根的过程中, 也常常需要采用复数运算, 增加了工作量. 因此, 抛物线法适用于当初值不太好时求方程(8-1)的复根的情况.

## § 8.5 非线性方程组的解法

设有非线性方程组

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0 \\ f_2(x_1, x_2, \dots, x_n) = 0 \\ \dots\dots\dots \\ f_n(x_1, x_2, \dots, x_n) = 0 \end{cases} \quad (8-25)$$

若记  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ ,  $\mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x}))^T$ , 则方程组(8-23)可简记成向量形式

$$\mathbf{F}(\mathbf{x}) = \mathbf{0} \quad (8-26)$$

解非线性方程组的方法有许多种, 其中一类是线性化的方法, 它是将方程组中的每个方程线性化得到一个线性方程组, 由此构造迭代格式, 求得方程组(8-25)的近似解. 这类方法中常用的一种是 Newton 法; 另一类方法是将解非线性方程组问题化成优化问题, 然后以最优化方法求解. 最速下降法是最基本的方法. 本节简单介绍这两种方法.

### 8.5.1 解非线性方程组的 Newton 法

这是第三节中 Newton 法的直接推广. 设给定方程组 (8-26) 的第  $k$  次近似解  $\mathbf{x}^{(k)} = (x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)})^T$ , 以函数  $F(\mathbf{x})$  在  $\mathbf{x}^{(k)}$  处的一阶 Taylor 多项式近似函数, 得到线性方程组

$$F(\mathbf{x}^{(k)}) + F'(\mathbf{x}^{(k)}) \Delta \mathbf{x}^{(k)} = \mathbf{0}$$

即

$$F'(\mathbf{x}^{(k)}) \Delta \mathbf{x}^{(k)} = -F(\mathbf{x}^{(k)}) \quad (8-27)$$

其中

$$F'(\mathbf{x}^{(k)}) = \left[ \begin{array}{cccc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{array} \right]_{x=\mathbf{x}^{(k)}}$$

称为向量函数  $F(\mathbf{x})$  的 Jacobi 矩阵,  $\Delta \mathbf{x}^{(k)} = \mathbf{x} - \mathbf{x}^{(k)} = (\Delta x_1^{(k)}, \dots, \Delta x_n^{(k)})^T$ . 线性方程组 (8-27) 称为 Newton 方程组. 如果  $F(\mathbf{x})$  的 Jacobi 矩阵在  $\mathbf{x}^{(k)}$  处非奇异, 方程组 (8-27) 有唯一解  $\Delta \mathbf{x}^{(k)}$ , 则  $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}^{(k)}$  为方程组 (8-26) 的第  $k+1$  次近似解. 按上述过程求方程组 (8-26) 的近似解称为 Newton 方法. 可以证明, 当初始值  $\mathbf{x}^{(0)}$  与解充分接近时, Newton 法是平方收敛的. 计算过程中的停步准则一般采用以下两种:

$$(1) \quad \|F(\mathbf{x}^{(k+1)})\|_{\infty} = \max_{1 \leq i \leq n} |f_i(x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_n^{(k+1)})| < \varepsilon;$$

$$(2) \quad \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty} = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon. \text{ 当 } \|\mathbf{x}^{(k)}\| \text{ 较大时, 也可}$$

用两次迭代的相对误差充分小作为停步准则.

**例 8** 求方程组

$$\begin{cases} 4x_1^2 + 3x_2^2 = 1 \\ x_1^3 - 8x_2^3 = 1 \end{cases}$$

邻近  $(0.25, -0.5)$  的解.

[解] 因为

$$F(\mathbf{x}) = \begin{bmatrix} 4x_1^2 + 3x_2^2 - 1 \\ x_1^3 - 8x_2^3 - 1 \end{bmatrix}, \quad F'(\mathbf{x}) = \begin{bmatrix} 8x_1 & 6x_2 \\ 3x_1^2 & -24x_2^2 \end{bmatrix}$$

$\mathbf{x}^{(0)} = (0.25, -0.5)^T$ , 故 Newton 方程组为

$$F'(\mathbf{x}^{(0)}) \Delta \mathbf{x}^{(0)} = \begin{bmatrix} 2 & -3 \\ 0.1875 & -6 \end{bmatrix} \begin{bmatrix} \Delta x_1^{(0)} \\ \Delta x_2^{(0)} \end{bmatrix} = - \begin{bmatrix} 0 \\ 0.015625 \end{bmatrix}$$

其解为  $\Delta x_1^{(0)} = 0.004\,098\,36$ ,  $\Delta x_2^{(0)} = 0.002\,732\,24$ , 所以

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \Delta \mathbf{x}^{(0)} = (0.254\ 098\ 36, -0.497\ 267\ 76)^T$$

继续迭代下去,可得近似解为  $\mathbf{x}^* \approx (0.254\ 078\ 6, -0.497\ 251\ 2)^T$ .

Newton 法具有收敛快,形式简单等优点,但它对初值的要求较高.另外,Newton 法每迭代一次,需计算  $n + n^2$  个函数值及导数值,求解一个  $n$  阶线性方程组,计算量较大.

### 8.5.2 最速下降法

设有非线性方程组

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0 \\ f_2(x_1, x_2, \dots, x_n) = 0 \\ \dots\dots\dots \\ f_m(x_1, x_2, \dots, x_n) = 0 \end{cases} \quad (8-28)$$

构造模函数

$$\Phi(\mathbf{x}) = \sum_{i=1}^m [f_i(\mathbf{x})]^2 \quad (8-29)$$

其中  $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ , 显然, 方程组 (8-28) 的解是模函数  $\Phi(\mathbf{x})$  的零点, 反之亦然. 因此, 求解非线性方程组 (8-28) 可以转化为求解下述最优化问题

$$\min_{\mathbf{x} \in \mathbb{R}^n} \Phi(\mathbf{x}) \quad (8-30)$$

这是一个无约束最优化问题, 有许多求解方法, 最基本的一种方法是最速下降法.

最速下降法的基本思想在第三章 § 3.6 中已介绍过. 为直观起见, 下面着重讨论二维的情形. 设

$$\Phi(x_1, x_2) = \sum_{i=1}^m [f_i(x_1, x_2)]^2 \quad (8-31)$$

记  $g_1(x_1, x_2) = \frac{\partial \Phi}{\partial x_1}$ ,  $g_2(x_1, x_2) = \frac{\partial \Phi}{\partial x_2}$ , 负梯度方向  $-\mathbf{G}(x_1, x_2) = -(g_1(x_1, x_2), g_2(x_1, x_2))$  就是点  $(x_1, x_2)$  处函数  $\Phi(x_1, x_2)$  的最速下降方向. 显然, 若  $\mathbf{G} \neq \mathbf{0}$ , 则沿  $-\mathbf{G}$  方向  $\Phi(x_1, x_2)$  的值必定是减少的. 因此, 从初始值  $(x_1^{(0)}, x_2^{(0)})$  出发, 沿方向  $-\mathbf{G}(x_1^{(0)}, x_2^{(0)})$  一定可以找到一点  $(x_1^{(1)}, x_2^{(1)})$ , 使得  $\Phi(x_1^{(1)}, x_2^{(1)}) < \Phi(x_1^{(0)}, x_2^{(0)})$ , 然后再从  $(x_1^{(1)}, x_2^{(1)})$  出发, 沿  $-\mathbf{G}(x_1^{(1)}, x_2^{(1)})$  方向找点  $(x_1^{(2)}, x_2^{(2)})$ , 使得  $\Phi(x_1^{(2)}, x_2^{(2)}) < \Phi(x_1^{(1)}, x_2^{(1)})$ . 如此继续下去, 就可能逼近到  $\Phi(x_1, x_2)$  的最小点  $(x_1^*, x_2^*)$ .

从点  $(x_1^{(k)}, x_2^{(k)})$  出发, 沿方向  $-\mathbf{G}(x_1^{(k)}, x_2^{(k)})$ , 如何求出新近似解  $(x_1^{(k+1)}, x_2^{(k+1)})$ ? 自然地, 我们希望  $(x_1^{(k+1)}, x_2^{(k+1)})$  是使得  $\Phi(x_1, x_2)$  的值下降最多的

点,即

$$\begin{aligned}\Phi(x_1^{(k+1)}, x_2^{(k+1)}) &= \min_{\lambda > 0} \Phi(x_1^{(k)} - \lambda g_1(x_1^{(k)}, x_2^{(k)}), x_2^{(k)} - \lambda g_2(x_1^{(k)}, x_2^{(k)})) \\ &= \min_{\lambda > 0} \varphi(\lambda)\end{aligned}\quad (8-32)$$

这是一个一维极值问题,称为一维搜索问题,射线方向  $-G(x_1^{(k)}, x_2^{(k)})$  称为搜索方向. 一维搜索的方法很多,如果  $\varphi(\lambda)$  的形式简单,可用一元函数求极值的方法求解;否则,可用对分搜索, Fibonacci 搜索、黄金分割、插值等方法求解. 求出问题(8-32)的解  $\lambda$  后,代入公式

$$x_1^{(k+1)} = x_1^{(k)} - \lambda g_1(x_1^{(k)}, x_2^{(k)}), x_2^{(k+1)} = x_2^{(k)} - \lambda g_2(x_1^{(k)}, x_2^{(k)})$$

即得第  $k+1$  次近似解. 按上述过程求问题(8-31)的解称为最速下降法,这一过程可直接推广到  $n$  维的情形.

如图 8-6 所示,  $u = \Phi(x_1, x_2)$  是一空间曲面,它与  $x_1x_2$  面的切点即为解点  $(x_1^*, x_2^*)$ . 图 8-6(b) 是曲面  $u = \Phi(x_1, x_2)$  的等高线图,同一条等高线上的  $\Phi$  值是相同的. 在解点  $(x_1^*, x_2^*)$  邻近,等高线是以  $(x_1^*, x_2^*)$  为内点的封闭曲线,且相应的  $\Phi$  值由外往里递减. 负梯度方向  $-G(x_1^{(k)}, x_2^{(k)})$  是等高线在点  $(x_1^{(k)}, x_2^{(k)})$  处的内法线方向. 从  $(x_1^{(k)}, x_2^{(k)})$  出发,沿方向为  $-G(x_1^{(k)}, x_2^{(k)})$  的射线行进到等高线与射线相切时,切点即为  $(x_1^{(k+1)}, x_2^{(k+1)})$ .

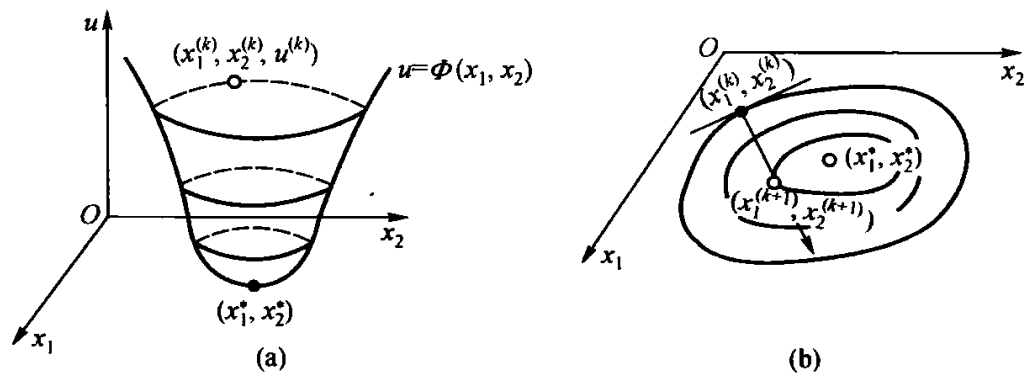


图 8-6

最速下降法的计算步骤为:

(1) 选取初始点  $x^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)})$ , 置  $k=0$ .

(2) 计算  $\Phi(x)$  在  $x^{(k)}$  处的梯度值

$$G_k = G(x^{(k)}) = \left( \frac{\partial \Phi}{\partial x_1}, \dots, \frac{\partial \Phi}{\partial x_n} \right)_{x=x^{(k)}}$$

(3) 若  $\|G_k\| < \varepsilon$ , 得近似解  $x^* = x^{(k)}$ ; 否则, 求解一维搜索问题  $\min_{\lambda > 0} \Phi(x^{(k)} - \lambda G_k)$ , 得解  $\lambda$ .

(4) 置  $x^{(k)} - \lambda G_k \Rightarrow x^{(k+1)}$ ,  $k+1 \Rightarrow k$ , 转 2.

最速下降法计算简单,收敛性好,但收敛速度较慢,它仅具有线性收敛速度.特别当  $\Phi(\mathbf{x})$  的等高线成狭长的谷时(见图 8-7),最速下降法收敛极慢.因此常常将它与 Newton 法联用,先用最速下降法求得较好的近似值,然后用 Newton 法继续求方程组(8-28)的解.

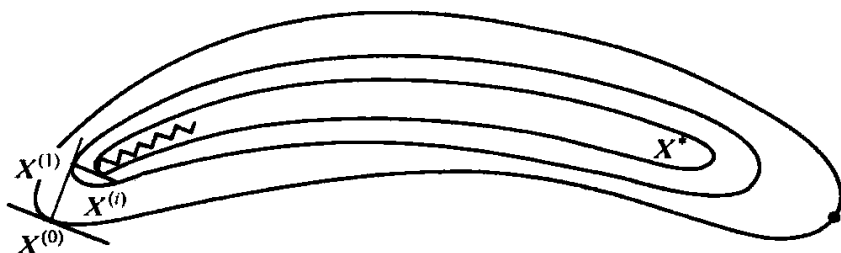


图 8-7

需要说明的是,最优化方法在实际使用时常常不作精确的一维搜索,而只求  $\mathbf{x}^{(k+1)}$  满足某些条件,如使  $\Phi(\mathbf{x})$  值下降且位移  $\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$  不太小等.

## § 8.6 应用实例

### 8.6.1 MATLAB 求解非线性方程(组)

MATLAB 软件中提供了多个函数求解方程的解析解或数值解.符号工具箱中给出的函数 solve() 对求解多项式方程(组)是十分有效的.

例如,求解带参数的方程组

$$\begin{cases} x^2 + ax + 6y^2 + b = 0 \\ y = a + x + 4 \end{cases}$$

只要在工作窗口键入命令:

```
>> syms a b x y;
```

```
>> [x,y] = solve('x^2 + a * x + 6 * y^2 + b = 0','y = a + x + 4','x,y')
```

MATLAB 反馈两组解析题:

$$\begin{aligned} x &= \left[ -24/7 - 13/14 * a + 1/14 * (-384 - 96 * a + a^2 - 28 * b)^{(1/2)} \right] \\ &\quad \left[ -24/7 - 13/14 * a - 1/14 * (-384 - 96 * a + a^2 - 28 * b)^{(1/2)} \right] \\ y &= \left[ 1/14 * a + 4/7 + 1/14 * (-384 - 96 * a + a^2 - 28 * b)^{(1/2)} \right] \\ &\quad \left[ 1/14 * a + 4/7 - 1/14 * (-384 - 96 * a + a^2 - 28 * b)^{(1/2)} \right] \end{aligned}$$

对于方程组 
$$\begin{cases} x + 3y^3 + 5z^2 = \frac{1}{2} \\ x^2 - 3y + 2z^3 = 2, \\ x^3 + 4y^2 + 2z = 5 \end{cases}$$
, 用命令得到的是数值解:

```
>> [x,y,z] = solve('x + 3 * y^3 + 5 * z^2 = 0.5','x^2 - 3 * y + 2 * z^3 = 2',  
'x^3 + 4 * y^2 + 2 * z = 5');
```

解得 27 组解, 其中一组解为:

$$\begin{cases} x_1 = 2.1894536962191751662310389046301 + 0.90750374776006492253615248356683 * i \\ y_1 = 1.2869793387508196518568412932804 - 1.0234657932573800703293002604836 * i \\ z_1 = -1.2607377127232910931556622005624 - 0.88305066883018633027093714600531 * i \end{cases}$$

将解  $[x, y, z]$  代入原方程组, 得残差向量的模为:

```
>> err = [x + 3 * y.^3 + 5 * z.^2 - 0.5, x.^2 - 3 * y + 2 * z.^3 - 2, x.^3 + 4 *  
          y.^2 + 2 * z - 5];  
>> norm(double(eval(err)))  
ans = 3.6745e - 030
```

可见解的精确度是非常高的.

`roots()` 是求多项式方程所有根的函数, 其调用形式为

```
>> roots(p);
```

其中  $p$  为多项式系数按降幂排列的向量.

对于一般的非线性方程(组), 有函数 `fzero()`、`fsolve()` 等. 例如, 求解方程  $e^{-3x} \sin(4x + 2) + 4e^{-\frac{1}{2}x} \cos(2x) = \frac{1}{2}$  在  $x_0 = 1$  附近的根, 可键入命令

```
>> fx = inline('exp(-3 * x) * sin(4 * x + 2) + 4 * exp(-0.5 * x) *  
cos(2 * x) - 0.5','x');  
>> [x,f] = fsolve(fx,1.0)
```

得到

```
x = 0.67374570499890  
f = 1.346045497285786e - 011
```

其中  $f$  为残差值. 事实上, 此题计算中默认的精度要求太低, 我们可以调整精度控制量, 得到更精确的解:

```
>> ff = optimset; ff.TolX = 1e - 16; ff.TolFun = 1e - 30; ff.Display = 'iter';  
>> [x,f] = fsolve(fx,1.0,ff)
```

迭代 5 步得到具有 16 为有效数值的近似解:

$$x = 0.6737457050013476$$

$$f = 0$$

最优化工具箱中的无约束优化函数 `fminsearch()`、`fminunc()` 也是有效的工具, 它们的调用形式与 `fsolve()` 相同.

### 8.6.2 高温炉的温度计算

机电工程中常需要制造半导体装置, 如太阳能电池、集成电路等, 这中间需要对一些材料进行高温热处理. 典型的高温炉由加热元件及炉墙构成. 设  $x$  表示高温炉内的温度,  $T$  表示炉外周围环境的温度. 由于热量的对流和辐射, 炉内温度将变化, 已知温度的变化率与炉内和炉外的温差成比率. 根据实验观察, 温度变化过程可处理为如下的模型:

$$\frac{dx}{dt} = \alpha(T - x) + \beta(T^4 - x^4) + \gamma u^2.$$

其中  $\alpha, \beta$  和  $\gamma$  是正常数, 依赖于高温炉的保温能力.  $\alpha(T - x)$  表示对流散热,  $\beta(T^4 - x^4)$  表示辐射散热,  $u$  表示加热元件的电流.

在许多热处理过程中, 温度的控制极为重要. 设加热元件的电流  $u$  是常数, 为得到稳定状态的温度  $x$ , 令  $\frac{dx}{dt} = 0$ , 上述微分方程转化为非线性方程

$$\alpha(T - x) + \beta(T^4 - x^4) + \gamma u^2 = 0$$

问题归结为求解非线性方程的根的问题. 现取数据如下:

$$u = 21 \text{ A}, \quad T = 20^\circ\text{C} = 293.15 \text{ K}, \quad \alpha = 50 \text{ min}^{-1}$$

$$\beta = 2 \times 10^{-7} \text{ min}^{-1} - \text{K}^{-3}, \quad \lambda = 800 \text{ K/min} - \text{A}^2$$

代入方程得

$$f(x) = 3.689 \times 10^5 - 50x - 2 \times 10^{-7}x^4 = 0$$

此求根问题可直接调用 MATLAB 中的多项式求根函数 `roots()`. 设多项式的系数向量

$$c = [-0.0000002 \quad 0 \quad 0 \quad -50 \quad 368900]$$

调用函数 `roots(c)`, 即得方程的四个根

$$x = \begin{bmatrix} -1210.5 \\ 46.02 + 1166.3i \\ 46.02 - 1166.3i \\ 1118.5 \end{bmatrix}.$$

显然前三个根没有物理意义, 故当加热元件的电流为  $u = 21 \text{ A}$  时, 相应的稳定状态温度为  $x_4 = 1118.5 \text{ K}$ .



### 8.6.3 悬索垂度与张力的计算

公路和铁路设计中常出现高架悬索桥梁,由于桥梁的重量,在设计中需计算各个支撑部件所承受的张力. 设高架悬索系统如图 8-8,其中  $a$  表示悬索的跨度,  $x$  是悬索的垂度,  $m$  是悬索承受的质量.

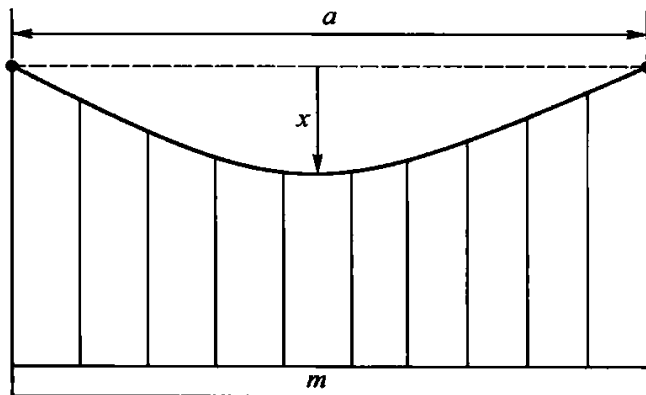


图 8-8

设悬索承受的重量是均匀分布的,  $g = 9.78 \text{ m/s}^2$  表示重力加速度,则悬索承受的负荷密度为  $w = \frac{mg}{a}$ . 若不计温度变化的影响,悬索端点的张力由公式

$$T = \frac{wa}{2} \sqrt{1 + \left(\frac{a}{x}\right)^2}$$

确定,为此需计算悬索的垂度  $x$ . 设悬索的长度为  $L > a$ ,垂度  $x$  近似满足如下的非线性代数方程

$$L = a \left[ 1 + \frac{8}{3} \left(\frac{x}{a}\right)^2 - \frac{32}{5} \left(\frac{x}{a}\right)^4 + \frac{256}{7} \left(\frac{x}{a}\right)^6 \right]$$

不妨设  $a = 120 \text{ m}$ ,  $L = 125 \text{ m}$ ,  $m = 1\,500 \text{ kg}$ ,则悬索的负荷密度  $w = 122.25 \text{ N/m}$ . 采用 Newton 迭代法求解非线性方程,取迭代初值  $x_0 = a/2$ ,误差精度  $\varepsilon = 10^{-6}$  开始计算,迭代到第 7 步,得到结果  $x = 15.27 \text{ m}$ ,悬索端点承受张力  $T = 16\,168.50 \text{ N}$ .

由张力计算公式知:悬索的垂度  $x$  越大,悬索受到的张力  $T$  越小,因此可调节悬索的长度  $L$ ,增加悬索的垂度  $x$ ,减小其承受的张力  $T$ . 表 8-5 显示了对于悬索的不同长度,数值计算所得悬索的垂度及其端点所受的张力.

表 8-5

| $L/m$ | 125      | 130      | 135      | 140      | 145     | 150     |
|-------|----------|----------|----------|----------|---------|---------|
| $x/m$ | 15.27    | 21.94    | 27.19    | 31.63    | 35.45   | 38.75   |
| $T/N$ | 16 168.5 | 12 426.8 | 10 922.1 | 10 109.1 | 9 608.6 | 9 275.9 |

程序如下:

```
function newton = newton(x0,tol)
    a = 120; % 悬索的跨度(m)
    m = 1 500; % 承受物的质量(kg)
    g = 9.78; % 重力加速度
    w = m * g/a;
    f = inline('a * (1 + (8/3) * (t/a)^2 - (32/5) * (t/a)^4 + (256/7) * (t/a)^6) - L','a','t','L'); % f(t) = 0
    df = inline('a * ((16/3) * t/a^2 - (32/5) * 4 * (t/a)^3/a + (256 * 6/7) * (t/a)^5/a)','a','t'); % f'(t)
    % 计算垂度和张力
    for j = 1:6
        L(j) = a + 5 * j;
        for i = 1:100 % 迭代求解非线性方程
            xi = x0 - f(a,x0,L(j))/df(a,x0);
            if abs(xi - x0) <= tol break; end;
            x0 = xi;
        end;
        x(j) = xi;
        T(j) = (w * a/2) * sqrt(1 + (a/(4 * x(j)))^2);
    end
```

## 评 注

本章重点讨论了单个非线性方程的情形,但在基本概念和收敛性方面,方程组的情形和单个方程的情形类似.

简单迭代法的理论基础是求迭代函数  $\varphi(x)$  的不动点. 迭代收敛的充分条件是在有根区间  $[a, b]$  上, 有  $|\varphi'(x)| < 1$ . 简单迭代法可推广到方程组的情形.

Newton 迭代法收敛速度很快, 在单根的情况下, Newton 迭代法是二阶收敛的. Newton 迭代法对初值的要求较高, 初值选得不好时, Newton 迭代序列有可能是发散的, 此时可先用二分法或简单迭代法获得有一定精度的近似值, 再将此近

似值作为 Newton 迭代法的初值开始迭代计算,或将方法改进为 Newton 下山法

$$x_{k+1} = x_k - \lambda \frac{f(x_k)}{f'(x_k)} (k = 0, 1, 2, \dots)$$

其中  $\lambda (0 < \lambda < 1)$  称为下山因子. 适当选择下山因子, 可使迭代序列收敛. 下山因子的选取没有统一公式, 一般通过试算确定.

弦截法和抛物线法是多点迭代法, 它们属于插值型方法. 弦截法的收敛阶是  $r = \frac{1+\sqrt{5}}{5} \approx 1.618$ , 抛物线法的收敛阶是  $r \approx 1.85$ . 抛物线法适用于求方程的复根.

Newton 迭代法可以方便地推广到  $n$  维非线性方程组的情形. 此时需要引入 Jacobi 矩阵  $J(\mathbf{x}) = F'(\mathbf{x}^{(k)})$ , 计算中可用中心差商近似  $J(\mathbf{x})$  中的各偏导数, 然后用直接法或用迭代法求解线性代数方程组. 针对 Newton 迭代法的弱点, 有许多改进的形式, 它们统称为 Newton 型方法, 例如, DFP 算法、BFGS 算法、Fletcher-Freeman 方法和 Gill-Murray 方法都是比较有效的<sup>[11]</sup>. 非线性方程组的求解问题可以转化为一个无约束优化问题, 本章仅介绍了最速下降法. 最速下降法是一个经典的下降算法, 它简单易行, 但收敛速度缓慢, 目前使用得不多. 然而在理论上, 它是其他下降算法的基础. 求解大型无约束问题的有效方法是共轭梯度法, 计算时需要求模函数的导数. 如果模函数不可导, 或计算导数过于复杂, 可以采用不需要计算导数的直接法, 如模式搜索法和单纯形法等.

## 习 题 八

1. 用对分区间法求解下列方程, 要求误差不超过  $10^{-5}$ :

- (1)  $x - \ln x = 2$  在区间  $[2, 4]$  内的根;
- (2)  $xe^x - 1 = 0$  在区间  $[0, 1]$  内的根;
- (3)  $x^3 + 4x^2 - 10 = 0$  在区间  $[1, 2]$  内的根.

2. 已知方程  $x^3 - x^2 - 1 = 0$  在  $x_0 = 1.5$  邻近有根, 试判断下列迭代格式:

- (1)  $x_{n+1} = 1 + \frac{1}{x_n^2}$
- (2)  $x_{n+1} = \sqrt{\frac{1}{x_n - 1}}$
- (3)  $x_{n+1} = \sqrt[3]{1 + x_n^2}$

在  $x_0 = 1.5$  邻近的收敛性. 选一种迭代格式计算 1.5 邻近的根, 要求结果具有 5 位有效数字.

3. 用简单迭代法求下列方程的根, 要求  $|x_{n+1} - x_n| < 10^{-5}$ :

(1)  $x - \cos x = 0$

(2)  $4x - e^{-x} = 0$

(3)  $x^3 - 2x - 5 = 0$

4. 用 Steffensen 方法求方程  $x^3 - x^2 - 1 = 0$  在区间  $[1, 2]$  内的根, 要求结果具有 5 位有效数字, 并与第 2 题的结果进行比较.

5. 分别用简单迭代法与 Steffensen 方法求方程  $xe^x - 1 = 0$  的根, 要求误差不超过  $10^{-5}$ .

6. 对于方程  $f(x) = 0$ , 使用迭代格式

$$x_{n+1} = x_n + cf(x_n) \quad (n = 0, 1, 2, \dots)$$

若  $f'(x) \neq 0$ , 如何选取  $c$  才能保证迭代收敛.

7. 设有解方程  $12 - 3x + 2\cos x = 0$  的迭代法

$$x_{n+1} = 4 + \frac{2}{3}\cos x_n$$

(1) 证明对  $\forall x_0 \in \mathbf{R}$ , 此迭代法均收敛到方程的根;

(2) 取  $x_0 = 4$ , 用此迭代法求方程根的近似值, 使其误差不超过  $\frac{1}{2} \times 10^{-5}$ ;

(3) 此迭代法的收敛阶是多少? 说明理由.

8. 证明由迭代格式

$$x_{n+1} = \frac{x_n}{2} + \frac{1}{x_n} \quad (n = 0, 1, 2, \dots)$$

产生的迭代序列  $\{x_n\}$ , 对任意  $x_0 > 1$  均收敛到  $\sqrt{2}$ , 且是二阶收敛的.

9. 证明当  $x_0$  充分接近  $\sqrt{a}$  时, 按迭代公式

$$x_{n+1} = \frac{x_n(x_n^2 + 3a)}{3x_n^2 + a} \quad (n = 0, 1, 2, \dots)$$

产生的迭代序列  $\{x_n\}$  收敛于  $\sqrt{a}$ , 且收敛阶为 3.

10. 用 Newton 法求解下列方程, 要求  $|x_{n+1} - x_n| < 10^{-6}$ :

(1)  $x - \ln x = 2$

(2)  $xe^x - 1 = 0$

(3)  $x + \sin x = 1$

11. 试用 Newton 法推导出  $\sqrt[p]{a}$  的迭代格式, 并求  $\sqrt[3]{3}$  的近似值, 要求误差不超过  $10^{-5}$ .

12. 证明方程  $f(x) = x^3 - x - 1 = 0$  在区间  $[1, 2]$  内仅有一个根. 取  $x_0 = 1.5$ , 用 Newton 法求此根, 要求精确到 5 位有效数字, 并说明其收敛阶数为 2.

13. 若  $x^*$  是方程  $f(x) = 0$  的  $r$  重根, 即  $f(x) = (x - x^*)^r \varphi(x)$ , 其中  $\varphi(x^*) \neq 0$ , 试证明:

(1) Newton 法在  $x^*$  邻近是线性收敛的;

(2) 改进的 Newton 法

$$x_{n+1} = x_n - \frac{xf'(x_n)}{f'(x_n)} \quad (n = 0, 1, 2, \dots)$$

在  $x^*$  邻近是至少二阶收敛的.

14. 设  $x^*$  是方程  $f(x) = 0$  的重根, 令

$$u(x) = \frac{f(x)}{f'(x)}$$

试说明  $x^*$  是方程  $u(x) = 0$  的单根, 并用 Newton 法导出相应的迭代格式.

15. 利用上题导出的迭代格式, 求方程

$$x^4 - 0.4x^3 + 1.04x^2 - 0.4x + 0.04 = 0$$

在 0.1 邻近的重根.

16. 试用割线法求解下列方程, 要求  $|x_{n+1} - x_n| < 10^{-5}$ :

(1)  $x - \ln x = 2$

(2)  $5x - \cos x - 1 = 0$

(3)  $x = 3 - 2^x$

17. 用 Newton 法求方程组

$$\begin{cases} x^2 + y^2 - 5 = 0 \\ (x+1)y - 3x - 1 = 0 \end{cases}$$

在 (1, 1) 邻近的解, 迭代 3 次.

18. 求下列方程组的解:

(1)  $\begin{cases} x = x^2 + y^2 \\ y = x^2 - y^2 \end{cases} \quad (x_0, y_0) = (0.8, 0.4)$

(2)  $\begin{cases} x - 0.7\sin x - 0.2\cos y = 0 \\ y - 0.7\cos x + 0.2\sin y = 0 \end{cases} \quad (x_0, y_0) = (0.5, 0.5)$

## 数值实验

1. 设有方程  $f(x) = x^2 + \sin 10x - 1 = 0$ , 先作出函数  $y = f(x)$  的图形, 观察方程的有根区间, 构造收敛的迭代格式并编程求出方程  $f(x) = 0$  的所有根.

2. 求方程  $x = e^{-x}$  在  $x = 0.5$  附近的根, 按如下算法编写程序, 求方程的近似解. 要求精确到  $10^{-8}$ :

(1) 取  $\varphi(x) = e^{-x}$ , 用简单迭代法  $x_{n+1} = \varphi(x_n)$  计算;

(2) 用加速收敛的迭代格式  $x_{n+1} = \lambda\varphi(x_n) + (1-\lambda)x_n, \lambda = 0.625$  计算.

3. 先作图分析下列方程的有根区间, 编写 Newton 法和弦截法的程序, 求解

下列方程:

$$(1) 5x^{23} - 6x^7 + 8x^6 - 5x^2 = 0;$$

$$(2) e^{-2x+6} \cos(10x) + e^{-3x} \sin(2x) = e^6.$$

4. 试编写求解非线性方程组的 Newton 法和最速下降法的程序, 求解下列方程组:

$$(1) \begin{cases} x^2 + y^2 = 1 \\ 0.75x^2 - y = -0.9 \end{cases} \text{在点} (-1, 0.1) \text{和} (0.5, 0.9) \text{附近的根};$$

$$(2) \begin{cases} x + 3y^3 + 2z^2 = \frac{1}{2} \\ x^2 + 3y + z^3 = 2 \\ x^3 + 2y + 2z^2 = \frac{1}{2} \end{cases} \text{在点} (-1, 0.1, 0.7) \text{附近的根};$$

$$(3) \begin{cases} x^2 e^{-\frac{xy^2}{2}} + e^{-\frac{x^2}{2}} \sin(xy) = 0 \\ x^2 \cos(x + y^2) + y^2 e^{x+y} = 0 \end{cases} \text{在点} (-1, 1), (-0.2, 4) \text{和} (0.1, -2) \text{附近}$$

的根.

## 第九章 常微分方程数值解法

许多实际问题的数学模型是微分方程或微分方程组的定解问题,如物体运动、电路振荡、化学反应及生物群体的变化等.能用解析方法求出精确解的微分方程为数不多,而且有的方程即使有解析解,也可能由于解表达式非常复杂而不易计算.因此有必要研究微分方程的数值解法.

本章主要讨论一阶常微分方程的初值问题,其一般形式是

$$\begin{cases} \frac{dy}{dx} = f(x, y), & a \leq x \leq b \\ y(a) = y_0 \end{cases} \quad (9-1)$$

在下面的讨论中,我们总假定函数  $f(x, y)$  连续,且关于  $y$  满足李普希兹 (Lipschitz) 条件,即存在常数  $L$ ,使得

$$|f(x, y) - f(x, \bar{y})| \leq L |y - \bar{y}|$$

这样,由常微分方程理论知,初值问题(9-1)的解必定存在唯一.

所谓数值解法,就是求问题(9-1)的解  $y(x)$  在若干点

$$a = x_0 < x_1 < x_2 < \cdots < x_N = b$$

处的近似值  $y_n (n=1, 2, \cdots, N)$  的方法,  $y_n (n=1, 2, \cdots, N)$  称为问题(9-1)的数值解,  $h_n = x_{n+1} - x_n$  称为由  $x_n$  到  $x_{n+1}$  的步长.今后如无特别说明,我们总取步长为常量  $h$ .

建立数值解法,首先要将微分方程离散化,一般采用以下几种方法:

(1) 用差商近似导数

若用向前差商  $\frac{y(x_{n+1}) - y(x_n)}{h}$  代替  $y'(x_n)$  代入(9-1)中的微分方程,则得

$$\frac{y(x_{n+1}) - y(x_n)}{h} \approx f(x_n, y(x_n)) \quad (n = 0, 1, \cdots)$$

化简得

$$y(x_{n+1}) \approx y(x_n) + hf(x_n, y(x_n))$$

如果用  $y(x_n)$  的近似值  $y_n$  代入上式右端,所得结果作为  $y(x_{n+1})$  的近似值,记为  $y_{n+1}$ ,则有

$$y_{n+1} = y_n + hf(x_n, y_n) \quad (n = 0, 1, \cdots) \quad (9-2)$$

这样,问题式(9-1)的近似解可通过求解下述问题

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & (n = 0, 1, \dots) \\ y_0 = y(a) \end{cases} \quad (9-3)$$

得到,按式(9-2)由初值  $y_0$  可逐次算出  $y_1, y_2, \dots$ . 式(9-3)是个离散化的问题,称为差分方程初值问题.

需要说明的是,用不同的差商近似导数,将得到不同的计算公式.

### (2) 用数值积分方法

将问题式(9-1)的解表成积分形式,用数值积分方法离散化. 例如,对微分方程两端积分,得

$$y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} f(x, y(x)) dx \quad (n = 0, 1, \dots) \quad (9-4)$$

用  $y_{n+1}, y_n$  代替  $y(x_{n+1}), y(x_n)$ , 对右端积分采用取左端点的矩形公式,即

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) dx \approx hf(x_n, y_n)$$

则由式(9-4)可得

$$y_{n+1} - y_n = hf(x_n, y_n)$$

于是问题式(9-1)的近似解可由以下公式求出

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & (n = 0, 1, \dots) \\ y_0 = y(a) \end{cases}$$

完全类似地,对右端积分采用取右端点的矩形公式或其他数值积分方法,又可得到不同的计算公式.

### (3) 用 Taylor 多项式近似.

将函数  $y(x)$  在  $x_n$  处展开,取一次 Taylor 多项式近似,则得

$$y(x_{n+1}) = y(x_n + h) \approx y(x_n) + hy'(x_n) = y(x_n) + hf(x_n, y(x_n))$$

再将  $y(x_n)$  的近似值  $y_n$  代入上式右端,所得结果作为  $y(x_{n+1})$  的近似值  $y_{n+1}$ ,得到离散化的计算公式

$$y_{n+1} = y_n + hf(x_n, y_n)$$

以上三种方法都是将微分方程离散化的常用方法,每一类方法又可导出不同形式的计算公式. 其中的 Taylor 展开法,不仅可以得到求数值解的公式,而且容易估计截断误差,故本章在推导数值求解公式时主要用这种方法.

在上面的例子中,我们给出了解初值问题式(9-1)的一种最简单的数值公式(9-3),虽然由于它的精度较低而不常使用,但这一公式的导出过程能较清楚地说明构造数值求解公式的基本思想,且几何意义明确. 因此我们首先介绍这种方法.



## § 9.1 欧拉 (Euler) 方法及其改进方法

### 9.1.1 Euler 方法

Euler 方法就是用差分方程初值问题

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) & (n = 0, 1, \dots) \\ y_0 = y(a) \end{cases}$$

的解来近似微分方程初值问题式(9-1)的解, 即由公式(9-2)依次算出  $y(x_n)$  的近似值  $y_n (n = 1, 2, \dots)$ .

例 1 用 Euler 方法求初值问题

$$\begin{cases} \frac{dy}{dx} = x - y, & 0 \leq x \leq 1 \\ y(0) = 0 \end{cases}$$

的数值解.

[解] 此问题的精确解为  $y = x + e^{-x} - 1$ , 对此问题 Euler 公式的具体形式为

$$y_{n+1} = y_n + h(x_n - y_n) \quad (n = 0, 1, \dots)$$

若取  $h = 0.1, N = 10$ , 由初值  $y_0 = 0$  出发按上式计算, 所得数值结果见表 9-1.

表 9-1

| $x_n$ | $y_n$     | $y(x_n)$  | $y(x_n) - y_n$ |
|-------|-----------|-----------|----------------|
| 0     | 0.000 000 | 0.000 000 | 0.000 000      |
| 0.1   | 0.000 000 | 0.004 837 | 0.004 837      |
| 0.2   | 0.010 000 | 0.018 731 | 0.008 731      |
| 0.3   | 0.029 000 | 0.040 818 | 0.011 818      |
| 0.4   | 0.056 100 | 0.070 320 | 0.014 220      |
| 0.5   | 0.090 490 | 0.106 531 | 0.016 041      |
| 0.6   | 0.131 441 | 0.148 812 | 0.017 371      |
| 0.7   | 0.178 297 | 0.196 585 | 0.018 288      |
| 0.8   | 0.230 467 | 0.249 329 | 0.018 862      |
| 0.9   | 0.287 420 | 0.306 570 | 0.019 150      |
| 1.0   | 0.348 678 | 0.367 879 | 0.019 201      |

从几何上看,微分方程  $y' = f(x, y)$  在  $xOy$  平面上确定了一个向量场:点  $(x, y)$  处的方向斜率为  $f(x, y)$ . 问题式(9-1)的解是过点  $(x_0, y_0)$  的积分曲线  $y = y(x)$ , 且此曲线上每点的切向都与向量场在这点的方向一致. 而 Euler 方法所给出的解是过点  $(x_0, y_0)$  的一条折线  $P_0P_1 \cdots P_N$  上的转折点, 折线上每一段的方向都与向量场在该线段的左端点处的方向一致(见图 9-1), 自然地, 可用折线  $P_0P_1 \cdots P_N$  近似问题式(9-1)的解曲线, 因此 Euler 方法又称为 Euler 折线法.

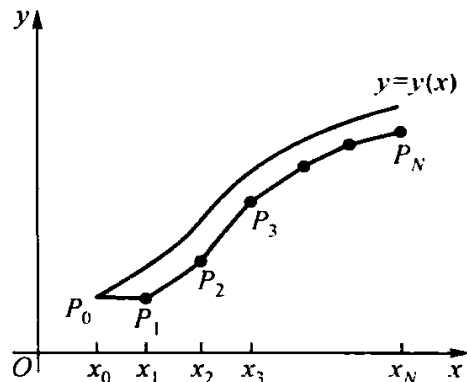


图 9-1

### 9.1.2 向后 Euler 法

如果在离散微分方程时,用向后差商近似导数,即

$$y'(x_{n+1}) \approx \frac{y(x_{n+1}) - y(x_n)}{h}$$

则得计算公式

$$\begin{cases} y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}) \\ y_0 = y(a) \end{cases} \quad (9-5)$$

这就是向后 Euler 方法.

向后 Euler 方法与 Euler 方法形式上相似,但计算时却复杂得多. Euler 公式是显式的,可以直接计算. 向后 Euler 公式的右端含有  $y_{n+1}$ , 不能直接计算出  $y_{n+1}$ , 称为隐式公式, 需要迭代求解. 一般先用 Euler 公式算出迭代初值  $y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$ , 再用向后 Euler 公式迭代

$$y_{n+1}^{(k+1)} = y_n + hf(x_{n+1}, y_{n+1}^{(k)}) \quad (k = 0, 1, \cdots)$$

当  $|y_{n+1}^{(k+1)} - y_{n+1}^{(k)}| < \varepsilon$  时, 终止迭代, 取  $y_{n+1} = y_{n+1}^{(k+1)}$ .

下面考虑向后 Euler 公式迭代的收敛性. 由于  $f(x, y)$  关于  $y$  满足 Lipschitz 条件, 即

$$|f(x, y) - f(x, \bar{y})| \leq L |y - \bar{y}|$$

那么

$$\begin{aligned} |y_{n+1}^{(k+1)} - y_{n+1}^{(k)}| &= h |f(x_{n+1}, y_{n+1}^{(k)}) - f(x_{n+1}, y_{n+1}^{(k-1)})| \\ &\leq hL |y_{n+1}^{(k)} - y_{n+1}^{(k-1)}| \leq \cdots \\ &\leq (hL)^{(k)} |y_{n+1}^{(1)} - y_{n+1}^{(0)}| \end{aligned}$$

由此得出, 当  $hL < 1$  时, 向后 Euler 方法迭代收敛, 称  $hL < 1$  为迭代收敛的条件.

### 9.1.3 梯形公式

利用数值积分方法离散微分方程时,如果用梯形公式计算式(9-4)中右端积分,即

$$\int_{x_n}^{x_{n+1}} f(x, y(x)) dx \approx \frac{h}{2} [f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))]$$

并用  $y_n, y_{n+1}$  代替  $y(x_n), y(x_{n+1})$ , 得求解初值问题式(9-1)的梯形公式

$$y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1})] \quad (9-6)$$

梯形公式也是隐式公式,需用迭代法求解,迭代格式为

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n) \\ y_{n+1}^{(k+1)} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(k)})] \end{cases} \quad (k = 0, 1, \dots) \quad (9-7)$$

同样由于  $f(x, y)$  关于  $y$  满足 Lipschitz 条件,格式(9-7)产生的序列  $\{y_{n+1}^{(k)}\}$  满足

$$\begin{aligned} |y_{n+1}^{(k+1)} - y_{n+1}^{(k)}| &= \frac{h}{2} |f(x_{n+1}, y_{n+1}^{(k)}) - f(x_{n+1}, y_{n+1}^{(k-1)})| \\ &\leq \dots \leq \left(\frac{hL}{2}\right)^{(k)} |y_{n+1}^{(1)} - y_{n+1}^{(0)}| \end{aligned}$$

因此,当  $\frac{hL}{2} < 1$  时,迭代格式(9-7)收敛.

若计算的精度要求不是太高,迭代公式每步仅迭代一次,从而导出一种新的方法——改进的 Euler 方法.

### 9.1.4 改进 Euler 法

按式(9-7)计算问题式(9-1)的数值解时,如果每步只迭代一次,相当于将 Euler 公式与梯形公式结合使用:先用 Euler 公式求  $y_{n+1}$  的一个初步近似值  $\bar{y}_{n+1}$ ,称为预测值,然后用梯形公式校正求得近似值  $y_{n+1}$ ,即

$$\begin{cases} \bar{y}_{n+1} = y_n + hf(x_n, y_n) & \text{预测} \\ y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, \bar{y}_{n+1})] & \text{校正} \end{cases} \quad (9-8)$$

式(9-8)称为由 Euler 公式和梯形公式得到的预测-校正系统,也叫改进 Euler 法,它是显式单步法.

为便于编制程序上机,式(9-8)常改写成

$$\begin{cases} y_p = y_n + hf(x_n, y_n) \\ y_q = y_n + hf(x_n + h, y_p) \\ y_{n+1} = \frac{1}{2}(y_p + y_q) \end{cases} \quad (9-9)$$

**算法 9.1**

(1) 输入  $a, b, f(x, y)$ , 整数  $N$ , 初值  $y_0$ .

(2) 置  $h = \frac{b-a}{N}, n=0, x=a, y=y_0$

输出  $(x, y)$ .

(3) 计算

$$y_p = y + hf(x, y)$$

$$y_q = y + hf(x+h, y_p)$$

$$\text{置 } \frac{1}{2}(y_p + y_q) \Rightarrow y, x+h \Rightarrow x$$

输出  $(x, y)$ .

(4) 若  $n < N-1$ , 置  $n+1 \Rightarrow n$ , 转 3; 否则停机.

**例 2** 用改进 Euler 法求初值问题

$$\begin{cases} y' = x - y, & 0 \leq x \leq 1 \\ y(0) = 0 \end{cases}$$

的数值解, 取  $h=0.1, N=10$ .

[解] 按式(9-9)

$$y_p = y_n + hf(x_n, y_n) = y_n + 0.1(x_n - y_n)$$

$$y_q = y_n + 0.1(x_n + 0.1 - y_p)$$

$$y_{n+1} = \frac{1}{2}(y_p + y_q)$$

数值结果见表 9-2.

**表 9-2**

| $x_n$ | $y_n$     | $y(x_n)$  | $y(x_n) - y_n$ |
|-------|-----------|-----------|----------------|
| 0     | 0.000 000 | 0.000 000 | 0.000 000      |
| 0.1   | 0.005 000 | 0.004 837 | -0.000 163     |
| 0.2   | 0.019 025 | 0.018 731 | -0.000 294     |
| 0.3   | 0.041 218 | 0.040 818 | -0.000 400     |
| 0.4   | 0.070 802 | 0.070 320 | -0.000 482     |
| 0.5   | 0.107 076 | 0.106 531 | -0.000 545     |
| 0.6   | 0.149 404 | 0.148 812 | -0.000 592     |
| 0.7   | 0.197 211 | 0.196 585 | -0.000 626     |
| 0.8   | 0.249 976 | 0.249 329 | -0.000 647     |

续表

| $x_n$ | $y_n$     | $y(x_n)$  | $y(x_n) - y_n$ |
|-------|-----------|-----------|----------------|
| 0.9   | 0.307 228 | 0.306 570 | -0.000 658     |
| 1.0   | 0.368 541 | 0.367 879 | -0.000 662     |

与例 1 的计算结果比较,改进 Euler 法的精度明显高于 Euler 法.

### 9.1.5 单步法的误差估计

前面介绍的几种方法,在计算  $y_{n+1}$  时都只用到前一步的值  $y_n$ ,这类方法统称为单步法,单步法的一般形式为

$$y_{n+1} = y_n + h\varphi(x_n, y_n, x_{n+1}, y_{n+1}, h) \quad (9-10)$$

其中函数  $\varphi$  与微分方程的右端项  $f(x, y)$  有关,若  $\varphi$  中不含  $y_{n+1}$ ,则方法为显式的,否则为隐式的. 为便于误差分析,先考虑计算一步所产生的误差.

**定义 9.1** 设  $y(x)$  是问题式(9-1)的精确解,则

$$R_{n+1} = y(x_{n+1}) - y(x_n) - h\varphi(x_n, y(x_n), x_{n+1}, y(x_{n+1}), h) \quad (9-11)$$

称为单步法(9-10)的局部截断误差,简称截断误差.

局部截断误差通常由 Taylor 展开得到,如 Euler 法有

$$\begin{aligned} R_{n+1} &= y(x_{n+1}) - y(x_n) - hf(x_n, y(x_n)) = y(x_n + h) - y(x_n) - hy'(x_n) \\ &= \frac{1}{2}h^2 y''(\xi) \end{aligned} \quad (9-12)$$

由误差表示的不同形式, Euler 法的局部截断误差也可表示为

$$R_{n+1} = \frac{1}{2}h^2 y''(x_n) + O(h^3) = O(h^2)$$

实际计算时,  $y_n$  是  $y(x_n)$  的近似值,因此,计算过程中除每步所产生的局部截断误差外,还有因前面的计算不准确而引起的误差.

**定义 9.2** 在不考虑舍入误差的情况下,问题式(9-1)的精确解  $y(x_{n+1})$  与数值解  $y_{n+1}$  之差称为整体截断误差,记为

$$e_{n+1} = y(x_{n+1}) - y_{n+1}$$

下面讨论 Euler 方法的整体截断误差.

为简便起见,假定函数  $f(x, y)$  充分光滑,问题式(9-1)的解  $y(x)$  在  $[a, b]$  上二阶连续可微,于是由式(9-12),局部截断误差有界,即存在  $M > 0$ ,使得对任意  $x \in [a, b]$ ,都有  $|y''(x)| \leq M$ ,从而有

$$\begin{aligned} |R_{n+1}| &= |y(x_{n+1}) - [y(x_n) + hf(x_n, y(x_n))]| \\ &= \frac{h^2}{2} |y''(\xi)| \leq \frac{h^2}{2} M \quad (n = 0, 1, 2, \dots) \end{aligned}$$

记  $\bar{y}_{n+1} = y(x_n) + hf(x_n, y(x_n))$ , 于是

$$\begin{aligned}
 |e_{n+1}| &= |y(x_{n+1}) - y_{n+1}| \\
 &\leq |y(x_{n+1}) - \bar{y}_{n+1}| + |\bar{y}_{n+1} - y_{n+1}| \\
 &= |R_{n+1}| + |y(x_n) + hf(x_n, y(x_n)) - y_n - hf(x_n, y_n)| \\
 &\leq |R_{n+1}| + |y(x_n) - y_n| + h|f(x_n, y(x_n)) - f(x_n, y_n)| \\
 &\leq |R_{n+1}| + (1 + hL)|y(x_n) - y_n| \\
 &\leq \frac{h^2}{2}M + (1 + hL)|e_n|
 \end{aligned}$$

反复递推得

$$\begin{aligned}
 |e_{n+1}| &\leq \frac{h^2}{2}M + (1 + hL)\left[\frac{h^2}{2}M + (1 + hL)|e_{n-1}|\right] \\
 &\leq \frac{h^2}{2}M \sum_{k=0}^n (1 + hL)^k \\
 &= \frac{h^2}{2}M \frac{(1 + hL)^{n+1} - 1}{(1 + hL) - 1} \\
 &= \frac{hM}{2L}[(1 + hL)^{n+1} - 1]
 \end{aligned}$$

因为  $(n+1)h \leq b-a$ , 从而有

$$(1 + hL)^{n+1} \leq (1 + hL)^{\frac{b-a}{h}} < e^{L(b-a)}$$

所以

$$|e_{n+1}| \leq \frac{hM}{2L}[e^{L(b-a)} - 1] \quad (9-13)$$

式(9-8)表明, Euler方法的整体截断误差与  $h$  同阶. 当  $h \rightarrow 0$  时,  $e_N \rightarrow 0$ .

**定义 9.3** 若单步法(9-10)的局部截断误差为  $O(h^{p+1})$ , 则称该方法是  $p$  阶的.

$p$  的值越大, 方法(9-10)的精度越高. 估计式(9-12)说明, Euler方法是一阶的, 它的精度不高.

梯形公式的局部截断误差为

$$\begin{aligned}
 R_{n+1} &= y(x_{n+1}) - \left\{ y(x_n) + \frac{h}{2}[f(x_n, y(x_n)) + f(x_{n+1}, y(x_{n+1}))] \right\} \\
 &= \left[ y(x_n) + hy'(x_n) + \frac{1}{2}h^2y''(x_n) + \frac{1}{3!}h^3y'''(x_n) + O(h^4) \right] \\
 &\quad - \left\{ y(x_n) + \frac{h}{2}\left[ y'(x_n) + y'(x_n) + hy''(x_n) + \frac{1}{2}h^2y'''(x_n) + O(h^3) \right] \right\} \\
 &= -\frac{h^3}{12}y'''(x_n) + O(h^4)
 \end{aligned}$$

故梯形公式是二阶的方法. 类似地可以证明, 向后 Euler 方法是一阶方法, 改进的 Euler 方法是二阶方法.

## § 9.2 龙格-库塔(Runge-Kutta)法

### 9.2.1 Runge-Kutta 法的基本思想

回顾 Euler 方法的误差估计, 用一阶 Taylor 多项式近似函数得到 Euler 公式, 故其局部截断误差为一阶 Taylor 余项  $O(h^2)$ . 完全类似地, 若用  $p$  阶 Taylor 多项式近似函数, 即

$$y_{n+1} = y(x_n) + hy'(x_n) + \frac{h^2}{2!}y''(x_n) + \cdots + \frac{h^p}{p!}y^{(p)}(x_n)$$

其中  $y'(x) = f(x, y)$ ,  $y''(x) = f'_x(x, y) + f'_y(x, y)f(x, y)$ ,  $\cdots$ , 则局部截断误差应为  $p$  阶 Taylor 余项  $O(h^{p+1})$ . 由此得到启示: 可以通过提高 Taylor 多项式的次数来提高算法的阶数, 以得到高精度的数值方法. 但若直接对  $y(x)$  用高次 Taylor 多项式近似, 则因公式中出现  $f$  的各阶偏导数导致计算太复杂, 计算量大而不实用. 如果将 Euler 公式与改进 Euler 公式分别写成以下形式

Euler 公式

$$\begin{cases} y_{n+1} = y_n + hK_1 \\ K_1 = f(x_n, y_n) \end{cases}$$

改进 Euler 公式

$$\begin{cases} y_{n+1} = y_n + h\left(\frac{1}{2}K_1 + \frac{1}{2}K_2\right) \\ K_1 = f(x_n, y_n) \\ K_2 = f(x_n + h, y_n + hK_1) \end{cases}$$

这两组公式都是用函数  $f(x, y)$  在某些点上的值的线性组合来计算  $y(x_{n+1})$  的近似值  $y_{n+1}$ . Euler 公式每步计算一次  $f(x, y)$  的值, 它是  $y(x_{n+1})$  在  $x_n$  处的一阶 Taylor 多项式, 因而是一阶方法. 改进 Euler 公式每步需计算二次  $f(x, y)$  的值, 它在  $(x_n, y_n)$  处的 Taylor 展开式与  $y(x_{n+1})$  在  $x_n$  处的 Taylor 展开式的前三项完全相同, 故是二阶方法. 于是可以考虑用函数  $f(x, y)$  在若干点上的函数值的线性组合来构造近似公式, 构造时要求近似公式在  $(x_n, y_n)$  处的 Taylor 展开式与解  $y(x)$  在  $x_n$  处的 Taylor 展开式的前面几项重合, 从而使近似公式达到所需要的阶数. 这样既避免了计算函数  $f(x, y)$  的偏导数, 又提高了方法的精度, 这就是 Runge-Kutta 方法(简称 RK 方法)的基本思想.

### 9.2.2 RK 方法的构造

一般地, RK 方法设近似公式为

$$\begin{cases} y_{n+1} = y_n + h \sum_{i=1}^p c_i K_i \\ K_1 = f(x_n, y_n) \\ K_i = f\left(x_n + a_i h, y_n + h \sum_{j=1}^{i-1} b_{ij} K_j\right) \quad (i = 2, 3, \dots, p) \end{cases} \quad (9-14)$$

其中  $a_i, b_{ij}, c_i$  都是参数, 确定它们的原则是使近似公式在  $(x_n, y_n)$  处的 Taylor 展开式与  $y(x)$  在  $x_n$  处的 Taylor 展开式的前面的项尽可能多地重合, 这样就使近似公式有尽可能高的精度. 以  $p=2$  为例, 近似公式为

$$\begin{cases} y_{n+1} = y_n + h(c_1 K_1 + c_2 K_2) \\ K_1 = f(x_n, y_n) \\ K_2 = f(x_n + a_2 h, y_n + h b_{21} K_1) \end{cases} \quad (9-15)$$

式(9-15)在  $(x_n, y_n)$  处的 Taylor 展开式为

$$\begin{aligned} y_{n+1} &= y_n + h[c_1 f(x_n, y_n) + c_2 f(x_n + a_2 h, y_n + h b_{21} f(x_n, y_n))] \\ &= y_n + h\{c_1 f(x_n, y_n) + c_2 [f(x_n, y_n) + a_2 h f'_x(x_n, y_n) \\ &\quad + b_{21} h f'_y(x_n, y_n) f(x_n, y_n)]\} + O(h^3) \\ &= y_n + (c_1 + c_2) f(x_n, y_n) h + c_2 [a_2 f'_x(x_n, y_n) + b_{21} f'_y(x_n, y_n) f(x_n, y_n)] h^2 \\ &\quad + O(h^3) \end{aligned} \quad (9-16)$$

$y(x_{n+1})$  在  $x_n$  处的 Taylor 展开式为

$$\begin{aligned} y(x_{n+1}) &= y(x_n) + h y'(x_n) + \frac{h^2}{2} y''(x_n) + O(h^3) \\ &= y_n + f(x_n, y_n) h + \frac{h^2}{2} [f'_x(x_n, y_n) + f'_y(x_n, y_n) f(x_n, y_n)] \\ &\quad + O(h^3) \end{aligned} \quad (9-17)$$

要使近似公式(9-15)的局部截断误差为  $O(h^3)$ , 则应要求式(9-16)和式(9-17)的前三项相同, 于是有

$$\begin{cases} c_1 + c_2 = 1 \\ c_2 a_2 = \frac{1}{2} \\ c_2 b_{21} = \frac{1}{2} \end{cases} \quad (9-18)$$

式(9-18)有无穷多组解, 它的每一组解代入式(9-15)得到的近似公式, 局部



截断误差均为  $O(h^3)$ , 故这些方法统称为二阶方法. 例如, 取  $c_1 = c_2 = \frac{1}{2}$ ,  $a_2 = b_{21} = 1$ , 近似公式为

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2) \\ K_1 = f(x_n, y_n) \\ K_2 = f(x_n + h, y_n + hK_1) \end{cases}$$

这就是改进 Euler 公式. 如果取  $c_1 = 0, c_2 = 1, a_2 = b_{21} = \frac{1}{2}$ , 则得

$$\begin{cases} y_{n+1} = y_n + hK_2 \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right) \end{cases}$$

这也是常用的二阶公式, 称为中点公式.

理论上可以证明, 无论怎样选取参数, 式(9-15)都不可能具有更高的精度. 因此, 每步计算二个函数值, 只能得出二阶公式.

类似地, 对  $p=3$  和  $p=4$  的情形, 通过更复杂的计算, 可以导出三阶和四阶 RK 公式, 其中常用的三阶和四阶 RK 公式为

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 4K_2 + K_3) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right) \\ K_3 = f(x_n + h, y_n - hK_1 + 2hK_2) \end{cases}$$

和

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right) \\ K_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2\right) \\ K_4 = f(x_n + h, y_n + hK_3) \end{cases} \quad (9-19)$$

式(9-19)称为经典形式的四阶 RK 公式, 通常说四阶 RK 方法就是指用式(9-19)求解.

**算法 9.2**

(1) 输入  $a, b, f(x, y), N, y_0$ .

(2) 置  $h = \frac{b-a}{N}, n = 1, x_0 = a$ .

(3) 计算

$$x = x_0 + h$$

$$K_1 = f(x_0, y_0)$$

$$K_2 = f\left(x_0 + \frac{h}{2}, y_0 + \frac{h}{2}K_1\right)$$

$$K_3 = f\left(x_0 + \frac{h}{2}, y_0 + \frac{h}{2}K_2\right)$$

$$K_4 = f(x_0 + h, y_0 + hK_3)$$

$$y = y_0 + \frac{h}{6}[K_1 + 2(K_2 + K_3) + K_4]$$

输出  $(x, y)$ .

(4) 若  $n < N$ , 则置  $n+1 \Rightarrow n, x \Rightarrow x_0, y \Rightarrow y_0$ , 转 3; 否则停机.

基于算法 9.2 的四阶 RK 方法的 MATLAB 程序如下:

```
function yrk4 = rk4(fxy, a, b, y0, n)
h = (b - a)/n; x = a:h:b; yrk4 = zeros(n, 1); yrk4(1) = y0;
for i = 1:n-1
    k1 = fxy(x(i), yrk4(i));
    k2 = fxy(x(i) + h/2, yrk4(i) + h * k1/2);
    k3 = fxy(x(i) + h/2, yrk4(i) + h * k2/2);
    k4 = fxy(x(i) + h, yrk4(i) + h * k3);
    yrk4(i+1) = yrk4(i) + h * (k1 + 2 * k2 + 2 * k3 + k4)/6;
end;
```

**例 3** 用四阶 RK 方法求初值问题

$$\begin{cases} y' = x - y, & 0 \leq x \leq 1 \\ y(0) = 0 \end{cases}$$

的数值解, 取  $h = 0.1, n = 10$ .

[解] 公式(9-19)在此题中的具体形式为

$$\begin{cases} y_{n+1} = y_n + \frac{0.1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = x_n - y_n \\ K_2 = x_n + \frac{h}{2} - \left(y_n + \frac{h}{2}K_1\right) = 0.95(x_n - y_n) + 0.05 \\ K_3 = x_n + \frac{h}{2} - \left(y_n + \frac{h}{2}K_2\right) = 0.9525(x_n - y_n) + 0.0475 \\ K_4 = x_n + \frac{h}{2} - \left(y_n + \frac{h}{2}K_3\right) = 0.952375(x_n - y_n) + 0.047625 \end{cases}$$

计算结果见表 9-3.

表 9-3

| $x_n$ | $y_n$                | $y(x_n)$             | $y(x_n) - y_n$         |
|-------|----------------------|----------------------|------------------------|
| 0     | 0                    | 0                    | 0                      |
| 0.1   | 0.004 837 500 000 00 | 0.004 837 418 035 96 | $-0.82 \times 10^{-7}$ |
| 0.2   | 0.018 730 901 406 25 | 0.018 730 753 077 98 | $-0.15 \times 10^{-6}$ |
| 0.3   | 0.040 818 422 001 18 | 0.040 818 220 681 72 | $-0.20 \times 10^{-6}$ |
| 0.4   | 0.070 320 288 917 49 | 0.070 320 046 035 64 | $-0.24 \times 10^{-6}$ |
| 0.5   | 0.106 530 934 423 38 | 0.106 530 659 712 63 | $-0.27 \times 10^{-6}$ |
| 0.6   | 0.148 811 934 376 32 | 0.148 811 636 094 03 | $-0.30 \times 10^{-6}$ |
| 0.7   | 0.196 585 618 671 23 | 0.196 585 303 791 41 | $-0.31 \times 10^{-6}$ |
| 0.8   | 0.249 329 289 734 43 | 0.249 328 964 117 22 | $-0.33 \times 10^{-6}$ |
| 0.9   | 0.306 569 991 200 08 | 0.306 569 659 740 60 | $-0.33 \times 10^{-6}$ |
| 1.0   | 0.367 879 774 412 50 | 0.367 879 441 171 44 | $-0.33 \times 10^{-6}$ |

与例 1、例 2 的计算结果相比,显然四阶 RK 方法的精度高得多.

需要说明的是,当  $p=1,2,3,4$  时,RK 公式的最高阶数恰好是  $p$ ,当  $p>4$  时,RK 公式的最高阶数不是  $p$ ,如  $p=5$  时,RK 公式的最高阶数仍为 4, $p=6$  时,RK 公式的最高阶数为 5. RK 方法的导出基于 Taylor 展开,故它要求所求问题的解具有较高的光滑度.当解充分光滑时,四阶 RK 方法确实优于改进 Euler 法.对大量实际问题来说,四阶 RK 方法一般可以达到精度要求.如果解的光滑性差,则用四阶 RK 方法求问题(9-1)的数值解的效果可能不如改进 Euler 法.因此,实际计算时,需根据问题的具体情况来选择合适的算法.

### 9.2.3 变步长的 RK 方法

由于问题式(9-1)的解函数  $y(x)$  的变化是不均匀的,如果用等步长求数值解,则可能产生有些点处精度过高,有些点处精度过低的情况,为保证一定的精度,必须取较小的步长.但这样既增加了计算量,也导致误差的增加,因而实际计算时,往往采用事后估计误差、自动调整步长的 RK 方法.

设用  $p$  阶方法计算  $y_{n+1}$ ,记从  $y_n$  出发,以步长  $h$  计算一步所得  $y(x_{n+1})$  的近似值为  $y_{n+1}^{(h)}$ ,以步长  $\frac{h}{2}$  计算两步得  $y(x_{n+1})$  的近似值为  $y_{n+1}^{(\frac{h}{2})}$ .由于  $p$  阶公式的局部截断误差为  $O(h^{p+1})$ ,且  $y^{(p+1)}(x)$  在小区间  $[x_n, x_{n+1}]$  上变化不大,故有

$$y(x_{n+1}) - y_{n+1}^{(h)} \approx ch^{p+1} \quad (9-20)$$

$$y(x_{n+1}) - y_{n+1}^{(\frac{h}{2})} \approx 2c\left(\frac{h}{2}\right)^{p+1} \quad (9-21)$$

将式(9-21)乘以  $2^p$  减式(9-20)得

$$(2^p - 1)y(x_{n+1}) - 2^p y_{n+1}^{(\frac{h}{2})} + y_{n+1}^{(h)} \approx 0$$

从而有

$$|y(x_{n+1}) - y_{n+1}^{(h)}| \approx \frac{2^p}{2^p - 1} |y_{n+1}^{(h)} - y_{n+1}^{(\frac{h}{2})}| = \Delta \quad (9-22)$$

这样,可以事后误差估计式(9-22)中  $\Delta$  的大小来选择合适的步长.

变步长的 RK 方法按以下过程进行:先以  $h$  为步长从点  $x_n$  出发求得  $y(x_n + h)$  的一个近似值  $y_{n+1}^{(h)}$ ,再以  $\frac{h}{2}$  为步长从  $x_n$  出发计算两步求得  $y(x_n + h)$  的又一个近似值  $y_{n+1}^{(\frac{h}{2})}$ .如果

$$\Delta = \frac{2^p}{2^p - 1} \times |y_{n+1}^{(h)} - y_{n+1}^{(\frac{h}{2})}| < \varepsilon$$

且相差不多,这里  $\varepsilon$  为事先给定的精度,则取  $y(x_{n+1}) \approx y_{n+1} = y_{n+1}^{(h)}$ ,并以  $h$  为步长计算  $x_{n+2}$  处的近似值  $y_{n+2}$ ;如果  $\Delta \ll \varepsilon$ ,则表明步长过小,应反复将步长加倍进行计算,直到  $\Delta > \varepsilon$  为止.此时前一次的计算结果为  $y_{n+1}$ ,步长即为合适步长,可以此步长继续计算;如果  $\Delta > \varepsilon$ ,则表明步长过大,应将步长减半再计算,直到满足  $\Delta < \varepsilon$  为止.

## § 9.3 线性多步法

以上所介绍的各种数值解法都是单步法,单步法在计算  $y_{n+1}$  时,只用到前一步的信息  $y_n$ .

为了提高精度,需重新计算多个点处的函数值,如 RK 方法,故计算量较大. 如何通过较多地利用前面的已知信息,如  $y_n, y_{n-1}, \dots, y_{n-r}$ , 来构造高精度的算法计算  $y_{n+1}$ , 这就是多步法的基本思想. 多步法中最常用的是线性多步法, 它的计算公式中只出现  $y_{n+1}, y_n, \dots, y_{n-r}$  及  $f(x_{n+1}, y_{n+1}), \dots, f(x_{n-r}, y_{n-r})$  的一次项, 其一般形式为

$$y_{n+1} = \sum_{i=0}^r \alpha_i y_{n-i} + h \sum_{i=1}^r \beta_i f_{n-i} \quad (9-23)$$

其中  $\alpha_i, \beta_i$  均为常数,  $f_k = f(x_k, y_k)$  ( $k = n+1, n, \dots, n-r$ ). 若  $\beta_{-1} = 0$ , 公式 (9-23) 为显式,  $\beta_{-1} \neq 0$ , 公式为隐式. 构造线性多步公式常用 Taylor 展开和数值积分的方法.

### 9.3.1 线性多步公式的导出

利用 Taylor 展开导出线性多步公式的基本方法是: 将线性多步公式 (9-22) 在  $x_n$  处进行 Taylor 展开, 然后与  $y(x_{n+1})$  在  $x_n$  处的 Taylor 展开式相比较, 要求它们前面的项重合, 由此确定参数  $\alpha_i, \beta_i$ . 下面以  $r=1$  为例说明.

设初值问题式 (9-1) 的解  $y(x)$  充分光滑, 待定的线性两步公式为

$$y_{n+1} = \alpha_0 y_n + \alpha_1 y_{n-1} + h(\beta_{-1} f_{n+1} + \beta_0 f_n + \beta_1 f_{n-1}) \quad (9-24)$$

记  $y_n^{(k)} = y^{(k)}(x_n)$  ( $k=1, 2, \dots$ ), 则  $y(x)$  在  $x_n$  处的 Taylor 展开式为

$$y(x) = y_n + y'_n(x - x_n) + \frac{y''_n}{2}(x - x_n)^2 + \dots + \frac{y_n^{(p)}}{p!}(x - x_n)^p + O((x - x_n)^{p+1}) \quad (9-25)$$

假设前  $n$  步计算结果都是准确的, 即  $y_i = y(x_i), y'(x_i) = f(x_i, y_i)$  ( $i \leq n$ ), 则

$$y_{n-1} = y(x_n - h) = y_n - y'_n h + \frac{y''_n}{2!} h^2 - \frac{y'''_n}{3!} h^3 + \frac{y_n^{(4)}}{4!} h^4 - \frac{y_n^{(5)}}{5!} h^5 + O(h^6)$$

$$f_{n-1} = f(x_{n-1}, y_{n-1}) = y'(x_{n-1}) = y'_n - y''_n h + \frac{y'''_n}{2!} h^2 - \frac{y_n^{(4)}}{3!} h^3 + \frac{y_n^{(5)}}{4!} h^4 + O(h^5)$$

$$f_n = f(x_n, y_n) = y'_n$$

$$f_{n+1} = f(x_{n+1}, y_{n+1}) \approx y'(x_{n+1}) = y'_n + y''_n h + \frac{y'''_n}{2!} h^2 + \frac{y_n^{(4)}}{3!} h^3 + \frac{y_n^{(5)}}{4!} h^4 + O(h^5)$$

将以上各式代入式 (9-24) 并整理, 得

$$\begin{aligned} y_{n+1} = & (\alpha_0 + \alpha_1) y_n + (-\alpha_1 + \beta_{-1} + \beta_0 + \beta_1) y'_n h + \left( \frac{\alpha_1}{2} + \beta_{-1} - \beta_1 \right) y''_n h^2 + \\ & \left( -\frac{\alpha_1}{6} + \frac{\beta_{-1}}{2} + \frac{\beta_1}{2} \right) y'''_n h^3 + \left( \frac{\alpha_1}{24} + \frac{\beta_{-1}}{6} - \frac{\beta_1}{6} \right) y_n^{(4)} h^4 + \\ & \left( -\frac{\alpha_1}{120} + \frac{\beta_{-1}}{24} + \frac{\beta_1}{24} \right) y_n^{(5)} h^5 + O(h^6) \end{aligned} \quad (9-26)$$

为使公式(9-24)有  $p$  阶精度,只需使式(9-26)与  $y(x_{n+1})$  在  $x_n$  处的 Taylor 展开式

$$y(x_{n+1}) = y_n + y'_n h + \frac{y''_n}{2!} h^2 + \cdots + \frac{y_n^{(5)}}{5!} h^5 + O(h^6) \quad (9-27)$$

的前  $p+1$  项重合,因为式(9-26)中有 5 个待定系数,对比 Taylor 展开式的系数可得方程组

$$\begin{cases} \alpha_0 + \alpha_1 = 1 \\ -\alpha_1 + \beta_{-1} + \beta_0 + \beta_1 = 1 \\ \frac{1}{2}\alpha_1 + \beta_{-1} - \beta_1 = \frac{1}{2} \\ -\frac{1}{6}\alpha_1 + \frac{1}{2}\beta_{-1} + \frac{1}{2}\beta_1 = \frac{1}{6} \\ \frac{1}{24}\alpha_1 + \frac{1}{6}\beta_{-1} - \frac{1}{6}\beta_1 = \frac{1}{24} \end{cases} \quad (9-28)$$

由推导过程知,如果选取参数  $\alpha_i, \beta_i$ ,使其满足式(9-28)中前  $p+1$  个方程( $p=1, 2, 3, 4$ ),则近似公式(9-24)为  $p$  阶公式. 如  $\alpha_0 = 1, \alpha_1 = 0, \beta_{-1} = \beta_0 = \frac{1}{2}, \beta_1 = 0$  满足方程组(9-28)的前三个方程,故公式

$$y_{n+1} = y_n + \frac{h}{2}(f_{n+1} + f_n)$$

为二阶公式. 解方程组(9-28)得

$$\alpha_0 = 0, \quad \alpha_1 = 1, \quad \beta_{-1} = \beta_1 = \frac{1}{3}, \quad \beta_0 = \frac{4}{3}$$

相应的线性二步公式为

$$y_{n+1} = y_{n-1} + \frac{h}{3}(f_{n+1} + 4f_n + f_{n-1}) \quad (9-29)$$

式(9-29)称为 Simpson 公式,它是四阶公式,局部截断误差为

$$R_{n+1} = -\frac{1}{90}h^5 y_n^{(5)} + O(h^6)$$

由此可知,线性二步公式至多是四阶公式.

一般地,线性多步公式(9-23)中有  $2r+3$  个待定参数,如果令式(9-23)右端在  $x_n$  处的 Taylor 展开式与  $y(x_{n+1})$  在  $x_n$  处的 Taylor 展开式的前  $p+1$  项系数对应相等,可得方程组

$$\begin{cases} \sum_{i=0}^r \alpha_i = 1 \\ \sum_{i=1}^r (-i)^k \alpha_i + k \sum_{i=-1}^r (-i)^{k-1} \beta_i = 1 \quad (k = 1, 2, \cdots, p) \end{cases} \quad (9-30)$$

其解所对应的公式(9-23)具有  $p$  阶精度,局部截断误差为

$$R_{n+1} = \frac{h^{p+1}}{(p+1)!} \left[ 1 - \sum_{i=1}^r (-i)^{p+1} \alpha_i - (p+1) \sum_{i=-1}^r (-i)^p \beta_i \right] y_n^{(p+1)} + O(h^{p+2}) \quad (9-31)$$

显然,线性多步公式(9-23)至多可达到  $2r+2$  阶精度.

### 9.3.2 常用的线性多步公式

#### (一) 亚当斯(Adams)公式

取  $r=3$ , 并令  $\alpha_1 = \alpha_2 = \alpha_3 = \beta_{-1} = 0$ , 由方程组

$$\begin{cases} \sum_{i=0}^3 \alpha_i = 1 \\ \sum_{i=1}^3 (-i)^k \alpha_i + k \sum_{i=-1}^3 (-i)^{k-1} \beta_i = 1 \quad (k=1, 2, 3, 4) \end{cases} \quad (9-32)$$

可解得

$$\alpha_0 = 1, \quad \beta_0 = \frac{55}{24}, \quad \beta_1 = -\frac{59}{24}, \quad \beta_2 = \frac{37}{24}, \quad \beta_3 = -\frac{9}{24}$$

相应的线性多步公式为

$$y_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \quad (9-33)$$

因为  $\beta_{-1}=0$ , 式(9-33)称为 Adams 显式公式, 它是四阶公式, 局部截断误差为

$$\begin{aligned} R_{n+1} &= \frac{h^5}{5!} \left[ 1 - \sum_{i=1}^3 (-i)^5 \alpha_i - 5 \sum_{i=-1}^3 (-i)^4 \beta_i \right] y_n^{(5)} + O(h^6) \\ &= \frac{251}{720} h^5 y_n^{(5)} + O(h^6) \end{aligned} \quad (9-34)$$

如果令  $\alpha_1 = \alpha_2 = \alpha_3 = \beta_3 = 0$ , 由方程组(9-32)可得解

$$\alpha_0 = 1, \quad \beta_{-1} = \frac{9}{24}, \quad \beta_0 = \frac{19}{24}, \quad \beta_1 = -\frac{5}{24}, \quad \beta_2 = \frac{1}{24}$$

相应的线性多步公式

$$y_{n+1} = y_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \quad (9-35)$$

称为四阶 Adams 隐式公式, 其局部截断误差为

$$R_{n+1} = -\frac{19}{720} h^5 y_n^{(5)} + O(h^6) \quad (9-36)$$

Adams 公式(9-33), 式(9-35)也可以利用数值积分方法得到, 其基本思想是首先将初值问题式(9-1)化成等价的积分形式

$$y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} f(x, y(x)) dx = \int_{x_n}^{x_{n+1}} F(x) dx \quad (9-37)$$

其中  $F(x) = f(x, y(x))$ , 然后用过节点  $x_n, x_{n-1}, \dots, x_{n-r}$  或  $x_{n+1}, x_n, \dots, x_{n-r+1}$  的  $F(x)$  的  $r$  次插值多项式  $\varphi_r(x)$  代替  $F(x)$  求积分, 即得  $r+1$  阶 Adams 显式或隐式公式. 例如, 当  $r=3$  时, 过节点  $x_n, x_{n-1}, x_{n-2}, x_{n-3}$ ,  $F(x)$  的三次插值多项式为

$$L_3(x) = \sum_{i=0}^3 l_i(x) F(x_{n-i})$$

其中

$$l_i(x) = \frac{(x-x_n)(x-x_{n-1})(x-x_{n-2})(x-x_{n-3})}{(x-x_{n-i}) \prod_{\substack{j=0 \\ j \neq i}}^3 (x_{n-i} - x_{n-j})} \quad (i=0, 1, 2, 3)$$

将  $L_3(x)$  代替  $F(x)$  计算式 (9-37) 右端的积分, 得

$$\begin{aligned} y(x_{n+1}) - y(x_n) &\approx \int_{x_n}^{x_{n+1}} L_3(x) dx \\ &= \sum_{i=0}^3 \left[ \int_{x_n}^{x_{n+1}} l_i(x) dx \right] F(x_{n-i}) \\ &= F(x_n) \int_{x_n}^{x_{n+1}} \frac{(x-x_{n-1})(x-x_{n-2})(x-x_{n-3})}{6h^3} dx + \\ &\quad F(x_{n-1}) \int_{x_n}^{x_{n+1}} \frac{(x-x_n)(x-x_{n-2})(x-x_{n-3})}{-2h^3} dx + \\ &\quad F(x_{n-2}) \int_{x_n}^{x_{n+1}} \frac{(x-x_n)(x-x_{n-1})(x-x_{n-3})}{2h^3} dx + \\ &\quad F(x_{n-3}) \int_{x_n}^{x_{n+1}} \frac{(x-x_n)(x-x_{n-1})(x-x_{n-2})}{-6h^3} dx \\ &= \frac{h}{24} [55F(x_n) - 59F(x_{n-1}) + 37F(x_{n-2}) - 9F(x_{n-3})] \end{aligned}$$

用  $y_n, y_{n+1}$  代替  $y(x_n), y(x_{n+1})$ ,  $f_k = f(x_k, y_k)$  代替  $F(x_k) = f(x_k, y(x_k))$  ( $k = n, n-1, n-2, n-3$ ), 则得

$$y_{n+1} = y_n + \frac{h}{24} (55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3})$$

这就是四阶 Adams 显式公式. 由于积分区间在插值区间  $[x_{n-3}, x_n]$  外面, 故又称为四阶 Adams 外插公式. 由插值余项公式可得其局部截断误差为

$$\begin{aligned} R_{n+1} &= \int_{x_n}^{x_{n+1}} \frac{F^{(4)}(\xi_x)}{4!} \prod_{j=0}^3 (x - x_{n-j}) dx \\ &= \int_{x_n}^{x_{n+1}} \frac{y^{(5)}(\xi_x)}{4!} \prod_{j=0}^3 (x - x_{n-j}) dx \end{aligned}$$

由积分中值定理, 存在  $\eta \in (x_{n-3}, x_{n+1})$ , 使得



$$R_{n+1} = \frac{y^{(5)}(\eta)}{4!} \int_{x_n}^{x_{n+1}} \prod_{j=0}^3 (x - x_{n-j}) dx = \frac{251}{720} h^5 y^{(5)}(\eta)$$

如果以过节点  $x_{n+1}, x_n, x_{n-1}, x_{n-2}$  的  $F(x)$  的三次插值多项式

$$L_3(x) = \sum_{i=-1}^2 l_i(x) F(x_{n-i})$$

其中

$$l_i(x) = \frac{\prod_{j=-1}^2 (x - x_{n-j})}{(x - x_{n-i}) \prod_{\substack{j=-1 \\ j \neq i}}^2 (x_{n-i} - x_{n-j})} \quad (i = -1, 0, 1, 2)$$

代替  $F(x)$  计算式(9-37)右端的积分, 即得四阶 Adams 隐式公式

$$y_{n+1} = y_n + \frac{h}{24} (9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2})$$

其局部截断误差为

$$R_{n+1} = -\frac{19}{720} h^5 y^{(5)}(\eta), \quad x_{n-2} < \eta < x_{n+1}$$

由于积分区间在插值区间  $[x_{n-2}, x_{n+1}]$  内, 故 Adams 隐式公式又称为 Adams 内插公式.

### (二) 米尔恩 (Milne) 公式

取  $r=3$ , 并令  $\alpha_0 = \alpha_1 = \alpha_2 = \beta_{-1} = 0$ , 由方程组(9-31)可解出  $\alpha_3 = 1, \beta_0 = \frac{8}{3}, \beta_1 = -\frac{4}{3}, \beta_2 = \frac{8}{3}, \beta_3 = 0$ , 相应的线性多步公式

$$y_{n+1} = y_{n-3} + \frac{4}{3} h (2f_n - f_{n-1} + 2f_{n-2}) \quad (9-38)$$

称为 Milne 公式, 其局部截断误差为

$$R_{n+1} = \frac{14}{45} h^5 y_n^{(5)} + O(h^6) \quad (9-39)$$

Milne 公式是四阶四步显式公式.

### (三) 汉明 (Hamming) 公式

取  $r=2$ , 并令  $\alpha_1 = \beta_2 = 0$ , 可得到 Hamming 公式

$$y_{n+1} = \frac{1}{8} (9y_n - y_{n-2}) + \frac{3}{8} h (f_{n+1} + 2f_n - f_{n-1}) \quad (9-40)$$

其局部截断误差为

$$R_{n+1} = -\frac{1}{40} h^5 y_n^{(5)} + O(h^6) \quad (9-41)$$

Hamming 公式是四阶三步隐式公式.

利用线性多步法求解初值问题式(9-1)时,必须先用其他方法算出开始几个点的近似值.一般可用同阶的单步法.

**例 4** 分别用四阶 Adams 显式和隐式公式求初值问题

$$\begin{cases} y' = x - y, & 0 \leq x \leq 1 \\ y(0) = 0 \end{cases}$$

的数值解,取  $h=0.1$ .

[解] 根据题意,  $x_n = nh = 0.1n$ ,  $f_n = x_n - y_n$ , 由四阶 Adams 显式公式(9-33)有

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) \\ &= \frac{1}{24}(18.5y_n + 5.9y_{n-1} - 3.7y_{n-2} + 0.9y_{n-3} + 0.24n + 0.12) \\ &\quad (n = 3, 4, \dots, 9) \end{aligned}$$

由四阶 Adams 隐式公式(9-35)有

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{24}(9f_{n+1} + 19f_n - 5f_{n-1} + f_{n-2}) \\ &= \frac{1}{24}(-0.9y_{n+1} + 22.1y_n + 0.5y_{n-1} - 0.1y_{n-2} + 0.24n + 0.12) \end{aligned}$$

由上式可解出

$$y_{n+1} = \frac{1}{24.9}(22.1y_n + 0.5y_{n-1} - 0.1y_{n-2} + 0.24n + 0.12) \quad (n = 2, 3, \dots, 9)$$

利用精确解  $y = e^{-x} + x - 1$  求出起步值(一般可用四阶 RK 公式计算起步值)后,按上面的公式计算,结果见表 9-4.

表 9-4

| $x_n$ | Adams 显式法    |                       | Adams 隐式法    |                      |
|-------|--------------|-----------------------|--------------|----------------------|
|       | $y_n$        | $ y(x_n) - y_n $      | $y_n$        | $ y(x_n) - y_n $     |
| 0.0   | 0            |                       | 0            |                      |
| 0.1   | 0.004 837 42 |                       | 0.004 837 42 |                      |
| 0.2   | 0.018 730 75 |                       | 0.018 730 75 |                      |
| 0.3   | 0.040 818 22 |                       | 0.040 818 01 | $2.1 \times 10^{-7}$ |
| 0.4   | 0.070 322 92 | $2.87 \times 10^{-6}$ | 0.070 319 66 | $3.8 \times 10^{-7}$ |
| 0.5   | 0.106 535 48 | $4.82 \times 10^{-6}$ | 0.106 530 14 | $5.2 \times 10^{-7}$ |
| 0.6   | 0.148 818 41 | $6.77 \times 10^{-6}$ | 0.148 811 01 | $6.3 \times 10^{-7}$ |
| 0.7   | 0.196 593 39 | $8.09 \times 10^{-6}$ | 0.196 584 59 | $7.1 \times 10^{-7}$ |

续表

| $x_n$ | Adams 显式法    |                        | Adams 隐式法    |                      |
|-------|--------------|------------------------|--------------|----------------------|
|       | $y_n$        | $ y(x_n) - y_n $       | $y_n$        | $ y(x_n) - y_n $     |
| 0.8   | 0.249 338 16 | $9.19 \times 10^{-6}$  | 0.249 328 19 | $7.7 \times 10^{-7}$ |
| 0.9   | 0.306 579 61 | $9.95 \times 10^{-6}$  | 0.306 568 85 | $8.1 \times 10^{-7}$ |
| 1.0   | 0.367 889 96 | $1.052 \times 10^{-6}$ | 0.367 878 60 | $8.4 \times 10^{-7}$ |

从表 9-4 可以看出,四阶 Adams 隐式法比显式法的精度高,比较这两种方法的局部截断误差公式(9-34)和式(9-36),可以说明这一现象.一般地,同阶的隐式法比显式法精确,而且数值稳定性也好.但在隐式公式中,通常很难解出  $y_{n+1}$ ,需要用迭代法求解,这样又增加了计算量.因此实际计算时,很少单独使用显式公式或隐式公式,而是将它们联合使用:先用显式公式求出  $y(x_{n+1})$  的预测值,记作  $\bar{y}_{n+1}$ ,再用隐式公式对预测值进行校正,求出  $y(x_{n+1})$  的近似值  $y_{n+1}$ .

### 9.3.3 预测-校正系统

用显式公式计算预测值,然后用隐式公式进行校正,得到近似值  $y_{n+1}$ ,这样一组计算公式称为预测-校正系统.一般地,采用同阶的隐式公式与显式公式.常用的预测-校正系统有两种:

Adams 预测-校正公式

$$\begin{cases} \bar{y}_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) & \text{预测} \\ y_{n+1} = y_n + \frac{h}{24}[9f(x_{n+1}, \bar{y}_{n+1}) + 19f_n - 5f_{n-1} + f_{n-2}] & \text{校正} \end{cases} \quad (9-42)$$

Milne-Hamming 预测-校正公式

$$\begin{cases} \bar{y}_{n+1} = y_{n-3} + \frac{4}{3}h(2f_n - f_{n-1} + 2f_{n-2}) \\ y_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3}{8}h[f(x_{n+1}, \bar{y}_{n+1}) + 2f_n - f_{n-1}] \end{cases} \quad (9-43)$$

说明:(1) 以上两种预测-校正公式均为四阶公式,其起步值通常用四阶 RK 公式计算;(2) 有时为提高精度,校正公式可迭代进行多次,但迭代次数一般不超过 3 次.

为减少一次迭代所产生的误差,常常用局部截断误差进一步修正预测值与校正值,得到更精确的预测-校正公式.下面分别对 Adams 预测-校正公式与 Milne-Hamming 公式进行讨论.

由 Adams 公式的局部截断误差公式

$$y(x_{n+1}) - \bar{y}_{n+1} = \frac{251}{720}h^5 y_n^{(5)} + O(h^6)$$

$$y(x_{n+1}) - y_{n+1} = -\frac{19}{720}h^5 y_n^{(5)} + O(h^6)$$

两式相减得

$$y_{n+1} - \bar{y}_{n+1} = \frac{270}{720}h^5 y_n^{(5)} + O(h^6)$$

从而有

$$h^5 y_n^{(5)} \approx \frac{720}{270}(y_{n+1} - \bar{y}_{n+1})$$

将上式代入式(9-34)和式(9-35),得

$$y(x_{n+1}) - \bar{y}_{n+1} \approx \frac{251}{270}(y_{n+1} - \bar{y}_{n+1}) \quad (9-44)$$

$$y(x_{n+1}) - y_{n+1} \approx -\frac{19}{270}(y_{n+1} - \bar{y}_{n+1}) \quad (9-45)$$

用式(9-44)和式(9-45)修正公式(9-42),就得到多环节的 Adams 预测-校正公式

$$\begin{cases} p_{n+1} = y_n + \frac{h}{24}(55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}) & \text{预测} \\ m_{n+1} = p_{n+1} + \frac{251}{270}(c_n - p_n) & \text{改进} \\ c_{n+1} = y_n + \frac{h}{24}[9f(x_{n+1}, m_{n+1}) + 19f_n - 5f_{n-1} + f_{n-2}] & \text{校正} \\ y_{n+1} = c_{n+1} - \frac{19}{270}(c_{n+1} - p_{n+1}) & \text{改进} \end{cases} \quad (9-46)$$

完全类似地,可以导出多环节的 Milne-Hamming 预测-校正公式(也称为 Hamming 方法).

$$\begin{cases} p_{n+1} = y_{n-3} + \frac{4}{3}h(2f_n - f_{n-1} + 2f_{n-2}) \\ m_{n+1} = p_{n+1} + \frac{112}{121}(c_n - p_n) \\ c_{n+1} = \frac{1}{8}(9y_n - y_{n-2}) + \frac{3}{8}h[f(x_{n+1}, m_{n+1}) + 2f_n - f_{n-1}] \\ y_{n+1} = c_{n+1} - \frac{9}{121}(c_{n+1} - p_{n+1}) \end{cases} \quad (9-47)$$

上述预测-校正公式的优点是每算一步只需计算两个函数值,计算量小于四阶 RK 方法,而且在计算过程中已大致估计出误差.它们不足之处在于必须借助于别的方法计算开始几个函数值,计算过程中不易变步长.

**算法 9.3**

(1) 输入  $a, b, f(x, y), N, y_0$ .

(2) 置  $h = \frac{b-a}{N}, x_0 = a, n = 1$ .

(3) 计算

$$f_{n-1} = f(x_{n-1}, y_{n-1})$$

$$K_1 = hf_{n-1}$$

$$K_2 = hf\left(x_{n-1} + \frac{h}{2}, y_{n-1} + \frac{K_1}{2}\right)$$

$$K_3 = hf\left(x_{n-1} + \frac{h}{2}, y_{n-1} + \frac{K_2}{2}\right)$$

$$K_4 = hf(x_{n-1} + h, y_{n-1} + K_3)$$

$$y_n = y_{n-1} + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

$$x_n = a + nh$$

(4) 输出  $(x_n, y_n)$ .

(5) 若  $n < 3$ , 置  $n+1 \Rightarrow n$ , 返回 3; 否则, 置  $n+1 \Rightarrow n, 0 \Rightarrow p_0, 0 \Rightarrow c_0$ , 转 6.

(6) 计算

$$f_3 = f(x_3, y_3)$$

$$x = x_3 + h$$

$$p = y_0 + \frac{4}{3}(2f_3 - f_2 + 2f_1)$$

$$m = p + \frac{112}{121}(c_0 - p_0)$$

$$c = \frac{1}{8}(9y_3 - y_1) + \frac{3}{8}h[f(x, m) + 2f_3 - f_2]$$

$$y = c - \frac{9}{121}(c - p)$$

输出  $(x, y)$ .

(7) 若  $n < N$ , 置  $n+1 \Rightarrow n, x_{j+1} \Rightarrow x_j, y_{j+1} \Rightarrow y_j, f_{j+1} \Rightarrow f_j (j = 0, 1, 2), x \Rightarrow x_3, y \Rightarrow y_3, p \Rightarrow p_0, c \Rightarrow c_0$ , 转 6; 否则停机.

## § 9.4 相容性、收敛性与稳定性

### 9.4.1 相容性与收敛性

上面所介绍的方法都是用离散化的手段, 将微分方程初值问题化为差分方

程初值问题来求解的. 这些转化是否合理? 即当  $h \rightarrow 0$  时, 差分方程是否能无限逼近微分方程, 差分方程的解  $y_n$  是否能无限逼近微分方程初值问题的准确解  $y(x_n)$ , 这就是相容性与收敛性问题. 这里仅对显式单步法进行讨论.

用显式单步法

$$y_{n+1} = y_n + h\varphi(x_n, y_n, h)$$

其中函数  $\varphi$  称为增量函数, 求解初值问题式(9-1), 即用差分方程初值问题

$$\begin{cases} y_{n+1} = y_n + h\varphi(x_n, y_n, h) \\ y(x_0) = y_0 \end{cases} \quad (9-48)$$

的解作为问题式(9-1)的近似解. 如果近似是合理的, 则应有

$$\frac{y(x+h) - y(x)}{h} - \varphi(x, y(x), h) \rightarrow 0 \quad (h \rightarrow 0) \quad (9-49)$$

其中  $y(x)$  为问题式(9-1)的解. 因为

$$\lim_{h \rightarrow 0} \frac{y(x+h) - y(x)}{h} = y'(x) = f(x, y)$$

故由式(9-49)得

$$\lim_{h \rightarrow 0} \varphi(x, y, h) = f(x, y)$$

如果增量函数  $\varphi(x, y, h)$  关于  $h$  连续, 则有

$$\varphi(x, y, 0) = f(x, y) \quad (9-50)$$

式(9-50)称为相容性条件. 如果增量函数  $\varphi(x, y, h)$  满足条件式(9-50), 则称显式单步法与初值问题式(9-1)相容. 显然, 满足相容性条件式(9-50)是可以单步法求解初值问题式(9-1)的必要条件.

容易验证, Euler 法与改进 Euler 法均满足相容性条件. 事实上, 对 Euler 法, 增量函数为

$$\varphi(x, y, h) = f(x, y)$$

自然满足条件式(9-50), 改进 Euler 法的增量函数为

$$\varphi(x, y, h) = \frac{1}{2}[f(x, y) + f(x+h, y+hf(x, y))]$$

因为  $f(x, y)$  连续, 从而有

$$\varphi(x, y, 0) = \frac{1}{2}[f(x, y) + f(x, y)] = f(x, y)$$

所以, Euler 法与改进 Euler 法均与初值问题式(9-1)相容. 一般地, 如果显式单步法有  $p$  阶精度( $p > 0$ ), 则其局部截断误差为

$$y(x+h) - [y(x) + h\varphi(x, y(x), h)] = O(h^{p+1})$$

上式两端同除以  $h$ , 得

$$\frac{y(x+h) - y(x)}{h} - \varphi(x, y, h) = O(h^p)$$

令  $h \rightarrow 0$ , 如果  $\varphi(x, y, h)$  连续, 则有

$$y'(x) - \varphi(x, y, 0) = 0$$

即

$$f(x, y) - \varphi(x, y, 0) = 0$$

所以  $p > 0$  的显式单步法均与问题式(9-1)相容. 由此即得各阶 RK 方法与初值问题式(9-1)相容.

一种数值方法称为是收敛的, 如果对于任意初值  $y_0$  及任意  $x \in (a, b]$ , 都有

$$\lim_{h \rightarrow 0} y_n = y(x) \quad (x = a + nh)$$

其中  $y(x)$  为初值问题式(9-1)的准确解.

按定义, 数值方法的收敛性需根据该方法的整体截断误差  $\varepsilon_{n+1}$  来判定.

§ 9.1 中已导出, Euler 方法的整体截断误差有估计式

$$|e_{n+1}| \leq \frac{hM}{2L} |e^{L(b-a)} - 1| = O(h)$$

当  $h \rightarrow 0$  时,  $e_n \rightarrow 0$ , 故 Euler 方法收敛. 类似地, 可以讨论显式单步法的收敛性.

**定理 9.1** 设显式单步法具有  $p$  阶精度, 其增量函数  $\varphi(x, y, h)$  关于  $y$  满足 Lipschitz 条件, 问题式(9-1)的初值是精确的, 即  $y(x_0) = y_0$ , 则显式单步法整体截断误差为

$$e_{n+1} = y(x_{n+1}) - y_{n+1} = O(h^p)$$

读者可仿 Euler 法整体截断误差估计过程自行证明此定理. 提示: 显式单步法具有  $p$  阶精度, 即其局部截断误差为  $O(h^{p+1})$ , 故存在  $M > 0$ , 使得

$$|R_{n+1}| = |y(x_{n+1}) - y(x_n) - h\varphi(x_n, y(x_n), h)| \leq Mh^{p+1}$$

**推论 1** 设显式单步法具有  $p$  阶精度, 增量函数  $\varphi(x, y, h)$  在区域

$$S: a \leq x \leq b, \quad -\infty < y < +\infty, \quad 0 \leq h \leq h_0$$

内连续, 且关于  $y$  满足 Lipschitz 条件, 则显式单步法是收敛的.

当  $f(x, y)$  在区域  $D: a \leq x \leq b, -\infty < y < +\infty$  上连续, 且关于  $y$  满足 Lipschitz 条件时, 改进 Euler 方法, 各阶 RK 方法的增量函数  $\varphi(x, y, h)$  在区域  $S$  上连续, 且关于  $y$  满足 Lipschitz 条件, 因而它们都是收敛的.

关于显式单步法收敛的更一般结果是:

**定理 9.2** 设增量函数  $\varphi(x, y, h)$  在区域  $S$  上连续, 且关于  $y$  满足 Lipschitz 条件, 则显式单步法收敛的充分必要条件是相容性条件式(9-50)成立.

证明请看参考文献[9].

#### 9.4.2 稳定性

在收敛性的讨论中, 我们是在每步计算都是准确的前提下, 估计由近似公式所产生的误差, 没有考虑舍入误差. 实际计算中, 除了由数值方法所产生的截断

误差外,还有因数字舍入而产生的误差.一种数值方法,即使它满足相容性条件,也是收敛的,但若在计算过程中,舍入误差的积累越来越大,那么它的解  $y_n$  作为问题式(9-1)的近似解也可能严重失真.影响舍入误差的因素很多,这里我们只关心它在传播过程中的增长情况,这就是数值方法的稳定性问题.具体地说,如果用某种数值方法求解初值问题,当步长  $h$  固定时,在节点值  $y_n$  上产生误差(扰动) $\delta$ ,而由此引起节点值  $y_m (m > n)$  的误差  $\delta_m$  的绝对值均不超过  $|\delta|$ ,则称此方法是绝对稳定的.稳定性问题比较复杂,而上面的定义又依赖于微分方程本身,实际讨论时,均就下述模型方程进行

$$y' = \lambda y \quad (9-51)$$

其中  $\lambda$  为复常数.

**定义 9.4** 用某种数值方法求解模型方程(9-51),对固定的步长  $h$ ,如果由计算节点值  $y_n$  时产生的误差  $\delta$  所引起后面节点值  $y_m (m > n)$  的误差  $\delta_m$  的绝对值均不超过  $|\delta|$ ,则称该方法对于所用步长  $h$  和复数  $\lambda$  是绝对稳定的.使得方法绝对稳定的  $h$  和  $\lambda$  的全体称为该方法的绝对稳定区域.特别地,如果绝对稳定区域包含整个左半平面,则称该方法是 A-稳定的.绝对稳定区域与实轴的交称为绝对稳定区间.

(1) Euler 方法的稳定性.

Euler 方法用于模型方程式(9-51),计算公式为

$$y_{n+1} = (1 + h\lambda)y_n \quad (9-52)$$

设实际计算  $y_n$  时产生误差  $\delta$ ,它使节点值  $y_{n+1}$  产生误差  $\delta_{n+1}$ ,记  $\tilde{y}_n = y_n + \delta, \tilde{y}_{n+1} = y_{n+1} + \delta_{n+1}$ ,由式(9-52)有

$$\tilde{y}_{n+1} = y_{n+1} + \delta_{n+1} = (1 + h\lambda)\tilde{y}_n = (1 + h\lambda)y_n + (1 + h\lambda)\delta$$

于是有

$$\delta_{n+1} = (1 + h\lambda)\delta \quad (9-53)$$

为使误差不扩大,仅需

$$|1 + h\lambda| \leq 1 \quad (9-54)$$

因此,Euler 方法的绝对稳定区域为  $|1 + h\lambda| \leq 1$ ,绝对稳定区间为  $-2 \leq h\lambda \leq 0$ .如图 9-2 所示,这是  $h\lambda$  平面上以  $-1$  为中心的单位圆.当  $h\lambda$  在此圆内时,Euler 方法是稳定的;若  $h\lambda$  在此圆外,则 Euler 方法不稳定.

(2) 梯形公式的稳定性.

对模型方程式(9-51),梯形公式的具体表达式为

$$y_{n+1} = y_n + \frac{h}{2}(\lambda y_n + \lambda y_{n+1})$$



即

$$y_{n+1} = \frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} y_n \quad (9-55)$$

类似于 Euler 方法的讨论可得, 梯形公式的绝对稳定区域为

$$\left| \frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}} \right| \leq 1 \quad (9-56)$$

即

$$\left| 1 + \frac{h\lambda}{2} \right| \leq \left| 1 - \frac{h\lambda}{2} \right|$$

化简得

$$\operatorname{Re}(h\lambda) \leq 0$$

因此, 梯形公式的绝对稳定区域为  $h\lambda$  平面的左半平面 (见图 9-3). 特别地, 当  $\lambda$  为负实数时, 对任意的  $h > 0$ , 梯形公式都是稳定的.

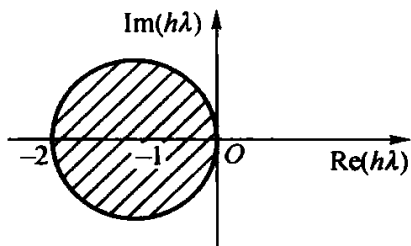


图 9-2

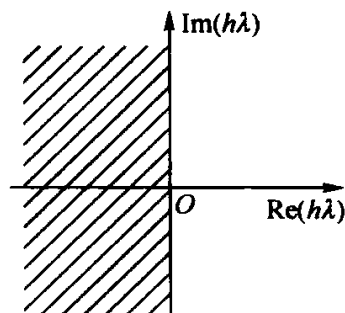


图 9-3

### (3) RK 方法的稳定性.

与前面的讨论相仿, 将 RK 方法用于模型方程 (9-51), 可得二、三、四阶 RK 方法的绝对稳定区域分别为

$$\begin{aligned} \left| 1 + h\lambda + \frac{1}{2}(h\lambda)^2 \right| &\leq 1 \\ \left| 1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 \right| &\leq 1 \\ \left| 1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 + \frac{1}{24}(h\lambda)^4 \right| &\leq 1 \end{aligned}$$

其图形见图 9-4, 当  $\lambda$  为实数时, 四阶 RK 方法的绝对稳定区间为  $[-2.78, 0]$ .

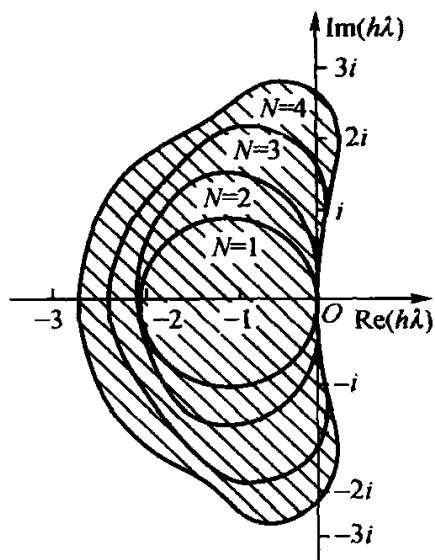


图 9-4

**例 5** 用四阶 RK 方法求解初值问题

$$\begin{cases} y' = -20y, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

分别取  $h=0.1, 0.2$ . 将所得结果与准确解  $y = e^{-20x}$  比较, 并说明比较结果.

[解] 对此问题, 四阶 RK 方法的计算公式为

$$K_1 = -20y_n, \quad K_2 = -20\left(y_n + \frac{h}{2}K_1\right)$$

$$K_3 = -20\left(y_n + \frac{h}{2}K_2\right), \quad K_4 = -20(y_n + hK_3)$$

$$y_{n+1} = y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

将初值  $y_0 = y(0) = 1$  代入, 计算结果见表 9-5.

表 9-5

| $x_n$ | $y_n(h=0.1)$ | $y(x_n) - y_n$ | $y_n(h=0.2)$ | $y(x_n) - y_n$ |
|-------|--------------|----------------|--------------|----------------|
| 0.0   | 1            |                | 1            |                |
| 0.1   | 0.333 333 3  | -0.197 998 1   |              |                |
| 0.2   | 0.111 111 1  | -0.092 795 5   | 5            | -4.981 684 4   |
| 0.3   | 0.037 037 0  | -0.034 558 2   |              |                |
| 0.4   | 0.012 345 6  | -0.012 010 1   | 25           | -24.999 665    |
| 0.5   | 0.004 115 2  | -0.004 069 8   |              |                |

续表

| $x_n$ | $y_n(h=0.1)$               | $y(x_n) - y_n$              | $y_n(h=0.2)$ | $y(x_n) - y_n$ |
|-------|----------------------------|-----------------------------|--------------|----------------|
| 0.6   | 0.001 371 7                | -0.001 365 6                | 125          | -124.999 9     |
| 0.7   | $4.572\ 47 \times 10^{-4}$ | $-4.564\ 15 \times 10^{-4}$ |              |                |
| 0.8   | $1.524\ 15 \times 10^{-4}$ | $-1.523\ 03 \times 10^{-4}$ | 625          | -625.000 0     |
| 0.9   | $5.080\ 52 \times 10^{-5}$ | $-5.079 \times 10^{-5}$     |              |                |
| 1.0   | $1.881\ 67 \times 10^{-6}$ | $-1.879\ 61 \times 10^{-6}$ | 3 125        | -3 125.000     |

从表 9-5 可以看出, 当  $h=0.1$  时, 虽然计算结果的相对误差很大, 但计算过程是稳定的. 而当  $h=0.2$  时, 计算过程不稳定. 这是因为  $\lambda = -20$ , 当  $h=0.1$  时,  $\lambda h = -2$  属于稳定区间  $[-2.78, 0]$ , 当  $h=0.2$  时,  $\lambda h = -4$  不属于稳定区间.

对于一般方程  $y' = f(x, y)$ , 在考虑稳定性对步长的限制时, 常用  $\frac{\partial f}{\partial y}$  代替  $\lambda$ , 只要步长的选取使得  $h\lambda$  在所用方法的绝对稳定区域内即可. 例如, 若用 Euler 方法求解初值问题

$$\begin{cases} y' = -\frac{20xy}{1+x^2}, & 0 \leq x \leq 2 \\ y(0) = 1 \end{cases}$$

因为  $\lambda = \frac{\partial f}{\partial y} = -\frac{20x}{1+x^2} < 0$ , 且  $\max_{0 \leq x \leq 2} |\lambda| = 10$ , 要使  $|1 + h\lambda| \leq 1$ , 只要  $|1 - 10h| \leq 1$ , 即  $0 \leq x \leq 0.2$ . 这就是说, 只要取步长  $0 < h \leq 0.2$ , 用 Euler 方法求解上述问题是绝对稳定的. 如果用四阶 RK 方法, 则需限制步长  $h$ , 使得  $-2.78 \leq h\lambda < 0$ , 即  $0 < h < 0.278$ .

从稳定性定义及数值结果可以看出, 绝对稳定区域越大, 对步长  $h$  的限制越小, 方法的稳定性越好. 因而梯形公式与 RK 方法的稳定性均优于 Euler 方法.

线性多步法的收敛性与稳定性问题比较复杂, 这里不作介绍, 有兴趣的读者可参看[2], [9].

## § 9.5 微分方程组的数值解法

### 9.5.1 一阶微分方程组的数值解法

设有一阶微分方程组的初值问题

$$\begin{cases} y'_i = f_i(x, y_1, y_2, \dots, y_m) & (i = 1, 2, \dots, m) \\ y_i(a) = y_{i0} \end{cases} \quad (9-57)$$

若记  $\mathbf{y} = (y_1, y_2, \dots, y_m)^T$ ,  $\mathbf{y}_0 = (y_{10}, y_{20}, \dots, y_{m0})^T$ ,  $\mathbf{f} = (f_1, f_2, \dots, f_m)^T$ , 则初值问题式(9-57)可写成如下向量形式

$$\begin{cases} \mathbf{y}' = \mathbf{f}(x, \mathbf{y}) \\ \mathbf{y}(a) = \mathbf{y}_0 \end{cases} \quad (9-58)$$

如果向量函数  $\mathbf{f}(x, \mathbf{y})$  在区域

$$D: a \leq x \leq b, \quad \mathbf{y} \in \mathbf{R}^m$$

连续, 且关于  $\mathbf{y}$  满足 Lipschitz 条件, 即存在  $L > 0$ , 使得对  $\forall x \in [a, b], \mathbf{y}_1, \mathbf{y}_2 \in \mathbf{R}^m$ , 都有

$$\|\mathbf{f}(x, \mathbf{y}_1) - \mathbf{f}(x, \mathbf{y}_2)\| \leq L \|\mathbf{y}_1 - \mathbf{y}_2\|$$

那么问题式(9-58)在  $[a, b]$  上存在唯一解  $\mathbf{y} = \mathbf{y}(x)$ .

问题式(9-58)和式(9-1)形式上完全相同, 故对初值问题式(9-1)所建立的各种数值解法可全部用于求解问题式(9-58), 只需将  $y$  换成向量  $\mathbf{y}$ ,  $f(x, y)$  换成向量函数  $\mathbf{f}(x, \mathbf{y})$  即可.

例如, 对问题式(9-58), Euler 方法的计算公式为

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(x_n, \mathbf{y}_n) \quad (9-59)$$

其中  $\mathbf{y}_n = (y_{1n}, y_{2n}, \dots, y_{mn})^T$ , 其分量形式为

$$y_{in+1} = y_{in} + hf_i(x_n, y_{1n}, y_{2n}, \dots, y_{mn}) \quad (i = 1, 2, \dots, m)$$

改进 Euler 法的计算公式为

$$\begin{cases} \mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{2}(\mathbf{K}_1 + \mathbf{K}_2) \\ \mathbf{K}_1 = \mathbf{f}(x_n, \mathbf{y}_n) \\ \mathbf{K}_2 = \mathbf{f}(x_n + h, \mathbf{y}_n + h\mathbf{K}_1) \end{cases} \quad (9-60)$$

经典四阶 RK 公式为

$$\begin{cases} \mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4) \\ \mathbf{K}_1 = \mathbf{f}(x_n, \mathbf{y}_n) \\ \mathbf{K}_2 = \mathbf{f}\left(x_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{K}_1\right) \\ \mathbf{K}_3 = \mathbf{f}\left(x_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{K}_2\right) \\ \mathbf{K}_4 = \mathbf{f}(x_n + h, \mathbf{y}_n + h\mathbf{K}_3) \end{cases} \quad (9-61)$$

其中  $\mathbf{K}_i = (K_{1i}, K_{2i}, \dots, K_{mi})^T (i = 1, 2, 3, 4)$ , 式(9-61)的分量形式为

$$\begin{cases} y_{in+1} = y_{in} + \frac{h}{6}(K_{i1} + 2K_{i2} + 2K_{i3} + K_{i4}) \\ K_{i1} = f_i(x_n, y_{1n}, y_{2n}, \dots, y_{mn}) \\ K_{i2} = f_i\left(x_n + \frac{h}{2}, y_{1n} + \frac{h}{2}K_{11}, \dots, y_{mn} + \frac{h}{2}K_{m1}\right) \\ K_{i3} = f_i\left(x_n + \frac{h}{2}, y_{1n} + \frac{h}{2}K_{12}, \dots, y_{mn} + \frac{h}{2}K_{m2}\right) \\ K_{i4} = f_i(x_n + h, y_{1n} + hK_{13}, \dots, y_{mn} + hK_{m3}) \end{cases} \quad (i = 1, 2, \dots, m)$$

(9-62)

一般地,用于问题式(9-58)的显式单步法公式为

$$y_{n+1} = y_n + h\Phi(x_n, y_n, h) \quad (9-63)$$

其中  $\Phi = (\varphi_1, \varphi_2, \dots, \varphi_m)^T$  是  $m+2$  元的  $m$  维向量函数. 为便于理解,下面用含有两个方程的方程组( $m=2$ )具体说明.

设有初值问题

$$\begin{cases} y' = f(x, y, z), & y(a) = y_0 \\ z' = g(x, y, z), & z(a) = z_0 \end{cases} \quad (9-64)$$

它的经典四阶 RK 公式为

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(K_{11} + 2K_{12} + 2K_{13} + K_{14}) \\ z_{n+1} = z_n + \frac{h}{6}(K_{21} + 2K_{22} + 2K_{23} + K_{24}) \end{cases} \quad (9-65)$$

$$\begin{cases} K_{11} = f(x_n, y_n, z_n) \\ K_{21} = g(x_n, y_n, z_n) \\ K_{12} = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_{11}, z_n + \frac{h}{2}K_{21}\right) \\ K_{22} = g\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_{11}, z_n + \frac{h}{2}K_{21}\right) \\ K_{13} = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_{12}, z_n + \frac{h}{2}K_{22}\right) \\ K_{23} = g\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_{12}, z_n + \frac{h}{2}K_{22}\right) \\ K_{14} = f(x_n + h, y_n + hK_{13}, z_n + hK_{23}) \\ K_{24} = g(x_n + h, y_n + hK_{13}, z_n + hK_{23}) \end{cases} \quad (9-66)$$

计算过程为从节点  $x_n$  处的近似解  $y_n, z_n$  出发,按式(9-66)顺序计算  $K_{11}, K_{21}, K_{12}, K_{22}, K_{13}, K_{23}, K_{14}, K_{24}$ ,将所得结果代入式(9-65),即得  $x_{n+1}$  处的近似解

$y_{n+1}, z_{n+1}$ .

类似于 § 9.4 的讨论, 可以建立一阶方程组的数值解法的相容性、收敛性及稳定性概念与结论.

### 9.5.2 高阶微分方程的数值解法

高阶微分方程的初值问题可以通过变量代换化为一阶微分方程组初值问题.

设有  $m$  阶常微分方程初值问题

$$\begin{cases} y^{(m)} = f(x, y, y', \dots, y^{(m-1)}), & a \leq x \leq b \\ y(a) = y_0, y'(a) = y_0^{(1)}, \dots, y^{(m-1)}(a) = y_0^{(m-1)} \end{cases} \quad (9-67)$$

引入新变量  $y_1 = y, y_2 = y', \dots, y_m = y^{(m-1)}$ , 问题式 (9-67) 就化为一阶微分方程组初值问题

$$\begin{cases} y_1' = y_2, & y_1(a) = y_0 \\ y_2' = y_3, & y_2(a) = y_0^{(1)} \\ \dots\dots\dots & \dots\dots\dots \\ y_{m-1}' = y_m, & y_{m-1}(a) = y_0^{(m-2)} \\ y_m' = f(x, y_1, y_2, \dots, y_m), & y_m(a) = y_0^{(m-1)} \end{cases} \quad (9-68)$$

然后用 9.5.1 中的数值方法求解问题式 (9-68), 就可以得到问题式 (9-67) 的数值解.

**例 6** 用四阶 RK 方法求解初值问题

$$\begin{cases} y'' = 5e^{2x} \sin x - 2y + 2y', & 0 \leq x \leq 1 \\ y(0) = -2, y'(0) = -3 \end{cases} \quad (9-69)$$

取  $h = 0.1$ .

[解] 引入新变量  $z = y'$ , 问题 (9-68) 化为如下形式

$$\begin{cases} y' = z, & y(0) = -2 \\ z' = 5e^{2x} \sin x - 2y + 2z, & z(0) = -3 \end{cases}$$

这是形如式 (9-64) 的问题, 按式 (9-65), 式 (9-66) 求解. 因为  $x_0 = 0, y_0 = -2, z_0 = -3$ , 由式 (9-66) 得

$$K_{11} = z_0 = -3$$

$$K_{21} = 5e^{2x_0} \sin x_0 - 2y_0 + 2z_0 = 4 - 6 = -2$$

$$K_{12} = z_0 + \frac{h}{2} K_{21} = -3 - 0.1 = -3.1$$

$$K_{22} = 5e^h \sin \frac{h}{2} - 2\left(y_0 + \frac{h}{2} K_{11}\right) + 2\left(z_0 + \frac{h}{2} K_{21}\right) = -1.623\ 822\ 4$$

$$K_{13} = z_0 + \frac{h}{2}K_{22} = -3.081\ 191\ 1$$

$$K_{23} = 5e^{0.1}\sin 0.05 - 2\left(-2 - \frac{0.1}{2} \times 3.1\right) + 2\left(-3 - \frac{0.1}{2} \times 1.623\ 822\ 4\right) \\ = -1.576\ 204\ 6$$

$$K_{14} = z_0 + hK_{23} = -3.157\ 620\ 5$$

$$K_{24} = 5e^{0.2}\sin 0.1 - 2(-2 - 0.1 \times 3.081\ 191\ 1) + 2(-3 - 0.1 \times 1.576\ 204\ 6) \\ = -1.089\ 318\ 6$$

将上述结果代入式(9-65)得

$$y_1 = y_0 + \frac{h}{6}(K_{11} + 2K_{12} + 2K_{13} + K_{14}) = -2.308\ 666\ 7$$

$$z_1 = z_0 + \frac{h}{6}(K_{21} + 2K_{22} + 2K_{23} + K_{24}) = -3.158\ 156\ 2$$

类似地,可计算以后各节点处的近似解,计算结果见表9-6.

表 9-6

| $x_n$ | $y_n$        | $z_n$        | $y(x_n) - y_n$        |
|-------|--------------|--------------|-----------------------|
| 0     | -2           | -3           | 0                     |
| 0.1   | -2.308 666 7 | -3.158 156 2 | $1.85 \times 10^{-6}$ |
| 0.2   | -2.627 799 4 | -3.200 744 8 | $4.39 \times 10^{-6}$ |
| 0.3   | -2.943 007 2 | -3.068 319 1 | $7.31 \times 10^{-6}$ |
| 0.4   | -3.233 061 6 | -2.682 910 2 | $1.06 \times 10^{-5}$ |
| 0.5   | -3.467 833 3 | -1.943 690 5 | $1.39 \times 10^{-5}$ |
| 0.6   | -3.605 759 5 | -0.721 904 4 | $1.72 \times 10^{-5}$ |
| 0.7   | -3.590 764 8 | 1.144 985 1  | $2.10 \times 10^{-5}$ |
| 0.8   | -3.348 556 7 | 3.859 959 0  | $2.29 \times 10^{-5}$ |
| 0.9   | -2.782 214 5 | 0.767 390 8  | $2.46 \times 10^{-5}$ |
| 1.0   | -1.766 994 3 | 1.289 383 2  | $2.26 \times 10^{-5}$ |

问题式(9-69)的精确解为  $y = e^{2x}(\sin x - 2\cos x)$ . 将它与近似解比较即得误差(见表9-6中最后一列).

### 9.5.3 刚性方程组

在自动控制、电力系统、化学反应等领域中,常会出现一类特殊的常微分方

程组. 例如, 初值问题

$$\begin{bmatrix} y_1' \\ y_2' \end{bmatrix} = \begin{bmatrix} -0.01 & -299.99 \\ 0 & -300 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}, \quad \begin{aligned} y_1(0) &= 2 \\ y_2(0) &= 1 \end{aligned} \quad (9-70)$$

其解析解为

$$\begin{cases} y_1 = e^{-0.01x} + e^{-300x} \\ y_2 = e^{-300x} \end{cases}$$

在解析解的表达式中, 当  $x \rightarrow +\infty$  时,  $e^{-300x}$  变化迅速, 称为解的快瞬态部分;  $e^{-0.01x}$  相对变化缓慢, 称为解的慢瞬态部分.  $y_1(x)$  含有慢瞬态部分和快瞬态部分, 整体变化缓慢,  $y_2(x)$  仅含有快瞬态部分, 迅速趋于零.

另一方面, 如果用数值方法求解这类问题, 由于方法绝对稳定域的限制, 将对步长的选取十分苛刻. 例如, 用四阶 RK 方法求解方程组 (9-70), 由于方程组 (9-70) 的系数矩阵的特征值为  $\lambda_1 = -300, \lambda_2 = -0.01$ , 为使计算稳定, 要求  $-300h \in (-2.785, 0)$ , 即要求步长  $h < 0.00928$ , 计算到  $x = 10$  时, 需要计算 1078 步, 这就需要相当大的计算量. 而且由于步长过小, 计算步数过多, 使得舍入误差积累, 影响解的真实性. 这就是微分方程的刚性 (stiff) 现象.

**定义 9.5** 设线性方程组

$$y'(x) = Ay(x) + b(x) \quad (9-71)$$

的系数矩阵  $A$  的特征值为  $\lambda_1, \lambda_2, \dots, \lambda_n$ . 若

$$(1) \operatorname{Re} \lambda_j < 0, j = 1, 2, \dots, n$$

$$(2) s = \frac{\max_{1 \leq j \leq n} |\operatorname{Re} \lambda_j|}{\min_{1 \leq j \leq n} |\operatorname{Re} \lambda_j|} \gg 1$$

则称方程组 (9-71) 为刚性方程组 (或病态方程组),  $s$  为刚性比.

方程组 (9-70) 的刚性比  $s = 30000$ , 是一个严重刚性的方程组.

对于刚性方程组的数值求解, 应采用步长  $h$  不受稳定性限制的 A-稳定方法, 如隐式 Euler 法、隐式多步法或隐式 Runge-Kutta 法, 并用 Newton 迭代法求解隐式方程组. 实际计算中常用的有以下几种方法:

二阶隐式 RK 方法:

$$\begin{cases} y_{n+1} = y_n + hK_1 \\ K_1 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right) \end{cases} \quad (9-72)$$

二阶隐式 RK 方法:



$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2) \\ K_1 = f(x_n, y_n) \\ K_2 = f\left(x_n + h, y_n + \frac{h}{2}(K_1 + K_2)\right) \end{cases} \quad (9-73)$$

四阶隐式 RK 方法:

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(K_1 + K_2) \\ K_1 = f\left(x_n + h, y_n + \frac{h}{4}\left(K_1 + \left(1 + \frac{2\sqrt{3}}{3}\right)K_2\right)\right) \\ K_2 = f\left(x_n + \left(\frac{1}{2} - \frac{\sqrt{3}}{6}\right)h, y_n + \frac{h}{4}\left(\left(1 - \frac{2\sqrt{3}}{3}\right)K_1 + K_2\right)\right) \end{cases} \quad (9-74)$$

例如,对于初值问题(9-70),采用隐式 RK 公式(9-72)迭代一次计算,用 MATLAB 编程如下:

```
function [x,y] = RK12(dyf,xspan,y0,h)
x = xspan(1):h:xspan(2); y = zeros(length(y0),length(x));
y(:,1) = y0(:);
for i = 1:length(x) - 1
    k1 = feval(dyf,x(i),y(:,i));
    y(:,i+1) = y(:,i) + h * k1
    k2 = feval(dyf,x(i+1),y(:,i+1));
    y(:,i+1) = y(:,i) + h * (k1 + k2)/2;
end
x = x'; y = y'
```

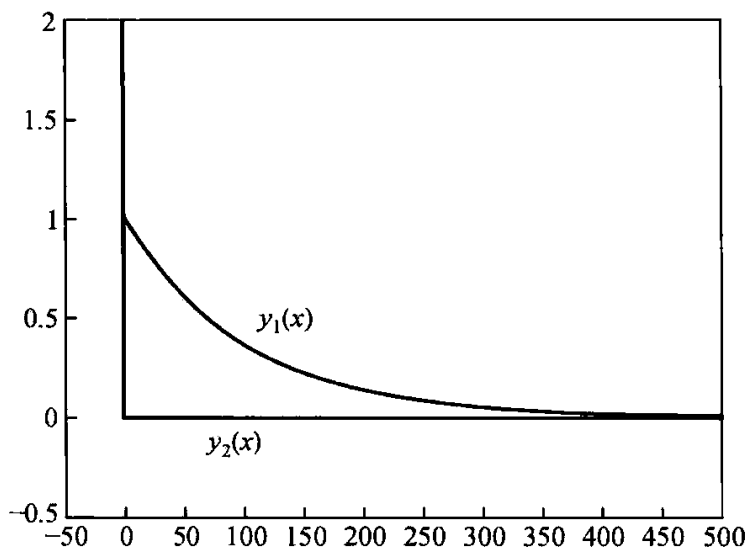
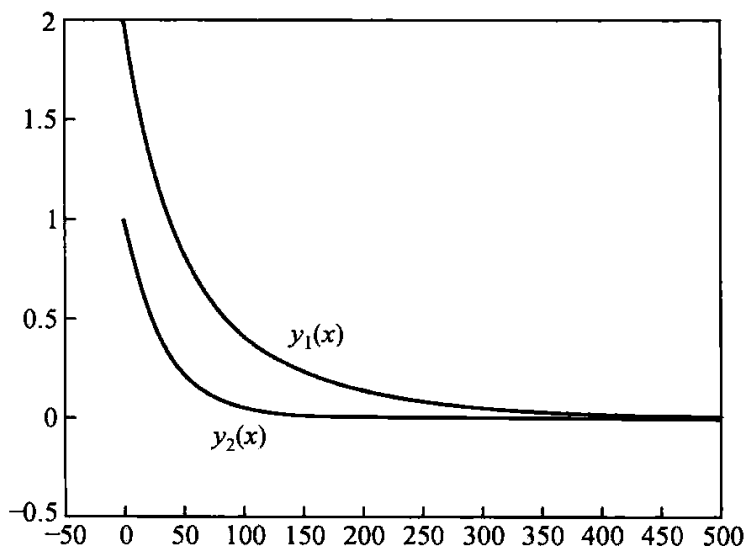
将微分方程(9-70)编为函数

```
function f = dyf(t,y)
f(1) = -0.01 * y(1) - 299.99 * y(2);
f(2) = -300 * y(2);
f = f(:);
```

取步长  $h = 0.003$ , 键入命令

```
>> [x,y] = RK12(@ dyf,[0 500],[2 1],0.001);
>> plot(x,y); axis([-50 500 -0.5 2])
```

得到较精确的计算结果,见图 9-5. 若取步长  $h = 0.006\ 67$ , 计算结果错误,见图 9-6; 若取步长  $h = 0.007$ , 计算将无法进行下去.

图 9-5 步长  $h = 0.003$ 图 9-6 步长  $h = 0.00667$ 

刚性微分方程的数值计算是一个复杂而有困难的过程,其他的一些数值计算方法可参阅[13].

## § 9.6 边值问题的数值解法

科学研究和工程计算中还会出现另外一类常微分方程求解问题.例如,二阶常微分方程

$$y'' = f(x, y, y'), \quad a < x < b \quad (9-75)$$

在自变量  $x$  的两个不同点处给出解  $y(x)$  的条件,这类问题称为边值问题,所给

条件称为边界条件. 方程式(9-75)的边界条件有三种:

$$\text{第一边界条件: } y(a) = \alpha, y(b) = \beta \quad (9-76a)$$

$$\text{第二边界条件: } y'(a) = \alpha, y'(b) = \beta \quad (9-76b)$$

$$\text{第三边界条件: } y(a) + \alpha_0 y'(a) = \alpha, y(b) + \beta_0 y'(b) = \beta \quad (9-76c)$$

其中  $\alpha, \beta, \alpha_0, \beta_0$  均为常数. 求解边值问题的方法很多, 本节简单介绍打靶法和有限差分法.

为简便起见, 本节专门讨论二阶线性微分方程的边值问题

$$\begin{cases} y'' + p(x)y' + q(x)y = f(x), & a < x < b \\ y(a) = \alpha, & y(b) = \beta \end{cases} \quad (9-77)$$

### 9.6.1 打靶法

打靶法可用于求解高阶线性或非线性常微分方程, 它的基本思路是: 将边值问题化为初值问题, 用前面介绍的求解初值问题的各种方法计算近似解, 适当调整初值条件, 使得近似解逼近给定的边界条件. 这种方法很像瞄准目标打靶, 故称为打靶法或试射法. 具体求解步骤如下:

(1) 先预测  $y(x)$  的导数在点  $x = a$  处的值  $y'(a) = t_1$ , 将边值问题(9-77)化为初值问题

$$\begin{cases} y'' + p(x)y' + q(x)y = f(x), & a < x < b \\ y(a) = \alpha, & y'(a) = t_1 \end{cases} \quad (9-78)$$

解初值问题(9-78), 得数值解  $y_1(x)$ .

(2) 比较  $y_1(b) = \beta_1$  与  $\beta$ , 若  $\beta_1 = \beta$  或  $|\beta_1 - \beta| < \varepsilon$ , 则  $y_1(x)$  即为所求的数值解.

(3) 若不是(2)的情况, 修改  $y'(a)$  的值. 例如, 取  $y'(a) = t_2 = \frac{\beta}{\beta_1} t_1$ , 用  $t_2$  代替方程(9-78)中的  $t_1$ , 再解初值问题(9-78)得解  $y_2(x)$ .

(4) 再比较  $y_2(b) = \beta_2$  与  $\beta$ , 若  $\beta_2 = \beta$  或  $|\beta_2 - \beta| < \varepsilon$ , 则  $y_2(x)$  即为所求的数值解; 否则继续修改  $t_2$ , 直至  $\beta_k = \beta$  或  $|\beta_k - \beta| < \varepsilon$ .

一般  $t_k$  可按公式

$$t_k = t_{k-1} - \frac{\beta_{k-1} - \beta}{\beta_{k-1} - \beta_{k-2}}(t_{k-1} - t_{k-2}), \quad k = 2, 3, \dots \quad (9-79)$$

逐步修正.

### 9.6.2 有限差分法

有限差分法的基本思想是将区间  $[a, b]$  剖分, 在各节点上用差商代替导数, 将微分方程离散成差分方程, 求解差分方程组, 从而得到解在各节点上的近似

值. 具体方法如下:

将区间  $[a, b]$  作  $n$  等分, 则步长  $h = (b - a)/n$ , 节点  $x_i = a + ih (i = 0, 1, \dots, n)$ . 在内部节点  $x_i (i = 1, 2, \dots, n-1)$  上, 方程组 (9-77) 为

$$y''(x_i) + p(x_i)y'(x_i) + q(x_i)y(x_i) = f(x_i), i = 1, 2, \dots, n-1 \quad (9-80)$$

用中心差商近似导数

$$y'(x_i) = \frac{1}{2h} [y(x_{i+1}) - y(x_{i-1})] - \frac{h^2}{6} y^{(3)}(\xi_i), \quad x_{i-1} < \xi_i < x_{i+1}$$

$$y''(x_i) = \frac{1}{h^2} [y(x_{i+1}) - 2y(x_i) + y(x_{i-1}))] - \frac{h^2}{12} y^{(4)}(\eta_i), \quad x_{i-1} < \eta_i < x_{i+1}$$

将以上两式代入式 (9-80) 得

$$\frac{1}{h^2} [y(x_{i+1}) - 2y(x_i) + y(x_{i-1}))] + \frac{1}{2h} p(x_i) [y(x_{i+1}) - y(x_{i-1}))] + q(x_i)y(x_i) = f(x_i) + O(h^2) \quad i = 1, 2, \dots, n-1$$

省略无穷小量  $O(h^2)$ , 分别以  $p_i, q_i, f_i, y_i$  代替  $p(x_i), q(x_i), f(x_i), y(x_i)$ , 若取边界条件  $y(a) = \alpha, y(b) = \beta$ , 得差分方程组

$$\begin{cases} (-2 + h^2 q_1) y_1 + \left(1 + \frac{h}{2} p_1\right) y_2 = h^2 f_1 - \left(1 - \frac{h}{2} p_1\right) \alpha \\ \left(1 - \frac{h}{2} p_i\right) y_{i-1} + (-2 + h^2 q_i) y_i + \left(1 + \frac{h}{2} p_i\right) y_{i+1} = h^2 f_i, \quad i = 2, 3, \dots, n-2 \\ \left(1 - \frac{h}{2} p_{n-1}\right) y_{n-2} + (-2 + h^2 q_{n-1}) y_{n-1} = h^2 f_{n-1} - \left(1 + \frac{h}{2} p_{n-1}\right) \beta \end{cases} \quad (9-81)$$

式 (9-81) 是三对角线性方程组, 可用追赶法求解. 其他边界条件也可得到形式相近的线性方程组.

可以证明: 若解函数  $y(x)$  充分光滑, 则差分格式 (9-81) 是二阶方法, 即

$$\max_{0 \leq i \leq n} |y(x_i) - y_i| = O(h^2)$$

例 7 试分别取  $n = 10, 20, 40$  用差分法求解边值问题

$$\begin{cases} y'' + \frac{x}{1-x} y' - \frac{1}{1-x} y = x - 1 \\ y(0) = 1, y(1) = 2e \end{cases}$$

并将计算结果与精确解  $y(x) = 2e^x + 2x - x^2 - 1$  进行比较.

[解] 取  $n = 10, h = 0.1$ , 式 (9-81) 为

$$\begin{bmatrix}
 -2.010\ 0 & 1.000\ 0 & & & & & & & \\
 0.994\ 4 & -2.011\ 1 & & & & & & & \\
 & 0.987\ 5 & & & & & & & \\
 & & \ddots & & & & & & \\
 & & & \ddots & & & & & \\
 & & & & \ddots & & & & \\
 & & & & & 1.116\ 7 & & & \\
 & & & & & 0.883\ 3 & -2.033\ 3 & 1.200\ 0 & \\
 & & & & & 0.800 & -2.050\ 0 & & 
 \end{bmatrix}
 \begin{bmatrix}
 y_1 \\
 y_2 \\
 y_3 \\
 y_4 \\
 y_5 \\
 y_6 \\
 y_7 \\
 y_8 \\
 y_9
 \end{bmatrix}
 =
 \begin{bmatrix}
 -0.010\ 0 \\
 -0.009\ 0 \\
 -0.008\ 0 \\
 -0.007\ 0 \\
 -0.006\ 0 \\
 -0.005\ 0 \\
 -0.004\ 0 \\
 -0.003\ 0 \\
 -0.002\ 0
 \end{bmatrix}$$

解方程组即可得近似解,计算结果见表 9-7. 根据表 9-7 中数据知:当  $h$  越来越小时,数值解的精度也在逐步提高.

表 9-7

| $x_n$ | $y_n(h=0.1)$ | $y_n(h=0.05)$ | $y_n(h=0.025)$ | $y(x_n)$ |
|-------|--------------|---------------|----------------|----------|
| 0.0   | 1.000 0      | 1.000 0       | 1.000 0        | 1.000 0  |
| 0.1   | 1.389 8      | 1.394 4       | 1.397 2        | 1.400 3  |
| 0.2   | 1.783 4      | 1.792 0       | 1.797 1        | 1.802 8  |
| 0.3   | 2.183 5      | 2.195 0       | 2.201 9        | 2.209 7  |
| 0.4   | 2.592 7      | 2.606 3       | 2.614 5        | 2.623 6  |
| 0.5   | 3.014 2      | 3.028 8       | 3.037 5        | 3.047 4  |
| 0.6   | 3.451 3      | 3.465 8       | 3.474 4        | 3.484 2  |
| 0.7   | 3.907 8      | 3.920 8       | 3.928 6        | 3.937 5  |
| 0.8   | 4.387 7      | 4.397 9       | 4.404 1        | 4.411 1  |
| 0.9   | 4.895 6      | 4.901 6       | 4.905 1        | 4.909 2  |
| 1.0   | 5.436 6      | 5.436 6       | 5.436 6        | 5.436 6  |

有限差分法的其他格式以及各种格式的稳定性、收敛性等问题可参阅偏微分方程数值解的相关教材.

## § 9.7 应用实例

### 9.7.1 MATLAB 求解微分方程

MATLAB 中提供了功能强大的求解常微分方程的符号运算函数 `dsolve`. `dsolve` 函数可以确定简单常微分方程的通解和带初值条件的特解,也可计算线性齐次或非齐次常微分方程组的通解和特解.

**例 8** 求微分方程  $\begin{cases} x^2 y'' - xy' + y = 6x \ln x \\ y(1) = 1, y'(1) = 2 \end{cases}$  的特解.

[解] 键入命令

```
>> y = dsolve('x^2 * D2y - x * Dy + y = 6 * x * log(x)', 'y(1) = 1, Dy(1) = 2', 'x')
```

得到解

$$y = x + x * \log(x) + \log(x)^3 * x$$

即解为  $y = x(1 + \ln x + (\ln x)^3)$ .

**例 9** 求微分方程组  $\begin{cases} x''(t) + 2x'(t) = x(t) + 2y(t) - e^{-t} \\ y'(t) = 4x(t) + 3y(t) + 4e^{-t} \end{cases}$  的通解.

[解] 键入命令

```
>> syms t x y;
>> [x,y] = dsolve('D2x + 2 * Dx = x + 2 * y - exp(-t)', 'Dy = 4 * x + 3 * y + 4 * exp(-t)', 't')
```

得到解

$$\begin{aligned} x &= -6 * t * \exp(-t) + C1 * \exp(-t) + C2 * \exp((1 + 6^{(1/2)}) * t) + \\ &C3 * \exp((1 - 6^{(1/2)}) * t) \\ y &= \\ &6 * t * \exp(-t) - C1 * \exp(-t) + 4 * C2 * \exp((1 + 6^{(1/2)}) * t) \\ &+ 2 * C2 * \exp((1 + 6^{(1/2)}) * t) * 6^{(1/2)} \\ &+ 4 * C3 * \exp((1 - 6^{(1/2)}) * t) - 2 * C3 * \exp((1 - 6^{(1/2)}) * t) * \\ &6^{(1/2)} + 1/2 * \exp(-t) \end{aligned}$$

即通解为

$$\begin{cases} x(t) = -6te^{-t} + C_1 e^{-t} + C_2 e^{(1+\sqrt{6})t} + C_3 e^{-(1+\sqrt{6})t} \\ y(t) = 6te^{-t} - C_1 e^{-t} + 2(2 + \sqrt{6})C_2 e^{(1+\sqrt{6})t} + \\ \quad 2(2 - \sqrt{6})C_3 e^{-(1+\sqrt{6})t} + \frac{1}{2}e^{-t} \end{cases}$$

对于复杂的常微分方程(组),dsolve 函数无能为力,或求得的符号解含有递推和特殊函数,难以解读,只能求数值解.数值求解微分方程的库函数有

ode23():低阶显式 Runge-Kutta 方法,低精度,能解一般非刚性问题;

ode45():变步长高阶显式 Runge-Kutta 方法,高精度,适用于大部分非刚性问题;

ode113():变阶数多步法,适用于误差要求严格的非刚性问题;

ode23s():低精度,可求解刚性问题;

ode23t():梯形公式,低精度,可求解适度刚性问题

这些函数调用的基本格式为

$$[x, y] = \text{ode}('fun', [x0 \ xn], [y0])$$

其中 fun 是给定微分方程的设定函数;[x0 xn]是自变量的变化区间,默认自适应的计算步长;[y0]是初值,对于微分方程组,[y0]应是一个向量.

**例 10** 设有微分方程组  $\begin{cases} x_1' \sin x_1 + x_2' \cos x_2 + x_1 = 1 \\ -x_1' \cos x_2 + x_2' \sin x_1 + x_2 = 0 \end{cases}$ , 且  $x_1(0) = x_2(0) = 0$ , 求方程的数值解.

[解] 设  $x = [x_1, x_2]$ , 原方程组可写为矩阵形式  $A(x)x' = B(x)$ , 其中

$$A(x) = \begin{bmatrix} \sin x_1 & \cos x_2 \\ -\cos x_2 & \sin x_1 \end{bmatrix}, \quad B(x) = \begin{bmatrix} 1 - x_1 \\ -x_2 \end{bmatrix}$$

假设  $A(x)$  非奇异, 则有  $x' = A^{-1}(x)B(x)$ , 先设置方程右端函数:

```
function dx = fun98(t, x)
A = [sin(x(1)) cos(x(2)); -cos(x(2)) sin(x(1))];
B = [1 - x(1); -x(2)];
dx = inv(A) * B;
```

再在命令窗口键入:

```
>> opt = odeset; opt. RelTol = 1e - 6;
>> [t, x] = ode45('fun98', [0 10], [0;0], opt);
plot(t, x)
```

计算结果如图 9-7.

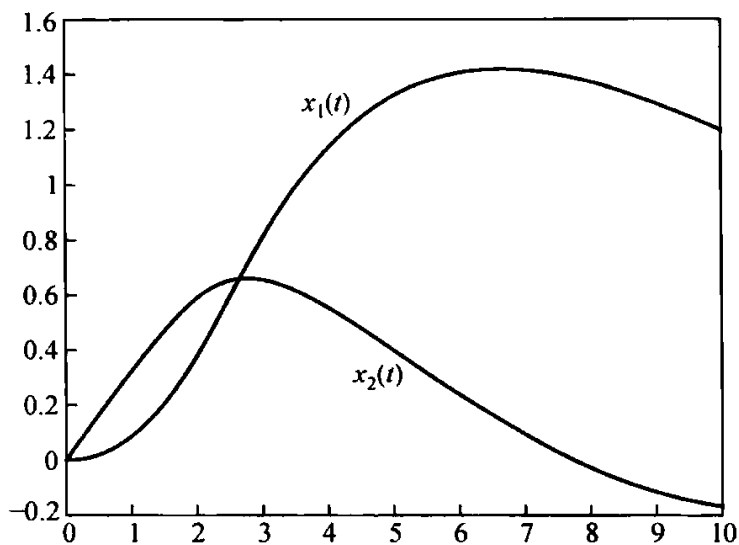


图 9-7

### 9.7.2 渡船的航行路线问题

小船渡河,河宽为  $a$ ,小船的航行始终对着出发对岸的码头. 设小船以匀速  $v_1$  航行,河水流速为常量  $v_2$ ,求小船航行的路线.

取坐标系如图 9-8,码头为坐标原点  $O$ ,初始时刻小船位于点  $A(a,0)$  处. 在时刻  $t$ ,小船位于  $P(x,y)$  处. 设小船的航行路线为  $y=y(x)$ ,其实际航行速度为  $\boldsymbol{v}=\boldsymbol{v}_1+\boldsymbol{v}_2$ ,其中  $\boldsymbol{v}_1$  表示船速,  $\boldsymbol{v}_2$  表示水流速. 航行方向为曲线  $y=y(x)$  在点  $P$  处的切线方向.

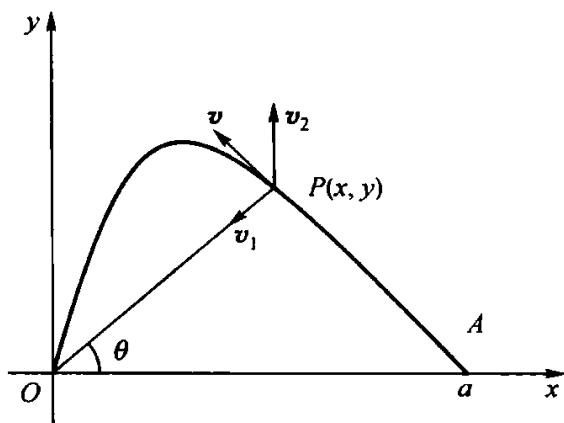


图 9-8

在  $x$  轴和  $y$  轴正向分解向量,得微分方程组



$$\begin{cases} \frac{dx}{dt} = -v_1 \cos \theta \\ \frac{dy}{dt} = v_2 - v_1 \sin \theta \end{cases}$$

消去时间参数  $t$ , 得方程

$$\frac{dy}{dx} = \tan \theta - k \frac{1}{\cos \theta}$$

其中  $k = \frac{v_2}{v_1}$ . 由  $\tan \theta = \frac{y}{x}$ ,  $\cos \theta = \frac{x}{\sqrt{x^2 + y^2}}$ , 得初值问题

$$\begin{cases} \frac{dy}{dx} = \frac{y}{x} - k \frac{\sqrt{x^2 + y^2}}{x} \\ y|_{x=a} = 0 \end{cases}$$

精确解为

$$y = \frac{a}{2} \left[ \left( \frac{x}{a} \right)^{1-k} - \left( \frac{x}{a} \right)^{1+k} \right]$$

为便于数值计算, 作变量替换  $z = -x$ , 问题化为

$$\begin{cases} \frac{dy}{dz} = \frac{y}{z} - k \frac{\sqrt{z^2 + y^2}}{z} \\ y|_{z=-a} = 0, \quad z \in [-a, 0] \end{cases}$$

采用改进的 Euler 方法和四阶经典 RK 方法, 用 MATLAB 编程数值模拟小船的航行路径. 取  $a = 10\text{m}$ ,  $n = 100$ ,  $k = 0.5, 1, 1.5$  计算.

当  $k < 1$ ,  $v_1 > v_2$  时, 若  $x = 0$  时, 有  $y = 0$ , 即小船可以到达点  $O$ . 两种数值解和精确解如图 9-9 所示, 数值解显示的路径与精确解曲线完全重合, 且误差小于  $10^{-5}$  或  $10^{-7}$ .

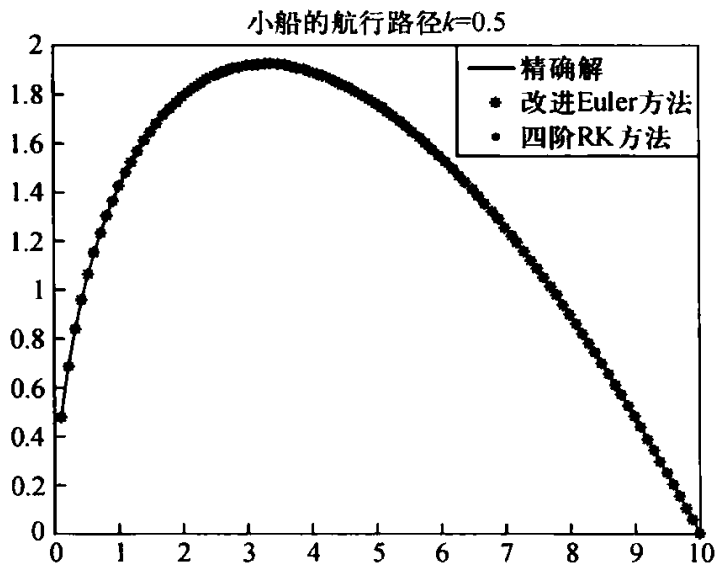


图 9-9

当  $k=1, v_1=v_2$  时, 若  $x=0$  时, 有  $y=5$ , 即小船可以到河岸的另一边, 但不能到达点  $O$ . 航行路径如图 9-10 所示.

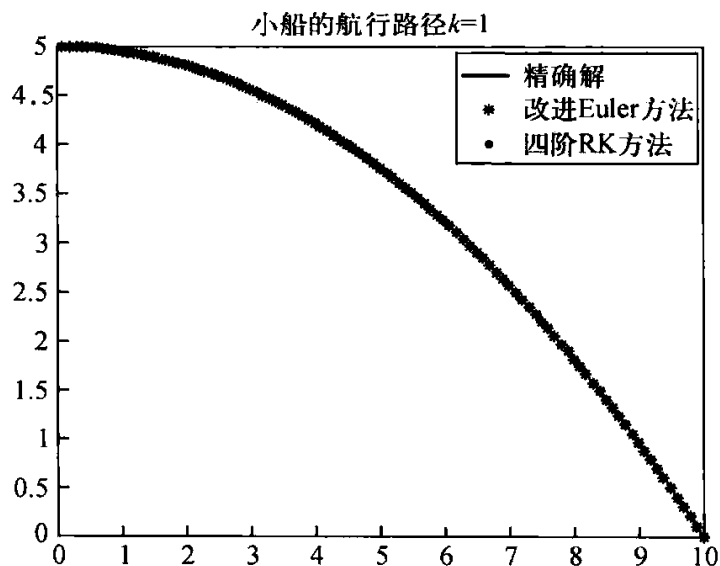


图 9-10

当  $k>1, v_1<v_2$  时, 若  $x\rightarrow 0$  时, 有  $y\rightarrow\infty$ , 即小船被河水冲向远处, 不可能在有限时间内到达对岸, 见图 9-11.

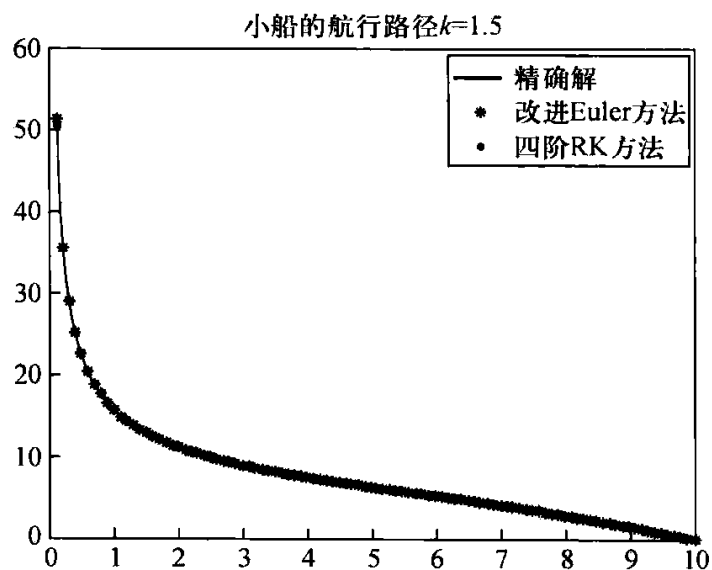


图 9-11

### 9.7.3 悬臂梁的挠度分析

悬臂梁在各种荷载及不同支撑下的挠度分析是建筑工程中常见的问题. 例如考虑如图 9-12 的一段被嵌入墙内的悬臂梁, 在固定点  $A$  处, 位移  $y$  和斜度  $\frac{dy}{dz}$  皆为零.

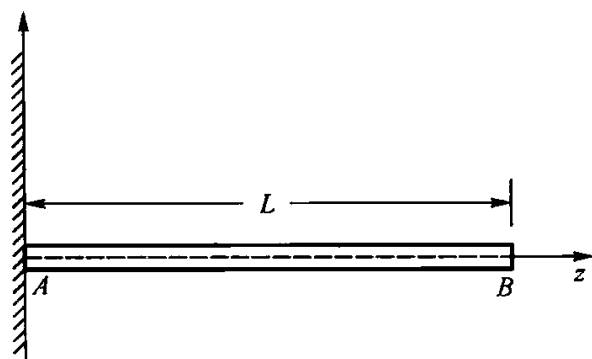


图 9-12

设梁是一根均匀细杆,其长度为  $L$ . 梁关于垂直方向  $y$  的挠度满足方程

$$JE \frac{d^2 y}{dz^2} = \rho g \left( 1 + \frac{dy}{dz} \right)^{3/2} \left[ z \left( z - \frac{L}{2} \right) + \frac{L^2}{2} \right]$$

其中  $J$  是梁的横截面关于其主轴的惯性力矩,  $E$  是 Young 模量, 常数  $\rho$  是梁的线密度,  $g$  是重力加速度. 方程的边界条件是  $y|_{z=0} = 0$ ,  $\frac{dy}{dz}|_{z=0} = 0$ . 引入状态变量

$x = \left[ y, \frac{dy}{dz} \right]^T$ , 上述二阶非线性边值问题转化为一阶微分方程组

$$\begin{cases} \frac{dx_1}{dz} = x_2 \\ \frac{dx_2}{dz} = \frac{\rho g}{JE} (1 + x_2^2)^{3/2} \left[ z \left( z - \frac{L}{2} \right) + \frac{L^2}{2} \right] \end{cases}$$

选取参数  $L = 2$  m, 线密度  $\rho = 10$  kg/m, 惯性力矩与 Young 模量的乘积  $JE = 2400$  kg·m<sup>3</sup>/s<sup>2</sup>. 将区间  $[0, 2]$  分成 100 等份, 采用经典四阶 RK 公式计算. 计算结果见图 9-13, 最大挠度在端点  $B$  出现,  $y(2) = 16.48$  cm.

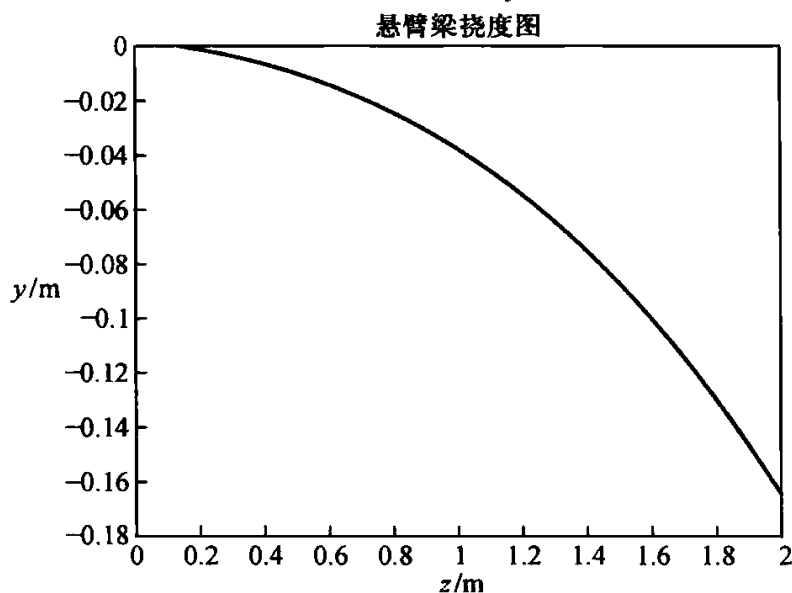


图 9-13

用 RK 法求解微分方程组的程序如下：

```
function RK4 = RK4(a,b,m)
    x1 = zeros(m+1,1); x2 = zeros(m+1,1);
    f = inline('v','v');
    g = inline('(98.1/2400)*(1+v^2)^(1.5)*(u*(u-2/2)+4/2)','v','u');
    h = 2/m;
    z = a:h:b;
    x1(1) = 0.0; x2(1) = 0.0;
    for n = 1:m
        k11 = f(x2(n));
        k21 = g(x2(n),z(n));
        k12 = f(x2(n) + h * k21/2);
        k22 = g(x2(n) + h * k21/2, z(n) + h/2);
        k13 = f(x2(n) + h * k22/2);
        k23 = g(x2(n) + h * k22/2, z(n) + h/2);
        k14 = f(x2(n) + h * k23);
        k24 = g(x2(n) + h * k23, z(n) + h);
        x1(n+1) = x1(n) + h * (k11 + 2 * k12 + 2 * k13 + k14)/6;
        x2(n+1) = x2(n) + h * (k21 + 2 * k22 + 2 * k23 + k24)/6;
    end;
    plot(z, -x1, 'k-');
    title('悬臂梁挠度图')
    xlabel('z/m'); ylabel('y/m');
```

#### 9.7.4 锁相环路的频率

锁相环路是电信工程中最常用的电路,它是一个非线性的反馈系统,可以检测和追踪输入的周期信号频率的变化. 实际生活中最常见的周期信号就是调频无线电信号. 例如,无线电信号的输入频率是  $f_0$  Hz,  $m(t)$  表示无线电波传输的音乐或语音,那么含有声音信息的调频无线电波可以表示为

$$z(t) = a \sin(2\pi[f_0 + m(t)]t)$$

接收信号的频率是  $f(t) = f_0 + m(t)$  (Hz), 振幅为  $a$ . 由于经过长距离的传输,信号损失严重,振幅会变得很小. 如果经传输所得的信号频率  $f(t)$  经过调频,音乐或语音信号  $m(t)$  可得到恢复、增强,从而在接收端还原音乐或语音信号.

设锁相环路如图 9-14, 其中  $u(t)$  是周期输入信号频率,  $y(t)$  是输出信号频率,  $\alpha$  是增强因子,  $\tau$  是时间常数,  $\omega_c$  是环路的中心频率. 电路的运行情况可表示为方程组

$$\begin{cases} \frac{dx_1}{dt} = \frac{\alpha u \cos x_2 - x_1}{\tau} \\ \frac{dx_2}{dt} = x_2 + \omega_c \\ y = \frac{x_1 + \omega_c}{2\pi} \end{cases}$$

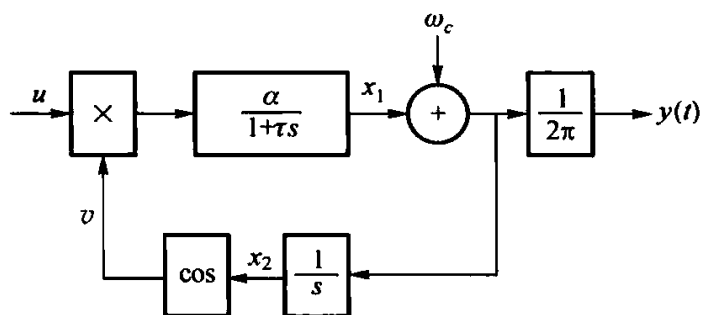


图 9-14

设输入信号  $u(t) = a \sin(\omega_0 t)$ , 如果  $\omega_0$  充分接近锁相环路的中心频率  $\omega_c$ , 锁相环路就被锁定到这一周期信号上. 锁相过程的实现与信号强度  $a$  和增强因子  $\alpha$  有关, 实际上, 当  $|\omega_0 - \omega_c| < a\alpha$  时, 锁相过程即可实现. 我们可采用经典四阶 RK 法求解上述常微分方程组, 进一步数值模拟锁相过程. 选取参数  $\omega_c = 2000\pi$  rad/s,  $\alpha = 1600$ ,  $\tau = 1.5$  msec, 输入信号振幅  $a = 0.5$ , 输入频率  $\omega_0 = 2100\pi$  rad/s, 并设初值条件  $x_1 = 0$ ,  $x_2 = 0$ ,  $y(0) = 1000$  Hz. 设时间区间  $0 \leq t \leq 25$  msec, 将其 1000 等分开始计算. 由数值模拟图 9-15, 我们发现大约 10 msec 时, 锁相成功实现.

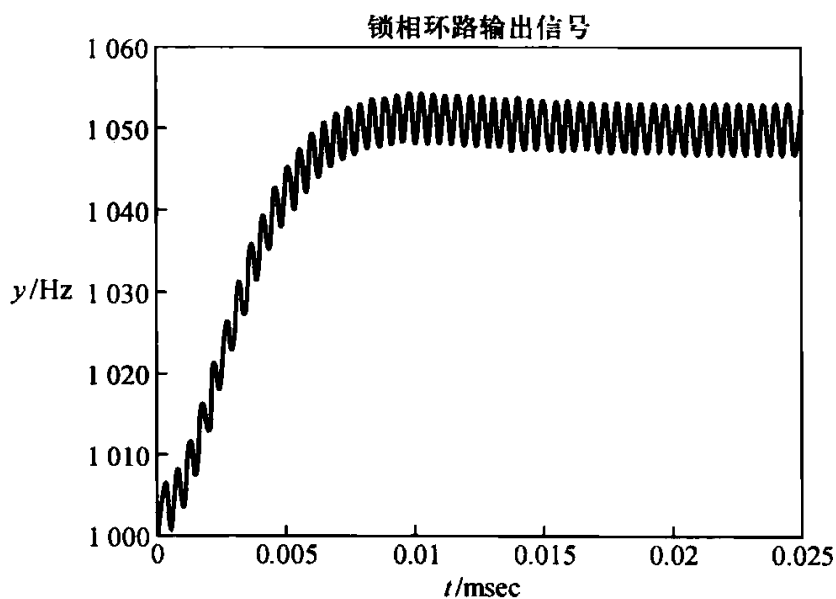


图 9-15

数值模拟锁相过程程序如下:

```
function PLL = PLL(t0,t1,n)
    h = (t1 - t0)/n;
    x1 = zeros(n + 1,1); x2 = zeros(n + 1,1); y = zeros(n + 1,1);
    t = t0:h:t1;
    f = inline('(1 600 * 0.5 * sin(2 * pi * 1 050 * t) * cos(x2) - x1)/(3/2 000)','t','x1','x2');
    g = inline('x1 + 2 * pi * 1 000','x1');
    y(1) = 1 000;
    for i = 1:n
        k11 = f(t(i),x1(i),x2(i));
        k21 = g(x1(i));
        k12 = f(t(i) + h/2,x1(i) + h * k11/2,x2(i) + h * k21/2);
        k22 = g(x1(i) + h * k11/2);
        k13 = f(t(i) + h/2,x1(i) + h * k12/2,x2(i) + h * k22/2);
        k23 = g(x1(i) + h * k12/2);
        k14 = f(t(i) + h/2,x1(i) + h * k13/2,x2(i) + h * k23/2);
        k24 = g(x1(i) + h * k13/2);
        x1(i + 1) = x1(i) + h * (k11 + 2 * k12 + 2 * k13 + k14)/6;
        x2(i + 1) = x2(i) + h * (k21 + 2 * k22 + 2 * k23 + k24)/6;
        y(i + 1) = (x1(i + 1) + 2 000 * pi)/2/pi;
    end;
    plot(t,y,'k-');
    title('锁相环路输出信号')
    xlabel('t(msec)');
    ylabel('y(Hz)');
```

## 评 注

常微分方程初值问题的数值解法主要分单步法和多步法. Euler 公式、改进的 Euler 公式以及各阶 Runge-Kutta 方法都是单步法. 对于一般的初值问题, 通常采用改进的 Euler 公式, 就能保证二阶精度. 如果方程的右端项  $f(x, y)$  足够光滑, 计算精度要求较高, 经典 Runge-Kutta 法是不错的方法. 实际计算中常用的是变步长的 Runge-Kutta 法, 这样既能保证精度, 又能节省计算量. 如果计算  $f(x, y)$  代价较高, 可选用线性多步法或预测-校正公式. 若要获得更高精度 (如误差小于  $10^{-10}$ ), 需要采用更高精度的计算公式, 再加上逐次分半步长或外推技巧. 收敛性、相容性和稳定性是微分方程数值解的重要概念, 实际计算中应先分析方

法的稳定性,选取步长  $h$  使得  $\lambda h$  落在稳定区域内,只有稳定的方法才能得到正确的结果.

对于微分方程组、刚性方程组以及边值问题,本章仅介绍了最基本的方法.

## 习 题 九

1. 用 Euler 方法求下列初值问题的数值解,取  $h = 0.1, N = 10$ , 计算结果保留 4 位小数,并与准确解进行比较:

$$(1) \begin{cases} y' = x + y, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

$$(2) \begin{cases} y' = y - \frac{2x}{y}, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

2. 用 Euler 方法求初值问题

$$\begin{cases} y' = 1 - \frac{2xy}{1+x^2}, & 0 \leq x \leq 2 \\ y(0) = 0 \end{cases}$$

的数值解,取  $h = 0.5$ , 计算结果取 5 位小数,并与精确解  $y(x) = \frac{x(3+x^2)}{3(1+x^2)}$  比较.

3. 用改进 Euler 法求下列问题的数值解,取  $h = 0.1$ , 计算结果保留 4 位小数:

$$(1) \begin{cases} y' = x + y, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

$$(2) \begin{cases} y' = y - \frac{2x}{y}, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

4. 分别用改进 Euler 法与四阶 RK 方法求初值问题

$$\begin{cases} y' = 1 - \frac{2xy}{1+x^2}, & 0 \leq x \leq 2 \\ y(0) = 0 \end{cases}$$

的数值解,取  $h = 0.5$ , 计算结果取 5 位小数,并与第 2 题的计算结果作比较.

5. 用 Euler 方法及改进 Euler 方法计算积分

$$I(x) = \int_0^x e^{t^2} dt$$

在  $x = 1$  处的近似值,取  $h = 0.1$ , 计算结果保留 4 位小数.

6. 用中点公式及四阶 RK 公式求初值问题

$$\begin{cases} y' = x + y \\ y(0) = 1 \end{cases}$$

的解  $y(x)$  在  $x=1$  处的近似值, 取  $h=0.2$ , 计算结果保留 4 位小数.

7. 用四阶 RK 方法求初值问题

$$\begin{cases} y' + y = 0, & 0 \leq x \leq 1 \\ y(0) = 1 \end{cases}$$

的数值解, 取  $h=0.1$ , 计算结果保留 4 位小数.

8. 选取参数  $a, b$ , 使下述形式的 RK 公式为二阶公式

$$\begin{cases} y_{n+1} = y_n + hK_2 \\ K_1 = f(x_n, y_n) \\ K_2 = f(x_n + ah, y_n + hbK_1) \end{cases}$$

9. 分别用四阶 Adams 显式和隐式公式求初值问题

$$\begin{cases} y' = -xy^2, & 0 \leq x \leq 1 \\ y(0) = 2 \end{cases}$$

的数值解, 取  $h=0.2$ .

10. 试证二步公式

$$y_{n+1} = \frac{1}{2}(y_n + y_{n-1}) + \frac{h}{4}(4f_{n+1} - f_n + 3f_{n-1}) \quad (n = 1, 2, \dots)$$

是二阶方法, 并估计其局部误差.

11. 设有求解常微分方程初值问题

$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

的二步公式

$$y_{n+1} = y_n + \frac{h}{12}(5f_{n+1} + 8f_n - f_{n-1})$$

试求该公式的局部截断误差, 并回答该方法是几阶精度的.

12. 试用 Taylor 展开构造形如

$$y_{n+1} = y_{n-3} + h(\alpha f_n + \beta f_{n-1})$$

的四步公式, 使它具有尽可能高的阶, 并写出其局部截断误差.

13. 对初值问题

$$\begin{cases} y' = y \\ y(0) = 1 \end{cases}$$

用改进 Euler 法求近似解的表达式, 并证明当步长  $h$  趋于 0 时, 近似解收敛于精确解  $y = e^x$ .

14. 试求向后 Euler 方法的绝对稳定区域.



## 15. 对初值问题

$$\begin{cases} y' = -1000(y - x^2) + 2x, & 0 \leq x \leq 1 \\ y(0) = 0 \end{cases}$$

用 Euler 方法求数值解, 分别取步长为  $h = 0.1, h = 0.01$ , 将所得结果与准确解  $y = x^2$  比较, 并用稳定性条件说明比较结果.

## 16. 用改进 Euler 法求问题

$$\begin{cases} y'' - 2y' + 2y = e^x \sin x, & 0 \leq x \leq 1 \\ y(0) = 0, y'(0) = -\frac{1}{2} \end{cases}$$

的数值解, 取  $h = 0.1$ .

## 17. 用四阶 RK 方法求解第 14 题中问题, 并比较结果.

## 18. 写出用四阶 RK 方法求解问题

$$\begin{cases} y''' + 2y'' - y' - 2y = e^x, & 0 \leq x \leq 3 \\ y(0) = 1, y'(0) = 2, y''(0) = 0 \end{cases}$$

的计算公式, 取  $h = 0.2$ .

19. 取  $t_0 = 2$ , 用打靶法求两点边值问题

$$\begin{cases} y'' - (y')^2 + x = 0 \\ y(0) = 1, y(0.4) = 1.8 \end{cases}$$

要求误差小于  $\varepsilon = 0.5 \times 10^{-4}$ .

20. 取步长  $h = 0.1$ , 用有限差分法求解两点边值问题

$$\begin{cases} -y'' + y = (\pi^2 + 1) \sin(\pi x) \\ y(0) = 0, y(1) = 0 \end{cases}$$

## 数值实验

## 1. 设有常微分方程初值问题

$$\begin{cases} y' = -y + 2\cos x, & 0 < x < \pi \\ y(0) = 1 \end{cases}$$

其精确解为  $y = \cos x + \sin x$ . 选取步长  $h$  使四阶 Adams 预测 - 校正算法和经典 RK 法均稳定, 分别用这两种方法求解微分方程, 将数值解与精确解进行比较, 输出结果. 其中多步法需要的初值由经典 RK 法提供.

## 2. 考虑 Rossler 化学反应方程

$$\begin{cases} x' = -y - z \\ y' = x + ay, & x(0) = y(0) = z(0) = d \\ z' = (x - c)z + b \end{cases}$$

将  $a, b, c, d$  设为参数, 编程求数值解. 取  $a = 0.2, b = 0.8, c = 4.5, d = 1$ , 绘制数值解的三维相轨迹, 并作出在  $xOy$  平面上的投影.

3. 选择状态函数, 将高阶非线性微分方程

$$\begin{cases} x'' - 2xzx' = 3x^2yt^2 \\ y'' - e^y y' = 4xzt^2 \\ z'' - 2tz' = 2te^{xy} \end{cases} \quad \begin{aligned} x(1) &= y(1) = z(1) = 3 \\ x'(1) &= y'(1) = z'(1) = 2 \end{aligned}$$

转化为一阶显式微分方程组, 编程求其数值解, 并绘制出解的相空间曲线.

4. 考虑反应速度方程组

$$\begin{cases} y_1' = -0.04y_1 + 10^4 y_2 y_3 \\ y_2' = 0.04y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \\ y_3' = 3 \times 10^7 y_2^2 \end{cases}$$

初值为  $y_1(0) = 1, y_2(0) = y_3(0) = 0$ . 这是一个刚性方程组, 试用 `ode45`( ) 计算, 结果如何? 如不能得到较精确的解应该如何处理?

5. 方程

$$x^2 y'' - x(x+2)y' + (x+2)y = 0$$

的通解为  $y = C_1 x + C_2 x e^x$ , 若边界条件为  $y(1) = e, y(2) = 2e^2$ , 确定方程的特解. 取  $h = 0.1, 0.05, 0.025$ , 用差分法编程求解该边值问题, 并研究步长  $h$  减半时误差的变化情况.

# 附录 MATLAB 数学软件简介

数学软件是以科学与工程计算为主要目的. 早期(20 世纪 70 年代)的数学软件多以软件包的形式出现, 它的特点是程序具有开放性, 但功能比较单一、用户界面简单、要求使用者熟悉相应的计算机高级语言. 另一类数学软件称为软件平台或软件工具, 它是近三十年发展起来的, 并迅速受到科学工作者和工程技术人员欢迎. 数学软件平台的优点是用户界面友好、功能较齐全而且使用者可以基本不了解软件的内部结构及编程. 目前, 较成熟和流行的数学软件工具有 MATLAB、Mathematica 和 Maple 等. MATLAB 软件特别适合矩阵运算和数值计算, 对处理大批数据效率较高. Mathematica 和 Maple 软件以符号运算见长, 但处理大量数据时效率较低.

MATLAB 是 Matrix Laboratory 的英文缩写, 意为矩阵实验室. 最初, 作为与 Linpack 和 Eispack 矩阵软件包的接口程序, MATLAB 是用 Fortran 语言编写的矩阵分析软件. 经过多年的开发, 它已具有强大的数值计算、图形绘制、数据处理以及符号运算功能, 成为适合多学科、多种工作的大型软件平台. 在欧美等发达国家的一些高校, MATLAB 已成为线性代数、自动控制理论、概率论、数理统计、时间序列分析、数字信号处理和动态系统仿真等课程的主要教学工具; 成为本科生、硕士和博士生必须掌握的基本技能.

## § 1 MATLAB 的基本功能

安装完 MATLAB 软件之后, 在桌面会自动生成 MATLAB 图标. 双击 MATLAB 图标, 即见到命令操作窗口, MATLAB 的一般计算和基本控制是通过命令窗口实现的. 此外, 左上端有一个工作空间窗口, 显示当前建立的所有变量的信息; 左下端有一个用来记录已使用过的历史命令窗口. 绘图时会自动生成图形窗口. 命令操作窗口的顶端是工具栏, 常用的有: file 实现各种文件操作; help 在线帮助, 以便了解 MATLAB 更多的信息.

### 1.1 基本数据格式、结构和输入方式

MATLAB 中的所有计算都是通过双精度量进行的, 在内存中的数据精度是

双精度的,但其显示格式可以不同.在命令窗口中,键入常用的命令 `format` 可以实现数据的不同显示格式之间的切换.`format` 命令的主要用法有:

附表 1

| 命令                          | 格式                                  |
|-----------------------------|-------------------------------------|
| <code>format</code>         | 默认值,数据显示格式与 <code>short</code> 格式相同 |
| <code>format short</code>   | 短格式,仅显示 5 位有效数字                     |
| <code>format long</code>    | 长格式,显示 15 位有效数字                     |
| <code>format short e</code> | 对任意小数,显示仅带 5 位有效数字和 3 位指数的数据        |
| <code>Format long e</code>  | 对任意小数,显示带 15 位有效数字和 3 位指数的数据        |

MATLAB 中的数据格式是矩阵,单个变量默认为是  $1 \times 1$  的矩阵.变量名是以字母开头,由字母、数字和下划线组成的字符串,字母的大小写是有区别的.MATLAB 中的特殊变量有:

附表 2

| 特殊变量名            | 说明                       |
|------------------|--------------------------|
| <code>pi</code>  | $\pi$ 值                  |
| <code>eps</code> | 机器极小值 $= 2.220\ 4e - 16$ |
| <code>ans</code> | 默认的代表计算结果的变量名            |
| <code>inf</code> | 无穷大                      |
| <code>NaN</code> | 不定量,如分母出现零               |
| <code>i,j</code> | 虚数符号 $i=j=\sqrt{-1}$     |

生成矩阵的方式有多种,生成小型矩阵的常用方法是直接从命令窗口输入.整个矩阵以“`[ ]`”作为首尾,矩阵的行与行之间用分号“`;`”相隔,每行中元素用逗号“`,`”或空格分隔.例如,在命令窗口键入:

```
>> a = [1 2 3;4 5 6;7 8 9]
```

命令窗口会显示结果:

```
a =
    1    2    3
    4    5    6
    7    8    9
```

表示变量  $a$  是一个三阶的方阵. 如果在上述命令的末尾加上“;”, 则同样生成矩阵  $a$ , 但并不在命令窗口中显示结果. 形成大规模矩阵的常用方法是使用 `load` 命令, 直接将数据文件读入. 对于按步长逐渐增加的一维数组, 有更简单的创建方式, 如

```
>> x = 0:0.1:10;
```

生成了从 0 开始, 每次递增 0.1, 直至 10 的 101 个数据构成的一位数组. 也可调用库函数 `linspace` 生成一维数组. 附表 3 列出了一些特殊函数, 以便形成常用的特殊矩阵.

附表 3

| 函数符号                    | 功能                          |
|-------------------------|-----------------------------|
| <code>zero(m, n)</code> | 生成 $m$ 行 $n$ 列的全零矩阵         |
| <code>ones(m, n)</code> | 生成 $m$ 行 $n$ 列的全 1 矩阵       |
| <code>eye(m, n)</code>  | 生成 $m$ 行 $n$ 列的对角线元素为 1 的矩阵 |
| <code>rand(m, n)</code> | 生成 $m$ 行 $n$ 列的随机矩阵         |

## 1.2 基本运算符

MATLAB 提供的常用运算符有:

附表 4

| 运算符 | 使用形式            | 功能                                        |
|-----|-----------------|-------------------------------------------|
| +   | $A + B$         | 执行矩阵加法运算, 要求 $A$ 、 $B$ 维数相同               |
| -   | $A - B$         | 执行矩阵减法运算, 要求 $A$ 、 $B$ 维数相同               |
| *   | $A * B$         | 执行矩阵相乘运算, 要求 $A$ 、 $B$ 维数满足矩阵相乘的条件        |
| /   | $B/A$           | 执行矩阵右除运算, 其值为矩阵 $B/A = BA^{-1}$           |
| \   | $A \setminus B$ | 执行矩阵左除运算, 其值为矩阵 $A \setminus B = A^{-1}B$ |
| .*  | $A .* B$        | 数组乘法, 运算结果为两个矩阵对应元素相乘, 要求 $A$ 、 $B$ 维数相同  |
| ./  | $B ./ A$        | 数组右除, 矩阵 $B$ 中各个元素除以矩阵 $A$ 中对应元素          |
| '   | $A'$            | 执行矩阵的转置                                   |

此外, 调用库函数 `inv(A)`、`det(A)`、`eig(A)`、`cond(A)` 和 `rank(A)`, 可分别得

到矩阵 A 的逆阵、行列式值、特征值、条件数和秩。

MATLAB 中的关系运算符和逻辑运算符有：

附表 5

| 关系运算符 | 说明   | 逻辑运算符 | 说明     |
|-------|------|-------|--------|
| ==    | 相等   | &     | 逻辑与运算  |
| ~=    | 不等于  |       | 逻辑或运算  |
| <     | 小于   | ~     | 逻辑非运算  |
| <=    | 小于等于 | xor   | 逻辑异或运算 |
| >     | 大于   |       |        |
| >=    | 大于等于 |       |        |

### 1.3 数学函数

MATLAB 所支持的部分常用的数学函数见附表 6。

附表 6

| 函数名     | 数学运算      | 函数名      | 数学运算        |
|---------|-----------|----------|-------------|
| sin(x)  | 正弦函数      | asin(x)  | 反正弦函数       |
| cos(x)  | 余弦函数      | acos(x)  | 反余弦函数       |
| tan(x)  | 正切函数      | atan(x)  | 反正切函数       |
| min(x)  | 最小值       | max(x)   | 最大值         |
| abs(x)  | 绝对值       | sqrt(x)  | 开平方         |
| log(x)  | 自然对数      | log10(x) | 以 10 为底的对数  |
| exp(x)  | 以 e 为底的指数 | sign(x)  | 符号函数        |
| real(x) | 求复数的实部    | imag(x)  | 求复数的虚部      |
| sum(x)  | 求所有元素的和   | round(x) | 四舍五入到最接近的整数 |
| fix(x)  | 向零方向取整数   | floor(x) | 向负方向取整数     |

MATLAB 的库函数是有限的,使用者可以自己编写函数,存为后缀为 m 的文件,或定义内联函数,之后即可像调用库函数一样随意调用。例如:将程序

```
function p = p(x,y)
p = sin(x^2 + y^2)/ exp(-x/2);
```

存为 p.m 文件之后,定义了二元函数

$$p(x, y) = \frac{\sin(x^2 + y^2)}{e^{-x/2}}$$

在命令窗口键入:

```
>> f = inline('30 + (1 - cos(pi * x))/2','x');
```

定义了函数:

$$f(x) = 30 + \frac{1 - \cos(\pi x)}{2}$$

## 1.4 控制语句

MATLAB 中的控制语句与其他语言中的类似,编写简洁,意义明确.

### (一) for 循环语句

例如

```
>> m = 10; n = 20;  
>> for i = 1:m  
    for j = 1:n  
        A(i,j) = 1/(i+j);  
    end  
end
```

通过循环语句,生成了 10 行 20 列的矩阵 A. 循环语句以 end 结束. 循环语句中的步长是可以选择的,如

```
>> for i = n:-1:-n  
    .....  
end;  
>> for x = 1:0.1:10  
    .....  
end;
```

都是允许的.

### (二) while 循环

程序

```
>> sum = 0;  
>> i = 1;  
>> while i <= 100  
    sum = sum + i;  
    i = i + 1;  
end;
```

利用 while 循环求得 1 ~ 100 之间所有整数的和.

### (三) 条件语句

条件语句的一般形式有

```
>> if x >= 0
    y = 1;
else y = -1;
end;
```

其中 else 部分可以缺省,也可嵌套使用.

## 1.5 帮助的使用

在使用 MATLAB 软件的过程中,往往会出现需要某个函数而又不了解该函数的具体用法的情况,一个简捷的方法就是利用 MATLAB 的在线查找功能. MATLAB 提供了非常完备的在线查找功能,了解和掌握在线查询,对运用软件是非常必要的. 查询系统的调用方式一般有两种:

(1) 从 MATLAB 的帮助窗口中获取帮助. 单击 help 或在命令窗口键入 helpwin 命令,可启动 help 窗口,逐层选择感兴趣的条目和函数,即可获得所需信息.

(2) 在命令窗口直接键入 help 命令,可获取需要的信息. 这种方式直截了当,获取信息简洁而迅速,最为常用. 例如,需要了解 plot 函数的用法,键入命令

```
>> help plot
```

立即得到关于函数 plot 的详细资料:

PLOT Linear plot.

PLOT(X,Y) plots vector Y versus vector X. If X or Y is a matrix, then the vector is plotted versus the rows or columns of the matrix, whichever line up. If X is a scalar and Y is a vector, length(Y) disconnected points are plotted.

.....

.....

也可在命令窗口中键入 doc、look for、exit、which、who、whos、what 等命令,对函数、文件、变量、和目录等信息进行查询. 读者可通过 help 命令了解这些命令的功能和使用方法.

## §2 绘图功能

MATLAB 提供了许多可以选用的图形工具,几乎满足了一般实际工程、科



学计算中的所有图形需要. 用户可以选择直角坐标系、极坐标系等不同的坐标系, 绘制平面曲线、空间曲线、空间网面图、空间表面图、直方图及向量图等. 这里仅介绍最基本的图形操作命令.

## 2.1 二维曲线绘图

`plot` 是最基本的图形命令, 一般使用格式为:

```
>> plot(x, y);
>> plot(x, y, s);
>> plot(x, y1, s1, x, y2, s2, x, y3, s3);
```

其中  $x, y, y1, y2, y3$  是一维数组. 执行命令 `plot(x, y)` 时, 以  $x$  的元素值为横坐标, 以  $y$  的元素值为纵坐标绘制连线图.  $s$  表示用户选定的线型和颜色. `plot(x, y1, s1, x, y2, s2, x, y3, s3)` 表示将多条曲线画在同一坐标系下. MATLAB 中可选用的线型和颜色选项见附表 7.

附表 7

| 线型选项 | 说明  | 颜色选项 | 说明  |
|------|-----|------|-----|
| -    | 实线  | r    | 红色  |
| --   | 虚线  | g    | 绿色  |
| :    | 点线  | b    | 蓝色  |
| -.   | 点划线 | y    | 黄色  |
| o    | 小圆圈 | k    | 黑色  |
| *    | 星号  | w    | 白色  |
| X    | 叉号  | c    | 青色  |
| +    | 加号  | m    | 洋红色 |
| S    | 小方块 |      |     |

MATLAB 还提供了一系列绘图辅助函数, 使得图形便于阅读. 常用的绘图辅助函数有:

附表 8

| 函数名                        | 功能               |
|----------------------------|------------------|
| <code>title('...')</code>  | 在图形的上方显示 '中指定的内容 |
| <code>xlabel('...')</code> | 将 '中指定的内容标在 x 轴上 |
| <code>ylabel('...')</code> | 将 '中指定的内容标在 y 轴上 |

续表

| 函数名                              | 功能                                          |
|----------------------------------|---------------------------------------------|
| <code>text(x,y,'...')</code>     | 将'中指定的内容显示在由 x,y 定义的位置上                     |
| <code>gtext('...')</code>        | 运行到该命令时,屏幕光标位置显示符号“+”等待,将'中指定的内容显示在鼠标指定的位置上 |
| <code>hold on</code>             | 保留现有图形窗口,并将随后的图形叠加在此图上                      |
| <code>hold off</code>            | 解除 hold on 命令,随后的 plot 命令将覆盖现有图形            |
| <code>Grid</code>                | 在图形上显示曲线网格                                  |
| <code>axis([x1 x2 y1 y2])</code> | 方括号中的四个数分别确定 x 和 y 方向显示的范围                  |

例如,下面一段程序实现了将区间 $[-1,1]$ 上前四个 Chebyshev 多项式

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_2(x) = 2x^2 - 1$$

$$T_3(x) = 4x^3 - 3x$$

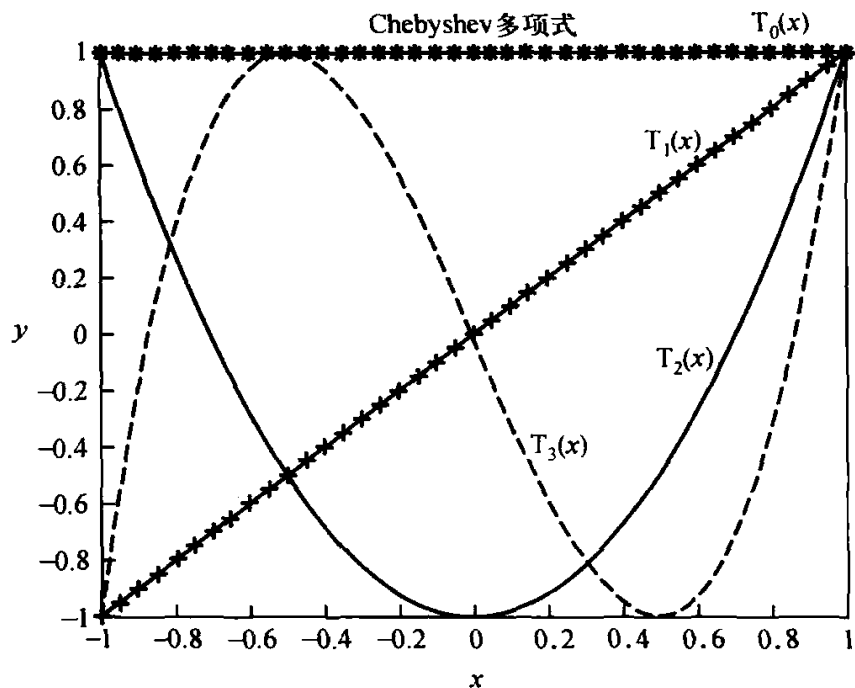
绘制在同一张图上的过程:

```
>> x = -1:0.05:1;
>> t0 = 1.0 + 0 * x;
>> t1 = x;
>> t2 = 2 * x.^2 - 1.0; t3 = 4 * x.^3 - 3 * x;
>> plot(x,t0,'* -'); gtext('T0');
>> title('Chebyshev 多项式');
>> xlabel('x'); ylabel('y');
>> hold on
>> plot(x,t1,'g +'); gtext('T1');
>> plot(x,t1,'g + -');
>> plot(x,t2,'r -',x,t3,'k -');
>> gtext('T2'); gtext('T3');
```

得到的图形如附图 1 所示.

利用命令 `subplot(m,n,i)` 可将图形窗口分成  $m$  行  $n$  列共  $m \times n$  个子窗口,子窗口按行排序,并在第  $i$  个子窗口上绘图. 若将上述程序中的绘图命令改作:

```
>> subplot(2,2,1); plot(x,t0); title('Chebyshev T0');
>> xlabel('x'); ylabel('y');
>> subplot(2,2,2); plot(x,t1); title('Chebyshev T1');
```



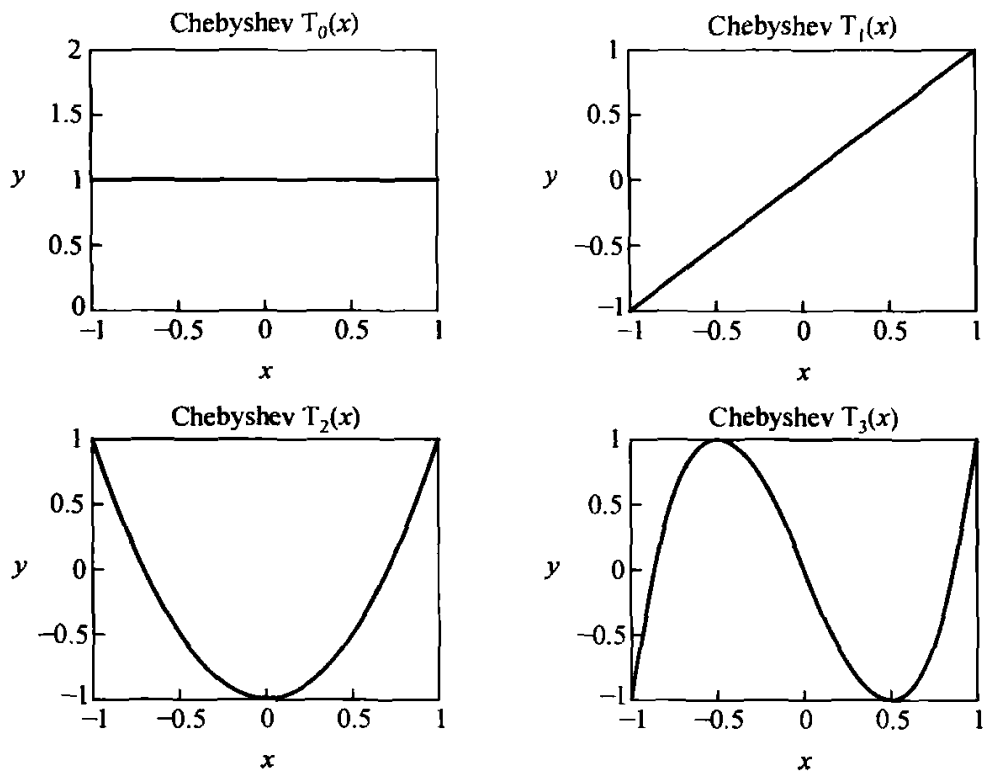
附图 1

```

>> xlabel('x'); ylabel('y');
>> subplot(2,2,3); plot(x,t2); title('Chebyshev T2');
>> xlabel('x'); ylabel('y');
>> subplot(2,2,4); plot(x,t3); title('Chebyshev T3');
>> xlabel('x'); ylabel('y');

```

可将四个 Chebyshev 多项式的图形分别画在四张子图上, 见附图 2.



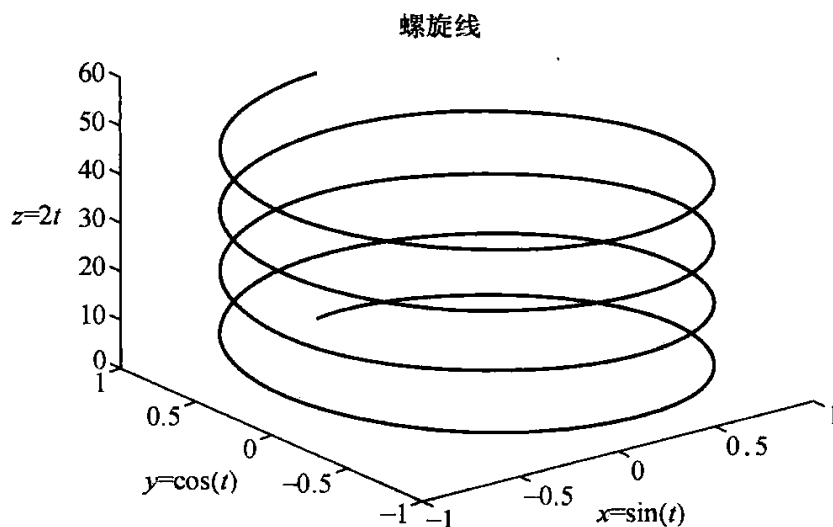
附图 2

## 2.2 三维图形

plot3 命令是绘制三维曲线的基本库函数,它的使用格式与 plot 命令类似,只是需增加 z 轴方向的数据.例如,程序

```
>> t=0:pi/60:8*pi;
>> plot3(sin(t),cos(t),2*t);
>> title('螺旋线')
>> xlabel('x=sin(t)');ylabel('y=cos(t)');zlabel('z=2t');
```

绘制了一条三位螺旋线,见附图 3.



附图 3

如果需要画一张三维曲面,应使用 mesh、meshgrid 和 surf 命令.绘图时先用 meshgrid 命令产生  $xOy$  平面的二维网格数据,再计算  $z$  轴的数据,然后进行三维曲面的作图.下面一段程序作出函数

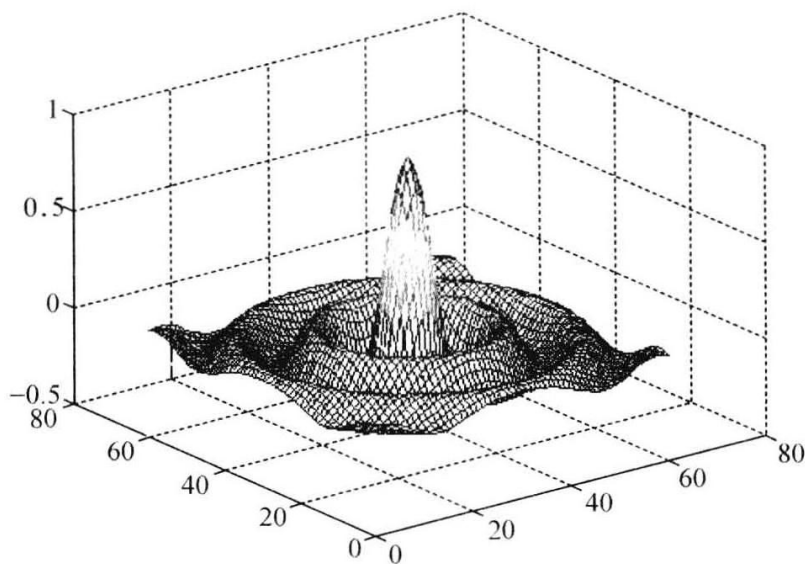
$$f(x,y) = \frac{\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}}$$

在区域  $[-15, 15] \times [-15, 15]$  上的三维图形.执行程序

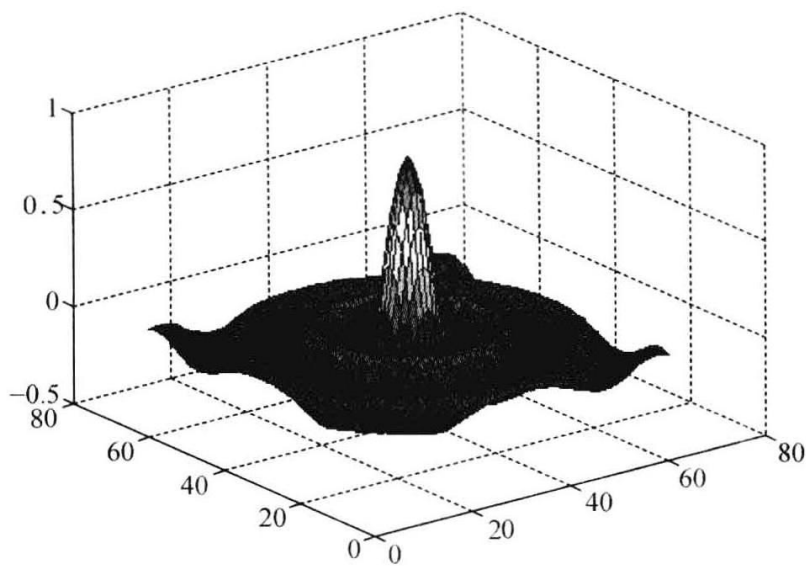
```
>> x = -15:0.5:15; y = x;
>> [X,Y] = meshgrid(x,y);
>> r = sqrt(X.^2 + Y.^2) + eps;
>> z = sin(r)./r;
>> mesh(z);
```

将得到函数的网格图,见附图 4. Surf 命令得到的是一张曲面图,而不是网格近

似图. 如将 `mesh(z)` 命令改为 `surf(z)`, 得到附图 5.



附图 4



附图 5

### § 3 高级运算功能

MATLAB 软件提供了丰富的具有符号运算功能和数值运算功能的库函数, 涉及数学和科学计算中既有代表性又有实用性的计算. 如符号运算包括代数运算、线性方程求解、矩阵运算、微积分中的极限、求导、积分和泰勒展开等. 数值运算几乎涵盖了数值计算方法中介绍的所有近似计算方法.

### 3.1 符号运算

#### (一) 符号变量的创建

MATLAB 中的符号运算对象是符号变量,符号变量可以代表表达式、函数、代数方程和微分方程等.创建符号变量一般用命令 `sym` 和 `syms`.符号变量中的参数或自变量应事先用 `syms` 命令说明,以便对符号变量进行运算.例如,程序

```
>> syms a b c x y
>> f1 = sym('a * x^2 + b * x + c');
>> g1 = sym('a * y^2 + b * c * x * y');
>> f1 + g1
ans = a * x^2 + b * x + c + a * y^2 + b * c * x * y
```

实现运算建立表达式  $ax^2 + bx + c$ ,  $ay^2 + bcxy$ , 并分别赋给变量 `f1`, `g1`, 再将两个表达式相加, 计算结果赋给默认变量 `ans`. 类似地, 可创建方程式和符号矩阵. 例如

```
>> eq1 = sym('a * x^2 + b * x + c = 0');           % 建立方程  $ax^2 + bx + c = 0$ ;
>> eq2 = sym('dy + y^3 = 1');                     % 建立常微分方程  $y' + y^3 = 1$ ;
>> msy = sym('[1 x y; sin(x) y/x f1; g1 eq1 eq2]'); % 建立符号矩阵 msy, 其形式为
```

```
msy =
[      1,      x,      y]
[ sin(x),  y/x,  a * x^2 + b * x + c]
[ a * y^2 + b * c * x * y,  a * x^2 + b * x + c = 0,  dy + y^3 = 1]
```

#### (二) 代数运算

常用的实现多项式运算的函数有:

附表 9

| 函数                             | 功能                                  |
|--------------------------------|-------------------------------------|
| <code>collect(表达式)</code>      | 合并表达式中的同类项                          |
| <code>collect(表达式, 'v')</code> | 合并表达式中的同类项, 指定 <code>v</code> 为独立变量 |
| <code>expand(表达式)</code>       | 将表达式展开为多项式                          |
| <code>factor(表达式)</code>       | 将表达式进行因式分解                          |
| <code>simply(表达式)</code>       | 将表达式简化                              |

下面给出一些例子:

```
>> s1 = sym('x^4 - 1');
>> s2 = sym('(x - 2)^3 + (y + 5)^2');
>> s3 = sym('36 * x * y^3 / (3 * y * x^2)');
>> factor(s1) % 对表达式 s1 进行因式分解, 得 (x - 1) * (x + 1) * (x^2 + 1)
>> expand(s2) % 展开表达式 s2, 得 x^3 - 6 * x^2 + 12 * x + 17 + y^2 + 10 * y
>> simple(s3) % 对表达式 s3 简化, 得 12/x * y^2
```

用符号运算求解方程和方程组的函数是 solve, 使用格式如下:

```
solve('f') % 解方程 f.
solve('f','x') % 对变量 x 解方程 f.
solve('f1',..., 'fn') % 解由 f1, ..., fn 组成的方程组.
solve('f1',..., 'fn','x1',..., 'xn') % 对指定变量解由 f1, ..., fn 组成的方程组.
```

例如, f1 是前面定义的一元二次方程, 执行命令

```
>> solve(f1)
```

得到符号解:

```
ans = [ 1/2/a * (-b + (b^2 - 4 * a * c)^(1/2)) ]
       [ 1/2/a * (-b - (b^2 - 4 * a * c)^(1/2)) ]
```

而解线性方程组的过程如下:

```
>> s = solve('3 * u + 2 * v - w = 10', '-u + 8 * v + 2 * w = 5', 'u + v + 7 * w = 1', 'u, v, w');
>> s.u, s.v, s.w
```

得解  $u = 53/21, v = 65/63, w = -23/63$ .

### (三) 微积分运算

微积分中的符号运算主要分为求极限、导数、积分、泰勒展开和常微分方程求解. 极限运算的格式为:

```
limit(f) % 求函数或表达式 f 当  $x \rightarrow 0$  时的极限.
limit(f, x, a) % 求函数或表达式 f 当  $x \rightarrow a$  时的极限.
limit(f, x, a, 'right') % 求函数或表达式 f 当  $x \rightarrow a$  时的右极限.
limit(f, x, inf) % 求函数或表达式 f 当  $x \rightarrow \infty$  时的极限.
```

例如:

```
>> dsin = (sin(y + x) - sin(y))/x;
>> limit(dsin, x, 0)
```

得到结果  $\text{ans} = \cos(y)$ .

求导运算的格式为:

```
diff(f)           % 将函数或表达式 f 关于默认变量求一次导数.
diff(f,'x')       % 将函数或表达式 f 关于变量 x 求一次导数.
diff(f,n)         % 将函数或表达式 f 关于默认变量求 n 次导数.
diff(f,'x',n)     % 将函数或表达式 f 关于变量 x 求 n 次导数.
```

例如:

```
>> g = x * sin(exp(y^2))/z;
>> diff(g,y)      % 将函数 g 对变量 y 求一阶偏导数,得结果
ans = 2 * x * cos(exp(y^2)) * y * exp(y^2)/z
>> diff(g,y,2)    % 将函数 g 对变量 y 求二阶偏导数,得结果
ans = -4 * x * sin(exp(y^2)) * y^2 * exp(y^2)^2/z + 2 * x * cos(exp(y^2))
      * exp(y^2)/z + 4 * x * cos(exp(y^2)) * y^2 * exp(y^2)/z
```

积分运算的格式为:

```
int(f)            % 将函数或表达式 f 关于默认变量积分.
int(f,'x')        % 将函数或表达式 f 关于变量 x 积分.
int(f,'x',a,b)    % 将函数或表达式 f 关于变量 x 积分,积分区间[a,b].
```

例如:

```
>> int(a * x^y,y,0,pi/6)      % 计算含参数的积分,得结果
ans = a * (x^(1/6 * pi) - 1)/log(x)
```

Taylor 函数可以实现任意阶的泰勒展开. 例如,求函数  $\sin(x) + e^x \cos(x)$  的展开至 8 阶的麦克劳林公式,执行语句

```
>> taylor(sin(x) + exp(x) * cos(x),8)
```

得结果

```
ans = 1 + 2 * x - 1/2 * x^3 - 1/6 * x^4 - 1/40 * x^5 + 1/720 * x^7
```

dsolve 函数可用来求解方程和简单常微分方程的解析解. 例如,求解微分方程

$$\begin{cases} y''' - y'' = x \\ y(1) = 8, \quad y'(1) = 7, \quad y''(1) = 4 \end{cases}$$

的特解由语句

```
>> sol_1 = dsolve('D3y - D2y = x','y(1) = 8','Dy(1) = 7','D2y(1) = 4')
```

实现,结果为

```
sol_1 = -1/2 * x * t^2 - 1/2 * x + 1 + 3 * t + (x + 4)/exp(1) * exp(t)
```



求解常微分方程  $x^2 + y + (x - 2y)y' = 0$  通解的语句为

```
>> sol_2 = dsolve('x^2 + y + (x - 2 * y) * Dy = 0','x')
```

通解为

```
sol_2 =  
[1/2 * x + 1/6 * (9 * x^2 + 36 * C1 + 12 * x^3)^(1/2)]  
[1/2 * x - 1/6 * (9 * x^2 + 36 * C1 + 12 * x^3)^(1/2)]
```

类似地,可求得常微分方程组的通解或特解.

## 3.2 数值计算

### (一) 插值及曲面拟合

MATLAB 的一维插值命令是 `interp1`,使用格式为 `interp1(x,y,xi,'method')`,其中  $x, y$  是已知一维数据,  $xi$  是插值计算点. `method` 参数可以有四种选择,它们是: `nearest`、`linear`、`cubic`、`spline`,分别表示最近点插值、线性插值、三次分段插值和三次样条插值. `method` 参数可以缺省,默认为 `linear`.

二维插值是对二元函数作插值近似,库函数有 `interp2`. 函数 `interp2` 的使用格式为 `interp2(x,y,z,xi,yi,'method')`,其中  $x, y$  是已知的二维数据,  $z$  是由  $x, y$  确定的被插值函数的数据,  $xi, yi$  是插值计算点, `method` 有四种选项: `nearest`、`bilinear`、`bicubic`、`spline`,分别表示选用最近点插值、双线性插值、双三次插值和样条插值计算近似值  $zi$ . 例如,下面一段程序利用二维插值函数对 `peak` 函数进行插值近似:

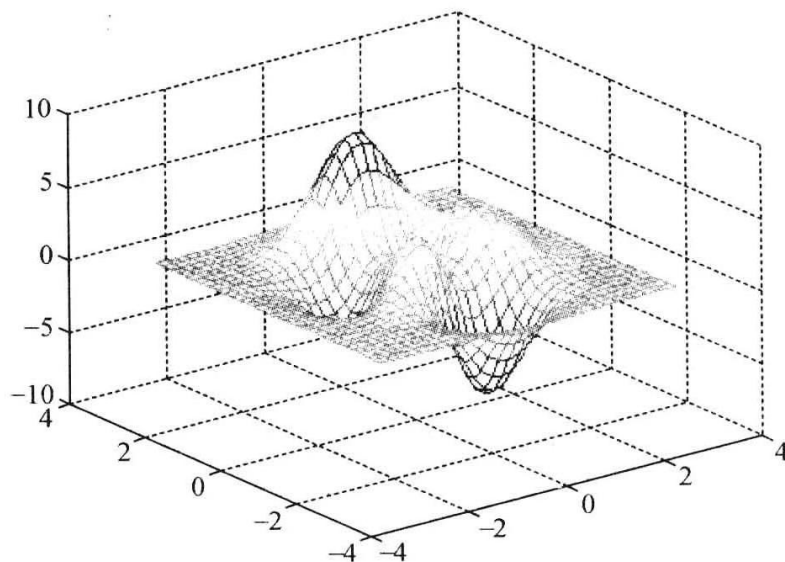
```
>> [X,Y] = meshgrid(-3:0.2:3);  
>> Z = peaks(X,Y);  
>> [xi,yi] = meshgrid(-3:0.1:3);  
>> zi = interp2(X,Y,Z,xi,yi,'bilinear');  
>> mesh(X,Y,Z)  
>> mesh(xi,yi,zi);
```

附图 6 是原函数的网格图,附图 7 显示的是双线插值结果.

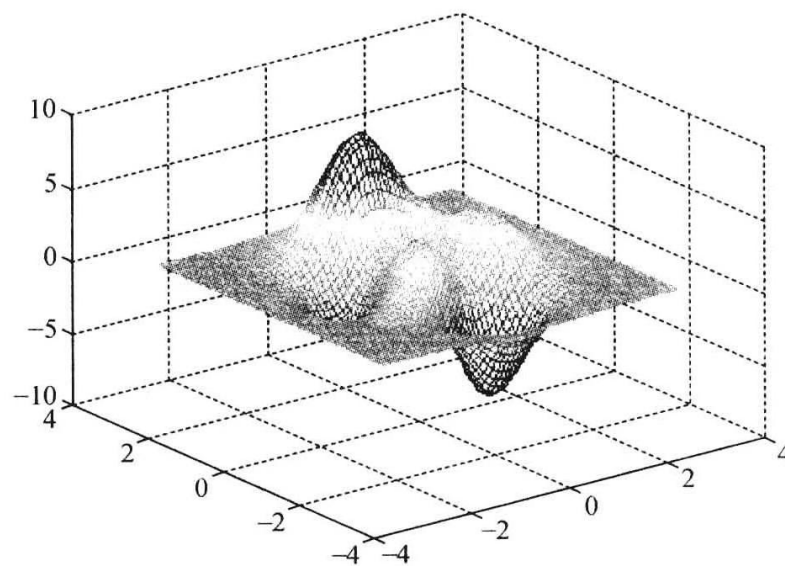
### (二) 数值积分

MATLAB 提供了应用复化梯形法、复化 Simpson 法计算数值积分的函数. 复化梯形法的函数是 `trapz`,使用格式为 `trapz(x,y)`,其中  $x, y$  是数组. 函数 `quad` 是用复化 Simpson 公式计算数值积分,使用格式为 `quad('f',a,b,tol)`,其中  $f$  表示已被定义且存为  $m$  文件的函数,  $a, b$  是积分限,  $tol$  是相对误差限. `quad8` 是利用高阶 Newton-Cotes 求积公式计算数值积分,使用格式与 `quad` 一样. 例如计算积

分  $I = \int_0^{\pi} \sqrt{100\sin^2 x + 25\cos^2 x} dx$ , 须先定义函数



附图 6



附图 7

```
function y = f(x)
y = sqrt(100 * sin(x).^2 + 25 * cos(x).^2);
```

将其存为文件 f.m, 然后调用函数

```
>> I = quad('f', 0, pi, 1.0^(-6));
```

得积分近似  $I = 24.220\ 641$ .

### (三) 常微分方程数值解

MATLAB 提供的求解常微分方程数值解的方法是以 Runge - Kutta 法为基础的, 函数有 ode23 和 ode45. ode23 是用二阶或三阶 Runge - Kutta 法求解, ode45

是用四阶或五阶 Runge - Kutta 方法求解. ode23 的使用格式为 `ode23('f',x0,xn,y0)`, 其中 `f` 是自定义的存为 `m` 文件的函数名, `x0,xn` 是求解区间  $[x_0, x_n]$  的两个端点, `y0` 是初始值. 例如求解常微分方程的初值问题

$$\begin{cases} y'' + (y^2 - 1)y' = 0 \\ y(0) = 0, y'(0) = 0.5 \end{cases} \quad (0 < x < 10)$$

令  $y_1' = y(x)$ ,  $y_2 = y'(x)$ , 则原初值问题化为常微分方程组

$$\begin{cases} y_1' = y_2 \\ y_2' = (1 - y_1^2)y_2 - y_1 \\ y_1(0) = 0, y_2(0) = 0.5 \end{cases} \quad (0 < x < 10)$$

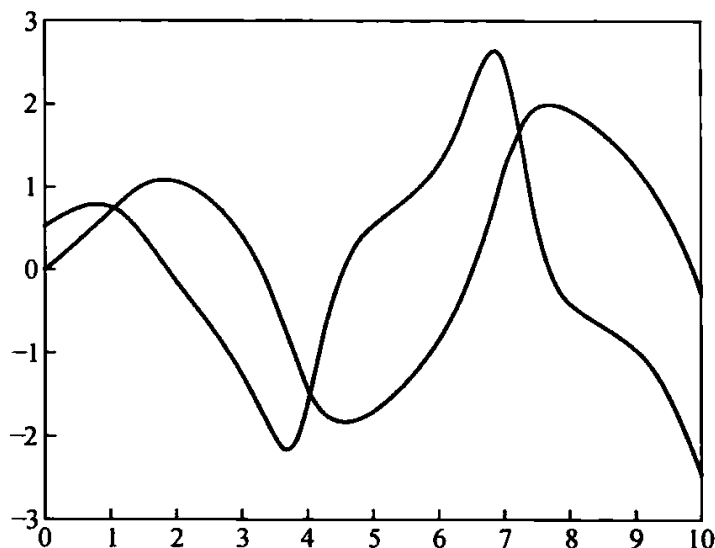
先建立微分方程组右端函数的 `m` 文件:

```
function z = pol(x,y)
    z = [y(2); (1 - y(1)^2) * y(2) - y(1)];
```

将其存为在 MATLAB 工作目录的 `m` 文件下, 然后在命令窗口键入:

```
>> y0 = [0;0.5];
>> [x,y] = ode23('pol',0,10,y0);
>> plot(x,y)
```

计算结果如附图 8.



附图 8

MATLAB 软件中更详细的数值计算函数与命令见前面各章节中.

MATLAB 软件还含有各种工具箱, 功能齐全的工具箱大致可分为两类, 功能型工具箱和领域型工具箱. 功能型工具箱主要用来扩充 MATLAB 的符号计算

功能、图形建模仿真功能、文件处理功能及与硬件实时交互功能,而领域型工具箱具有很强的专业性.如通讯工具箱、控制系统工具箱、信号处理工具箱、图像处理工具箱、神经网络工具箱、优化工具箱、微分方程工具箱、统计工具箱、符号数学工具箱、小波工具箱等.有兴趣的读者可查阅相关参考书.

## 参 考 文 献

- [1] 豪斯霍尔德 A. S. 数值分析中的矩阵论. 孙家昶, 译. 北京: 科学出版社, 1986.
- [2] 奥特加 J. M. 数值分析. 张丽君等, 译. 北京: 高等教育出版社, 1984.
- [3] 徐树方. 矩阵计算的理论与方法. 北京: 北京大学出版社, 1995.
- [4] Burden R. L. Numerical Analysis. 7th ed. 北京: 高等教育出版社, 2001.
- [5] Bjorck A. Least Squares Methods. in: P. G. Ciarlet and J. L. Lions eds. Handbook of Numerical Analysis, Vol. 1, North-Holland, Austerdam, 1990.
- [6] 阿特金森 K. E. 数值分析引论. 匡蛟勋, 王国荣, 译. 上海: 上海科学出版社, 1986.
- [7] 黄友谦. 数值实验. 北京: 高等教育出版社, 1989.
- [8] Michael T. H. Scientific Computing, An Introductory Survey, 2nd ed. 北京: 清华大学出版社, 2001.
- [9] 胡祖炽, 林源渠. 数值分析. 北京: 高等教育出版社, 1986.
- [10] 孙志忠, 袁慰平, 闻震初. 数值分析. 第二版. 南京: 东南大学出版社, 2002.
- [11] 袁亚湘, 孙文瑜. 最优化理论与方法. 北京: 科学出版社, 1999.
- [12] 余德浩, 汤涛. 微分方程数值解法. 北京: 科学出版社, 2003.
- [13] 袁鼎兆. 刚性常微分方程初值问题的数值解法. 北京: 科学出版社, 1987.
- [14] 胡祖炽, 雷功炎. 偏微分方程初值问题差分方法. 北京: 北京大学出版社, 1988.
- [15] Gear C. W. 常微分方程初值问题的数值解法. 费景高等, 译. 北京: 科学出版社, 1971.
- [16] Evans D. S. Sparsity and Its Applications. Cambridge: Cambridge University Press, 1985.
- [17] Tewarson R. D. Sparse Matrices. New York: Academic Press, 1973.
- [18] 戈卢布 G. H, 范洛恩 C. F. 矩阵计算. 袁亚湘等, 译. 北京: 科学出版社, 2001.

- 
- [19] Wilkinson J. H. 代数特征值问题. 石钟慈等, 译. 北京: 科学出版社, 1987.
  - [20] 曹志浩, 张玉德, 李瑞遐. 矩阵计算与方程求根. 第二版, 北京: 高等教育出版社, 1984.
  - [21] Schilling R. J, Harris S. L. Applied Numerical Methods for Engineers Using MATLAB and C. 北京: 机械工业出版社, 2004.
  - [22] Numberger G. Approximationly Spline Function. Berlin: Spring-Verlag, 1989.
  - [23] 王德人, 杨忠华. 函数逼近论. 北京: 高等教育出版社, 1990.
  - [24] 关治, 陆金甫. 数值分析基础. 北京: 高等教育出版社, 1998.
  - [25] Davis P. J, Rabinowitz P. Methods of Numerical Integration. 2nd ed. Orlando: Academic Press, 1984.

Images have been losslessly embedded. Information about the original file can be found in PDF attachments. Some stats (more in the PDF attachments):

```
{
  "filename": "MTI4ODg5ODcuemlw",
  "filename_decoded": "12888987.zip",
  "filesize": 18810560,
  "md5": "f059a9d9530be59adb0224cc60c63705",
  "header_md5": "6d0704655dc9d1e84fb7a3fdc86e8483",
  "sha1": "19042fde2cbbdbd0b3bdb876c21d3e4728a38cbb",
  "sha256": "9c6f11a2e32281faef0a2933e43287b84d2c390b47c7ce0d81acd8525c233ff4",
  "crc32": 325721025,
  "zip_password": "",
  "uncompressed_size": 19716930,
  "pdg_dir_name": "_12888987",
  "pdg_main_pages_found": 363,
  "pdg_main_pages_max": 363,
  "total_pages": 373,
  "total_pixels": 1813500400,
  "pdf_generation_missing_pages": false
}
```